目　录

第 1 章　入门 …… 1

1.1　Java技术现象 …… 1

1.1.1　关于Java技术 …… 1

1.1.2　Java技术能做什么 …… 3

1.1.3　Java技术将如何改变我们的生活 …… 3

1.2　“Hello World!”应用程序 …… 4

1.2.1　使用NetBeans IDE编写“Hello World!” …… 4

1.2.2　在Microsoft Windows平台上编写“Hello World!” …… 9

1.2.3　在Solaris OS和Linux平台上编写“Hello World!” …… 12

1.3　分析“Hello World!”应用程序 …… 15

1.3.1　源代码注释 …… 16

1.3.2　HelloWorldApp类定义 …… 16

1.3.3　main方法 …… 16

1.4　常见问题（和解决方案） …… 17

1.4.1　编译器问题 …… 17

1.4.2　运行时问题 …… 18

问题和练习：入门 …… 19

第 2 章　面向对象的编程概念 …… 21

2.1　对象是什么 …… 21

2.2　类是什么 …… 22

2.3　继承是什么 …… 24

2.4　接口是什么 …… 24

2.5　包是什么 …… 25

问题和练习：面向对象的编程概念 …… 25

第 3 章　语言基础 …… 27

3.1　变量 …… 27

3.1.1　命名 …… 28

3.1.2　基本数据类型 …… 28

3.1.3　数组 …… 30

3.1.4　变量小结 …… 33

问题和练习：变量 …… 34

3.2　操作符 …… 34

3.2.1　赋值、算术和一元操作符 …… 35

3.2.2　相等、关系和条件操作符 …… 37

3.2.3　位操作符和移位操作符 …… 39

3.2.4　操作符小结 …… 40

问题和练习：操作符 …… 41

3.3　表达式、语句和块 …… 42

3.3.1　表达式 …… 42

3.3.2　语句 …… 42

3.3.3　块 …… 43

问题和练习：表达式、语句和块 …… 43

3.4　控制流语句 …… 44

3.4.1　if-then和if-then-else语句 …… 44

3.4.2　switch语句 …… 45

3.4.3　while和do-while语句 …… 47

3.4.4　for语句 …… 48

3.4.5　分支语句 …… 49

3.4.6　控制流语句小结 …… 52

问题和练习：控制流语句 …… 52

第 4 章　类和对象 …… 54

4.1　类 …… 54

4.1.1　声明类 …… 55

4.1.2　声明成员变量 …… 55

4.1.3　定义方法 …… 57

4.1.4　为类提供构造器 …… 58

4.1.5　把信息传递给方法或构造器 …… 59

4.2　对象 …… 62

4.2.1　创建对象 …… 63

4.2.2　使用对象 …… 66

4.3　关于类的更多介绍 …… 67

4.3.1　从方法返回值 …… 68

4.3.2 使用关键字this ……69
4.3.3 控制对类成员的访问 ……70
4.3.4 了解实例成员和类成员 ……72
4.3.5 初始化字段 ……75
4.3.6 创建和使用类和对象的小结 ……76
问题和练习：类 ……76
问题和练习：对象 ……77
4.4 嵌套类 ……78
4.4.1 为什么使用嵌套类 ……78
4.4.2 静态嵌套类 ……79
4.4.3 内部类 ……79
4.4.4 内部类示例 ……80
4.4.5 嵌套类小结 ……81
问题和练习：嵌套类 ……82
4.5 枚举类型 ……83
问题和练习：枚举类型 ……85
4.6 注解 ……86
4.6.1 文档 ……86
4.6.2 编译器使用的注解 ……87
4.6.3 注解处理 ……88
问题和练习：注解 ……88

第5章 接口和继承 ……90

5.1 接口 ……90
5.1.1 Java中的接口 ……90
5.1.2 接口作为API ……91
5.1.3 接口和多重继承 ……91
5.1.4 定义接口 ……91
5.1.5 实现接口 ……92
5.1.6 使用接口作为类型 ……93
5.1.7 重新编写接口 ……94
5.1.8 接口小结 ……95
问题和练习：接口 ……95
5.2 继承 ……95
5.2.1 Java平台类层次结构 ……96
5.2.2 继承示例 ……96
5.2.3 在子类中能进行什么操作 ……97
5.2.4 子类中的私有成员 ……98
5.2.5 对象的类型转换 ……98
5.2.6 覆盖和隐藏方法 ……99
5.2.7 隐藏字段 ……100
5.2.8 使用关键字super ……100
5.2.9 Object作为超类 ……101
5.2.10 编写最终类和方法 ……104
5.2.11 抽象方法和抽象类 ……104
5.2.12 继承小结 ……106
问题和练习：继承 ……106

第6章 泛型 ……108

6.1 简介 ……108
6.2 泛型类型 ……109
6.3 泛型方法和构造器 ……111
6.4 受限类型参数 ……112
6.5 子类型化 ……113
6.6 通配符 ……114
6.7 类型擦除 ……115
6.8 泛型小结 ……116
问题和练习：泛型 ……117

第7章 包 ……118

7.1 创建和使用包 ……118
7.1.1 创建包 ……119
7.1.2 命名包 ……120
7.1.3 使用包成员 ……121
7.1.4 管理源文件和类文件 ……123
7.1.5 创建和使用包小结 ……124
问题和练习：创建和使用包 ……124

第8章 数字和字符串 ……126

8.1 数字 ……126
8.1.1 数字类 ……126
8.1.2 格式化数字输出 ……128
8.1.3 高级算术功能 ……131
8.1.4 数字小结 ……134
问题和练习：数字 ……134
8.2 字符 ……136
8.3 字符串 ……137
8.3.1 创建字符串 ……138
8.3.2 字符串长度 ……138
8.3.3 连接字符串 ……139
8.3.4 创建格式化字符串 ……139
8.3.5 在数字和字符串之间进行转换 ……140
8.3.6 处理字符串中的字符 ……141

8.3.7　比较字符串和部分字符串 …… 145
8.3.8　StringBuilder类 …… 146
8.3.9　字符和字符串小结 …… 149
问题和练习：字符和字符串 …… 150

第 9 章　异常 …… 151

9.1　异常是什么 …… 151
9.2　捕获或者指定需求 …… 152
9.2.1　异常的三种类型 …… 152
9.2.2　绕过捕获或者指定 …… 153
9.3　捕获和处理异常 …… 153
9.3.1　try块 …… 154
9.3.2　catch块 …… 154
9.3.3　finally块 …… 155
9.3.4　结合使用 …… 156
9.4　指定方法抛出的异常 …… 159
9.5　如何抛出异常 …… 159
9.5.1　throw语句 …… 160
9.5.2　Throwable类及其子类 …… 160
9.5.3　Error类 …… 161
9.5.4　Exception类 …… 161
9.5.5　链式异常 …… 161
9.5.6　创建异常类 …… 162
9.6　关于不可控异常的争论 …… 163
9.7　异常的优点 …… 164
9.7.1　优点一：把错误处理代码和“常规”代码分离开 …… 164
9.7.2　优点二：把错误沿调用栈向上传递 …… 165
9.7.3　优点三：对错误类型进行分组和区分 …… 166
9.8　小结 …… 167
问题和练习：异常 …… 167

第 10 章　基本 I/O …… 169

10.1　I/O流 …… 169
10.1.1　字节流 …… 170
10.1.2　字符流 …… 171
10.1.3　缓冲流 …… 174
10.1.4　扫描和格式化 …… 174
10.1.5　命令行I/O …… 179
10.1.6　数据流 …… 180
10.1.7　对象流 …… 182
10.2　文件I/O …… 185
10.2.1　File对象 …… 185
10.2.2　随机访问文件 …… 187
10.3　新I/O包 …… 189
10.4　小结 …… 189
问题和练习：基本I/O …… 189

第 11 章　集合 …… 190

11.1　集合简介 …… 190
11.1.1　集合框架是什么 …… 190
11.1.2　Java集合框架的优势 …… 190
11.2　接口 …… 191
11.2.1　Collection接口 …… 192
11.2.2　Set接口 …… 195
11.2.3　List接口 …… 198
11.2.4　Queue接口 …… 204
11.2.5　Map接口 …… 206
11.2.6　对象排序 …… 212
11.2.7　SortedSet接口 …… 216
11.2.8　SortedMap接口 …… 219
11.2.9　接口小结 …… 219
问题和练习：接口 …… 220
11.3　实现 …… 221
11.3.1　Set实现 …… 222
11.3.2　List实现 …… 223
11.3.3　Map实现 …… 224
11.3.4　Queue实现 …… 225
11.3.5　包装器实现 …… 226
11.3.6　简便实现 …… 227
11.3.7　实现小结 …… 228
问题和练习：实现 …… 229
11.4　算法 …… 229
11.4.1　排序 …… 229
11.4.2　混序 …… 231
11.4.3　常规数据操作 …… 231
11.4.4　搜索 …… 231
11.4.5　组合 …… 232
11.4.6　查找极值 …… 232
11.5　定制集合实现 …… 232
11.5.1　编写实现的原因 …… 232
11.5.2　如何编写定制实现 …… 233

11.6 互操作性……234
11.6.1 兼容性……235
11.6.2 API设计……236

第 12 章 并发……238

12.1 进程和线程……238
12.1.1 进程……238
12.1.2 线程……238
12.2 Thread对象……239
12.2.1 定义和启动线程……239
12.2.2 使用sleep暂停执行……240
12.2.3 中断……240
12.2.4 join……241
12.2.5 SimpleThreads示例……242
12.3 同步……243
12.3.1 线程干扰……243
12.3.2 内存一致性错误……244
12.3.3 同步方法……245
12.3.4 内在锁和同步……246
12.3.5 原子访问……247
12.4 活性……247
12.4.1 死锁……247
12.4.2 饿死和活锁……248
12.5 保护块……249
12.6 不可变对象……252
12.6.1 同步类示例……252
12.6.2 定义不可变对象的策略……253
12.7 高级并发对象……255
12.7.1 Lock对象……255
12.7.2 执行器……257
12.7.3 并发集合……258
12.7.4 原子变量……259
12.8 推荐阅读……260
问题和练习：并发……260

第 13 章 正则表达式……262

13.1 简介……262
13.1.1 正则表达式是什么……262
13.1.2 这个包如何表示正则表达式……262
13.2 测试示例……263
13.3 字符串字面量……263
13.4 字符类……264
13.5 预定义字符类……268
13.6 量词……270
13.6.1 零长度匹配……270
13.6.2 使用量词的捕获组和字符类……273
13.6.3 greedy、reluctant和possessive量词的区别……273
13.7 捕获组……274
13.7.1 编号……274
13.7.2 反向引用……275
13.8 边界匹配器……275
13.9 Pattern类的方法……277
13.9.1 创建带有标志的模式……277
13.9.2 嵌入标志表达式……278
13.9.3 使用matches(String, CharSequence)方法……279
13.9.4 使用split(String)方法……279
13.9.5 其他工具方法……280
13.9.6 java.lang.String中和Pattren等同的方法……280
13.10 Matcher类的方法……280
13.10.1 索引方法……280
13.10.2 研究方法……281
13.10.3 替换方法……281
13.10.4 使用start和end方法……281
13.10.5 使用matches和lookingAt方法……282
13.10.6 使用replaceFirst(String)和replaceAll(String)……283
13.10.7 appendReplacement(StringBuffer,String)和appendTail(StringBuffer)方法……284
13.10.8 java.lang.String中和Matcher等同的方法……284
13.11 PatternSyntaxException类的方法……284
13.12 小结……286
13.13 附加资源……287

问题和练习：正则表达式 …… 287

第 14 章　平台环境 …… 288

14.1　配置工具 …… 288
14.1.1　属性 …… 288
14.1.2　命令行参数 …… 291
14.1.3　环境变量 …… 292
14.1.4　其他配置工具 …… 293
14.2　系统工具 …… 294
14.2.1　命令行I/O对象 …… 294
14.2.2　系统属性 …… 294
14.2.3　安全管理器 …… 296
14.2.4　系统中的杂项方法 …… 297
14.3　PATH和CLASSPATH …… 297
14.3.1　更新PATH变量（Microsoft Windows NT / 2000 / XP） …… 298
14.3.2　更新PATH变量（Solaris和Linux） …… 298
14.3.3　检查CLASSPATH变量（所有平台） …… 299
问题和练习：平台环境 …… 299

第 15 章　Swing …… 301

15.1　Swing包概述 …… 301
15.1.1　Swing是什么 …… 301
15.1.2　Swing演示程序 …… 302
15.2　Swing的特性 …… 305
15.2.1　Swing组件的可视化指导 …… 306
15.2.2　可插拔的界面外观 …… 310
15.2.3　拖放和数据传输 …… 311
15.2.4　国际化和本地化 …… 311
15.2.5　可访问性 …… 312
15.2.6　和桌面集成 …… 312
15.2.7　系统托盘图标支持 …… 313
问题和练习：图形化用户界面 …… 313

第 16 章　把程序打包到 JAR 文件中 …… 314

16.1　使用JAR文件：基础 …… 314
16.1.1　创建JAR文件 …… 315
16.1.2　查看JAR文件的内容 …… 318
16.1.3　提取JAR文件的内容 …… 318
16.1.4　更新JAR文件 …… 319
16.1.5　运行JAR打包的软件 …… 321
16.2　处理清单文件：基础 …… 322
16.2.1　了解默认的清单 …… 322
16.2.2　修改清单文件 …… 322
16.2.3　设置应用程序的入口点 …… 323
16.2.4　把类添加到JAR文件的类路径 …… 324
16.2.5　设置包版本信息 …… 324
16.2.6　在JAR文件中密封包 …… 326
16.3　JAR文件的签名和验证 …… 326
16.3.1　了解签名和验证 …… 327
16.3.2　签名JAR文件 …… 329
16.3.3　验证签名的JAR文件 …… 330
16.4　使用和JAR相关的API …… 330
16.4.1　示例——应用程序JarRunner …… 330
16.4.2　JarClassLoader类 …… 331
16.4.3　JarRunner类 …… 333
问题和练习：JAR文件 …… 334

第 17 章　Java Web Start …… 335

17.1　运行Java Web Start应用程序 …… 336
17.1.1　从浏览器运行Java Web Start应用程序 …… 336
17.1.2　从Java缓存查看器运行Java Web Start应用程序 …… 336
17.1.3　从桌面运行Java Web Start应用程序 …… 337
17.2　部署Java Web Start应用程序 …… 337
17.2.1　设置Web服务器 …… 337
17.2.2　创建JNLP文件 …… 337
17.2.3　把应用程序放到服务器上 …… 342
17.2.4　创建Web页面 …… 342
17.3　开发Java Web Start应用程序 …… 342
17.3.1　把应用程序打包到JAR文件中 …… 342
17.3.2　读取JAR文件中的资源 …… 343
17.3.3　不被信任的应用程序 …… 343
17.4　JNLP API …… 343
17.5　Java Web Start和安全 …… 344
17.5.1　Java Web Start安全基础 …… 344

17.5.2 Java Web Start部署的签名JAR文件……345
17.5.3 安全和JNLP文件……345
17.5.4 HTTPS证书的动态下载……345
17.6 常见的Java Web Start问题……345
问题和练习：Java Web Start……346

第 18 章 applet……347

18.1 applet入门……348
18.1.1 扩展Applet或者JApplet…349
18.1.2 applet的生存周期……349
18.1.3 里程碑方法……351
18.1.4 用于绘制和事件处理的方法…352
18.1.5 用于添加UI组件的方法……354
18.1.6 applet能做什么，不能做什么…357
18.2 利用applet API的优势……357
18.2.1 查找和加载数据文件……358
18.2.2 显示简短状态字符串……359
18.2.3 在浏览器中显示文档……359
18.2.4 向其他applet发送消息……360
18.2.5 播放声音……363
18.2.6 定义和使用applet参数……365
18.2.7 使用APPLET标记……367
18.3 编写applet时的考虑事项……370
18.3.1 安全限制……370
18.3.2 创建用户界面……371
18.3.3 获得系统属性……373
18.3.4 applet中的线程……374
18.3.5 处理服务器端应用程序……376
18.4 完成applet……378
18.4.1 发布applet前的工作……378
18.4.2 圆满完成的applet……379
18.5 部署applet……380
18.5.1 一般的部署考虑事项……380
18.5.2 使用applet标记……380
18.5.3 使用object标记……380
18.5.4 使用embed标记……381
18.5.5 在混合浏览器环境中部署applet……382
18.6 解决常见的applet问题……383
问题和练习：Java applet……384

附录 A Java 语言关键字……385
附录 B Java 编程语言认证考试的准备……386
索引……394

第1章 入　门

1.1 Java技术现象

到处都在谈论Java技术，但是Java技术到底是什么呢？下面的章节将说明Java技术为何既是一种编程语言又是一个平台，并且概述这种技术能为你做什么。

1.1.1 关于Java技术

Java技术既是一种编程语言，也是一个平台。

1. Java编程语言

Java编程语言是一种高级语言，其特点可以用下面几个词语来归纳：

- 简单；
- 平台独立；
- 面向对象；
- 可移植；
- 分布式；
- 高性能；
- 多线程；
- 健壮；
- 动态；
- 安全。

在James Gosling和Henry McGilton撰写的*Java Language Environment*[1]一文中，对上面的每个词语都给出了解释。

在Java编程语言中，首先把所有源代码都写入到纯文本文件中，以.java扩展名结束。然后，使用javac编译器把这些源文件编译为.class文件。.class文件不包含对应于处理器的本机代码，而是包含字节码（bytecode）——Java虚拟机（Java Virtual Machine，Java VM）的机器语言[2]。然后java启动器工具通过Java虚拟机的一个实例运行应用程序（见图1-1）。

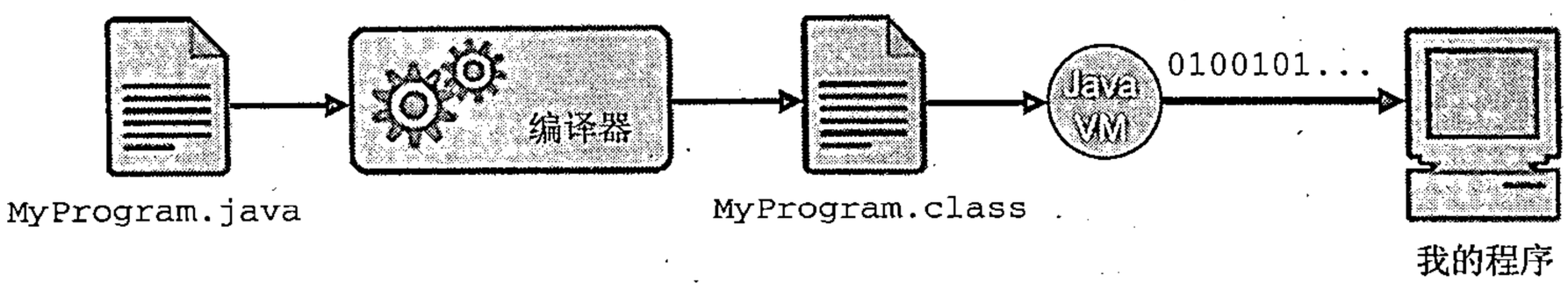


图1-1　编译和运行应用程序

1. http://java.sun.com/docs/white/langenv/
2. 术语“Java虚拟机”和“JVM”都表示Java平台上的虚拟机（Virtual Machine）。

因为在很多不同的操作系统上都有Java VM，所以相同的.class文件可以运行在Microsoft Windows、Solaris OS、Linux或者Mac OS上。一些虚拟机，比如Java HotSpot虚拟机[1]，在运行时还会执行额外的步骤，以提高应用程序的性能。这些步骤包括各种任务，比如查找性能瓶颈和将频繁使用的代码段落重编译（为本机代码）（见图1-2）。

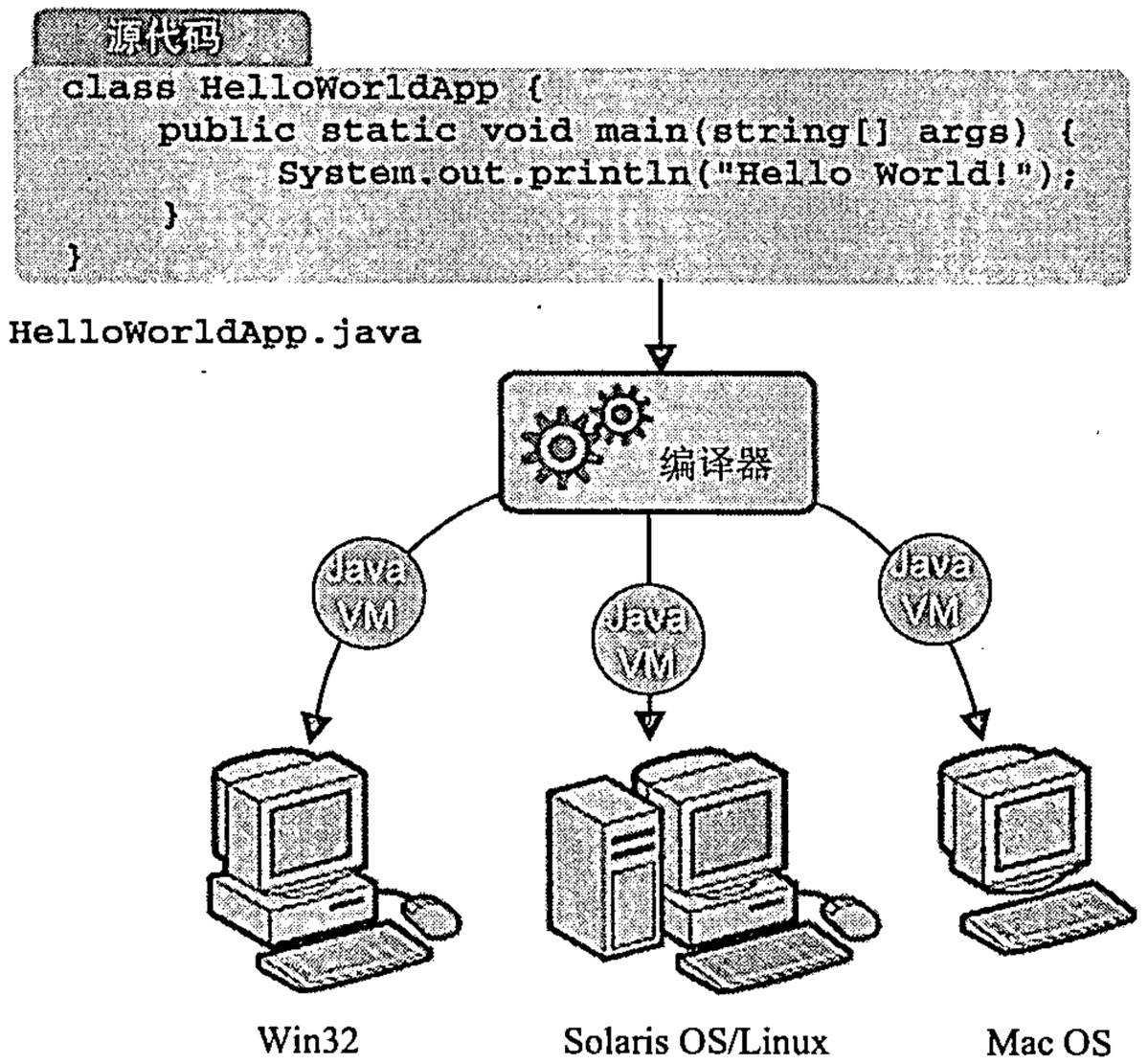


图1-2　通过Java VM，同一个应用程序能够运行在多种平台上

2. Java平台

平台（platform）是程序在其中运行的硬件或软件环境。我们已经提到了一些最流行的平台，比如Microsoft Windows、Linux、Solaris OS和Mac OS。大多数平台可以被描述为操作系统和底层硬件的组合。Java平台与大多数其他平台的不同之处在于，它是一种运行在其他基于硬件平台之上的纯软件平台。

Java平台有两个组成部分：

- Java虚拟机（Java Virtual Machine，Java VM）。
- Java应用程序编程接口（Application Programming Interface，API）。

我们已经介绍了Java虚拟机，它是Java平台的基础，已经被移植到了各种硬件平台上（见图1-3）。

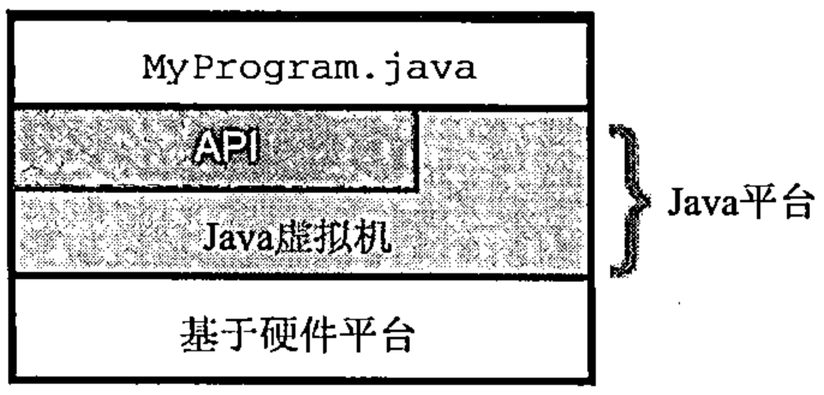


图1-3　API和Java虚拟机把程序和底层硬件隔离开

API是已经开发好的软件组件的大型集合，它们提供了许多有用的功能。API按相关性类和

1. http://java.sun.com/javase/technology/hotspot.jsp

接口分为多个库，这些库被称为包（package）。下一节重点介绍API提供的一些功能。

作为独立于平台的环境，Java平台比本机代码稍慢一些。但是，编译器和虚拟机技术的进展已经使Java平台的性能与本机代码接近，而且不会有损于可移植性。

1.1.2 Java 技术能做什么

通用的高级Java编程语言是一个功能强大的软件平台。Java平台的每个完整实现都能提供如下特性：

- **开发工具**——开发工具提供进行编译、运行、监视、调试应用程序和编写应用程序文档所需的所有功能。作为开发新手，你将使用的主要工具是javac编译器、java启动器和javadoc文档工具。
- **API**——API提供Java编程语言的核心功能。它提供大量已编写好的类，可以用于你自己的应用程序。核心API非常大；要大致了解API包含的内容，请参考Java SE开发工具包6（Java SE Development Kit 6，JDK 6）文档[1]。
- **部署技术**——JDK软件提供标准机制，比如Java Web Start软件和Java Plug-In软件，用于把应用程序部署给最终用户。
- **用户界面工具包**——Swing和Java 2D工具包使创建复杂的图形用户界面（Graphical User Interface，GUI）成为可能。
- **集成库**——集成库，比如Java IDL API、JDBC API、Java命名和目录接口（Java Naming and Directory Interface，JNDI）API、Java RMI和Java RMI-IIOP技术（Java Remote Method Invocation over Internet Inter-ORB Protocol Technology），支持数据库访问和远程对象的操纵。

1.1.3 Java 技术将如何改变我们的生活

我们不能保证如果你学习了Java编程语言就能够得到荣誉、财富，甚至工作。但是，它很可能使你的程序更好，而需要付出的努力却比其他语言少。我们相信Java技术会给你带来以下好处：

- **入门迅速**——尽管Java编程语言是一种强大的面向对象语言，但是它容易学，特别是对于熟悉C或C++的程序员。
- **编写的代码更少**——程序度量（比如类的数量、方法的数量等）的对比表明，用Java编程语言写的程序比用C++写的相同程序小4倍。
- **编写的代码更好**——Java编程语言鼓励良好的编程实践，而且它的自动垃圾回收功能有助于避免内存泄漏。它的面向对象性质、JavaBeans组件架构以及易于扩展的大量API，使你可以重用已有的、经过测试的代码并且减少错误。
- **开发程序更快**——Java编程语言比C++简单，因为这个原因，使用Java进行开发的速度可能要快上一倍。你的程序所需的代码行数也更少。
- **平台无关性**——因为不需要使用由其他语言编写的库，可以保持程序的可移植性。
- **一次编写，处处运行**——因为Java程序被编译为与机器无关的字节码，所以它们可以在任何Java平台上以一致的方式运行。
- **发布软件更容易**——使用Java Web Start软件，用户只需通过点击鼠标就可启动应用程序。启动时的自动版本检查使用户总能更新到软件的最新版本。如果有可用的更新，那么Java Web Start软件就会自动更新安装。

1. docs/index.html

1.2　“Hello World!”应用程序

下面几节详细讲解如何编译和运行简单的应用程序“Hello World!”。第1节介绍使用NetBeans IDE开始程序设计，NetBeans IDE是一个集成开发环境，它极大地简化了软件开发过程。其余几节介绍在不使用集成开发环境的情况下，如何在特定平台上开始编程。

1.2.1　使用 NetBeans IDE 编写“Hello World!”

现在开始编写你的第一个应用程序！下面的详细说明适用于NetBeans IDE用户。NetBeans IDE运行在Java平台上，这就是说你可以在能够运行JDK 6的任何操作系统上运行它。这些操作系统包括Microsoft Windows、Solaris OS、Linux和Mac OS X。

如果你不喜欢使用IDE编写程序，请参阅1.2.2节或者1.2.3节。

1. 检查清单

为了编写第一个程序，你需要：

- JDK 6；
- NetBeans IDE。

你可以下载JDK和NetBeans IDE的捆绑软件包[1]。有用于Microsoft Windows、Solaris OS和Linux操作系统的软件包安装程序。

也可以分别下载和安装JDK和IDE。可以在下面网址单独下载JDK：

```
http://java.sun.com/javase/6/download.jsp
```

（注意确保你下载的是JDK，而不是JRE。）

可以在下面网址单独下载NetBeans IDE：

```
http://www.netbeans.org/downloads/
```

6

注意　本节中显示的屏幕截图反映的是运行在JDK 6 上的NetBeans IDE 5.0。编写本书时，5.0是最新的IDE版本。你阅读本书时，可能有了更新的IDE版本，但是这里演示的工作流程应该是相同的。

2. 创建第一个应用程序

第一个应用程序HelloWorldApp简单地显示问候语“Hello World!”。为了创建这个程序，你需要：

- **创建IDE工程**。创建IDE工程时，将创建一个环境，在其中构建（build）[2]和运行应用程序。使用IDE工程免去了通常与javac编译器和java启动器工具相关的配置工作。通过选择IDE中的一个菜单项，就可以构建和运行应用程序。
- **把代码添加到生成的源文件**。源文件包含使用Java编程语言编写的代码，你和其他程序员都能够理解它。作为IDE工程创建工作的一部分，会自动生成一个框架源文件。然后你修改这个源文件，加上“Hello World!”消息。
- **把源文件编译为.class文件**。IDE调用Java编程语言编译器（javac），它处理源文件并且把源文件的文本转换为Java虚拟机能够理解的指令。这个文件中包含的指令被称为字节

1. http://java.sun.com/javase/6/download.jsp
2. 构建指的是将源文件转换为可执行代码的过程。——译者注

码（bytecode）。

- **运行程序**。IDE调用Java应用程序启动器工具（java），它使用Java虚拟机运行应用程序。

创建IDE工程

创建IDE工程，操作步骤如下。

(1) 启动NetBeans IDE。

- 在Microsoft Windows系统中，可以在“Start”菜单中选择NetBeans IDE菜单项。
- 在Solaris OS和Linux系统中，进入IDE的bin目录并且输入./netbeans执行IDE启动器脚本。
- 在Mac OS X系统中，点击NetBeans IDE应用程序图标。

(2) 在NetBeans IDE中，选择File | New Project（见图1-4）。

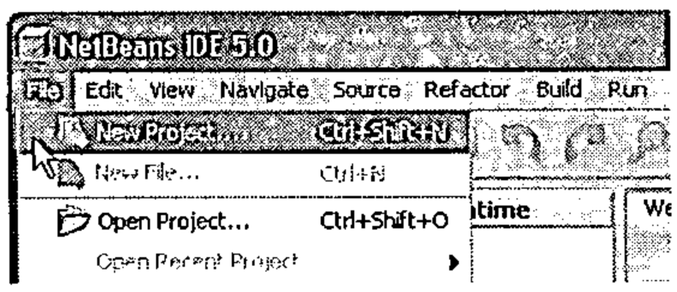


图1-4 选择NetBeans IDE的File | New Project菜单项

(3) 在New Project向导中，展开General分类项，选择Java Application（见图1-5）。

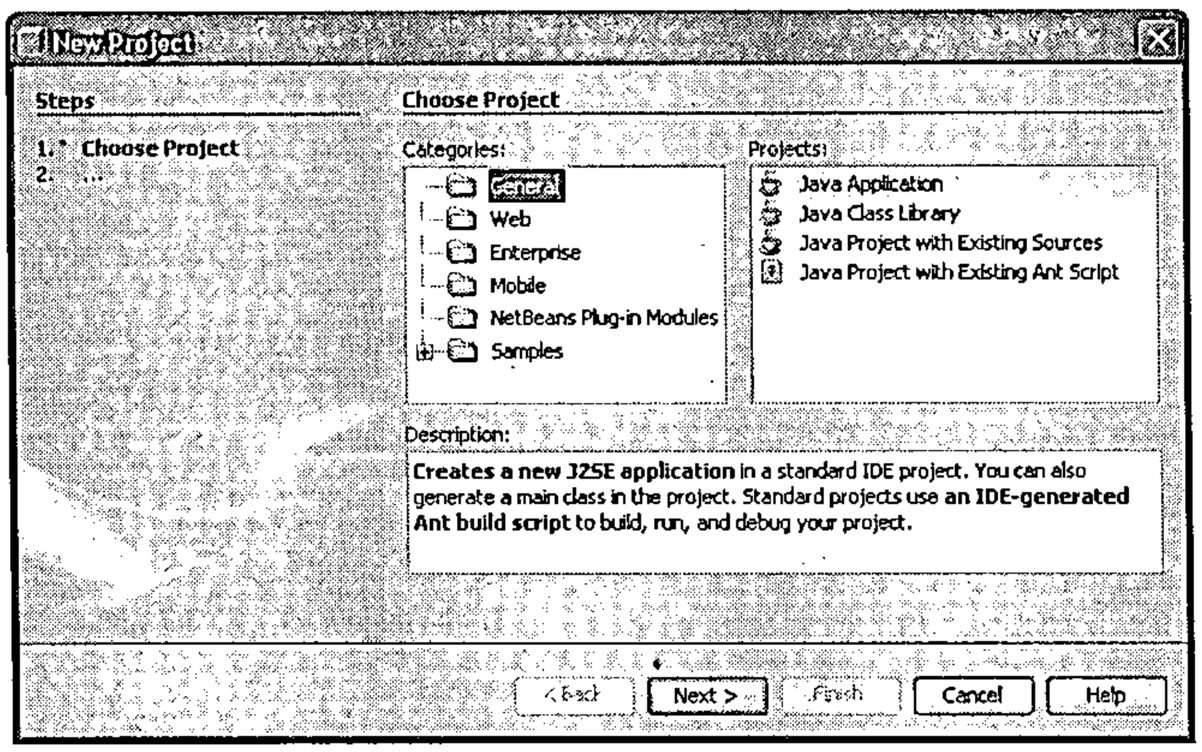


图1-5 在NetBeans IDE中New Project向导的Choose Project页面

(4) 在向导的Name and Location页面中，进行如下操作（见图1-6）：

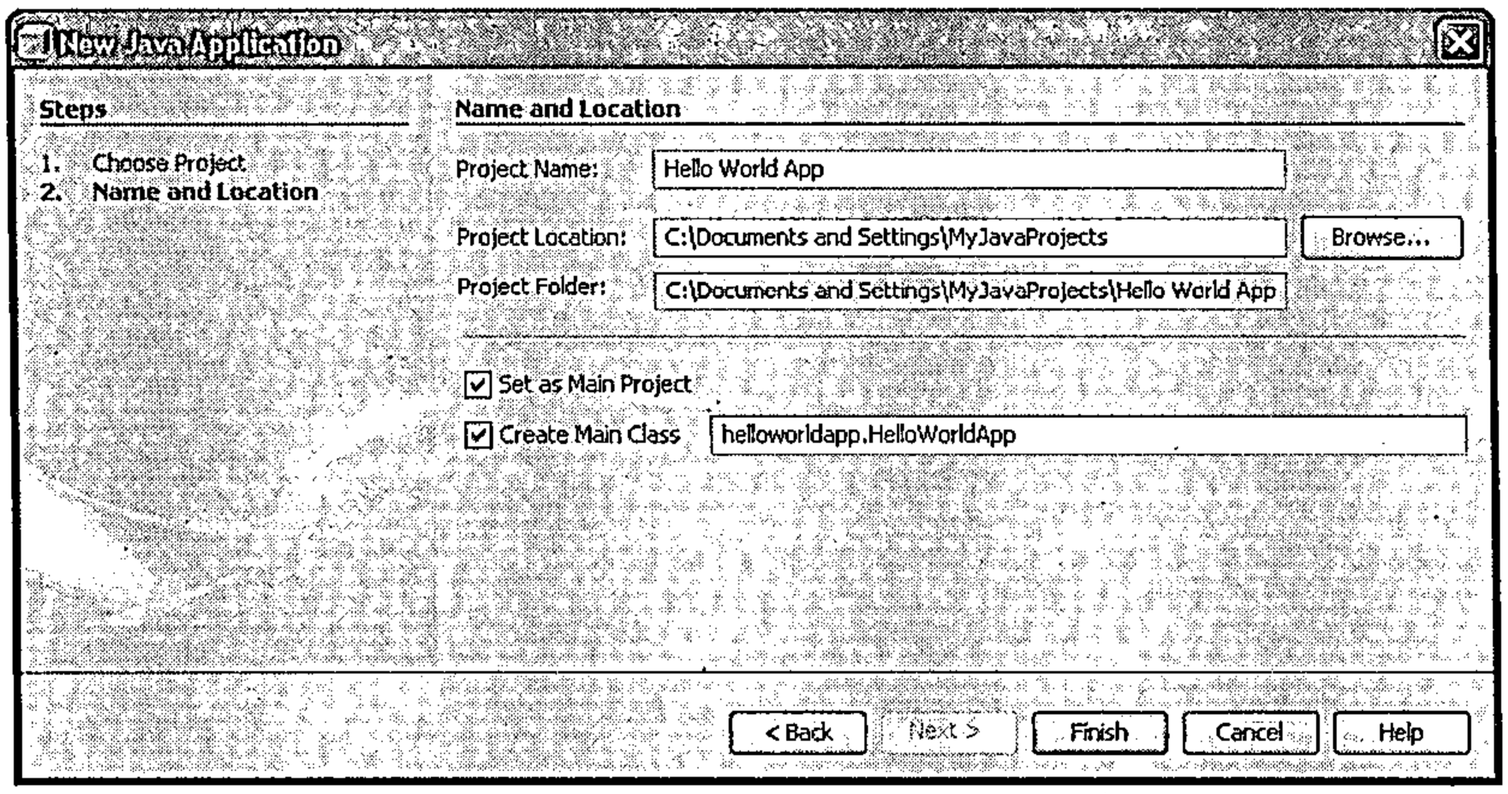


图1-6 NetBeans IDE中New Project向导的Name and Location页面

- 在Project Name字段中，输入Hello World App。

- 在Create Main Class字段中，输入helloworldapp.HelloWorldApp。
- 选中Set as Main Project复选框。

(5) 点击Finish。

现在已经创建了工程，并且在IDE中打开了。你应该看到如下组件：

- Projects窗口，它包含工程组件的树形视图，包括源文件、代码依赖的库等。
- Source Editor窗口，其中打开了名为HelloWorldApp的文件。
- Navigator窗口，在其中可以在被选中类的元素之间快速移动（见图1-7）。

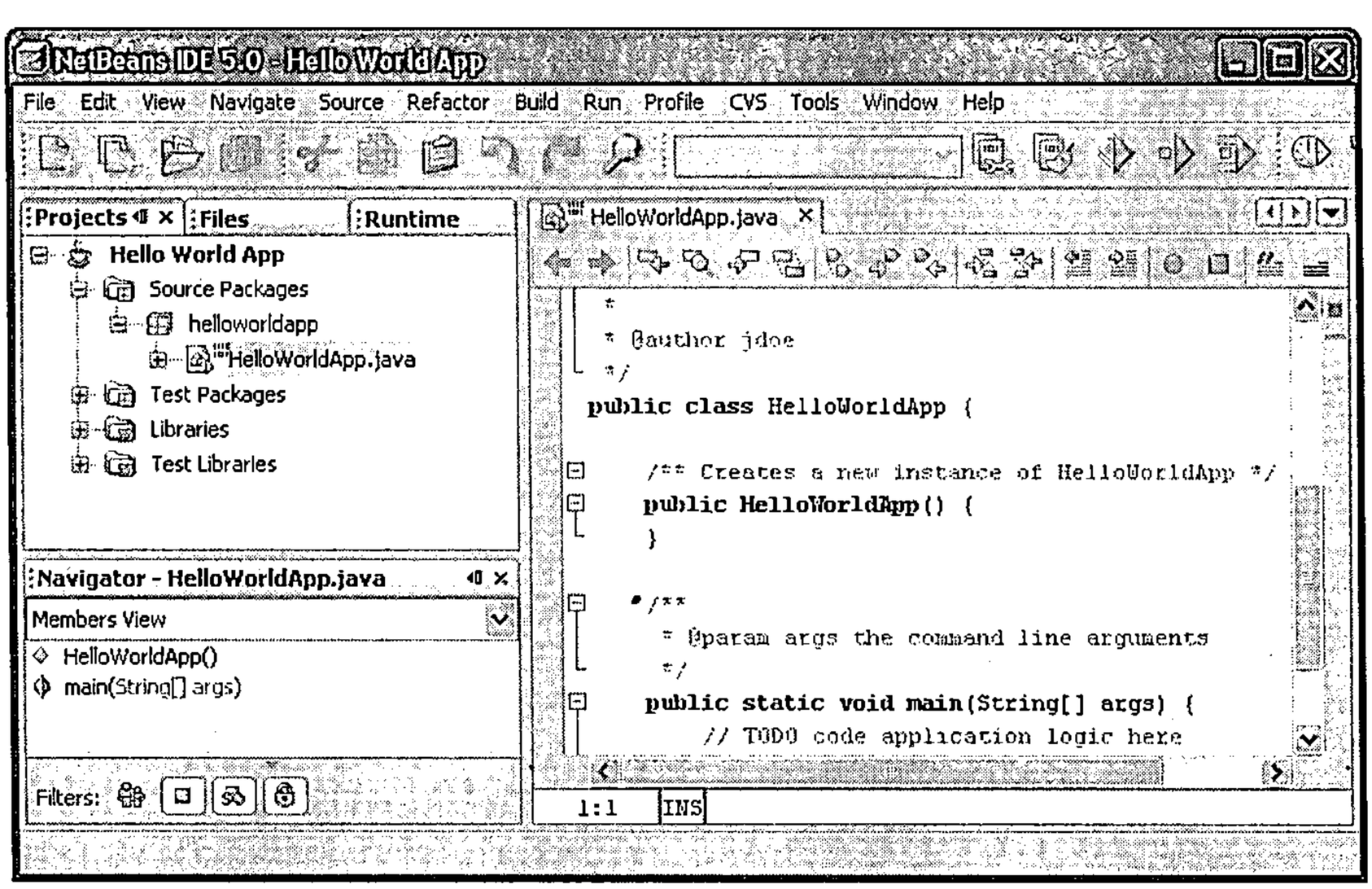


图1-7　打开了HelloWorldApp工程的NetBeans IDE

把代码添加到生成的源文件

因为我们在New Project向导中选中了Create Main Class复选框，所以IDE已经为你创建了框架类。通过把下面这一行

```
// TODO code application logic here
```

替换为

```
System.out.println("Hello World!"); // Display the string.
```

7～9 就可以把“Hello World!”消息添加到框架代码中。

还可以把代码

```
/**
 *
 * @author
 */
```

替换为

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
```

10 这4行是代码注释，不会影响程序的运行方式。后面的章节将讲解代码注释的使用和格式。

输入代码时要谨慎 严格按照前面所显示的输入所有代码、注释和文件名。编译器（javac）和启动器（java）是区分大小写的，所以必须保持大小写的一致性。换句话说，HelloWorldApp不等同于helloworldapp。

选择File | Save保存文件修改。

文件应该如下所示：

```
/*
 * HelloWorldApp.java
 *
 * Created on February 5, 2006, 6:43 PM
 *
 * To change this template, choose Tools | Template Manager
 * and open the template in the editor.
 */

package helloworldapp;

/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
public class HelloWorldApp {

  /** Creates a new instance of HelloWorldApp */
  public HelloWorldApp() {
  }

  /**
   * @param args the command line arguments
   */
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
  }

}
```

把源文件编译为.class文件

要编译源文件，选择IDE主菜单的Build | Build Main Project菜单项。

Output窗口打开，并且显示类似图1-8所示的输出。

11

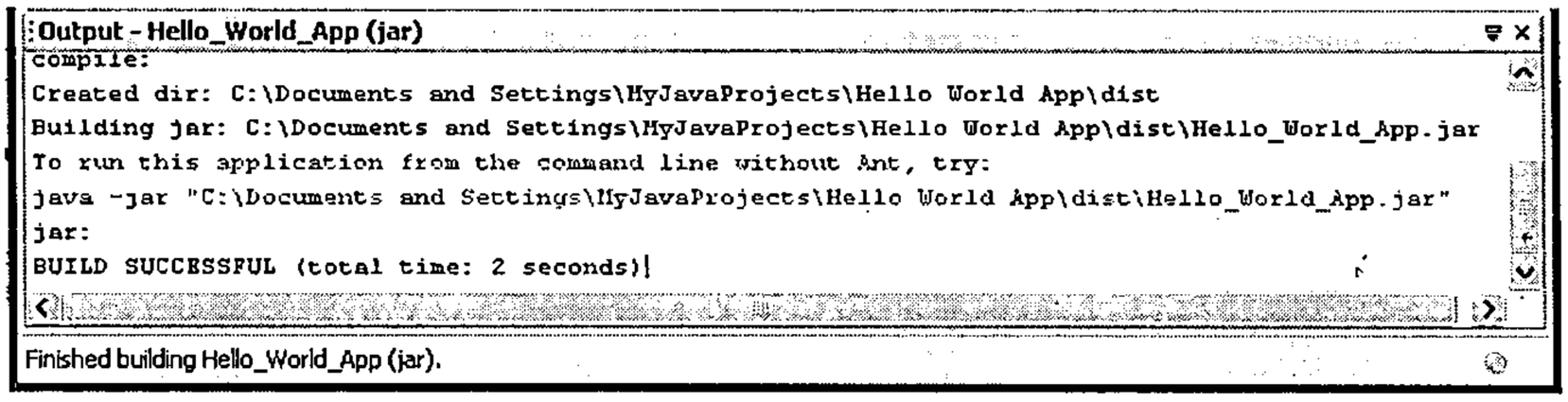


图1-8 Output窗口显示构建HelloWorld工程的结果

如果构建输出以“BUILD SUCCESSFUL”语句结束，那么祝贺你！你已经成功地编译了程序！

如果构建输出以“BUILD FAILED”语句结束，那么代码中可能有语法错误。错误在Output窗口中显示为超链接文本。双击这样的超链接就会定位到出现错误的源代码。你可以纠正错误，然后再次选择Build | Build Main Project。

构建工程时生成了HelloWorldApp.class字节码文件。打开Files窗口，然后展开Hello World App/build/classes/helloworldapp节点，就能看到生成的新文件在什么位置，如图1-9所示。

现在，编译完工程，可以运行应用程序了。

运行程序

在IDE的菜单栏中选择Run | Run Main Project。

12

图1-10显示你将看到的画面。



图1-9　Files窗口，显示生成的.class文件

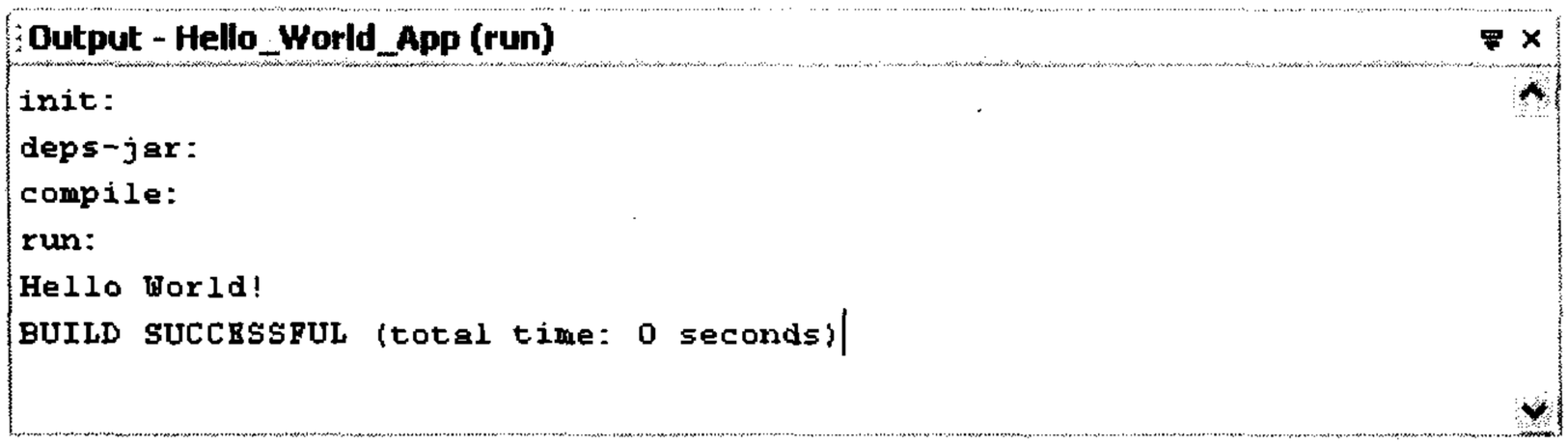


图1-10　程序在Output窗口中输出“Hello World!”（以及编译脚本的其他输出）

祝贺你！你的程序能够工作了！

3. NetBeans IDE的进一步介绍

在1.3节中将分析这个简单应用程序中的代码。虽然本书的其余部分没有专门说明如何使用NetBeans IDE，但是你可以轻松地使用这个IDE编写和运行示例代码。下面是使用这种IDE的一些技巧和可能用到的一些IDE特性。

- 在IDE中创建了工程之后，你可以使用New File向导把文件添加到工程中。选择File | New File，然后在向导中选择一个模板，比如Empty Java File模板。
- 使用IDE的Compile File（F9）和Run File（Shift-F6）命令，可以编译和运行单个文件（而不是整个工程）。如果使用Run Main Project命令，IDE就会运行被IDE关联为主工程的主类的文件。因此，如果在HelloWorldApp工程中创建了额外的类，然后试图使用Run Main Project命令运行这个文件，那么IDE就会运行HelloWorldApp，而不是这个文件。
- 对于包含一个以上源文件的示例应用程序，你可能希望创建单独的IDE工程。
- 在IDE中输入代码时，可能会定期地出现代码补全框。你可以忽略代码补全框并且继续输入，也可以选择使用它建议的表达式。如果不希望代码补全框自动出现，可以关闭这个特性。选择Tools | Options，点击Editor选项卡，然后去掉Auto Popup Completion Window复选框的选定。
- 如果试图改变Projects窗口中源文件的节点名称，IDE会显示Rename对话框，引导你重新命名类和更新引用这个类的代码。点击Next显示Refactoring窗口，它包含可以进行的修改的树形视图。然后点击Do Refactoring进行修改[1]。如果工程中只有一个类，那么这一系列的点击似乎是多余的，但是当你的改动影响到大型工程中其余部分的代码时，这些操作就非常有用了。

13

1. 这里的修改就是所谓重构（refactoring），特指改变代码的内部结构，但不影响其外部行为的修改。——译者注

- 要获得NetBeans IDE特性的更详尽说明，请参阅NetBeans IDE的Docs和Support页面[1]，或者浏览IDE的Help菜单的文档。

1.2.2 在Microsoft Windows平台上编写“Hello World!”

现在开始编写你的第一个应用程序！下面的说明适用于Windows XP专业版、Windows XP家庭版、Windows Server 2003、Windows 2000专业版和Windows Vista的用户。Solaris OS和Linux平台的编程说明请参阅1.2.3节。

1. 检查清单

为了编写第一个程序，你需要：

(1) JDK 6

现在可以下载Windows版本[2]。（注意，确保你下载的是JDK，而不是JRE。）请参考安装说明[3]。

(2) **文本编辑器**

在这个例子中我们使用记事本（NotePad），它是Windows平台自带的简单编辑器。如果你使用其他文本编辑器，这些说明也适用。

这两个工具完全能够满足你编写第一个应用程序的需要。

2. 创建第一个应用程序

第一个应用程序`HelloWorldApp`简单地显示问候语“Hello World!”。要创建这个程序，你需要：

- **创建源文件**，源文件包含使用Java编程语言编写的代码，你和其他程序员都能够理解它。你可以使用任何文本编辑器创建和编辑源文件。
- **把源文件编译为.class文件**，Java编程语言编译器（`javac`）处理源文件并且把源文件的文本转换为Java虚拟机能够理解的指令。这个文件中包含的指令被称为字节码（bytecode）。
- **运行程序**，Java应用程序启动器工具（`java`）使用Java虚拟机运行应用程序。

14

创建源文件

创建源文件，有两个做法可以选择：

- 把文件`HelloWorldApp.java`[4]保存到你的计算机上，这样可以避免很多输入操作。然后，可以直接跳到“把源文件编译为.class文件”部分。
- 或者，可以按照下面的指导（操作要多一些）。

首先打开编辑器。可以选择Start菜单的Programs > Accessories > Notepad打开记事本。在新文档中，输入如下代码：

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
```

---

1. http://www.netbeans.org/kb/
2. http://java.sun.com/javase/6/download.jsp
3. http://java.sun.com/javase/6/webnotes/install/
4. tutorial/getStarted/application/examples/HelloWorldApp.java

```
    }
}
```

输入代码时要谨慎　严格按照前面所显示的输入代码、注释和文件名。编译器（javac）和启动器（java）是区分大小写的，所以必须保持大小写的一致性。换句话说，HelloWorldApp不等同于helloworldapp。

把代码保存到名为HelloWorldApp.java的文件中。在记事本中的操作步骤是：首先选择File > Save As菜单项。然后，在Save As对话框中：

(1) 使用Save in下拉框，指定要保存文件的文件夹（目录）。在这个例子中，目录是C盘的java目录。

(2) 在File name文本框中，输入"HelloWorldApp.java"，包括引号。

(3) 在Save as type下拉框中，选择Text Documents (*.txt)。

15

(4) 在Encoding下拉框中，保持编码为ANSI不变。

完成操作时，对话框应该如图1-11所示。

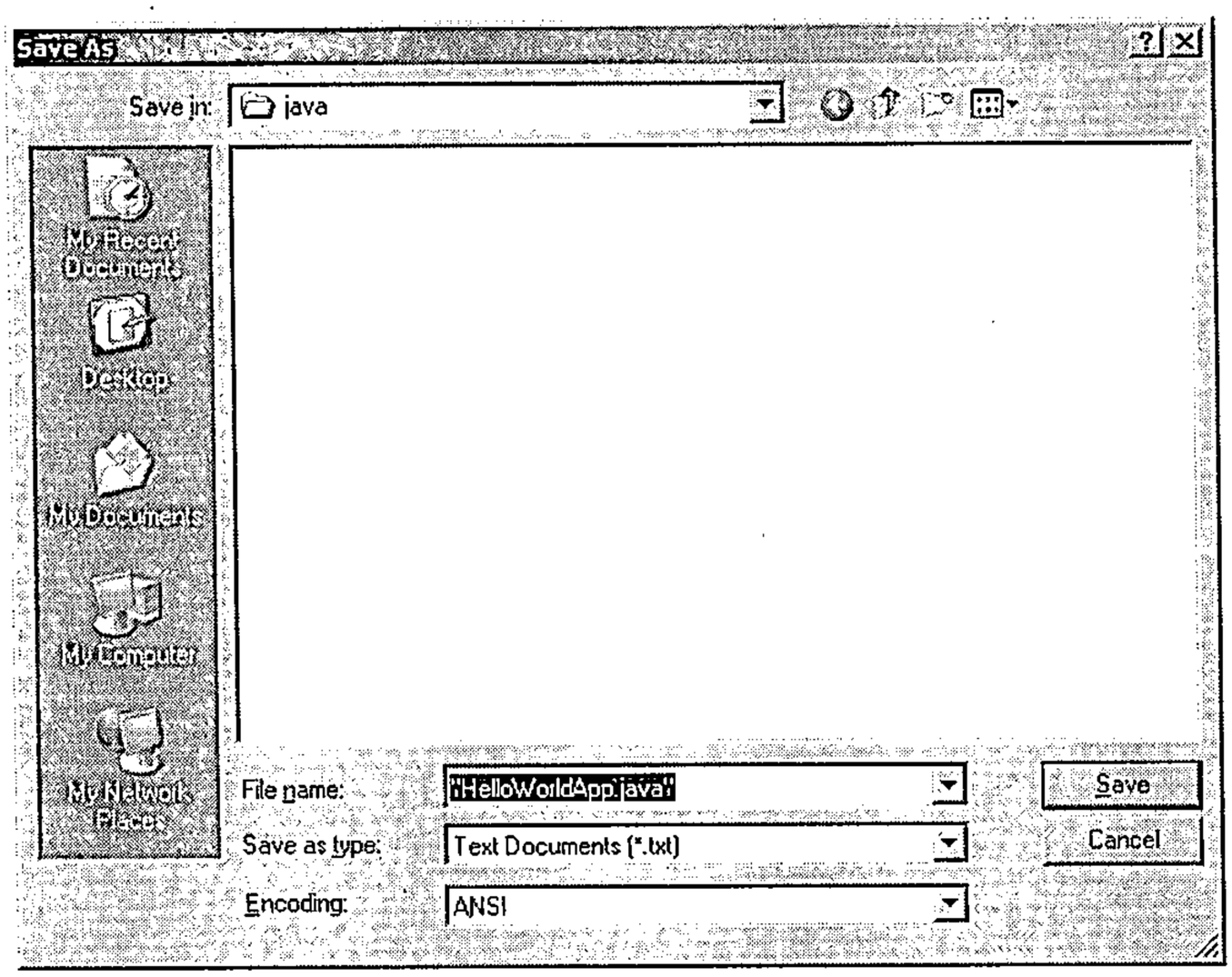


图1-11　点击Save之前的Save As对话框

现在点击Save，退出记事本。

把源文件编译为.class文件

打开shell，或者说“命令”窗口。为了进行这一操作，可以在Start菜单选择Command Prompt（Windows XP），或者选择Run，然后输入cmd。shell窗口类似于图1-12所示。

命令提示符显示你的当前目录（current directory）。在打开命令提示符时，你的当前目录通

16

常是Windows XP的主目录（如图1-12所示）。

要编译源代码文件，应该将当前目录改变为文件所在的目录。例如，如果你的源代码目录是C盘上的java目录，那么在命令提示符中输入以下命令并按回车键：

```
cd C:\java
```

现在，命令提示符应该变成C:\java>。

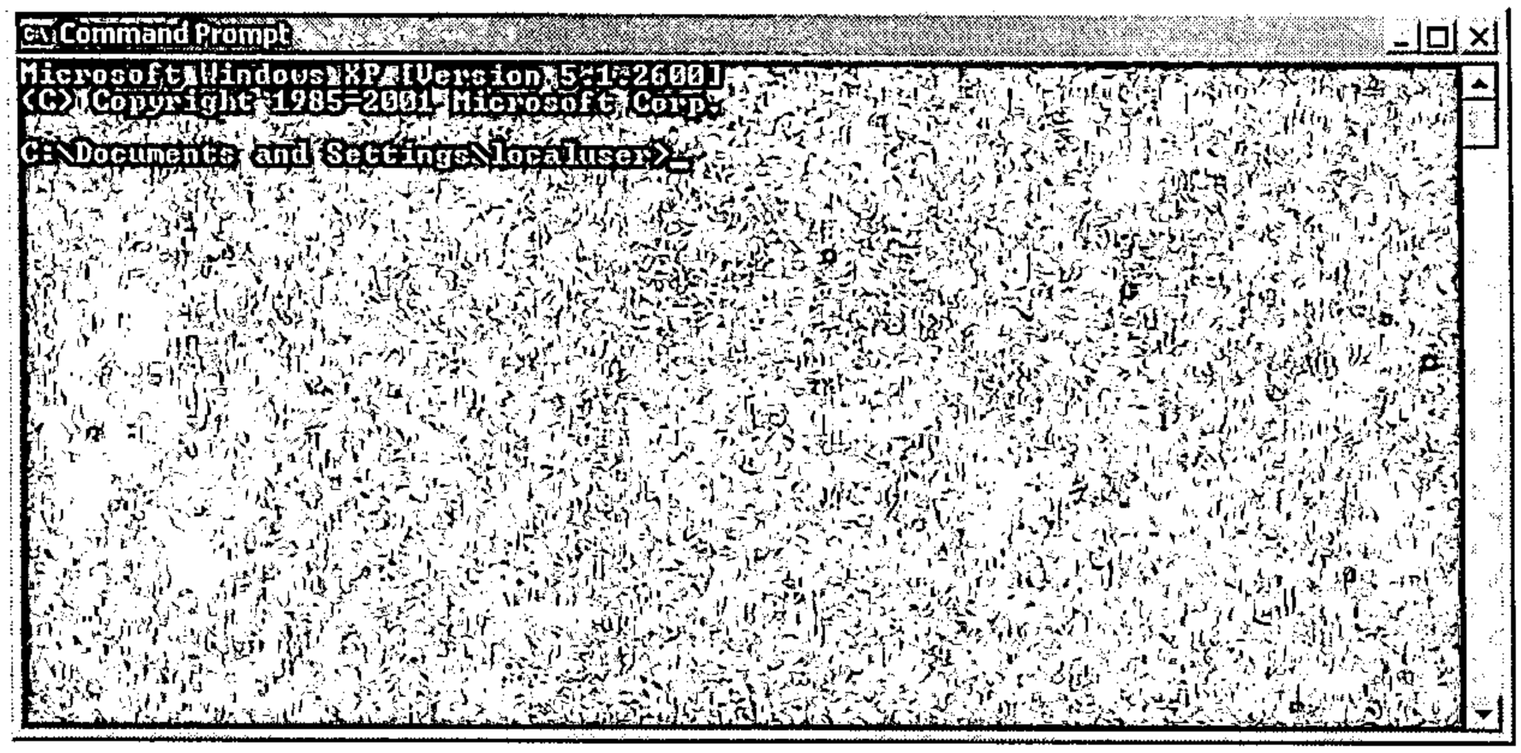


图1-12 shell窗口

注意 要改变为另一个驱动器上的目录，就必须多输入一个命令：驱动器的名称。例如，要改变为D盘上的java目录，就必须输入驱动器符号D: ，如图1-13所示。

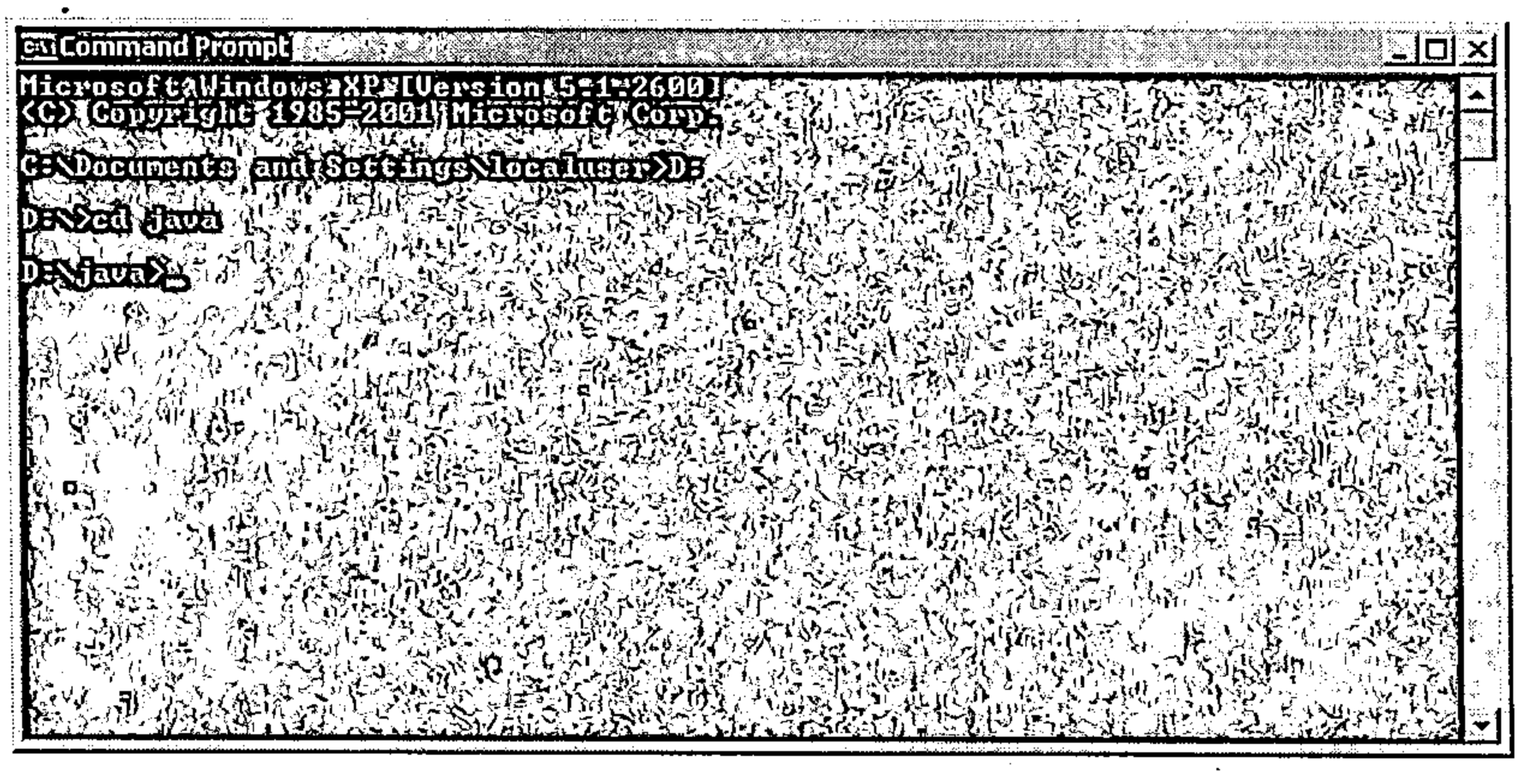


图1-13 改变到另一个驱动器上的目录

如果在命令提示符中输入dir，也应该能看到源文件，如图1-14所示。

17

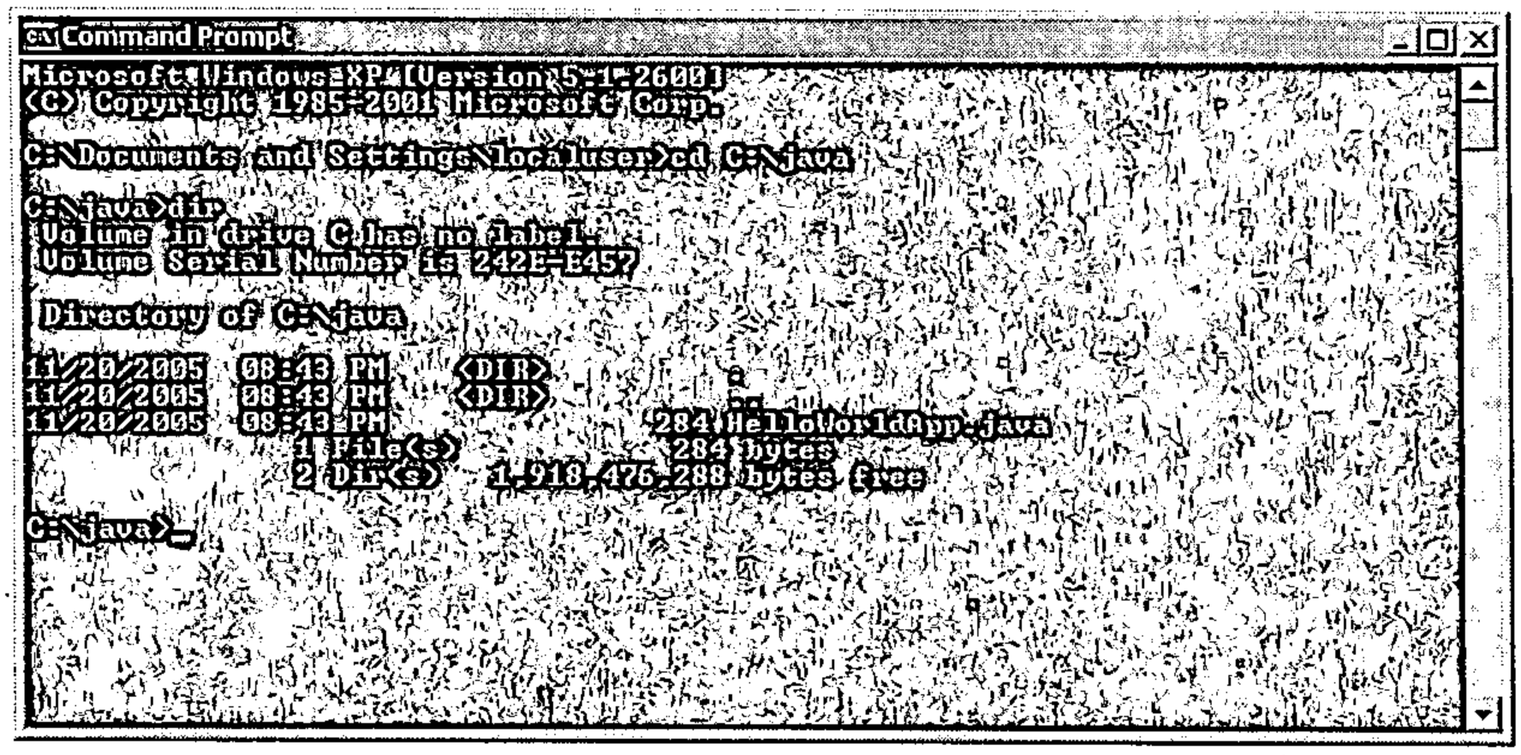


图1-14 目录清单显示.java源文件

现在准备进行编译。在命令提示符中，输入如下命令并且按回车键：

```
javac HelloWorldApp.java
```

编译器生成字节码文件HelloWorldApp.class。在命令提示符中输入dir查看生成的新文件，如图1-15所示。

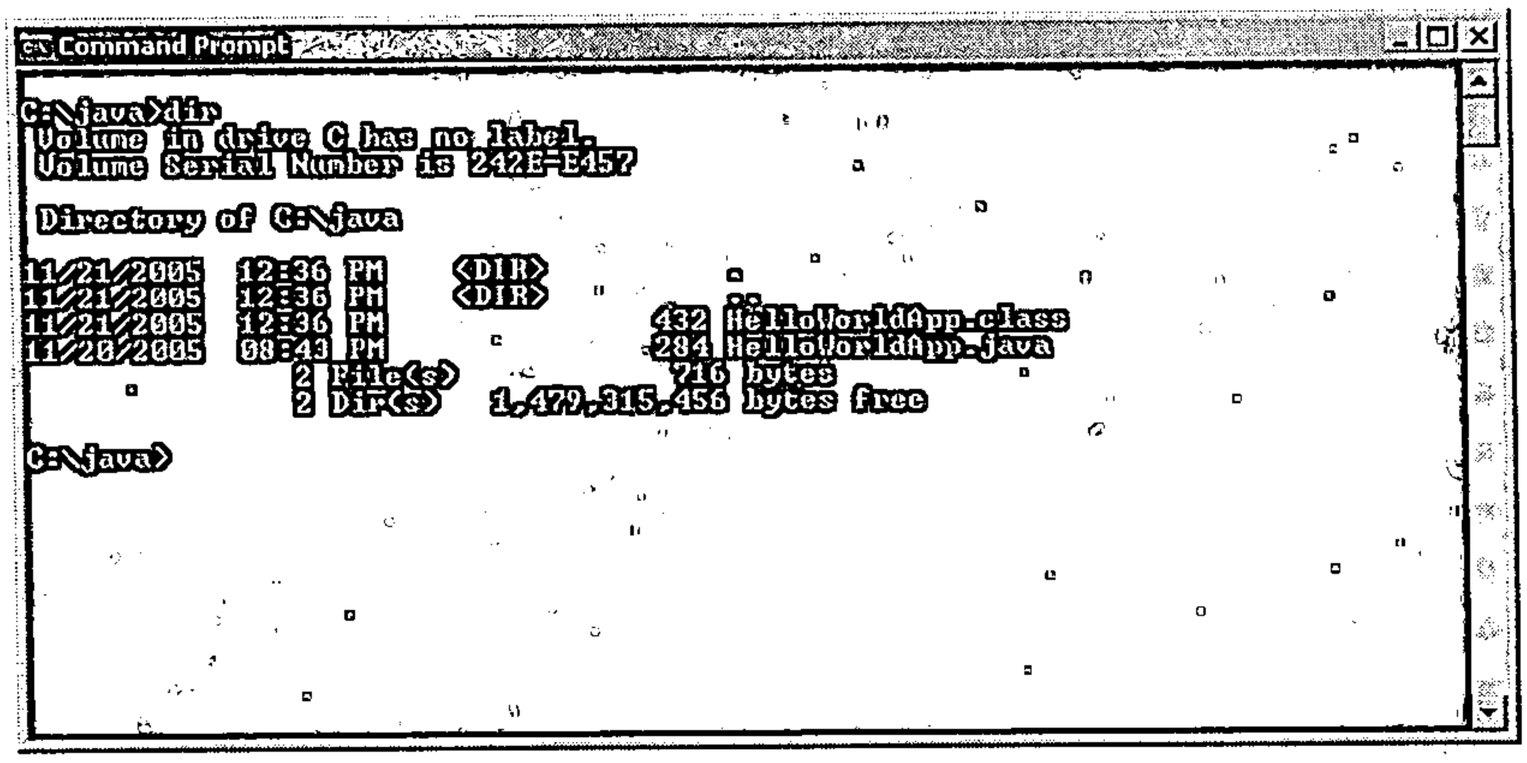


图1-15　目录清单显示生成的.class文件

有了.class文件，就可以运行程序了。

运行程序

在相同的目录中，在命令提示符中输入以下命令：


```
java HelloWorldApp
```

图1-16所示为应该看到的画面。

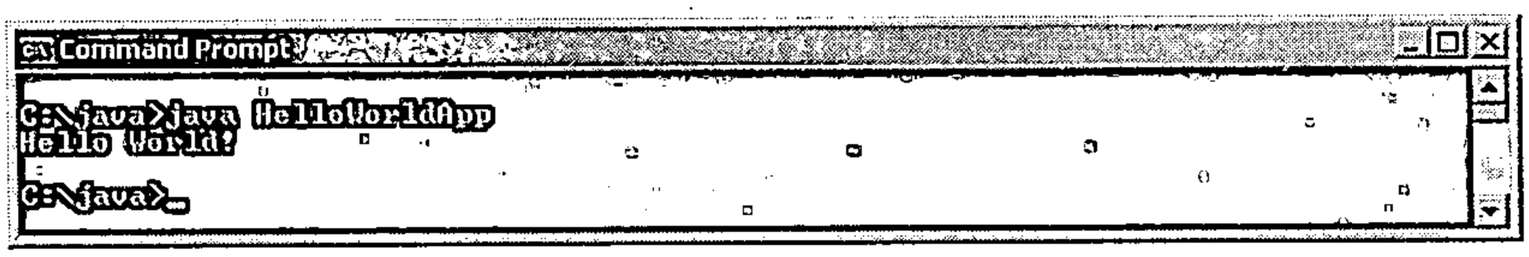


图1-16　程序在屏幕上输出“Hello World!”

祝贺你！你的程序能够工作了！

1.2.3　在 Solaris OS 和 Linux 平台上编写“Hello World!”

现在开始编写你的第一个应用程序！下面的说明适用于Solaris OS和Linux用户。Microsoft Windows的编程说明请参阅1.2.2节。

1. 检查清单

为了编写第一个程序，你需要：

(1) JDK 6

现在你可以下载Solaris OS或者Linux版本[1]。（注意，确保你下载的是JDK，而不是JRE。）请参考安装说明[2]。

(2) 文本编辑器

在这个例子中我们使用Pico，它是很多基于UNIX的平台上的编辑器。如果你使用其他文本

---

1. http://java.sun.com/javase/6/download.jsp
2. http://java.sun.com/javase/6/webnotes/install/

编辑器，比如vi或者emacs，这些说明也适用。

这两个工具能够完全满足你编写第一个应用程序的需要。

19

2. 创建第一个应用程序

第一个应用程序HelloWorldApp简单地显示问候语“Hello World!”。为了创建这个程序，你需要：

- **创建源文件**。源文件包含使用Java编程语言编写的代码，你和其他程序员都能够理解它。可以使用任何文本编辑器创建和编辑源文件。
- **把源文件编译为.class文件**。Java编程语言编译器（javac）处理源文件并且把源文件的文本转换为Java虚拟机能够理解的指令。这个.class文件中包含的指令被称为字节码（bytecode）。
- **运行程序**。Java应用程序启动器工具（java）使用Java虚拟机运行应用程序。

创建源文件

创建源文件，有两个做法可以选择：

- 把文件HelloWorldApp.java[1]保存到你的计算机上，这样可以避免很多输入操作。然后，可以直接跳到“把源文件编译为.class文件”部分。
- 或者，可以按照下面的指导（操作要多一些）。

首先打开shell，或者说“终端（terminal）”窗口（见图1-17）。

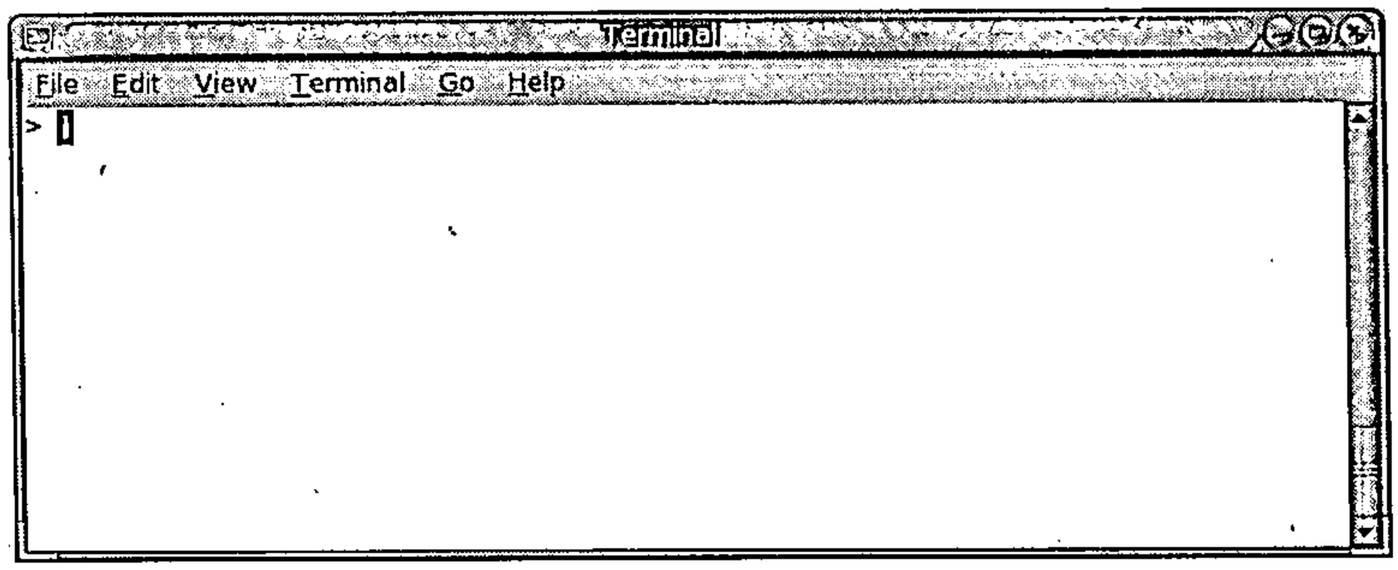


图1-17 新的终端窗口

20

打开命令提示符时，当前目录（current directory）通常是主目录（home directory）。任何时候都可以在命令提示符中输入cd并且按回车键，把当前目录改变为主目录。

应该把创建的源文件保存在单独的目录中。可以使用mkdir命令创建一个目录。例如，为了在主目录中创建java目录，可以使用如下命令：

```
cd
mkdir java
```

为了把当前目录改变为这个新的目录，可以输入：

```
cd java
```

现在可以开始创建源文件了。

在命令提示符中输入pico并且按下回车键，打开Pico。如果系统回应的信息为：“pico: command not found”，那么很可能是因为Pico不可用。更多的信息请联系你的系统管理员，或者使用另一种编辑器。

启动Pico时，它会显示新的空白缓冲区（buffer），你在这个区域中输入代码。

---

1. tutorial/getStarted/application/examples/HelloWorldApp.java

在新的缓冲区中输入下列代码：

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
  }
}
```

输入代码时要谨慎　严格按照前面所显示的输入代码、注释和文件名。编译器（javac）和启动器（java）是区分大小写的，所以必须保持大小写的一致性。换句话说，HelloWorldApp不等同于helloworldapp。

把代码保存在名为HelloWorldApp.java的文件中。要在Pico编辑器中完成这一操作，首先按下Ctrl+O，然后在命令提示符的底部将看到“File Name to write:”，输入你希望文件
21 所在的目录，后面跟着HelloWorldApp.java。例如，如果希望把文件HelloWorldApp.java保存到目录/home/jdoe/java中，就输入/home/jdoe/java/HelloWorldApp.java并且按下回车键。

按下Ctrl+X退出Pico。

把源文件编译为.class文件

打开另一个shell窗口。为了编译源文件，需要把当前目录改变为你的文件所在的目录。例如，如果源文件目录是/home/jdoe/java，就在命令提示符中输入如下命令并且按下回车键：

```
cd /home/jdoe/java
```

如果在命令提示符中输入pwd，就应该看到当前目录，在这个例子中应该已经改变到了/home/jdoe/java。

如果在命令提示符中输入ls，就应该看到你的文件（见图1-18）。

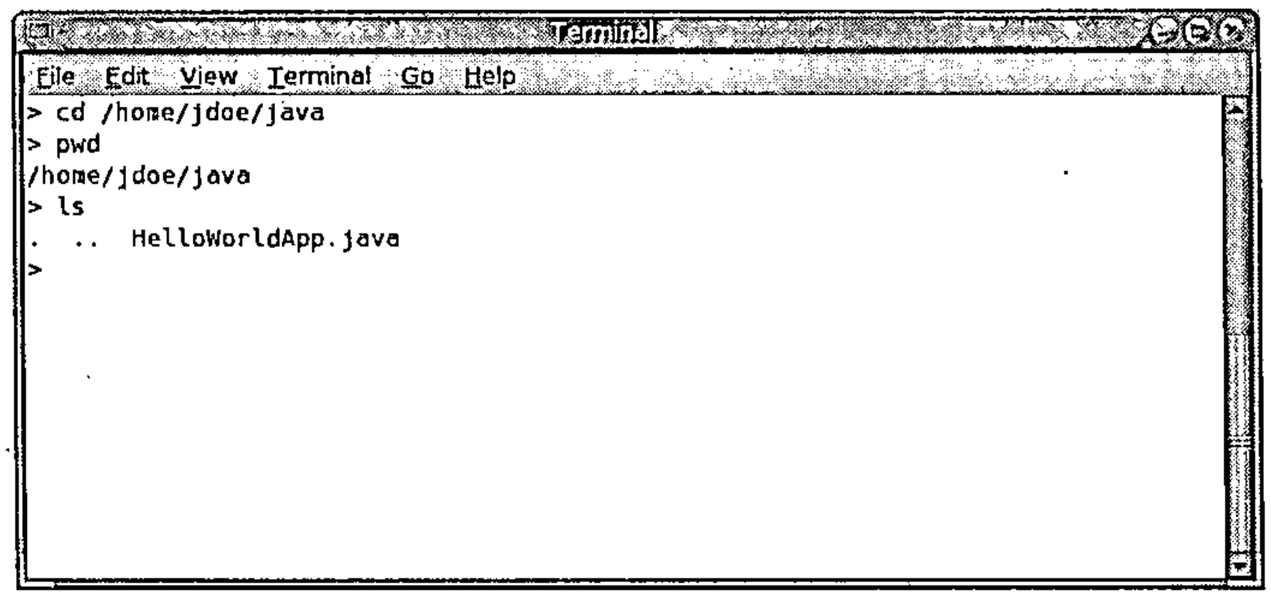


图1-18　ls命令的结果，显示.java源文件

现在准备编译源文件。在命令提示符中，输入如下命令并且按下回车键：

```
javac HelloWorldApp.java
```

编译器生成字节码文件HelloWorldApp.class。在命令提示符中输入ls查看生成的新文
22 件（见图1-19）。

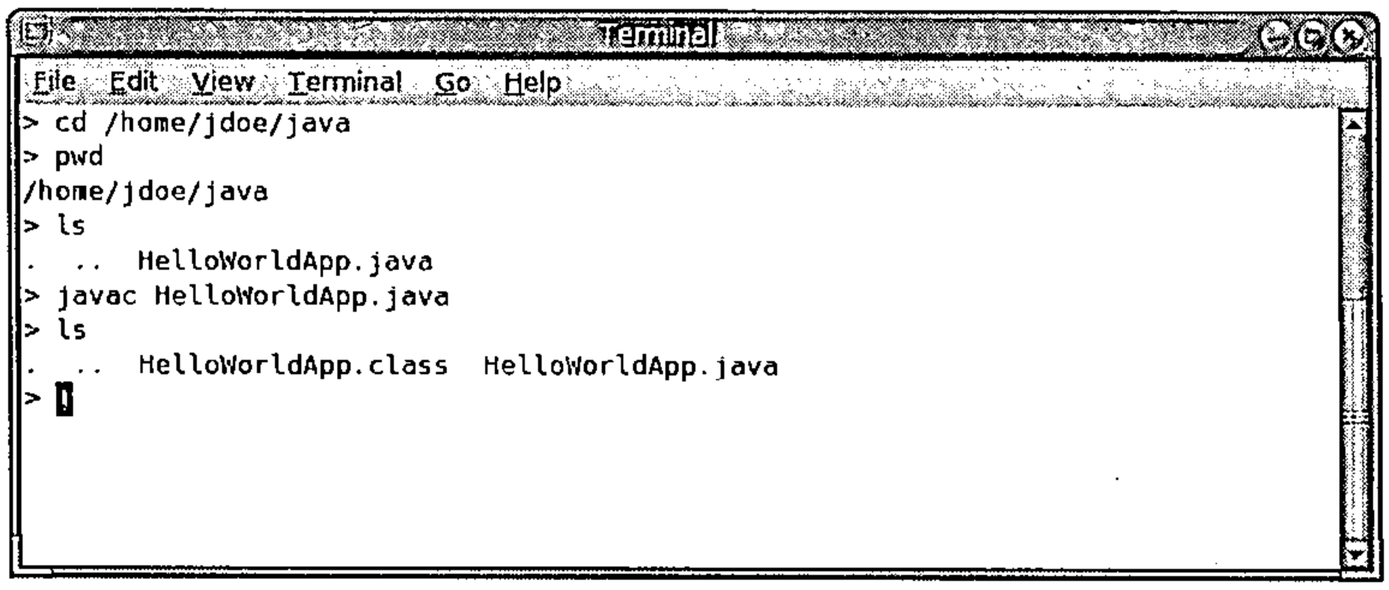


图1-19 ls命令的结果，显示生成的.class文件

准备好.class文件，就可以运行程序了。

运行程序

在相同的目录中，在命令提示符中输入如下命令：

```
java HelloWorldApp
```

图1-20所示为应该看到的画面。

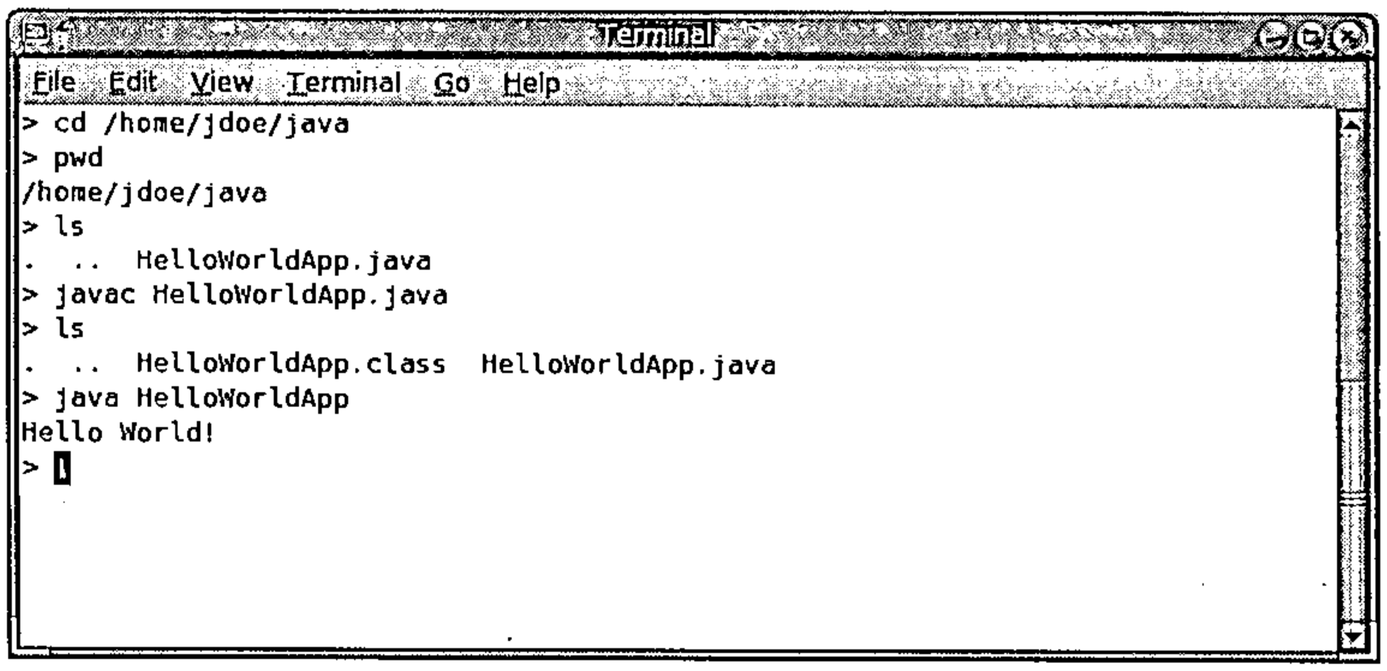


图1-20 屏幕上输出“Hello World!”

祝贺你！你的程序能够工作了！

23

1.3 分析“Hello World!”应用程序

现在你已经看过了“Hello World!”应用程序（并且可能已经编译和运行了），你可能希望了解它是如何工作的。下面再次列出它的代码：

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
  }
}
```

应用程序“Hello World!”由3个主要部分构成：源代码注释、HelloWorldApp类定义和main方法。下面的讲解将使你对这些代码有一个基本的了解，在阅读后续的章节时，将理解代码中更深层次的含义。

1.3.1 源代码注释

下面的粗体文本定义应用程序“Hello World!”的注释（comment）。

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
  }
}
```

注释会被编译器忽略，但是对其他程序员非常有用。Java编程语言支持三种注释类型：

24

- **/* *text* */**　编译器忽略从/*到*/之间的所有内容。
- **/** *documentation* */**　这表示文档注释（documentation comment，简写为doc comment）。编译器忽略这种类型的注释，就像忽略/*和*/类型的注释一样。当javadoc工具准备自动生成的文档时，它会使用文档注释。关于javadoc的更多信息请参阅Javadoc工具文档[1]。
- **//** *text*　编译器忽略从//到行尾的所有内容。

1.3.2 HelloWorldApp类定义

下面的粗体文本是应用程序“Hello World!”的类定义块的开始。

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
    System.out.println("Hello World!"); // Display the string.
  }
}
```

如上所示，类定义的最基本形式是：

```
class name {
  ...
}
```

关键字class开始名为name的类的类定义，每个类的代码出现在前面用粗体标出的前后花括号之间。第2章概述类的概念，第4章详细地讨论类。现在，知道每个应用程序都以类定义开始就足够了。

1.3.3 main方法

25

下面的粗体文本是main方法的定义。

```
/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp {
  public static void main(String[] args) {
```

---

1. docs/guide/javadoc/index.html

```
    System.out.println("Hello World!"); // Display the string.
  }
}
```

在Java编程语言中，每个应用程序都必须包含main方法，它的签名如下：

```
public static void main(String[] args)
```

可以任意安排修饰符public和static的前后顺序（public static或者static public），但是习惯的用法是public static。可以把参数命名为任何名称，但是大多数程序员选择使用“args”或者“argv”。

main方法类似于C和C++语言中的main函数；它是应用程序的入口点，随后它将调用你的程序需要的所有其他方法。

main方法接受单一参数：String类型的元素的数组。

```
public static void main(String[] args)
```

这个数组是一种机制，运行时系统通过它把信息传递给你的应用程序。数组中的每个字符串称为命令行参数（command-line argument）。命令行参数使用户可以影响应用程序的操作，而无需重新编译它。例如，一个排序程序可以允许用户使用如下命令行参数指定按照降序对数据进行排序：

```
-descending
```

应用程序“Hello World!”忽略它的命令行参数，但是你要认识到这样的参数的确存在。

最后，代码

```
System.out.println("Hello World!");
```

使用API的System类把消息“Hello World!”发送到标准输出。

1.4 常见问题（和解决方案）

1.4.1 编译器问题

1. Microsoft Windows系统中的常见错误消息

```
'javac' is not recognized as an internal or external command, operable program or batch file
```

如果出现这个错误，就表示Windows找不到编译器（javac）。

有一种方法通知Windows在哪里寻找javac。假设你把JDK安装在C:\jdk6。在命令提示符中输入如下命令，然后按下回车键：

```
C:\jdk6\bin\javac HelloWorldApp.java
```

如果你选择这种方式，每次编译或者运行程序时，都必须在javac和java命令前面加上C:\jdk6\bin\。为了避免额外的输入，可以参考JDK 6安装说明的“更新PATH变量（Update the PATH variable）”一节[1]。

2. UNIX系统中的常见错误消息

```
javac: Command not found
```

如果出现这个错误，就表示UNIX找不到编译器（javac）。

有一种方法通知UNIX在哪里寻找javac。假设你把JDK安装在/usr/local/jdk6。在命令提示符中输入如下命令，然后按下回车键：

1. http://java.sun.com/javase/6/webnotes/install/jdk/install-windows.html

```
/usr/local/jdk6/javac HelloWorldApp.java
```

注意　如果你选择这种方式，每次编译或者运行程序时，都必须在javac和java命令前面加上/usr/local/jdk6。为了避免额外的输入，可以把这些信息添加到PATH变量中。完成这一操作的步骤因当前运行的shell不同而不同。

3. 语法错误（所有平台）

如果你错误地输入了程序的某个部分，编译器可能会指出语法（syntax）错误。错误消息通常显示错误类型、检测到的错误出现的行号、该行的代码和代码中错误的位置。下面是语句末尾遗漏了分号（;）所导致的错误：

```
testing.java:14: ';' expected.
System.out.println("Input has " + count + " chars.")
                                                    ^
1 error
```

有时候编译器猜测不出你的意图，就会输出含混的错误消息；有时候错误跨越了若干行代码，就会输出多个错误消息。例如，下面的代码段中粗体显示的行遗漏了分号（;）：

```
while (System.in.read() != -1)
    count++
System.out.println("Input has " + count + " chars.");
```

在处理这段代码时，编译器给出两个错误消息：

```
testing.java:13: Invalid type expression.
        count++
              ^
testing.java:14: Invalid declaration.
    System.out.println("Input has " + count + " chars.");
                       ^
2 errors
```

因为编译器处理到count++之后时，编译器的状态表示它处理到了表达式的中间，所以它给出两个错误消息。没有分号，编译器就没办法知道语句结束了。

如果发现任何编译器错误，那么程序就不会被成功编译，并且不会创建.class文件。小心检查程序，修改所有错误，然后再次进行尝试。

4. 语义错误

除了检验程序在语法上的正确性之外，编译器还会检查其他方面的基本正确性。例如，每当你使用未经初始化的变量时，编译器都会给出警告：

```
testing.java:13: Variable count may not have been initialized.
        count++
        ^
testing.java:14: Variable count may not have been initialized.
    System.out.println("Input has " + count + " chars.");
                                      ^
2 errors
```

程序同样不会被成功编译，并且编译器不会创建.class文件。你需要修改错误，再次进行尝试。

1.4.2　运行时问题

1. Microsoft Windows系统上的错误消息

```
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp
```

如果出现这个错误，就表示java找不到你的字节码文件HelloWorldApp.class。

java试图查找.class文件的一个位置是当前目录。所以，如果你的.class文件位于C:\java，就应该把当前目录改变到这里。要改变目录，可以在命令提示符中输入如下命令并且按下回车键：

```
cd c:\java
```

命令提示符应该改变为C:\java>。如果在命令提示符中输入dir，就应该看到你的.java和.class文件。现在再次输入java HelloWorldApp。

如果问题仍然存在，可能必须改变你的CLASSPATH变量。为了查看是否必须这样做，使用如下命令清空类路径：

```
set CLASSPATH=
```

现在再次输入java HelloWorldApp。如果程序能够工作，就必须改变CLASSPATH变量。要设置这个变量，可以参考JDK 6安装说明的“更新PATH变量（Update the PATH variable）”一节。按照相同的方式设置CLASSPATH变量。

2. UNIX系统上的错误消息

```
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp
```

如果出现这个错误，就表示java找不到你的字节码文件HelloWorldApp.class。

java试图查找.class文件的一个位置是当前目录。所以，如果你的.class文件位于/home/jdoe/java，就应该把当前目录改变到这里。为了改变目录，可以在命令提示符中输入如下命令并且按下回车键：

29

```
cd /home/jdoe/java
```

如果在命令提示符中输入pwd，就应该看到/home/jdoe/java。如果在命令提示符中输入ls，就应该看到你的.java和.class文件。现在再次输入java HelloWorldApp。

如果问题仍然存在，可能必须改变你的CLASSPATH环境变量。为了查看是否必须这样做，使用如下命令清空类路径：

```
unset CLASSPATH
```

再次输入java HelloWorldApp。如果程序能够工作，就必须使用改变PATH变量的方式改变CLASSPATH变量。

初级程序员常犯的错误是试图对编译器生成的.class文件运行java启动器。例如，如果你试图使用java HelloWorldApp.class（而不是java HelloWorldApp）命令运行程序，就会出现如下错误消息：

```
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp/class
```

参数应该是你希望使用的类的名称，而不是文件名。

3. 没有定义main方法

Java虚拟机要求执行的类具有main方法，从这里开始执行应用程序。1.3节详细地讨论了main方法。如果你遗漏了这个方法，就会在运行时出现如下错误：

```
Exception in thread "main" java.lang.NoSuchMethodError: main
```

问题和练习：入门

问题

1. 当你编译一个用Java编程语言写的程序时，编译器将供人阅读的源代码文件转换为Java虚拟机可以理解的与平台无关的代码。这种与平台无关的代码叫作什么？

30

2. 以下哪项不是合法的注释：
   a. `/** comment */`
   b. `/* comment */`
   c. `/* comment`
   d. `// comment`
3. 如果运行时出现如下错误，你应该最先检查什么？

```
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorldApp.java
```

4. main方法的签名必须包含关键字public和static。这两个关键字的前后顺序重要吗？
5. main方法接受单一参数。按照惯例，这个参数的名称是什么？

练习

1. 修改HelloWorldApp.java程序，使它显示“Hola Mundo!”而不是“Hello World!”。
2. 下面是稍经修改的HelloWorldApp版本：

```
// HelloWorldApp2.java

// INTENTIONALLY UNCOMPILABLE!

/**
 * The HelloWorldApp class implements an application that
 * simply prints "Hello World!" to standard output.
 */
class HelloWorldApp2 {
  public static void main(String[] args) {
    System.out.println("Hello World!); // Display the string.
  }
}
```

这个程序有一个错误。修改这个错误，使程序能够成功地编译并运行。这个错误是什么？

答案

可以在以下位置找到“问题”和“练习”的答案：


```
tutorial/getStarted/QandE/answers.html
```

第2章

面向对象的编程概念

如果你以前从未使用过面向对象的编程语言，那么在开始编写代码之前就需要理解几个基础概念。本章向你介绍对象、类、继承、接口和包。每段讨论都把重点放在用现实世界的例子阐述这些概念，同时介绍Java编程语言的语法。

2.1 对象是什么

对象是理解面向对象（object-oriented）技术的关键。现在环顾四周，你能发现很多现实世界对象的例子：你的狗、你的桌子、你的电视机和你的自行车等。

现实世界的对象具有两个性质：它们都具有状态（state）和行为（behavior）。例如，狗具有状态（名字、颜色、品种和是否饥饿）和行为（叫、抓东西和摇尾巴）。自行车也具有状态（当前挡位、当前脚踏板步调[1]和当前速度）和行为（换挡、改变脚踏板步调和使用刹车）。认识现实世界中对象的状态和行为，是理解“面向对象编程”这个术语非常好的途径。

现在花点儿时间观察你周围现实世界的对象。对于看到的每个对象，都问自己两个问题：“这个对象可能处于什么状态？”和“这个对象可能执行什么行为？”把你的观察结果写下来。这样，你会注意到现实世界的对象的多样性；你的台灯可能具有两个可能状态（开和关）和两个可能的行为（打开和关闭），但是你的收音机可能具有更多状态（开、关、当前音量和当前频道）和行为（打开、关闭、增加音量、降低音量、搜索、扫描和调谐频道）。接下来，可能你还会注意到一些对象包含其他对象。这些对现实世界的观察都被转化到了面向对象编程的世界中。

软件对象（见图2-1）在概念上类似于现实世界对象：它们也由状态和相关的行为构成。对象把它的状态保存在字段（field）（即一些编程语言中所说的变量）中，并且通过方法（即一些编程语言中所说的函数）表现它的行为。方法操作对象的内部状态，并且作为对象到对象通信的主要机制。隐藏内部状态并且要求所有交互都通过对象的方法来进行，这被称之为数据封装（data encapsulation）——面向对象编程的一项基本原理。

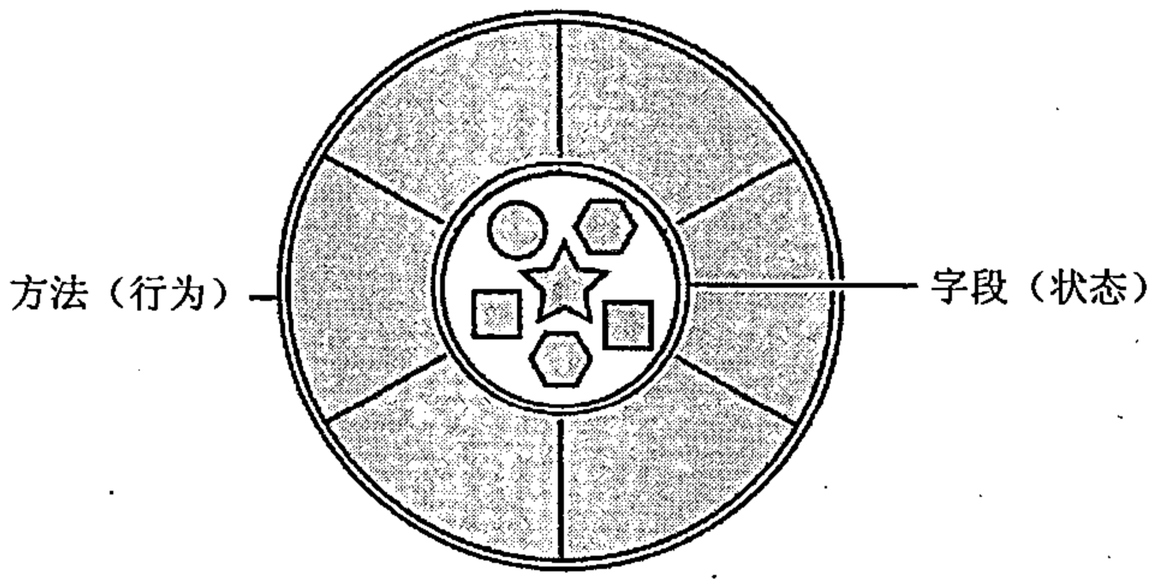


图2-1 软件对象

1. 指脚踏板每分钟的转数。

考虑一下自行车（见图2-2）。通过标识状态（当前速度、当前脚踏板步调和当前挡位）并且提供改变这些状态的方法，就能够控制外部世界如何使用它。例如，如果自行车只有6个挡位，那么改变挡位的方法就不能接受小于1或者大于6的其他值。

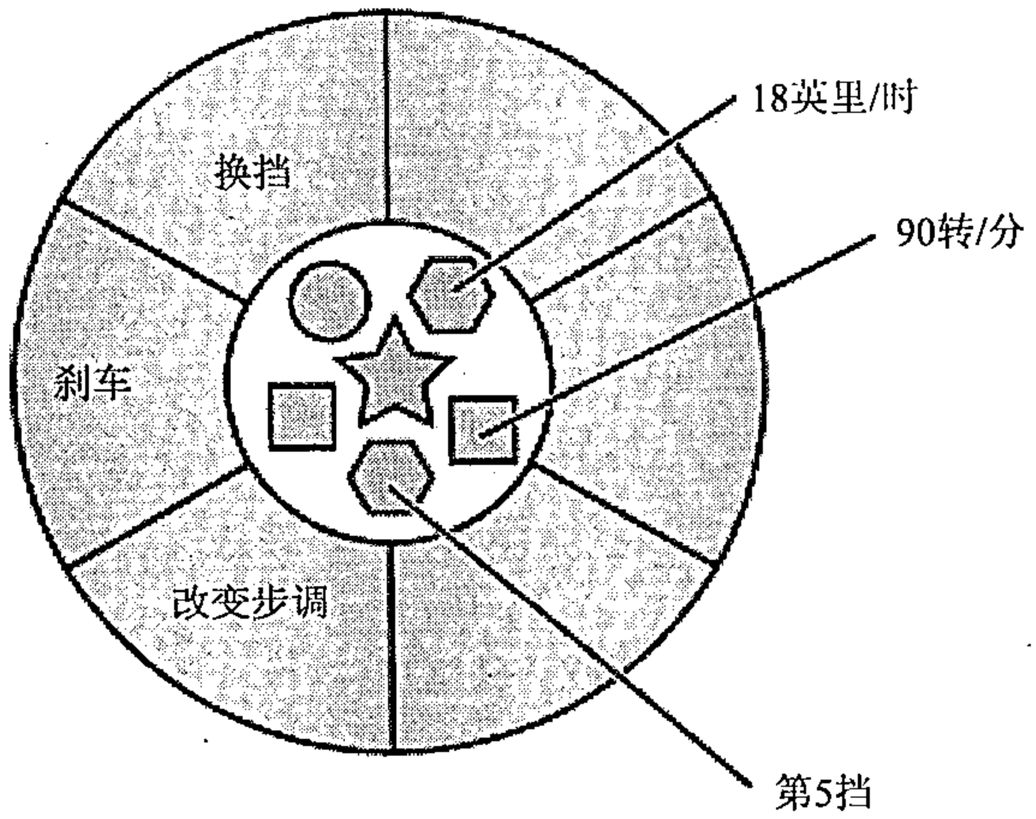


图2-2　作为软件对象的自行车模型

把代码包装到独立的软件对象中提供了很多优势，这些优势包括：

- **模块化**——一个对象的源代码可以独立于其他对象的源代码进行编写和维护。一经创建，对象可以很容易地在系统中传递。
- **隐藏信息**——通过只和对象的方法进行交互，对象内部的实现细节对外部世界是隐藏的。
- **代码重用**——如果对象已经存在（可能由其他软件开发者编写），你可以在自己的程序中使用这个对象。这样可以由专家实现 / 测试 / 调试复杂的、任务特定的对象，然后你可以在自己的代码中放心地运行这些对象。
- **可插拔性和容易调试**——如果特定对象出现问题，你可以简单地把它从应用程序中删除，然后插入其他对象来替换它。这类似于解决现实世界的机械问题。如果螺栓断了，你就替换它，而不是替换整台机器。

2.2　类是什么

在现实世界中，你会经常发现很多独立对象都属于相同类型。可能有数千辆自行车，它们的制造商和型号都相同。每辆自行车都出于相同的蓝图，因此包含相同的组件。在面向对象的术语中，我们说你的自行车是被称为自行车的对象类（class of object）的一个实例（instance）。类（class）是创建独立对象的蓝图。

下面的Bicycle[1]类是自行车的一个实现：

```
class Bicycle {

  int cadence = 0;
  int speed = 0;
  int gear = 1;

  void changeCadence(int newValue) {
```

1. tutorial/java/concepts/examples/Bicycle.java

```
  cadence = newValue;
}

void changeGear(int newValue) {
  gear = newValue;
}

void speedUp(int increment) {
  speed = speed + increment;
}

void applyBrakes(int decrement) {
  speed = speed - decrement;
}

void printStates() {
  System.out.println("cadence:"+cadence+"
                      speed:"+speed+" gear:"+gear);
}
}
```

尽管你不熟悉Java编程语言的语法，但是这个类的设计基于前面对自行车对象的讨论。字段cadence、speed和gear代表对象的状态，而方法（changeCadence、changeGear、speedUp等）定义对象和外部世界的交互。

也许你已经注意到，Bicycle类没有包含main方法。这是因为它不是完整的应用程序；它只是可能用于其他应用程序中的自行车的蓝图。创建和使用新的Bicycle对象的责任属于你的应用程序中的其他一些类。

下面是BicycleDemo[1]类，它创建两个独立的Bicycle对象并且调用它们的方法：

```
class BicycleDemo {
  public static void main(String[] args) {

    // Create two different Bicycle objects
    Bicycle bike1 = new Bicycle();
    Bicycle bike2 = new Bicycle();

    // Invoke methods on those objects
    bike1.changeCadence(50);
    bike1.speedUp(10);
    bike1.changeGear(2);
    bike1.printStates();

    bike2.changeCadence(50);
    bike2.speedUp(10);
    bike2.changeGear(2);
    bike2.changeCadence(40);
    bike2.speedUp(10);
    bike2.changeGear(3);
    bike2.printStates();
  }
}
```

这段测试代码输出两辆自行车的脚踏板步调、速度和挡位：

```
cadence:50 speed:10 gear:2
cadence:40 speed:20 gear:3
```

1. tutorial/java/concepts/examples/BicycleDemo.java

2.3　继承是什么

不同类型的对象相互之间经常具有某些共性。例如，山地自行车、公路自行车和双座自行车都具有自行车的性质（当前速度、当前脚踏板步调和当前挡位）。但是每种自行车也都定义了额外特性，使之不同于其他种类：双座自行车有两个座位和两个车把；公路自行车使用降低的车把；一些山地自行车有额外的链条环，使之具有更低的挡位系数。

面向对象的编程允许类从其他类继承（inherit）共同的状态和行为。在这个例子中，Bicycle（自行车）成为MountainBike（山地自行车）、RoadBike（公路自行车）和TandemBike（双座自行车）的超类（superclass）。在Java编程语言中，允许每个类具有一个直接超类，允许每个
37 超类具有潜在的数量不限的子类（subclass）（见图2-3）。

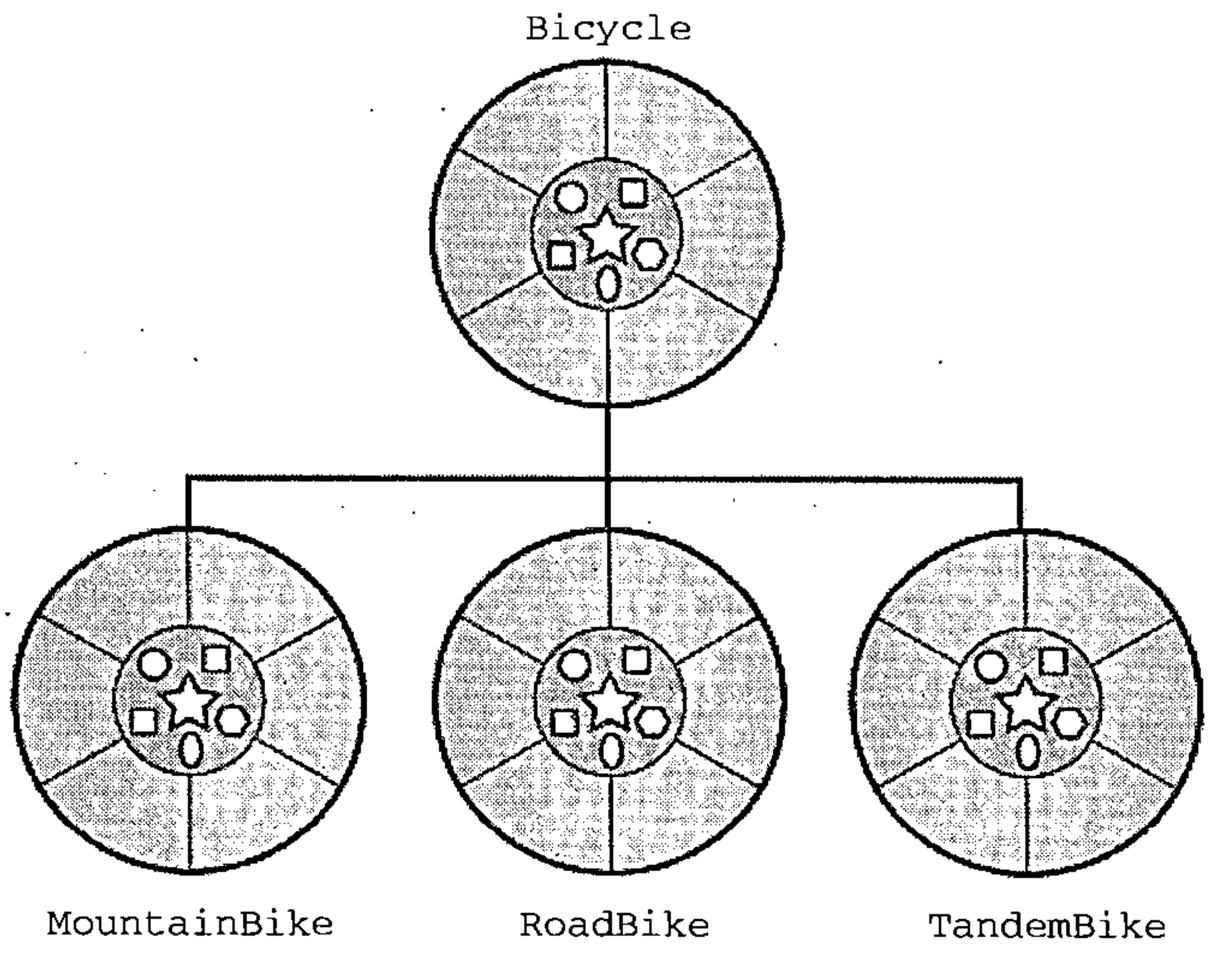


图2-3　自行车类的继承

创建子类的语法很简单。在类定义的开头使用关键字extends，后面是被继承的类的名称：

```
class MountainBike extends Bicycle {

  // new fields and methods defining a mountain bike
  // would go here

}
```

这使MountainBike具有和Bicycle相同的所有字段和方法，也允许它的代码具有独特的专有特性。这使子类的代码易于阅读。但是，你要谨慎地正确编写每个子类定义的状态和行为的文档，因为这些代码不会出现在每个子类的源文件中。

2.4　接口是什么

你已经学过，对象通过暴露的方法定义它们和外部世界的交互。方法形成对象和外部世界的接口（interface）；例如，电视机前面的按钮就是你和电视机塑料外壳内部的电气线路之间的
38 接口。按下“电源”按钮可以控制电视机的开和关。

接口常常是一组具有空方法体的相关方法。如果指定为接口，那么自行车的行为可能像下面这样：

```
interface Bicycle {
  void changeCadence(int newValue);

  void changeGear(int newValue);

  void speedUp(int increment);

  void applyBrakes(int decrement);
}
```

为了实现这个接口，应该改变类的名称（比如改为ACMEBicycle），并且需要在类定义中使用关键字implements：

```
class ACMEBicycle implements Bicycle {

  // remainder of this class implemented as before

}
```

通过实现接口，类可以更加正式地说明它承诺提供的行为。接口构成了类和外部世界之间的契约，并且这个契约在编译时由编译器强制实施。如果你的类声明实现一个接口，那么这个接口定义的所有方法都必须出现在其源代码中，否则类就不能成功编译。

2.5 包是什么

包（package）是组织一组相关类和接口的名称空间。从概念上讲，你可以认为包类似于计算机上的不同文件夹。你可能把HTML页面存放在一个文件夹中，图片放在另一个文件夹中，脚本和应用程序又在另一个文件夹中。因为Java编程语言编写的软件可能由数百个或者数千个独立的类构成，所以把相关类和接口存放在包中，以组织这些内容，这是很有意义的。

Java平台提供了数量庞大的类库（包的集合），可以用于应用程序的开发。在第1章中讲过，这种库被称为“应用程序编程接口（Application Programming Interface）”，或者简写为“API”。它的包代表和通用编程相关的最常见的任务。例如，String对象包含字符串的状态和行为；File对象允许程序员方便地创建、删除、检查、比较或者修改文件系统中的文件；Socket对象允许创建和使用网络套接字；各种GUI对象控制按钮和复选框，以及和图形用户界面相关的所有事项。可以选择的类差不多有数千个。这使程序员可以把精力集中于特定的应用程序设计，而不必在基础设施上浪费时间。

Java平台API规范（Java Platform API Specification）[1]包含Java SE 6的所有包、接口、类、字段和方法的详细清单。在浏览器中加载这个页面，并且把它设成书签。对于程序员，它将成为最重要的一个参考文档。

问题和练习：面向对象的编程概念

问题

1. 现实世界对象包含___和___。
2. 软件对象的状态存储在___之中。
3. 软件对象的行为通过___暴露。
4. 对外部世界隐藏内部数据，只通过公共的方法访问内部数据，这被称为数据___。

---

1. docs/api/index.html

5. 软件对象的蓝图称为___。
6. 共同行为可以定义在___中，并且使用___关键字继承到___中。
7. 不带有实现的方法的集合称为___。
8. 按照功能组织类和接口的名称空间称为___。
9. 术语API代表___。

练习

1. 为你在本章开头观察的每个现实世界对象创建新的类。如果忘记了所需的语法，请参考Bicycle类。
2. 为你在上个练习中创建的每个新类创建定义其行为的接口，然后要求你的类实现它。省略一个或者两个方法然后进行编译。出现了什么错误？

40

答案

可以在以下位置找到“问题”和“练习”的答案：

41

```
tutorial/java/concepts/QandE/answers.html
```

第3章

语言基础

3.1 变量

在前一章中讲过，对象把它的状态存储在字段（field）中：

```
int cadence = 0;
int speed = 0;
int gear = 1;
```

2.1节介绍了字段，但是你可能还有几个问题，比如：命名字段的规则和惯例是什么？除了int之外，还有什么其他数据类型？声明的时候必须对字段进行初始化吗？如果没有明确地初始化，那么会给字段分配默认值吗？我们将在本章中回答这些问题，但是，首先你必须了解几个技术上的区别。在Java编程语言中，“字段”和“变量”这两个术语都用到了；这是编程新手经常混淆的地方，因为这两个术语似乎指的是同一种东西。

Java编程语言定义下面这些变量类型。

- **实例变量（非静态字段）**。从技术上讲，对象把它们的各个状态存储在“非静态字段”中，这就是说，不使用关键字static声明字段。非静态字段也被称为实例变量（instance variable），因为它们的值对类的每个实例（instance）是唯一的（换句话说，就是对每个对象是唯一的）；一辆自行车的currentSpeed独立于另一辆自行车的currentSpeed。

- **类变量（静态字段）**。类变量（class variable）是使用static修饰符声明的任何字段；这就通知编译器，不管这个类被实例化了多少次，都只存在这个变量的一个副本。定义特定类型自行车的挡位数量的字段就可以被标记为static，因为在概念上所有实例都使用相同数量的挡位。代码static int numGears = 6;就创建了这样的静态字段。另外，可以添加关键字final，表示永远都不能改动挡位数量。
- **局部变量**。和对象把其状态存储在字段中类似，方法经常把其临时状态存储在局部变量（local variable）中。声明局部变量的语法和声明字段的语法类似（例如int count = 0;）。没有专门关键字用于说明变量是局部变量；确定局部变量的方式完全取决于声明变量的位置——在方法的前后括号之间。这样，局部变量只对声明它的方法是可见的；不能从类的其他部分访问局部变量。
- **参数**。在Bicycle类和应用程序“Hello World!”的main方法中，你已经见到了参数的例子。main方法的签名是public static void main (String[] args)。这里，变量args是这个方法的参数。要牢记，这些参数总是被归类为“变量”，而不是“字段”，这很重要。这也适用于其他接受参数的结构（比如构造器和异常处理代码），你将在本书后面学到它们。

当本书的其余部分讨论到字段和变量时，遵循如下总的指导方针。如果我们谈到“一般字段”（不包括局部变量和参数），我们可能简单地说“字段”。如果讨论到“前面所有内容”，我们可能简单地说“变量”。如果上下文要求作出区分，我们会适当地使用特定术语（静态字段、

局部变量，等等）。有时候你可能会看到“成员”这个术语。类型的字段、方法和嵌套类型被统称为这个类型的成员（member）。

3.1.1 命名

每种编程语言都有其自己的一套规则和惯例，用于规定允许使用的名称类型，Java编程语言也不例外。下面总结了命名变量的规则和惯例：

- 变量名称是区分大小写的。变量名称可以是任何合法的标识符——长度不限的Unicode字母和数字的序列，以字母、美元符号“$”或者下划线字符“_”开头。但是，按照惯例，总是以字母开始变量名称，而不使用“$”或者“_”。另外，按照惯例，不要使用美元符号。你可能在一些自动生成的名称中看到美元符号，但是在你自己的变量名称中应该避免使用它。对下划线字符也有类似的惯例；虽然从技术上说，以“_”开头的变量名称是合法的，但是不推荐这样做。空白是不允许使用的。
- 后续字符可以是字母、数字、美元符号或者下划线字符。当选择变量的名称时，使用完整的单词，而不是难以理解的简写。这样做可以使你的代码容易阅读和理解。很多情况下，也使你的代码能够自我解释；例如，和简写的s、c和g相比，名为cadence、speed和gear的字段显得直观得多。还要牢记，你选择的名称必须不是关键字或者保留字。请参见附录A。
- 如果你选择的名称只有一个单词，那么就全使用小写字母。如果它包含多个单词，那么就把每个后续单词的第一个字母大写。名称gearRatio和currentGear是很好的例子。如果你的变量用于存储常量值，比如static final int NUM_GEARS = 6，那么惯例就稍有变化，每个字母都大写，并且使用下划线分隔后续的单词。按照惯例，其他情况下不使用下划线字符。

3.1.2 基本数据类型

Java编程语言是类型严格（即“强类型”）的语言，这就是说所有变量在使用之前都必须声明。这涉及到声明变量的类型和名称，就像你看到过的：

```
int gear = 1;
```

这告诉程序，有一个名为“gear”的字段，它保存数值数据，并且其初始值为1。变量的数据类型决定它可能包含的值，以及可能对它执行的操作。除了int之外，Java编程语言还支持另外7种基本数据类型（primitive data type）。原始类型是语言预定义的，并且以保留关键字命名。原始值不和其他原始值共享状态。Java编程语言支持的8种基本数据类型如下：

- **byte**。byte数据类型是8位带符号二进制补码整数。其最小值为-128；最大值为127（含）。在内存紧张的情况下，将byte数据类型用于大型数组对节省内存很有帮助。我们将在3.1.3节讲解数组。也可以使用byte替换int，这样它们的限制能够帮助说明你的代码；变量的范围受到限制的事实可以作为文档的一种形式。
- **short**。short数据类型是16位带符号二进制补码整数。其最小值为-32 768；最大值为32 767（含）。它具有和byte一样的指导方针：你可以在内存紧张的情况下，将short用于大型数组以便节省内存。
- **int**。int数据类型是32位带符号二进制补码整数。其最小值为-2 147 483 648；最大值为2 147 483 647（含）。对于整数值，这个数据类型一般是默认选择，除非出于某种原因（比如节省内存）而选择其他类型。对你的程序要使用的数字来说，这种数据类型很可能已经足够大了，但是如果你的值需要更大范围，就使用long类型。

- **long**。long数据类型是64位带符号二进制补码整数。其最小值为-9 223 372 036 854 775 808；最大值为9 223 372 036 854 775 807（含）。当你的值需要的范围超过int提供的范围时，可以使用这个数据类型。
- **float**。float数据类型是单精度32位IEEE 754浮点数。对它的详细介绍超出了这里的讨论范围，但是在Java语言规范（Java Language Specification）[1]的4.2.3节中作出了规定。和对byte以及short类型的建议一样，如果在使用大型的浮点数数组时需要节省内存，就要使用float（而不是double）。永远都不要把这种数据类型用于精确值，比如货币。对于精确值，你需要使用java.math.BigDecimal类[2]。第8章讲解BigDecimal和Java平台提供的其他有用的类。
- **double**。double数据类型是双精度64位IEEE 754浮点数。对它的详细介绍超出了这里的讨论范围，但是在Java语言规范（Java Language Specification）的4.2.3节中作出了规定。对于小数，这个数据类型是一般的默认选择。和前面讲的一样，永远不要把这种数据类型用于精确值，比如货币。
- **boolean**。boolean数据类型只有两个可能值：true和false。这种数据类型被用作跟踪真 / 假条件的简单标志。这种数据类型表示1位信息，但是它的“长度”没有确切地定义。
- **char**。char数据类型是单一16位Unicode字符。其最小值为'\u0000'（即0）；最大值为'\uffff'（即65 535（含））。

46

除了前面列出的8种基本数据类型之外，Java编程语言还通过java.lang.String类[3]，提供对字符串的专门支持。把字符串包围在双引号中，就会自动地创建新的String对象；例如String s = "this is a string";。String对象是不可变的（immutable），这就是说创建之后，就不能改变它们的值。从技术上说，String类不是基本数据类型，但是考虑到语言给它提供的专门支持，你也可以把它看作基本数据类型。在第8章中将更多地介绍String类。

1. 默认值

声明字段时，不必非要为其赋值。被声明但是没有初始化的字段会被编译器设置为合理的默认值。总的来说，根据数据类型，这个默认值是0或者null。但是，依靠这样的默认值一般被认为是不良的编程方式。

表3-1总结了前述数据类型的默认值。

表3-1 数据类型及其默认值

数据类型	默认值（用于字段）
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String（或者任何对象）	null
boolean	false

1. docs/books/jls/third_edition/html/typesValues.html
2. docs/api/java/math/BigDecimal.html
3. docs/api/java/lang/String.html

局部变量稍有不同；编译器永远都不会给未初始化的局部变量分配默认值。如果在声明局部变量时不能初始化它，就要确保在使用之前为其赋值。访问未经初始化的局部变量将导致编
47 译时错误。

2. 字面量

你或许会注意到，当初始化原始类型的变量时，没有使用关键字new。原始类型是语言内置的特殊数据类型；它们不是从类创建的对象。字面量（literal）是固定值的源代码表现形式；字面量直接出现在代码中，无需计算。如下所示，可以为基本数据类型的变量分配字面量。

```
boolean result = true;
char capitalC = 'C';
byte b = 100;
short s = 10000;
int i = 100000;
```

整数类型（byte、short、int和long）使用十进制、八进制或者十六进制计数系统。十进制是你每天都使用的计数系统；它的基数为10，使用的数字从0到9。八进制计数系统的基数为8，由数字0到7构成。十六进制计数系统的基数为16，使用数字0到9和字母A到F。对于通用程序设计来说，十进制系统可能是你将使用的唯一一种计数系统。但是，如果你需要八进制或者十六进制，下面的例子演示了正确的语法。前缀0表示八进制，而前缀0x表示十六进制。

```
int decVal = 26;    // The number 26, in decimal
int octVal = 032;   // The number 26, in octal
int hexVal = 0x1a;  // The number 26, in hexadecimal
```

浮点数类型（float和double）也可以用E或者e（用于科学记数法）、F或者f（32位浮点字面量）和D或者d（64位双精度字面量；这是默认的，并且按照惯例可以省略）表示。

```
double d1 = 123.4;
double d2 = 1.234e2; // same value as d1,
                     // but in scientific notation
float f1  = 123.4f;
```

char和String类型的字面量可能包含任意Unicode（UTF-16）字符。如果你的编辑器和文件系统允许，就可以在代码中直接使用这样的字符。如果不允许，可以使用“Unicode转义符”，比如'\u0108'（带有升调符号的大写C），或者"S\u00ED se\u00F1or"（西班牙语的Sí Señor）。对于char类型的字面量，总要使用单引号（'）；对于String类型的字面量，总要使用双引号（"）。Unicode转义序列可以用在程序中的其他位置（比如字段名称中），并非只能在char和String
48 字面量中使用。

Java编程语言还支持几个特殊的char和String字面量的转义序列：\b（退格）、\t（制表符）、\n（换行）、\f（换页）、\r（回车）、\"（双引号）、\'（单引号）和\\（反斜线）。

还有一个特殊的null字面量，它可以用作任何引用类型的值。null可以赋值给除属于原始类型的变量之外的任何变量。除了测试其存在之外，对null值没有什么可做的操作。因此，在程序中经常使用null作为标识，表示某个对象不可用。

最后，还有一种特殊类型的字面量，称为类字面量（class literal），它的构成方式是类名称加后缀“.class”，比如String.class。这引用表示类型本身的对象（类型为Class）。

3.1.3 数组

数组（array）是存储数量固定的单一类型值的容器对象。数组的长度在创建数组时就固定下来了。在创建之后，数组的长度是不可变的。你在应用程序“Hello World!”的main方法中看到过数组的例子。这一节更加详细地讨论数组。

数组中的每个项称为元素（element），通过每个元素的数字索引（也称下标）访问元素。如图3-1所示，索引的数字从0开始。因此，在索引8的位置访问第9个元素。

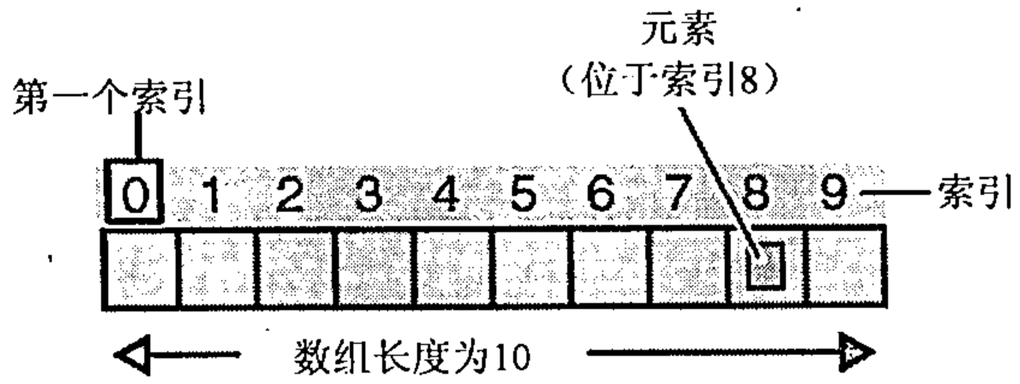


图3-1 具有10个元素的数组

下面的程序ArrayDemo[1]创建一个由整数构成的数组，它把一些值存放到数组中，然后把每个值发送到标准输出。

49

```
class ArrayDemo {
  public static void main(String[] args) {
    int[] anArray;              // declares an array of integers

    anArray = new int[10];      // allocates memory for 10 integers

    anArray[0] = 100; // initialize first element
    anArray[1] = 200; // initialize second element
    anArray[2] = 300; // etc.
    anArray[3] = 400;
    anArray[4] = 500;
    anArray[5] = 600;
    anArray[6] = 700;
    anArray[7] = 800;
    anArray[8] = 900;
    anArray[9] = 1000;

    System.out.println("Element at index 0: " + anArray[0]);
    System.out.println("Element at index 1: " + anArray[1]);
    System.out.println("Element at index 2: " + anArray[2]);
    System.out.println("Element at index 3: " + anArray[3]);
    System.out.println("Element at index 4: " + anArray[4]);
    System.out.println("Element at index 5: " + anArray[5]);
    System.out.println("Element at index 6: " + anArray[6]);
    System.out.println("Element at index 7: " + anArray[7]);
    System.out.println("Element at index 8: " + anArray[8]);
    System.out.println("Element at index 9: " + anArray[9]);
  }
}
```

这个程序的输出是：

```
Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```

1. tutorial/java/nutsandbolts/examples/ArrayDemo.java

在现实的编程工作中，可以使用一种循环结构（looping construct）迭代处理数组的每个元素，而不是像前面这样单独地编写每一行。但是，这个示例清楚地演示了数组的语法。你将在
50 3.4节中学习各种循环结构（for、while和do-while）。

1. 声明引用数组的变量

前面的程序使用下面这行代码声明anArray：

```
int[] anArray;          // declares an array of integers
```

和其他类型变量的声明一样，数组的声明由两个部分构成：数组类型和数组名称。数组类型写作*type*[]，其中*type*是被包含元素的数据类型；方括号是指示这个变量包含数组的专用符号。数组的长度不是其类型的一部分（这就是括号中为空白的原因）。数组的名称可以是你想使用的任何名称，只要名称遵循前面3.1.1节中讨论的规则和惯例即可。和其他类型的变量一样，声明不会实际创建一个数组——它只是通知编译器，这个变量将包含指定类型的数组。

类似的，你可以声明其他类型的数组：

```
byte[] anArrayOfBytes;
short[] anArrayOfShorts;
long[] anArrayOfLongs;
float[] anArrayOfFloats;
double[] anArrayOfDoubles;
boolean[] anArrayOfBooleans;
char[] anArrayOfChars;
String[] anArrayOfStrings;
```

你也可以把方括号放在数组名称的后面：

```
float anArrayOfFloats[];  // this form is discouraged
```

但是，按照惯例不推荐这样做；括号标识数组类型，应该和类型签名一起出现。

2. 创建、初始化和访问数组

创建数组的一个途径是使用new操作符。ArrayDemo程序中的下列语句为包含10个整数元素的数组分配足够的内存，并且把该数组赋值给anArray变量。

```
anArray = new int[10];  // create an array of integers
```

如果遗漏了这行语句，编译器就会输出如下错误消息，并且编译会失败：

```
ArrayDemo.java:4: Variable anArray may
                      not have been initialized.
```
51

下面这几行代码为数组的每个元素赋值：

```
anArray[0] = 100; // initialize first element
anArray[1] = 200; // initialize second element
anArray[2] = 300; // etc.
```

可以通过每个数组元素的数字索引访问它们：

```
System.out.println("Element 1 at index 0: " + anArray[0]);
System.out.println("Element 2 at index 1: " + anArray[1]);
System.out.println("Element 3 at index 2: " + anArray[2]);
```

另外，也可以使用简捷语法创建和初始化数组：

```
int[] anArray = {100, 200, 300, 400, 500, 600, 700, 800, 900, 1000};
```

其中，数组的长度由{和}之间值的数量确定。

也可以声明由数组构成的数组[也称为多维（multidimensional）数组]，声明方法是使用两组或者两组以上的方括号，比如String[][] names。因此，必须通过对应的数字索引值访问每个元素。

在Java编程语言中，多维数组仅仅是其组成元素本身也是数组的数组。这和C或者Fortran

语言中的数组不同。其结果是允许行的长度可变，如下面的程序MultiDimArrayDemo[1]所示：

```
class MultiDimArrayDemo {
  public static void main(String[] args) {
    String[][] names = {{"Mr. ", "Mrs. ", "Ms. "},
                        {"Smith", "Jones"}};
    System.out.println(names[0][0] + names[1][0]); // Mr. Smith
    System.out.println(names[0][2] + names[1][1]); // Ms. Jones
  }
}
```

这个程序的输出如下：

```
Mr. Smith
Ms. Jones
```

52

最后，你可以使用内置的length属性确定数组的长度。代码

```
System.out.println (anArray.length) ;
```

将把数组长度发送到标准输出。

3. 复制数组

System类有一个arraycopy方法，可以使用它把数据从一个数组复制到另一个数组：

```
public static void arraycopy(Object src,
                             int srcPos,
                             Object dest,
                             int destPos,
                             int length)
```

两个Object参数分别指定被复制的数组和复制到的数组。三个int参数分别指定源数组中的开始位置、目的数组中的开始位置和要复制的数组元素的数量。

下面的程序ArrayCopyDemo声明一个包含char元素的数组，这些字符拼写出单词“decaffeinated”。它使用arraycopy把数组元素的子序列复制到第二个数组：

```
class ArrayCopyDemo {
  public static void main(String[] args) {
    char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
                        'i', 'n', 'a', 't', 'e', 'd' };
    char[] copyTo = new char[7];

    System.arraycopy(copyFrom, 2, copyTo, 0, 7);
    System.out.println(new String(copyTo));
  }
}
```

这个程序的输出是：

```
caffein
```

3.1.4 变量小结

“字段”和“变量”都是Java语言中的术语。实例变量（非静态字段）对于类的每个实例是唯一的。类变量（静态字段）使用修饰符static声明；不管类被实例化了多少次，类变量严格地都只有一个副本。局部变量存储方法内部的临时状态。参数是为方法提供额外信息的变量；局部变量和参数总被分类为“变量”（而不是“字段”）。命名字段或者变量时，应该（或者必须）遵守一些规则和惯例。

53

1. tutorial/java/nutsandbolts/examples/MultiDimArrayDemo.java

8种基本数据类型是：byte、short、int、long、float、double、boolean和char。java.lang.String类代表字符串。编译器会为上述类型的字段分配合理的默认值；对于局部变量，不会为其分配默认值。字面量是固定值的源代码表现形式。数组是存储数量固定的单一类型值的容器对象。在创建数组时设置数组的长度。创建之后，数组长度是固定不变的。

问题和练习：变量

问题

1. 术语“实例变量”是___的另一个名称。
2. 术语“类变量”是___的另一个名称。
3. 局部变量存储临时状态；在___内声明它。
4. 在方法签名的前后括号中声明的变量被称为___。
5. Java编程语言支持的8种基本数据类型是什么？
6. 字符串由___类表示。
7. ___是存储数量固定的单一类型值的容器对象。

练习

1. 创建一个定义一些字段的小程序。尝试创建一些非法字段名称，然后查看编译器产生了什么样的错误消息。使用命名规则和惯例作为指导方针。
2. 在练习1创建的程序中，不初始化字段，然后输出它们的值。对局部变量进行同样的操作，看看编译器会产生什么样的错误消息。熟悉常见的编译器错误，这能够帮助你更容易地识别出代码中的错误。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/nutsandbolts/QandE/answers_variables.html

3.2　操作符

既然已经学习了如何声明和初始化变量，或许你希望了解如何对变量进行某种操作。从Java编程语言的操作符开始学习是个好主意。操作符是特定的符号，可以使用它们对一个、两个或者三个操作数（operand）进行特定操作，然后返回结果。

在学习Java编程语言的操作符之前，首先了解哪些操作符具有最高优先级是有帮助的。表3-2中的操作符是按照它们的优先级顺序列出的。操作符出现的位置越靠近表的顶部，其优先级就越高。优先级高的操作符先于优先级相对低的操作符进行计算。位于同一行的操作符具有相同优先级。优先级相同的操作符出现在同一表达式中时，必须给出规则，从而确定先计算哪个操作符。除赋值操作符之外，所有二元操作符都按照从左到右的顺序计算；赋值操作符按照从右到左的顺序计算。

表3-2　操作符优先级

操作符	优先级
后缀	*expr++ expr--*
一元	*++expr --expr +expr -expr* ~ !
乘	* / %

（续）

操作符	优先级
加	+ -
移位	<< >> >>>
关系	< > <= >= instanceof
相等	== !=
按位与	&
按位异或	^
按位或	|
逻辑与	&&
逻辑或	||
三元	? :
赋值	= += -= *= /= %= &= ^= |= <<= >>= >>>=

55

在通用编程中，某些操作符出现的频率高于其他操作符；例如，赋值操作符“=”要比无符号右移位操作符“>>>”常见得多。了解这些之后，下面的讨论首先介绍最常用的操作符，然后介绍不怎么常见的操作符。后面的讨论都带有示例代码，你可以编译并运行它们。分析代码的输出能帮助你强化学到的知识。

3.2.1 赋值、算术和一元操作符

1. 简单的赋值操作符

你将用到的最常见的操作符之一是简单的赋值操作符“=”。在Bicycle类中已经见过了这个操作符；它把其右侧操作数的值赋给左侧操作数：

```
int cadence = 0;
int speed = 0;
int gear = 1;
```

也可以对对象使用这个操作符，进行对象引用（object reference）的赋值，这将在第8章中讨论。

2. 算术操作符

Java编程语言提供执行加法、减法、乘法和除法操作的操作符。在基本数学运算中能经常看到它们。你感觉陌生的唯一符号可能是“%”，它用一个操作数除另一个操作数，然后返回结果的余数。

```
+       加法操作符（也用于拼接字符串）
-       减法操作符
*       乘法操作符
/       除法操作符
%       取模操作符
```

下面的程序ArithmeticDemo[1]测试算术操作符。

56

```
class ArithmeticDemo {

  public static void main (String[] args){

    int result = 1 + 2; // result is now 3
    System.out.println(result);
```

1. tutorial/java/nutsandbolts/examples/ArithmeticDemo.java

```
    result = result - 1; // result is now 2
    System.out.println(result);

    result = result * 2; // result is now 4
    System.out.println(result);

    result = result / 2; // result is now 2
    System.out.println(result);

    result = result + 8; // result is now 10
    result = result % 7; // result is now 3
    System.out.println(result);
  }
}
```

也可以把算术操作符和简单的赋值操作符组合在一起，从而创建复合赋值（compound assignment）。例如，x+=1;和x=x+1;都表示x的值加1。

“+”操作符也可以用于把两个字符串串联（连接）在一起，如下面的程序ConcatDemo[1]所示：

```
class ConcatDemo {
  public static void main(String[] args){
    String firstString = "This is";
    String secondString = " a concatenated string.";
    String thirdString = firstString+secondString;
    System.out.println(thirdString);
  }
}
```

在程序的结尾，变量thirdString包含“This is a concatenated string.”，它被发送到标准输出。

57

3. 一元操作符

一元操作符只需要一个操作数；它们执行各种操作，比如使值递增/递减1、反转表达式或者对布尔值求反。

+	一元正操作符，指示正值（但不带这个操作符的数也是正的）
-	一元负操作符，对表达式求反
++	递增操作符，使值增加1
--	递减操作符，使值减去1
!	逻辑求补操作符，反转布尔值

下面的程序UnaryDemo[2]测试一元操作符：

```
class UnaryDemo {

  public static void main(String[] args){
    int result = +1; // result is now 1
    System.out.println(result);
    result--; // result is now 0
    System.out.println(result);
    result++; // result is now 1
    System.out.println(result);
    result = -result; // result is now -1
    System.out.println(result);
    boolean success = false;
    System.out.println(success); // false
    System.out.println(!success); // true
```

---

1. tutorial/java/nutsandbolts/examples/ConcatDemo.java
2. tutorial/java/nutsandbolts/examples/UnaryDemo.java

```
    }
}
```

递增/递减操作符可以在操作数之前（前缀）或者之后（后缀）使用。代码result++;和++result;都将使result加1。唯一的区别在于，前缀版本（++result）对增加后的值进行计算；而后缀版本（result++）对原始值进行计算。如果你仅仅执行简单的递增/递减操作，那么选择哪个版本就没什么关系。但是如果使用这个操作符作为更大的表达式的一部分，那么选择不同版本就会造成很大区别。

下面的程序PrePostDemo演示前缀/后缀版本的一元递增操作符：

```
class PrePostDemo {
  public static void main(String[] args){
    int i = 3;
    i++;
    System.out.println(i);      // "4"
    ++i;
    System.out.println(i);      // "5"
    System.out.println(++i);    // "6"
    System.out.println(i++);    // "6"
    System.out.println(i);      // "7"
  }
}
```

3.2.2 相等、关系和条件操作符

1. 相等和关系操作符

相等和关系操作符判断一个操作数是否大于、小于、等于或者不等于另一个操作数。你可能熟悉大多数这类操作符。但要牢记，在检测两个原始值是否相等时，必须使用“==”，而不是“=”。

==	等于
!=	不等于
>	大于
>=	大于或等于
<	小于
<=	小于或等于

下面的程序ComparisonDemo[1]测试比较操作符：

```
class ComparisonDemo {

  public static void main(String[] args) {
    int value1 = 1;
    int value2 = 2;
    if(value1 == value2)
                        System.out.println("value1 == value2");
    if(value1 != value2)
                        System.out.println("value1 != value2");
    if(value1 > value2) System.out.println("value1 > value2");
    if(value1 < value2) System.out.println("value1 < value2");
    if(value1 <= value2)
                    System.out.println("value1 <= value2");
  }
}
```

输出：

```
value1 != value2
```

1. tutorial/java/nutsandbolts/examples/ComparisonDemo.java

```
value1 < value2
value1 <= value2
```

2. 条件操作符

“&&”和“||”操作符对两个布尔表达式执行条件与（Conditional-AND）和条件或（Conditional-OR）操作。这两个操作符具有“短路”行为，也就是说只在必要时才计算第二个操作数。

&&	条件与
||	条件或

下面的程序ConditionalDemo1[1]测试这两个操作符：

```
class ConditionalDemo1 {

  public static void main(String[] args){
    int value1 = 1;
    int value2 = 2;
    if((value1 == 1) && (value2 == 2))
      System.out.println("value1 is 1 AND value2 is 2");
    if((value1 == 1) || (value2 == 1))
      System.out.println("value1 is 1 OR value2 is 1");

  }
}
```

另一个条件操作符是?:，可以把它看作if-then-else语句（在3.4节中讨论）的简写方式。这个操作符被称为三元操作符（ternary operator），因为它使用三个操作数。在下面的示例中，应该把这个操作符读作：“如果someCondition为true，就把value1的值赋给result。否则，把value2的值赋给result”。

60　下面的程序ConditionalDemo2[2]测试?:操作符：

```
class ConditionalDemo2 {

  public static void main(String[] args){
    int value1 = 1;
    int value2 = 2;
    int result;
    boolean someCondition = true;
    result = someCondition ? value1 : value2;
    System.out.println(result);
  }
}
```

因为someCondition为true，所以程序把“1”输出到屏幕。如果能使代码的可读性更强，就使用?:操作符，而不使用if-then-else语句；例如，当表达式紧凑并且没有副作用（比如赋值）时。

3. 类型比较操作符instanceof

instanceof操作符把对象和指定的类型进行比较。你可以使用它测试某个对象是否是一个类的实例、子类的实例或者实现特定接口的类的实例。

下面的程序InstanceofDemo[3]定义了父类（名为Parent）、简单接口（名为MyInterface）和子类（名为Child），子类Child继承父类并且实现接口。

---

1. tutorial/java/nutsandbolts/examples/ConditionalDemo1.java
2. tutorial/java/nutsandbolts/examples/ConditionalDemo2.java
3. tutorial/java/nutsandbolts/examples/InstanceofDemo.java

```
class InstanceofDemo {
  public static void main(String args[]) {
    Parent obj1 = new Parent();
    Parent obj2 = new Child();

    System.out.println("obj1 instanceof Parent: " +
                       (obj1 instanceof Parent));
    System.out.println("obj1 instanceof Child: " +
                       (obj1 instanceof Child));
    System.out.println("obj1 instanceof MyInterface: " +
                       (obj1 instanceof MyInterface));
    System.out.println("obj2 instanceof Parent: " +
                       (obj2 instanceof Parent));
    System.out.println("obj2 instanceof Child: " +
                       (obj2 instanceof Child));
    System.out.println("obj2 instanceof MyInterface: " +
                       (obj2 instanceof MyInterface));
  }
}

class Parent{}
class Child extends Parent implements MyInterface{}
interface MyInterface{}
```

61

这个程序的输出为：

```
obj1 instanceof Parent: true
obj1 instanceof Child: false
obj1 instanceof MyInterface: false
obj2 instanceof Parent: true
obj2 instanceof Child: true
obj2 instanceof MyInterface: true
```

使用instanceof操作符时，要牢记null不是任何对象的实例。

3.2.3 位操作符和移位操作符

Java编程语言还针对整数类型提供了位操作符和移位操作符。这一节中讨论的操作符不怎么常见。因此这里只简要地介绍它们，目的是让你知道有这些操作符。

一元位求补操作符“~”反转位模式；它可以应用于任何整数类型，把每个“0”转换成“1”，把每个“1”转换成“0”。例如，一个byte包含8位，使用这个操作符处理位模式为“00000000”的值时，将把它的位模式转换为“11111111”。

带符号左移位操作符“<<”把位模式向左移位，带符号右移位操作符“>>”把位模式向右移位。位模式由左侧操作数给定，移位的位置数量由右侧操作数给定。无符号右移位操作符“>>>”把0移位到最左侧的位置，而“>>”操作之后最左侧位置的状态取决于符号扩展。

位&操作符执行位与操作。

位^操作符执行位异或操作。

位|操作符执行位或操作。

对于前面的&，当两个操作数的某位都为1时，结果中的这一位就为1。

对于前面的|，当一个操作数的某位为1时，结果中的这一位就为1[1]。

下面的程序BitDemo[2]使用位与操作符把数字“2”输出到标准输出。

62

```
class BitDemo {
```

1. 原书为“对于前面的&和|，当两个操作数的某位都为1时，结果中的这一位就为1”，表达错误。——译者注
2. tutorial/java/nutsandbolts/examples/BitDemo.java

```
    public static void main(String[] args) {
      int bitmask = 0x000F;
      int val = 0x2222;
      System.out.println(val & bitmask); // prints "2"
    }
}
```

3.2.4 操作符小结

下面是Java编程语言支持的操作符的参考小结。

简单赋值操作符

=　简单赋值操作符

算术操作符

+　加法操作符（也用于拼接字符串）

-　减法操作符

*　乘法操作符

/　除法操作符

%　取模操作符

一元操作符

+　一元正操作符；指示正值（但不带这个操作符的数也是正的）

-　一元负操作符；对表达式求反

++　递增操作符；使值增加1

--　递减操作符；使值减去1

!　逻辑求补操作符；反转布尔值

相等和关系操作符

==　等于

!=　不等于

>　大于

>=　大于或等于

<　小于

<=　小于或等于

条件操作符

&&　条件与

||　条件或

?:　三元（if-then-else语句的简写方式）

类型比较操作符instanceof

instanceof把对象和指定的类型进行比较

位和位移位操作符

~　一元位求补

<<　带符号左移位

>>　带符号右移位

>>>　无符号右移位

&　位与

^　位异或

| 位或

问题和练习：操作符

问题

1. 查看下面的代码段。

```
arrayOfInts[j] > arrayOfInts[j+1]
```

代码中包含哪些操作符？

2. 查看下面的代码段。

```
int i = 10;
int n = i++%5;
```

a. 执行代码后，i和n的值是什么？

b. 不使用后缀递增操作符（i++），而是使用前缀版本（++i），这时i和n的最终值是什么？

3. 为了反转布尔值，应该使用哪个操作符？
4. 比较两个值时应该使用哪个操作符？“=”还是“==”？
5. 解释下面的代码示例：

```
result = someCondition ? value1 : value2;
```

练习

1. 把下面的程序改为使用复合赋值：

```
class ArithmeticDemo {

  public static void main (String[] args){

    int result = 1 + 2; // result is now 3
    System.out.println(result);

    result = result - 1; // result is now 2
    System.out.println(result);

    result = result * 2; // result is now 4
    System.out.println(result);

    result = result / 2; // result is now 2
    System.out.println(result);

    result = result + 8; // result is now 10
    result = result % 7; // result is now 3
    System.out.println(result);

  }
}
```

2. 在下面的程序中，解释为什么“6”这个值在输出中出现了两次：

```
class PrePostDemo {
  public static void main(String[] args){
    int i = 3;
    i++;
    System.out.println(i);      // "4"
    ++i;
    System.out.println(i);      // "5"
    System.out.println(++i);    // "6"
    System.out.println(i++);    // "6"
    System.out.println(i);      // "7"
  }
}
```

答案

可以在以下位置找到“问题”和“练习”的答案：

65　tutorial/java/nutsandbolts/QandE/questions_expressions.html

3.3　表达式、语句和块

现在你已经了解了变量和操作符，下面开始学习表达式（expression）、语句（statement）和块（block）。操作符可以用来构建表达式，用表达式来计算值；表达式是语句的核心成分；而语句又可以分为多个块。

3.3.1　表达式

表达式（expression）是由变量、操作符和方法调用构成的结构，表达式是按照语言的语法构成的，它计算出单一值。你已经见到过表达式的例子，在下面用粗体显示：

```
int cadence = 0 ;
anArray[0] = 100 ;
System.out.println("Element 1 at index 0: " + anArray[0]);

int result = 1 + 2 ; // result is now 3
if(value1 == value2 ) System.out.println("value1 == value2" );
```

表达式的返回值的数据类型取决于表达式中使用的元素。表达式cadence = 0返回int类型，因为赋值操作符的返回值的数据类型与其左侧操作数的数据类型相同，在这个例子中，cadence是int类型。从其他表达式中可以看到，表达式也可以返回其他类型的值，如boolean或者String。

Java编程语言允许用各种较小的表达式构成复合表达式，条件是表达式各个部分的数据类型要相匹配。下面是一个复合表达式的例子：

```
1 * 2 * 3
```

在这个特定的例子中，表达式的运算次序并不重要，因为相乘的结果与次序无关；无论相乘的次序是什么，结果总是一样的。但是，并非所有表达式都是这样。例如，对于下面的表达式，以不同的次序执行加法和除法操作，会得出不同的结果：

66

```
x + y / 100 // ambiguous
```

可以通过使用成对的圆括号“(”和“)”精确地指定表达式的运算次序。例如，要使前面的表达式意思明确，可以这样写：

```
(x + y) / 100 // unambiguous, recommended
```

如果不明确地指定操作符的执行次序，那么执行次序由表达式中使用的操作符的优先级决定。具有较高优先级的操作符先被运算。例如，除法操作符的优先级比加法操作符高，所以下面两个语句是等效的：

```
x + y / 100
x + (y / 100)  // unambiguous, recommended
```

在写复合表达式时，应该用圆括号明确指定哪些操作符应该先被运算。这种编程习惯可以使代码更易于阅读和维护。

3.3.2　语句

语句大致相当于自然语言中的句子。语句（statement）组成了一个完整的执行单元。下面

的表达式类型以一个分号（;）结尾时可以组成一个语句：

- 赋值表达式
- ++或--
- 方法调用
- 对象创建表达式

这些语句类型被称为表达式语句（expression statement）。下面是一些表达式语句的例子：

```
aValue = 8933.234;                    // assignment statement
aValue++;                             // increment statement
System.out.println("Hello World!");   // method invocation
                                      // statement
Bicycle myBike = new Bicycle();       // object creation
                                      // statement
```

除了这些表达式语句，还有另外两种语句。声明语句（declaration statement）和控制流语句（control flow statement）。声明语句声明一个变量。你已经见过许多声明语句的例子。

```
double aValue = 8933.234; // declaration statement
```

67

最后，控制流语句控制语句的执行次序。3.4节将讲解控制流语句。

3.3.3 块

块（block）是位于成对括号之间的零个或者多个语句的语句组，可以在允许使用单一语句的任何位置使用块。下面的示例BlockDemo[1]演示块的使用：

```
class BlockDemo {
  public static void main(String[] args) {
    boolean condition = true;
    if (condition) { // begin block 1
      System.out.println("Condition is true.");
    } // end block one
    else { // begin block 2
      System.out.println("Condition is false.");
    } // end block 2
    }
}
```

问题和练习：表达式、语句和块

问题

1. 操作符可以用于构成___，它计算出值。
2. 表达式是___的核心成分。
3. 语句可以分组为___。
4. 下面的代码段是___表达式的例子。

```
1 * 2 * 3
```

5. 语句大致相当于自然语言中的句子，但是语句不是以点号结束，而是以___结束。
6. 块是位于成对___之间的零个或者多个语句的语句组，可以在允许使用单一语句的任何位置使用块。

68

练习

识别下面表达式语句的类型：

---

1. tutorial/java/nutsandbolts/examples/BlockDemo.java

- aValue = 8933.234;
- aValue++;
- System.out.println("Hello World!");
- Bicycle myBike = new Bicycle();

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/nutsandbolts/QandE/answers_expressions.html

3.4　控制流语句

通常，源文件中的语句按照它们出现的顺序从头到尾执行。但是，控制流语句（control flow statement）通过使用决策、循环和分支来改变执行流程，使程序按照条件执行特定的代码块。这一节介绍Java编程语句提供的决策语句（if-then、if-then-else和switch）、循环语句（for、while和do-while）以及分支语句（break、continue和return）。

3.4.1　if-then 和 if-then-else 语句

1. if-then语句

if-then是所有控制流语句中最基础的语句。只有特定检测结果为true时，它才通知程序执行特定代码段落。例如，只在自行车处于行进状态时，Bicycle类才允许使用刹车降低自行车的速度。下面是applyBrakes方法的一种可能实现：


```
void applyBrakes(){
  if (isMoving){          // the "if" clause: bicycle must moving
     currentSpeed--;      // the "then" clause:
                          // decrease current speed
  }
}
```

如果测试结果为false（就是说自行车不在行进状态），那么控制就跳转到if-then语句的结尾。

另外，如果then子句只包含一个语句，那么前后括号是可选的：

```
void applyBrakes(){
  if (isMoving) currentSpeed--;     // same as above,
                                    // but without braces
}
```

决定什么时候省略括号是个人习惯问题。省略括号会导致代码更加脆弱。如果以后在then子句中加入第二个语句，常见的错误是忘记加上现在必需的括号。编译器不能捕获这样的错误；你将会得到错误的结果。

2. if-then-else语句

当if子句的计算结果为false时，if-then-else语句提供第二个执行路径。在applyBrakes方法中，如果自行车在没有行进时使用刹车，你可以使用if-then-else语句采取某些措施。在这个例子中，采取的措施是简单地输出错误消息，说明自行车已经停住了。

```
void applyBrakes(){
  if (isMoving) {
     currentSpeed--;
  } else {
     System.err.println("The bicycle has already stopped!");
```

```
    }
}
```

下面的程序IfElseDemo[1]根据考试成绩的值评出等级：如果得分为90或以上，则等级为A；如果得分80或以上，则为B，依此类推。

70

```
class IfElseDemo {
  public static void main(String[] args) {

    int testscore = 76;
    char grade;

    if (testscore >= 90) {
        grade = 'A';
    } else if (testscore >= 80) {
        grade = 'B';
    } else if (testscore >= 70) {
        grade = 'C';
    } else if (testscore >= 60) {
        grade = 'D';
    } else {
        grade = 'F';
    }
    System.out.println("Grade = " + grade);
  }
}
```

程序的输出为：

```
Grade = C
```

可能你注意到，testscore的值能满足复合语句中的多个表达式：76 >= 70和76 >= 60。但是，一旦满足条件，会执行适当语句（grade = 'C';），而不会计算其余条件。

3.4.2 switch语句

与if-then和if-then-else语句不同，switch语句允许使用任意数量的可能执行路径。switch语句可以处理byte、short、char和int基本数据类型。它也可以处理4.5节中讨论的枚举类型（enumerated type），还可以处理“包装”特定原始类型的几个特殊类：Character[2]、Byte[3]、Short[4]和Integer[5]，第8章中讨论这些类。

71

下面的程序SwitchDemo[6]声明int类型的month变量，这个变量表示一年中的月份。程序使用switch语句，按照month的值输出月份名称。

```
class SwitchDemo {
  public static void main(String[] args) {

    int month = 8;
    switch (month) {
      case 1:  System.out.println("January"); break;
      case 2:  System.out.println("February"); break;
      case 3:  System.out.println("March"); break;
      case 4:  System.out.println("April"); break;
```

---

1. tutorial/java/nutsandbolts/examples/IfElseDemo.java
2. docs/api/java/lang/Character.html
3. docs/api/java/lang/Byte.html
4. docs/api/java/lang/Short.html
5. docs/api/java/lang/Integer.html
6. tutorial/java/nutsandbolts/examples/SwitchDemo.java

```
            case 5:  System.out.println("May"); break;
            case 6:  System.out.println("June"); break;
            case 7:  System.out.println("July"); break;
            case 8:  System.out.println("August"); break;
            case 9:  System.out.println("September"); break;
            case 10: System.out.println("October"); break;
            case 11: System.out.println("November"); break;
            case 12: System.out.println("December"); break;
            default: System.out.println("Invalid month.");break;
        }
    }
}
```

在这个例子中，“August”被发送到标准输出。

switch的语句体被称为switch块（switch block）。switch块直接包含的任何语句都可以使用一个或者多个case或default标签加以标记。switch语句计算其表达式，然后执行适当的case语句。

当然，你也可以使用if-then-else语句实现相同操作：

```
int month = 8;
if (month == 1) {
    System.out.println("January");
} else if (month == 2) {
    System.out.println("February");
}
... // and so on
```

决定使用if-then-else语句还是switch语句，有时候是判断的问题。你可以根据可读性和其他因素决定采用哪种语句。If-then-else语句可以用于根据某范围的值或条件进行判断，而switch语句只用于根据单个整数值或者枚举值进行判断。

72

switch语句中另一个有意思的地方是每个case语句后面的break语句。break语句终止包含它的switch语句，控制流程从switch代码块后的第一个语句继续执行。break语句是必需的，因为如果没有break语句，将继续执行下面的case语句。也就是说，如果没有显式的break语句，流程控制将一个接一个地执行后续的case语句。下面的程序SwitchDemo2[1]演示让case语句连续执行为什么可能是有用的：

```
class SwitchDemo2 {
  public static void main(String[] args) {

    int month = 2;
    int year = 2000;
    int numDays = 0;

    switch (month) {
      case 1:
      case 3:
      case 5:
      case 7:
      case 8:
      case 10:
      case 12:
        numDays = 31;
        break;
      case 4:
      case 6:
```

1. tutorial/java/nutsandbolts/examples/SwitchDemo2.java

```
case 9:
case 11:
  numDays = 30;
  break;
case 2:
  if ( ((year % 4 == 0) && !(year % 100 == 0))
       || (year % 400 == 0) )
    numDays = 29;
  else
    numDays = 28;
  break;
default:
    System.out.println("Invalid month.");
    break;
  }
  System.out.println("Number of Days = " + numDays);
 }
}
```

73

程序的输出是：

```
Number of Days = 29
```

从技术上说，最后的break语句不是必需的，因为流程总会执行到switch语句之外。但是，我们建议在最后一个case语句中也使用break语句，以便更容易修改代码并减少出错的可能。default部分处理没有被任何case语句显式处理的所有值。

3.4.3 while 和 do-while 语句

当特定条件为true时，while语句持续执行一个代码块。其语法可以表示为：

```
while (expression) {
  statement(s)
}
```

while语句计算表达式*expression*，它必须返回一个布尔值。如果表达式的计算结果为true，那么while语句就执行while块中的语句*statement(s)*。while语句继续检测表达式并且执行它的块，直到表达式的计算结果为false。下面的程序WhileDemo[1]使用while语句输出从1到10的值：

```
class WhileDemo {
  public static void main(String[] args){
    int count = 1;
    while (count < 11) {
      System.out.println("Count is: " + count);
      count++;
    }
  }
}
```

还可以使用while语句实现无限循环，如下：

```
while (true){
  // your code goes here
}
```

Java编程语言还提供do-while语句，可以表示为：

74

```
do {
     statement(s)
```

1. tutorial/java/nutsandbolts/examples/WhileDemo.java

```
} while (expression);
```

do-while和while语句之间的区别在于，do-while在循环结尾处（而不是开始处）计算其表达式。因此，do块中的语句总会至少执行一次，如下面的程序DoWhileDemo[1]所示：

```
class DoWhileDemo {
  public static void main(String[] args){
    int count = 1;
    do {
        System.out.println("Count is: " + count);
        count++;
    } while (count <= 11);
  }
}
```

3.4.4 for语句

for语句提供了对一个范围内的值进行迭代的紧凑方式。程序员经常把它称为“for循环”，这是因为它的运行方式是重复循环，直到满足特定条件为止。for语句的一般形式可以表示为：

```
for (initialization; termination; increment) {
  statement(s)
}
```

在使用for语句的这个版本时，要牢记：

- *Initialization*（初始化）表达式对循环进行初始化；它在循环开始时执行一次。
- 当*termination*（终止）表达式计算结果为false时，循环终止。
- 在循环的每次迭代之后调用*increment*表达式；最常见的方式是使用这个表达式递增或者递减一个值。

75

下面的程序ForDemo[2]使用for语句的一般形式把数字1到10发送到标准输出：

```
class ForDemo {
  public static void main(String[] args){
    for(int i=1; i<11; i++){
      System.out.println("Count is: " + i);
    }
  }
}
```

这个程序的输出为：

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
```

注意初始化表达式中声明变量的代码。这个变量的作用域是从其声明开始，直到for语句控制块的结束，所以它也可以用在终止和递增表达式中。如果在循环之外不需要控制for语句的变量，那么最好在初始化表达式中声明变量。i、j和k这样的名称经常用于控制for循环；在初始化表达式中声明它们，就限制了它们的生存周期并且减少了错误的可能。

---

1. tutorial/java/nutsandbolts/examples/DoWhileDemo.java
2. tutorial/java/nutsandbolts/examples/ForDemo.java

for循环的三个表达式都是可选的；可以像下面这样创建无限循环：

```
for ( ; ; ) {     // infinite loop

  // your code goes here
}
```

for语句还有另外一种形式，这种形式用于在集合（参见11章）和数组（参见3.1.3节）之中进行迭代。有时候把这种形式称为增强的for（enhanced for）语句，它可以使循环更加紧凑和容易阅读。为了演示，参考下面的数组，它保存数字1到10：

```
int[] numbers = {1,2,3,4,5,6,7,8,9,10};
```

下面的程序EnhancedForDemo[1]使用增强for循环遍历这个数组：

```
class EnhancedForDemo {
  public static void main(String[] args){
    int[] numbers = {1,2,3,4,5,6,7,8,9,10};
    for (int item : numbers) {
      System.out.println("Count is: " + item);
    }
  }
}
```

在这个例子中，变量item保存从数字数组获得的当前值。这个程序的输出和前一个程序相同：

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
```

我们建议，只要情况允许，就使用for语句的这种形式，而不使用其一般形式。

3.4.5 分支语句

1. break语句

break语句有两种形式：带标签和无标签。在前面讨论switch语句时，你已经见到了无标签形式。

也可以使用无标签的break语句终止for、while或者do-while循环，如下面的程序BreakDemo[2]所示：

```
class BreakDemo {
  public static void main(String[] args) {

    int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076,
                          2000, 8, 622, 127 };
    int searchfor = 12;

    int i;
    boolean foundIt = false;
```

1. tutorial/java/nutsandbolts/examples/EnhancedForDemo.java
2. tutorial/java/nutsandbolts/examples/BreakDemo.java

```
for (i = 0; i < arrayOfInts.length; i++) {
  if (arrayOfInts[i] == searchfor) {
    foundIt = true;
    break;
  }
}

if (foundIt) {
    System.out.println("Found " + searchfor +
                                    " at index " + i);
} else {
    System.out.println(searchfor + " not in the array");
  }
}
```

这个程序在数组中搜索数字12。当找到这个值时，以粗体显示的break语句终止循环。然后控制流转到程序结尾的输出语句。

这个程序的输出是：

```
Found 12 at index 4
```

无标签break语句终止最内层的switch、for、while或do-while语句，而带标签的break语句终止外层语句。下面的程序BreakWithLabelDemo[1]和前一个程序类似，但是它使用嵌套for循环在二维数组中搜索一个值。当找到这个值时，带标签break语句终止外层for循环（标签为“search”）：

78

```
class BreakWithLabelDemo {
  public static void main(String[] args) {

    int[][] arrayOfInts = { { 32, 87, 3, 589 },
                            { 12, 1076, 2000, 8 },
                            { 622, 127, 77, 955 }
                          };
  int searchfor = 12;

  int i;
  int j = 0;
  boolean foundIt = false;

search:
  for (i = 0; i < arrayOfInts.length; i++) {
     for (j = 0; j < arrayOfInts[i].length; j++) {
       if (arrayOfInts[i][j] == searchfor) {
         foundIt = true;
         break search;
       }
     }
  }

  if (foundIt) {
      System.out.println("Found " +
                         searchfor + " at " + i + ", " + j);
  } else {
      System.out.println(searchfor + " not in the array");
  }
 }
}
```

1. tutorial/java/nutsandbolts/examples/BreakWithLabelDemo.java

程序的输出为：

```
Found 12 at 1, 0
```

break语句终止带标签的语句；它不把控制流转到标签位置。控制流被转到紧跟在带标签（被终止的）语句后面的语句。

2. continue语句

continue语句跳过for、while或者do-while循环的当前迭代。无标签形式跳到最内层循环体的结尾，并且计算控制循环的布尔表达式。下面的程序ContinueDemo[1]遍历一个字符串，计算字母“p”的出现次数。如果当前字母不是p，continue语句就跳过循环的剩余部分，然后处理下一个字符。如果是p，程序就递增字母计数器。

79

```
class ContinueDemo {
  public static void main(String[] args) {

    String searchMe = "peter piper picked a peck of " +
                      "pickled peppers";
    int max = searchMe.length();
    int numPs = 0;

    for (int i = 0; i < max; i++) {
      //interested only in p's
      if (searchMe.charAt(i) != 'p')
          continue;

      //process p's
      numPs++;
    }
    System.out.println("Found " + numPs +
                       " p's in the string.");
  }
}
```

这个程序的输出是：

```
Found 9 p's in the string.
```

为了更清楚地了解这种处理效果，可以删除continue语句，重新进行编译。再次运行程序时，计数器将是错误的，它指出找到了35个p，而不是9个。

带标签的continue语句跳过标有给定标签的外层循环的当前迭代。下面的示例程序ContinueWithLabelDemo[2]使用嵌套的循环在一个字符串中搜索子字符串。这里需要两个嵌套的循环：一个迭代子字符串，另一个迭代正在被搜索的字符串。下面的程序ContinueWithLabelDemo使用带标签的continue语句跳过外层循环的一次迭代。

80

```
class ContinueWithLabelDemo {
  public static void main(String[] args) {

    String searchMe = "Look for a substring in me";
    String substring = "sub";
    boolean foundIt = false;

    int max = searchMe.length() - substring.length();

  test:
    for (int i = 0; i <= max; i++) {
```

1. tutorial/java/nutsandbolts/examples/ContinueDemo.java
2. tutorial/java/nutsandbolts/examples/ContinueWithLabelDemo.java

```
        int n = substring.length();
        int j = i;
        int k = 0;
        while (n-- != 0) {
          if (searchMe.charAt(j++) != substring.charAt(k++)) {
            continue test;
          }
        }
        foundIt = true;
        break test;
      }
      System.out.println(foundIt ? "Found it" :
                                   "Didn't find it");
    }
  }
```

下面是这个程序的输出：

```
Found it
```

3. return语句

最后一个分支语句是return语句。return语句退出当前方法，控制流返回到调用这个方法的位置。return语句有两种形式：一种返回一个值，另一种不返回值。要返回一个值，只需将此值（或一个计算出此值的表达式）放在return关键字后面：

```
return ++count;
```

return的返回值的数据类型必须匹配方法声明的返回值类型。当方法被声明为void时，使用return的不返回值形式：

```
return;
```

81　4.2.2节的第2小节将讨论编写方法所需的所有知识。

3.4.6　控制流语句小结

if-then是所有控制流语句中最基础的语句。它通知程序，只有特定检测结果为true时才执行特定代码段落。

if-then-else语句提供当if子句结果为false时的第二个执行路径。

与if-then和if-then-else语句不同，switch语句允许任意数量的可能执行路径。

当特定条件为true时，while和do-while语句持续执行一个语句块。

do-while和while之间的不同之处在于，do-while在循环的结尾处（而不是开始处）计算其表达式。因此，do块内的语句总会执行至少一次。

for语句提供对一个范围内的值进行迭代的紧凑方式。它有两种形式，其中一种用于遍历集合和数组。

问题和练习：控制流语句

问题

1. Java编程语句支持的最基础的控制流语句是___语句。
2. ___语句允许使用任意数量的可能执行路径。
3. ___语句和while语句类似，但是在循环的___计算其表达式。
4. 如何使用for语句编写无限循环？
5. 如何使用while语句编写无限循环？

练习

1. 查看下面的代码段。

```
if (aNumber >= 0)
  if (aNumber == 0) System.out.println("first string");
else System.out.println("second string");
System.out.println("third string");
```

a. 如果aNumber为3，你认为代码将会输出什么结果？

b. 编写一个测试程序包含前面的代码段，使aNumber为3。这个程序的输出是什么？这是你预测的输出吗？解释输出为什么是这样；换句话说，代码段的控制流是怎样的？ 82

c. 只使用空格和换行，重新格式化代码段，使控制流更易于理解。

d. 使用括号“{”和“}”，进一步使代码段更清楚明了。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/nutsandbolts/QandE/answers_flow.html 83

第4章

类和对象

了解了Java编程语言的基础知识之后，现在你可以学习编写自己的类了。在本章中，你将了解关于定义自己的类的信息，包括声明成员变量、方法和构造器。

你将学习如何使用类创建对象以及如何使用创建的对象。

本章还将介绍嵌套在其他类之内的嵌套类以及枚举和注解。

4.1 类

第2章介绍面向对象的概念时使用自行车类作为示例，它具有比赛自行车、山地自行车和双座自行车等子类。下面是Bicycle类的可能实现，用来给出类的声明的概貌。本章的后续小节将回过头来一步一步地讲解类的声明。现在，你不必过多地考虑细节问题。


```
public class Bicycle {
    // the Bicycle class has three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int startCadence, int startSpeed, int startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    // the Bicycle class has four methods
    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

MountainBike类是Bicycle的子类，它的类声明可能像这样：

```
public class MountainBike extends Bicycle {
    // the MountainBike subclass has one field
```

```
public int seatHeight;

// the MountainBike subclass has one constructor
public MountainBike(int startHeight, int startCadence,
                    int startSpeed, int startGear) {

    super(startCadence, startSpeed, startGear);
    seatHeight = startHeight;
}

// the MountainBike subclass has one method
public void setHeight(int newValue) {
    seatHeight = newValue;
}
}
```

MountainBike继承Bicycle的所有字段和方法，并且添加了seatHeight字段和设置这个字段的方法（山地自行车的车座可以根据地形的要求上调或者下调）。

4.1.1 声明类

你见过以如下方式定义的类：

```
class MyClass {
  // field, constructor, and method declarations
}
```

这是*类声明*（class declaration）。类体（class body）（括号之间的内容）包含用于从这个类创建的对象的生存周期的所有代码：用于初始化新对象的构造器，用于提供类及其对象的状态的字段的声明，以及实现类及其对象的行为的方法。

前面的类声明是最基本的——它只包含类声明必需的成分。在类声明的开头，你可以提供关于类的更多信息，比如其超类的名称、是否实现某个接口，等等。例如：

```
class MyClass extends MySuperClass implements YourInterface {
  // field, constructor, and method declarations
}
```

这表示MyClass是MySuperClass的子类，它实现YourInterface接口。

在最开头，你还可以使用public或者private这样的修饰符——所以你会发现类声明的起始行可能变得非常复杂。修饰符public和private（它们确定其他的哪些类可以访问MyClass）将在本章后面讨论。第5章将介绍如何和为什么要使用extends和implements关键字。目前，你不必考虑这些额外的复杂内容。

总的来说，类声明可以包含下面这些组件，按照顺序是：

(1) 修饰符，比如public、private，以后还有很多其他的修饰符。

(2) 类的名称，按照惯例首字母大写。

(3) 类的父类（超类）的名称（如果有父类的话），前面加关键字extends。一个类只能扩展（extend）一个父类。

(4) 类实现的接口清单（如果有接口的话），以逗号分隔，前面加关键字implements。一个类可以实现（implement）一个以上的接口。

(5) 类体，用花括号{}括起来。

4.1.2 声明成员变量

变量有下面几种：

- 类中的成员变量——它们称为字段（field）。
- 方法或者代码块中的变量——它们称为局部变量（local variable）。
- 方法声明中的变量——它们称为参数（形参）（parameter）。

Bicycle类使用如下代码行定义其字段：

```
public int cadence;
public int gear;
public int speed;
```

字段声明由3部分组成，按照顺序是：

(1) 零个或者多个修饰符，比如public或者private。

(2) 字段类型。

(3) 字段名称。

Bicycle类的字段名为cadence、gear和speed，它们都属于整数数据类型(int)。public关键字表示这些字段是公共成员，可以被能够访问这个类的任何对象访问。

1. 访问修饰符

第一个（最左侧的）修饰符让你可以控制其他哪些类能够访问成员字段。目前，只需考虑public和private即可。稍后将讨论其他访问修饰符。

- public修饰符——可以从所有类访问这个字段。
- private修饰符——只能从它自己的类中访问这个字段。

按照封装的思想，通常把字段声明为私有的。这就是说，只能从Bicycle类直接访问它们。

88 但是，我们还是需要从外部访问这些值。这可以通过添加获取字段值的公共方法间接地做到：

```
public class Bicycle {

private int cadence;
private int gear;
private int speed;

public Bicycle(int startCadence,
               int startSpeed, int startGear) {
  gear = startGear;
  cadence = startCadence;
  speed = startSpeed;
}

public int getCadence() {
  return cadence;
}

public void setCadence(int newValue) {
  cadence = newValue;
}

public int getGear() {
  return gear;
}

public void setGear(int newValue) {
  gear = newValue;
}

public int getspeed() {
  return speed;
}
```

```
  public void applyBrake(int decrement) {
    speed -= decrement;
  }

  public void speedUp(int increment) {
    speed += increment;
  }

}
```

2. 类型

所有变量都必须具有类型。你可以使用原始类型，比如int、float和boolean等，也可以使用引用类型，比如字符串、数组或者对象。 89

3. 变量名称

所有变量，不管它们是字段、局部变量还是参数，都遵守3.1.1节介绍的命名规则和惯例。

在本章中请注意，相同的命名规则和惯例也用于方法和类的名称，但是有以下例外：

- 类名称的第一个字母应该大写。
- 方法名称中的第一个（或者唯一的）单词应该是动词。

4.1.3 定义方法

这是方法声明的一个典型例子：

```
public double calculateAnswer(double wingSpan,
    int numberOfEngines, double length, double grossTons) {
  // do the calculation here
}
```

方法声明必需的元素是方法的返回类型、名称、一对圆括号()以及花括号{}之间的方法体。

更加常见的方法声明有6个组成部分，按照顺序是：

(1) 修饰符——比如public、private，以后将学习其他修饰符。

(2) 返回类型——方法返回值的数据类型；或者，如果方法没有返回值，就使用void。

(3) 方法名称——字段名称的命名规则也适用于方法名称，但是惯例稍有不同。

(4) 圆括号中的参数清单——括在圆括号()中的用逗号分隔的输入参数清单，参数名称前附加它们的数据类型。如果没有参数，则必须使用空的圆括号。

(5) 异常清单——以后讨论它。

(6) 括在花括号中的方法体——方法的代码（包括局部变量的声明）出现在这里。

本章后面将讨论修饰符、返回类型和参数。第9章中讨论异常。 90

定义 *方法签名（method signature）由方法声明的两个部分构成——方法的名称和参数类型。*

前面声明的方法的签名是：

```
calculateAnswer(double, int, double, double)
```

1. 命名方法

虽然方法名称可以是任何合法的标识符，但是编码惯例对方法名称有所限制。按照惯例，方法名称应该是一个小写的动词，或者是多个单词，以小写动词开始，后面是形容词、名词等。在多单词名称中，第二个单词和后续单词的第一个字母应该大写。下面是一些例子：

```
run
runFast
getBackground
getFinalData
```

```
compareTo
setX
isEmpty
```

典型情况下，方法在其类之内使用唯一名称。但是，因为可以进行方法重载（method overload），所以某个方法的名称可能和其他方法一样。

2. 重载方法

Java编程语言支持重载方法，并且Java能够区分使用不同方法签名的方法。这就是说，如果类之内的方法具有不同的参数清单，那么这些方法可以使用相同名称。（第5章将讨论这种情况的一些限定条件。）

假设你有一个类，用于描述各种类型的数据（字符串、整数等），它包含描述各种数据类型的方法。每个方法都使用新名称会显得很繁琐。例如drawString、drawInteger、drawFloat等。在Java编程语言中，所有描述方法都可以使用相同的名称，但是传递给每个方法的参数清单可以不同。所以，数据描述类就可以声明4个名为draw的方法，每个方法的参数清单各不相同：

91

```
public class DataArtist {
  ...
  public void draw(String s) {
    ...
  }
  public void draw(int i) {
    ...
  }
  public void draw(double f) {
    ...
  }
  public void draw(int i, double f) {
    ...
  }
}
```

通过传递到方法中的参数数量和类型，重载的方法被加以区分。在这个代码示例中，draw(String s)和draw(int i)是不同的方法，因为它们需要不同的参数类型。

不能使用相同名称声明参数类型和数量相同的多个方法，因为编译器不能区分它们。

在区分方法时，编译器不会考虑返回类型，所以你不能声明两个签名相同的方法，即使它们具有不同的返回类型。

注意　使用重载的方法时应该有节制，因为它们使代码的可读性降低了很多。

4.1.4　为类提供构造器

类包含构造器，调用构造器可以从类“蓝图”创建对象。构造器的声明和方法的声明类似——只是它们使用类的名称，而且没有返回类型。例如，Bicycle类有一个构造器：

```
public Bicycle(int startCadence,
               int startSpeed, int startGear) {
  gear = startGear;
  cadence = startCadence;
  speed = startSpeed;
}
```

为了创建新的名为myBike的Bicycle对象，使用new操作符调用构造器：

```
Bicycle myBike = new Bicycle(30, 0, 8);
```

92

new Bicycle(30, 0, 8)在内存中为对象创建空间并且初始化其字段。

虽然Bicycle只有一个构造器，但其实它可以具有其他构造器，例如无实参构造器：

```
public Bicycle() {
  gear = 1;
  cadence = 10;
  speed = 0;
}
```

Bicycle yourBike = new Bicycle();调用无实参构造器，以便创建名为yourBike的新Bicycle对象。

Bicycle中可以同时声明这两个构造器，因为它们的参数清单不同。和方法一样，Java平台根据构造器实参清单中的实参数量和类型区分构造器。不能在同一个类中编写实参数量和类型都相同的两个构造器，因为平台不能区分它们。这样做会导致编译时错误。

不必非要为类提供任何构造器，但是这样做的时候要谨慎。编译器会自动为没有构造器的所有类提供无实参的默认构造器。这个默认构造器将调用超类的无实参构造器。在这种情况下，如果超类没有无实参构造器，编译器就会指出问题，所以你必须检查超类是否有无实参构造器。如果你的类没有显式超类，那么就使用Object作为隐式超类，这个超类确实具有无实参构造器。

可以使用自己的超类构造器。本章开始的MountainBike类就是这样做的。第5章中将讨论这种情况。

可以在构造器的声明中使用访问修饰符，从而控制其他哪些类可以调用这个构造器。

注意 如果另一个类不能调用MyClass构造器，它就不能直接创建MyClass对象。

4.1.5 把信息传递给方法或构造器

在方法或者构造器的声明中需要声明方法或者构造器的参数的数量和类型。例如下面的方法，它根据贷款数额、利率和贷款时间长度（周期数量）以及贷款的未来值，计算家庭贷款的月还款额：

93

```
public double computePayment(double loanAmt,
                             double rate,
                             double futureValue,
                             int numPeriods) {
  double interest = rate / 100.0;
  double partial1 = Math.pow((1 + interest), -numPeriods);
  double denominator = (1 - partial1) / interest;
  double answer = (-loanAmt / denominator) -
                  ((futureValue * partial1) / denominator);
  return answer;
}
```

这个方法有4个形参：贷款数额、利率、未来值和周期数量。前3个形参是双精度浮点数，第4个是整数。形参用在方法体中，并且在运行时将采用传递进来的实参值。

注意 形参（parameter）指的是方法声明中的变量清单。实参（argument）是调用方法时传递给方法的实际值。调用方法时，在类型和顺序方面，实参必须和声明的形参相匹配。有时形参和实参笼统地称为参数。

---

1. 参数类型

方法或者构造器的参数可以使用任何数据类型，这包括基本数据类型，比如双精度浮点数、浮点数和整数，就像你在computePayment方法中看到的；还包括引用数据类型，比如对象和数组。

下面是一个方法的例子，它接收数组作为实参。在这个例子中，方法从Point对象数组创建新的Polygon对象并初始化它（假设Point是一个描述x、y坐标的类）：

```
public Polygon polygonFrom(Point[] corners) {
  // method body goes here
}
```

注意　Java编程语言不允许把方法传递给方法。但是可以把对象传递给方法，然后调用这个对象的方法。

2. 任意数量的实参

你可以使用名为可变参数（vararg）的结构把任意数量的值传递给方法。当你不知道有多少个特定类型的实参要传递给方法时，可以使用可变参数。这是手工创建数组的便捷方式（前面
94 的方法可以使用可变参数，而不使用数组）。

为了使用可变参数，在最后一个参数的类型之后加上省略号（3个点号...），然后是空格和参数名称，然后就可以使用任意数量的参数（包括没有参数）调用这个方法。

```
public Polygon polygonFrom(Point... corners) {
  int numberOfSides = corners.length;
  double squareOfSide1, lengthOfSide1;
  squareOfSide1 =
  (corners[1].x - corners[0].x)*(corners[1].x - corners[0].x) +
  (corners[1].y - corners[0].y)*(corners[1].y - corners[0].y);
  lengthOfSide1 = Math.sqrt(squareOfSide1);
  // more method body code follows that creates
  // and returns a polygon connecting the Points
}
```

你能够发现，在这个方法中corners被作为数组处理。可以使用数组，也可以使用一系列实参调用这个方法。在这两种情况下，这个方法体中的代码都将把参数作为数组处理。

最常使用可变参数的地方是输出方法——例如下面这个printf方法：

```
public PrintStream printf(String format, Object... args)
```

允许你输出任意数量的对象。可以这样调用它：

```
System.out.printf("%s: %d, %s%n", name, idnum, address);
```

或者这样调用：

```
System.out.printf("%s: %d, %s, %s, %s%n", name,
                                idnum, address, phone, email);
```

或者使用不同数量的实参。

3. 参数名称

当声明方法或者构造器的参数时，需要为参数提供名称。在方法体之内使用这个名称引用传递进来的实参。

在参数的作用域内，参数名称必须是唯一的。同一个方法或者构造器的任意两个参数名称不能相同，并且参数名称不能和方法或者构造器内的局部变量名称相同。

95 参数名称可以和一个类的字段名称相同。如果这样使用，那么参数被称为使这个字段遮蔽

(shadow)。遮蔽的字段会使代码难以阅读，并且按照惯例只在设置特定字段的构造器或者方法中使用。例如，考虑下面的Circle类和它的setOrigin方法：

```
public class Circle {
  private int x, y, radius;
  public void setOrigin(int x, int y) {
    ...
  }
}
```

Circle类有3个字段：x、y和radius。setOrigin方法有两个参数，每个参数的名称都和一个字段名称相同。每个方法参数都使同名的字段遮蔽了。所以如果在方法体内使用简单名称x或者y，则是引用参数，而不是引用字段。为了访问字段，必须使用限定名称。我们将在本章后面4.3.2节中讨论这种情况。

4. 传递基本数据类型实参

原始实参，比如int或者double，是按值（by value）传递给方法的。这就是说，对参数值的任何改动都只限于这个方法的作用域内。当方法返回时，参数消失并且对参数的所有改动都会丢失。下面是一个例子：

```
public class PassPrimitiveByValue {

  public static void main(String[] args) {

    int x = 3;

    // invoke passMethod() with x as argument
    passMethod(x);

    // print x to see if its value has changed
    System.out.println("After invoking passMethod, x = " + x);

  }

  // change parameter in passMethod()
  public static void passMethod(int p) {
    p = 10;
  }
}
```

运行这个程序的输出为：

```
After invoking passMethod, x = 3
```

96

5. 传递引用数据类型实参

引用数据类型参数，比如对象，也是按值传递给方法的。这就是说，当方法返回时，传递给方法的引用仍然像以前那样引用相同的对象。但是，在方法中可以改变对象的字段值，如果它们具有适当访问级别的话。

例如，考虑任意类中移动Circle对象的方法：

```
public void moveCircle(Circle circle, int deltaX, int deltaY) {

  // code to move origin of circle to x+deltaX, y+deltaY
  circle.setX(circle.getX() + deltaX);
  circle.setY(circle.getY() + deltaY);

  // code to assign a new reference to circle
  circle = new Circle(0, 0);
}
```

使用3个实参调用这个方法：

```
moveCircle(myCircle, 23, 56)
```

在方法内，circle最初引用myCircle。方法把circle引用的对象（即myCircle）的x和y坐标分别改动了23和56。（这些改动在方法返回时维持不变。）使用x = y = 0把circle赋值为引用新的Circle对象。但是，这个赋值不具有永久性，因为引用是按值传递的，并且不能改动。在方法内，circle指向的对象改动了，但是，当方法返回时，myCircle仍然引用相同的Circle对象，就像调用方法之前一样。

4.2　对象

典型的Java程序会创建很多对象，我们知道，它们之间是通过调用方法进行交互的。

通过这些对象之间的交互，程序可以完成各种任务，比如实现图形用户界面（GUI）、运行动画或者通过网络发送和接收信息。对象完成了它要完成的工作之后，对象的资源就被回收，
97 以供其他对象使用。

下面是一个小程序，名为CreateObjectDemo[1]，它创建3个对象：一个Point对象和两个Rectangle对象。在4.2.1节的第3小节列出了Point和Rectangle类。

```
public class CreateObjectDemo {

  public static void main(String[] args) {

    // Declare and create a point object
    // and two rectangle objects.
    Point originOne = new Point(23, 94);
    Rectangle rectOne = new Rectangle(originOne, 100, 200);
    Rectangle rectTwo = new Rectangle(50, 100);

    // display rectOne's width, height, and area
    System.out.println("Width of rectOne: " + rectOne.width);
    System.out.println("Height of rectOne: " + rectOne.height);
    System.out.println("Area of rectOne: " +
                                       rectOne.getArea());

    // set rectTwo's position
    rectTwo.origin = originOne;

    // display rectTwo's position
    System.out.println("X Position of rectTwo: " +
                                          rectTwo.origin.x);
    System.out.println("Y Position of rectTwo: " +
                                          rectTwo.origin.y);

    // move rectTwo and display its new position
    rectTwo.move(40, 72);
    System.out.println("X Position of rectTwo: " +
                                          rectTwo.origin.x);
    System.out.println("Y Position of rectTwo: " +
                                          rectTwo.origin.y);
  }
}
```

98 这个程序创建、处理各个对象，并且显示各个对象的信息。下面是输出：

---

1. tutorial/java/javaOO/examples/CreateObjectDemo.java

```
Width of rectOne: 100
Height of rectOne: 200
Area of rectOne: 20000
X Position of rectTwo: 23
Y Position of rectTwo: 94
X Position of rectTwo: 40
Y Position of rectTwo: 72
```

下面3节使用前面的例子描述对象在程序内的生存周期。通过这些讲解，你将学到如何编写代码在自己的程序中创建和使用对象。你还将学习在对象的生存周期结束后，系统如何清理对象。

4.2.1 创建对象

我们知道，类为对象提供蓝图；对象从类中创建。下列是CreateObjectDemo程序中的语句，每行语句创建一个对象，并且把它赋值给变量：

```
Point originOne = new Point(23, 94);
Rectangle rectOne = new Rectangle(originOne, 100, 200);
Rectangle rectTwo = new Rectangle(50, 100);
```

第一行创建Point类的对象，第二行和第三行分别创建Rectangle类的对象。

这3行语句都有3个部分（下面详细讨论）：

(1) **声明**。粗体显示的代码都是变量声明，它们把变量名称和对象类型关联起来。

(2) **实例化**。new关键字是用于创建对象的Java操作符。

(3) **初始化**。new操作符后面调用构造器，构造器初始化新对象。

1. 声明引用对象的变量

在前面讲过，为了声明变量，需要编写：

```
type name;
```

这是在通知编译器，你将使用name引用类型为type的数据。在使用原始变量时，这个声明还为变量保留适当数量的内存。

99

也可以单独声明引用变量。例如：

```
Point originOne;
```

如果这样声明originOne，那么在实际创建对象并且把对象赋值给该变量之前，该变量的值是未确定的。仅仅声明引用变量并不会创建对象。创建对象需要使用new操作符，这将在下一节讨论。在代码中使用originOne之前，必须把对象赋值给它。否则，就会出现编译器错误。

这个变量当前不引用任何对象，处于这一状态的变量如图4-1所示（变量名称originOne，加上不指向任何内容的引用）。



图4-1 originOne为null

2. 实例化类

new操作符为新的对象分配内存，并且返回对这段内存的引用，从而实例化类。new操作符还调用对象的构造器。

注意 短语“实例化类”的含义和“创建对象”相同。当你创建对象时，就创建了类的“实例”，从而“实例化”了类。

---

new需要单一的后缀实参：调用构造器。构造器的名称提供要实例化的类的名称。

new操作符返回对它创建的对象的引用。通常把这个引用赋给适当类型的变量，比如：

```
Point originOne = new Point(23, 94);
```

new操作符返回的引用不一定要赋值给变量。可以在表达式中直接使用引用。例如：

```
int height = new Rectangle().height;
```

100 这个语句将在下一节中讨论。

3. 初始化对象

下面是Point类的代码[1]：

```
public class Point {
  public int x = 0;
  public int y = 0;
  // constructor
  public Point(int a, int b) {
  x = a;
  y = b;
  }
}
```

这个类包含单一构造器。你能够识别出构造器，因为它的声明使用的名称与类名称相同，并且没有返回类型。Point类中的构造器采用两个整数实参，由代码（int a, int b）声明。下面的语句提供23和94作为这些实参的值：

```
Point originOne = new Point(23, 94);
```

执行这行语句的结果如图4-2所示。

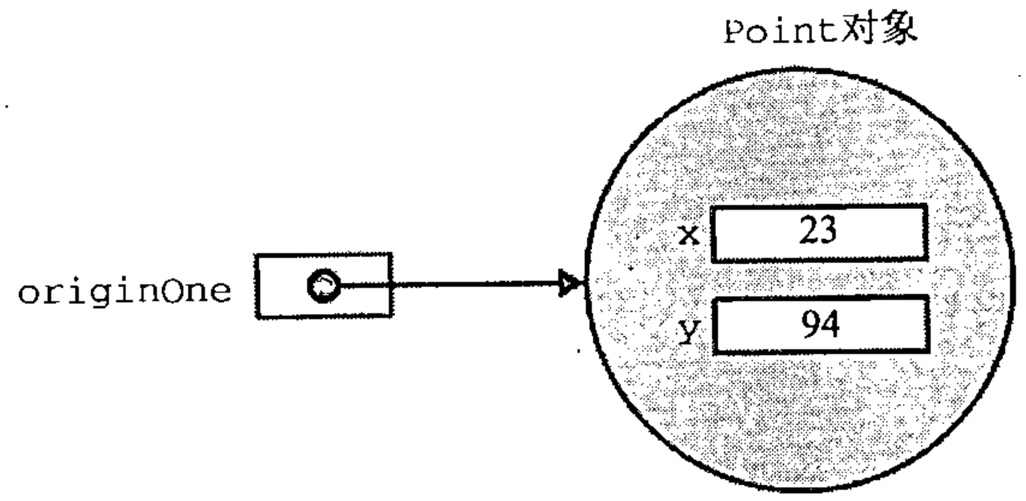


图4-2　现在originOne指向Point对象

101 下面是Rectangle类的代码[2]，它包含4个构造器：

```
public class Rectangle {
  public int width = 0;
  public int height = 0;
  public Point origin;

  // four constructors
  public Rectangle() {
    origin = new Point(0, 0);
  }
  public Rectangle(Point p) {
    origin = p;
  }
  public Rectangle(int w, int h) {
```

1. tutorial/java/javaOO/examples/Point.java
2. tutorial/java/javaOO/examples/Rectangle.java

```
    origin = new Point(0, 0);
    width = w;
    height = h;
  }
  public Rectangle(Point p, int w, int h) {
    origin = p;
    width = w;
    height = h;
  }

  // a method for moving the rectangle
  public void move(int x, int y) {
    origin.x = x;
    origin.y = y;
  }

  // a method for computing the area of the rectangle
  public int getArea() {
    return width * height;
  }
}
```

每个构造器都允许你为矩形的大小和宽度提供初始值，并且用到了原始类型和引用类型。如果一个类有多个构造器，那么构造器的签名必须不同。Java编译器根据实参的数量和类型区分构造器。当Java编译器遇到下面的代码时，它知道应该调用Rectangle类中需要Point实参以及两个整数实参的那个构造器。

```
Rectangle rectOne = new Rectangle(originOne, 100, 200);
```

这会调用Rectangle的构造器之一，把origin初始化为originOne。而且，这个构造器把width设置为100，height设置为200。现在，有指向相同Point对象的两个引用；多个引用可以指向同一个对象，如图4-3所示。 102

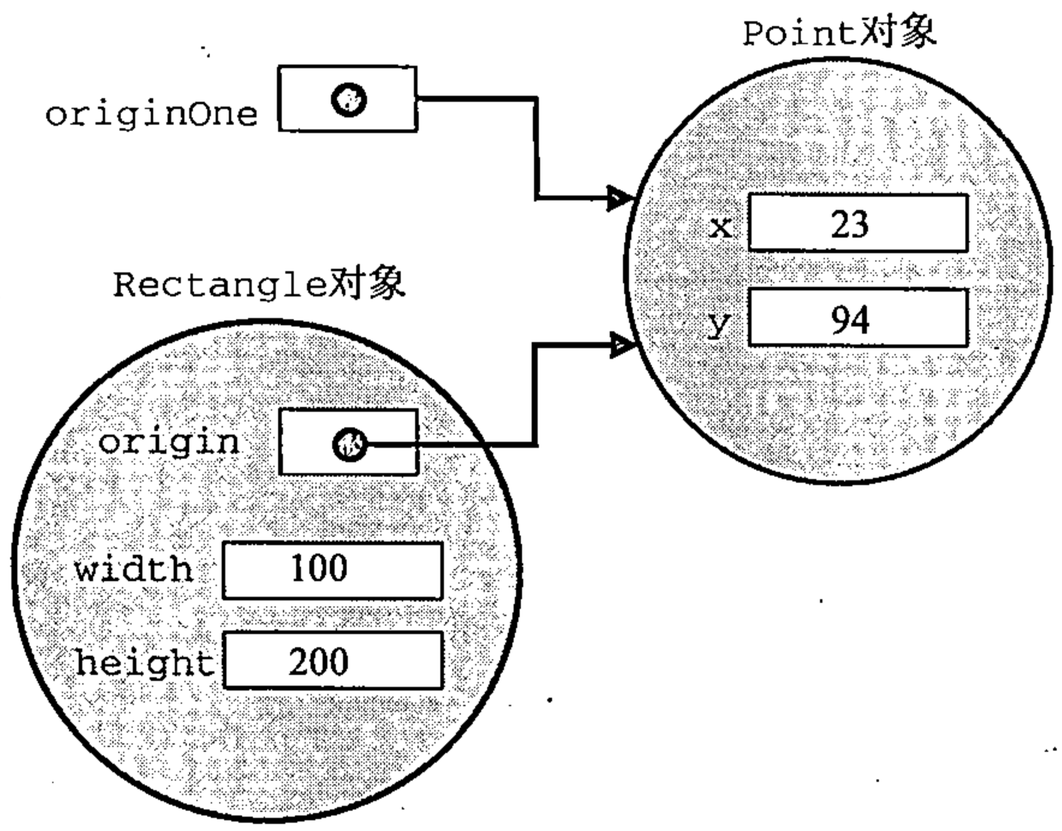


图4-3 现在originOne指向Point对象

下面这行代码调用需要两个整数实参的Rectangle构造器，该构造器为width和height提供初始值。如果你查看这个构造器中的代码，就会发现它创建新的Point对象，对象的x和y值被初始化为0。

```
Rectangle rectTwo = new Rectangle(50, 100);
```

下面语句中使用的Rectangle构造器不采用任何实参，所以它被称为无实参构造器

(no-argument constructor)：

```
Rectangle rect = new Rectangle();
```

所有类都必须具有至少一个构造器。如果一个类没有明确地声明任何构造器，那么Java编译器就会自动地提供无实参构造器，称为默认构造器（default constructor）。这个构造器调用类的父类的无实参构造器；或者，如果类没有其他父类，则调用Object的构造器。如果父类没有构造器（Object有构造器），编译器就会拒绝编译程序。

4.2.2　使用对象

在创建对象之后，你可能希望将它用于某种目的。可能需要使用一个对象字段的值、改动
103 一个对象字段或者调用对象的方法来执行一个操作。

1. 引用对象的字段

对象字段是通过其名称来引用的。必须使用含义明确的名称。

可以在字段所在的类之内使用字段的简单名称。例如，我们可以在Rectangle类之内添加一行语句，它输出width和height：

```
System.out.println("Width and height are: " +
                                width + ", " + height);
```

在这个例子中，width和height是简单名称。

对象的类之外的代码必须使用对象引用或者表达式，后面跟着点号（.）操作符，再后面是简单字段名称，例如：

```
objectReference.fieldname
```

例如，CreateObjectDemo类中的代码在Rectangle类之外。所以，为了引用名为rectOne的Rectangle对象内的origin、width和height字段，CreateObjectDemo类必须分别使用名称rectOne.origin、rectOne.width和rectOne.height。程序使用其中的两个名称显示rectOne的宽度和高度：

```
System.out.println("Width of rectOne: " + rectOne.width);
System.out.println("Height of rectOne: " + rectOne.height);
```

试图在CreateObjectDemo类的代码中使用简单名称width和height是没有意义的——这些字段只存在于Rectangle对象中，并且会导致编译器错误。

后面，程序使用类似代码显示关于rectTwo的信息。类型相同的对象具有相同实例字段的它们自己的副本。因此，每个Rectangle对象都具有名为origin、width和height的字段。当你通过对象引用访问实例字段时，就引用了特定对象的字段。CreateObjectDemo程序中的rectOne和rectTwo这两个对象具有不同的origin、width和height字段。

为了访问字段，可以使用对象的命名引用，就像在前面的例子中那样；也可以使用返回对象引用的任何表达式。前面讲过，new操作符返回对象的引用，所以你可以使用new操作符的返回值访问新对象的字段：

```
int height = new Rectangle().height;
```

这行语句创建新的Rectangle对象，并且立即获得其高度。从本质上讲，这行语句计算Rectangle的默认高度。注意，在执行完这行语句之后，程序再也不具有对创建出的Rectangle的引用，因为程序从未把该引用存储到任何位置。对象被解除引用，并且其资源被Java虚拟机
104 回收。

2. 调用对象的方法

也可以使用对象引用调用对象的方法。把方法的简单名称附加到对象引用之后，中间使用点号操作符（.）连接。另外，在封闭的括号内提供方法需要的所有实参。如果方法不需要任何实参，则使用空括号。示例如下：

```
objectReference.methodName(argumentList);
```

或者

```
objectReference.methodName();
```

Rectangle类有两个方法：getArea()计算矩形的面积，move()改变矩形的原点。下面是CreateObjectDemo代码中调用这两个方法的部分：

```
System.out.println("Area of rectOne: " + rectOne.getArea());
...
rectTwo.move(40, 72);
```

第一行语句调用rectOne的getArea()方法，并且显示调用结果。第二行语句移动rectTwo，因为move()方法把新值赋给了对象的origin.x和origin.y。

与实例字段一样，对象引用（objectReference）必须引用一个对象。你可以使用变量名称，但是也可以使用返回对象引用的任何表达式。因为new操作符返回对象引用，所以你可以使用new的返回值调用新对象的方法：

```
new Rectangle(100, 50).getArea()
```

表达式new Rectangle(100, 50)返回指向Rectangle对象的对象引用。如上所示，你可以使用点号调用新Rectangle对象的getArea()方法，用以计算新矩形的面积。

一些方法，比如getArea()，返回一个值。对于返回值的方法，你可以在表达式中使用方法调用。可以把返回值赋给变量、使用它作出判断或者控制循环。下面这行代码把getArea()的返回值赋给变量areaOfRectangle：

```
int areaOfRectangle = new Rectangle(100, 50).getArea();
```

105

记住，调用特定对象的方法等同于把消息发送给这个对象。在这个示例中，被调用getArea()方法的对象是构造器返回的那个矩形。

3. 垃圾收集器

一些面向对象的语言要求你跟踪创建的所有对象，并在不再需要它们时显式地销毁它们。显式地管理内存是很令人乏味的，而且容易出错。借助Java平台，你可以创建任意数量的对象（当然会受到系统处理能力的限制），而且不必为销毁它们而操心。Java运行时环境在判断出已不再需要的对象时，会销毁它们。这个过程称为垃圾收集（garbage collection）。

当一个对象已经没有引用指向它时，它就符合垃圾收集的条件了。被保存在变量中的引用常常在变量超出作用域时被销毁。你也可以通过将变量设置为特殊值null来显式地销毁对象引用。记住，程序中可以有多个引用指向相同的对象；一个对象的所有引用都必须被销毁，对象才符合垃圾收集的条件。

Java运行时环境有一个垃圾收集器，它定期释放已经不再被引用的对象使用的内存。当垃圾收集器认为时机恰当时，它会自动地执行回收任务。

4.3 关于类的更多介绍

本节介绍有关类的其他方面的知识，这些知识和你在前面学习的使用对象引用和点号操作符有关。

- 从方法返回值。
- this关键字。
- 类成员和实例成员。
- 访问控制。

4.3.1　从方法返回值

在下面这些情况下，方法将返回到调用它的代码：

- 方法中的所有语句执行完毕。
- 到达return语句。
- 抛出异常（以后介绍）。

106

不管满足哪个条件，方法都会返回。

在方法声明中声明方法的返回类型。在方法体中，使用return语句返回一个值。

所有被声明为void的方法都没有返回值。并不要求方法一定包含return语句，但是也可以包含。在这种情况下，return语句可以用于跳出控制流块，并且退出方法，可以这样简单地使用它：

```
return;
```

如果试图从被声明为void的方法返回值，就会出现编译器错误。

未被声明为void的方法都必须包含带有相应返回值的return语句，比如：

```
return returnValue;
```

返回值的数据类型必须匹配方法声明的返回类型；例如，不能从被声明为返回一个布尔值的方法返回整数值。

下面是4.2节中讨论的Rectangle类中的getArea()方法，该方法返回一个整数：

```
// a method for computing the area of the rectangle
public int getArea() {
  return width * height;
}
```

这个方法返回表达式width * height计算出的整数。

getArea方法返回原始类型。方法也可以返回引用类型。例如，在处理Bicycle对象的程序中，我们可以使用这样的方法：

```
public Bicycle seeWhosFastest(Bicycle myBike,
                Bicycle yourBike, Environment env) {
  Bicycle fastest;
  // code to calculate which bike is faster, given
  // each bike's gear and cadence and given
  // the environment (terrain and wind)
  return fastest;
}
```

107

返回类或者接口

子类和接口将在第5章中讨论。如果这一节对你有一定难度，那么就跳过它，等到读完第5章之后回来再读。

当方法使用一个类名作为返回类型时，比如whosFastest方法这样，那么返回对象的类型类必须是返回类型指定的类或它的子类。假设存在如图4-4所示的类层次结构，其中ImaginaryNumber是java.lang.Number的子类，而java.lang.Number是Object的子类。

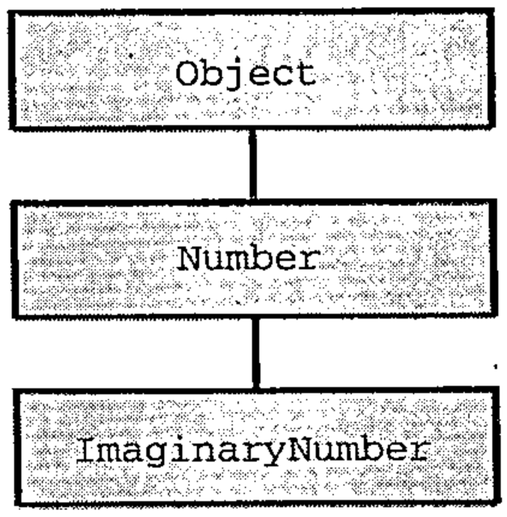


图4-4 ImaginaryNumber的类层次结构

现在假设你的一个方法被声明为返回Number:

```
public Number returnANumber() {
  ...
}
```

returnANumber方法可以返回ImaginaryNumber，但不能返回Object。因为ImaginaryNumber是Number的子类，所以它是一个Number对象。但Object对象不一定是Number对象——它可以是String或其他类型。

可以覆盖方法并且将它定义为返回原始方法的子类，就像下面这样:

```
public ImaginaryNumber returnANumber() {
  ...
}
```

这种技术称为协变返回类型（covariant return type），就是说允许返回类型向和子类相同的方向变化。 108

注意 也可以使用接口名称作为返回类型。在这种情况下，返回的对象必须实现指定的接口。

4.3.2 使用关键字 this

在实例方法或构造器中，this引用当前对象（current object），即被调用的方法或构造器所属的对象。通过使用this关键字，可以在实例方法或构造器中引用当前对象的任何成员。

1. 对字段使用this关键字

使用this关键字的最常见原因是，方法或构造器的参数遮蔽了对象的成员字段。

例如，Point类写成这样:

```
public class Point {
  public int x = 0;
  public int y = 0;

  // constructor
  public Point(int a, int b) {
    x = a;
    y = b;
  }
}
```

但是也可以被编写成这样:

```
public class Point {
  public int x = 0;
  public int y = 0;
```

```
  // constructor
  public Point(int x, int y) {
    this.x = x;
    this.y = y;
  }
}
```

第二个构造器的每个实参都遮蔽对象的一个字段——构造器内的**x**是构造器的第一个实参
109 的局部副本。为了引用Point字段**x**，构造器必须使用**this.x**。

2. 对构造器使用this

在构造器中，还可以使用this关键字调用同一个类中的另一个构造器。这种做法被称为显式构造器调用（**explicit constructor invocation**）。下面是另一个Rectangle类，它与4.2节中的Rectangle类的实现不同：

```
public class Rectangle {
  private int x, y;
  private int width, height;

  public Rectangle() {
    this(0, 0, 0, 0);
  }
  public Rectangle(int width, int height) {
    this(0, 0, width, height);
  }
  public Rectangle(int x, int y, int width, int height) {
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
  }
  ...
}
```

这个类包含一组构造器。每个构造器初始化矩形的一些或全部成员变量。构造器为没有通过实参提供初始值的成员变量提供默认值。例如，无实参构造器使用4个0值调用四实参构造器，两实参构造器使用2个0值调用四实参构造器。像以前一样，编译器根据实参的数量和类型判断应该调用哪个构造器。

如果存在，对另一个构造器的调用必须是构造器中的第一行。

4.3.3 控制对类成员的访问

访问级别修饰符决定其他类是否可以使用特定字段或者调用特定方法。访问控制有两个级别：

- 最高级别——public，或者包私有（**package-private**）（无显式修饰符）。
- 成员级别——public、private、protected或者包私有（无显式修饰符）。

110

可以使用修饰符public声明类，在这种情况下，这个类对任何位置的所有类都是可见的。如果类没有修饰符（这是默认情况，也称为包私有），那么它只在它自己的包内是可见的（包是相关类的命名组，将在第7章中介绍）。

在成员级别，也可以像最高级别一样使用修饰符public或者无修饰符（包私有），并且具有相同含义。对于成员，有两个附加的访问修饰符：private和protected。修饰符private指定成员只在它自己的类中是可以访问的。修饰符protected指定成员只在它自己的包中是可以访问的（和包私有一样），并且可以被另一个包中其类的子类访问。

表4-1列出每个修饰符允许的成员访问方式。

表4-1 访问级别

修饰符	类	包	子类	所有类
public	是	是	是	是
protected	是	是	是	否
无修饰符	是	是	否	否
private	是	否	否	否

第一列（类）指定类本身是否可以访问此访问修饰符定义的成员。可以看到，类总是能够访问它自己的成员。第二列（包）指定与这个类在相同包中的其他类（无论它们的派生关系如何）是否可以访问此成员。第三列（子类）指定类的子类（在这个包之外声明）是否可以访问此成员。第四列（所有类）指定外界所有类是否可以访问此成员。

访问级别在两方面有影响。首先，当你使用来自另一个来源的类（比如Java平台中的类）时，访问级别决定你的类可以使用这些类中的哪些成员。其次，在你编写类时，需要决定类中的每个成员变量和方法应该具有什么访问级别。

我们来看一组类，看看访问级别如何影响可见性。图4-5显示了组成这个例子的4个类以及它们的相互关系。 111

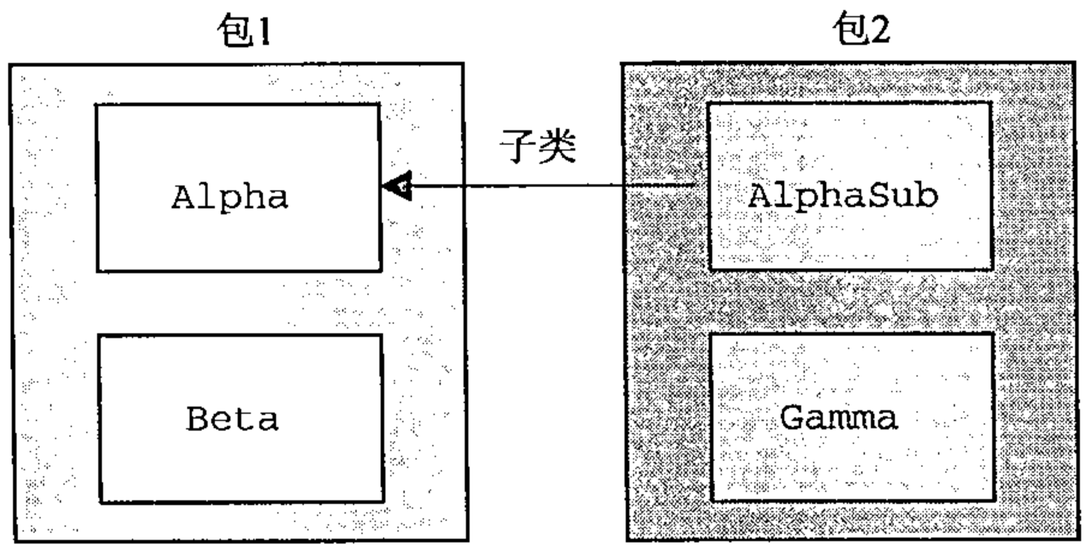


图4-5 类和包的示例，用于演示访问级别

表4-2显示对于可以应用的每个访问修饰符，Alpha类的成员在哪里是可见的。

表4-2 可见性

修饰符	Alpha	Beta	Alphasub	Gamma
public	是	是	是	是
protected	是	是	是	否
无修饰符	是	是	否	否
private	是	否	否	否

关于选择访问级别的提示 如果其他程序员使用你的类，你会希望确保他们不会误用你的类。在这方面，访问级别可以帮助你。

- 对特定成员使用最严格的访问级别。如果没有充分的理由，就使用private级别。
- 除了常量以外，避免使用public字段。（本书中的很多示例都使用public字段。这可以帮助简明地演示某些重点，但是不建议用于生产代码。）公共字段倾向于把你引向特定实现，并且会限制改动代码时的灵活性。

4.3.4　了解实例成员和类成员

本节讨论使用static关键字创建属于类（而不是属于类的实例）的字段和方法。

1. 类变量

从同一个类蓝图创建数个对象时，它们都具有属于它们自己的实例变量（instance varible）的独特副本。在Bicycle类的例子中，实例变量是cadence、gear和speed。每个Bicycle对
112 象都具有属于自己的这些变量的值，它们存储在不同的内存位置中。

有时候，你会希望变量对所有对象是公共的。这是通过修饰符static实现的。在声明中使用static修饰符的字段被称为静态字段（static field）或者类变量（class variable）。它们和类相关，而不和任何对象相关。类的每个实例共享类变量，它位于内存的一个固定位置。任何对象都可以改变类变量的值，但是在不创建类的实例的情况下，也可以操作类变量。

例如，假设你创建若干Bicycle对象，并且把序号赋给每个对象，第一个对象的序号从1开始。这个ID数字对每个对象是唯一的，因此它是实例变量。同时，你需要一个字段用于跟踪已经创建了多少个Bicycle对象，以便能知道把什么ID值赋给下一个对象。这样的字段和任何独立对象都不相关，但是和类的整体相关。在这种情况下，就需要一个类变量numberOfBicycles，如下所示：

```
public class Bicycle{

  private int cadence;
  private int gear;
  private int speed;

  // add an instance variable for the object ID
  private int id;

  // add a class variable for the number
  // of Bicycle objects instantiated
  private static int numberOfBicycles = 0;
    ...
}
```

通过类名称本身引用类变量，例如：

```
Bicycle.numberOfBicycles
```

这就清楚地表示它们是类变量。

注意　也可以使用对象引用来引用静态字段，比如：

```
myBike.numberOfBicycles
```

但是不建议这样做，因为这没有清楚地表示出它们是类变量。

---

113 可以使用Bicycle构造器设置id实例变量，并且递增numberOfBicycles类变量的值：

```
public class Bicycle{

  private int cadence;
  private int gear;
  private int speed;
  private int id;
  private static int numberOfBicycles = 0;
```

```
public Bicycle(int startCadence,
               int startSpeed, int startGear){
  gear = startGear;
  cadence = startCadence;
  speed = startSpeed;

  // increment number of Bicycles and assign ID number
  id = ++numberOfBicycles;
}

// new method to return the ID instance variable
  public int getID() {
    return id;
  }
  ...
}
```

2. 类方法

和静态变量一样，Java编程语言也支持静态方法。静态方法在它们的声明中使用static修饰符，应该使用类名称调用静态方法，而无需创建类的实例，比如：

```
ClassName.methodName (args)
```

注意 也可以使用对象引用来引用静态方法，比如：

```
instanceName.methodName (args)
```

但是不建议这样做，因为这没有清楚地表示出它们是类方法。

静态方法经常用于访问静态字段。例如，我们可以为Bicycle类添加一个静态方法，用于访问numberOfBicycles静态字段：

```
public static int getNumberOfBicycles() {
  return numberOfBicycles;
}
```

114

并非实例、类变量和方法的所有组合都被允许使用：

- 实例方法可以直接访问实例变量和实例方法。
- 实例方法可以直接访问类变量和类方法。
- 类方法可以直接访问类变量和类方法。
- 类方法不能直接访问实例变量或实例方法——它们必须使用对象引用。另外，类方法不能使用this关键字，因为没有实例可供this引用。

3. 常量

static修饰符和final修饰符组合在一起也可以用于定义常量。

final修饰符指定这个字段的值不能被改动。

例如，下面的变量声明定义一个名为PI的常量，其值是圆周率（圆的周长与其直径的比值）的近似值：

```
static final double PI = 3.141592653589793;
```

通过这种方式定义的常量不能被重新赋值，如果你的程序试图这样做，就会导致编译时错误。按照惯例，常量值的名称使用大写字母拼写。如果名称由一个以上的单词构成，那么使用下划线（_）分隔各个单词。

注意　如果原始类型或者字符串被定义为常量，并且这个值在编译时是已知的，那么编译器就会使用这个值替换代码中出现的每个该常量名称。这被称为编译时常量（compile-time constant）。如果常量值在外部被改变了（例如，如果规定圆周率应该是3.975），就需要重新编译使用这个常量的所有类，以便使之具有当前值。

4. Bicycle类

115 通过本节的修改，现在Bicycle类成了下面这样：

```
public class Bicycle{
  private int cadence;
  private int gear;
  private int speed;
  private int id;
  private static int numberOfBicycles = 0;

  public Bicycle(int startCadence,
                 int startSpeed, int startGear) {
    gear = startGear;
    cadence = startCadence;
    speed = startSpeed;
    id = ++numberOfBicycles;
  }

  public int getID() {
    return id;
  }

public static int getNumberOfBicycles() {
  return numberOfBicycles;
}

public int getCadence(){
  return cadence;
}

public void setCadence(int newValue){
  cadence = newValue;
}

public int getGear(){
return gear;
}

public void setGear(int newValue){
  gear = newValue;
}

public int getspeed(){
  return speed;
}

public void applyBrake(int decrement){
  speed -= decrement;
}

public void speedUp(int increment){
  speed += increment;
}
```

```
}
```

116

4.3.5 初始化字段

你已经知道，可以在字段的声明中为字段提供初始值：

```
public class BedAndBreakfast {
  public static int capacity = 10;      // initialize to 10
  private boolean full = false;         // initialize to false
}
```

当初始值可用，并且初始化过程可以放在一行代码中时，这种方式很不错。但是，因为初始化的这种形式很简单，所以有其局限性。如果初始化需要某些逻辑（例如，错误处理或者使用for循环填充复杂数组），那么简单的赋值就不够用了。可以在构造器中初始化实例变量，在这里可以使用错误处理或者其他逻辑。为了给类变量提供相同功能，Java编程语言包含了静态初始化块（static initialization block）。

注意 不必在类定义的开头声明字段，虽然这是最常见的编程习惯。只需在使用字段之前声明和初始化它们即可。

---

1. 静态初始化块

静态初始化块（static initialization block）被放在花括号中，前面是static关键字。下面是一个例子：

```
static {
  // whatever code is needed for initialization goes here
}
```

类可以具有任意数量的静态初始化块，并且它们可以出现在类体内的任何位置。运行时系统保证按照静态初始化块在源代码中出现的顺序执行它们。

下面是静态块的一个替换方案——你可以编写私有静态方法：

```
class Whatever {
  public static varType myVar = initializeClassVariable();
  private static varType initializeClassVariable() {
    // initialization code goes here
  }
}
```

私有静态方法的优势是，如果你需要重新初始化类变量，可以在以后重用它们。

117

2. 初始化实例成员

通常把初始化实例变量的代码放在构造器中。对于使用构造器初始化实例变量，有两个替换方案：初始化器块和final方法。

用于实例变量的初始化器块和静态初始化块差不多，只是前者没有static关键字：

```
{
  // whatever code is needed for initialization goes here
}
```

Java编译器把初始化器块复制到每个构造器中。因此，这种方式可以用于在多个构造器之间共享一个代码块。

不能在子类中覆盖final方法（final method）。这在第5章中讨论。下面是使用final方法初始化实例变量的一个例子：

```
class Whatever {
```

```
  private varType myVar = initializeInstanceVariable();
  protected final varType initializeInstanceVariable() {
    // initialization code goes here
  }
}
```

如果子类希望重用初始化方法，这种方式就特别有用。这个方法是final方法，因为在实例初始化过程中调用非final方法可能导致问题。Joshua Bloch在*Effective Java*[1]一书中详细地讨论了这个问题。

4.3.6　创建和使用类和对象的小结

类声明可以命名类，并且把类体括在花括号中。类名称前面可以使用修饰符。类体包含类的字段、方法和构造器。类使用字段包含状态信息，使用方法实现行为。构造器使用类的名称，它们对类的新实例进行初始化，构造器看上去就像没有返回类型的方法。

通过相同的方式控制对类和成员的访问：在它们的声明中使用访问修饰符，比如public。

通过在成员声明中使用static关键字指定类变量或者类方法。未被声明为static的成员是隐式的实例成员。类变量被类的所有实例共享，可以通过类名称或实例引用访问类变量。类
118 的实例获得属于它们自己的每个实例变量的副本，必须通过实例引用访问它们。

通过使用new操作符和构造器从类创建对象。new操作符返回创建好的对象的引用。可以把引用赋给变量，或者直接使用它们。

对于可以从所属的类之外的代码访问的实例变量和方法，可以使用限定名称引用它们。实例变量的限定名称就像下面这样：

```
objectReference.variableName
```

方法的限定名称就像下面这样：

```
objectReference.methodName(argumentList)
```

或者

```
objectReference.methodName()
```

垃圾收集器自动清除不再使用的对象。如果程序不再保存指向对象的引用，那么这个对象就不再被使用。通过把保存引用的变量设置为null，可以显式地清除引用。

问题和练习：类

问题

考虑下面的类：

```
public class IdentifyMyParts {
  public static int x = 7;
  public int y = 3;
}
```

a. 类变量是什么？

b. 实例变量是什么？

119 c. 下面代码的输出是什么？

```
IdentifyMyParts a = new IdentifyMyParts();
IdentifyMyParts b = new IdentifyMyParts();
a.y = 5;
b.y = 6;
```

---

1. docs/books/effective

```
IdentifyMyParts.x = 1;
b.x = 2;
System.out.println("a.y = " + a.y);
System.out.println("b.y = " + b.y);
System.out.println("IdentifyMyParts.x = " + a.x);
System.out.println("b.x = " + b.x);
```

练习

1. 编写一个类，它的实例代表一副扑克牌中的一张牌。扑克牌有两个独特的属性：点数和花色。请保存好你的解决方案，因为在“问题和练习：枚举类型”部分中将要求你重新编写它。

提示 可以使用assert语句检查你的赋值。编写：

```
assert (boolean expression to test);
```

如果布尔表达式为false，就会出现错误消息。例如，

```
assert toString(ACE) == "Ace";
```

应该返回true，所以不会出现错误消息。

如果使用assert语句，就必须带ea标志运行程序：

```
java -ea YourProgram.class
```

---

2. 编写一个类，它的实例代表**完整**的一副牌。你也应该保存好这个解决方案。
3. 编写一个小程序来测试你的整副牌类和单张牌类。这个程序应该可以简单地创建整副牌并且显示其中的单张牌。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/javaOO/QandE/creating-answers.html 120

问题和练习：对象

问题

1. 下面的程序错在哪里？

```
public class SomethingIsWrong {
  public static void main(String[] args) {
    Rectangle myRect;
    myRect.width = 40;
    myRect.height = 50;
    System.out.println("myRect's area is " + myRect.area());
  }
}
```

2. 下面的代码创建一个Point对象和一个Rectangle对象。代码执行完之后，有多少个引用指向这些对象？哪个对象符合垃圾收集的条件？

```
...
Point point = new Point(2,4);
Rectangle rectangle = new Rectangle(point, 20, 20);
point = null;
...
```

3. 程序如何销毁由它创建的对象？

练习

1. 改正“问题1”中给出的程序SomethingIsWrong。

2. 下面的类名为NumberHolder[1]，编写代码来创建这个类的一个实例，初始化它的两个成员变量，然后显示每个成员变量的值：

```
public class NumberHolder {
  public int anInt;
  public float aFloat;
}
```

答案

可以在以下位置找到“问题”和“练习”的答案：

121 tutorial/java/javaOO/QandE/objects-answers.html

4.4 嵌套类

Java编程语言允许你在一个类中定义另一个类。这样的类被称为嵌套类（nested class）：

```
class OuterClass {
  ...
  class NestedClass {
    ...
  }
}
```

嵌套类是包含它的外围类的成员，因此嵌套类可以访问外围类的其他成员，即使这些成员被声明为私有的。作为OuterClass类的成员，嵌套类可以被声明为private、public、protected或者包私有。（前面讲过，外部类只能声明为public或者包私有。）

术语 嵌套类分为两种类型：静态和非静态。声明为static的嵌套类被称为静态嵌套类（static nested class）。非静态嵌套类被称为内部类（inner class）。

```
class OuterClass {
  ...
  static class StaticNestedClass {
    ...
  }
  class InnerClass {
    ...
  }
}
```

4.4.1 为什么使用嵌套类

使用嵌套类出于几个原因，包括：

- 这是对只在一个位置使用的类进行逻辑分组的一种途径。
- 增强封装。
- 嵌套类可以提高代码的可读性和可维护性。

类的逻辑分组——如果一个类只对另一个类有用，那么把这个类嵌入另一个类并且将两个
122 类放在一起是符合逻辑的。嵌套这样的“辅助类”会使它们的包更加合理。

增强封装——考虑两个顶级类A和B，这里，B需要访问被声明为private的A的成员。通过把B类隐藏在A类中，A的成员就可以声明为private，并且B能够访问它们。另外，可以对

1. tutorial/java/javaOO/QandE/NumberHolder.java

外部的所有类隐藏B本身。

可读性和可维护性更强的代码——把小型类嵌入顶级类中，可以使代码更加靠近被使用的位置。

4.4.2 静态嵌套类

与类方法和类变量一样，静态嵌套类和它的外部类相关联。和静态类方法一样，静态嵌套类不能直接引用包含它的类中定义的实例变量和方法——静态嵌套类只能通过对象引用使用它们。

注意 静态嵌套类和包含它的类（以及其他类）的实例成员进行交互，就像所有其他顶级类一样。实际上，静态嵌套类在行为上是一个顶级类，只是为了便于打包嵌套到了另一个顶级类中。

使用外部类的名称访问静态嵌套类：

```
OuterClass.StaticNestedClass
```

例如，为了创建静态嵌套类的对象，使用如下语法：

```
OuterClass.StaticNestedClass nestedObject =
                    new OuterClass.StaticNestedClass();
```

4.4.3 内部类

与实例方法和实例变量一样，内部类与包含它的类的一个实例相关联，内部类可以直接访问此对象的变量和方法。另外，因为内部类与实例相关联，所以它不能定义任何静态成员。

内部类的实例存在于外部类的实例之内。考虑下面的类：

123

```
class OuterClass {
  ...
  class InnerClass {
    ...
  }
}
```

InnerClass的实例只存在于OuterClass的实例之内，并且可以直接访问包含它的实例的方法和字段。图4-6演示了这一概念。

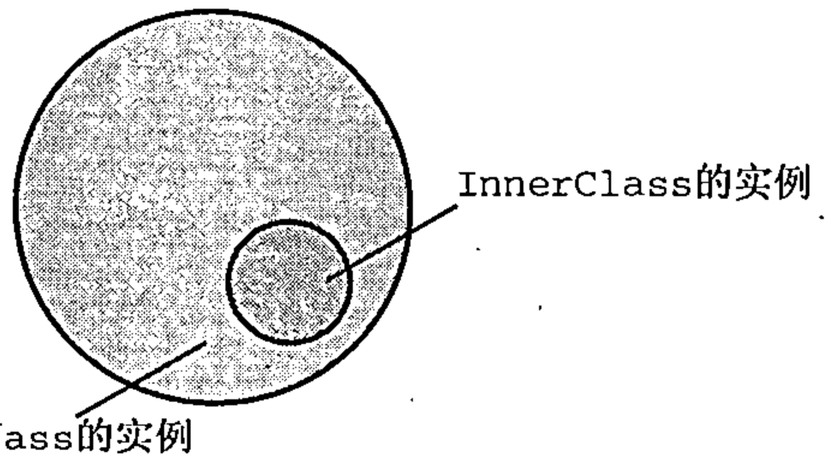


图4-6 InnerClass存在于OuterClass的实例内

要实例化内部类，必须首先实例化外部类。然后使用如下语法在外部对象内创建内部对象：

```
OuterClass.InnerClass innerObject =
                    outerObject.new InnerClass();
```

另外，有两种特殊类型的内部类：局部类和匿名类（也称为匿名内部类）。下一节简要讨论这两种内部类。

4.4.4　内部类示例

为了了解内部类的使用，我们考虑一个简单的整数栈（stack），栈是程序设计中常见的数据结构，它的名称很形象——栈就像一“叠”盘子。把一个盘子放到一叠盘子中时，你把它放在顶上；取出一个盘子时，从顶上取出。这种操作的首字母缩写是LIFO（last in，first out，后进先出）。上面的盘子常常被放进和拿出，而底部的盘子有可能留在原处相当长时间。

124

下面的StackOfInts类实现一个数组。当你添加一个整数（称为“压入”）时，它进入第一个可用空元素。当你删除一个整数（称为“弹出”）时，就删除数组中的最后一个整数。

下面的StackOfInts（它是一个应用程序）由以下部分构成：

- 外部类StackOfInts，它包含把整数压入栈、把整数弹出栈和检测栈是否为空的方法。
- StepThrough内部类和标准Java迭代器（iterator）类似。迭代器用于遍历数据结构，典型情况下，迭代器具有测试最后元素、获得当前元素和移动到下一个元素的方法。
- main方法实例化StackOfInts数组（stackOne），用整数（0、2、4等）填充该数组，然后实例化StepThrough对象（iterator），并且使用它输出stackOne的内容。

```
public class StackOfInts {

  private int[] stack;
  private int next = 0; // index of last item in stack + 1

  public StackOfInts(int size) {
    // create an array large enough to hold the stack
    stack = new int[size];
  }

  public void push(int on) {
    if (next < stack.length)
      stack[next++] = on;
  }
  public boolean isEmpty() {
  return (next == 0);
  }

  public int pop(){
    if (!isEmpty())
      return stack[--next]; // top item on stack
    else
      return 0;
  }

  public int getStackSize() {
    return next;
  }
  private class StepThrough {
    // start stepping through at i=0
    private int i = 0;

    // increment index
    public void increment() {
      if ( i < stack.length)
        i++;
    }

    // retrieve current element
    public int current() {
      return stack[i];
```

125

```
  }

  // last element on stack?
  public boolean isLast(){
    if (i == getStackSize() - 1)
      return true;
    else
      return false;
  }
}

public StepThrough stepThrough() {
  return new StepThrough();
}

public static void main(String[] args) {

  // instantiate outer class as "stackOne"
  StackOfInts stackOne = new StackOfInts(15);

  // populate stackOne
  for (int j = 0 ; j < 15 ; j++) {
    stackOne.push(2*j);
  }

  // instantiate inner class as "iterator"
  StepThrough iterator = stackOne.stepThrough();

  // print out stackOne[i], one per line
  while(!iterator.isLast()) {
    System.out.print(iterator.current() + " ");
    iterator.increment();
  }
  System.out.println();
 }
}
```

程序的输出为：

```
0 2 4 6 8 10 12 14 16 18 20 22 24 26
```

126

注意，StepThrough类直接引用StackOfInts的Stack实例变量。

内部类主要用于实现辅助类，比如前面例子中显示的辅助类。如果你计划处理用户界面事件，就需要了解内部类的使用，因为事件处理机制广泛地使用了内部类。

1. 局部内部类和匿名内部类

有两种其他类型的内部类。可以在方法体中声明内部类。这样的类称为*局部内部类*（local inner class）。还可以在方法体中不命名地声明内部类。这样的类称为*匿名内部类*（anonymous inner class）。在高级Java编程中会遇到这些类。

2. 修饰符

可以把用于外部类的其他成员的修饰符用于内部类。例如，就像对其他类成员那样，可以使用访问说明符（private、public和protected）限制对内部类的访问。

4.4.5 嵌套类小结

在另一个类中定义的类被称为嵌套类。与类的其他成员一样，嵌套类可以被声明为静态或非静态的。非静态的嵌套类被称为内部类。内部类的实例只能在包含它的类的实例内存在，而且内部类的实例可以访问包含它的类的成员，甚至那些被声明为私有的成员。

表4-3列出了嵌套类的类型。

表4-3　嵌套类的类型

类　型	作用域	内　部
静态嵌套类	成员	否
内部[非静态]类	成员	是
局部类	局部	是
匿名类	只在定义它的位置	是

127

问题和练习：嵌套类

问题

1. 下面的程序Problem.java[1]不能通过编译。需要进行什么修改才能使它通过编译？为什么？

```
public class Problem {
  String s;
  static class Inner {
    void testMethod() {
       s = "Set from Inner";
    }
  }
}
```

2. 使用Box类（在javax.swing包中）的Java API文档帮助你回答下列问题：
   a. Box类定义的静态嵌套类是什么？
   b. Box类定义的内部类是什么？
   c. Box类的内部类的超类是什么？
   d. 可以在任何类中使用的Box类的嵌套类有哪些？
   e. 如何创建Box的Filler类的实例？

练习

编译下面的Class1.java[2]，并且运行Class1。输出是什么？

```
public class Class1 {
    protected InnerClass1 ic;

    public Class1() {
  ic = new InnerClass1();
    }

    public void displayStrings() {
System.out.println(ic.getString() + ".");
System.out.println(ic.getAnotherString() + ".");
  }

  static public void main(String[] args) {
      Class1 c1 = new Class1();
c1.displayStrings();
  }
```

---

1. tutorial/java/javaOO/QandE/Problem.java
2. tutorial/java/javaOO/QandE/Class1.java

```
    protected class InnerClass1 {
  public String getString() {
      return "InnerClass1: getString invoked";
  }

  public String getAnotherString() {
      return "InnerClass1: getAnotherString invoked";
  }
    }
}
```

答案

可以在以下位置找到“问题”和“练习”的答案：

```
tutorial/java/javaOO/QandE/nested-answers.html
```

4.5 枚举类型

枚举类型（enum type）是其字段由一组不变的常量构成的一种类型。常见的例子包括指南针方向（值为NORTH、SOUTH、EAST和WEST）和一周中的星期一到星期日。

因为是常量，所以枚举类型的字段名称使用大写字母。

在Java编程语言中，使用enum关键字定义枚举类型。例如，可以像下面这样指定表示一周的枚举类型：

128

```
public enum Day {
  SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
  THURSDAY, FRIDAY, SATURDAY
}
```

如果需要表示一组不变常量，就应该使用枚举类型。这包括自然界的枚举类型，比如太阳系中的行星，以及你在编译时知道的所有可能值的数据集合，例如菜单的选项、命令行标志等。

下面是一段代码，它演示如何使用前面定义的Day枚举类型：

```
public class EnumTest {
  Day day;

  public EnumTest(Day day) {
    this.day = day;
  }

  public void tellItLikeItIs() {
    switch (day) {
      case MONDAY: System.out.println("Mondays are bad.");
                   break;

      case FRIDAY: System.out.println("Fridays are better.");
                   break;

      case SATURDAY:
      case SUNDAY: System.out.println("Weekends are best.");
                   break;

      default: System.out.println("Midweek days are so-so.");
               break;
    }
  }
```

```
public static void main(String[] args) {
  EnumTest firstDay = new EnumTest(Day.MONDAY);
  firstDay.tellItLikeItIs();
  EnumTest thirdDay = new EnumTest(Day.WEDNESDAY);
  thirdDay.tellItLikeItIs();
  EnumTest fifthDay = new EnumTest(Day.FRIDAY);
  fifthDay.tellItLikeItIs();
  EnumTest sixthDay = new EnumTest(Day.SATURDAY);
  sixthDay.tellItLikeItIs();
  EnumTest seventhDay = new EnumTest(Day.SUNDAY);
  seventhDay.tellItLikeItIs();
 }
}
```

129

输出为：

```
Mondays are bad.
Midweek days are so-so.
Fridays are better.
Weekends are best.
Weekends are best.
```

和其他语言的对应特性相比，Java编程语言的枚举类型的功能要强大得多。枚举声明定义一个类（称为枚举类型）。枚举类的体可以包含方法和其他字段。当创建枚举类型时，编译器自动添加一些专用方法。例如，静态的values方法返回包含枚举类型所有值的数组，其中的值按照值声明的顺序排序。这个方法经常和for-each结构组合在一起，用于迭代枚举类型的值。例如，Planet类示例中的下列代码迭代太阳系中的所有行星。

```
for (Planet p : Planet.values()) {
  System.out.printf("Your weight on %s is %f%n",
                    p, p.surfaceWeight(mass));
}
```

注意　所有枚举类型都隐式地扩展java.lang.Enum。因为Java不支持多重继承，所以枚举类型不能扩展任何其他类。

在下面的例子中，Planet是表示太阳系中行星的枚举类型。它们的定义带有不变的质量和半径属性。

每个枚举常量的声明都带有质量和半径参数值。创建常量时，这些值被传递给构造器。Java要求在定义任何其他字段或者方法之前定义常量字段。另外，当具有字段和方法时，枚举常量的清单以分号结束。

注意　枚举类型的构造器必须是包私有或者私有访问的。构造器自动创建在枚举体的开头定义的常量。你自己不能调用枚举构造器。

除了属性和构造器之外，Planet还具有方法，允许你获得每个行星上的物体的表面引力和重量。下面是一个示例程序，它接收你在地球上的重量（可以使用任何单位），计算并且输出你在所有行星上的重量（以相同单位）：

130

```
public enum Planet {
  MERCURY (3.303e+23, 2.4397e6),
  VENUS   (4.869e+24, 6.0518e6),
  EARTH   (5.976e+24, 6.37814e6),
```

```
MARS    (6.421e+23, 3.3972e6),
JUPITER (1.9e+27, 7.1492e7),
SATURN  (5.688e+26, 6.0268e7),
URANUS  (8.686e+25, 2.5559e7),
NEPTUNE (1.024e+26, 2.4746e7),
PLUTO   (1.27e+22, 1.137e6);

private final double mass;   // in kilograms
private final double radius; // in meters
Planet(double mass, double radius) {
  this.mass = mass;
  this.radius = radius;
}
private double mass()   { return mass; }
private double radius() { return radius; }

// universal gravitational constant  (m3 kg-1 s-2)
public static final double G = 6.67300E-11;

double surfaceGravity() {
  return G * mass / (radius * radius);
}
double surfaceWeight(double otherMass) {
  return otherMass * surfaceGravity();
}
public static void main(String[] args) {
  double earthWeight = Double.parseDouble(args[0]);
  double mass = earthWeight/EARTH.surfaceGravity();
  for (Planet p : Planet.values())
    System.out.printf("Your weight on %s is %f%n",
                      p, p.surfaceWeight(mass));
}
}
```

如果你从命令行使用实参175运行Planet.class，就会得到如下输出：

```
$ java Planet 175
Your weight on MERCURY is 66.107583
Your weight on VENUS is 158.374842
Your weight on EARTH is 175.000000
Your weight on MARS is 66.279007
Your weight on JUPITER is 442.847567
Your weight on SATURN is 186.552719
Your weight on URANUS is 158.397260
Your weight on NEPTUNE is 199.207413
Your weight on PLUTO is 11.703031
```

131

问题和练习：枚举类型

练习

1. 重新编写你在“问题和练习：类”部分中编写的Card类，让它使用枚举类型表示扑克牌的点数和花色。
2. 重新编写Deck类。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/javaOO/QandE/enum-answers.html

4.6　注解

注解（annotation）提供和程序有关、但不是程序本身组成部分的数据。注解对它们所注解的代码的执行没有直接影响。

注解有很多用处，其中包括：

- **编译器使用的信息**——编译器可以使用注解检测错误或者抑制警告。
- **编译时和部署时的处理**——软件工具可以处理注解信息来生成代码、XML文件等。
- **运行时处理**——一些注解可以用于运行时的检查。

注解可以应用于程序中类、字段、方法和其他程序元素的声明。

注解首先出现，（按照惯例）经常有自己的代码行，并且可以包含带有命名或者未命名值的元素，例如：

```
@Author(
  name = "Benjamin Franklin",
  date = "3/27/2003"
)
class MyClass() { }
```

或者

```
@SuppressWarnings(value = "unchecked")
void myMethod() { }
```

132

如果只有一个名为“value”的元素，就可以省略名称，比如：

```
@SuppressWarnings("unchecked")
void myMethod() { }
```

另外，如果注解没有元素，就可以省略括号，比如：

```
@Override
void mySuperMethod() { }
```

4.6.1　文档

很多注解能够替换过去在代码中使用注释表达的内容。

假设一个软件开发团队传统上在每个类体的开头使用注释提供重要信息：

```
public class Generation3List extends Generation2List {

  // Author: John Doe
  // Date: 3/17/2002
  // Current revision: 6
  // Last modified: 4/12/2004
  // By: Jane Doe
  // Reviewers: Alice, Bill, Cindy

  // class code goes here
}
```

为了使用注解添加相同的元数据，你必须首先定义注解类型（annotation type）。进行这一操作的语法如下：

```
@interface ClassPreamble {
  String author();
  String date();
  int currentRevision() default 1;
  String lastModified() default "N/A";
  String lastModifiedBy() default "N/A";
```

```
  String[] reviewers();  // Note use of array
}
```

注解类型的定义看上去就像接口的定义，其中interface关键字的前面加上了@字符（@=“AT”，即Annotation Type的首字母）。实际上，注解类型是接口（interface）的一种形式，接口将在后面的章节中讲解，目前你不需要了解接口的知识。 133

上面的注解体定义包含注解类型元素（annotation type element）声明，看上去很像方法。注意，它们可以定义可选的默认值。

定义了注解类型之后，就可以使用这种类型的注解，填写其中的值，如下：

```
@ClassPreamble (
  author = "John Doe",
  date = "3/17/2002",
  currentRevision = 6,
  lastModified = "4/12/2004",
  lastModifiedBy = "Jane Doe"
  reviewers = {"Alice", "Bob", "Cindy"} // Note array notation
)
public class Generation3List extends Generation2List {

  // class code goes here

}
```

注意 为了使@ClassPreamble中的信息出现在Javadoc生成的文档中，你必须使用@Documented注解@ClassPreamble定义本身：

```
import java.lang.annotation.*; // import this to use @Documented
@Documented
@interface ClassPreamble {

  // Annotation element definitions

}
```

4.6.2 编译器使用的注解

语言规范本身预定义了3种注解类型：@Deprecated、@Override和@SuppressWarnings。

@Deprecated[1]注解表示被标记的元素是已废弃（deprecated）的，并且不应该再使用。当程序使用带有@Deprecated注解的方法、类或者字段时，编译器就会生成警告。元素被废弃时， 134
还应该使用Javadoc的@deprecated标记进行文档标识，如下面的例子所示。在Javadoc注释和注解中都使用“@”符号并不是巧合——它们在概念上是相关的。另外，注意Javadoc标记以小写“d”开头，而注解以大写“D”开头。

```
// Javadoc comment follows
 /**
  * @deprecated
  * explanation of why it was deprecated
  */
 @Deprecated
 static void deprecatedMethod() { }
}
```

1. docs/api/java/lang/Deprecated.html

@Override[1]注解通知编译器，这个元素要覆盖超类中声明的元素。（覆盖方法将在第5章中讨论。）

```
// mark method as a superclass method
// that has been overridden
@Override
int overriddenMethod() { }
```

虽然在覆盖方法时，不严格要求使用这一注解，但是它能帮助你防止发生错误。如果被标记为@Override的方法没有正确地覆盖其超类中的方法，编译器就会生成一个错误。

@SuppressWarnings[2]注解通知编译器，抑制在其他情况下会产生的特定警告。在下面的例子中，使用了已废弃的方法，通常编译器会生成警告。但是由于这个例子中使用了该注解，警告被抑制了。

```
// use a deprecated method and tell
// compiler not to generate a warning
@SuppressWarnings("deprecation")
void useDeprecatedMethod() {
  objectOne.deprecatedMethod(); // deprecation warning
                                // suppressed
}
```

Java语言规范（Java Language Specification）列出了编译器警告的两个类别："废弃"（deprecation）和"未检查"（unchecked）。当使用在泛型（在第6章中讨论）出现之前编写的遗留代码时，可能出现"未检查"警告。为了抑制多种类别的警告，使用如下语法：

135

```
@SuppressWarnings({"unchecked", "deprecation"})
```

4.6.3　注解处理

注解的更高级应用包括编写注解处理器（annotation processor），它能够阅读Java程序并且根据程序的注解进行操作。例如，注解处理器可以生成辅助源代码，使程序员不必创建总是遵循可预期模式的样板代码。为了完成这一任务，JDK 5.0中包含一种注解处理工具，称为apt。在JDK 6.0中，apt的功能是Java编译器的标准组成部分。

为了在运行时可以利用注解信息，必须使用@Retention(RetentionPolicy.RUNTIME)注解注解类型本身，如下：

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@interface AnnotationForRuntime {

  // Elements that give information
  // for runtime processing
}
```

问题和练习：注解

问题

1. 下面的接口有什么错误？

```
public interface House {
  @Deprecated
```

---

1. docs/api/java/lang/Override.html
2. docs/api/java/lang/SuppressWarnings.html

```
  void open();
  void openFrontDoor();
  void openBackDoor();
}
```

136

2. 编译下面这个程序：

```
interface Closable {
  void close();
}

class File implements Closable {
  @Override
  public void close() {
    // ... close this file...
  }
}
```

结果是什么？为什么会产生这样的结果？

3. 考虑“问题1”中显示的House接口的下列实现：

```
public class MyHouse implements House {
  public void open() {}
  public void openFrontDoor() {}
  public void openBackDoor() {}
}
```

如果编译这个程序，编译器会指出open（在接口中）已被废弃了。怎么做才能抑制这个警告？

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/javaOO/QandE/annotations-answers.html

137

第5章 接口和继承

在前一章中，你已经见到了接口实现的示例。在这一章中，你将更多地了解接口——接口用于什么目的、为什么要编写接口以及如何编写接口。

本章还介绍从一个类派生另一个类的途径，也就是子类（subclass）如何从超类（superclass）继承字段和方法。你将了解所有类都是从Object类派生出来的，以及如何修改子类从超类中继承的方法。本章还将介绍类似于接口的抽象（abstract）类。

5.1 接口

在软件开发中的很多情况下，不同的程序员团队之间遵守详细说明软件如何交互的“契约”很重要。在完全不了解其他团队如何编写代码的情况下，每个团队都应该能够编写他们自己的代码。总的来说，接口（interface）就是这种契约。

例如，想象一下未来社会，那时候计算机控制的机器人汽车在城市的街道上运送乘客，无需人类进行操作。汽车生产厂商编写软件（当然是Java）操纵汽车——停止、启动、加速、左转等。另一个厂商——电子导航设备生产厂商——生产计算机系统，计算机系统获得全球定位卫星（Global Positioning Satellite，GPS）发出的定位数据和有关交通情况的无线传输数据，并且使用这些信息驾驶汽车。

汽车生产厂商必须发布工业标准的接口，详细说明可以调用什么方法来开动汽车（所有生产厂商制造的所有汽车）。然后，导航设备生产厂商编写软件，调用接口中描述的方法来驾驶汽车。任何一个厂商都不需要了解另一个厂商如何实现他们的软件。实际上，每个厂商都认为其软件是高度专有的，并且保留在任何时候修改其软件的权利，只要继续遵守已发布的接口即可。

5.1.1 Java中的接口

在Java编程语言中，接口（interface）是类似于类的一种引用类型，它只能包含常量、方法签名和嵌套类型，其中没有方法体。接口不能被实例化——它们只能被类实现（implement）或者被其他接口扩展（extend）。（在本章后面讲解扩展。）

定义接口和创建新类相似：

```
public interface OperateCar {

  // constant declarations, if any

  // method signatures
  int turn(Direction direction, // An enum with values
                                // RIGHT, LEFT
  double radius, double startSpeed, double endSpeed);
  int changeLanes(Direction direction,
                  double startSpeed, double endSpeed);
  int signalTurn(Direction direction, boolean signalOn);
```

```
    int getRadarFront(double distanceToCar, double speedOfCar);
    int getRadarRear(double distanceToCar, double speedOfCar);
      ...
    // more method signatures
}
```

注意，方法签名不带花括号，并且以分号结束。

为了使用接口，你需要编写实现接口的类。当可实例化的类实现接口时，它为接口中声明的每个方法提供方法体。例如：

```
public class OperateBMW760i implements OperateCar {

  // the OperateCar method signatures, with implementation --
  // for example:
  int signalTurn(Direction direction, boolean signalOn) {
    //code to turn BMW's LEFT turn indicator lights on
    //code to turn BMW's LEFT turn indicator lights off
    //code to turn BMW's RIGHT turn indicator lights on
    //code to turn BMW's RIGHT turn indicator lights off
  }

  // other members, as needed -- for example, helper classes
  // not visible to clients of the interface
}
```

在前面机器人汽车的例子中，接口由汽车生产厂商实现。当然在本质上，雪佛兰牌汽车的实现和丰田汽车的实现不同，但是这两个汽车生产厂商都应用相同的接口。导航设备生产厂商是接口的客户，它构建的系统将使用GPS数据提供的汽车位置、数字街道地图和交通状况数据来驾驶汽车。这样，导航系统就会调用接口的方法：转向、改变车道、刹车、加速等。

5.1.2 接口作为 API

机器人汽车的例子演示了接口被用作工业标准的API。API也常见于商业软件产品中。典型情况下，一个公司销售包含复杂方法的软件包，另一个公司希望在其自己的软件产品中使用这些软件包。例如，数字图像处理方法的软件包被卖给制作最终用户图片程序的公司。图像处理软件公司编写类来实现接口，并且使接口对其客户公开。然后，图片公司使用接口中定义的签名和返回类型调用图像处理方法。虽然图像处理软件公司的API对其用户是公开的，但是API的实现是严格保密的——实际上，只要实现还继续实现用户依赖的原始接口，就可以在以后修改该实现。

5.1.3 接口和多重继承

接口在Java编程语言中扮演了一个非常重要的角色。接口不是类层次结构的一部分，但它们和类结合在一起工作。Java编程语言不允许多重继承（继承将在本章后面讨论），但是接口提供了替代方式。

在Java中，类只能继承一个类，但是类可以实现一个以上的接口。因此，对象可以具有多重类型：它们自己类的类型和它们实现的所有接口的类型。这就是说，如果变量被声明为接口的类型，那么它的值就可以引用从实现这个接口的任何类实例化而来的任何对象。我们将在后面的5.1.6节中讨论这个问题。

5.1.4 定义接口

接口声明由修饰符、关键字interface、接口名称、以逗号分隔的父接口清单（如果有父

接口的话）和接口体构成。例如：

```
public interface GroupedInterface extends Interface1,
                                          Interface2, Interface3 {

  // constant declarations
  double E = 2.718282;  // base of natural logarithms

  // method signatures
  void doSomething (int i, double x);
  int doSomethingElse(String s);
}
```

访问说明符public表明，这个接口可以被任何包中的任何类使用。如果没有把接口指定为公共的，就只能从定义该接口的同一个包中定义的类访问这个接口。

接口可以扩展其他接口，就像类可以扩展或者子类化其他类一样。但是，类只能扩展一个其他类，而接口可以扩展任意数量的接口。接口声明中包含它扩展的所有接口的清单，以逗号分隔。

接口体

接口体包含接口中所有方法的方法声明。因为接口没有为其中声明的方法提供实现，所以接口中方法声明的后面有一个分号，但是没有花括号。接口中声明的所有方法被隐式地声明为
142 public，所以可以省略public修饰符。

除了方法声明之外，接口还可以包含常量声明。接口中定义的所有常量值被隐式地声明为public、static和final。同样，可以省略这些修饰符。

5.1.5　实现接口

为了声明实现接口的类，需要在类声明中包含implements子句。你的类可以实现一个以上的接口，所以implements关键字后面是以逗号分隔的类实现的接口清单。

按照惯例，如果有extends子句，那么implements子句在extends子句后面。

1. 接口示例Relatable

考虑一个接口，它定义如何比较对象的大小。

```
public interface Relatable {

  // this (object calling isLargerThan) and
  // other must be instances of the same class
  // returns 1, 0, -1 if this is greater
  // than, equal to, or less than other
  public int isLargerThan(Relatable other);
}
```

如果你希望能够比较类似对象的大小，那么不管对象是什么，实例化对象的类都应该实现Relatable接口。

如果能够通过某种方式比较从类实例化出来的对象的相对“大小”，那么任何类都可以实现Relatable。对于字符串，可以比较字符数；对于书籍，可以比较页数；对于学生，可以比较体重；等等。对于二维几何对象，面积是一个不错的选择（参见后面的RectanglePlus类），而对于三维几何对象，可以比较体积。所有这些类都可以实现isLargerThan()方法。

如果你知道一个类实现了Relatable，就知道了可以比较从这个类实例化出来的对象的大小。

2. 实现Relatable接口

143 下面是4.2.1节中出现的Rectangle类，我们重新编写它来实现Relatable：

```
public class RectanglePlus implements Relatable {
```

```
public int width = 0;
public int height = 0;
public Point origin;

// four constructors
public RectanglePlus() {
  origin = new Point(0, 0);
}
public RectanglePlus(Point p) {
  origin = p;
}
public RectanglePlus(int w, int h) {
  origin = new Point(0, 0);
  width = w;
  height = h;
}
public RectanglePlus(Point p, int w, int h) {
  origin = p;
  width = w;
  height = h;
}

// a method for moving the rectangle
public void move(int x, int y) {
  origin.x = x;
  origin.y = y;
}

// a method for computing the area of the rectangle
public int getArea() {
  return width * height;
}

// a method to implement Relatable
public int isLargerThan(Relatable other) {
  RectanglePlus otherRect = (RectanglePlus)other;
  if (this.getArea() < otherRect.getArea())
    return -1;
  else if (this.getArea() > otherRect.getArea())
    return 1;
  else
    return 0;
}
}
```

因为RectanglePlus类实现了Relatable接口，所以可以比较任意两个RectanglePlus对象的大小。 144

5.1.6 使用接口作为类型

定义新的接口时，就是在定义新的引用数据类型。在可以使用所有其他数据类型名称的任何位置，都可以使用接口名称。如果你定义引用变量的类型为一个接口，那么赋给它的任何对象都必须是实现这个接口的类的实例。

下面是用于查找两个对象中的最大者的方法，可以用于从实现Relatable的类实例化而来的任何对象：

```
public Object findLargest(Object object1, Object object2) {
  Relatable obj1 = (Relatable)object1;
  Relatable obj2 = (Relatable)object2;
```

```
    if ((obj1).isLargerThan(obj2) > 0)
      return object1;
    else
      return object2;
  }
```

通过把object1类型强制转换为Relatable类型，就可以调用isLargerThan方法。

如果你在各种各样的类中实现Relatable，那么就可以使用findLargest()方法比较从任何这些类实例化而来的对象——但前提是两个对象都实例化自同一个类。类似地，可以使用如下方法比较所有这些类实例化的对象：

```
public Object findSmallest(Object object1, Object object2) {
  Relatable obj1 = (Relatable)object1;
  Relatable obj2 = (Relatable)object2;
  if ( (obj1).isLargerThan(obj2) < 0)
    return object1;
  else
    return object2;
}

public boolean isEqual(Object object1, Object object2) {
  Relatable obj1 = (Relatable)object1;
  Relatable obj2 = (Relatable)object2;
  if ( (obj1).isLargerThan(obj2) == 0)
    return true;
  else
    return false;
}
```

这些方法可以用于任何“可比较”对象，而不管它们的类层次结构如何。当类实现Relatable接口时，它们可以是它们自己类（或者超类）的类型，也可以是Relatable类型。这就使类具有了多重继承的某些优势，它们可以同时具有超类和接口的行为。

145

5.1.7　重新编写接口

考虑你已经开发的名为DoIt的接口：

```
public interface DoIt {
  void doSomething(int i, double x);
  int doSomethingElse(String s);
}
```

假设以后你希望把第三个方法添加到DoIt，那么该接口就变成了：

```
public interface DoIt {
  void doSomething(int i, double x);
  int doSomethingElse(String s);
  boolean didItWork(int i, double x, String s);
}
```

如果这样改动，实现旧的DoIt接口的所有类都将无法工作，因为它们不再实现这个接口了。使用这个接口的程序员会提出强烈抗议。

请尝试预测接口的所有用途，并且从一开始就完整地指定它。因为这一要求经常难以做到，所以，以后你可能需要创建更多接口。例如，可以创建扩展DoIt的DoItPlus接口：

```
public interface DoItPlus extends DoIt {

  boolean didItWork(int i, double x, String s);
}
```

现在使用你的代码的用户可以选择继续使用旧的接口，或者选择更新到新的接口。

5.1.8 接口小结

接口定义在两个对象之间进行通信的协议。

接口声明中包含一组方法的签名，但是不包含这些方法的实现，并且可能包含常量定义。

实现接口的类必须实现这个接口中声明的所有方法。

接口名称可以用在可以使用类型的任何位置。

146

问题和练习：接口

问题

1. 实现java.lang.CharSequence[1]接口的类必须实现什么方法？
2. 下面的接口有什么错误？

```
public interface SomethingIsWrong {
  void aMethod(int aValue){
    System.out.println("Hi Mom");
  }
}
```

3. 改正“问题2”中接口的错误。
4. 下面的接口合法吗？

```
public interface Marker {
}
```

练习

1. 编写一个类，实现java.lang包中的CharSequence接口。选择本书中的一组语句用作数据。编写一个小的main方法来测试你的类；确保调用所有4个方法。
2. 假设你编写了一个时间服务器，它定期向客户通报当前日期和时间。编写服务器使用的一个接口，强制其客户使用特定协议。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/IandI/QandE/interfaces-answers.html

5.2 继承

在前面的章节中，曾经几次提到继承（inheritance）。在Java语言中，某个类可以从其他类派生（derive）出来，因此它就继承（inherit）了这些类的字段和方法。

147

定义 从另一个类派生而来的类被称为子类（subclass），也被称为派生类（derived class）、扩展类（extended class）或者子类（child class）。派生子类的类被称为超类（superclass），也被称为基类（base class）或者父类（parent class）。

除了Object（它没有超类）之外，所有类都有且只有一个直接超类（单一继承）。在没有任何其他显式超类的情况下，每个类都是Object的隐式子类。

类可以从另一个类的派生类的派生类的派生类……派生而来，最终派生于最顶级的类Object。这样的类被称为继承链条中所有类的后裔，继承链条一直延伸到Object。

1. docs/api/java/lang/CharSequence.html

继承的概念非常简单，但是其功能非常强大：当你希望创建新的类，而且已经有一个类包含你要使用的一些代码时，就可以从这个现有的类派生出你的新类。通过这种方式，就可以重用现有类的字段和方法，而无需自己编写（和调试）它们。

子类继承其父类的所有公共（public）和受保护（protected）成员（字段、方法和嵌套类），而不管子类在哪个包中。如果子类和它的父类在同一个包中，还会继承父类的包私有（package-private）成员。构造器不是成员，所以子类不继承构造器，但是可以从子类调用父类的构造器。

5.2.1　Java平台类层次结构

java.lang包中定义的Object类[1]定义和实现所有类（包括你编写的类）通用的行为。在Java平台中，从Object类派生出很多类，这些类又派生出很多其他类，依此类推，这样形成了类的层次结构（见图5-1）。

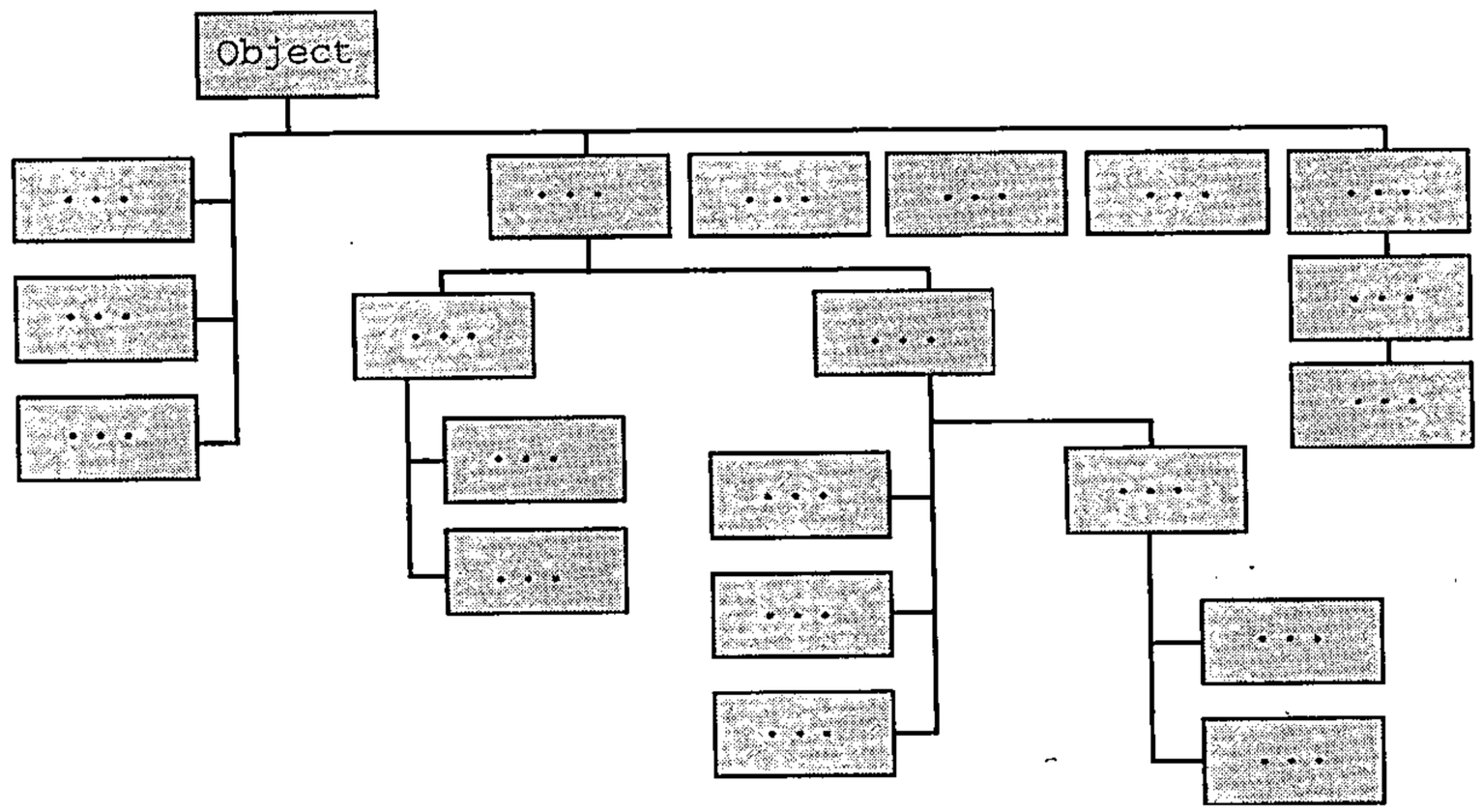


图5-1　Java平台中的所有类都是Object的后裔

在这个层次结构的顶端，Object是所有类中最一般化的类。接近层次结构底部的类提供更加专门化的行为。

5.2.2　继承示例

下面是第4章中出现过的Bicycle类的可能实现的示例代码：

```
public class Bicycle {

    // the Bicycle class has three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int startCadence,
                   int startSpeed, int startGear) {
        gear = startGear;
        cadence = startCadence;
```

1. docs/api/java/lang/Object.html

```
    speed = startSpeed;
}

// the Bicycle class has four methods
public void setCadence(int newValue) {
    cadence = newValue;
}

public void setGear(int newValue) {
    gear = newValue;
}

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

148
⋮
149

MountainBike类是Bicycle的子类，它的类声明可能像下面这样：

```
public class MountainBike extends Bicycle {

    // the MountainBike subclass adds one field
    public int seatHeight;

    // the MountainBike subclass has one constructor

    public MountainBike(int startHeight, int startCadence,
                        int startSpeed, int startGear) {
        super(startCadence, startSpeed, startGear);
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one method
    public void setHeight(int newValue) {
        seatHeight = newValue;
    }
}
```

MountainBike继承了Bicycle的所有字段和方法，并且添加了字段seatHeight和设置这个字段的方法。除了构造器之外，这就像你完全从头开始编写新的MountainBike，包括4个字段和5个方法。但是，不必自己完成所有工作。如果Bicycle类中的方法非常复杂，并且需要大量时间进行调试，那么这种方式就具有非常大的价值。

5.2.3 在子类中能进行什么操作

子类继承其父类的所有公共（public）和受保护（protected）成员，而不管子类在哪个包中。如果子类及其父类在同一个包中，那么子类还继承父类的包私有（package-private）成员。子类可以使用继承而来的成员、替换它们、隐藏它们或者使用新成员补充它们。

- 可以像对所有其他字段那样直接使用继承的字段。
- 可以在子类中声明与超类字段同名的字段，从而隐藏（hide）超类字段（不推荐这样做）。
- 可以在子类中声明超类中没有的新字段。
- 可以直接使用继承的方法。
- 可以在子类中编写与超类方法的签名相同的新的实例（instance）方法，从而覆盖（override）超类方法。

150

- 可以在子类中编写与超类方法的签名相同的新的静态方法，从而隐藏超类方法。
- 可以在子类中声明超类中没有的新方法。
- 可以编写子类构造器来调用超类构造器，可以通过隐式方式，也可以使用关键字super。

本章下面几节将深入讨论这些主题。

5.2.4　子类中的私有成员

子类并不继承其父类的private成员。但是，如果超类具有公共或者受保护方法用于访问其私有字段，那么子类也可以使用这些方法。

嵌套类可以访问包含它的类的所有私有成员——包括字段和方法。因此，子类继承的公共或者受保护嵌套类可以间接访问超类的所有私有成员。

5.2.5　对象的类型转换

我们介绍过，对象属于实例化它的类的数据类型。例如，如果我们编写：

```
public MountainBike myBike = new MountainBike();
```

那么myBike就属于MountainBike类型。

MountainBike从Bicycle和Object派生而来，因此MountainBike是Bicycle，也是Object，并且可以在能够调用Bicycle或者Object对象的任何位置使用它。

相反的情况并不一定成立：Bicycle可能是MountainBike，但并不一定是。类似地，Object可能是Bicycle或者MountainBike，但并不一定是。

类型转换（cast）允许在一种类型的位置上使用另一种类型的对象，但是类型转换只能在继承和实现允许转换的对象之间进行。例如，如果编写：

```
Object obj = new MountainBike();
```

那么obj既是Object也是MountainBike（除非obj被赋值为另一个不是MountainBike的对象）。这被称为隐式类型转换（implicit cast）。

另一方面，如果编写：

151

```
MountainBike myBike = obj;
```

就会出现编译时错误，因为编译器不知道obj是MountainBike。但是，通过显式类型转换（explicit cast），我们可以通知编译器，我们承诺把MountainBike赋值给obj：

```
MountainBike myBike = (MountainBike) obj;
```

这种类型转换插入一个对obj是否被赋值为MountainBike的运行时检查，以便编译器能够安全地假设obj是MountainBike。如果在运行时obj不是MountainBike，就会抛出异常。

注意　可以使用instanceof操作符对特定对象的类型进行逻辑检查。这可以避免因不适当的类型转换而产生的运行时错误。例如：

```
if (obj instanceof MountainBike) {
   MountainBike myBike = (MountainBike) obj;
}
```

这里，instanceof操作符检查obj是否引用MountainBike，这样我们就可以使类型转换操作不会抛出运行时异常。

5.2.6 覆盖和隐藏方法

1. 实例方法

如果子类中实例方法的签名（名称，还有参数的数量和类型）和返回类型与超类中实例方法的签名和返回类型一样，那么子类中的方法就覆盖（override）超类的方法。

子类能够覆盖方法的能力使类可以继承与其行为“足够接近”的超类，然后按照需要修改行为。覆盖后的方法的名称、参数数量和类型以及返回类型与被覆盖的方法相同。覆盖方法也可以返回被覆盖方法的返回类型的子类型，这被称为协变返回类型（covariant return type）。

当覆盖方法时，你可能希望使用@Override注解，它指示编译器你的目的是覆盖超类中的方法。如果出于某种原因，编译器检测到这个方法不存在于一个超类中，它就会生成错误。关于@Override的更多信息请参考4.6节。

2. 类方法

如果子类定义的类方法的签名和超类中类方法的签名相同，那么子类中的方法就隐藏（hide）超类中的方法。

152

隐藏和覆盖之间的区别有重要意义。被调用的覆盖方法的版本是子类中的版本。被调用的隐藏方法的版本取决于是从超类调用，还是从子类调用。我们来看一个包含两个类的例子。第一个类是Animal，它包含一个实例方法和一个类方法：

```
public class Animal {
  public static void testClassMethod() {
    System.out.println("The class method in Animal.");
  }
  public void testInstanceMethod() {
    System.out.println("The instance method in Animal.");
  }
}
```

第二个类是Animal的子类，名为Cat：

```
public class Cat extends Animal {
  public static void testClassMethod() {
    System.out.println("The class method in Cat.");
  }
  public void testInstanceMethod() {
    System.out.println("The instance method in Cat.");
  }

  public static void main(String[] args) {
    Cat myCat = new Cat();
    Animal myAnimal = myCat;
    Animal.testClassMethod();
    myAnimal.testInstanceMethod();
  }
}
```

Cat类覆盖Animal类的实例方法，并且隐藏Animal类的类方法。这个类中的main方法创建Cat的实例，并且调用类的testClassMethod()，调用实例的testInstanceMethod()。

这个程序的输出如下：

```
The class method in Animal.
The instance method in Cat.
```

正如预料，调用的隐藏方法的版本是超类中的版本，而调用的覆盖方法的版本是子类中的版本。

153

3. 修饰符

与被覆盖的方法相比，我们允许覆盖方法的访问说明符更宽松，而不允许更严格。例如，超类中的受保护实例方法在子类中可以被设置为公共，但是不能被设置为私有。

如果你试图把超类中的实例方法改变为子类中的类方法，或者进行相反操作，就会出现编译时错误。

4. 小结

表5-1总结了使用和超类方法相同的签名来定义方法时所发生的情况。

表5-1　使用和超类方法相同的签名定义方法

	超类实例方法	超类静态方法
子类实例方法	覆盖	生成编译时错误
子类静态方法	生成编译时错误	隐藏

注意　在子类中，可以重载从超类继承而来的方法。这样，重载的方法既不隐藏也不覆盖超类方法——它们是新的方法，是子类特有的。

5.2.7　隐藏字段

在一个类中，与超类字段同名的字段会隐藏超类的字段，即使它们的类型不同。在子类中，不能使用简单名称引用超类中的字段。相反，必须通过super访问超类的字段，在下一节将讲解这个关键字。总的来说，我们不建议隐藏字段，因为这样会使代码难以阅读。

5.2.8　使用关键字 super

1. 访问超类方法

如果你的方法覆盖超类的一个方法，那么可以通过关键字super调用被覆盖的方法。也可
154 以使用super引用隐藏字段（虽然不建议隐藏字段）。考虑下面这个类Superclass：

```
public class Superclass {

  public void printMethod() {
    System.out.println("Printed in Superclass.");
  }
}
```

下面是子类Subclass，它覆盖printMethod()：

```
public class Subclass extends Superclass {

  public void printMethod() {
    //overrides printMethod in Superclass
    super.printMethod();
    System.out.println("Printed in Subclass");
  }
  public static void main(String[] args) {

  Subclass s = new Subclass();
  s.printMethod();
  }

}
```

在Subclass中，简单名称printMethod()引用Subclass声明的那个方法，而覆盖

Superclass中的同名方法。所以，为了引用从Superclass继承而来的printMethod()，Subclass必须使用包含super的限定名称，如上所示。编译和执行Subclass的输出如下：

```
Printed in Superclass.
Printed in Subclass
```

2. 子类构造器

下面的例子演示如何使用super关键字调用超类的构造器。回忆Bicycle的例子(见5.2节)，其中MountainBike是Bicycle的子类。下面的MountainBike（子类）构造器调用超类构造器，然后添加它自己的初始化代码：

```
public MountainBike(int startHeight, int startCadence,
                    int startSpeed, int startGear) {
  super(startCadence, startSpeed, startGear);
  seatHeight = startHeight;
}
```

用于调用超类构造器的代码必须位于子类构造器的第一行。

调用超类构造器的语法如下：

155

```
super();
```

或者

```
super (parameter list) ;
```

使用super()时，调用超类的无实参构造器。使用super (parameter list) 时，调用带有匹配的参数列表的超类构造器。

注意 如果构造器没有显式调用超类构造器，Java编译器就会自动插入对超类的无实参构造器的调用。如果超类没有无实参构造器，就会出现编译时错误。Object确实具有无实参构造器，所以如果Object是唯一超类，则没有问题。

如果子类构造器通过显式方式或者隐式方式调用其超类的构造器，你可以认为将有一系列构造器被调用，调用路径一直延伸到Object的构造器。实际上，情况就是如此。这被称为构造器链（constructor chaining），并且当类继承的路径很长时，你需要认识到这一点。

5.2.9 Object 作为超类

java.lang包中的Object类位于类层次结构树的顶端。每个类都是Object类的直接或者间接后裔。你使用或者编写的每个类都继承Object的实例方法。你可以不使用这些方法，但是，如果选择使用它们，可能就需要使用你的类专用的代码覆盖它们。这一节中讨论的从Object继承而来的方法如下：

- **protected Object clone() throws CloneNotSupportedException**——创建和返回这个对象的副本。
- **public boolean equals (Object obj)**——指出另一个对象是否“等于”这个对象。
- **protected void finalize() throws Throwable**——当垃圾收集器判断没有对某个对象的引用时，垃圾收集器对这个对象调用此方法。
- **public final Class getClass()**——返回对象的运行时类。
- **public int hashCode()**——返回对象的散列码值。
- **public String toString()**——返回对象的字符串表示。

156

Object的notify、notifyAll和wait方法都用于同步程序中独立运行的线程活动，我们

将在第12章中讨论它们，这里不多做介绍。这类方法有5个：

- public final void notify()
- public final void notifyAll()
- public final void wait()
- public final void wait(long timeout)
- public final void wait(long timeout, int nanos)

注意　很多这类方法（特别是clone方法）有些微妙的性质。从Josh Bloch撰写的*Effective Java*[1]一书中能够获得正确使用这些方法的信息。

1. clone()方法

如果一个类或者它的一个超类实现了Cloneable接口，就可以使用clone()方法从现有对象创建一个副本。为了创建克隆，可以编写：

```
aCloneableObject.clone();
```

Object的这个方法的实现检查调用clone()方法的对象是否实现了Cloneable接口。如果对象没有实现Cloneable接口，方法就会抛出CloneNotSupportedException异常。异常处理将在第9章中讲解。目前你只需要知道，如果要编写一个clone()方法覆盖Object的这个方法，clone()方法必须被声明为：

```
protected Object clone() throws CloneNotSupportedException
```

或者

```
public Object clone() throws CloneNotSupportedException
```

如果调用clone()方法的对象确实实现了Cloneable接口，那么Object的clone()方法的实现就创建和原始对象属于同一个类的对象，并且使用和原始对象的对应成员变量相同的值初始化新对象的成员变量。

157

使类可以克隆的最简单途径是把implements Cloneable添加到类的声明中。然后你的对象就能调用clone()方法了。

对于一些类，Object的clone()方法的默认行为工作得很好。但是，如果对象包含对外部对象（比如ObjExternal）的引用，就可能需要覆盖clone()以便获得正确行为。否则，一个对象对ObjExternal做出的改动在其克隆对象中也是可见的。这就是说，原始对象及其克隆对象不是相互独立的——为了分离它们，你必须覆盖clone()，以便它克隆这个对象和ObjExternal。这样，原始对象引用ObjExternal，而克隆对象引用ObjExternal的克隆，这样，原始对象与其克隆对象彼此真正地独立。

2. equals()方法

equals()方法比较两个对象的等同性，如果它们相等，则返回true。Object类提供的equals()方法使用相等操作符（==）确定两个对象是否相等。对于基本数据类型，这能给出正确结果。但是对于对象，这就不适用了。Object提供的equals()方法检查对象引用（reference）是否相等——就是说，被比较的两个对象是否严格地是同一个对象。

为了检查两个对象是否在等价（equivalency）的意义上相等（即包含相同信息），必须覆盖equals()方法。下面是覆盖equals()方法的Book类的例子：

---

1. docs/books/effective

```
public class Book {
  ...
  public boolean equals(Object obj) {
    if (obj instanceof Book)
      return ISBN.equals((Book)obj.getISBN());
    else
      return false;
  }
}
```

查看下面检查两个Book类的实例是否相等的代码：

```
Book firstBook = new Book("0201914670");
  // Swing Tutorial, 2nd edition
Book secondBook = new Book("0201914670");
if (firstBook.equals(secondBook)) {
  System.out.println("objects are equal");
}
else {
  System.out.println("objects are not equal");
}
```

158

尽管firstBook和secondBook引用两个完全不同的对象，这个程序也会输出“objects are equal”。因为被比较的两个对象包含相同的ISBN，所以认为它们相等。

如果相等操作符不适用于你的类，那么总应该覆盖equals()方法。

注意 如果你覆盖equals()，就必须也覆盖hashCode()。

3. finalize()方法

Object类提供一个回调方法finalize()，当对象变成垃圾时可能对这个对象调用这个方法。Object的finalize()实现不进行任何操作——你可以覆盖finalize()以便进行清理操作，比如释放资源。

系统可能自动调用finalize()方法，但是什么时候调用它（甚至是否调用它）是不确定的。因此，不要依赖这个方法为你进行清理操作。例如，如果在你的代码中，在执行I/O操作之后没有关闭文件描述符，而指望finalize()关闭它们，那么就有可能用光文件描述符。

4. getClass()方法

不能覆盖getClass()方法。getClass()方法返回一个Class对象，可以使用这个对象的方法获得关于类的信息，比如类的名称（getSimpleName()）、其超类（getSuperclass()）以及它实现的接口（getInterfaces()）。例如，下面的方法获得并且显示对象的类名称：

```
void printClassName(Object obj) {
  System.out.println("The object's class is "
                     obj.getClass().getSimpleName());
}
```

java.lang包中的Class类[1]具有很多方法（超过50个）。例如，你可以查看类是否是注解（isAnnotation()）、是否是接口（isInterface()）或者是否是枚举（isEnum()）。可以查看对象的字段是什么（getFields()），或者它的方法是什么（getMethods()），等等。

159

5. hashCode()方法

hashCode()方法的返回值是对象的散列码，它是对象的十六进制内存地址。

1. docs/api/java/lang/Class.html

按照定义，如果两个对象相等，它们的散列码必须也相等。如果你覆盖了equals()方法，就改变了两个对象相等的方式，Object的hashCode()实现就不再合法。因此，如果你覆盖equals()方法，就必须同时覆盖hashCode()方法。

6. toString()方法

你应该总是考虑覆盖你的类中的toString()方法。

Object的toString()方法返回对象的String表示，这对于调试非常有用。对象的String表示完全取决于对象，这就是需要覆盖你的类中的toString()的原因。

可以把toString()和System.out.println()一起使用，显示对象的文本表示，比如Book的一个实例：

```
System.out.println(firstBook.toString());
```

对于正确覆盖的toString()方法，上述代码输出一些有用的信息，比如：

```
ISBN: 0201914670; The JFC Swing Tutorial; A Guide to
                                Constructing GUIs, 2nd Edition
```

5.2.10　编写最终类和方法

可以把类的一些或者全部方法声明为最终（final）。在方法声明中使用关键字final可以指定这个方法不能被子类覆盖。Object类就是这样做的——它的很多方法被声明为final。

如果方法的实现不应该被改动，并且方法的实现对对象的状态一致性至关重要，你就可能希望使这个方法成为最终的。例如，你可能希望使getFirstPlayer方法在这个ChessAlgorithm类中是最终的：

```
class ChessAlgorithm {
  enum ChessPlayer { WHITE, BLACK }
  ...
  final ChessPlayer getFirstPlayer() {
    return ChessPlayer.WHITE;
  }
  ...
}
```

160

从构造器调用的方法一般应该被声明为最终的。如果构造器调用非最终方法，子类就可能重新定义这个方法，从而导致意外的或者不期望的结果。

注意，也可以声明整个类为最终的——这会防止这个类被子类化。例如，当创建不可变的类（比如String类）时，这就非常有用。

5.2.11　抽象方法和抽象类

抽象类（abstract class）是被声明为abstract的类——它可能包含也可能不包含抽象方法。抽象类不能被实例化，但是可以被子类化。

抽象方法（abstract method）是被声明为不具有实现（没有花括号，后面跟着分号）的方法，比如：

```
abstract void moveTo(double deltaX, double deltaY);
```

如果类包含抽象方法，那么该类本身必须被声明为abstract，比如：

```
public abstract class GraphicObject {
  // declare fields
  // declare non-abstract methods
  abstract void draw();
}
```

当抽象类被子类化时，子类通常提供其父类中所有抽象方法的实现。但是，如果子类没有这么做，那么子类就必须也被声明为abstract。

注意 接口（interface）（见5.1节）中的所有方法隐式地（implicitly）声明为抽象的，所以接口方法不使用abstract修饰符（也可以使用，但是没有必要）。

1. 抽象类和接口

和接口不同，抽象类可以包含非static和final的字段，并且它们可以包含实现了的方法。这样的抽象类类似于接口，但是它们提供了部分实现，把完整实现留给子类去完成。如果抽象类只包含抽象方法声明，就应该把它声明为接口。

类可以在类层次结构中的任何位置实现多个接口，不管它们是否按任何形式相互关联。例如Comparable或者Cloneable。

比较来说，抽象类经常被子类化以便共享一些实现。单一抽象类被具有很多共同点（抽象类实现了的部分）但也有一些不同点（抽象方法）的相似类子类化。 161

2. 抽象类示例

在面向对象的绘图应用程序中，你可以绘制圆、矩形、直线、贝塞尔（Bezier）曲线和很多其他图形对象。这些对象都具有相同的特定状态（例如位置、方向、线条颜色和填充颜色）和行为（例如移动、旋转、改变大小和绘制）。一些状态和行为对所有图形是相同的，例如位置、填充颜色和移动。另一些状态和行为需要不同的实现，例如改变大小和绘制。所有GraphicObject都必须知道如何绘制自己或者如何改变自己的大小；区别仅仅在于如何完成这些操作。

这是应用抽象超类的理想情况。你可以利用相似性的优势，声明所有图形对象继承同一个抽象父对象——例如GraphicObject，如图5-2所示。

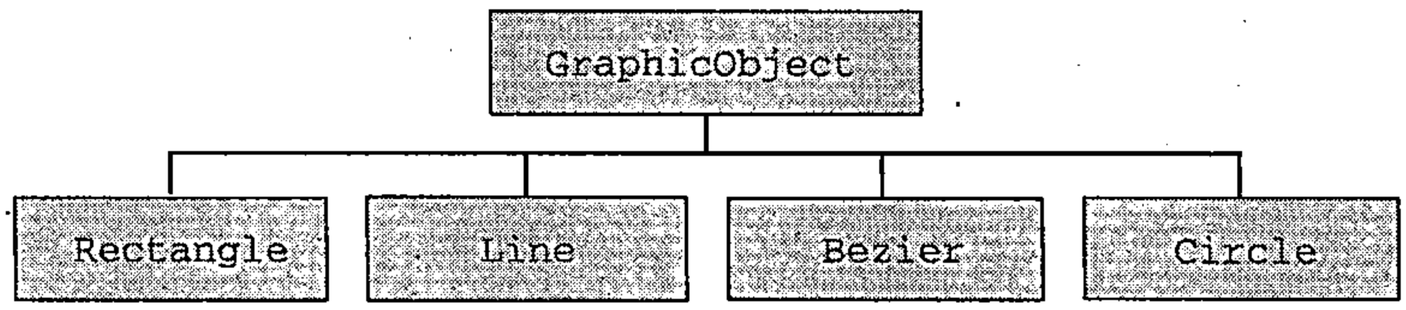


图5-2 Rectangle、Line、Bezier和Circle都继承GraphicObject

首先，声明抽象类GraphicObject提供所有子类都共享的成员变量和方法，比如当前位置和moveTo方法。GraphicObject还声明抽象方法，比如draw或者resize，要求所有子类实现它们，但是必须通过不同方式实现。GraphicObject类像下面这样：

```
abstract class GraphicObject {
  int x, y;
  ...
  void moveTo(int newX, int newY) {
    ...
  }
  abstract void draw();
  abstract void resize();
}
```

162

GraphicObject的每个非抽象子类（比如Circle和Rectangle）都必须为draw和resize方法提供实现：

```
class Circle extends GraphicObject {
  void draw() {
    ...
  }
  void resize() {
    ...
  }
}
class Rectangle extends GraphicObject {
  void draw() {
    ...
  }
  void resize() {
    ...
  }
}
```

3. 当抽象类实现接口时

在5.1节中，曾经强调实现接口的类必须实现接口的所有方法。但是，实际上可以定义不实现所有接口方法的类，前提是这个类被声明为abstract。例如：

```
abstract class X implements Y {
  // implements all but one method of Y
}

class XX extends X {
  // implements the remaining method in Y
}
```

在这个例子中，X类必须是abstract，因为它没有完全实现Y，但是实际上，XX类的确实现了Y。

4. 类成员

抽象类可能具有static字段和static方法。可以通过类引用使用这些静态成员，比如
163 AbstractClass.StaticMethod()，就像对任何其他类那样。

5.2.12　继承小结

除Object类以外，每个类都只有一个直接超类。类（直接或间接地）从它的所有超类继承字段和方法。子类可以覆盖它继承的方法，也可以隐藏它继承的字段或方法。（注意，隐藏字段通常是不良的编程习惯。）

5.2.6节中的表5-1显示了使用和超类方法相同的签名声明一个方法的效果。

Object类位于类层次结构的顶端。所有类都是这个类的后裔并从它继承方法。从Object继承的有用方法包括toString()、equals()、clone()和getClass()。

通过在类声明中使用final关键字，可以防止类被子类化。类似地，可以通过将方法声明为最终方法，防止它被子类覆盖。

抽象类只能被子类化，不能被实例化。抽象类可以包含抽象方法，这种方法只是被声明但没有被实现。然后，子类为抽象方法提供实现。

问题和练习：继承

问题

1. 查看下面两个类：

```
public class ClassA {
```

```
  public void methodOne(int i) {
  }
  public void methodTwo(int i) {
  }
  public static void methodThree(int i) {
  }
  public static void methodFour(int i) {
  }
}

public class ClassB extends ClassA {
  public static void methodOne(int i) {
  }
  public void methodTwo(int i) {
  }
  public void methodThree(int i) {
  }
  public static void methodFour(int i) {
  }
}
```

a. 哪个方法覆盖了超类中的方法？

b. 哪个方法隐藏了超类中的方法？

c. 其他方法会有什么效果？

2. 查看“问题和练习：类”中编写的Card[1]、Deck[2]和DisplayDeck[3]类。这些类应该覆盖Object的什么方法？

练习

编写“问题2”中回答出的方法的实现。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/java/IandI/QandE/inherit-answers.html

1. tutorial/java/IandI/QandE/Card.java
2. tutorial/java/IandI/QandE/Deck.java
3. tutorial/java/IandI/QandE/DisplayDeck.java

第6章 泛型

6.1 简介

在一些重要的大型软件项目中，错误是不可避免的。谨慎地进行计划、程序设计和测试能够帮助减少错误，但是出于某些原因，在某些地方，错误总能不知不觉地跑到你的代码里。当引入了新特性并且你的代码基变大和变复杂时，这种情况尤其明显。

幸运的是，某些错误比其他错误更容易检测。例如，编译时错误会立即通知你出了问题；可以使用编译器的错误消息立即找出问题所在并且改正问题。但是，运行时错误的问题要大得多：它们不总是马上出现，当它们出现时，表现出的错误位置可能远离真正导致问题的位置。

泛型（generic）为你的代码添加了更容易在编译时检测出错误的能力。一些程序员选择通过学习Java集合框架（Java Collections Framework）来学习泛型；毕竟这些类的确非常多地使用了泛型。但是，因为我们还没有讲到集合，所以本章将重点介绍简单的“类似集合”的示例，我们将从头开始设计这些示例。通过这种实践方式介绍必需的语法和术语，同时演示设计泛型用于解决的各种各样的问题。

简单的Box类

我们从设计非泛型的Box类开始，这个类对任意类型的对象进行操作。它只需要提供两个方法：add（把一个对象添加到盒子中）和get（获得添加的对象）：

```
public class Box {

  private Object object;

  public void add(Object object) {
    this.object = object;
  }

  public Object get() {
    return object;
  }
}
```

因为类的方法接受或者返回Object，所以可以随意地把你希望的任何内容传递给它，但不能传递原始类型。但是，如果需要把包含的类型限制在某个特定范围内（比如Integer），唯一的选择是在文档中（或者，像这个例子一样，就在注释中）指定这一要求，当然这未能让编译器知道你的限制[1]：

```
public class BoxDemo1 {

  public static void main(String[] args) {
```

1. tutorial/java/generics/examples/BoxDemo1.java

```
    // ONLY place Integer objects into this box!
    Box integerBox = new Box();

    integerBox.add(new Integer(10));
    Integer someInteger = (Integer)integerBox.get();
    System.out.println(someInteger);
  }
}
```

程序BoxDemo1创建Integer对象，把它传递给add，然后通过get的返回值把同一个对象赋值给someInteger。然后程序把对象的值（10）发送到标准输出。我们知道，从Object到Integer的类型转换是正确的，因为我们在注释中指定了“契约”。但是要记住，编译器完全不知道这些——它只是信任我们的类型转换是正确的。此外，编译器不会进行保护措施，来防止粗心的程序员把类型错误的对象传递给这个类，比如String[1]：

168

```
public class BoxDemo2 {

  public static void main(String[] args) {

    // ONLY place Integer objects into this box!
    Box integerBox = new Box();

    // Imagine this is one part of a large application
    // modified by one programmer.
    integerBox.add("10"); // note how the type is now String

    // ... and this is another, perhaps written
    // by a different programmer
    Integer someInteger = (Integer)integerBox.get();
    System.out.println(someInteger);
  }
}
```

在这里，我们把数字10存储为String，这是GUI收集用户输入时可能发生的情况。但是，现有的从Object到Integer的类型转换被错误地忽略了。这明显是一个错误，但是因为代码仍然能够通过编译，所以在运行时之前你不会发现任何错误，在运行时应用程序会因ClassCastException异常而崩溃：

```
Exception in thread "main"
  java.lang.ClassCastException:
    java.lang.String cannot be cast to java.lang.Integer
    at BoxDemo2.main(BoxDemo2.java:6)
```

如果把Box类设计为使用泛型，编译器就能发现这个错误，而不会导致应用程序在运行时崩溃。

6.2 泛型类型

我们更新Box类，让它使用泛型。我们首先通过把代码public class Box改为public class Box<T>，创建泛型类型声明（generic type declaration）；这样就引入了一个名为T的类型变量（type variable），可以在类内部的任何位置使用它。相同的技术也可以用于接口。这种概念没有特别复杂的地方。实际上，这和你已经了解的一般变量非常类似。只要把T看作特殊类型

169

1. tutorial/java/generics/examples/BoxDemo2.java

的变量就可以了，它的“值”将是你传递过来的任何类型；这可以是任何类类型、任何接口类型，或者甚至可以是另一种变量类型。但它不能是任何基本数据类型。在这个例子的上下文中，我们也可以说T是Box类的正式类型参数（formal type parameter）：

```
/**
 * Generic version of the Box class.
 */
public class Box<T> {

  private T t; // T stands for "Type"

  public void add(T t) {
    this.t = t;
  }

  public T get() {
    return t;
  }
}
```

如你所见，我们把出现Object的所有位置都替换成了T。为了在你自己的代码中引用这个泛型类，必须执行泛型类型调用（generic type invocation），它把T替换为某个具体值，比如Integer：

```
Box<Integer> integerBox;
```

可以把泛型类型调用看作类似于普通的方法调用，但不是把实参传递给方法，而是把类型实参（type argument）——在这个例子中是Integer——传递给Box类本身。和所有其他变量声明一样，这段代码没有实际创建新的Box对象。它只是声明integerBox将保存对“Integer的Box”（这是Box<Integer>的读法）的引用。

泛型类型的调用通常被称为参数化类型（parameterized type）。为了实例化这个类，需要像以往那样使用new关键字，但是要在类名称和括号之间加上<Integer>：

170
```
integerBox = new Box<Integer>();
```

或者，可以把整个语句写在一行中，比如：

```
Box<Integer> integerBox = new Box<Integer>();
```

初始化integerBox之后，就可以随意地调用它的get方法，无需提供类型转换，如BoxDemo3[1]所示：

```
public class BoxDemo3 {

  public static void main(String[] args) {
    Box<Integer> integerBox = new Box<Integer>();
    integerBox.add(new Integer(10));
    Integer someInteger = integerBox.get(); // no cast!
    System.out.println(someInteger);
  }
}
```

另外，如果你试图把不兼容的类型（比如String）添加到盒子中，编译就会失败，同时告诉你什么代码将会产生运行时错误：

```
BoxDemo3.java:5: add(java.lang.Integer)
                 in Box<java.lang.Integer>
cannot be applied to (java.lang.String)
```

---

1. tutorial/java/generics/examples/BoxDemo3.java

```
    integerBox.add("10");
               ^
1 error
```

实际上，类型变量不是类型本身，了解这一点很重要。在前面的例子中，你在文件系统上的任何位置都找不到T.java或者T.class。而且，T不是Box类名称的一部分。在实际的编译过程中，所有泛型信息都将被完全删除，文件系统上只会留下Box.class。我们将在后面的6.7节中讨论这种情况。

还要注意，泛型类型可能具有多个类型参数，但是每个参数在声明它的类或者接口中都必须是唯一的。例如，Box<T,T>的声明将在第二个T的位置产生错误，但是允许使用Box<T,U>。

类型参数惯例

按照惯例，类型参数是单一的大写字母。这和你已经了解的变量命名惯例（见3.1.1节）形成了鲜明对比，这样做有充分的理由：如果不遵循这一惯例，就难以区分类型变量和普通类名称或接口名称之间的区别。

最常用的类型参数名称如下：

171

E Element（被Java集合框架广泛地使用），元素

K Key，键

N Number，数

T Type，类型

V Value，值

S、U、V等 第二、三、四个类型

你会经常在Java SE API和本书的其余部分中看到这些名称的使用。

6.3 泛型方法和构造器

也可以在方法和构造器的签名内声明类型参数，从而创建泛型方法（generic method）和泛型构造器（generic constructor）。这和声明泛型类型类似，但是类型参数的作用域局限于声明它的方法或者构造器内：

```
/**
 * This version introduces a generic method.
 */,
public class Box<T> {

  private T t;

  public void add(T t) {
    this.t = t;
  }

  public T get() {
    return t;
  }

  public <U> void inspect(U u){
    System.out.println("T: " + t.getClass().getName());
    System.out.println("U: " + u.getClass().getName());
  }

  public static void main(String[] args) {
```

```
    Box<Integer> integerBox = new Box<Integer>();
    integerBox.add(new Integer(10));
    integerBox.inspect("some text");
  }
}
```

这里，我们添加了一个名为inspect的泛型方法，它定义一个名为U的类型参数。这个方法
172 接受一个对象，把对象的类型发送到标准输出。为了进行对比，它也输出类型T。为了方便，这个类现在也具有main方法，以便可以像应用程序那样运行。

这个程序的输出是：

```
T: java.lang.Integer
U: java.lang.String
```

传递不同的类型，程序输出也会相应地改变。

泛型方法的更实际的应用可能像下面的程序这样：它定义一个静态方法，把对单一项的引用插入到多个盒子里：

```
public static <U> void fillBoxes(U u, List<Box<U>> boxes) {
  for (Box<U> box : boxes) {
    box.add(u);
  }
}
```

为了使用这个方法，你的代码可以像这样：

```
Crayon red = ...;
List<Box<Crayon>> crayonBoxes = ...;
```

调用这个方法的完整语法是：

```
Box.<Crayon>fillBoxes(red, crayonBoxes);
```

这里，我们显式地提供用作U的类型，但是通常可以省略，编译器将推断出所需的类型：

```
Box.fillBoxes(red, crayonBoxes);   // compiler infers that
                                   // U is Crayon
```

这一特性被称为类型推断（type inference），它允许你像调用普通方法那样调用泛型方法，无需在尖括号之间指定类型。

6.4　受限类型参数

有时候你可能希望限制允许传递给类型参数的类型种类。例如，对数字进行操作的方法可能只希望接受Number或者其子类的实例。这就是受限类型参数（bounded type parameter）的用途。

173 要声明受限类型参数，首先列出类型参数的名称，后面是extends关键字，再后面是其上限（upper bound），在这个例子中是Number。注意，在这种上下文中，按一般意义，使用extends要么表示“扩展”（在类中），要么表示“实现”（在接口中）：

```
/**
 * This version introduces a bounded type parameter.
 */
public class Box<T> {

  private T t;

  public void add(T t) {
    this.t = t;
  }

  public T get() {
```

```
    return t;
  }

  public <U extends Number> void inspect(U u){
    System.out.println("T: " + t.getClass().getName());
    System.out.println("U: " + u.getClass().getName());
  }

  public static void main(String[] args) {
    Box<Integer> integerBox = new Box<Integer>();
    integerBox.add(new Integer(10));
    integerBox.inspect("some text");    // error: this is
                                        // still String!
  }
}
```

修改我们的泛型方法，使其包含这个受限类型参数后，编译就会失败，因为对inspect的调用仍然包含String：

```
Box.java:21: <U>inspect(U) in Box<java.lang.Integer> cannot
  be applied to (java.lang.String)
                        integerBox.inspect("10");
                                   ^
1 error
```

为了指定必须实现的额外接口，需要使用&字符，比如：

```
<U extends Number & MyInterface>
```

174

6.5 子类型化

你已经知道，可以把一种类型的对象赋值给另一种类型的对象，前提是两种类型兼容。例如，可以把Integer赋值给Object，因为Object是Integer的一个超类型：

```
Object someObject = new Object();
Integer someInteger = new Integer(10);
someObject = someInteger; // OK
```

在面向对象的术语中，这被称为“is-a”关系。因为Integer是Object的一种，所以允许这样赋值。但是Integer也是Number的一种，所以下列代码也是合法的：

```
public void someMethod(Number n){
  // method body omitted
}

someMethod(new Integer(10));  // OK
someMethod(new Double(10.1)); // OK
```

对于泛型来说，情况也一样。你可以执行泛型类型调用，把Number作为其类型实参，如果实参和Number兼容，就允许对add进行任何后续调用：

```
Box<Number> box = new Box<Number>();
box.add(new Integer(10));  // OK
box.add(new Double(10.1)); // OK
```

现在查看如下方法：

```
public void boxTest(Box<Number> n){
  // method body omitted
}
```

它接受什么类型的实参？通过查看其签名，你会发现它接受类型为Box<Number>的单一实参。但是确切的含义是什么？是否像你预期的那样，允许传递Box<Integer>或者Box<Double>？令人惊奇的是，回答是“否”。因为Box<Integer>和Box<Double>不是

Box<Number>的子类型。

如果你考虑现实对象（可以实际描述的事物，比如笼子），理解这种情况就要简单得多：

```
// A cage is a collection of things, with bars to keep them in.
interface Cage<E> extends Collection<E>;
```

175

注意　Collection接口是集合层次结构（collection hierarchy）的根接口；它代表一组对象。因为笼子能用来装对象（动物）的集合，所以在这个例子中使用它是有意义的。

狮子是动物的一种，所以Lion是Animal的子类型：

```
interface Lion extends Animal {}
Lion king = ...;
```

当需要动物时，我们可以提供狮子：

```
Animal a = king;
```

当然，能把狮子装到狮子笼里：

```
Cage<Lion> lionCage = ...;
lionCage.add(king);
```

也可以把蝴蝶装到蝴蝶笼里：

```
interface Butterfly extends Animal {}
Butterfly monarch = ...;
Cage<Butterfly> butterflyCage = ...;
butterflyCage.add(monarch);
```

但是“动物笼子”是什么意思呢？这个词是含义不清的，所以为了准确，我们假设讨论的是“所有动物的笼子”：

```
Cage<Animal> animalCage = ...;
```

这是用来同时混装所有种类的动物的笼子。它的栏杆必须足够结实，才能装得了狮子，而且栏杆间隙必须足够小，才能装得了蝴蝶。这样的笼子甚至根本造不出来，但是如果造得出来，那么：

```
animalCage.add(king);
animalCage.add(monarch);
```

因为狮子是一种动物（Lion是Animal的子类型），问题就成了“狮子笼是一种动物笼子吗？Cage<Lion>是Cage<Animal>的子类型吗？”按照上面“动物笼子”的定义，回答必然是“否”。这很令人意外！但是当你考虑到下面这一点时，就非常合理了：狮子笼不能装蝴蝶，而蝴蝶笼也不能装狮子。因此，这两种笼子都不能被看作“所有动物的笼子”：

```
animalCage = lionCage;       // compile-time error
animalCage = butterflyCage;  // compile-time error
```

176

没有泛型，就会把动物装到错误类型的笼子中，可能会使动物跑掉。

6.6　通配符

前面我们提到，词可能是含义不清的。“动物笼子”这个说法可以合理地表示“所有动物的笼子”，但是也可以暗示完全不同的概念：不是为任何类型的动物设计的笼子，而是为某个未知类型的动物设计的笼子。在泛型中，使用通配符（wildcard）“？”表示未知类型。

为了指定能够装某个种类的动物的笼子：

```
Cage<? extends Animal> someCage = ...;
```

?extends Animal读作“未知类型是Animal的子类型，也可能是Animal本身”，这归结为“某个类型的动物”。这是受限通配符的一个例子，其中Animal形成预期类型的上限。如果你只要求装某个类型的动物的笼子，就可以提供狮子笼或者蝴蝶笼。

注意 通过使用super关键字，而不是extends，也可以指定下限（lower bound）。因此，代码<? super Animal>将读作“未知类型是Animal的超类型，也可能是Animal本身”。你也可以使用不受限通配符（unbounded wildcard）指定未知类型，不受限通配符表示为<?>。不受限通配符在本质上和<? extends Object>表达的意思一样。

虽然Cage<Lion>和Cage<Butterfly>不是Cage<Animal>的子类型，但是实际上它们都是Cage<? extends Animal>的子类型：

```
someCage = lionCage;          // OK
someCage = butterflyCage;     // OK
```

所以现在问题变成了“可以把蝴蝶和狮子直接添加到someCage吗？”你可能猜得出，这个问题的回答是“否”：

```
someCage.add(king);           // compiler-time error
someCage.add(monarch);        // compiler-time error
```

如果someCage是蝴蝶笼，那么它能很好地装蝴蝶，但是狮子会破坏笼子逃跑。如果它是狮子笼，那么装狮子很合适，但是蝴蝶能飞出去。那么，如果根本不能把任何种类的动物放在someCage中；它不就没用了吗？不是这样的，因为你仍然可以读取其内容：

```
void feedAnimals(Cage<? extends Animal> someCage) {
  for (Animal a : someCage)
    a.feedMe();
}
```

因此，如前面所示，你可以把动物装在适当的笼子里，并且首先对狮子调用如下方法，然后对蝴蝶调用这个方法：

```
feedAnimals(lionCage);
feedAnimals(butterflyCage);
```

或者，你可以选择把动物都混装到所有动物的笼子中：

```
feedAnimals(animalCage);
```

6.7 类型擦除

当实例化泛型类型时，编译器用一种被称为类型擦除（type erasure）的技术转换这些类型。类型擦除是编译器删除类或者方法内与类型参数和类型实参相关的所有信息的过程。类型擦除确保使用泛型的Java应用程序与泛型出现之前创建的Java库和应用程序保持二进制兼容性。

例如，Box<String>被转换为类型Box，这被称为原始类型（raw type）——原始类型是没有任何类型实参的泛型类或者实例名称。这就是说，你识别不出泛型类在运行时使用的是Object的什么类型。下列操作是不可能的：

```
public class MyClass<E> {
  public static void myMethod(Object item) {
    if (item instanceof E) {    // Compiler error
      ...
    }
    E item2 = new E();          // Compiler error
```

```
    E[] iArray = new E[10];   // Compiler error
    E obj = (E)new Object();  // Unchecked cast warning
  }
}
```

粗体显示的操作在运行时没有意义，因为编译器在编译时删除了和实际类型实参（由类型
178 参数E表示）相关的所有信息。

类型擦除使新代码可以继续和旧代码进行交互。由于任何其他原因而使用原始类型都被认为是不良的编程习惯，应该尽可能避免。

把旧代码和泛型代码混合在一起时，可能遇到类似下面的警告消息：

```
Note: WarningDemo.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
```

在使用对原始类型进行操作的旧的API时，可能出现这种情况，如下面的WarningDemo[1]程序所示：

```
public class WarningDemo {
  public static void main(String[] args){
    Box<Integer> bi;
    bi = createBox();
  }

  /**
   * Pretend that this method is part of an old library,
   * written before generics. It returns
   * Box instead of Box<T>.
   */
  static Box createBox(){
    return new Box();
  }
}
```

使用-Xlint:unchecked重新编译会得到如下额外信息：

```
WarningDemo.java:4: warning: [unchecked] unchecked conversion
found   : Box
required: Box<java.lang.Integer>
       bi = createBox();
                    ^
1 warning
```

6.8　泛型小结

本章描述了下列问题：有一个Box类，我们将它编写为用于一般用途，以便它能够处理
179 Object。我们需要一个只处理Integer的实例。注释说明只处理Integer，所以程序员知道（或者说应该知道）这一点，但是编译器完全不知道。这就是说，编译器不能捕获某些人错误地添加String的情况。当我们读取值并且把它转换为Integer时，就会抛出异常，但是这很不理想，因为从位置和时间来看，异常的出现可能远离错误的发生点：

(1) 调试可能变得困难，因为抛出异常的代码位置可能远离错误代码所在的位置。

(2) 在编译时捕获错误总比在运行时捕获要好。

尤其是，你学习到泛型类型声明可以包含一个或者多个类型参数；使用泛型类型时可以为每个类型参数提供一个类型实参。你还学习到这个类型参数可以用于定义泛型方法和构造器。

---

1. tutorial/java/generics/examples/WarningDemo.java

受限类型参数限制传递给类型参数的类型种类，它们可以只指定上限。通配符代表未知类型，它们可以指定上限或者下限。在编译过程中，类型擦除删除泛型类或者接口中的所有泛型信息，只留下它们的原始类型。这使泛型代码和旧代码进行交互成为可能，但是在很多情况下，编译器会发出警告，通知你使用提供更多详细信息的专门标志重新进行编译。

关于更多信息，请参考Gilad Bracha撰写的*Generics*[1]。

问题和练习：泛型

问题

查看下面的类：

```
public class AnimalHouse<E> {
  private E animal;
  public void setAnimal(E x) {
    animal = x;
  }
  public E getAnimal() {
    return animal;
  }
}

public class Animal{
}
public class Cat extends Animal {
}

public class Dog extends Animal {
}
```

对于下面的代码段，判断代码是否：

(1) 编译失败。

(2) 编译出现警告。

(3) 运行时生成错误。

(4) 不属于上面所有情况（编译和运行都没有问题）。

a. `AnimalHouse<Animal> house = new AnimalHouse<Cat>();`

b. `AnimalHouse<Dog> house = new AnimalHouse<Animal>();`

c.
```
AnimalHouse<?> house = new AnimalHouse<Cat>();
house.setAnimal(new Cat());
```

d.
```
AnimalHouse house = new AnimalHouse();
house.setAnimal(new Dog());
```

练习

设计一个类，作为下列媒体类型的库：书籍、录像和报纸。提供一个使用泛型的类版本和一个不使用泛型的类版本。可以随意使用任何附加API来存储和检索媒体。

答案

可以在以下位置找到“问题”和“练习”的答案：

```
tutorial/java/generics/QandE/generics-answers.html
```

1. tutorial/extra/generics/index.html

第7章 包

本章介绍如何把类和接口打包到包中，如何使用包中的类，以及如何管理文件系统以便编译器能够找到你的源文件。

7.1 创建和使用包

为了更容易地找到和使用类型、避免名称冲突以及控制访问，程序员将一组相关的类型放到包中。

定义 包（package）是一组相关的类型，它可以提供访问保护和名称空间管理。注意，这里的类型（type）指的是类、接口、枚举和注解类型。枚举和注解类型分别是特殊种类的类和接口，所以本章中的类型指的是类和接口（class and interface）。

类型是Java平台的组成部分，它们是按照功能把类打包在一起而形成的各种包的成员：基础类在java.lang中，用于读和写（输入和输出）的类在java.io中，等等。你也可以将自己的类型放进包中。

假设你编写了一组类，它们代表图形对象，比如圆、矩形、直线和点。你还写了一个接口Draggable，如果可以用鼠标拖动类的实例，那么类应该实现此接口：


```
//in the Draggable.java file
public interface Draggable {
    . . .
}

//in the Graphic.java file
public abstract class Graphic {
    . . .
}

//in the Circle.java file
public class Circle extends Graphic implements Draggable {
    . . .
}

//in the Rectangle.java file
public class Rectangle extends Graphic implements Draggable {
    . . .
}

//in the Point.java file
public class Point extends Graphic implements Draggable {
    . . .
}
```

```
//in the Line.java file
public class Line extends Graphic implements Draggable {
    . . .
}
```

应该将这些类和接口打包到一个包中，理由如下：

- 你和其他程序员可以很容易地确定这些类型是相关的。
- 你和其他程序员可以知道到哪里寻找提供图形相关功能的类型。
- 因为包创建了一个新的名称空间，所以你的类型名称不会与其他包中的类型名称发生冲突。
- 可以允许包内的类型不受限制地相互访问，同时仍然严格限制从包外对这些类型的访问。

7.1.1 创建包

为了创建包，需要为包选择一个名称（命名惯例见下一节），并且把带有这个名称的package语句放在包含你希望放在包中的类型（类、接口、枚举和注解类型）的每个源文件的开头。

package语句（例如package graphics;）必须是源文件的第一行。每个源文件中只能有一个package语句，并且它应用于这个文件中的所有类型。 184

注意 如果把多个类型放在单一源文件中，那么只能有一个类型是public的，并且其名称必须和源文件的名称一样。例如，你可以在Circle.java文件中定义public class Circle，在Draggable.java文件中定义public interface Draggable，在Day.java文件中定义public enum Day，等等。

可以在包含公共类型的同一个文件中包含非公共类型（但是，强烈反对这样做，除非非公共类型很小并且和公共类型密切相关），但是从包外只能访问公共类型。所有最高级别的非公共类型都是包私有的。

如果你把前面列出的图形接口和类都放在名为graphics的包中，就需要6个源文件，如下：

```
//in the Draggable.java file
package graphics;
public interface Draggable {
    ...
}

//in the Graphic.java file
package graphics;
public abstract class Graphic {
    ...
}

//in the Circle.java file
package graphics;
public class Circle extends Graphic implements Draggable {
    ...
}

//in the Rectangle.java file
package graphics;
```

```
public class Rectangle extends Graphic implements Draggable {
    ...
}

//in the Point.java file
package graphics;
public class Point extends Graphic implements Draggable {
    ...
}

//in the Line.java file
package graphics;
public class Line extends Graphic implements Draggable {
    ...
}
```

185

如果没有使用package语句，你的类型就包含在未命名的包中。总的来说，未命名的包只用于小型或者临时应用程序，或者在开发过程的初期使用。其他情况下，类和接口应该放在命名的包中。

7.1.2　命名包

因为世界各地的程序员都在使用Java编程语言编写类和接口，所以很可能有两个程序员对不同类型使用相同名称。实际上，上一个例子就是这样：它定义了一个Rectangle类，而java.awt包中已经有了一个Rectangle类。但是，如果两个类在不同的包中，编译器仍然允许两个类同名。每个Rectangle类的完全限定名都包含包的名称。也就是说，graphics包中的Rectangle类的完全限定名是graphics.Rectangle，而java.awt包中的Rectangle类的完全限定名是java.awt.Rectangle。

这种机制一般很有效，除非两个程序员为他们的包使用相同的名称。如何避免这个问题呢？回答是按照惯例。

命名惯例

包名称都使用小写字母，以避免和类名称或者接口名称发生冲突。

公司使用它们的因特网域名的反向顺序形式作为它们包名称的开头，例如，com.example.orion表示example.com公司的程序员创建的名为orion的包。

出现在一个公司内的命名冲突需要按照这个公司内部的惯例来处理，可以在公司名称后面加上部门或者工程的名称（比如com.company.region.package）。

Java语言本身中的包用java.或者javax.开头。

186　在某些情况下，因特网域名可能不是合法的包名称。例如，如果域名包含连字符或者其他特殊字符，包名称以数字或者其他不能作为Java名称开头的字符开头，或者包名称包含Java保留关键字（比如“int”），就会出现这种情况。在这种情况下，我们建议的惯例是添加下划线（见表7-1）。

表7-1　使用特殊域名作为包名称

域　名	包名称前缀
clipart-open.org	org.clipart_open
free.fonts.int	int_.fonts.free
poetry.7days.com	com._7days.poetry

7.1.3 使用包成员

构成包的类型被称为包成员（package member）。

要从包外使用public包成员，必须采用以下做法之一：

- 用成员的完全限定名称引用它。
- 导入包成员。
- 导入成员所属的整个包。

这些做法分别适用不同的情况，下面分别解释。

1. 通过完全限定名称引用包成员

到目前为止，本书中的大多数例子一直使用简单名称（比如Rectangle和StackOfInts）引用类型。如果你正在编写的代码与它要使用的成员在相同的包中，或者成员已经被导入，那么可以使用包成员的简单名称。

但是，如果你打算使用另一个包中的成员，而且这个包还没有被导入，那么必须使用成员的完全限定名称，这其中包含包的名称。在前一个例子中，graphics包中声明的Rectangle类的完全限定名称是：

```
graphics.Rectangle
```

可以使用这个完全限定名称创建graphics.Rectangle的实例：

```
graphics.Rectangle myRect = new graphics.Rectangle();
```

187

在不经常使用名称的情况下，限定名称很好用。但是，当频繁地使用名称时，重复地输入名称就变得很麻烦，并且代码难以阅读。一种替换方案是，可以导入（import）成员或者它的包，然后使用它的简单名称。

2. 导入包成员

为了把特定的成员导入到当前文件中，应该在文件的开头，任何类或接口的定义之前，但在package语句（如果有的话）之后放一个import语句。下面的代码导入前一节创建的graphics包中的Rectangle类：

```
import graphics.Rectangle;
```

现在，可以使用简单名称引用Rectangle类：

```
Rectangle myRectangle = new Rectangle();
```

如果你只需要使用graphics包中的几个成员，那么这种方法很合适。但是，如果要使用一个包中的许多类型，那么就应该导入整个包。

3. 导入整个包

要导入某个包中包含的所有类型，应该使用带有星号（*）通配符的import语句：

```
import graphics.*;
```

现在，可以使用简单名称引用graphics包中的任何类或接口：

```
Circle myCircle = new Circle();
Rectangle myRectangle = new Rectangle();
```

如上所示，import语句中的星号只能用于指定一个包中的所有类。它不能用于匹配包中的类的子集。例如，下面的代码行并不匹配graphics包中所有以“A”开头的类：

```
import graphics.A*;     // does not work
```

相反，它导致一个编译器错误。用import语句只能导入单个包成员或整个包。

188

注意　import语句的另一种不常见的形式允许你导入类中的公共嵌套类。例如，如果graphics.Rectangle类包含有用的嵌套类（比如Rectangle.DoubleWide和Rectangle.Square），你可以使用下面两个语句导入Rectangle及其嵌套类：

```
import graphics.Rectangle;
import graphics.Rectangle.*;
```

注意，第二个import语句不会导入Rectangle。

import的另一种不常见的形式是静态导入语句（static import statement），这将在本节末尾讨论。

按照惯例，Java编译器自动地为每个源文件导入3个完整包：（1）没有名称的包；（2）java.lang包；（3）当前包（当前文件所在的包）。

4. 包没有层次结构

最初看上去，包好像具有层次结构，但其实不然。例如，Java API包含java.awt包、java.awt.color包、java.awt.font包以及很多以java.awt开头的包。但是，java.awt.color包、java.awt.font包和其他java.awt.xxxx包都不包含在java.awt包之内。前缀java.awt（Java Abstract Window Toolkit，Java抽象窗口工具包）用于很多相关包，只是为了使它们的关系更明显，但是不表示它们有包含关系。

导入java.awt.*就导入了java.awt包中的所有类型，但是不会导入java.awt.color、java.awt.font或者任何其他java.awt.xxxx包。如果你计划使用java.awt.color包中的类和其他类型，同时还要使用java.awt包中的类型，就必须导入这两个包中的所有文件：

```
import java.awt.*;
import java.awt.color.*;
```

5. 名称多义性

如果两个包中都具有某个名称相同的成员，而且这两个包都被导入了，那么你必须用完全限定名称引用每个成员。例如，在graphics包中定义了一个Rectangle类。java.awt包中也包含一个Rectangle类。如果graphics包和java.awt包都被导入了，那么下面的代码具有多义性：

```
Rectangle rect;
```

189

在这种情况下，必须使用成员的完全限定名明确指出你要使用哪个Rectangle类。例如：

```
graphics.Rectangle rect;
```

6. 静态导入语句

有这样的情况：你需要频繁地访问一个或者两个类的静态最终字段（常量）和静态方法。一再地为这些类的名称加上前缀会导致代码混乱。静态导入（static import）语句提供一种途径，使你可以导入希望使用的常量和静态方法，而不用为这些类的名称加上前缀。

java.lang.Math类定义PI常量和很多静态方法，包括用于计算正弦、余弦、正切、平方根、最大值、最小值、指数和很多其他用途的方法。例如：

```
public static final double PI 3.141592653589793
public static double cos(double a)
```

通常，为了从另一个类中使用这些对象，需要为类名称加上前缀，如下：

```
double r = Math.cos(Math.PI * theta);
```

你可以使用静态导入语句导入java.lang.Math的静态成员，以避免为类名称加上前缀Math。Math的静态成员可以分别导入：

```
import static java.lang.Math.PI;
```

也可以成批导入：

```
import static java.lang.Math.*;
```

被导入之后，就可以不带限定地使用静态成员。例如，前面的代码段就变成：

```
double r = cos(PI * theta);
```

显然，你可以编写一个自己的类，其中包含频繁使用的常量和静态方法，然后使用静态导入语句。例如：

```
import static mypackage.MyConstants.*;
```

190

注意 要非常保守地使用静态导入。过度使用静态导入会导致代码难以阅读和维护，因为阅读代码的人不知道哪个类定义了特定的静态对象。适当使用静态导入，通过消除重复的类名称，会使代码的可读性更高。

7.1.4 管理源文件和类文件

Java平台的很多实现依靠层次化的文件系统管理源文件和类文件，虽然Java语言规范（Java Language Specification）并未要求这么做。其策略如下：

把类、接口、枚举或者注解类型的源代码放在文本文件中，这个文本文件的名称是类型的简单名称，扩展名为.java。例如：

```
// in the Rectangle.java file
package graphics;
public class Rectangle() {
  ...
}
```

然后，把源文件放在一个目录中，这个目录的名称反映类型所属的包的名称：

```
...\graphics\Rectangle.java
```

包成员的限定名称和保存文件的路径名称是对应的，使用Microsoft Windows文件名称的反斜线分隔符（在UNIX系统中使用正斜线），如下：

类名称	graphics.Rectangle
文件路径	graphics\Rectangle.java

前面讲过，按照惯例，一个公司使用其因特网域名的反向顺序形式作为其包名。Example公司的因特网域名为example.com，它就在其所有包名称前面加上com.example。包名称的每个部分对应一个子目录。所以，如果Example公司有一个包含Rectangle.java源文件的com.example.graphics包，它就应该保存在如下的子目录序列中：

```
...\com\example\graphics\Rectangle.java
```

191

当你编译源文件时，编译器为其中定义的每个类型创建不同的输出文件。输出文件的基本名就是类型的名称，其扩展名是.class。例如，如果源文件是：

```
// in the Rectangle.java file
package com.example.graphics;
public class Rectangle{
  ...
}
class Helper{
  ...
```

```
}
```

那么编译后的文件就应该位于：

```
<path to the parent directory of the output files>\com\example\graphics\Rectangle.class

<path to the parent directory of the output files>\com\example\graphics\Helper.class
```

和.java源文件一样，编译后的.class文件应该位于反映包名称的一系列目录中。但是，.class文件不必和.java源文件位于相同路径。你可以分别管理源文件和类文件路径，比如：

```
<path_one>\sources\com\example\graphics\Rectangle.java

<path_two>\classes\com\example\graphics\Rectangle.class
```

通过这样管理，可以把classes目录交给其他程序员，而无需暴露你的源文件。你还需要通过相同的方式管理源文件和类文件，以便使编译器和Java虚拟机（Java Virtual Machine，JVM）能够找到你的程序所需的所有类型。

classes目录的完整路径<path_two>\classes被称为类路径（class path），使用CLASSPATH系统变量设置。编译器和JVM都通过把包名称添加到类路径来构建.class文件的路径。例如，如果<path_two>\classes是你的类路径，而包名称是com.example.graphics，那么编译器和JVM就在<path_two>\classes\com\example\graphics中查找.class文件。

192

类路径可能包含若干路径，中间以分号（Windows系统）或者冒号（UNIX系统）分隔。默认情况下，编译器和JVM搜索当前目录和包含Java平台类的JAR文件，所以这些目录自动包含在你的类路径中。

设置CLASSPATH系统变量

为了显示当前的CLASSPATH变量，在Windows和UNIX（Bourne shell）系统中使用如下命令：

```
In Windows:    C:\> set CLASSPATH
In Unix:       % echo &#36CLASSPATH
```

为了删除CLASSPATH变量的当前内容，使用如下命令：

```
In Windows:    C:\> set CLASSPATH=
In Unix:       % unset CLASSPATH; export CLASSPATH
```

为了设置CLASSPATH变量，使用如下命令（其中的目录名称只用于演示）：

```
In Windows:    C:\> set CLASSPATH=C:\users\george\java\classes
In Unix:       % CLASSPATH=/home/george/java/classes; export CLASSPATH
```

7.1.5　创建和使用包小结

要为类型创建包，需要把package语句放在包含这个类型（类、接口、枚举或者注解类型）的源文件的第一行。

为了使用位于不同包中的公共类型，有3种方法：(1) 使用类型的完全限定名称，(2) 导入类型，(3) 导入类型所属的整个包。

包的源文件和类文件的路径名称反映包的名称。

193

可能必须设置CLASSPATH变量，以便使编译器和JVM能够找到类型的.class文件。

问题和练习：创建和使用包

问题

假设你已经编写了一些类。后来，你决定把它们分开放到3个包中，如表7-2所示。另外，假

设当前这些类在默认包中（即，它们没有使用package语句）。

表7-2 包名称和类名称

包名称	类名称
mygame.server	Server
mygame.shared	Utilities
mygame.client	Client

1. 为了把每个类放在正确的包中，你需要在每个源文件中添加什么代码？
2. 为了遵守目录结构的规定，需要在开发目录中创建一些子目录，然后把源文件存放在正确的子目录中。必须创建哪些子目录？把每个源文件存放到哪个子目录中？
3. 为了使源文件能够正确地编译，你认为需要对源文件作出任何其他改动吗？如果需要改动，要改动什么？

练习

下载下面列出的源文件：

- **Client:** `tutorial/java/package/QandE/question/Client.java`
- **Server:** `tutorial/java/package/QandE/question/Server.java`
- **Utilities:** `tutorial/java/package/QandE/question/Utilities.java`

1. 对刚刚下载的源文件进行你在问题1到3中考虑的改动。
2. 编译修改后的源文件。（提示：如果你从命令行调用编译器[而不是使用开发工具]，就从包含你刚刚创建的mygame目录的目录调用编译器。）

答案

可以在以下位置找到“问题”和“练习”的答案：

`tutorial/java/package/QandE/packages-answers.html`

第8章 数字和字符串

8.1 数字

在本章的开始，将讨论java.lang包中的Number[1]类及其子类，以及在什么情况下应该使用这些类的实例，而不使用基本数据类型。

这一节还介绍PrintStream[2]和DecimalFormat[3]类，它们提供用于编写格式化数字输出的方法。

最后讨论java.lang包中的Math[4]类。它包含的算术函数补充了语言内置的操作符。这个类包含的方法用于三角函数、指数函数等。

8.1.1 数字类

处理数字时，你的代码中最常用的是基本数据类型。例如：

```
int i = 500;
float gpa = 3.65;
byte mask = 0xff;
```

但是出于某些原因，可能需要在使用基本数据类型的位置使用对象，因此Java平台为每种基本数据类型提供了包装器（wrapper）类。这些类把基本数据类型“包装”在一个对象中。包装操作通常是由编译器完成的；如果你在需要对象的地方使用了原始类型，编译器就替你把原始类型装箱（box）到包装器类中。类似的，如果你在需要原始类型的地方使用了数字对象，编译器就替你将对象拆箱（unbox）。

下面是装箱和拆箱的一个例子：

```
Integer x, y;
x = 12;
y = 15;
System.out.println (x+y) ;
```

当x和y被赋为整数值时，编译器对整数进行装箱，因为x和y是整数对象。在println()语句中，x和y被拆箱，以便作为整数相加。

所有数字包装器类都是抽象类Number的子类，如图8-1所示。

注意 Number还有4个其他子类，这里没有讨论。BigDecimal和BigInteger用于高精度计算。AtomicInteger和AtomicLong用于多线程应用程序。

1. docs/api/java/lang/Number.html
2. docs/api/java/io/PrintStream.html
3. docs/api/java/text/DecimalFormat.html
4. docs/api/java/lang/Math.html

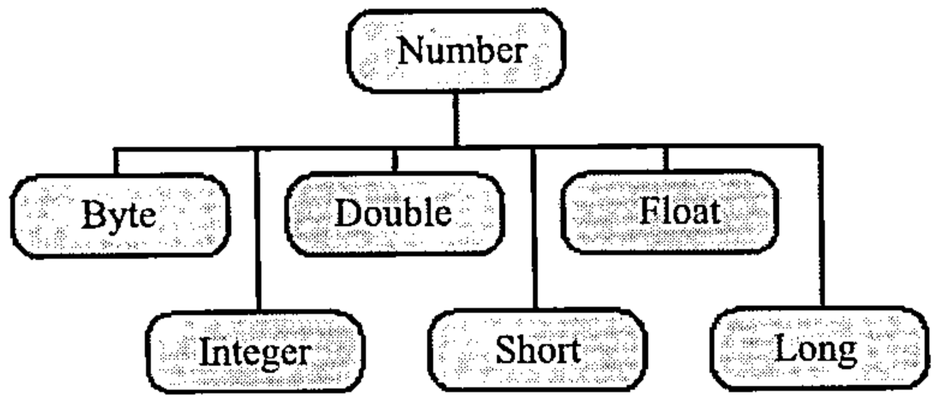


图8-1 Number的类层次结构

196

出于下面三个原因，你可能使用Number对象，而不使用原始类型：

- 当方法的实参需要对象时（通常在操作数字的集合时使用）。
- 需要使用类定义的常量，比如MIN_VALUE和MAX_VALUE，它们定义这个数据类型的上限和下限。
- 需要使用类方法，将值转换成其他原始类型（或者相反）、转换成字符串（或者相反）以及在数字系统（十进制、八进制、十六进制和二进制）之间进行转换。

表8-1列出了Number类的所有子类都实现的实例方法。

表8-1 Number类的所有子类实现的方法

方 法	描 述
byte byteValue() short shortValue() int intValue() long longValue() float floatValue() double doubleValue()	把这个Number对象的值转换为基本数据类型，并且返回
int compareTo (Byte anotherByte) int compareTo (Double anotherDouble) int compareTo (Float anotherFloat) int compareTo(Integer anotherInteger) int compareTo (Long anotherLong) int compareTo (Short anotherShort)	把这个Number对象和实参进行比较
boolean equals (Object obj)	确定这个数字对象是否等于实参。如果实参不是null，并且实参是属于相同类型的对象，并且具有相同的数字值，则这个方法返回true。对Double和Float对象有一些额外要求，详细介绍参见Java API文档

每个Number类都包含另一些方法，用于在数字和字符串之间以及在数字系统之间进行转换。表8-2列出Integer类中的这些方法。另一些Number子类中的方法与之类似。

197

表8-2 Integer类的转换方法

方 法	描 述
static Integer decode (String s)	把字符串解码为整数。可以接受十进制、八进制或者十六进制数字的字符串表示作为输入
static int parseInt (String s)	返回整数（只有十进制）
static int parseInt (String s, int radix)	返回整数，接受十进制、二进制、八进制或者十六进制数字（radix分别等于10、2、8或者16）的字符串表示作为输入
String toString()	返回表示这个整数值的String对象
static String toString (int i)	返回表示指定的整数的String对象
static Integer valueOf (int i)	返回包含指定的原始类型值的Integer对象

（续）

方　法	描　述
static Integer valueOf (String s)	返回包含指定的字符串表示的值的Integer对象
static Integer valueOf(String s, int radix)	返回包含指定的字符串表示的整数值的Integer对象，按照radix的值进行解析。例如，如果s = "333"并且radix = 8，那么这个方法就返回与八进制数字333相等的以10为基数的整数

8.1.2　格式化数字输出

前面你见到过使用print()和println()方法把字符串发送到标准输出（System.out）。因为所有数字都能被转换为字符串（本章后面将介绍），你可以使用这些方法输出字符串和数字的任意混合形式。但是，Java编程语言具有其他方法，使你在处理数字时对输出有更多的控制能力。

1. printf()和format()方法

java.io包包含PrintStream类，它有两个格式化方法，可以用于替换print()和println()。这两个方法是format()和printf()，它们是相互等效的。你曾经使用过的熟悉的System.out正好是PrintStream对象，所以你可以对System.out调用PrintStream方法。因此，在以前的代码中用到print()或者println()的任何位置，你都可以使用format()或者printf()。例如：

198

```
System.out.format (.....) ;
```

这两个java.io.PrintStream方法的语法相同：

```
public PrintStream format (String format, Object... args)
```

其中format指定要使用的格式，而args是要使用这个格式输出的变量清单。下面是一个例子：

```
System.out.format ("The value of the float variable is %f, " +
                   "while the value of the integer variable " +
                   "is %d, and the string is %s",
                   floatVar, intVar, stringVar) ;
```

第一个参数format是指定如何对第二个参数args进行格式化的格式化字符串。格式化字符串包含纯文本和格式化说明符（format specifier），格式化说明符是用于格式化实参Object...args的特殊字符。Object...args被称为可变参数（vararg），它表示实参的数量可能变化。

格式化说明符以百分号（%）开头，以转换符号（conversion）结束。转换符号是指定要格式化的实参类型的字符。在百分号（%）和转换符号之间可以加上可选的标志和说明符。有很多转换符号、标志和说明符，在java.util.Formatter[1]的文档中介绍了它们。

下面是一个例子：

```
int i = 461012;
System.out.format ("The value of i is: %d%n", i) ;
```

%d说明单一变量是十进制整数。%n是与平台无关的换行字符。代码的输出是：

```
The value of i is: 461012
```

printf()和format()方法被重载了。每个方法都有使用如下语法的版本：

199

```
public PrintStream format (Locale l, String format,
                                          Object... args)
```

---

1. docs/api/java/util/Formatter.html

例如，为了在法语系统（其中，在浮点数的英语表示法中使用小数点的位置使用逗号）中输出数字，可以使用：

```
System.out.format
   (Locale.FRANCE, "The value of the float variable is %f, " +
    "while the value of the integer variable is %d, and the " +
    "string is %s%n", floatVar, intVar, stringVar);
```

2. 示例

表8-3列出示例程序TestFormat.java中用到的一些转换符号和标志。

表8-3 TestFormat.java中用到的转换符号和标志

转换字符	标 志	解 释
d		十进制整数
f		浮点数
n		适合于运行应用程序的平台的换行字符。总应该使用%n，而不使用\n
tB		日期和时间转换符字——地区特定的月份的完整名称
td、te		日期和时间转换符字——2位数字的月的日期。如果需要，td具有前导的0，te则没有
ty、tY		日期和时间转换符字——ty＝2位年份，tY＝4位年份
tl		日期和时间转换符字——以12小时方式表示小时
tM		日期和时间转换符字——以2位表示分钟，如果需要，则带有前导的0
tp		日期和时间转换符字——地区特定的am / pm（小写）
tm		日期和时间转换符字——以2位表示月份，如果需要，则带有前导的0
tD		日期和时间转换符字——日期表示为%tm%td%ty
08		字符长度为8位，如果需要，则带有前导的0
+		包含符号，正或者负
,		包含地区特定的分组符号
-		左对齐
.3		小数点后有3位
10.3		字符长度为10位，右对齐，小数点后有3位

下面的程序演示你可以使用format()进行的一些格式化操作。必须导入java.util包，因为它包含Locale和Calendar类：

```
import java.util.*;

public class TestFormat {

  public static void main(String[] args) {
    long n = 461012;
    System.out.format("%d%n", n);
    System.out.format("%08d%n", n);
    System.out.format("%+8d%n", n);
    System.out.format("%,8d%n", n);
    System.out.format("%+,8d%n%n", n);

    double pi = Math.PI;
    System.out.format("%f%n", pi);
    System.out.format("%.3f%n", pi);
    System.out.format("%10.3f%n", pi);
    System.out.format("%-10.3f%n", pi);
    System.out.format(Locale.FRANCE, "%-10.4f%n%n", pi);

    Calendar c = Calendar.getInstance();
```

```
    System.out.format("%tB %te, %tY%n", c, c, c);
    System.out.format("%tl:%tM %tp%n", c, c, c);
    System.out.format("%tD%n", c);
  }
}
```

程序的输出为：

```
461012
00461012
 +461012
 461,012
+461,012

3.141593
3.142
   · 3.142
3.142
3,1416

May 29, 2006
2:34 am
05/29/06
```


注意　本节中仅仅介绍了format()和printf()方法的基本应用。进一步的详细信息请参考10.1.4节中的第2小节。

3. DecimalFormat类

可以使用java.text.DecimalFormat控制前导0和末尾的0、前缀和后缀、分组（千位）分隔符和小数分隔符的显示。DecimalFormat为数字的格式化提供了非常大的灵活性，但是它会使代码变得更加复杂。

下面的例子通过把模式字符串传递给DecimalFormat构造器，从而创建DecimalFormat对象myFormatter。然后，myFormatter调用DecimalFormat从NumberFormat继承而来的format()方法，这个方法接受double值作为实参，并且返回格式化后的数字字符串。

下面是演示DecimalFormat使用方法的一个示例程序：

```
import java.text.*;

public class DecimalFormatDemo {

  static public void customFormat(String pattern,
     double value ) {
    DecimalFormat myFormatter = new DecimalFormat(pattern);
    String output = myFormatter.format(value);
    System.out.println(value + "  " + pattern + "  " + output);
  }

  static public void main(String[] args) {

    customFormat("###,###.###", 123456.789);
    customFormat("###.##", 123456.789);
    customFormat("000000.000", 123.78);
    customFormat("$###,###.###", 12345.67);
  }
}
```


这个程序的输出是：

```
123456.789  ###,###.###  123,456.789
123456.789  ###.##  123456.79
123.78  000000.000  000123.780
12345.67  $###,###.###  $12,345.67
```

表8-4解释每行输出。

表8-4 DecimalFormat.java的输出

值	样 式	输 出	解 释
123456.789	###,###.###	123,456.789	磅符号（#）表示一位数字，逗号是分组分隔符的占位符，点号是小数分隔符的占位符
123456.789	###.##	123456.79	value在小数点右边有三位，但是pattern只有两位。format方法通过向上取整处理此情况
123.78	000000.000	000123.780	pattern指定前导和末尾的0，因为用字符0替代了磅符号（#）
12345.67	$###,###.###	$12,345.67	pattern中的第一个字符是美元符号（$）。注意，它位于格式化后的output的最左一位前面

8.1.3 高级算术功能

Java编程语言通过其算术操作符（+、-、×、/和%）支持基本算术功能。java.lang包中的Math类提供的方法和常量用于更加高级的数学运算。

Math类中的方法都是静态的，所以你可以直接从这个类调用它们，比如：

```
Math.cos(angle);
```

注意 通过使用static import语言特性（参见7.1.3节中的第6小节），你不必在每个算术函数之前都加上Math：

```
import static java.lang.Math.*;
```

这使你可以通过它们的简单名称调用Math类的方法。例如：

```
cos(angle);
```

203

1. 常量和基本方法

Math类包含两个常量：

- Math.E——它是自然对数的基数。
- Math.PI——它是圆的周长与其直径的比值。

Math类还包含40个以上的静态方法。表8-5列出其中的一些基本方法。

表8-5 基本Math方法

方 法	描 述
double abs (double d) float abs (float f) int abs (int i) long abs (long lng)	返回实参的绝对值
double ceil (double d)	返回大于或者等于实参的最小整数。返回类型为双精度
double floor (double d)	返回小于或者等于实参的最大整数。返回类型为双精度
double rint (double d)	返回最接近实参值的整数。返回类型为双精度
long round (double d) int round (float f)	返回最接近实参的long或者int，由方法的返回类型决定

（续）

方　法	描　述
double min (double arg1, double arg2)	返回两个实参中比较小的那个
float min (float arg1, float arg2)	
int min (int arg1, int arg2)	
long min (long arg1, long arg2)	
double max (double arg1, double arg2)	返回两个实参中比较大的那个
float max (float arg1, float arg2)	
int max (int arg1, int arg2)	
long max (long arg1, long arg2)	

204

下面的程序BasicMathDemo[1]演示其中一些方法的使用。

```
public class BasicMathDemo {
  public static void main(String[] args) {
    double a = -191.635;
    double b = 43.74;
    int c = 16, d = 45;

    System.out.printf("The absolute value of %.3f is %.3f%n",
                                                a, Math.abs(a));
    System.out.printf("The ceiling of %.2f is %.0f%n",
                                                b, Math.ceil(b));
    System.out.printf("The floor of %.2f is %.0f%n",
                                               b, Math.floor(b));
    System.out.printf("The rint of %.2f is %.0f%n",
                                                b, Math.rint(b));
    System.out.printf("The max of %d and %d is %d%n",
                                             c, d, Math.max(c, d));
    System.out.printf("The min of of %d and %d is %d%n",
                                             c, d, Math.min(c, d));
  }
}
```

下面是这个程序的输出：

```
The absolute value of -191.635 is 191.635
The ceiling of 43.74 is 44
The floor of 43.74 is 43
The rint of 43.74 is 44
The max of 16 and 45 is 45
The min of 16 and 45 is 16
```

2. 指数和对数方法

表8-6列出了Math类的指数和对数方法。

表8-6　指数和对数方法

方　法	描　述
double exp (double d)	返回以自然对数e为基数，以实参为幂的结果
double log (double d)	返回实参的自然对数
double pow (double base, double exponent)	返回以第一个实参为基数，以第二个实参为幂的结果
205 double sqrt (double d)	返回实参的平方根

1. tutorial/java/data/examples/BasicMathDemo.java

下面的程序ExponentialDemo[1]显示e的值，然后对随意选择的数字调用表8-6中列出的每个方法：

```
public class ExponentialDemo {

  public static void main(String[] args) {

    double x = 11.635;
    double y = 2.76;

    System.out.printf("The value of e is %.4f%n", Math.E);
    System.out.printf("exp(%.3f) is %.3f%n", x, Math.exp(x));
    System.out.printf("log(%.3f) is %.3f%n", x, Math.log(x));
    System.out.printf("pow(%.3f, %.3f) is %.3f%n",
                                          x, y, Math.pow(x, y));
    System.out.printf("sqrt(%.3f) is %.3f%n", x, Math.sqrt(x));
  }
}
```

下面是运行ExponentialDemo的输出：

```
The value of e is 2.7183
exp(11.635) is 112983.831
log(11.635) is 2.454
pow(11.635, 2.760) is 874.008
sqrt(11.635) is 3.411
```

3. 三角方法

Math类还提供一组三角函数，表8-7总结了这些方法。传递给每个方法的值是以弧度表示的角度。你可以使用toRadians方法把角度转换为弧度。

206

表8-7 三角方法

方 法	描 述
double sin (double d)	返回指定双精度值的正弦值
double cos (double d)	返回指定双精度值的余弦值
double tan (double d)	返回指定双精度值的正切值
double asin (double d)	返回指定双精度值的反正弦值
double acos (double d)	返回指定双精度值的反余弦值
double atan (double d)	返回指定双精度值的反正切值
double atan2 (double y, double x)	把直角坐标（x, y）转换为极坐标（r, theta）并且返回theta
double toDegrees (double d)	把实参转换为角度或者弧度
double toRadians (double d)	

下面的程序TrigonometricDemo[2]使用上面的每个方法计算45度角的各个三角函数值：

```
public class TrigonometricDemo {
  public static void main(String[] args) {
    double degrees = 45.0;
    double radians = Math.toRadians(degrees);

    System.out.format("The value of pi is %.4f%n", Math.PI);
    System.out.format("The sine of %.1f degrees is %.4f%n",
      degrees, Math.sin(radians));
    System.out.format("The cosine of %.1f degrees is %.4f%n",
```

1. tutorial/java/data/examples/ExponentialDemo.java
2. tutorial/java/data/examples/TrigonometricDemo.java

```
        degrees, Math.cos(radians));
    System.out.format("The tangent of %.1f degrees is %.4f%n",
        degrees, Math.tan(radians));
    System.out.format("The arcsine of %.4f is %.4f degrees %n",
        Math.sin(radians),
        Math.toDegrees(Math.asin(Math.sin(radians))));
    System.out.format("The arccosine of %.4f is %.4f " +
        degrees %n", Math.cos(radians),
        Math.toDegrees(Math.acos(Math.cos(radians))));
    System.out.format("The arctangent of %.4f is %.4f " +
        degrees %n", Math.tan(radians),
        Math.toDegrees(Math.atan(Math.tan(radians))));
  }
}
```

207

这个程序的输出如下：

```
The value of pi is 3.1416
The sine of 45.0 degrees is 0.7071
The cosine of 45.0 degrees is 0.7071
The tangent of 45.0 degrees is 1.0000
The arcsine of 0.7071 is 45.0000 degrees
The arccosine of 0.7071 is 45.0000 degrees
The arctangent of 1.0000 is 45.0000 degrees
```

4. 随机数

random()方法返回一个0.0到1.0之间的伪随机数。这个范围包括0.0，但不包括1.0。换句话说就是0.0 <= Math.random() < 1.0。要得到另一个范围内的随机数，可以对random方法的返回值执行算术操作。例如，下面的代码产生1~9的随机整数：

```
int number = (int) (Math.random() * 10);
```

通过将random方法的返回值乘以10，可能值的范围变成0.0 <= number < 10.0。编译器对原始双精度值（Math.random() * 10）进行装箱，然后使用Double类的intValue()方法把它转换为原始类型。

如果你需要生成单一随机数，使用Math.random就很好。如果你需要生成一系列随机数，就应该创建java.util.Random的实例，然后对这个对象调用方法以便生成数字。

8.1.4　数字小结

使用一个包装器类（Byte、Double、Float、Integer、Long或者Short）把原始类型的数字包装到对象中。如果必要的话，Java编译器自动地为你包装（装箱）原始类型或者对它们进行拆箱。

Number类包含常量和有用的类方法。MIN_VALUE和MAX_VALUE常量包含一个类型的对象能够包含的最小值和最大值。byteValue、shortValue和类似方法把一种数字类型转换为另一种。valueOf方法把字符串转换为数字，而toString方法把数字转换为字符串。

为了格式化包含数字的字符串以便输出，可以使用PrintStream类中的printf()和format()方法。另一种途径是使用NumberFormat类，使用模式定制数字格式。

Math类包含很多类方法，用于执行数学计算功能，包括指数、对数和三角方法。Math还包含基本算术功能，比如绝对值和舍入，还有用于生成随机数的random()方法。

208

问题和练习：数字

问题

1. 使用API文档查找下列问题的答案：

a. 可以使用哪个Integer方法把int转换为以十六进制表示数字的字符串？例如，哪个方法把整数65转换为字符串“41”？
b. 使用哪个Integer方法把以5为基数表示的字符串转换为相等的int？例如，如何把字符串“230”转换为整数值65？写出完成这个任务的代码。
c. 使用哪个Double方法检查一个浮点数是否具有特殊值NaN（Not a Number，非数字）？

2. 以下表达式的值是什么？为什么？

```
Integer.valueOf(1).equals(Long.valueOf(1))
```

练习

1. 改动下面的MaxVariablesDemo[1]，显示最小值，而不是最大值。你可以删除与变量aChar和aBoolean相关的所有代码。输出是什么？

```
public class MaxVariablesDemo {
    public static void main(String args[]) {

        //integers
        byte largestByte = Byte.MAX_VALUE;
        short largestShort = Short.MAX_VALUE;
        int largestInteger = Integer.MAX_VALUE;
        long largestLong = Long.MAX_VALUE;

        //real numbers
        float largestFloat = Float.MAX_VALUE;
        double largestDouble = Double.MAX_VALUE;

        //other primitive types
        char aChar = 'S';
        boolean aBoolean = true;

        //Display them all.
        System.out.println("The largest byte value is "
                           + largestByte + ".");
        System.out.println("The largest short value is "
                           + largestShort + ".");
        System.out.println("The largest integer value is "
                           + largestInteger + ".");
        System.out.println("The largest long value is "
                           + largestLong + ".");

        System.out.println("The largest float value is "
                           + largestFloat + ".");
        System.out.println("The largest double value is "
                           + largestDouble + ".");

        if (Character.isUpperCase(aChar)) {
            System.out.println("The character " + aChar
                               + " is uppercase.");
        } else {
            System.out.println("The character " + aChar
                               + " is lowercase.");
        }
        System.out.println("The value of aBoolean is "
                           + aBoolean + ".");
    }
}
```

---

1. tutorial/java/data/QandE/MaxVariablesDemo.java

2. 创建一个程序，从命令行读取数量不确定的整数实参，把它们相加。例如，假设输入：

```
java Adder 1 3 2 10
```

程序应该显示16，然后退出。如果用户只输入一个实参，程序应该显示错误消息。可以在ValueOfDemo（参见8.3.5节的第1小节）的基础上编写你的程序。

209 3. 创建一个和前一个程序类似的程序，二者的不同之处在于：

- 不读取整数实参，而是读取浮点数实参。
- 它显示实参的总和，结果精确到小数点右两位。

例如，如果输入：

```
java FPAdder 1 1e2 3.0 4.754
```

程序就应该显示108.75。根据你所在的地区，小数点可能是逗号（,）而不是点号（.）。

答案

可以在以下位置找到“问题”和“练习”的答案：

```
tutorial/java/data/QandE/numbers-answers.html
```

8.2 字符

在大多数情况下，如果你使用单一字符值，可以使用原始char类型，例如：

```
char ch = 'a';
char uniChar = '\u039A';  // Unicode for uppercase Greek
                          // omega character
char[] charArray ={ 'a', 'b', 'c', 'd', 'e' };  // an array
                                                // of chars
```

但是有时候，你需要使用一个字符作为对象，比如要求方法实参是对象的时候。Java编程语言提供包装器（wrapper）类，它把char“包装”到Character对象中来达到这一目的。类型为Character的对象包含单一字段，其类型为char。这个Character[1]类还提供很多有用的类方法（即静态方法）用于处理字符。

可以使用Character构造器创建Character对象。

210

```
Character ch = new Character('a');
```

在某些情况下，Java编译器也会为你创建Character对象。例如，如果你把原始类型char传递给要求对象作为参数的方法，编译器就自动地为你把char转换为Character。这个特性被称为自动装箱（autobox）；如果做相反方向的转换，就称为拆箱（unbox）。

下面是装箱的一个例子：

```
Character ch = 'a';   // the primitive char 'a'
                      // is boxed into the Character object ch
```

下面是装箱和拆箱的例子：

```
Character test(Character c) {...}   // method parameter and
                                    // return type =
                                    // Character object

char c = test('x');      // primitive 'x' is boxed for method test,
                         // return is unboxed to char 'c'
```

1. docs/api/java/lang/Character.html

注意 Character类是不可变的，所以创建之后，不能改动Character对象。

表8-8列出Character类中一些最有用的方法，但是没有全部列出。这个类中方法的完整清单（方法超过50个）请参阅java.lang.Character **API**规范。

表8-8 Character类中有用的方法

方 法	描 述
boolean isLetter (char ch)	分别判断指定的字符值是否为字母或者数字
boolean isDigit (char ch)	
boolean isWhitespace (char ch)	判断指定的字符值是否为空白
boolean isUpperCase (char ch)	分别判断指定的字符值是否为大写或者小写
boolean isLowerCase (char ch)	
char toUpperCase (char ch)	返回指定字符值的大写或者小写形式
char toLowerCase (char ch)	
toString (char ch)	返回表示指定字符值的String对象——即包含一个字符的字符串

211

转义序列

前面加上反斜线（\）的字符是转义序列（escape sequence），对编译器具有特殊含义。在本书的System.out.println()语句中经常使用换行字符（\n）在输出字符串之后转入下一行。表8-9显示Java的转义序列。

表8-9 转义序列

转义序列	描 述
\t	在文本中的这个位置插入制表符
\b	在文本中的这个位置插入退格
\n	在文本中的这个位置插入换行
\r	在文本中的这个位置插入回车
\f	在文本中的这个位置插入换页
\'	在文本中的这个位置插入单引号字符
\"	在文本中的这个位置插入双引号字符
\\	在文本中的这个位置插入反斜线字符

在输出语句中遇到转义序列时，编译器相应地解释它。例如，如果你希望在引号内插入引号，就必须在引号之内使用转义序列\"。要输出下面这个句子：

```
She said "Hello!" to me.
```

可以写为：

```
System.out.println ("She said \"Hello!\" to me.") ;
```

8.3 字符串

字符串被广泛地应用在Java编程中，字符串是字符序列。在Java编程语言中字符串是对象。Java平台提供String[1]类用于创建和处理字符串。

212

1. docs/api/java/lang/String.html

8.3.1　创建字符串

创建字符串最直接的方式是：

```
String greeting = "Hello world!";
```

在这个例子中，“Hello world!”是字符串字面量（string literal）——代码中用双引号括起来的一系列字符。遇到代码中的字符串字面量时，编译器就会使用其值创建String对象——在这个例子中就是Hello world!。

和任何其他对象一样，可以使用new关键字和构造器来创建String对象。String对象有11个构造器，允许你使用不同来源（比如字符数组）提供字符串的初始值：

```
char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.'};
String helloString = new String(helloArray);
System.out.println(helloString);
```

上面代码段的最后一行显示hello.。

注意　String类是不可变的，所以创建String对象之后就不能改动它。String类有很多方法看上去可以修改字符串，后面将介绍其中一些方法。因为字符串不可变，所以实际上这些方法进行的操作是创建并且返回新的字符串，新字符串包含操作的结果。

8.3.2　字符串长度

用于获得对象信息的方法被称为访问器方法（accessor method）。对字符串可以使用的一个访问器方法是length()方法，它返回字符串对象中包含的字符数量。执行完下面两行代码后，len等于17：

```
String palindrome = "Dot saw I was Tod";
int len = palindrome.length();
```

回文（palindrome）是对称的词或者句子——在忽略大小写和标点的情况下，它的正向拼写和反向拼写是一样的。下面是一个短小的低效率程序，它反转回文字符串。程序调用String方法charAt(i)，它返回字符串中的第i个字符，编号从0开始：

213

```
public class StringDemo {

  public static void main(String[] args) {
    String palindrome = "Dot saw I was Tod";
    int len = palindrome.length();
    char[] tempCharArray = new char[len];
    char[] charArray = new char[len];

    // put original string in an array of chars
    for (int i = 0; i < len; i++) {
      tempCharArray[i] = palindrome.charAt(i);
    }

    // reverse array of chars
    for (int j = 0; j < len; j++) {
      charArray[j] = tempCharArray[len - 1 - j];
    }

    String reversePalindrome =  new String(charArray);
    System.out.println(reversePalindrome);
  }
}
```

运行程序得到如下输出：

```
do'I' saw I was toD
```

为了完成字符串反转操作，程序必须把字符串转换为字符数组（第一个for循环），把这个数组反转得到第二个数组（第二个for循环），然后把第二个数组转换为字符串。String类包含一个getChars()方法用于把字符串（或者字符串的一部分）转换为字符数组，所以我们可以把上面程序的第一个for循环替换为：

```
palindrome.getChars(0, len - 1, tempCharArray, 0);
```

8.3.3 连接字符串

String类提供一个用于连接两个字符串的方法：

```
string1.concat(string2);
```

它返回一个新的字符串，即string2连接到string1末尾的结果。 214

也可以对字符串字面量使用concat()方法，比如：

```
"My name is ".concat("Rumplestiltskin");
```

更常见的方式是使用“+”操作符连接字符串，比如：

```
"Hello," + " world" + "!"
```

其结果是：

```
"Hello, world!"
```

在print语句中广泛地使用“+”操作符。例如：

```
String string1 = "saw I was ";
System.out.println("Dot " + string1 + "Tod");
```

语句的输出是：

```
Dot saw I was Tod
```

这样的连接可以是任何对象的混合。对于不是String的每个对象，会调用其toString()方法把它转换为String。

注意 Java编程语言不允许字符串字面量在源文件中跨越多行，所以在跨越多行的字符串的每行末尾，都必须使用“+”连接操作符。例如：

```
String quote = "Now is the time for all good " +
               "men to come to the aid of their country.";
```

同样，在print语句中，使用“+”连接操作符分割多行字符串是非常常见的。

8.3.4 创建格式化字符串

你已经看到过使用printf()和format()方法输出格式化数字的例子。String类有和它们等同的类方法format()，它返回String对象，而不是PrintStream对象。 215

使用String的静态方法format()允许你创建可以重用的格式化字符串，而不是一次性的输出语句。例如，可以不使用如下代码：

```
System.out.printf("The value of the float variable is " +
                  "%f, while the value of the integer " +
                       "variable is %d, and the string " +
                       "is %s", floatVar, intVar, stringVar);
```

而使用：

```
String fs;
fs = String.format("The value of the float variable is " +
                   "%f, while the value of the integer " +
                        "variable is %d, and the string " +
                        "is %s", floatVar, intVar, stringVar);
System.out.println(fs);
```

8.3.5　在数字和字符串之间进行转换

1. 把字符串转换为数字

程序执行中经常遇到这种情况：数字数据保存在字符串对象中，比如用户输入的一个值。包装原始数字类型的每个Number子类（Byte[1]、Integer[2]、Double[3]、Float[4]、Long[5]和Short[6]）都提供一个名为valueOf的类方法，它把字符串转换为属于这个类型的对象。下面的例子ValueOfDemo[7]从命令行获得两个字符串，把它们转换为数字，然后对数字值执行算术操作：

216

```
public class ValueOfDemo {
  public static void main(String[] args) {

    // this program requires two arguments on the command line
    if (args.length == 2) {

        // convert strings to numbers
        float a = (Float.valueOf(args[0])).floatValue();
        float b = (Float.valueOf(args[1])).floatValue();

        // do some arithmetic
        System.out.println("a + b = " + (a + b) );
        System.out.println("a - b = " + (a - b) );
        System.out.println("a * b = " + (a * b) );
        System.out.println("a / b = " + (a / b) );
        System.out.println("a % b = " + (a % b) );
    } else {
        System.out.println("This program requires " +
                           "two command-line arguments.");
    }
  }
}
```

下面是当使用4.5和87.2作为命令行参数时程序的输出：

```
a + b = 91.7
a - b = -82.7
a * b = 392.4
a / b = 0.0516055
a % b = 4.5
```

注意　包装原始数字类型的每个Number子类还提供一个parseXXX()方法（比如parseFloat()），可以使用它把字符串转换为原始数字。因为返回原始类型而不是对象，所以parseFloat()方法比valueOf()方法更加直接。例如，在ValueOfDemo程序中，我们可以使用：

---

1. docs/api/java/lang/Byte.html
2. docs/api/java/lang/Integer.html
3. docs/api/java/lang/Double.html
4. docs/api/java/lang/Float.html
5. docs/api/java/lang/Long.html
6. docs/api/java/lang/Short.html
7. tutorial/java/data/examples/ValueOfDemo.java

```
float a = Float.parseFloat(args[0]);
float b = Float.parseFloat(args[1]);
```

2. 把数字转换为字符串

有时候，因为你需要对字符串形式的值进行操作，所以要把数字转换为字符串。可以通过几种方式把数字转换为字符串：

```
int i;
String s1 = "" + i;    // Concatenate "i" with an empty string;
                       // conversion is handled for you.
```

217

或者

```
String s2 = String.valueOf(i);  // The valueOf class method.
```

每个Number子类都包含一个toString()类方法，它把其原始类型转换为字符串。例如：

```
int i;
double d;
String s3 = Integer.toString(i);
String s4 = Double.toString(d);
```

示例ToStringDemo[1]使用toString方法把数字转化为字符串。这个程序然后使用一些字符串方法计算小数点之前和之后的位数：

```
public class ToStringDemo {

  public static void main(String[] args) {
    double d = 858.48;
    String s = Double.toString(d);

    int dot = s.indexOf('.');

    System.out.println(dot + " digits before decimal point.");
    System.out.println( (s.length() - dot - 1) +
                        " digits after decimal point.");
  }
}
```

这个程序的输出为：

```
3 digits before decimal point.
2 digits after decimal point.
```

8.3.6 处理字符串中的字符

String类有很多方法，用于查看字符串的内容、在字符串内查找字符或者子字符串、改变大小写以及其他任务。

1. 按照索引获得字符或者子字符串

通过调用访问器方法charAt()，可以获得字符串中位于特定索引位置的字符。第一个字符的索引为0，而最后一个字符的索引为length()-1。例如，下面的代码获得字符串中索引为9的字符：

218

```
String anotherPalindrome = "Niagara. O roar again!";
char aChar = anotherPalindrome.charAt(9);
```

索引从0开始，所以索引为9的字符是“O”，如图8-2所示。

如果你希望从字符串获得一个以上的连续字符，可以使用substring方法。substring有两个版本，如表8-10所示。

---

1. tutorial/java/data/examples/ToStringDemo.java

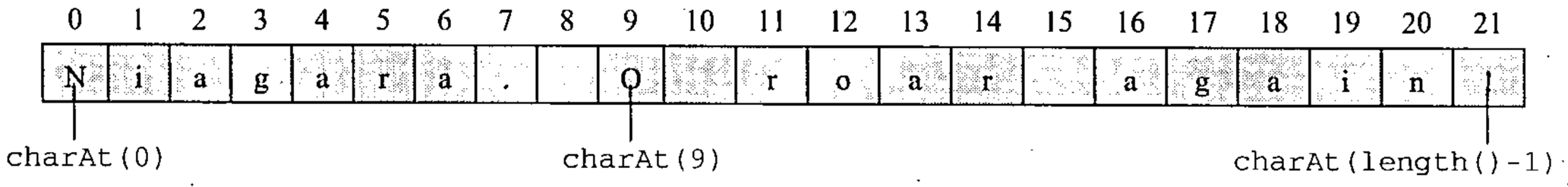


图8-2　使用charAt方法获得位于特定索引位置的字符

表8-10　String类的substring方法

方　法	描　述
String substring(int beginIndex, int endIndex)	返回新的字符串，它是这个字符串的子字符串。第一个整数实参指定第一个字符的索引。第二个整数实参是最后一个字符的索引加1
String substring(int beginIndex)	返回新的字符串，它是这个字符串的子字符串。整数实参指定第一个字符的索引。这里，返回的子字符串一直扩展到原始字符串的结尾

下面的代码从Niagara回文获得子字符串，它从索引11扩展到（但并不包括）索引15，即单词“roar”（见图8-3）：

```
String anotherPalindrome = "Niagara. O roar again!";
String roar = anotherPalindrome.substring (11, 15) ;
```

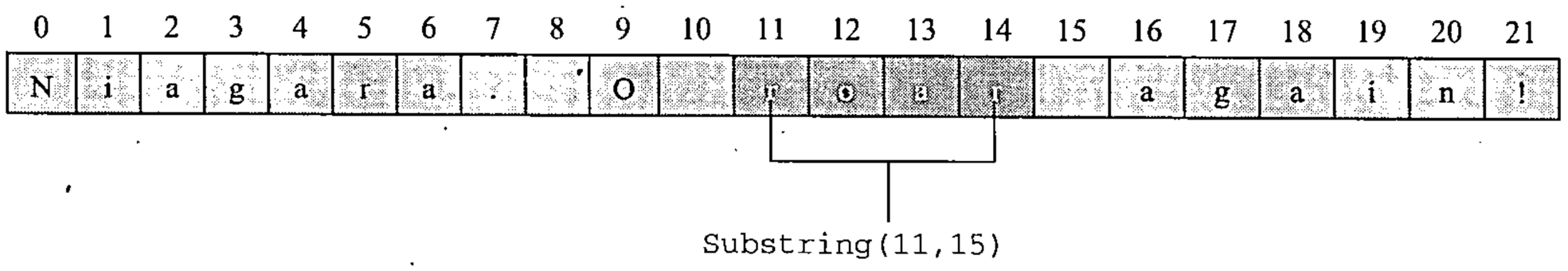


图8-3　使用substring方法获得部分字符串

2. 处理字符串的其他方法

表8-11列出了用于处理字符串的几个其他String方法。

表8-11　String类中用于处理字符串的其他方法

方　法	描　述
String[] split(String regex) String[] split(String regex, int limit)	搜索字符串实参（它包含正则表达式）指定的匹配字符串，并且相应地把这个字符串拆分为字符串数组。可选的整数实参指定返回数组的最大长度。正则表达式将在第13章中介绍
CharSequence subSequence(int beginIndex, int endIndex)	返回从索引beginIndex到endIndex-1构成的新字符序列
String trim()	返回这个字符串删除了前后空白的副本
String toLowerCase() String toUpperCase()	返回把这个字符串转换为小写或者大写后的副本。如果不需要转换，则返回原始字符串

3. 在字符串中搜索字符和子字符串

下面是另外一些String方法，用于在字符串内搜索字符或者子字符串。String类提供访问器方法，用于返回特定字符或者子字符串在字符串中的位置：indexOf()和lastIndexOf()。indexOf()方法从字符串的开头向后搜索，而lastIndexOf()方法从字符串的末尾向前搜索。如果没有找到字符或者子字符串，那么indexOf()和lastIndexOf()返回-1。

String类还提供一个搜索方法contains，如果字符串包含特定字符序列，这个方法就返回true。当你只需要知道字符串是否包含字符序列，而字符序列的出现位置并不重要时，可以使用这个方法。

表8-12描述各种字符串搜索方法。

表8-12 String类中的搜索方法

方 法	描 述
`int indexOf(int ch)` `int lastIndexOf(int ch)`	返回指定字符在第一次（最后一次）出现时的位置索引
`int indexOf(int ch, int fromIndex)` `int lastIndexOf(int ch, int fromIndex)`	返回指定字符在第一次（最后一次）出现时的位置索引，从指定索引位置向前（向后）开始搜索
`int indexOf(String str)` `int lastIndexOf(String str)`	返回指定子字符串在第一次（最后一次）出现时的位置索引
`int indexOf(String str, int fromIndex)` `int lastIndexOf(String str, int fromIndex)`	返回指定子字符串在第一次（最后一次）出现时的位置索引，从指定索引位置向前（向后）开始搜索
`boolean contains(CharSequence s)`	如果字符串包含指定的字符序列，则返回true

注意 CharSequence是String类实现的一个接口。因此，你可以使用字符串作为contains()方法的实参。

4. 替换字符串中的字符和子字符串

String类提供的用于把字符或者子字符串插入字符串的方法非常少。一般不需要它们：通过把从字符串中取出的子字符串和你希望插入的子字符串连接在一起，可以创建新的字符串。

但是，String类的确提供了用于替换找到的字符或者子字符串的4个方法。表8-13中列出了它们。

表8-13 String类中用于处理字符串的方法

方 法	描 述
`String replace(char oldChar, char newChar)`	返回新的字符串，它是把这个字符串中的所有oldChar都替换为newChar的结果
`String replace(CharSequence target, CharSequence replacement)`	把这个字符串中和字面量目标序列匹配的每个子字符串都替换为指定的字面量替换序列
`String replaceAll(String regex, String replacement)`	把这个字符串中和给定的正则表达式匹配的每个子字符串都替换为给定的替换序列
`String replaceFirst(String regex, String replacement)`	把这个字符串中和给定的正则表达式匹配的第一个子字符串替换为给定的替换序列

220 ~ 221

5. 示例

下面的类Filename[1]演示如何使用lastIndexOf()和substring()方法隔离文件名的不同部分。

注意 下面Filename类中的方法没有进行任何错误检查，并且假设它们的实参包含完整目录路径和带有扩展名的文件名。如果这些方法是产品代码，就应该检验它们的实参构成是否正确。

```
/**
```

1. tutorial/java/data/examples/Filename.java

```
 * This class assumes that the string used to initialize
 * fullPath has a directory path, filename, and extension.
 * The methods won't work if it doesn't.
 */
public class Filename {

  private String fullPath;
  private char pathSeparator, extensionSeparator;

  public Filename(String str, char sep, char ext) {
    fullPath = str;
    pathSeparator = sep;
    extensionSeparator = ext;
  }

  public String extension() {
    int dot = fullPath.lastIndexOf(extensionSeparator);
    return fullPath.substring(dot + 1);
  }

  public String filename() { // gets filename without extension
    int dot = fullPath.lastIndexOf(extensionSeparator);
    int sep = fullPath.lastIndexOf(pathSeparator);
    return fullPath.substring(sep + 1, dot);
  }

  public String path() {
    int sep = fullPath.lastIndexOf(pathSeparator);
    return fullPath.substring(0, sep);
  }
}
```

222

下面的程序FilenameDemo[1]构造Filename对象并且调用它的所有方法：

```
public class FilenameDemo {
  public static void main(String[] args) {
    final String FPATH = "/home/mem/index.html";
    Filename myHomePage = new Filename(FPATH,
                                       '/', '.');
    System.out.println("Extension = " +
                       myHomePage.extension());
    System.out.println("Filename = " +
                       myHomePage.filename());
    System.out.println("Path = " + myHomePage.path());
  }
}
```

下面是这个程序的输出：

```
Extension = html
Filename = index
Path = /home/mem
```

如图8-4所示，extension方法使用lastIndexOf定位点号（.）在文件名中最后一次出现的位置。然后substring使用lastIndexOf的返回值提取文件的扩展名——就是从点号开始到字符串结尾的子字符串。这段代码假设文件名中有点号，如果文件名不包含点号，lastIndexOf就会返回-1，并且substring方法会抛出StringIndexOutOfBoundsException异常。

---

1. tutorial/java/data/examples/FilenameDemo.java

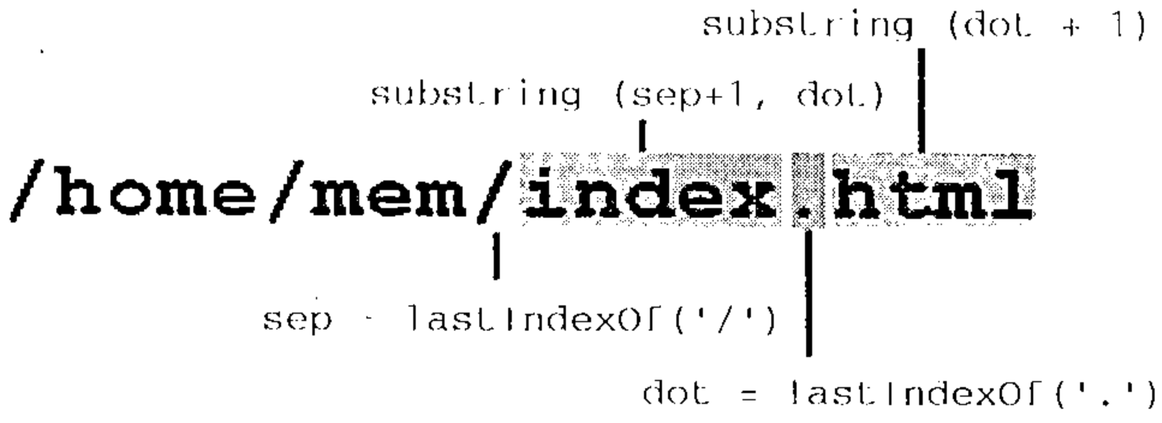


图8-4 在Filename类的extension方法中使用lastIndexOf和substring 223

另外，注意extension方法使用dot+1作为substring的实参。如果点号符号（.）是字符串中的最后一个字符，那么dot+1就等于字符串的长度，它等于字符串的最大索引加1（因为索引从0开始）。这对substring来说是合法实参，因为这个方法接受等于但不能大于字符串长度的索引，并且把它解释为“字符串的末尾”。

8.3.7 比较字符串和部分字符串

String类有很多用于比较字符串和部分字符串的方法。表8-14列出了这些方法。

表8-14 用于比较字符串的方法

方法	描述
`boolean endsWith(String suffix)` `boolean startsWith(String prefix)`	如果这个字符串以被指定为方法的实参的子字符串结束或者开始，则返回true
`boolean startsWith(String prefix, int offset)`	从索引offset的位置开始检查，如果这个字符串以被指定为实参的子字符串开始，则返回true
`int compareTo(String anotherString)`	按照字母表顺序比较两个字符串。返回一个整数，表示这个字符串是大于（结果大于0）、等于（结果等于0）还是小于（结果小于0）实参
`int compareToIgnoreCase(String str)`	按照字母表顺序比较两个字符串，忽略大小写的不同。返回一个整数，表示这个字符串是大于（结果大于0）、等于（结果等于0）还是小于（结果小于0）实参
`boolean equals(Object anObject)`	如果实参是String对象，而且它表示的字符序列和这个对象相同，则返回true
`boolean equalsIgnoreCase(String anotherString)`	如果实参是String对象，而且它表示的字符序列和这个对象相同，则返回true，比较时忽略大小写的不同
`boolean regionMatches(int toffset, String other, int ooffset, int len)`	检查这个字符串的指定部分是否和String实参的指定部分匹配。 字符串部分的长度为len，这个字符串从toffset索引开始，另一个字符串从ooffset索引开始
`boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)`	检查这个字符串的指定部分是否和String实参的指定部分匹配。 字符串部分的长度为len，这个字符串从toffset索引开始，另一个字符串从ooffset索引开始。 布尔参数指定是否忽略大小写；如果为true，则在比较字符时忽略大小写
`boolean matches(String regex)`	检查这个字符串是否和指定的正则表达式匹配。正则表达式在第13章中讨论

下面的程序RegionMatchesDemo[1]使用regionMatches方法在一个字符串内搜索另一个

1. tutorial/java/data/examples/RegionMatchesDemo.java

字符串：

```
public class RegionMatchesDemo {
  public static void main(String[] args) {
    String searchMe = "Green Eggs and Ham";
    String findMe = "Eggs";
    int searchMeLength = searchMe.length();
    int findMeLength = findMe.length();
    boolean foundIt = false;
    for (int i = 0; i <=
        (searchMeLength - findMeLength); i++) {
      if (searchMe.regionMatches(i, findMe, 0, findMeLength)) {
        foundIt = true;
        System.out.println(searchMe.substring(i,
                                              i + findMeLength));
        break;
      }
    }

    if (!foundIt) System.out.println("No match found.");
  }
}
```

这个程序的输出为Eggs。

224～225 这个程序处理searchMe引用的字符串，一次检查一个字符。对每个字符，程序都调用regionMatches方法，判断以当前字符开头的子字符串是否匹配程序正在搜索的字符串。

8.3.8 StringBuilder类

StringBuilder[1]对象和String对象类似，只是，前者可以修改。在内部，这些对象就像包含字符序列的可变长度数组。任何时候都可以通过方法调用改变字符序列的长度和内容。

应该总使用字符串，除非字符串构建器在简化代码（参见本节结尾的示例程序）或者提高性能方面能够提供优势。例如，如果你需要连接大量字符串，那么附加到一个StringBuilder对象效率会更高。

1. 长度和容量

StringBuilder类和String类一样也具有length()方法，它返回构建器中字符序列的长度。

和字符串不同，每个字符串构建器中还具有容量（capacity），它是已经分配的字符空间的数量。容量由capacity()方法返回，它总是大于或者等于长度（通常大于长度），并且根据需求自动扩展以适应对字符串构建器的添加（见表8-15）。

表8-15　StringBuilder构造器

构造器	描　述
StringBuilder()	创建容量为16的空字符串构建器（16个空元素）
StringBuilder(CharSequence cs)	构造包含和指定CharSequence相同的字符的字符串构建器，并且在CharSequence末尾加上16个额外的空元素
StringBuilder(int initCapacity)	创建具有指定的初始容量的空字符串构建器
StringBuilder(String s)	创建字符串构建器，使用指定的字符串初始化它的值，在字符串末尾加上16个额外的空元素

1. docs/api/java/lang/StringBuilder.html

例如，下面的代码：

```
StringBuilder sb = new StringBuilder(); // creates empty
                                        // builder, capacity 16
sb.append("Greetings"); // adds 9 character string at beginning
```

226

将生成长度为9、容量为16的字符串构建器，如图8-5所示。

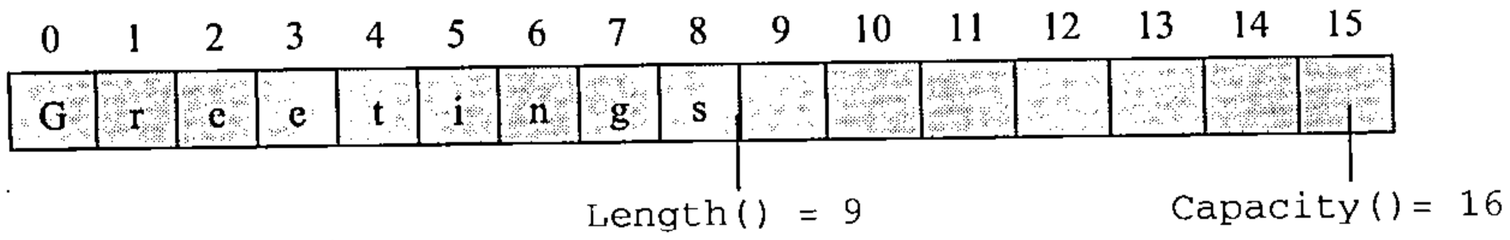


图8-5 字符串构建器的长度是它包含的字符数量；字符串构建器的容量是已经分配的字符空间的数量

StringBuilder类具有一些与长度和容量相关的方法，String类没有这些方法；见表8-16。

表8-16 长度和容量方法

方 法	描 述
void setLength(int newLength)	设置字符序列的长度。如果newLength小于length()，就把字符序列中最后的字符截掉。如果newLength大于length()，就把（newLengthlength()）个null字符添加到字符序列的末尾
void ensureCapacity(int minCapacity)	确保容量至少等于指定的最小值

很多操作（比如append()、insert()或者setLength()）都能够增加字符串构建器中字符序列的长度，使产生的length()大于当前的capacity()。当发生这种情况时，容量会自动增加。

2. StringBuilder操作

StringBuilder具有但在String中没有的主要操作是append()和insert()方法，它们可以被重载以便接受任何类型的数据。这两个方法都把其实参转换为字符串，然后把这个字符串附加或者插入到字符串构建器中的字符序列。append()方法把这些字符附加到现有字符序列的末尾，而insert()方法把字符添加到指定的位置。

表8-17列出StringBuilder类的一些方法。

227

表8-17 各种StringBuilder方法

方 法	描 述
StringBuilder append(boolean b) StringBuilder append(char c) StringBuilder append(char[] str) StringBuilder append(char[] str, int offset, int len) StringBuilder append(double d) StringBuilder append(float f) StringBuilder append(int i) StringBuilder append(long lng) StringBuilder append(Object obj) StringBuilder append(String s)	把实参附加到这个字符串构建器。进行附加操作之前把数据转换为字符串
StringBuilder delete(int start, int end) StringBuilder deleteCharAt(int index)	删除这个字符串构建器中指定的字符

（续）

方　法	描　述
StringBuilder insert(int offset, boolean b) StringBuilder insert(int offset, char c) StringBuilder insert(int offset, char[] str) StringBuilder insert(int index, char[] str, int offset, int len) StringBuilder insert(int offset, double d) StringBuilder insert(int offset, float f) StringBuilder insert(int offset, int i) StringBuilder insert(int offset, long lng) StringBuilder insert(int offset, Object obj) StringBuilder insert(int offset, String s)	把第二个实参插入到这个字符串构建器。第一个整数实参指定数据插入位置前面的索引。进行插入操作之前把数据转换为字符串
StringBuilder replace(int start, int end, String s)	替换这个字符串构建器中指定的字符
void setCharAt(int index, char c)	
StringBuilder reverse()	反转这个字符串构建器中的字符序列
String toString()	返回包含构建器中字符序列的字符串

注意　首先使用StringBuilder类的toString()方法把字符串构建器转换为字符串，然后就可以对StringBuilder对象使用任何String方法。然后使用StringBuilder(String str)构造器把字符串转换回字符串构建器。

3. 示例

在8.3节列出的程序StringDemo中，如果使用StringBuilder，而不使用String，那么这个示例程序就更有效率。

228　StringDemo用于反转一句回文。这里再次列出它：

```
public class StringDemo {
  public static void main(String[] args) {
    String palindrome = "Dot saw I was Tod";
    int len = palindrome.length();
    char[] tempCharArray = new char[len];
    char[] charArray = new char[len];

    // put original string in an array of chars
    for (int i = 0; i < len; i++) {
      tempCharArray[i] = palindrome.charAt(i);
    }

    // reverse array of chars
    for (int j = 0; j < len; j++) {
      charArray[j] = tempCharArray[len - 1 - j];
    }

    String reversePalindrome =  new String(charArray);
    System.out.println(reversePalindrome);
  }
}
```

运行程序得到如下输出：

```
doT saw I was toD
```

为了完成字符串反转功能，程序把字符串转换为字符数组（第一个for循环），把这个数组反转得到第二个数组（第二个for循环），然后把数组转换回字符串。

如果把palindrome从字符串转换为字符串构建器，就可以使用StringBuilder类中的reverse()方法。这使代码更加简单，也更容易阅读：

```
public class StringBuilderDemo {
  public static void main(String[] args) {
    String palindrome = "Dot saw I was Tod";

    StringBuilder sb = new StringBuilder(palindrome);

    sb.reverse();  // reverse it

    System.out.println(sb);
  }
}
```

运行这个程序得到相同的输出：

```
doT saw I was toD
```

注意，println()输出字符串构建器，比如在下列代码中： 229

```
System.out.println (sb) ;
```

因为这时会隐式地调用sb.toString()，就像在println()调用中对待任何其他对象一样。

注意 还有一个和StringBuilder类完全相同的StringBuffer类，只是它通过使其方法同步，具有线程安全性。线程将在第12章中讨论。

8.3.9 字符和字符串小结

大多数情况下，如果你使用单一字符值，就可以使用原始char类型。但是有时候，你需要将一个字符作为对象使用——比如要求方法实参是对象时。Java编程语言提供包装器（wrapper）类把char“包装”到Character对象中以达到这一目的。类型为Character的对象包含单一字段，其类型为char。这个Character类还提供很多有用的类方法（即静态方法）用于处理字符。

字符串是字符序列，广泛地用于Java编程中。在Java编程语言中，字符串是对象。String类具有超过60个方法和13个构造器。

更常见的是，使用下面这样的语句创建字符串：

```
String s = "Hello world!";
```

而不是使用某个String构造器。

String类有很多方法用于查找和获得子字符串；然后可以使用“+”拼接操作符，容易地使用这些子字符串重新生成新的字符串。

String类还包含很多实用方法，其中包括split()、toLowerCase()、toUpperCase()和valueOf()。在把用户输入的字符串转换为数字的操作中，valueOf()方法是必不可少的。Number子类也提供用于在字符串和数字之间进行相互转换的方法。

除了String类之外，还有StringBuilder类。有时候，处理StringBuilder对象要比处理字符串的效率更高。StringBuilder类提供几个处理字符串的有用方法，其中包括reverse()方法。但是总的来说，String类提供的方法其应用范围更广。

可以使用StringBuilder构造器把字符串转换为字符串构建器。可以使用toString()方法把字符串构建器转换为字符串。 230

问题和练习：字符和字符串

问题

1. 下面的字符串构建器的初始容量是多少？

```
StringBuilder sb = new StringBuilder("Able was I " +
                                     "ere I saw Elba.");
```

2. 查看如下字符串：

```
String hannah = "Did Hannah see bees? Hannah did.";
```

   a. 表达式hannah.length()显示的值是什么？

   b. 调用方法hannah.charAt(12)的返回值是什么？

   c. 编写一个表达式，引用hannah所引用的字符串中的字母b。

3. 下列表达式返回的字符串的长度是多少？这个字符串是什么？

```
"Was it a car or a cat I saw?".substring(9, 12)
```

4. 在下面的程序ComputeResult.java[1]中，每个编号的代码行执行之后，result的值是什么？

```
public class ComputeResult {
  public static void main(String[] args) {
    String original = "software";
    StringBuilder result = new StringBuilder("hi");
    int index = original.indexOf('a');

/*1*/  result.setCharAt(0, original.charAt(0));
/*2*/  result.setCharAt(1,
           original.charAt(original.length()-1));
/*3*/  result.insert(1, original.charAt(4));
/*4*/  result.append(original.substring(1,4));
/*5*/  result.insert(3, (original.substring(index, index+2)
                                              + " "));

     System.out.println(result);
  }
}
```

231

练习

1. 以两种方式把下面两个字符串拼接在一起，以得到“Hi, mom.”：

```
String hi = "Hi, ";
String mom = "mom.";
```

2. 编写一个程序，从你的全名得出姓名缩写并显示出来。

3. 变位词是改变一个单词或者短语的字母顺序而得到的单词或者短语；比如“parliament”就是“partial men”的变位词，而“software”是“swear oft”的变位词。编写一个程序，判断一个字符串是否是另一个字符串的变位词。这个程序应该忽略空格和标点符号。

答案

可以在以下位置找到“问题”和“练习”的答案：

232　tutorial/java/data/QandE/characters-answers.html

---

1. tutorial/java/data/QandE/ComputeResult.java

第9章

异　　常

Java编程语言使用异常（exception）处理错误和其他异常事件。本章介绍何时和如何使用异常。

9.1　异常是什么

异常（exception）这个术语是“异常事件”的简称。

定义　异常是一个事件，它出现在程序执行的过程中，中断程序指令的正常流程。

当一个方法中发生错误时，此方法创建一个对象并将它交给运行时系统。此对象被称为异常对象（exception object），它包含关于错误的信息，包括错误的类型和错误发生时程序的状态。创建异常对象并将它交给运行时系统被称为抛出异常（throw an exception）。

在方法抛出一个异常之后，运行时系统尝试寻找对此异常进行处理的某些机制。对异常进行处理的这一套“机制”是一系列有序的方法，是为了到达发生错误的方法而调用的一系列方法。这个方法列表被称为调用栈（call stack）（见图9-1）。

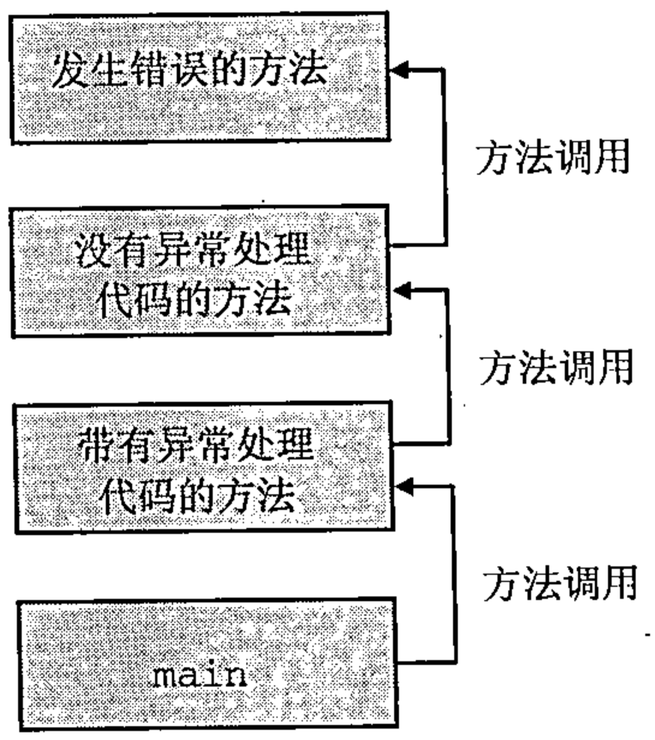
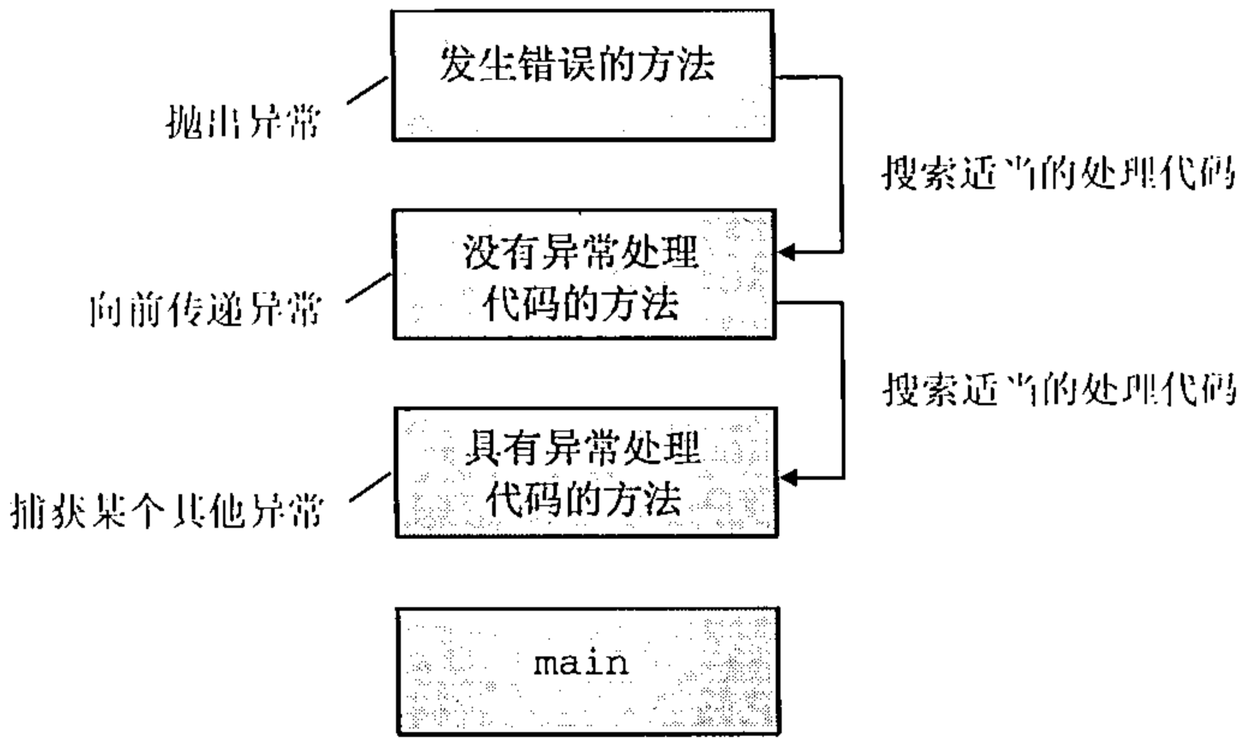


图9-1　调用栈

运行时系统搜索调用栈，寻找一个包含可以处理异常的代码块的方法。这个代码块被称为异常处理代码（exception handler）。搜索从发生错误的方法开始，按照与方法被调用次序相反的次序遍历调用栈。当找到合适的处理器时，运行时系统将异常传递给此处理器。如果被抛出的异常对象的类型和某个异常处理代码可以处理的类型匹配，那么此处理器就被认为是合适的。

选择合适的异常处理代码被称为捕获异常（catch the exception）。如果运行时系统彻底搜索了调用栈中的所有方法，但没有找到合适的异常处理代码（如图9-2所示），那么运行时系统和程序就会终止。



233
~
234

图9-2　在调用栈中搜索异常处理代码

与传统的错误管理技术相比，使用异常来管理错误有一些优点。更多的信息请参考9.7节。

9.2　捕获或者指定需求

有效的Java编程语言代码必须遵守捕获或者指定需求（Catch or Specify Requirement）。也就是说，可能抛出特定异常的代码必须具有如下任一条件：

- 捕获异常的try语句。try必须提供异常处理代码，对它的介绍见9.3节。
- 指定可能抛出异常的方法。这个方法必须提供throws子句来列出异常，对它的介绍见9.4节。

不遵守“捕获或者指定需求”的代码不能通过编译。

并不是所有异常都遵循捕获或者指定需求。为了了解其原因，我们需要了解异常的三种基本类型，其中只有一种遵循这个需求。

9.2.1　异常的三种类型

异常的第一种类型是可控异常（checked exception）。这些是编写良好的应用程序应该能够预测并且能够从中恢复的异常情况。例如，假设应用程序要求用户输入文件名，然后把文件名传递给java.io.FileReader的构造器以便打开这个文件。通常，用户提供现有的、可读的文件名，所以FileReader对象会被成功构造，并且应用程序正常执行。但是有时候用户提供了不存在的文件名，那么构造器就会抛出java.io.FileNotFoundException异常。编写良好的程序会捕获这个异常，并且通知用户出现了错误，可能会提示用户输入正确的文件名。

可控异常遵循捕获或者指定需求。所有异常都是可控的，除了由Error、RuntimeException和它们的子类表示的异常。

异常的第二种类型是错误（error）。这些是应用程序外部的异常情况，应用程序通常不能预
235
测它们并且不能从中恢复。例如，假设应用程序成功地打开了一个文件用于输入，但是因为硬件故障或者系统故障不能读取文件。不成功的读取会抛出java.io.IOError异常。应用程序可以选择捕获这种异常，以便通知用户出现了问题——但是应用程序也可以输出栈踪迹并且退出，这也是有意义的。

错误不遵循捕获或者指定需求。错误是由Error或者其子类表示的异常。

异常的第三种类型是运行时异常（runtime exception）。这些是应用程序内部的异常情况，

应用程序通常不能预测它们并且不能从中恢复。这些异常情况通常表示编程错误，比如逻辑错误或者API的不正确应用。例如，考虑前面介绍的应用程序，它把文件名传递给FileReader的构造器。如果逻辑错误导致把null传递给构造器，构造器就会抛出NullPointerException异常。应用程序能够捕获这个异常，但是消除导致异常的错误可能更有意义。

运行时异常不遵循捕获或者指定需求。运行时异常是由RuntimeException及其子类表示的异常。

错误和运行时异常合称为不可控异常。

9.2.2 绕过捕获或者指定

一些程序员认为捕获或者指定需求是异常机制中的重大缺陷，可以通过在可控异常的位置使用不可控异常从而绕过它们。总的来说不建议这样做。在9.6节中讨论什么时候适合使用不可控异常。

9.3 捕获和处理异常

本节讲解如何使用异常处理代码的三个组件try、catch和finally块编写异常处理代码。本节的最后一部分讨论一个例子，并分析各种情况下的处理过程。

下面的程序定义并实现了一个名为ListOfNumbers[1]的类。构造好之后，ListOfNumbers创建一个Vector，它包含10个Integer元素，这些元素的值为从0到9。ListOfNumbers类还定义了一个名为writeList的方法，此方法将数字列表写到文本文件OutFile.txt中。这个例子使用java.io中定义的输出类，第10章中讨论这些类。 236

```
// Note: This class won't compile by design!
import java.io.*;
import java.util.Vector;

public class ListOfNumbers {

  private Vector vector;
  private static final int SIZE = 10;

  public ListOfNumbers () {
    vector = new Vector(SIZE);
    for (int i = 0; i < SIZE; i++) {
      vector.addElement(new Integer(i));
    }
  }

  public void writeList() {
    PrintWriter out = new PrintWriter(
                          new FileWriter("OutFile.txt"));

    for (int i = 0; i < SIZE; i++) {
      out.println("Value at: " + i + " = " +
                                    vector.elementAt(i));
    }

    out.close();
  }
}
```

1. tutorial/essential/exceptions/examples/ListOfNumbers.java

第一行以粗体显示的代码调用一个构造器。此构造器初始化一个文件的输出流。如果文件不能被打开，那么构造器抛出一个IOException异常。第二行以粗体显示的代码调用Vector类的elementAt方法，如果实参值太小（小于0）或太大（大于Vector当前包含的元素数），那么此方法抛出一个ArrayIndexOutOfBoundsException异常。

如果试图编译ListOfNumbers类，那么编译器会输出一个错误消息，描述由FileWriter构造器抛出的异常。但是对于elementAt方法抛出的异常，编译器没有显示错误消息。这是因为构造器抛出的异常IOException是一个可控异常，而elementAt方法抛出的异常ArrayIndexOutOfBoundsException是一个不可控异常。

既然你已经熟悉了ListOfNumbers类，而且知道了其中什么地方可以抛出异常，下面就开始学习如何编写异常处理代码来捕获和处理这些异常。

237

9.3.1　try块

构造异常处理代码的第一步是，将可能抛出异常的代码封装在try块中。try块一般是下面这样的：

```
try {
  code
}
catch and finally blocks . . .
```

标为*code*的段落包含一行或多行能抛出异常的语句（下面两节讲解catch和finally块）。

要为ListOfNumbers类的writeList方法构造异常处理代码，你需要将writeList方法中可能抛出异常的语句包围在try块中。这可以通过多种办法实现。可以将每行可能抛出异常的代码放在它自己的try块中，并为它们提供单独的异常处理代码。或者，可以将所有writeList代码放在一个try块中，并使多个异常处理代码与它相关联。下面的例子因为涉及的代码非常短，所以让整个方法使用一个try块：

```
private Vector vector;
private static final int SIZE = 10;

PrintWriter out = null;

try {
  System.out.println("Entered try statement");
  out = new PrintWriter(new FileWriter("OutFile.txt"));
  for (int i = 0; i < SIZE; i++) {
    out.println("Value at: " + i + " = " +
                                    vector.elementAt(i));
  }
}
catch and finally statements . . .
```

如果某个try块内发生异常，异常由与该try块相关联的异常处理代码处理。要使异常处理代码与try块相关联，必须在它后面放一个catch块。下一节讲解具体做法。

238

9.3.2　catch块

将异常处理代码与try块相关联的办法是，在try块后面直接提供一个或多个catch块。try块的结尾与第一个catch语句之间不能有任何代码

```
try {
```

```
} catch (ExceptionType name) {

} catch (ExceptionType name) {

}
```

每个catch块都是一个异常处理代码，它处理由其实参指定的异常类型。实参类型*ExceptionType*声明处理器可以处理的异常类型，它必须是一个从Throwable类派生而来的类的名称。处理器可以用*name*引用异常。

当异常处理代码被调用时，执行catch块包含的代码。如果异常处理代码是调用栈中其*ExceptionType*与被抛出的异常类型匹配的第一个处理器，那么运行时系统就调用这个处理器。如果被抛出的对象可以被合法地赋给异常处理代码的实参，那么系统就认为它是匹配的。

下面是用于writeList方法的两个异常处理代码，每个处理器分别对应try语句中可能抛出的两种可控异常类型：

```
try {

} catch (FileNotFoundException e) {
    System.err.println("FileNotFoundException: " + e.getMessage());
    throw new SampleException(e);

} catch (IOException e) {
    System.err.println("Caught IOException: " + e.getMessage());
}
```

两个处理器都显示一条错误消息。第二个处理器除此之外不进行任何操作。通过捕获第一个处理器没有捕获的任何IOException异常，第二个处理器允许程序继续执行。

第一个处理器除了输出消息之外，还抛出一个用户定义的异常。（抛出异常将在后面的9.5节中详细介绍。）在这个例子中，当捕获到FileNotFoundException异常时，就导致抛出名为SampleException的用户定义的异常。如果你希望程序在这种情况下按照指定的方式处理一个异常，就可能这样做。

除了输出错误消息或者终止程序之外，异常处理代码能够进行更多操作。它们可以进行错误恢复、提示用户作出决定，或者使用链式异常把错误传递给更高级别的处理器，这在9.5.5节中讨论。

9.3.3 finally块

存在try块时总会执行finally块。这确保即使出现未预料到的异常，也会执行finally块。但是，finally除了处理异常之外还有很多作用——它使程序员能够避免因清理代码而偶然被return、continue或者break绕过。在finally块中加上清理代码总是良好的习惯，即使在预期不会出现异常的情况下。

你曾经见过writeList方法的try块打开了PrintWriter。程序应该在退出writeList方法之前关闭这个流。这就引发了一个有点儿复杂的问题，因为writeList方法的try块可能以3种方式退出：

(1) new FileWriter语句失败，抛出一个IOException异常。

(2) vector.elementAt(i)语句失败，抛出一个ArrayIndexOutOfBoundsException异常。

(3) 所有语句成功执行，try块正常退出。

无论try块中是否发生异常，运行时系统总会执行finally块中的语句。所以这里是执行清理工作的理想位置。

下面是writeList方法的finally块，它清理并关闭PrintWriter：

```
finally {
  if (out != null) {
     System.out.println("Closing PrintWriter");
     out.close();
  } else {
     System.out.println("PrintWriter not open");
  }
}
```

240

在writeList例子中，可以提供清理功能而不必借助于finally块。例如，可以将关闭PrintWriter的代码放在try块的末尾，并且再次放在处理ArrayIndexOutOfBounds-Exception的异常处理代码中，如下所示：

```
try {

     out.close();     // Don't do this; it duplicates code.

} catch (FileNotFoundException e) {
     out.close();     // Don't do this; it duplicates code..

   System.err.println("Caught: FileNotFoundException: " +
                                              e.getMessage());
     throw new RuntimeException(e);

} catch (IOException e) {
     System.err.println("Caught IOException: " +
                                          e.getMessage());
}
```

但是，这造成了代码重复，因而使代码难以阅读，而且以后修改代码时可能出错。例如，如果在try块中添加了新代码，它们可以抛出另一种异常，那么你必须记得在新的异常处理代码中关闭PrintWriter。

重点　finally块是防止资源泄漏的关键工具。当关闭文件或者通过其他方式回收资源时，在finally块中加入代码，以便确保资源总是被回收。

9.3.4　结合使用

前几节分别讨论了如何为ListOfNumbers类中的writeList方法构造try、catch和finally代码块。

现在，我们完整地查看代码，并且研究可能发生什么情况。

241

将所有组件结合起来后，writeList方法类似下面这样：

```
public void writeList() {
  PrintWriter out = null;

  try {
    System.out.println("Entering try statement");
    out = new PrintWriter(
                  new FileWriter("OutFile.txt"));
      for (int i = 0; i < SIZE; i++)
        out.println("Value at: " + i + " = " +
```

```
                                vector.elementAt(i));

        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Caught " +
                               "ArrayIndexOutOfBoundsException: " +
                                                    e.getMessage());
        } catch (IOException e) {
            System.err.println("Caught IOException: " +
                                                e.getMessage());
        } finally {
            if (out != null) {
              System.out.println("Closing PrintWriter");
              out.close();
            }
            else {
              System.out.println("PrintWriter not open");
            }
        }
    }
```

如前所述，这个方法中的try块有3种不同的可能退出方式，下面是其中的两种：

(1) try语句中的代码失败并且抛出异常。这可能是new FileWriter语句导致的IOException异常，也可能是for循环中的错误索引值导致的ArrayIndexOutOfBounds-Exception异常。

(2) 所有语句成功执行，try块正常退出。

我们来看看在这两种可能的退出方式下，writeList方法中发生了什么情况。 242

1. 情况1：出现异常

创建FileWriter的语句可能因为多种原因失败。例如，如果程序不能创建或者写入指定的文件，那么FileWriter的构造器就会抛出一个IOException异常。

当FileWriter抛出IOException异常时，运行时系统立即停止执行try块，正在执行的方法调用没有完成。然后，运行时系统从方法调用栈的顶部开始搜索合适的异常处理代码。在这个例子中，当发生IOException异常时，FileWriter构造器位于调用栈的顶端。但是FileWriter构造器没有合适的异常处理代码，所以运行时系统检查方法调用栈中的下一个方法——writeList方法。writeList方法有两个异常处理代码：一个针对IOException异常，一个针对ArrayIndexOutOfBoundsException异常。

运行时系统按照writeList的异常处理代码在try语句后出现的次序检查处理器。第一个异常处理代码的实参是ArrayIndexOutOfBoundsException。因为这不匹配被抛出的异常的类型，所以运行时系统继续检查下一个异常处理代码——IOException。它匹配被抛出的异常的类型，所以运行时系统停止对适当异常处理代码的搜索。现在运行时系统找到了适当的处理器，所以执行对应catch块中的代码。

在执行异常处理代码后，运行时系统将控制传递给finally块。不管前面是否捕获到异常，finally块中的代码都会执行。在此情况下，PrintWriter从未被打开过，也就无需关闭。在执行完finally块之后，程序继续执行finally块后的第一个语句。

下面是抛出IOException异常时ListOfNumbers程序的完整输出：

```
Entering try statement
Caught IOException: OutFile.txt
PrintWriter not open
```

下面代码中以粗体显示的部分是在此情况下执行的语句： 243

```
public void writeList() {
  PrintWriter out = null;
```

```
try {
  System.out.println("Entering try statement");
  out = new PrintWriter(
            new FileWriter("OutFile.txt"));
  for (int i = 0; i < SIZE; i++)
    out.println("Value at: " + i + " = " +
                                    vector.elementAt(i));

} catch (ArrayIndexOutOfBoundsException e) {
    System.err.println("Caught " +
                       "ArrayIndexOutOfBoundsException: " +
                                          e.getMessage());
  } catch (IOException e) {
    System.err.println("Caught IOException: " +
                                    e.getMessage());
  } finally {
    if (out != null) {
       System.out.println("Closing PrintWriter");
       out.close();
    }
    else {
      System.out.println("PrintWriter not open");
    }
  }
}
```

2. 情况2：try块正常退出

在此情况下，try块作用域内的所有语句成功地执行，没有抛出异常。一直执行到try块的末尾，运行时系统将控制传递给finally块。因为所有语句都成功地执行，所以当控制到达finally块时PrintWriter是打开的，于是finally块关闭它。同样，在执行完finally块之后，程序继续执行finally块后的第一个语句。

下面是没有抛出异常时ListOfNumbers程序的完整输出：

```
Entering try statement
Closing PrintWriter
```

244　下面代码中以粗体显示的部分是在此情况下执行的语句：

```
public void writeList() {
  PrintWriter out = null;
  try {
    System.out.println("Entering try statement");
    out = new PrintWriter(
              new FileWriter("OutFile.txt"));
    for (int i = 0; i < SIZE; i++)
      out.println("Value at: " + i + " = " +
                                  vector.elementAt(i));

  } catch (ArrayIndexOutOfBoundsException e) {
      System.err.println("Caught " +
                         "ArrayIndexOutOfBoundsException: " +
                                            e.getMessage());

  } catch (IOException e) {
      System.err.println("Caught IOException: " +
                                      e.getMessage());
  } finally {
      if (out != null) {
        System.out.println("Closing PrintWriter");
        out.close();
      }
```

```
        else {
          System.out.println("PrintWriter not open");
        }
    }
}
```

9.4 指定方法抛出的异常

前一节讲解了如何为ListOfNumbers类中的writeList方法编写异常处理代码。有时，代码应该捕获其中发生的异常。但在其他情况下，让调用栈中的上层方法来处理异常会更好。例如，如果你将ListOfNumbers类设置为一个包的一部分，你可能无法预先知道这个包的所有用户的需求。在此情况下，最好不要捕获异常，而是允许调用栈中的上层方法处理它。

如果writeList方法不捕获其中发生的可控异常，那么writeList方法必须表明它可以抛出这些异常。我们修改原来的writeList方法，指定它可以抛出的异常，而不是捕获它们。为了便于参考，下面列出不能编译的原版本的writeList方法：

```
// Note: This method won't compile by design!
public void writeList() {
  PrintWriter out =
    new PrintWriter(new FileWriter("OutFile.txt"));
  for (int i = 0; i < SIZE; i++) {
    out.println("Value at: " + i + " = " +
                                vector.elementAt(i));
  }
  out.close();
}
```

为了指定writeList方法可以抛出两个异常，需要在writeList方法的方法声明中添加一个throws子句。throws子句由throws关键字和一个以逗号分隔的异常清单组成，这个清单列出此方法抛出的所有异常。此子句跟在方法名称和实参列表后面，在定义方法作用域的花括号之前，下面是一个例子：

```
public void writeList() throws IOException,
                               ArrayIndexOutOfBoundsException {
```

记住，ArrayIndexOutOfBoundsException是一个不可控异常，并不强制在throws子句中指定它。可以这么写：

```
public void writeList() throws IOException {
```

9.5 如何抛出异常

在能够捕获异常之前，必须有某些代码抛出了异常。任何代码都可能抛出异常：你的代码、其他人写的包（比如Java平台附带的包）中的代码或者Java运行时环境。不管是什么代码抛出了异常，都是使用throw语句抛出的。

你可能已经注意到了，Java平台提供了许多异常类。这些类都是Throwable类[1]的后裔。这些类使程序可以区分程序执行期间发生的各种异常。

你还可以创建自己的异常类来代表你写的类中可能发生的问题。实际上，如果你从事包的开发，就可能必须创建一组自己的异常类，以便使你的用户可以区分你的包中发生的错误与Java

1. docs/api/java/lang/Throwable.html

246 平台或其他包中发生的错误。

也可以创建链式异常。关于更多信息请参阅9.5.5节。

9.5.1 throw语句

所有方法都使用throw语句抛出异常。throw语句需要单一实参：一个可抛出的对象。可抛出的对象是Throwable类的任何子类的实例。下面是一个throw语句的例子：

```
throw someThrowableObject;
```

我们来看看实际代码中的throw语句。下面的pop方法取自一个实现常见的栈对象的类。此方法删除栈顶端的元素并返回这个对象：

```
public Object pop() {
  Object obj;

  if (size == 0) {
    throw new EmptyStackException();
  }

  obj = objectAt(size - 1);
  setObjectAt(size - 1, null);
  size--;
  return obj;
}
```

pop方法检查栈中是否有元素。如果栈是空的（它的大小为0），那么pop方法实例化一个新的EmptyStackException对象（它是java.util的成员）并抛出它。在9.5.6节中将解释如何创建自己的异常类。目前，你需要记住的是，只能抛出从java.lang.Throwable类继承而来的对象。

注意，pop方法的声明并不包含throws子句。因为EmptyStackException不是一个可控异常，所以不要求pop方法声明中出现它。

9.5.2 Throwable类及其子类

从Throwable类派生的对象包括直接后裔（直接从Throwable类继承而来的对象）和间接
247 后裔（从Throwable类的子类或孙类继承而来的对象）。图9-3显示Throwable类和它最主要的子类的类层次结构。如你所见，Throwable类有两个直接后裔：Error[1]和Exception[2]。

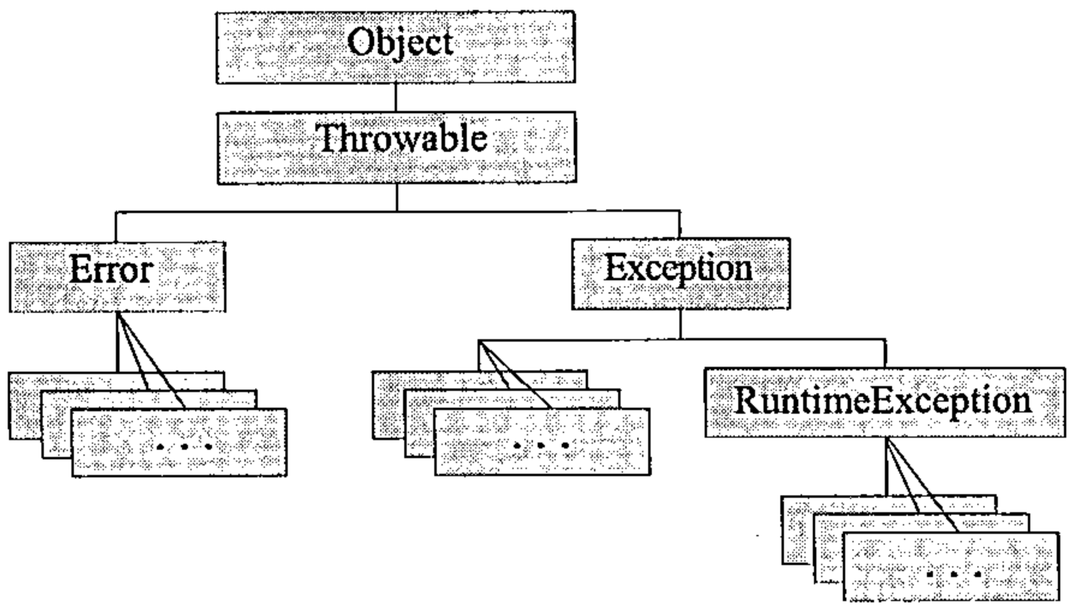


图9-3　Throwable类

1. docs/api/java/lang/Error.html
2. docs/api/java/lang/Exception.html

9.5.3 Error类

当Java虚拟机中发生动态链接故障或其他硬故障时，虚拟机抛出一个Error。典型情况下，简单程序不会捕获或者抛出Error。

9.5.4 Exception类

大多数程序抛出并捕获从Exception类派生的对象。Exception表示发生了一个问题，但并不是严重的系统问题。和Error相反，你编写的大多数程序将抛出并捕获Exception。

Java平台定义了Exception类的许多后裔，这些后裔表示各种可能发生的异常。例如，IllegalAccessException表示无法找到某个方法，而NegativeArraySizeException表示程序试图创建大小为负数的数组。

Exception的一个子类RuntimeException用来表示API的错误应用。运行时异常的一个例子是NullPointerException，当方法试图通过null引用访问对象成员时，发生这种异常。9.6节讨论为什么大多数应用程序都不应该抛出运行时异常或者子类RuntimeException。 248

9.5.5 链式异常

应用程序经常通过抛出一个异常对另一个异常作出响应。实际上，第一个异常导致了第二个异常。了解什么时候一个异常导致另一个异常会非常有帮助。链式异常（chained exception）帮助程序员完成这一工作。

下面是Throwable中支持链式异常的方法和构造器：

```
Throwable getCause()
Throwable initCause(Throwable)
Throwable(String, Throwable)
Throwable(Throwable)
```

initCause方法和Throwable构造器的Throwable实参是导致当前异常的异常。getCause方法返回导致当前异常的异常，而initCause返回当前异常。

下面是显示如何使用链式异常的一个例子：

```
try {

} catch (IOException e) {
    throw new SampleException("Other IOException", e);
}
```

在这个例子中，当捕获到IOException异常时，就会创建新的SampleException异常，导致这个异常的原始异常附加在其后，并且向上抛出链式异常，交给下一个更高级别的异常处理代码。

1. 访问栈踪迹信息

现在我们假设更高级别的异常处理代码希望按照它自己的格式转储栈踪迹。

定义 栈踪迹（stack trace）提供当前线程的执行历史信息，并且列出当出现异常时被调用的类和方法的名称。当异常被抛出时，栈踪迹是有用的调试工具。 249

下列代码显示如何对异常对象调用getStackTrace方法：

```
catch (Exception cause) {
  StackTraceElement elements[] = cause.getStackTrace();
```

```
    for (int i = 0, n = elements.length; i < n; i++) {
      System.err.println(elements[i].getFileName() + ":" +
                         elements[i].getLineNumber() +
                            ">> " +
                            elements[i].getMethodName() + "()");
    }
  }
```

2. 日志记录API

下一个代码段把catch块内什么位置出现异常的情况记录到日志。但是，它没有手工地解析栈踪迹并把输出发送给System.err()，而是使用java.util.logging[1]包中的日志记录工具把输出发送给一个文件：

```
try {
    Handler handler = new FileHandler("OutFile.log");
    Logger.getLogger("").addHandler(handler);

} catch (IOException e) {
    Logger logger = Logger.getLogger("package.name");
    StackTraceElement elements[] = e.getStackTrace();
    for (int i = 0; n = elements.length; i < n; i++) {
      logger.log(Level.WARNING, elements[i].getMethodName());
    }
}
```

9.5.6　创建异常类

选择要抛出的异常的类型时，你可以使用其他人编写的异常类（Java平台提供了许多可以使用的异常类），也可以编写自己的异常类。如果你对下面的任一个问题回答“是”，那么就应该编写自己的异常类，否则就应该使用其他人编写的异常类。

- 你需要的异常类型不能用Java平台中的异常类表示吗？
- 如果让你的用户能够区分你的异常与其他开发者编写的类抛出的异常，对于他们有帮助吗？
- 你的代码要抛出多个相关的异常吗？
- 如果你使用其他人的异常，用户可以访问这些异常吗？换个问法就是，你的包应该是独立的或自包含的吗？

250

1. 示例

假设你正在编写一个链表类。你的链表类支持以下方法：

- **objectAt(int n)**——返回链表中第n个位置上的对象。如果实参小于0或者大于链表中当前的对象数量，那么抛出一个异常。
- **firstObject()**——返回链表中的第一个对象。如果链表中不包含对象，那么抛出一个异常。
- **indexOf(Object o)**——搜索链表，寻找指定的对象并返回它在链表中的位置。如果传递给此方法的对象不在链表中，那么抛出一个异常。

这个链表类可以抛出多个异常，用一个异常处理代码捕获链表抛出的所有异常是比较方便的。另外，如果你打算在一个包中发布这个链表类，那么所有相关的代码应该被打包在一起。所以，这个链表类应该提供一组它自己的异常类。

对于这个链表类抛出的异常，图9-4显示这些异常的一种可能的类层次结构。

---

1. docs/api/java/util/logging/package-summary.html

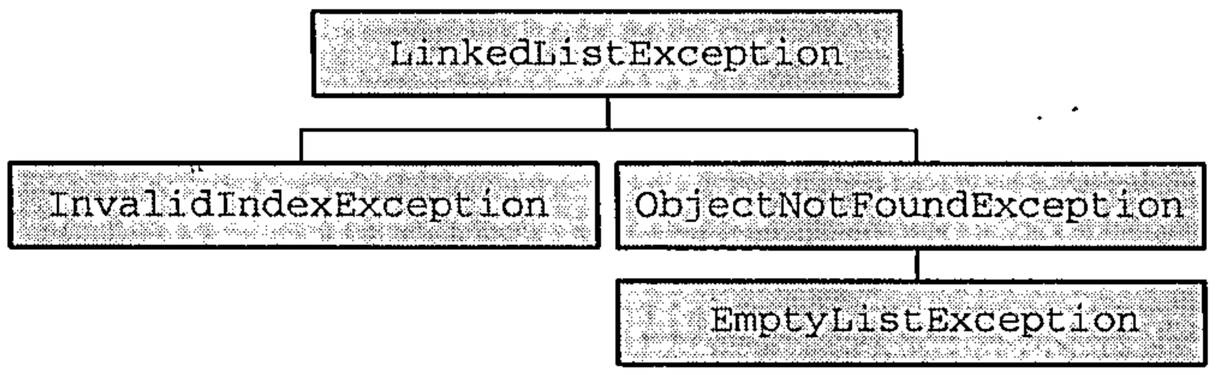


图9-4 示例异常类的层次结构

2. 选择超类

任何Exception子类都可以作为LinkedListException的父类。但是快速地研究这些子类，就会发现它们都不合适，因为它们要么是太特殊化，要么与LinkedListException完全无关。因此，LinkedListException的父类应该是Exception。

251

你编写的大多数applet和应用程序都将抛出Exception对象。通常，Error用于系统中严重的硬错误，比如那些阻止Java虚拟机运行的错误。

注意 为了使代码易于阅读，良好的编程习惯是在从Exception类（直接或间接）派生的所有类的名称后面加上字符串Exception。

9.6 关于不可控异常的争论

因为Java编程语言不要求方法捕获或指定不可控异常（RuntimeException、Error以及它们的子类），所以程序员可能试图编写只抛出可控异常的代码，或者使他们的所有异常子类都从RuntimeException派生。这两种捷径允许程序员编写代码而不必受到来自编译器的错误消息的干扰，也不必费心指定或捕获任何异常。尽管这对于程序员似乎很方便，但是它回避了“捕获或指定需求”的意图，而且会给使用你的类的其他人带来问题。

设计者为什么决定强制方法指定其作用域内可以抛出的所有未被捕获的可控异常？方法可以抛出的任何Exception都是此方法的公共编程接口的一部分。方法的调用者必须了解方法可以抛出的异常，这样才能决定如何处理它们。这些异常与方法的参数和返回值一样，都是方法的编程接口的一部分。

那么，下一个问题可能是：“既然将方法的API，（包括它可以抛出的异常）记入文档是一种好习惯，那么为什么不指定运行时异常呢？”运行时异常代表由编程问题和类似原因导致的问题，不能期望API客户代码通过任何途径合理地从这些问题中恢复或者处理这些问题。这样的问题包括：算术异常（比如被零除）、指针异常（比如试图通过null引用访问对象）和索引异常（比如试图通过太大或太小的索引访问数组元素）。

运行时异常可能在程序中的任何地方发生，而且在典型的程序中这种异常的数量可能非常大。必须在每个方法声明中都加上运行时异常会降低程序的清晰度。所以编译器不要求捕获或指定运行时异常（虽然可以这么做）。

一个常见的编程习惯是，当用户错误地调用某个方法时抛出RuntimeException。例如， 252
方法可以检查它的一个实参是否错误地被指定为null。如果一个实参为null，方法就可能抛出NullPointerException，它是不可控异常。

总的来说，不应该只是因为嫌麻烦，不愿意指定方法可以抛出的异常，就抛出一个RuntimeException或创建一个RuntimeException子类。

基本原则是：如果可以期望客户合理地从异常中恢复，就使它为可控异常。如果客户无法

从异常中恢复，就使它为不可控异常。

9.7　异常的优点

既然你已经了解了什么是异常以及如何使用它们，现在就来了解在程序中使用异常的优点。

9.7.1　优点一：把错误处理代码和“常规”代码分离开

异常提供了把非正常情况下的处理代码与程序的主逻辑分离的途径。在传统的程序设计中，错误的检测、报告和处理经常导致代码混乱。例如，下面的伪代码方法将整个文件读入内存：

```
readFile {
    open the file;
    determine its size;
    allocate that much memory;
    read the file into memory;
    close the file;
}
```

初看起来，这个函数好像很简单，但是它忽略了所有潜在的错误如下：

- 如果无法打开文件，会发生什么情况？
- 如果无法判断文件的长度，会发生什么情况？
- 如果无法分配足够的内存，会发生什么情况？
- 如果读操作失败，会发生什么情况？
- 如果无法关闭文件，会发生什么情况？

为了处理这些情况，readFile函数必须添加更多的代码进行错误的检测、报告和处理。此
253 函数的一个例子可能如下：

```
errorCodeType readFile {
  initialize errorCode = 0;
  open the file;
   if (theFileIsOpen) {
      determine the length of the file;
      if (gotTheFileLength) {
         allocate that much memory;
         if (gotEnoughMemory) {
            read the file into memory;
            if (readFailed) {
               errorCode = -1;
            }
         } else {
            errorCode = -2;
         }
      } else {
         errorCode = -3;
      }
      close the file;
      if (theFileDidntClose && errorCode == 0) {
         errorCode = -4;
      } else {
         errorCode = errorCode and -4;
      }
   } else {
      errorCode = -5;
   }
   return errorCode;
}
```

现在，有了这么多错误检测、报告和返回的代码，原来的7行代码被淹没在这一团乱麻中了。更糟糕的是，代码的逻辑流程也变得非常不清楚，很难判断代码的逻辑是否正确：如果此函数无法分配足够的内存，文件确实会被关闭吗？如果你在编写此函数的三个月后修改它，那么很难确保代码仍然正确。大多数程序员对此问题的解决方式是，忽略它，直到程序崩溃时才报告错误。

异常处理机制使你能够编写代码的主流程，同时在其他地方处理异常情况。如果readFile函数使用异常处理机制而不是传统的错误处理技术，那么它应该像下面这样：

254

```
readFile {
    try {
        open the file;
        determine its size;
        allocate that much memory;
        read the file into memory;
        close the file;
    } catch (fileOpenFailed) {
        doSomething;
    } catch (sizeDeterminationFailed) {
        doSomething;
    } catch (memoryAllocationFailed) {
        doSomething;
    } catch (readFailed) {
        doSomething;
    } catch (fileCloseFailed) {
        doSomething;
    }
}
```

注意，使用异常并不意味着不为检测、报告和处理错误作出努力，但是异常确实能够帮助你更加有效地组织工作。

9.7.2 优点二：把错误沿调用栈向上传递

异常的第二个优点是，可以将错误报告沿调用栈向上传递。假设readFile方法是主程序做的一系列嵌套的方法调用中的第4个方法：method1调用method2，method2调用method3，method3最终调用readFile：

```
method1 {
    call method2;
}
method2 {
    call method3;
}
method3 {
    call readFile;
}
```

假设method1是对readFile中可能发生的错误感兴趣的唯一方法。传统的错误通知技术迫使method2和method3将readFile返回的错误编码沿调用栈向上传递，直到最终到达method1——唯一对错误感兴趣的方法：

255

```
method1 {
    errorCodeType error;
    error = call method2;
    if (error)
        doErrorProcessing;
    else
        proceed;
}
errorCodeType method2 {
```

```
    errorCodeType error;
    error = call method3;
    if (error)
      return error;
    else
      proceed;
}

errorCodeType method3 {
    errorCodeType error;
    error = call readFile;
    if (error)
       return error;
    else
       proceed;
}
```

我们说过，Java运行时环境会沿调用栈往回搜索，寻找可以处理特定异常的方法。一个方法可以不理会其中抛出的任何异常，因为异常会沿调用栈向上传递直至被捕获。因此，只有关心错误情况的方法才必须为检测异常操心：

```
method1 {
    try {
       call method2;
    } catch (exception e) {
       doErrorProcessing;
    }
}
method2 throws exception {
    call method3;
}
method3 throws exception {
    call readFile;
}
```

256

但是，如上面的伪代码所示，回避异常要求在中间的方法中做一些工作。一个方法中可以抛出的任何可控异常都必须在这个方法的throws子句中指定。

9.7.3　优点三：对错误类型进行分组和区分

因为程序内抛出的所有异常都是对象，所以类层次结构的一个自然结果就是对异常进行分组和区分。例如，在Java平台上有一组相关的异常类IOException及其后裔，它们是在java.io中定义的。IOException是最常见的异常类，它代表在执行I/O（输出/输入）任务时可能发生的任何错误类型。它的后裔代表更特殊化的错误。例如，FileNotFoundException表示无法定位磁盘上的一个文件。

方法可以编写特殊化的处理器，以便处理非常特殊的异常。FileNotFoundException类没有后裔，所以下面的处理器只能处理一种类型的异常：

```
catch (FileNotFoundException e) {
  ...
}
```

方法可以在catch语句中指定任何异常的超类来设置异常组或一般类型，进而根据异常组或一般类型捕获异常。例如，要捕获所有I/O异常（无论它们的具体类型是什么），那么异常处理代码可以指定一个IOException实参：

```
catch (IOException e) {
  ...
}
```

此处理器能够捕获所有I/O异常，包括FileNotFoundException、EOFException等。通过查询传递给异常处理代码的实参，你可以了解关于异常的详细信息。例如，下面代码可以输出栈踪迹：

```
catch (IOException e) {
  e.printStackTrace();                  // Output goes to System.err.
  e.printStackTrace(System.out);        // Send trace to stdout.
}
```

甚至可以建立一个处理所有Exception的异常处理代码：

```
catch (Exception e) {    // A (too) general exception handler
  ...
}
```

Exception类接近Throwable类层次结构的顶端。因此，这个处理器除了它真正应该捕获的异常之外，还将捕获许多其他异常。如果你希望程序这样做，例如希望程序向用户输出错误消息，然后退出，可能就想通过这种方式处理异常。

但是在大多数情况下，你会希望异常处理代码尽可能专门化。其原因是，异常处理器必须先判断出现的异常的类型，然后才能决定采用何种最佳的恢复策略。实际上，如果不捕获特定的异常，那么处理器不得不应付任何可能性。过于一般化的异常处理代码会增加代码出错的可能性，因为它们也会捕获和处理程序员没有预料到的那些异常，以及处理器不打算处理的异常。

如前所述，你可以创建异常组并以一般化的方式处理异常，也可以使用特定的异常类型来区分异常，并以更有针对性的方式处理异常。

9.8 小结

程序可以使用异常表明发生了错误。为了抛出异常，可以使用throw语句并且为它提供异常对象——Throwable的后裔，使用这个对象提供关于发生的特定错误的信息。对于抛出未被捕获的可控异常的方法，它的声明中必须包含一个throws子句。

程序可以结合使用try、catch和finally块捕获异常。

- try块标识出可能发生异常的代码块。
- catch语句标识出可以处理特定类型的异常的代码块，这个代码块被称为异常处理代码。
- finally块标识出确保会执行的代码块，这里是在try块代码之后关闭文件、回收资源和通过其他方式进行清理操作的理想位置。

try语句应该至少包含一个catch块或一个finally块，它也可能有多个catch块。

异常对象的类表明被抛出的异常的类型。异常对象可以包含关于错误的更多信息，包括错误消息。使用链式异常，异常A可以指出导致它的另一个异常B，然后可以依次指出导致异常B的异常C，等等。

问题和练习：异常

问题

1. 下列代码合法吗？

```
try {

} finally {

}
```

2. 下列处理器能够捕获什么类型的异常？

```
catch (Exception e) {

}
```

使用这种类型的异常处理代码有什么错误？

3. 下列异常处理代码有什么错误吗？这段代码能编译吗？

```
try {

} catch (Exception e) {

} catch (ArithmeticException a) {

}
```

4. 把下列字母标号清单的每个项和数字标号清单的项匹配起来：

a.
```
int[] A;
A[0] = 0;
```

b. Java虚拟机开始运行你的程序，但是Java虚拟机找不到Java平台类。（Java平台类存放在classes.zip或者rt.jar中。）

c. 程序读取一个流并且到达“end of stream”标记。

d. 在关闭流和到达“end of stream”标记之前，程序试图再次读取流。

(1) ___错误

259 (2) ___可控异常

(3) ___编译错误

(4) ___无异常

练习

1. 在ListOfNumbers.java中添加一个readList方法。这个方法应该从一个文件读取int值，输出这些值，并将它们附加到向量的末尾。你应该捕获所有适当的错误。你还需要一个包含数字的文本文件以供读取。
2. 修改下面的cat方法，使它可以被编译：

```
public static void cat(File file) {
  RandomAccessFile input = null;
  String line = null;

  try {
      input = new RandomAccessFile(named, "r");
      while ((line = input.readLine()) != null) {
        System.out.println(line);
      }
      return;
    } finally {
      if (input != null) {
        input.close();
      }
    }
}
```

答案

可以在以下位置找到“问题”和“练习”的答案：

260 tutorial/essential/exceptions/QandE/answers.html

第10章

基本I/O

本章介绍Java平台中用于基本I/O的类。这一章重点介绍I/O流（I/O stream），这是一个极大简化了I/O操作的功能强大的概念。本章还介绍串行化，它使程序能够把整个对象写出到流中，再读取回来。然后介绍一些文件系统操作，包括随机访问文件。最后，简要介绍新I/O API的高级特性。本章中讲解的大多数类都在`java.io`包中。

安全考虑 一些I/O操作需要当前安全管理器的批准。下面章节中包含的示例程序是独立的应用程序，在默认情况下，这些程序没有安全管理器。为了在applet中应用，大多数这些示例程序都必须经过修改。关于applet的安全限制的更多信息，请参阅18.3.1节。

10.1 I/O流

I/O流（I/O stream）表示一个输入来源或者输出目标。流可以表示很多不同类型的来源或者目的，包括磁盘文件、设备、其他程序和内存数组。

流支持很多不同类型的数据，包括简单字节、基本数据类型、本地化字符和对象。一些流简单地传递数据；而另一些流按照有用的方式处理和转换数据。

不管流在内部如何工作，对于使用流的程序来说，所有流都表示相同的简单模型：流是一系列数据。程序使用输入流（input stream）从来源读取数据，一次一个项（见图10-1）。

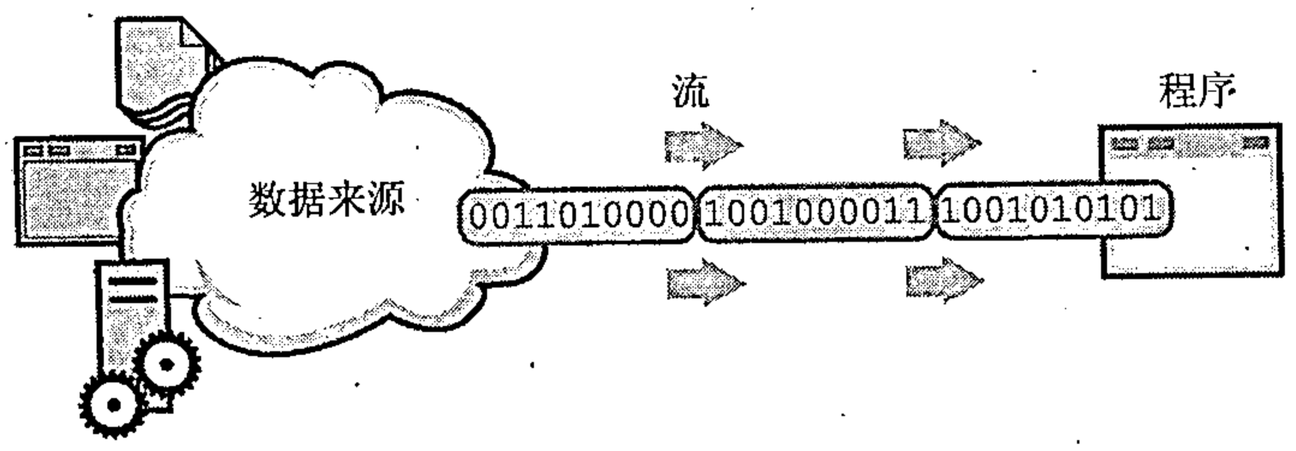


图10-1 把信息读入程序

程序使用输出流（output stream）把数据写入目标，一次一个项（见图10-2）。

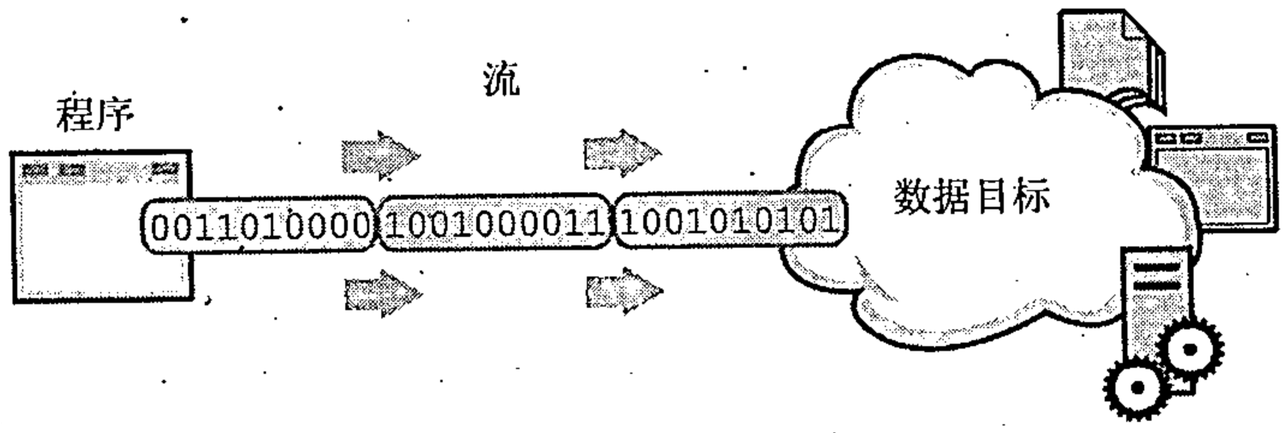


图10-2 从程序写出信息

在本章中，我们将介绍能够处理所有类型的数据的流，这些数据从原始类型到高级对象。

图10-1和图10-2所示的数据来源和数据目标可以是保存、生成或者使用数据的任何位置。显然，它们包括磁盘文件，但是来源或者目标也可以是另一个程序、外部设备、网络套接字或者数组。

在下一节中，我们使用最基本的流类型——字节流——演示I/O流的常见操作。我们使用示例文件xanadu.txt[1]作为示例输入，它包含如下诗句：

```
In Xanadu did Kubla Khan
A stately pleasure-dome decree:
Where Alph, the sacred river, ran
Through caverns measureless to man
Down to a sunless sea.
```

262

10.1.1　字节流

程序使用字节流（byte stream）执行8位字节的输入和输出。所有字节流类都派生于InputStream[2]和OutputStream[3]。

有很多字节流类。为了讲解字节流如何工作，我们重点讲解文件I/O字节流——FileInputStream[4]和FileOutputStream[5]。其他种类的字节流的使用大同小异；它们的区别主要在于构造的方式。

1. 使用字节流

我们将通过分析一个示例程序讲解FileInputStream和FileOutputStream，这个示例程序名为CopyBytes[6]，它使用字节流复制xanadu.txt：

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopyBytes {

  public static void main(String[] args) throws IOException {
    FileInputStream in = null;
    FileOutputStream out = null;
    try {
        in = new FileInputStream("xanadu.txt");
        out = new FileOutputStream("outagain.txt");
        int c;
        while ((c = in.read()) != -1) {
          out.write(c);
        }
    } finally {
        if (in != null) {
          in.close();
        }
        if (out != null) {
          out.close();
        }
```

1. tutorial/essential/io/examples/xanadu.txt
2. docs/api/java/io/InputStream.html
3. docs/api/java/io/OutputStream.html
4. docs/api/java/io/FileInputStream.html
5. docs/api/java/io/FileOutputStream.html
6. tutorial/essential/io/examples/CopyBytes.java

```
        }
    }
}
```

263

CopyBytes的大多数工作都在一个简单的循环中完成，这个循环读取输入流并且写入输出流，一次处理一个字节，如图10-3所示。

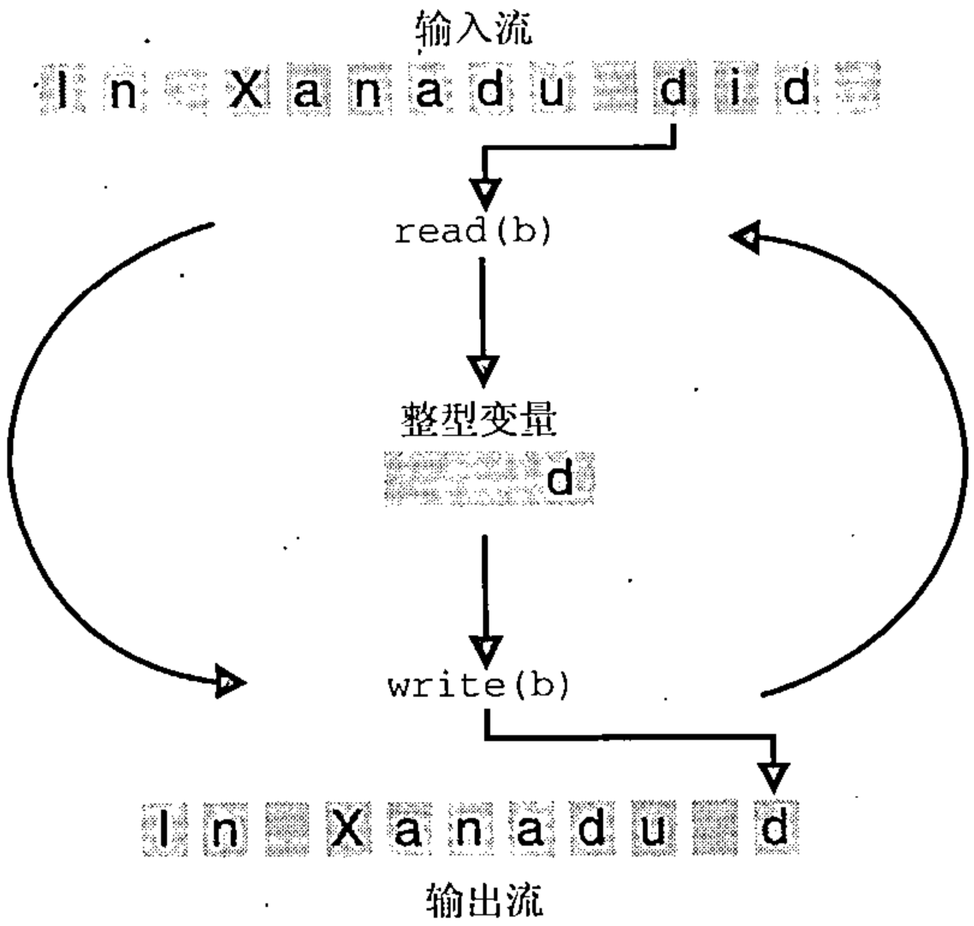


图10-3 简单字节流的输入和输出

注意，read()返回一个int值。如果输入的是字节流，那么read()为什么不返回一个byte值呢？使用int作为返回类型允许read()使用-1表示它到达了流的末尾。

2. 总要关闭流

当不再需要流时要关闭它，这非常重要——所以CopyBytes使用finally块确保两个流都会被关闭，即使出现了错误。这种编程习惯也能够帮助避免资源泄漏。

CopyBytes可能遇到的一个错误是，不能打开一个文件或者两个文件都打不开。出现这种情况时，和文件对应的流变量不会改变其初始的null值。CopyBytes通过这种方式，确保在调用close之前每个流变量都包含对象引用。

264

3. 什么时候不使用字节流

CopyBytes看上去像一个普通程序，但是实际上它代表你应该避免使用的低级I/O类型。因为xanadu.txt包含字符数据，所以最好的途径是使用字符流，见下一节的讨论。还有用于更加复杂的数据类型的流。字节流只应该用于最原始的I/O。

那么我们为什么还要介绍字节流？因为所有其他流类型都是在字节流的基础上构成的。

10.1.2 字符流

Java平台使用Unicode约定存储字符。字符流I/O自动地在这种内部格式和本地字符集之间进行转换。在西方地区，本地字符集通常是ASCII码的8位超集。

对于大多数应用程序来说，使用字符流的I/O和使用字节流的I/O一样简单。流类执行的输入和输出自动地在Unicode字符和本地字符集之间进行转换。使用字符流替换字节流的程序自动适应本地字符集，并且已经准备好进行国际化，无需程序员进行额外处理。

如果国际化不是重要问题，那么可以简单地使用字符流类，无需过多关注字符集的问题。以后，如果国际化的问题变得重要起来，你的程序也能够改编，无需很多重编码工作。关于国

际化请参阅在线教程[1]。

1. 使用字符流

所有字符流类都派生于Reader[2]和Writer[3]。和字节流一样，有专门用于文件I/O的字符流
265 类：FileReader[4]和FileWriter[5]。示例程序CopyCharacters[6]演示这些类：

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopyCharacters {

  public static void main(String[] args) throws IOException {

    FileReader inputStream = null;
    FileWriter outputStream = null;

    try {
        inputStream = new FileReader("xanadu.txt");
        outputStream = new FileWriter("characteroutput.txt");

        int c;
        while ((c = inputStream.read()) != -1) {
          outputStream.write(c);
        }
    } finally {
        if (inputStream != null) {
          inputStream.close();
        }
        if (outputStream != null) {
          outputStream.close();
        }
    }
  }

}
```

CopyCharacters和CopyBytes非常类似。最重要的区别是，CopyCharacters使用FileReader和FileWriter替代FileInputStream和FileOutputStream进行输入和输出。CopyBytes和CopyCharacters都使用int变量读取和写入。但是在CopyCharacters中，int变
266 量在其最后16位中保存一个字符值；而在CopyBytes中，int变量在其最后8位中保存一个字节值。

2. 使用字节流的字符流

字符流常常是字节流的“包装器”。字符流使用字节流执行物理I/O，这时字符流处理字符和字节之间的转换。例如，FileReader使用FileInputStream，而FileWriter使用FileOutputStream。

有两个通用的字节到字符的“桥梁”流：InputStreamReader[7]和OutputStreamWriter[8]。

---

1. docs/books/tutorial/i18n/index.html
2. docs/api/java/io/Reader.html
3. docs/api/java/io/Writer.html
4. docs/api/java/io/FileReader.html
5. docs/api/java/io/FileWriter.html
6. tutorial/essential/io/examples/CopyCharacters.java
7. docs/api/java/io/InputStreamReader.html
8. docs/api/java/io/OutputStreamWriter.html

当没有预打包的字符流类能够满足你的需求时，可以使用它们创建字符流。有一个从网络字节流创建字符流的例子，请参阅在线的套接字课程[1]。

3. 面向行的I/O

字符I/O通常出现在单位大于单一字符的情况下。一个常见的单位就是行：末尾带有行终止符的字符串。行终止符可以是回车 / 换行序列("\r\n")、单一回车("\r")或者单一换行("\n")。通过支持所有可能的行终止符，使程序可以读取在任何广泛使用的操作系统上创建的文本文件。

我们修改示例程序CopyCharacters，让它使用面向行的I/O。为了完成这个工作，我们必须使用两个以前没有见过的类BufferedReader[2]和PrintWriter[3]。在"10.1.3节和10.1.4节的第2小节中，我们将深入讨论这些类。目前，只对它们支持面向行的I/O的特性感兴趣。

示例CopyLines[4]调用BufferedReader.readLine和PrintWriter.println，一次输入和输出一行：

267

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.io.IOException;

public class CopyLines {

  public static void main(String[] args) throws IOException {
    BufferedReader inputStream = null;
    PrintWriter outputStream = null;

    try {
        inputStream =
          new BufferedReader(new FileReader("xanadu.txt"));
        outputStream =
          new PrintWriter(new
                          FileWriter("characteroutput.txt"));
   String l;
   while ((l = inputStream.readLine()) != null) {
     outputStream.println(l);
       }
   } finally {
       if (inputStream != null) {
         inputStream.close();
       }
       if (outputStream != null) {
         outputStream.close();
       }
   }
}
```

调用readLine返回一行文本，其中的行终止符被删除了。CopyLines使用println输出每行文本，它在每行末尾附加当前操作系统使用的行终止符。这个行终止符可能和输入文件中使用的行终止符不同。

---

1. docs/books/tutorial/networking/sockets/readingWriting.html
2. docs/api/java/io/BufferedReader.html
3. docs/api/java/io/PrintWriter.html
4. tutorial/essential/io/examples/CopyLines.java

268 除了字符和行之外，还有很多构成文本输入和输出的方式。关于更多信息，请参阅10.1.4节。

10.1.3 缓冲流

到目前为止，我们见过的大多数示例使用的都是非缓冲（unbuffered）I/O。这就是说，每个读取和写入请求都是由底层操作系统直接处理的。这使程序的效率非常低，因为这样的每个请求通常都会触发磁盘访问、网络活动或者其他一些代价相当高的操作。

为了降低这种类型的开销，Java平台实现了缓冲（buffered）I/O流。缓冲输入流从一个内存区域（称为缓冲区）读取数据；只当缓冲区为空时，才调用本机输入API。类似的，缓冲输出流把数据写入缓冲区，并且只当缓冲区填满时，才调用本机输出API。

程序可以使用我们已经用过几次的包装概念把非缓冲流转换为缓冲流，把非缓冲流对象传递给缓冲流类的构造器。可以像下面这样修改示例CopyCharacters中的构造器调用，让它使用缓冲I/O：

```
inputStream =
  new BufferedReader(new FileReader("xanadu.txt"));
outputStream =
  new BufferedWriter(new FileWriter("characteroutput.txt"));
```

有4个用来包装非缓冲流的缓冲流类：BufferedInputStream[1]和BufferedOutputStream[2]创建缓冲字节流，而BufferedReader和BufferedWriter[3]创建缓冲字符流。

刷新缓冲流

在关键位置写出缓冲，而不等到缓冲区填满，这经常是有意义的。这种操作被称为刷新（flush）缓冲。

一些缓冲输出类支持自动刷新（autoflush），这由一个可选的构造器实参指定。当启用自动刷新时，特定的关键事件会导致缓冲被刷新。例如，自动刷新的PrintWriter对象在每次调用println或者format时刷新缓冲区。关于这些方法的更多介绍，请参阅10.1.4节的第2小节。

要手动刷新流，可以调用它的flush方法。flush方法对任何输出流都是合法的，但是如
269 果流没有被缓冲，它就没有作用。

10.1.4 扫描和格式化

I/O编程经常涉及到人可以阅读的简洁的格式化数据转换。为了帮助你处理这些烦琐的问题，Java平台提供两个API。扫描器API把输入分割为和数据位相关联的记号。格式化API组合数据，形成人可以阅读的简洁的格式化形式。

1. 扫描

Scanner[4]类型的对象用于把格式化输入分割为记号，并且按照它们的数据类型转换各个记号。

(1) 把输入分割为记号

默认情况下，扫描器使用空白分隔记号。（空白字符包括空格、制表符、行终止符。完整清单请参阅Character.isWhitespace的文档[5]。）为了了解扫描如何工作，我们来分析程序ScanXan，它读取xanadu.txt中的各个单词并且输出它们，一行输出一个单词：

---

1. docs/api/java/io/BufferedInputStream.html
2. docs/api/java/io/BufferedOutputStream.html
3. docs/api/java/io/BufferedWriter.html
4. docs/api/java/util/Scanner.html
5. docs/api/java/lang/Character.html

```
import java.io.*;
import java.util.Scanner;

public class ScanXan {

  public static void main(String[] args) throws IOException {
    Scanner s = null;

    try {
      s = new Scanner(new BufferedReader(new
                    FileReader("xanadu.txt")));

      while (s.hasNext()) {
        System.out.println(s.next());
      }
    } finally {
      if (s != null) {
        s.close();
      }
    }
  }
}
```

270

注意，当Scanner对象的工作完成时，ScanXan调用Scanner的close方法。尽管扫描器不是一个流，你也需要关闭它，以便表示完成了对其底层流的处理。

ScanXan的输出就像下面这样：

```
In
Xanadu
did
Kubla
Khan
A
stately
pleasure-dome
...
```

为了使用不同的记号分隔符，可以调用useDelimiter()指定一个正则表达式。例如，假设你希望记号分隔符是一个逗号，后面跟着可选的空白，可以调用：

```
s.useDelimiter(",\\s*");
```

(2) 转换各个记号

ScanXan示例程序把所有输入记号都当作简单的String值对待。Scanner还支持Java语言的所有原始类型（char除外）的记号，也包括BigInteger和BigDecimal。另外，数字值可以使用千位分隔符。因此，当地区为US时，Scanner可以正确地读取表示整数值的字符串“32,767”。

我们必须提到地区，因为千位分隔符和小数点符号是地区特定的。所以，如果我们没有指定扫描器应该使用US地区，下面的例子就不能正确地用于所有地区。通常，你不必担心这类问题，因为输入数据的来源地区和你所在的地区通常是相同的。但是这个示例是本书的一部分，而本书发布到了世界各地。

示例ScanSum[1]读取一系列双精度值，并且把这些值相加。下面是源代码：

271

```
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
import java.util.Scanner;
```

1. tutorial/essential/io/examples/ScanSum.java

```
import java.util.Locale;

public class ScanSum {
  public static void main(String[] args) throws IOException {
    Scanner s = null;
    double sum = 0;
    try {
      s = new Scanner(new
            BufferedReader(new FileReader("usnumbers.txt")));
      s.useLocale(Locale.US);

      while (s.hasNext()) {
        if (s.hasNextDouble()) {
            sum += s.nextDouble();
          } else {
            s.next();
          }
        }
    } finally {
      s.close();
    }
    System.out.println(sum);
  }
}
```

下面是示例输入文件usnumbers.txt[1]：

```
8.5
32,767
3.14159
1,000,000.1
```

输出字符串为“1032778.74159”。在某些地区，其中的小数点会是其他字符，因为System.out是一个PrintStream对象，并且这个类没有提供覆盖默认地区的方式。我们可以覆盖整个程序的地区——或者可以使用格式化，如下一节所示。

2. 格式化

实现格式化的流对象要么是PrintWriter（一个字符流类）的实例，要么是PrintStream[2]（一个字节流类）的实例。

272

注意　你可能需要的PrintStream对象只有System.out和System.err[3]。关于这些对象的更多信息请参阅10.1.5节。当你需要创建格式化输出流时，实例化PrintWriter，而不是PrintStream。

与所有字节流和字符流对象一样，PrintStream和PrintWriter的实例实现一组标准的write方法，用于简单的字节和字符输出。另外，PrintStream和PrintWriter都实现一组相同的方法，用于把内部数据转换为格式化输出。它们提供两个级别的格式化：

- print和println按照标准方式格式化单个值。
- format按照格式字符串格式化几乎任意数量的值，它带有很多用于精确格式化的选项。

(1) print和println方法

使用适当的toString方法对值进行转换之后，调用print和println方法输出单一值。我

1. tutorial/essential/io/examples/usnumbers.txt
2. docs/api/java/io/PrintStream.html
3. docs/api/java/lang/System.html

们通过示例Root[1]讲解这种情况：

```
public class Root {
  public static void main(String[] args) {
    int i = 2;
    double r = Math.sqrt(i);

    System.out.print("The square root of ");
    System.out.print(i);
    System.out.print(" is ");
    System.out.print(r);
    System.out.println(".");

    i = 5;
    r = Math.sqrt(i);
    System.out.println("The square root of " +
                       i + " is " + r + ".");
  }
}
```

下面是Root的输出：

```
The square root of 2 is 1.4142135623730951.
The square root of 5 is 2.23606797749979.
```

273

i和r变量被格式化了两次：第一次使用重载的print中的代码，第二次使用Java编译器自动生成的转换代码，它也使用了toString。你可以通过这种方式格式化任何值，但是对结果没有很多控制能力。

(2) format方法

format方法根据格式字符串（format string）格式化多个实参。格式字符串由嵌入了格式说明符（format specifier）的静态文本构成；除了格式说明符之外，格式字符串在输出中是不变的。

格式字符串支持很多特性。在本书中，我们只介绍一些基本特性。完整介绍请参阅API规范中的格式字符串语法（Format String Syntax）[2]。

Root2[3]示例程序使用一个format调用格式化两个值：

```
public class Root2 {
  public static void main(String[] args) {
    int i = 2;
    double r = Math.sqrt(i);

    System.out.format("The square root of %d is %f.%n", i, r);
  }
}
```

下面是输出：

```
The square root of 2 is 1.414214.
```

就像这个示例中使用的三个格式说明符一样，所有格式说明符都以%开头，以一个或者两个字符转换（conversion）结束，它们指定生成的格式化输出的种类。这里用到的三个转换是：

- d：把整数值格式化为小数值。
- f：把浮点值格式化为小数值。
- n：输出平台特定的行终止符。

---

1. tutorial/essential/io/examples/Root.java
2. docs/api/java/util/Formatter.html
3. tutorial/essential/io/examples/Root2.java

下面是一些其他转换：

- x：把整数格式化为十六进制值。
- s：把任何值格式化为字符串。
- tB：把整数格式化为地区特定的月份名称。

274

还有很多其他转换。

注意　除了%%和%n之外，所有格式说明符都必须和一个参数匹配。如果没有匹配，就会抛出异常。

在Java编程语言中，转义符\n总会生成换行符（\u000A）。除非你明确希望使用换行符，否则不要使用\n。为了获得适用于本地平台的正确的行分隔符，需要使用%n。

除了转换之外，格式说明符还可以包含若干附加元素，它们进一步定制格式化输出。下面是一个示例程序Format[1]，它使用了每个可能的元素类型：

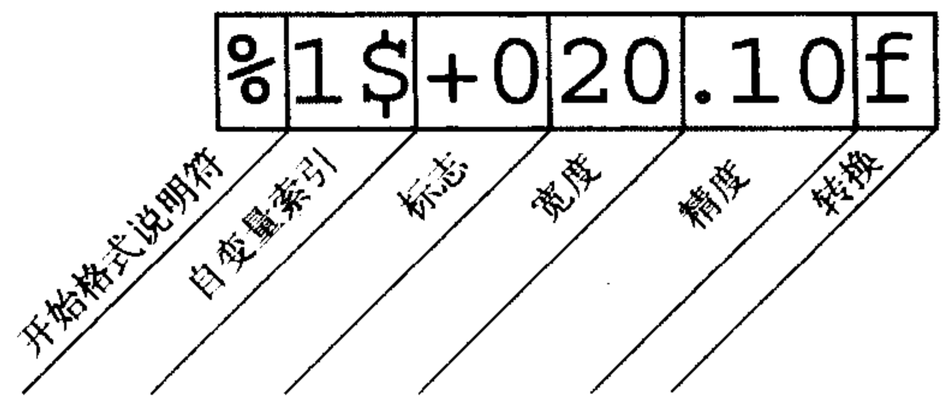
```
public class Format {
  public static void main(String[] args) {
    System.out.format("%f, %1$+020.10f %n", Math.PI);
  }
}
```

下面是程序的输出：

```
3.141593, +00000003.1415926536
```

附加的元素都是可选的。

图10-4显示更长的说明符如何分隔为元素。



275

图10-4　格式说明符的元素

元素出现的顺序必须按照上图显示的顺序。从右侧开始，可选的元素是：

- **精度**——用于浮点值，它是格式化值的数学精度。对于s和其他一般转换，它是格式化值的最大宽度；如果必要，对值进行右侧截断。
- **宽度**——格式化值的最小宽度；如果必要，就对值进行填补。默认情况下，使用空格从左侧填补值。
- **标志**——指定附加的格式化选项。在Format示例中，“+”标志指定总是把数字格式化为带有符号，而“0”标志指定填补字符为0。其他标志包括“-”（填补到右侧）和“,”（使用地区特定的千位分隔符格式化数字）。注意，一些标志不能和其他特定的标志或者特定转换一起使用。
- **实参索引**——允许明确地匹配指定的实参。你也可以使用“<”匹配和前一个说明符相同的实参。因此这个示例可以写为：

1. tutorial/essential/io/examples/Format.java

```
System.out.format("%f, %<+020.10f %n", Math.PI);
```

10.1.5 命令行 I/O

程序经常从命令行运行，并且在命令行环境中和用户进行交互。Java平台通过两种方式提供这种类型的交互：标准流（Standard Stream）和控制台（Console）。

1. 标准流

标准流是很多操作系统具有的特性。默认情况下，它们从键盘读取输入，并且把输出写入到显示设备。它们还支持对文件的I/O和程序之间的I/O，但是这一特性是由命令行解释器控制的，而不是由程序控制。

Java平台支持三种标准流：标准输入（Standard Input），通过System.in访问；标准输出（Standard Output），通过System.out访问；标准错误（Standard Error），通过System.err访问。这些对象都是自动定义的，并且不需要打开它们。标准输出和标准错误都用于输出；单独地输出错误使用户可以把常规输出转移到文件，同时仍然可以阅读错误消息。关于更多信息，请参阅你的命令行解释器的文档。 276

你可能期望标准流是字符流，但是出于历史原因，它们是字节流。System.out和System.err被定义为PrintStream对象。虽然从技术上说它是字节流，但是PrintStream使用内部的字符流对象模拟字符流的很多特性。

与之相反，System.in是不具有字符流特性的字节流。为了把标准输入用作字符流，需要将System.in包装在InputStreamReader中：

```
InputStreamReader cin = new InputStreamReader(System.in);
```

2. 控制台

替代标准流的一种更高级的方式是控制台。它是Console[1]类型的单一预定义对象，它具有标准流提供的大部分特性，还有一些其他特性。控制台特别适用于安全密码的输入。通过其reader和writer方法，控制台对象还提供输入和输出真正的字符流的功能。

程序在能够使用控制台之前，必须通过调用System.console()尝试获得Console对象。如果Console对象可用，这个方法就返回它。如果System.console返回NULL，那么不允许进行控制台操作，这要么是因为操作系统不支持它们，要么因为程序是在非交互环境中启动的。

ReadPassword

Console对象通过其readPassword方法支持安全密码的输入。这个方法通过两种方式帮助保证密码的输入的安全。首先，它抑制回显，所以密码在用户的屏幕上是不可见的。其次，readPassword返回一个字符数组，而不是String，所以不能重写密码，当不再需要时就把它从内存中删除。

Password[2]示例是一个原型程序，用于改动用户密码。它演示几个Console方法： 277

```
import java.io.Console;
import java.util.Arrays;
import java.io.IOException;

public class Password {

  public static void main (String args[]) throws IOException {
```

---

1. docs/api/java/io/Console.html
2. tutorial/essential/io/examples/Password.java

```
Console c = System.console();
if (c == null) {
  System.err.println("No console.");
  System.exit(1);
}

String login = c.readLine("Enter your login: ");
char [] oldPassword = c.readPassword("Enter your " +
                                     old password: ");

if (verify(login, oldPassword)) {
  boolean noMatch;
  do {
    char [] newPassword1 =
        c.readPassword("Enter your new password: ");
    char [] newPassword2 =
        c.readPassword("Enter new password again: ");
    noMatch = ! Arrays.equals(newPassword1, newPassword2);
    if (noMatch) {
        c.format("Passwords don't match. Try again.%n");
    } else {
        change(login, newPassword1);
        c.format("Password for %s changed.%n", login);
    }
    Arrays.fill(newPassword1, ' ');
    Arrays.fill(newPassword2, ' ');
  } while (noMatch);
}

Arrays.fill(oldPassword, ' ');
}

// Dummy verify method.
static boolean verify(String login, char[] password) {
  return true;
}

// Dummy change method.
static void change(String login, char[] password) {}
}
```

278 Password执行如下操作步骤：

(1) 尝试获得Console对象。如果这个对象不可用，则退出。

(2) 调用Console.readLine，提示用户输入登录名并且读取它。

(3) 调用Console.readPassword，提示用户输入现有密码并且读取它。

(4) 调用verify确定用户有权改变密码。(在这个例子中，verify是伪方法，它总是返回true。)

(5) 重复下列步骤，直到用户两次输入相同密码：

   a. 两次调用Console.readPassword，提示用户输入新密码并且读取它。

   b. 如果用户两次输入的密码相同，就调用change把密码改为新密码。(这里的change也是伪方法。)

   c. 使用空格覆盖两个密码。

(6) 使用空格覆盖旧密码。

10.1.6 数据流

数据流支持基本数据类型值(boolean、char、byte、short、int、long、float和double)

和String值的二进制I/O。所有数据流要么实现DataInput[1]接口，要么实现DataOutput[2]接口。这一节重点介绍这些接口的使用最为广泛的实现，DataInputStream[3]和DataOutputStream[4]。

下面的例子通过写出一组数据记录，然后再读入它们，从而演示数据流。每个记录由与发票上的一个项相关的三个值构成，如表10-1所示。

279

表10-1 DataStreams示例中的字段

记录顺序	数据类型	数据描述	输出方法	输入方法	示例值
1	double	商品价格	DataOutputStream.writeDouble	DataInputStream.readDouble	19.99
2	int	单位数量	DataOutputStream.writeInt	DataInputStream.readInt	12
3	String	商品描述	DataOutputStream.writeUTF	DataInputStream.readUTF	"Java T-Shirt"

这个例子很长，所以我们分段解释它。程序的开始定义一些常量，包括数据文件的名称和要写入文件的数据：

```
public class DataStreams {
   static final String dataFile = "invoicedata";

   static final double[] prices =
     { 19.99, 9.99, 15.99, 3.99, 4.99 };
   static final int[] units = { 12, 8, 13, 29, 50 };
   static final String[] descs = { "Java T-shirt",
                                   "Java Mug",
                                   "Duke Juggling Dolls",
                                   "Java Pin",
                                   "Java Key Chain" };
```

DataStreams打开一个输出流。因为DataOutputStream只能被创建为现有字节流对象的包装器，所以DataStreams提供一个缓冲的文件输出字节流：

```
   public static void main(String[] args) throws IOException {
     DataOutputStream out = null;

     try {
       out = new DataOutputStream(new
         BufferedOutputStream(new FileOutputStream(dataFile)));
```

DataStreams写出记录并且关闭输出流：

280

```
     for (int i = 0; i < prices.length; i ++) {
       out.writeDouble(prices[i]);
       out.writeInt(units[i]);
       out.writeUTF(descs[i]);
     }
   } finally {
       out.close();
   }
```

writeUTF方法按照UTF-8的一种修订形式写出String值。这是一种宽度可变的字符编码，对于常用的西方字符，只需要一个字节。

---

1. docs/api/java/io/DataInput.html
2. docs/api/java/io/DataOutput.html
3. docs/api/java/io/DataInputStream.html
4. docs/api/java/io/DataOutputStream.html

现在DataStreams再把数据读入。首先必须提供一个输入流和一个保存输入数据的变量。和DataOutputStream一样，DataInputStream也必须构造为字节流的包装器：

```
DataInputStream in = null;
double total = 0.0;
try {
  in = new DataInputStream(new
    BufferedInputStream(new FileInputStream(dataFile)));

  double price;
  int unit;
  String desc;
```

现在DataStreams可以读取流中的每个记录，报告它得到的数据：

```
  try {
    while (true) {
      price = in.readDouble();
      unit = in.readInt();
      desc = in.readUTF();
      System.out.format("You ordered %d units of %s " +
                        "at $%.2f%n", unit, desc, price);
      total += unit * price;
    }
  } catch (EOFException e) { }
  System.out.format("For a TOTAL of: $%.2f%n", total);
}
finally {
  in.close();
}
```

281

注意，DataStreams通过捕获EOFException[1]异常检测文件结束条件，而不是通过检查非法返回值。DataInput方法的所有实现都使用EOFException，而不是返回值。

还要注意，DataStreams中每个专门的write都严格地和相应的read匹配。由程序员确保输出类型和输入类型按照这种方式匹配：输入流由简单二进制数据构成，没有办法表明各个值的类型或者它们从流中的什么位置开始。

DataStreams使用了一种非常不好的编程技术：它使用浮点数表示货币值。总的来说，浮点数不适用于精确值。对于十进制小数尤其不好，因为常见值（比如0.1）没有二进制表示法。

用于货币值的正确类型是java.math.BigDecimal[2]。不幸的是，BigDecimal是对象类型，所以它不能处理数据流。但是BigDecimal能够处理对象流，我们将在下一节中讨论对象流。

10.1.7　对象流

就像数据流支持基本数据类型的I/O一样，对象流支持对象的I/O。大多数（但不是全部）标准类都支持它们的对象的串行化。支持串行化的类实现标志型接口Serializable[3]。

对象流类是ObjectInputStream[4]和ObjectOutputStream[5]。这些类实现ObjectInput[6]

---

1. docs/api/java/io/EOFException.html
2. docs/api/java/math/BigDecimal.html
3. docs/api/java/io/Serializable.html
4. docs/api/java/io/ObjectInputStream.html
5. docs/api/java/io/ObjectOutputStream.html
6. docs/api/java/io/ObjectInput.html

和ObjectOutput[1]，它们是DataInput和DataOutput的子接口。这意味着，10.1.6节中介绍的所有原始数据I/O方法在对象流中也实现了。所以，对象流可以包含原始值和对象值的组合。示例ObjectStreams[2]演示这种情况：

282

```
import java.io.*;
import java.math.BigDecimal;
import java.util.Calendar;

public class ObjectStreams {

static final String dataFile = "invoicedata";

static final BigDecimal[] prices = {
  new BigDecimal("19.99"),
  new BigDecimal("9.99"),
  new BigDecimal("15.99"),
  new BigDecimal("3.99"),
  new BigDecimal("4.99") };

static final int[] units = { 12, 8, 13, 29, 50 };

static final String[] descs = {
  "Java T-shirt",
  "Java Mug",
  "Duke Juggling Dolls",
  "Java Pin",
  "Java Key Chain" };

public static void main(String[] args)
  throws IOException, ClassNotFoundException {

  ObjectOutputStream out = null;
  try {
    out = new ObjectOutputStream(new
      BufferedOutputStream(new FileOutputStream(dataFile)));

    out.writeObject(Calendar.getInstance());
    for (int i = 0; i < prices.length; i ++) {
      out.writeObject(prices[i]);
      out.writeInt(units[i]);
      out.writeUTF(descs[i]);
    }
  } finally {
    out.close();
  }

  ObjectInputStream in = null;
  try {
    in = new ObjectInputStream(new
      BufferedInputStream(new FileInputStream(dataFile)));

    Calendar date = null;
    BigDecimal price;
    int unit;
    String desc;
```

283

1. docs/api/java/io/ObjectOutput.html
2. tutorial/essential/io/examples/ObjectStreams.java

```
BigDecimal total = new BigDecimal(0);

date = (Calendar) in.readObject();

System.out.format ("On %tA, %<tB %<te, %<tY:%n", date);

try {
  while (true) {
    price = (BigDecimal) in.readObject();
    unit = in.readInt();
    desc = in.readUTF();
    System.out.format("You ordered %d units of %s " +
                      "at $%.2f%n", unit, desc, price);
    total = total.add(price.multiply(new
                                     BigDecimal(unit)));
  }
 } catch (EOFException e) {}
 System.out.format("For a TOTAL of: $%.2f%n", total);
} finally {
  in.close();
}
 }
}
```

ObjectStreams创建和DataStreams功能相同的应用程序，其中有几处改动。首先，现在价格是BigDecimal对象。其次，Calendar[1]对象被写入到数据文件，它表示发票日期。

如果readObject()没有返回预期的对象类型，试图把它转换为正确类型可能会抛出ClassNotFoundException[2]异常。在这个简单的例子中不会出现这种情况，所以我们没有试图捕获这个异常。我们的做法是，通过把ClassNotFoundException添加到main方法的throws子句，从而通知编译器我们注意到了这个问题。

复杂对象的输出和输入

writeObject和readObject的使用很简单，但是它们包含一些非常复杂的对象管理逻辑。这对Calendar这样的类（它仅仅封装了原始值）并不重要。但是很多对象都包含对其他对象的引用。如果readObject要从流重新构成一个对象，它必须能够重新构成原始对象引用的所有对象。这些额外的对象可能具有它们自己的引用，等等。在这种情况下，writeObject遍历对象引用的整个网络，并且把网络中的所有对象都写入到流中。因此对writeObject的一次调用可能导致把很大数量的对象写入到流中。

284

图10-5演示这种情况，其中，调用writeObject是为了写入名为a的单一对象。这个对象包含对对象b和c的引用，而且b包含对d和e的引用。调用writeObject(a)不仅仅写入a，而且还有重新构成a所必需的所有对象，所以这个网络中的其他四个对象也被写入了。当readObject读取回a时，其他四个对象也被读取回来，并且保留所有原始的对象引用。

你可能想知道，如果同一个流中的两个对象都包含对一个对象的引用，会发生什么情况？当把它们读取回来时，它们还都引用这个单一对象吗？回答是“是的”。一个流只能包含对象的一个副本，而它可以包含对这个对象的任意数量的引用。因此，如果你明确地两次把一个对象写入流中，实际写入的只是两次引用。例如，如下代码两次把对象ob写入流中：

```
Object ob = new Object();
out.writeObject(ob);
out.writeObject(ob);
```

---

1. docs/api/java/util/Calendar.html
2. docs/api/java/lang/ClassNotFoundException.html

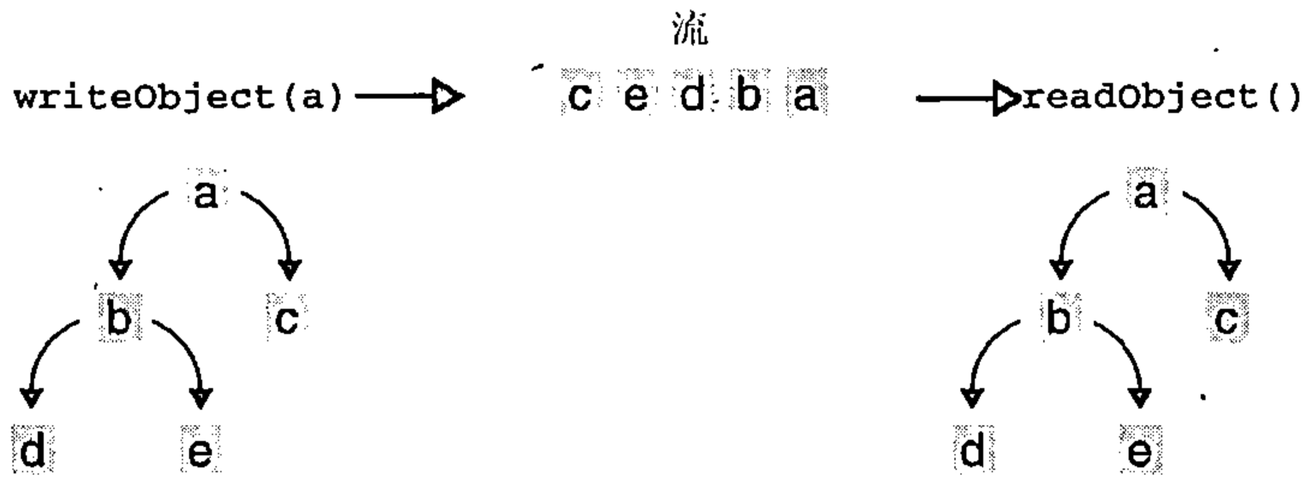


图10-5 多个引用对象的I/O

每个writeObject都必须和一个readObject匹配，所以读回流的代码必须像下面这样：

```
Object ob1 = in.readObject();
Object ob2 = in.readObject();
```

结果是两个变量ob1和ob2，它们引用一个对象。

但是，如果把一个对象写入两个不同的流中，实际上就复制了对象——读取这两个流的单一程序会看到两个不同的对象。

285

10.2 文件 I/O

到目前为止，我们讨论的重点都在流上，流提供用于读取和写入数据的简单模型。流处理大量的数据来源和数据目标，包括磁盘文件。但是，流并不支持对磁盘文件的所有常见操作。在本章的这一部分，我们将重点介绍非流的文件I/O。这包括两个主题：

- File是一个类，它帮助编写用于检查和处理文件和目录的、与平台无关的代码。
- 随机访问文件支持对磁盘文件数据的非顺序访问。

10.2.1 File 对象

File[1]类使编写检查和处理文件的、与平台无关的代码更加容易。这个类的名称有些误导：File实例表示文件名，而不是文件。和文件名对应的文件甚至可能不存在。

为什么对根本不存在的文件创建File对象呢？原因在于，程序可以使用这个对象解析文件名。而且，可以把File对象传递给一些类（比如FileWriter）的构造器，从而创建文件。

如果文件确实存在，程序可以检查其属性，并且对文件执行各种操作，比如重命名、删除或者改变其权限。

1. 具有很多名称的文件

File对象包含用于构造它的文件名字符串。在对象的生存周期中，这个字符串是永远不变的。程序可以使用File对象获得文件名的其他版本，其中一些版本可能和传递给构造器的原始文件名字符串相同，也可能不同。

假设一个程序通过调用构造器创建File对象：

```
File a = new File("xanadu.txt");
```

程序调用很多方法获得文件名的不同版本。然后程序可以运行在Microsoft Windows系统（在目录c:\java\examples中）和Solaris系统（在目录/home/cafe/java/examples中）上。表10-2列出方法的返回值。

286

1. docs/api/java/io/File.html

表 10-2　文件方法示例

调用的方法	Microsoft Windows 系统上的返回值	Solaris 系统上的返回值
a.toString()	xanadu.txt	xanadu.txt
a.getName()	xanadu.txt	xanadu.txt
a.getParent()	NULL	NULL
a.getAbsolutePath()	c:\java\examples\xanadu.txt	/home/cafe/java/examples/xanadu.txt
a.getCanonicalPath()	c:\java\examples\xanadu.txt	/home/cafe/java/examples/xanadu.txt

然后同一个程序从更加复杂的文件名构造File对象，使用File.separator按照与平台无关的方式指定文件名：

```
File b = new File(".." + File.separator + "examples" +
                  File.separator + "xanadu.txt");
```

虽然b和a引用同一文件，但是这个方法的返回值稍有不同，如表10-3所示。

表 10-3　更多文件方法示例

调用的方法	Microsoft Windows 系统上的返回值	Solaris 系统上的返回值
b.toString()	..\examples\xanadu.txt	../examples/xanadu.txt
b.getName()	xanadu.txt	xanadu.txt
b.getParent()	..\examples	../examples
b.getAbsolutePath()	c:\java\examples\..\examples\xanadu.txt	/home/cafe/java/examples/../examples/xanadu.txt
b.getCanonicalPath()	c:\java\examples\xanadu.txt	/home/cafe/java/examples/xanadu.txt

在Linux系统上运行同一个程序会得到和Solaris系统类似的结果。

这里需要解释一下，File.compareTo()不会认为a和b是相同的。尽管它们引用同一个文
287 件，但是构造它们所使用的名称是不同的。

FileStuff[1]示例程序使用从命令行传递来的名称创建File对象，然后对这些对象执行获得信息的各种方法。对各种各样的文件名运行FileStuff，你会发现这个程序具有指导性。要确保包含目录名称，并且使用实际上不存在的文件名。尝试把各种相对和绝对路径名传递给FileStuff：

```
import java.io.File;
import java.io.IOException;
import static java.lang.System.out;

public class FileStuff {

  public static void main(String args[]) throws IOException {

    out.print("File system roots: ");
    for (File root : File.listRoots()) {
      out.format("%s ", root);
    }
    out.println();

    for (String fileName : args) {
      out.format("%n------ %nnew File(%s)%n", fileName);
      File f = new File(fileName);
```

1. tutorial/essential/io/examples/FileStuff.java

```
            out.format("toString(): %s%n", f);
            out.format("exists(): %b%n", f.exists());
            out.format("lastModified(): %tc%n", f.lastModified());
            out.format("isFile(): %b%n", f.isFile());
            out.format("isDirectory(): %b%n", f.isDirectory());
            out.format("isHidden(): %b%n", f.isHidden());
            out.format("canRead(): %b%n", f.canRead());
            out.format("canWrite(): %b%n", f.canWrite());
            out.format("canExecute(): %b%n", f.canExecute());
            out.format("isAbsolute(): %b%n", f.isAbsolute());
            out.format("length(): %d%n", f.length());
            out.format("getName(): %s%n", f.getName());
            out.format("getPath(): %s%n", f.getPath());
            out.format("getAbsolutePath(): %s%n",
                               f.getAbsolutePath());
            out.format("getCanonicalPath(): %s%n",
                               f.getCanonicalPath());
            out.format("getParent(): %s%n", f.getParent());
            out.format("toURI: %s%n", f.toURI());
        }
    }
}
```

288

2. 处理文件

如果File对象中的名称是实际存在的文件，那么程序就可以使用它对文件执行很多有用的操作，其中包括把对象传递给流的构造器，以便打开文件进行读取和写入。

delete方法立即删除文件，而deleteOnExit方法在虚拟机终止时删除文件。

setLastModified设置文件的修改日期/时间。例如，为了把xanadu.txt的修改时间设置为当前时间，程序可以这样写：

```
new File("xanadu.txt").setLastModified(new Date().getTime());
```

renameTo()方法重命名文件。注意，File对象背后的文件名字符串保持不变，所以File对象将不引用重命名后的文件。

3. 处理目录

File有一些用于处理目录的有用方法。

mkdir方法创建一个目录。mkdirs方法首先创建还不存在的所有父目录，然后进行相同操作。

list和listFiles方法列出目录的内容。list方法返回String文件名构成的一个数组，而listFiles返回File对象构成的一个数组。

4. 静态方法

File包含一些有用的静态方法。

createTempFile方法使用唯一名称创建一个新文件，并且返回引用这个文件的File对象。

listRoots返回文件系统根的名称清单。在Microsoft Windows系统上，它是已加载的磁盘的根目录，比如a:\和c:\。在UNIX和Linux系统上，它是根目录/。

10.2.2 随机访问文件

随机访问文件（random access file）允许不按顺序（或者说随机）地访问文件的内容。

考虑名为ZIP的归档格式。ZIP档案包含文件，在典型情况下经过压缩以节省空间。它的尾部还包含一个目录项，用于指出ZIP档案中包含的各个文件从何处开始。此结构见图10-6。

289

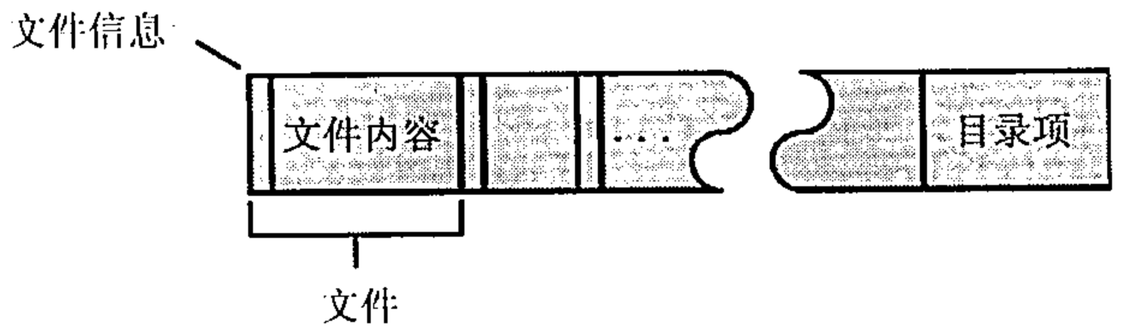


图10-6　ZIP档案

假设你要从ZIP档案中提取某个文件。如果使用顺序访问流，那么必须：

(1) 打开ZIP档案。

(2) 搜索ZIP档案，直至找到要提取的文件。

(3) 提取此文件。

(4) 关闭ZIP档案。

在使用这种过程时，平均来说，必须读过ZIP档案的一半才能找到要提取的文件。使用随机访问文件的搜索特性，可以更高效地从ZIP档案中提取相同的文件，步骤如下：

(1) 打开ZIP档案。

(2) 搜索目录项，找到要从ZIP档案中提取的文件的入口。

(3) 在ZIP档案中（向回）定位到要提取的文件的位置。

(4) 提取此文件。

(5) 关闭ZIP档案。

因为只需要读取目录项和要提取的文件，所以这种算法的效率更高。

`java.io.RandomAccessFile`[1]类实现`DataInput`和`DataOutput`接口，因此它既可以用于读，也可以用于写。`RandomAccessFile`与`FileInputStream`和`FileOutputStream`类似，在创建它时指定本机文件系统上要打开的一个文件。在创建`RandomAccessFile`时，必须指出你的目标是仅仅读取此文件，还是还要写入此文件。（必须能够读取文件，才能写入它。）下面
290 的代码创建一个读取文件`farrago.txt`的`RandomAccessFile`：

```
new RandomAccessFile("xanadu.txt", "r");
```

下面的语句打开相同的文件以便进行读取和写入：

```
new RandomAccessFile("xanadu.txt", "rw");
```

打开文件后，可以使用`DataInput`和`DataOutput`接口中定义的通用`read`和`write`方法对文件执行I/O操作。

`RandomAccessFile`支持文件指针（file pointer）的概念（见图10-7）。文件指针指出文件中的当前位置。在首次创建文件时，文件指针被设置为0，表示文件的开头。调用`read`和`write`方法时，按照读取或者写入的字节数量调整文件指针。

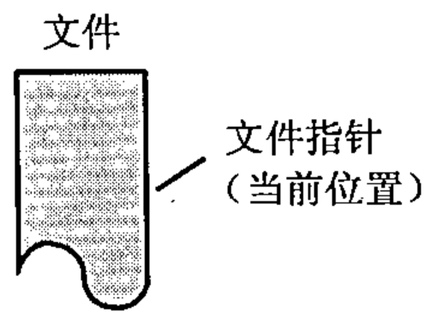


图10-7　具有当前文件指针概念的ZIP文件

除了那些在操作发生时隐式地移动文件指针的一般文件I/O方法之外，`RandomAccessFile`

1. docs/api/java/io/RandomAccessFile.html

还包含三个用于显式地操作文件指针的方法：

- **int skipBytes(int)**——将文件指针向前移动指定的字节数。
- **void seek(long)**——将文件指针定位到指定字节的前面。
- **long getFilePointer()**——返回文件指针的当前字节位置。

10.3 新I/O包

本章讨论的几乎都是java.io包。这个包提供大多数程序员需要的所有I/O特性。通过简单且容易使用的API，这个包实现了基本I/O服务——数据流、随机访问文件、字符转换和缓冲。

但是，编写高性能应用程序的一些程序员需要比java.io包特性更高的灵活性。可以在java.nio.*包中找到这样的特性。这些包提供的API用于可伸缩的I/O、快速缓冲字节和字符I/O以及字符集转换。

291

10.4 小结

java.io包中包含许多可以用来读取和写入数据的类。这些类中的大多数实现顺序访问流。顺序访问流可以分为两组：一组读取和写入字节，一组读取和写入Unicode字符。每种顺序访问流都有其专长，比如读取或者写入文件、在读取或者写入数据时对数据进行过滤，或者对对象进行串行化。

RandomAccessFile类实现对文件的随机输入/输出访问。此类型的对象维护一个文件指针，文件指针指出进行数据读取和写入的当前位置。

问题和练习：基本I/O

问题

1. 要读取靠近大型文件尾部的一个已知位置的几段数据，应该使用哪个类？
2. 在format调用中，表示换行的最佳途径是什么？
3. 如何把数据附加到文件的末尾？写出你要使用的类的构造器，并且解释它。

练习

1. 实现一对类（一个Reader和一个Writer），它们计算特定字符（比如e）被读取和写入的次数。当创建流时可以指定这个字符。编写一个程序测试你的类。可以使用xanadu.txt作为输入文件。
2. 文件datafile[1]的开头是一个long值，它表示同一个文件中数据的int部分的偏移量。使用RandomAccessFile类编写一个程序，获得这段int数据。这段int数据是什么？

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/essential/io/QandE/answers.html

292

1. tutorial/essential/io/QandE/datafile

第11章 集 合

本章介绍Java集合框架（Java Collections Framework）。你将学习集合是什么，以及它们如何简化你的工作并优化程序。你将学习构成集合框架的核心元素：接口、实现和算法。

11.1 集合简介

集合（collection）有时称为容器，是一个将多个元素组合到一个单元的对象。集合用于存储、获取、操纵聚合数据，以及与聚合数据通信。典型情况下，集合代表一个自然的组中的数据项，比如一副牌（纸牌的集合）、一个邮件包（信件的集合）或一个电话簿（名字到电话号码的映射）。

如果你以前使用过Java编程语言（或任何其他编程语言），那么可能已经熟悉集合的概念。在Java平台的早期版本中（版本1.2之前），集合的实现包括Vector[1]（向量）、Hashtable[2]（散列表）和数组[3]。但是，这些早期版本不包含集合框架。

11.1.1 集合框架是什么

集合框架是一个用于表示和操作集合的统一的体系结构。所有集合框架都包含如下元素：

- **接口**——它们是代表集合的抽象数据类型。接口允许操作集合而不涉及集合所代表的细节。在面向对象的语言中，这些接口一般组成一个层次结构。
- **实现**——它们是集合接口的具体实现。它们实质上是可重用的数据结构。
- **算法**——它们是在实现集合接口的对象上执行运算的方法，比如搜索和排序。这些算法被称为是多态的（polymorphic）：也就是说，相同的方法可以用于处理某个集合接口的许多不同实现。实质上，算法是可重用的功能。

除了Java的集合框架之外，最著名的集合框架例子是C++的标准模板库（Standard Template Library，STL）和Smalltalk的集合层次结构。在历史上，集合框架曾经非常复杂，以难学习著称。我们相信Java集合框架打破了这一传统认识，你将在本章中了解到这一点。

11.1.2 Java 集合框架的优势

1. 减少编程工作量

集合框架提供了有用的数据结构和算法，这样你就可以把精力集中于程序的重要部分，而不必费心处理编程工作所需的低级别细节。Java集合框架还促进了无关的API之间的互操作性，这样你就不必为了连接API而编写大量适配器对象或转换代码。

---

1. docs/api/java/util/Vector.html
2. docs/api/java/util/Hashtable.html
3. 参见3.1.3节。

2. 提高程序的运行速度和质量

这种集合框架提供了有用的数据结构和算法的高性能、高质量实现。因为每个接口的各种实现是可以互换的，所以可以很容易地通过更换集合的实现来调整程序。另外，因为你不必辛苦地编写自己的数据结构，所以有更多的时间来设法提高程序的质量和性能。

294

3. 允许无关的API之间的互操作性

集合接口是API之间交流的本地语言，API通过集合接口来回传递集合。如果我的网络管理API提供一个节点名称的集合，而你的GUI工具箱需要一个列标题的集合，那么这两个API可以无缝地互操作，尽管它们是独立编写出来的。

4. 减少学习和使用新API的难度

许多API很自然地采用集合作为输入，并且提供集合作为输出。过去，每个这样的API都有一个小的子API，用来操作它的集合。而这些特殊的子API之间几乎完全没有一致性，所以你不得不从头学习每个子API，而且使用它们时很容易出错。有了标准的集合接口，这个问题就解决了。

5. 减少设计新API的工作量

这是前一个优点的一个侧面。在每次创建依赖于集合的API时，设计人员和实现者不必从头开始，他们可以使用标准的集合接口。

6. 促进软件重用

符合标准集合接口的新数据结构天生就是可重用的。在实现这些接口的对象上进行操作的新算法也是可重用的。

11.2 接口

核心集合接口（core collection interface）封装不同类型的集合，如图11-1所示。这些接口允许独立地操作集合，而不涉及集合所代表的细节。核心集合接口是Java集合框架的基础。如图11-1所示，核心集合接口形成了一个层次结构。

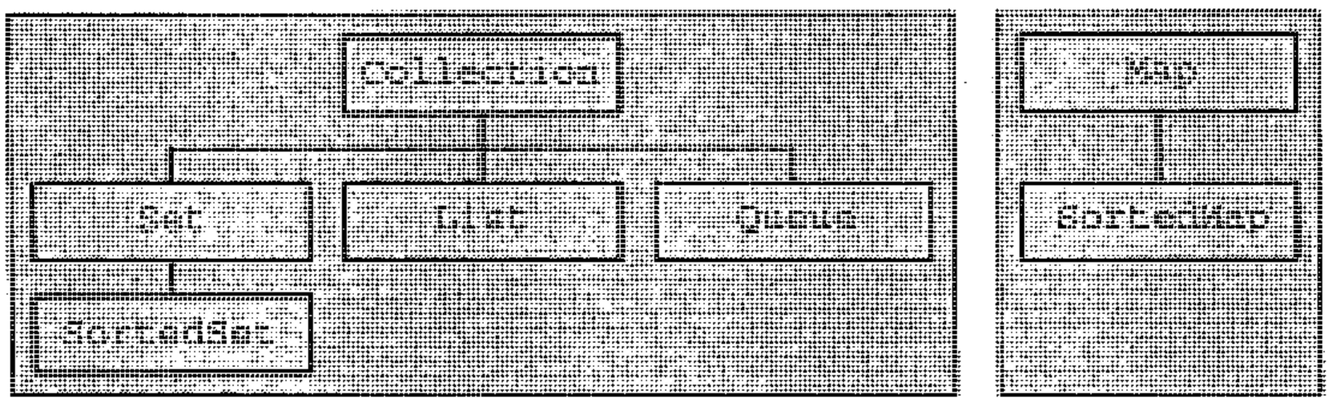


图11-1 核心集合接口

Set是一种特殊的Collection，SortedSet是一种特殊的Set，依此类推。还要注意，这个层次结构由两个独立的树构成：Map并不是真正的Collection。

注意，所有核心集合接口都是泛型。例如，下面是Collection接口的声明：

295

```
public interface Collection<E>...
```

<E>语法通知你这个接口是一个泛型。当你声明Collection实例时，可以并且应该指定这个集合中包含的对象的类型。指定类型允许编译器验证（在编译时）你放到这个集合中的对象类型是否正确，这样就减少了运行时错误。关于泛型的更多信息请参阅第6章。

当你了解了如何使用这些接口后，就在很大程度上了解了Java集合框架。这一节讨论有效使用接口的总指导方针，包括何时使用哪个接口。你还将学习每个接口的编程概念，以便充分

地利用接口。

为了将核心集合接口的数量控制在可管理的范围内，Java平台没有为每个集合类型的每个变体提供单独的接口。（这样的变体可能包含不可变的、大小固定的或只能进行附加操作的集合。）相反，每个接口中的修改操作是可选的——给定的实现可能选择不支持这些操作。如果调用一个未被支持的操作，集合就会抛出一个UnsupportedOperationException[1]异常。接口的实现要负责用文档记录它们支持的可选操作。Java平台的所有通用实现都支持所有可选操作。

下面的清单描述核心集合接口：

- **Collection**——集合层次结构的根，它代表一组对象，这些对象被称为它的元素（element）。Collection接口是所有接口实现的基础，在需要最大程度的通用性时可以使用它传递和操作集合。某些类型的集合允许重复的元素，而其他则不允许。某些是有序的，而其他则是无序的。Java平台没有提供这个接口的直接实现，但是提供了更特殊化的子接口（比如Set和List）的实现。参见11.2.1节。

296

- **Set**——不包含重复元素的集合。如你所期望的，这个接口模拟数学上集的概念，用于代表集，比如组成一副纸牌的各张牌、组成课程表的各门课程或在机器上运行的进程。参见11.2.2节。
- **List**——有次序的集合（有时被称为序列）。List可以包含重复元素。List的用户一般可以精确地控制每个元素被插入到List中的什么位置，并且可以通过元素的整数索引（位置）访问元素。如果你使用过Vector，那么可能熟悉List的一般风格。参见11.2.3节。
- **Queue**——在处理之前用于保存多个元素的集合。除了基本的Collection操作之外，Queue还提供附加的插入、提取和检查操作。

  典型情况下（但不是必须的），队列按照FIFO（先进先出）的方式管理元素。例外的情况是优先级队列，它按照提供的比较器或者元素的自然顺序对元素进行排序。不管使用什么排序顺序，队列的头都是被remove或者poll调用删除的元素。在FIFO队列中，所有新元素都插入到队列的末尾。其他种类的队列可能使用不同的安排原则。每个Queue实现都必须指定其排序属性。参见11.2.4节。
- **Map**——将键映射到值的对象。Map不能包含重复的键：每个键最多能映射到一个值。如果你使用过Hashtable，那么可能熟悉Map的基础知识。参见11.2.5节。

  下面是最后两个核心集合接口，它们只是Set和Map的有序版本：
- **SortedSet**——按照升序维护其元素的Set。它提供了几种附加操作来利用排序。有序集用于具有自然次序的集，比如词汇表或花名册。参见11.2.7节。
- **SortedMap**——按键的升序维护其映射的Map。它是SortedSet的Map模拟。有序映射用于具有自然次序的键/值对的集合，比如字典和电话簿。参见11.2.8节。

297　要想了解有序接口如何维护其元素的顺序，请参阅11.2.6节。

11.2.1　Collection接口

Collection[2]代表一组对象，这些对象称为它的元素。在需要最大程度的通用性的情况下，使用Collection接口传递对象的集合。例如，按照约定，所有通用集合实现都具有采用Collection实参的构造器。这个构造器被称为转换构造器（conversion constructor），它将新集

1. docs/api/java/lang/UnsupportedOperationException.html
2. docs/api/java/util/Collection.html

合初始化为包含指定集合中的所有元素，而不管给定集合的子接口或者实现类型是什么。换句话说，它允许你转换接口的类型。

例如，假设你有一个Collection<String> c，它可能是一个List、一个Set或者另一种类型的Collection。下面的代码创建新的ArrayList（List接口的实现），它初始化为包含c中的所有元素：

```
List<String> list = new ArrayList<String>(c);
```

下面显示Collection接口：

```
public interface Collection<E> extends Iterable<E> {
  // Basic operations
  int size();
  boolean isEmpty();
  boolean contains(Object element);
  boolean add(E element);          // optional
  boolean remove(Object element);  // optional
  Iterator<E> iterator();

  // Bulk operations
  boolean containsAll(Collection<?> c);
  boolean addAll(Collection<? extends E> c);     // optional
  boolean removeAll(Collection<?> c);            // optional
  boolean retainAll(Collection<?> c);            // optional
  void clear();                                  // optional

  // Array operations
  Object[] toArray();
  <T> T[] toArray(T[] a);
}
```

Collection代表一组对象，这个接口会完成你期望的操作。此接口的方法可以告诉你集合中有多少个元素（size和isEmpty），可以检查集合中是否有给定的对象（contains），可以在集合中添加和删除一个元素（add和remove），并可以提供针对集合的迭代器（iterator）。 298

add方法被定义为通用的，这样它对于允许和不允许重复元素的集合都有意义。它确保在调用完成后，Collection包含指定的元素；如果对Collection的修改成功，那么此方法返回true。类似的，remove方法用于从Collection删除指定元素的单一实例，前提是这个Collection包含此元素；如果对Collection的修改成功，那么此方法返回true。

1. 遍历集合

可以通过两种方式遍历集合：(1)使用for-each结构，(2)使用Iterator。

(1) for-each结构

for-each结构允许你使用for循环（见3.4.4节）简洁地遍历一个集合或数组。下面的代码使用for-each结构输出集合的每个元素，一行一个元素：

```
for (Object o : collection)
  System.out.println(o);
```

(2) 迭代器

Iterator[1]是一个对象，它允许你遍历一个集合，如果你愿意，还可以选择性地删除集合的元素。通过调用集合的iterator方法获得集合的Iterator。下面是Iterator接口：

```
public interface Iterator<E> {
  boolean hasNext();
```

1. docs/api/java/util/Iterator.html

```
    E next();
    void remove(); // optional
}
```

如果迭代中有更多的元素，hasNext方法就返回true，next方法返回迭代中的下一个元素。remove方法删除next方法从底层Collection返回的最后一个元素。对于next方法的每次调用，只能调用一次remove方法；如果违反这条规则，就会抛出一个异常。

注意，在迭代过程中，Iterator.remove是修改集合的唯一的安全途径；如果在迭代执行
299 期间通过其他任何方式修改底层集合，那么产生的行为将是不确定的。

当需要进行如下操作时，需要使用Iterator，而不要使用for-each结构：

- 删除当前元素。for-each结构隐藏迭代器，所以不能调用remove。因此，for-each结构不能用于过滤。
- 并行地迭代多个集合。

下面的方法显示如何使用Iterator过滤一个任意的Collection——就是说，在遍历集合的同时删除特定元素：

```
static void filter(Collection<?> c) {
  for (Iterator<?> it = c.iterator(); it.hasNext(); )
    if (!cond(it.next()))
      it.remove();
}
```

这个代码段是多态的，这就是说无论实现如何，它都可以处理任何Collection。这个例子展示了使用Java集合框架编写多态算法有多么简单。

2. Collection接口的批量操作

批量操作（bulk operation）对整个Collection执行操作。可以使用基本操作实现这些简捷操作，但是在大多数情况下，这样的实现的效率比较低。下面是批量操作。

- **containsAll**——如果目标Collection包含指定的Collection中的所有元素，那么此方法返回true。
- **addAll**——将指定的Collection中的所有元素添加到目标Collection。
- **removeAll**——从目标Collection中删除在指定的Collection中包含的所有元素。
- **retainAll**——从目标Collection中删除在指定的Collection中不包含的所有元素。也就是说，它只保留目标Collection和指定Collection中同时存在的那些元素。
- **clear**——从Collection中删除所有元素。

如果在执行操作的过程中目标Collection被修改了，那么addAll、removeAll和retainAll都返回true。

作为批量操作的简单例子，考虑以下语句，此语句从Collection c中删除指定元素e的所
300 有实例：

```
c.removeAll(Collections.singleton(e));
```

再看一个更特殊化的例子，假设你要从一个Collection中删除所有null元素：

```
c.removeAll(Collections.singleton(null));
```

此语句使用了Collections.singleton，这是一个静态工厂方法，它返回一个只包含指定元素的不可变的Set。

3. Collection接口的数组操作

Collection接口提供了toArray方法，作为集合与以数组作为输入的旧式API之间的桥梁。数组操作允许将Collection的内容转换为数组。没有参数的简单形式创建一个新的

Object数组。复杂一些的形式允许调用者提供一个数组或者选择输出数组的运行时类型。

例如，假设c是一个Collection。下面的代码段将c的内容转储到一个新分配的Object数组中，此数组的长度等于c中的元素数量：

```
Object[] a = c.toArray();
```

假设知道c只包含字符串（可能因为c的类型是Collection<String>）。下面的代码段将c的内容转储到一个新分配的String数组中，此数组的长度等于c中的元素数量：

```
String[] a = c.toArray(new String[0]);
```

11.2.2 Set 接口

Set[1]是不包含重复元素的Collection。Set模拟数学上的集（set）概念。Set接口只包含从Collection继承的方法，并且添加了禁止重复元素这一限制。Set还在equals和hashCode操作上添加了更强的约束，这允许对Set实例进行有意义的比较（即使它们的实现类型不同）。如果两个Set对象包含相同的元素，那么它们就是相等的。

下面是Set接口：

301

```
public interface Set<E> extends Collection<E> {
  // Basic operations
  int size();
  boolean isEmpty();
  boolean contains(Object element);
  boolean add(E element);                    // optional
  boolean remove(Object element);            // optional
  Iterator<E> iterator();

  // Bulk operations
  boolean containsAll(Collection<?> c);
  boolean addAll(Collection<? extends E> c);     // optional
  boolean removeAll(Collection<?> c);            // optional
  boolean retainAll(Collection<?> c);            // optional
  void clear();                                  // optional

  // Array Operations
  Object[] toArray();
  <T> T[] toArray(T[] a);
}
```

Java平台包含三个通用的Set实现：HashSet[2]、treeSet[3]和LinkedHashSet[4]。HashSet在散列表中存储其元素，它是性能最好的实现；但是它不能保证迭代的顺序。TreeSet在红-黑树中存储其元素，按照元素的值排列元素顺序；实质上它比HashSet要慢一些。LinkedHashSet实现其中带有链表的散列表，按照元素被插入到集中的顺序（插入顺序）排列元素顺序。LinkedHashSet使其用户不必使用HashSet提供的未指定的、通常乱序的集，其代价稍微高些。

下面是一个简单但有用的Set语句。假设你有一个Collection c，想要创建另一个包含相同元素的Collection，但是要去掉所有重复的元素。下面这行代码就实现此功能：

```
Collection<Type> noDups = new HashSet<Type>(c);
```

---

1. docs/api/java/util/Set.html
2. docs/api/java/util/HashSet.html
3. docs/api/java/util/TreeSet.html
4. docs/api/java/util/LinkedHashSet.html

它创建一个Set（根据定义，Set不能包含重复元素），此Set被初始化为包含c中的所有元
302 素。它使用11.2.1节中描述的标准转换构造器。

下面是这个语句的一个小变体，它保持原始集合中元素的顺序，同时删除重复元素：

```
Collection<Type> noDups = new LinkedHashSet<Type>(c);
```

下面是一个泛型方法，它封装前面的语句，返回类型与传递给它的泛型类型相同的Set：

```
public static <E> Set<E> removeDups(Collection<E> c) {
  return new LinkedHashSet<E>(c);
}
```

1. Set接口的基本操作

size操作返回Set中的元素数量（它的基数[cardinality]）。isEmpty当然用于检查Set是否为空。add方法将指定的元素添加到Set，条件是此元素在当前Set中不存在。它返回一个布尔值，表示元素是否被添加了。类似的，remove方法从Set中删除指定的元素（如果它存在的话），并且返回一个布尔值，表示元素是否被删除。iterator方法返回Set上的一个Iterator对象。

下面这个程序[1]从它的实参列表得到一些单词，并且显示重复的单词、不重复单词的数量以及消除重复后的单词列表：

```
import java.util.*;

public class FindDups {
  public static void main(String[] args) {
    Set<String> s = new HashSet<String>();
    for (String a : args)
      if (!s.add(a))
        System.out.println("Duplicate detected: " + a);
    System.out.println(s.size() + " distinct words: " + s);
  }
}
```

现在运行这个程序：

303

```
java FindDups i came i saw i left
```

产生的输出如下：

```
Duplicate detected: i
Duplicate detected: i
4 distinct words: [i, left, saw, came]
```

注意，此代码总是通过它的接口类型（Set）引用集合，而不是通过它的实现类型（HashSet）。**强烈推荐采用这种编程方式，因为它使你只通过改变构造器就能改变实现。如果用于存储集合的变量或用于传递给集合的参数被声明为Collection的实现类型，而不是接口类型，那么要想改变集合的实现类型，就必须修改所有这样的变量和参数。**

此外，这种情况下得到的程序不一定能够工作。如果程序使用了任何非标准的操作，这种操作在原来的实现中存在，而在新的实现中不存在，那么程序将失败。只通过接口引用集合可以禁止程序使用任何非标准操作。

在前面的例子中，Set的实现类型是HashSet，此实现类型不保证Set中的元素的次序。如果你想让程序按字母表顺序输出单词列表，那么只需将Set的实现类型从HashSet改为TreeSet。只修改一行就能使前面例子中的命令行产生以下输出：

```
java FindDups i came i saw i left
```

---

1. tutorial/collections/interfaces/examples/FindDups.java

```
Duplicate detected: i
Duplicate detected: i
4 distinct words: [came, i, left, saw]
```

2. Set接口的批量操作

批量操作特别适合Set；在应用于Set时，它们执行标准的集代数操作。假设s1和s2是Set。下面是各种批量操作的功能：

- **s1.containsAll(s2)**——如果s2是s1的**子集**，那么此方法返回true。（如果集s1包含s2中的所有元素，那么s2就是s1的子集。）
- **s1.addAll(s2)**——将s1转换为s1和s2的**并集**。（两个集的并集包含这两个集中的所有元素。）
- **s1.retainAll(s2)**——将s1转换为s1和s2的**交集**。（两个集的交集只包含这两个集中都有的元素。）
- **s1.removeAll(s2)**——将s1转换为s1和s2的（不对称的）差集。（例如，s1减去s2的差集包含在s1中存在但s2中不存在的所有元素。）

304

为了在不造成破坏的情况下计算两个集合的并集、交集和差集（也就是说，不改变这些集本身），调用者必须在调用合适的批量操作前复制一个集。语句如下：

```
Set<Type> union = new HashSet<Type>(s1);
union.addAll(s2);

Set<Type> intersection = new HashSet<Type>(s1);
intersection.retainAll(s2);

Set<Type> difference = new HashSet<Type>(s1);
difference.removeAll(s2);
```

在前面的代码中，产生的Set的实现类型是HashSet，我们说过，HashSet是Java平台中最好的Set实现。但是，也可以改为使用任何通用Set实现。

我们再来看看FindDups程序。假设你想知道哪些单词在实参列表中只出现了一次，哪些单词出现多次，但是不想重复输出任何单词。要达到此目标需要产生两个集，一个集包含实参列表中的每个单词，另一个集只包含重复的单词。只出现一次的单词是这两个集的差集，我们知道如何计算出它。产生的程序[1]如下所示：

```
import java.util.*;

public class FindDups2 {
  public static void main(String[] args) {
    Set<String> uniques = new HashSet<String>();
    Set<String> dups    = new HashSet<String>();

    for (String a : args)
      if (!uniques.add(a))
        dups.add(a);

    // Destructive set-difference
    uniques.removeAll(dups);

    System.out.println("Unique words:    " + uniques);
    System.out.println("Duplicate words: " + dups);
  }
}
```

305

1. tutorial/collections/interfaces/examples/FindDups2.java

仍然使用以前使用的实参列表来运行此程序（i came i saw i left），产生的输出如下：

```
Unique words:    [left, saw, came]
Duplicate words: [i]
```

一种不太常用的集代数操作是对称的差集（symmetric set difference）：这种集的元素存在于指定的两个集之一，但是不同时存在于这两个集中。下面的代码在不造成破坏的情况下计算两个集的对称差集：

```
Set<Type> symmetricDiff = new HashSet<Type>(s1);
symmetricDiff.addAll(s2);
Set<Type> tmp = new HashSet<Type>(s1);
tmp.retainAll(s2));
symmetricDiff.removeAll(tmp);
```

3. Set接口的数组操作

Set的数组操作没有什么特殊之处，它们的作用与在其他任何Collection上的数组操作一样。11.2.1节的第3小节中描述了数组操作。

11.2.3 List 接口

List[1]是一个有序的Collection（它有时被称为序列[sequence]）。列表可能包含重复的元素。除了从Collection继承的操作，List接口还包含以下操作：

- **位置访问**——根据元素在列表中的编号位置操作元素。
- **搜索**——在列表中搜索指定的元素并返回它的编号位置。
- **迭代**——扩展Iterator语义以便利用列表的顺序性质。
- **范围视图**——在列表上执行任意的范围操作（range operation）。

306

List接口如下所示：

```
public interface List<E> extends Collection<E> {
  // Positional access
  E get(int index);
  E set(int index, E element);      // optional
  boolean add(E element);           // optional
  void add(int index, E element);   // optional
  E remove(int index);              // optional
  boolean addAll(int index,
    Collection<? extends E> c);     // optional

  // Search
  int indexOf(Object o);
  int lastIndexOf(Object o);

  // Iteration
  ListIterator<E> listIterator();
  ListIterator<E> listIterator(int index);

  // Range-view
  List<E> subList(int from, int to);
}
```

Java平台包含两个通用的List实现：ArrayList[2]和LinkedList[3]，ArrayList一般是性

1. docs/api/java/util/List.html
2. docs/api/java/util/ArrayList.html
3. docs/api/java/util/LinkedList.html

能较好的实现，而LinkedList在某些环境下具有较好的性能。另外，Vector已经为实现List而进行了改进。

1. 与Vector比较

如果你使用过Vector，那么应该熟悉List的基础知识。（当然，List是一个接口，而Vector是一个具体的实现。）List还修正了Vector中几个次要的API的缺陷。经常使用的Vector操作（比如elementAt和setElementAt）的名称已经被换成了较短的名称。你可以把这两个操作看成List在模拟数组的方括号，这样就能够看出较短的名称方便多了。查看以下的赋值语句：

```
a[i] = a[j].times(a[k]);
```

307

等价的Vector形式是：

```
v.setElementAt(v.elementAt(j).times(v.elementAt(k)), i);
```

等价的List形式是：

```
v.set(i, v.get(j).times(v.get(k)));
```

你可能已经注意到了，set方法（它取代Vector方法setElementAt）颠倒了实参的次序，以便它们与对应的数组操作匹配。考虑以下的赋值语句：

```
gift[5] = "golden rings";
```

等价的Vector形式是：

```
gift.setElementAt("golden rings", 5);
```

等价的List形式是：

```
gift.set(5, "golden rings");
```

为了保持一致性，add(int，E)方法（它替代insertElementAt(Object，int)方法）也颠倒了实参的次序。

Vector中的各种范围操作（indexOf、lastIndexOf和setSize）已经被替换为单个范围视图操作（subList），subList的功能更强大而且一致性更高。

2. 集合操作

如果你已经熟悉了Collection中的操作，就会发现从Collection继承的操作的作用和你想像得一样。如果你不熟悉Collection中的操作，那么最好读一读11.2.1节的第3小节。remove操作总是从列表中删除指定元素的第一个实例。add和addAll操作总是在列表的末尾附加新元素。因此，下列代码的作用是将两个列表联结起来：

```
list1.addAll(list2);
```

下面是上述语句的不造成破坏的形式，它建立第三个List，其中包含第二个列表附加到第一个列表后形成的列表：

308

```
List<Type> list3 = new ArrayList<Type>(list1);
list3.addAll(list2);
```

注意，非破坏性形式的语句利用了ArrayList的标准转换构造器。

与Set接口一样，List也增强了对equals和hashCode方法的要求，这样就可以比较两个List对象在逻辑上是否相等，而不管它们的实现类如何。如果两个List对象以相同的次序包含相同的元素，那么它们就是相等的。

3. 位置访问和搜索操作

基本的位置访问（positional access）操作（get、set、add和remove）的行为与Vector中长名称的等效操作(elementAt、setElementAt、insertElementAt和removeElementAt)相似，但有一个值得注意的差异：set和remove操作返回被覆盖或删除的旧值；而Vector中的

等效操作（setElementAt和removeElementAt）不返回任何值（void）。搜索（search）操作indexOf和lastIndexOf的行为与Vector中的等效操作完全一样。

addAll操作从指定的位置开始插入指定Collection中的所有元素。元素被插入的次序就是指定Collection的迭代器返回它们的次序。这个调用是Collection的addAll操作的位置访问模拟。

下面这个小方法交换List中两个带索引的值。你在上编程入门课程时应该见过这样的语法：

```
public static <E> void swap(List<E> a, int i, int j) {
  E tmp = a.get(i);
  a.set(i, a.get(j));
  a.set(j, tmp);
}
```

当然，这里有一个重要的差异。这是一个多态算法：它可以交换任何List中的两个元素，无论它们的实现类型是什么。下面是另一个使用前面的swap方法的多态算法：

```
public static void shuffle(List<?> list, Random rnd) {
  for (int i = list.size(); i > 1; i--)
    swap(list, i - 1, rnd.nextInt(i));
}
```

309

这个算法包含在Java平台的Collections[1]类中，它使用指定的随机源随机地交换指定的列表。它采用了一点小技巧：从列表的底部向上运行，反复地将随机选择的元素交换到当前位置。与大多数幼稚的混序方法不同，这种算法既公平（如果采用一个公平的随机源，那么所有排列的出现几率是相同的）又迅速（实际上只需要进行list.size()-1次交换）。下面这个小程序使用此算法以随机次序输出它的实参列表中的单词：

```
import java.util.*;

public class Shuffle {
  public static void main(String[] args) {
    List<String> list = new ArrayList<String>();
    for (String a : args)
      list.add(a);
    Collections.shuffle(list, new Random());
    System.out.println(list);
  }
}
```

实际上，我们甚至可以使这个程序更短、更快。Arrays[2]类有一个静态的工厂方法asList，它允许数组被作为List查看。这个方法并不复制数组；List中的修改直接写到数组中，反之亦然。产生的List不是一个通用的List实现，在其中没有实现（可选的）add和remove操作：数组不能改变大小。通过利用Arrays.asList和调用shuffle的库版本（它使用一个默认的随机源），可以得到下面这个短小的程序[3]，它的行为与前一个程序完全一样：

```
import java.util.*;

public class Shuffle {
  public static void main(String[] args) {
    List<String> list = Arrays.asList(args);
    Collections.shuffle(list);
    System.out.println(list);
  }
```

---

1. docs/api/java/util/Collections.html
2. docs/api/java/util/Arrays.html
3. tutorial/collections/interfaces/examples/Shuffle.java

}

310

4. 迭代器

你可以想到，List的iterator操作返回的Iterator会以正确的次序返回列表元素。List还提供了一个功能更丰富的迭代器，称为ListIterator，它允许你双向遍历列表、在迭代期间修改列表以及获取迭代器的当前位置。ListIterator接口如下所示：

```
public interface ListIterator<E> extends Iterator<E> {
  boolean hasNext();
  E next();
  boolean hasPrevious();
  E previous();
  int nextIndex();
  int previousIndex();
  void remove(); // optional
  void set(E e); // optional
  void add(E e); // optional
}
```

ListIterator从Iterator继承了三个方法（hasNext、next和remove），它们的行为在这两个接口中是完全一样的。hasPrevious和previous操作精确地模拟了hasNext和next，只是方向不同。前两个操作指向（隐含的）游标之前的元素，后两个操作指向游标之后的元素。previous操作将游标向后移动，而next操作将游标向前移动。

下面是在列表中向后迭代的标准语法：

```
for (ListIterator<Type> it = list.listIterator(list.size());
                                        it.hasPrevious(); ) {
  Type t = it.previous();
  ...
}
```

注意上面语法中listIterator的实参。List接口有两种形式的listIterator方法。没有实参的形式返回一个定位到列表开头的ListIterator；具有int实参的形式返回一个定位到指定索引位置的ListIterator。此索引指向第一次调用next将返回的元素。第一次调用previous将返回索引为index-1的元素。在一个长度为n的列表中，有n+1个合法的index值（从0到n，包含两端）。

311

游标总是位于两个元素之间：一个元素由调用previous返回，一个元素由调用next返回。n+1个合法索引值分别对应元素之间的n+1个空隙，从第一个元素前面的空隙到最后一个元素后面的空隙。图11-2显示了一个包含四个元素的列表，其中有5个可能的游标位置。

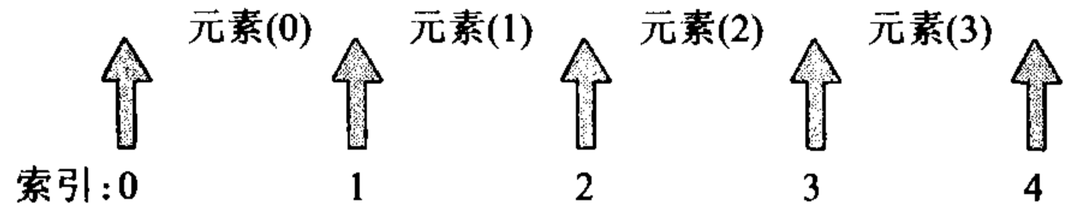


图11-2 5个可能的游标位置

对next和previous的调用可以被混合在一起，但是要小心。第一次调用previous会返回与最后一次调用next相同的元素。类似，在一系列previous调用之后，第一次调用next会返回与最后一次调用previous相同的元素。

理所当然，nextIndex方法返回下一次调用next将返回的元素的索引，previousIndex方法返回下一次调用previous将返回的元素的索引。这些调用通常用于报告找到某元素的位置，或者记录ListIterator的位置，以便可以在相同的位置创建另一个ListIterator。

nextIndex返回的数字总是比previousIndex返回的数字大1。这解释了以下两种情况下

的行为：（1）当游标位于第一个元素之前时，对previousIndex的调用返回-1；（2）当游标位于最后一个元素之后时，对nextIndex的调用返回list.size()。为了使你的认识更具体，下面给出List.indexOf的一个可能实现：

```
public int indexOf(E e) {
  for (ListIterator<E> it = listIterator(); it.hasNext(); )
    if (e == null ? it.next() == null : e.equals(it.next()))
      return it.previousIndex();
  return -1;  // Element not found
}
```

注意，indexOf方法返回it.previousIndex()，尽管它是向前遍历列表。原因是it.
312 nextIndex()返回我们将要检查的元素的索引，而我们想要的是我们刚检查完的元素的索引。

Iterator接口提供了remove操作，用于从Collection中删除next返回的最后一个元素。对于ListIterator，此操作删除next或previous返回的最后一个元素。ListIterator接口提供了另外两个用于修改列表的操作：set和add。set方法用指定的元素覆盖next或previous返回的最后一个元素。下面的多态算法使用set，以另一个值替换指定值的所有实例：

```
public static <E> void replace(List<E> list, E val, E newVal) {
  for (ListIterator<E> it = list.listIterator(); it.hasNext();)
    if (val == null ?
        it.next() == null : val.equals(it.next()))
      it.set(newVal);
}
```

这个例子中唯一有技巧性的地方是val和it.next之间的相等性测试。我们必须专门处理val值为null的情况，以防止抛出NullPointerException异常。

add方法在列表中当前游标位置之前插入一个新元素。下面的多态算法使用add方法，用指定列表中包含的一系列值替换指定值的所有实例：

```
public static <E> void replace(List<E> list, E val,
                               List<? extends E> newVals) {
  for (ListIterator<E>
       it = list.listIterator(); it.hasNext(); ){
    if (val == null ?
        it.next() == null : val.equals(it.next())) {
      it.remove();
      for (E e : newVals)
        it.add(e);
    }
  }
}
```

5. 范围视图操作

范围视图（range-view）操作subList(int fromIndex, int toIndex)返回列表一部分的List视图（view），索引范围是从fromIndex（含）到toIndex（不含）。这种半开放的范围（half-open range）与典型的for循环相同：

```
for (int i = fromIndex; i < toIndex; i++) {
  ...
}
```

313

正如视图（view）这个词所暗示的，返回的List由调用subList的List支持，所以对前者的修改会反映到后者中。

有了这个方法，就不需要进行显式的范围操作（数组常常用这些操作进行排序）。需要List的任何操作都可以作为范围操作使用，办法是传递一个subList视图而不是整个List。例如，

下面的语法从列表中删除一个范围内的元素：

```
list.subList(fromIndex, toIndex).clear();
```

可以构造类似的语法在一个范围内搜索一个元素：

```
int i = list.subList(fromIndex, toIndex).indexOf(o);
int j = list.subList(fromIndex, toIndex).lastIndexOf(o);
```

注意，前面的语法返回要寻找的元素在subList中的索引，而不是在后端List中的索引。

任何在List上操作的多态算法（比如replace和shuffle例子）都可以使用subList返回的List。

下面是一个多态算法，它的实现使用subList进行发牌。也就是说，它返回一个新的List（一把牌），其中包含从指定的List（一副牌）的末尾取得的指定数量的元素。返回的一把牌中的元素被从这副牌中删除：

```
public static <E> List<E> dealHand(List<E> deck, int n) {
  int deckSize = deck.size();
  List<E> handView = deck.subList(deckSize - n, deckSize);
  List<E> hand = new ArrayList<E>(handView);
  handView.clear();
  return hand;
}
```

注意，此算法从一副牌的末尾删除牌。对于许多常见的List实现，比如ArrayList，从列表的末尾删除元素本质上比从列表的开头删除元素性能更好。

下面的程序[1]结合使用dealHand方法和Collections.shuffle，从普通的52张的一副牌生成几把牌。这个程序采用两个命令行参数：(1)要发牌的把的数量，(2)每把牌中牌的数量。 314

```
import java.util.*;

class Deal {
  public static void main(String[] args) {
    int numHands = Integer.parseInt(args[0]);
    int cardsPerHand = Integer.parseInt(args[1]);

    // Make a normal 52-card deck.
    String[] suit = new String[]
      {"spades", "hearts", "diamonds", "clubs"};
    String[] rank = new String[]
      {"ace","2","3","4","5","6","7","8",
          "9","10","jack","queen","king"};
    List<String> deck = new ArrayList<String>();
    for (int i = 0; i < suit.length; i++)
      for (int j = 0; j < rank.length; j++)
        deck.add(rank[j] + " of " + suit[i]);

    Collections.shuffle(deck);

    for (int i=0; i < numHands; i++)
      System.out.println(dealHand(deck, cardsPerHand));
  }
}
```

运行程序生成如下输出：

```
% java Deal 4 5

[8 of hearts, jack of spades, 3 of spades, 4 of spades,
```

1. tutorial/collections/interfaces/examples/Deal.java

```
                                                    king of diamonds]
[4 of diamonds, ace of clubs, 6 of clubs, jack of hearts,
                                                  queen of hearts]
[7 of spades, 5 of spades, 2 of diamonds, queen of diamonds,
                                                    9 of clubs]
[8 of spades, 6 of diamonds, ace of spades, 3 of hearts,
                                                  ace of hearts]
```

尽管subList操作相当强大，但是在使用时必须小心。如果以其他方式（而不是通过返回的List）在后端的List中添加或删除元素，那么subList返回的List的语义就变成未定义的。所以，强烈建议你只将subList返回的List作为临时对象使用——用于在后端的List上执行一个或一系列范围操作。使用subList实例的时间越长，由于直接修改后端的List或通过另一个subList对象修改后端的List而造成损害的可能性就越大。注意，修改子列表的子列表并继续
315 使用原来的子列表（但并不是同时进行）是合法的。

6. 列表算法

Collections类中的大多数多态算法特别适用于List。掌握这些算法会使列表的处理非常容易。下面是这些算法的总结，11.4节中有更详细的说明。

- **sort**——使用一种合并的排序算法对List进行排序，它提供快速稳定的排序。（稳定排序[stable sort]不对相同的元素重新排序。）
- **shuffle**——随机交换List中的元素。
- **reverse**——颠倒List中元素的次序。
- **rotate**——按照指定的距离旋转List中的所有元素。
- **swap**——交换List中位于指定位置的元素。
- **replaceAll**——使用一个值替换另一个指定值的所有实例。
- **fill**——用指定的值覆盖List中的每个元素。
- **copy**——将源List复制进目标List。
- **binarySearch**——使用二进制搜索算法在一个有序的List中搜索元素。
- **indexOfSubList**——返回一个List中和另一个子列表相等的第一个子列表的索引。
- **lastIndexOfSubList**——返回一个List中和另一个子列表相等的最后一个子列表的索引。

11.2.4 Queue接口

Queue[1]是在处理之前保存元素的集合。除了基本的Collection操作之外，队列还提供附加的插入、删除和检查操作。Queue的接口如下：

```
public interface Queue<E> extends Collection<E> {
  E element();
  boolean offer(E e);
  E peek();
  E poll();
  E remove();
}
```
316

每个Queue方法都有两种形式：(1)一种在操作失败的情况下抛出异常，(2)另一种在操作失败的情况下返回一个特殊值（根据操作的不同，要么返回null，要么返回false）。这个接口的常规结构如表11-1所示。

---

1. docs/api/java/util/Queue.html

表 11-1 队列接口结构

	抛出异常	返回特殊值
插入	add(e)	offer(e)
删除	remove()	poll()
检查	element()	peek()

典型情况下（但并非必须），队列按照FIFO（先进先出）的形式管理元素。其中的一个例外是优先级队列，它按照元素的值对元素进行排序——更多的信息请参阅11.2.6节。不管使用哪种排序方式，调用remove或者poll都会删除队列的头元素。在FIFO队列中，所有新元素都被插入到队列的末尾位置。其他类型的队列可能使用不同的安排原则。每个Queue实现都必须指定其排序属性。

Queue实现可能限制它保存的元素数量；这样的队列被称为有界（bounded）的。java.util.concurrent中的一些Queue实现是有界的，但java.util中的实现不是。

add方法是Queue从Collection继承而来的，它插入一个元素，除非这样做违反队列的容量限制（在这种情况下，会抛出IllegalStateException异常）。offer方法只用于有界队列，它和add方法的唯一区别在于，它通过返回false指出插入元素的操作失败。

remove和poll方法删除队列的头元素并且返回这个元素。到底删除哪个元素由队列的排序策略决定。remove和poll方法的区别只出现在队列为空的时候，在这种情况下，remove抛出NoSuchElementException异常，而poll返回null。

element和peek方法返回（但是不删除）队列的头元素。二者之间的区别与remove和poll之间的区别一样。如果队列为空，element就抛出NoSuchElementException异常，而peek返回null。

317

Queue实现通常不允许插入null元素。LinkedList实现是一个例外，它被修改以便实现Queue。出于历史的原因，它允许null元素，但是你应该避免利用这一特性，因为null被用作poll和peek方法的特殊返回值。

Queue实现通常不定义基于元素的equals和hashCode方法版本，但是从Object继承了基于等同性的版本。

Queue接口没有定义阻塞队列的方法，这种方法在并发编程中是很常见的。这些方法等待元素出现或者等待空间可用，在扩展了Queue的java.util.concurrent.BlockingQueue[1]中定义了它们。

在下面的示例程序中，使用队列实现倒计时器。队列预先加载从命令行获得的一个指定值到0的所有整数值，按照降序排序。然后，按照1秒的间隔从队列删除这些值并且输出它们。这个程序是模拟的，因为不使用队列完成这一功能会更加自然，但是它能够演示在后续处理之前使用队列存储元素的特性：

```
import java.util.*;

public class Countdown {
  public static void main(String[] args)
      throws InterruptedException {
    int time = Integer.parseInt(args[0]);
    Queue<Integer> queue = new LinkedList<Integer>();
    for (int i = time; i >= 0; i--)
      queue.add(i);
```

1. docs/api/java/util/concurrent/BlockingQueue.html

```
    while (!queue.isEmpty()) {
      System.out.println(queue.remove());
      Thread.sleep(1000);
    }
  }
}
```

在下面的例子中，使用优先级队列对元素的集合进行排序。这个程序也是模拟的，因为有了Collection中提供的sort方法，就没有必要使用这种方式，但是这个程序能够演示优先级
318 队列的行为：

```
static <E> List<E> heapSort(Collection<E> c) {
  Queue<E> queue = new PriorityQueue<E>(c);
  List<E> result = new ArrayList<E>();
  while (!queue.isEmpty())
    result.add(queue.remove());
  return result;
}
```

11.2.5 Map接口

Map[1]是一个将键映射到值的对象。映射不能包含重复的键：每个键最多可以映射到一个值。映射模拟数学函数（function）抽象。Map接口如下所示：

```
public interface Map<K,V> {

  // Basic operations
  V put(K key, V value);
  V get(Object key);
  V remove(Object key);
  boolean containsKey(Object key);
  boolean containsValue(Object value);
  int size();
  boolean isEmpty();

  // Bulk operations
  void putAll(Map<? extends K, ? extends V> m);
  void clear();

  // Collection Views
  public Set<K> keySet();
  public Collection<V> values();
  public Set<Map.Entry<K,V>> entrySet();

  // Interface for entrySet elements
  public interface Entry {
    K getKey();
    V getValue();
    V setValue(V value);
  }
}
```
319

Java平台包含三个通用的Map实现：HashMap[2]、TreeMap[3]和LinkedHashMap[4]。它们的行为精确地模拟11.2.2节中描述的HashSet、TreeSet和LinkedHashSet。另外，HashMap已经

---

1. docs/api/java/util/Map.html
2. docs/api/java/util/HashMap.html
3. docs/api/java/util/TreeMap.html
4. docs/api/java/util/LinkedHashMap.html

为了实现Map进行了优化。

1. 与Hashtable比较

如果你使用过Hashtable，那么应该熟悉Map的基本特性。（当然，Map是一个接口，而Hashtable是一个具体的实现）。下面是它们的主要差异：

- Map提供Collection视图，而不是通过Enumeration对象直接支持迭代。Collection视图大大增强了此接口的表达能力，请参考本节后面的说明。
- Map允许对键、值或键—值对进行迭代；Hashtable没有提供第三个选项。
- Map提供了一种在迭代期间删除项目的安全方式；而Hashtable没有提供。

最后，Map弥补了Hashtable接口中一个次要的欠缺。Hashtable有一个称为contains的方法，如果Hashtable包含给定的值（value），那么这个方法返回true。根据contains这个名字，你可能会误以为此方法在Hashtable包含给定的键（key）时返回true，因为键是Hashtable的主要访问机制。Map接口将此方法重新命名为containsValue，这样就消除了这个误解的根源。另外，这也改进了接口的一致性——containsValue可以与containsKey共存。

2. Map接口的基本操作

Map接口的基本操作（put、get、containsKey、containsValue、size和isEmpty）的行为与Hashtable中的对应操作完全一样。下面这个程序[1]产生一个频率表，其中包含它的实参列表中单词的出现频率。频率表将每个单词映射到它在实参列表中的出现次数：

320

```
import java.util.*;

public class Freq {
  public static void main(String[] args) {
    Map<String, Integer> m = new HashMap<String, Integer>();

    // Initialize frequency table from command line
    for (String a : args) {
      Integer freq = m.get(a);
      m.put(a, (freq == null) ? 1 : freq + 1);
    }

    System.out.println(m.size() + " distinct words:");
    System.out.println(m);
  }
}
```

在这个程序中，唯一有点技巧的地方是put语句的第二个实参。这个实参是一个条件表达式，它的作用是：如果此单词以前没有出现过，那么将频率设置为1；如果此单词以前出现过，那么将频率设置为当前值加1。用以下命令行运行此程序：

```
java Freq if it is to be it is up to me to delegate
```

程序产生以下输出：

```
8 distinct words:
{to=3, delegate=1, be=1, it=2, up=1, if=1, me=1, is=2}
```

假设你想看到按字母表次序显示的频率表，需要做的仅仅是将Map的实现类型从HashMap改为TreeMap。只需修改这四个字符，就能用相同的命令行使程序产生以下输出：

```
8 distinct words:
{be=1, delegate=1, if=1, is=2, it=2, me=1, to=3, up=1}
```

---

1. tutorial/collections/interfaces/examples/Freq.java

同样，把映射的实现类型改为LinkedHashMap，就可以简单地使程序按照单词在命令行中第一次出现的顺序输出频率表。这样改动后的输出如下：

```
8 distinct words:
{if=1, it=2, is=2, to=3, be=1, up=1, me=1, delegate=1}
```

321

这种灵活性有力地表现出基于接口的框架的强大功能。

与Set和List接口一样，Map加强了对equals和hashCode方法的要求，从而可以比较两个Map对象在逻辑上是否相等，而不管它们的实现类型如何。如果两个Map实例代表相同的键一值映射对，那么它们就是相等的。

根据约定，所有通用的Map实现都提供了一种构造器，此构造器接受一个Map对象，并将新的Map初始化为包含指定Map中的所有键一值映射对。这个标准的Map转换构造器与标准Collection构造器类似：它允许调用者创建一个指定实现类型的Map，初始时包含另一个Map中的所有映射对（无论另一个Map的实现类型是什么）。例如，假设你有一个名为m的Map。下面这行代码创建一个新的HashMap，它最初包含与m相同的所有键一值映射对：

```
Map<K, V> copy = new HashMap<K, V>(m);
```

3. Map接口的批量操作

clear操作的作用与你的设想完全一样：它从Map中删除所有映射对。putAll操作与Collection接口的addAll操作对应，除了将一个Map转储到另一个Map中外，它还有另一个更微妙的用途。假设一个Map用于代表一个属性一值对的集合；putAll操作（与Map的转换构造器结合使用）提供一种创建具有默认值的属性映射的简单方式。下面这个静态工厂方法演示了此技术：

```
static <K, V> Map<K, V> newAttributeMap(
    Map<K, V>defaults, Map<K, V> overrides) {
  Map<K, V> result = new HashMap<K, V>(defaults);
  result.putAll(overrides);
  return result;
}
```

4. 集合视图

Collection视图方法允许以如下三种方式将Map看作Collection：

- **keySet**——Map中包含的键的Set。
- **values**——Map中包含的值的Collection。这个Collection不是Set，因为多个键可能映射到相同的值。
- **entrySet**——Map中包含的键一值对的Set。Map接口提供了一个小的嵌套接口Map.Entry，这个Set中的元素就是这种类型的。

322

Collection视图提供了迭代Map的唯一办法。下面这个例子演示迭代Map中的键的标准语法，使用的是for-each结构：

```
for (KeyType key : m.keySet())
  System.out.println(key);
```

下面是使用迭代器的语法：

```
// Filter a map based on some property of its keys.
for (Iterator<Type> it = m.keySet().iterator(); it.hasNext(); )
  if (it.next().isBogus())
    it.remove();
```

对值进行迭代的语法与之类似。下面是迭代键一值对的语法：

```
for (Map.Entry<KeyType, ValType> e : m.entrySet())
  System.out.println(e.getKey() + ": " + e.getValue());
```

最初，许多人担心这些语法可能很慢，因为每次调用Collection视图操作时Map必须创建一个新的Collection实例。请放心：每次向Map请求给定的Collection视图时，它可以总是返回相同的对象。java.util中的Map实现就是这么做的。

通过使用这三个Collection视图，调用Iterator的remove操作会从后端的Map中删除相关联的项目（假设后端的Map支持从指定位置删除元素）。前面的过滤语法演示了这种情况。

通过使用entrySet视图，还可以在迭代期间通过调用Map.Entry的setValue方法修改与一个键相关联的值（仍然假设后端的Map支持在指定位置修改值）。注意，这些是在迭代期间修改Map的唯一的安全方式；如果在迭代执行期间以其他任何方式修改底层的Map，那么产生的行为是不确定的。

Collection视图支持各种形式的元素删除操作：remove、removeAll、retainAll和clear操作，以及Iterator.remove操作。（仍然假设后端的Map支持元素删除操作。）

在任何环境下，Collection视图都不支持元素添加操作。元素添加操作对于keySet和values视图是没有意义的，而对于entrySet视图是不必要的，因为后端的Map的put和putAll方法提供相同的功能。 323

5. Collection视图的巧妙使用：Map代数运算

批量操作（containsAll、removeAll和retainAll）在应用于Collection视图时是非常有力的工具。对于初学者，假设你想知道一个Map是否是另一个Map的子映射，也就是说，一个Map是否包含另一个Map中的所有键一值映射对，可以使用下面的语法：

```
if (m1.entrySet().containsAll(m2.entrySet())) {
  ...
}
```

类似的，假设你想知道两个Map对象是否包含所有相同键的映射对，那么使用以下语法：

```
if (m1.keySet().equals(m2.keySet())) {
  ...
}
```

假设你有一个代表属性一值对集合的Map，以及两个代表必需的属性和允许的属性的Set。（允许的属性包括必需的属性。）下面的代码段判断属性映射是否符合这些约束（即其中的属性是否都是允许的属性，以及必需的属性是否全有了），如果不符合这些约束，那么输出详细的错误消息：

```
static <K, V> boolean validate(Map<K, V> attrMap,
          Set<K> requiredAttrs, Set<K>permittedAttrs) {
  boolean valid = true;
  Set<K> attrs = attrMap.keySet();
  if (!attrs.containsAll(requiredAttrs)) {
    Set<K> missing = new HashSet<K>(requiredAttrs);
    missing.removeAll(attrs);
    System.out.println("Missing attributes: " + missing);
    valid = false;
  }
  if (!permittedAttrs.containsAll(attrs)) {
    Set<K> illegal = new HashSet<K>(attrs);
    illegal.removeAll(permittedAttrs);
    System.out.println("Illegal attributes: " + illegal);
    valid = false;
  }
  return valid;
}
```

假设你想知道两个Map对象中都有的键，那么使用以下语法： 324

```
Set<KeyType>commonKeys = new HashSet<KeyType>(m1.keySet());
commonKeys.retainAll(m2.keySet());
```

类似的语法可以获取共有的值。

到目前为止给出的所有语法都是非破坏性的；也就是说，它们不修改后端的Map。我们现在就来修改后端的Map。假设你想从一个Map中删除另一个Map中也有的所有键—值对：

```
m1.entrySet().removeAll(m2.entrySet());
```

假设你想从一个Map中删除在另一个Map中有映射对的所有键：

```
m1.keySet().removeAll(m2.keySet());
```

当你在同一个批量操作中混合使用键和值时，会发生什么？假设你有一个Map——managers，它将一个公司中的每个雇员映射到此雇员的经理。我们故意不指明键和值对象的类型。这没问题，只要它们是相同的。现在，假设你想知道“独立工作者”（或者不是经理的人）有哪些。下面的代码段可以告诉你答案：

```
Set<Employee> individualContributors =
  new HashSet<Employee>(managers.keySet());
individualContributors.removeAll(managers.values());
```

假设你想解雇直接隶属于经理Simon的所有雇员，那么使用下面的代码：

```
Employee simon = ... ;
managers.values().removeAll(Collections.singleton(simon));
```

注意，此语法使用了Collections.singleton，这是一个静态工厂方法，它返回一个具有单个指定元素的不可变的Set。

完成这些工作之后，你可能发现一些雇员的经理已经不再为本公司工作了（如果Simon的任何下属都自己管理自己）。下面的代码告诉你哪些雇员的经理已经不再为本公司工作了：

325

```
Map<Employee, Employee> m =
  new HashMap<Employee, Employee>(managers);
m.values().removeAll(managers.keySet());
Set<Employee> slackers = m.keySet();
```

这个例子中有点儿技巧。首先，它建立一个Map的临时副本，然后从这个临时副本中删除其映射值（经理）是原Map中的键的所有项目。记住，原来的Map对于每个雇员都有一个项目。因此，临时的Map中余下的项目就是原始Map中那些其映射值（经理）不再是雇员的项目。现在，临时副本中的键正好代表我们要寻找的那些雇员。

像本节中这样的代码还有许许多多，把它们全列出来是不现实的，而且很冗长乏味。一旦你掌握了诀窍，那么在需要时提供合适的代码并不难。

6. 多映射

多映射（multimap）与Map类似，但它可以将每个键映射到多个值。Java集合框架不包含用于多映射的接口，因为它们不常用。简单的解决方案是，将一个其值为List实例的Map作为多映射使用。下一个例子演示此技术，这个例子读一个每行包含一个单词（全部小写）的单词列表，并且输出满足某个尺寸准则的所有变位词组。变位词组（anagram group）是一组单词，这些单词都包含完全相同的字母，但是次序不同。此程序从命令行取得两个实参：（1）字典文件的名字；（2）要输出的变位词组的最小尺寸。包含的单词数少于指定的最小值的变位词组不被输出。

寻找变位词组的标准技巧是：对于字典中的每个单词，依字母表顺序排列单词中的每个字母（也就是说，将单词的字母重新排序为字母表顺序），并在一个多映射中放一个项目，此项目将依字母表顺序排列后的单词映射到原来的单词。例如，单词“bad”导致在多映射中放一个

项目，此项目将“abd”映射到“bad”。很明显，映射到一个给定键的所有单词组成了一个变位词组。接下来的工作就简单了：迭代多映射中的键，输出符合尺寸要求的每个变位词组。

下面的程序[1]是此技术的一种很直观的实现。 326

```
import java.util.*;
import java.io.*;

public class Anagrams {

  public static void main(String[] args) {
    int minGroupSize = Integer.parseInt(args[1]);

// Read words from file and put into a simulated multimap
Map<String, List<String>> m
  = new HashMap<String, List<String>>();
try {
  Scanner s = new Scanner(new File(args[0]));
  while (s.hasNext()) {
    String word = s.next();
    String alpha = alphabetize(word);
    List<String> l = m.get(alpha);
    if (l == null)
      m.put(alpha, l=new ArrayList<String>());
    l.add(word);
  }
} catch (IOException e) {
    System.err.println(e);
    System.exit(1);
}

    // Print all permutation groups above size threshold
    for (List<String> l : m.values())
      if (l.size() >= minGroupSize)
        System.out.println(l.size() + ": " + l);
  }

  private static String alphabetize(String s) {
    char[] a = s.toCharArray();
    Arrays.sort(a);
    return new String(a);
  }
}
```

对一个包含173 000个单词的字典文件运行此程序，设置变位词组的最小尺寸为8，会产生以下输出： 327

```
9: [estrin, inerts, insert, inters, niters, nitres, sinter,
                                                triens, trines]
8: [lapse, leaps, pales, peals, pleas, salep, sepal, spale]
8: [aspers, parses, passer, prases, repass, spares, sparse,
                                                    spears]
10: [least, setal, slate, stale, steal, stela, taels, tales,
                                                  teals, tesla]
8: [enters, nester, renest, rentes, resent, tenser, ternes,
                                                    treens]
8: [arles, earls, lares, laser, lears, rales, reals, seral]
8: [earings, erasing, gainers, reagins, regains, reginas,
                                                  searing, seringa]
```

1. tutorial/collections/interfaces/examples/Anagrams.java

```
8: [peris, piers, pries, prise, ripes, speir, spier, spire]
12: [apers, apres, asper, pares, parse, pears, prase, presa,
                                        rapes, reaps, spare, spear]
11: [alerts, alters, artels, estral, laster, ratels, salter,
                                      slater, staler, stelar, talers]
9: [capers, crapes, escarp, pacers, parsec, recaps, scrape,
                                                  secpar, spacer]
9: [palest, palets, pastel, petals, plates, pleats, septal,
                                                  staple, tepals]
9: [anestri, antsier, nastier, ratines, retains, retinas,
                                        retsina, stainer, stearin]
8: [ates, east, eats, etas, sate, seat, seta, teas]
8: [carets, cartes, caster, caters, crates, reacts, recast,
                                                      traces]
```

其中的许多单词有点像是伪造的，但这不是程序的错误；这些单词就在我们使用的字典文件[1]中。这个字典来源于公共域启用基准参考（Public Domain ENABLE benchmark reference）的单词列表[2]。

11.2.6　对象排序

可以像下面这样对List l进行排序：

```
Collections.sort (l);
```

如果此List由String元素组成，那么它将按字母表次序排序。如果它由Date元素组成，那么它将按时间顺序排序。这是如何完成的呢？String和Date都实现了Comparable[3]接口，它为类提供自然顺序（natural ordering），允许对此类的对象进行自动排序。表11-2总结了实现Comparable接口的一些Java平台类。

328

表 11-2　实现 Comparable 接口的类

类	自然顺序
Byte	有符号的数字顺序
Character	无符号的数字顺序
Long	有符号的数字顺序
Integer	有符号的数字顺序
Short	有符号的数字顺序
Double	有符号的数字顺序
Float	有符号的数字顺序
BigInteger	有符号的数字顺序
BigDecimal	有符号的数字顺序
Boolean	Boolean.FALSE < Boolean.TRUE
File	路径名上与系统相关的字典顺序
String	字典顺序
Date	时间顺序
CollationKey	地区特定的字典顺序

如果你要排序的列表的元素没有实现Comparable接口，那么Collections.sort(list)

1. tutorial/collections/interfaces/examples/dictionary.txt
2. http://personal.riverusers.com/~thegrendel/
3. docs/api/java/lang/Comparable.html

将抛出一个ClassCastException异常[1]。类似的，如果你要排序的列表的元素无法使用comparator相互进行比较，那么Collections.sort(list, comparator)将抛出一个ClassCastException异常。可以被相互比较的元素被称为可相互比较的（mutually comparable）。尽管不同类型的元素可能是可相互比较的，但是这里列出的类都不允许类间比较。

如果你只想对可比较元素的列表进行排序，或者创建它们的有序集合，那么关于Comparable接口只需要知道这么多。如果你想实现自己的Comparable类型，那么会对下一节感兴趣。

329

1. 编写自己的Comparable类型

Comparable接口由如下方法组成：

```
public interface Comparable<T> {
  public int compareTo(T o);
}
```

compareTo方法比较接收的对象和指定的对象，根据接收的对象小于、等于或大于指定的对象，而返回一个负整数、零或正整数。如果指定的对象无法与接收的对象进行比较，那么此方法抛出一个ClassCastException异常。

下面这个代表人名的类[2]实现了Comparable接口：

```
import java.util.*;

public class Name implements Comparable<Name> {
  private final String firstName, lastName;

  public Name(String firstName, String lastName) {
    if (firstName == null || lastName == null)
      throw new NullPointerException();
    this.firstName = firstName;
    this.lastName = lastName;
  }

  public String firstName() { return firstName; }
  public String lastName()  { return lastName;  }

  public boolean equals(Object o) {
    if (!(o instanceof Name))
      return false;
    Name n = (Name)o;
    return n.firstName.equals(firstName) &&
                              n.lastName.equals(lastName);
  }
  public int hashCode() {
    return 31*firstName.hashCode() + lastName.hashCode();
  }
  public String toString() {
    return firstName + " " + lastName;
  }
  public int compareTo(Name n) {
    int lastCmp = lastName.compareTo(n.lastName);
    return (lastCmp != 0 ? lastCmp :
            firstName.compareTo(n.firstName));
  }
}
```

330

1. docs/api/java/lang/ClassCastException.html
2. tutorial/collections/interfaces/examples/Name.java

为了使这个例子简短一些，这个类受到了限制：它不支持中间名，它要求人名由姓和名组成，它没有以任何方式进行国际化。尽管如此，它仍然演示了以下几个重点：

- Name对象是不可变的（immutable）。对于所有被比较的内容，其不可变的类型是必须的，特别是那些将被用作Set中的元素或Map中的键的对象。如果你修改集合中的元素或键，那么这些集合将被破坏。
- 构造器检查它的实参是否是null。这可以确保所有Name对象的构成是良好的，从而使其他方法不会抛出NullPointerException异常。
- hashCode方法被重新定义了。这对于重新定义了equals方法的任何类都是必要的。（相等的对象必须有相等的散列码。）
- 如果指定的对象是null或者是不合适的类型，那么equals方法返回false。在此情况下，compareTo方法抛出一个运行时异常。这两种行为是这两个方法的常规约定所要求的。
- toString方法已经被重新定义，以便以人可读的形式输出人名。这一直是个好主意，特别是对于将被放入集合的对象。各种集合类型的toString方法依赖于它们的元素、键和值的toString方法。

既然本节的主题是安排元素的顺序，我们就来稍微多讨论一下Name的compareTo方法。它实现了标准的人名排序算法，先按姓排序，然后按名排序。这就是我们想要的自然顺序。如果自然顺序不自然，就会引起很大程度的混乱！

看看compareTo方法是如何实现的，它相当典型。首先，比较对象的最重要部分（在本例中是姓）。通常只需使用这部分的类型的自然顺序。在本例中，这部分是个String，其自然顺序是字典顺序。如果比较的结果不是零（零表示相等），那么就返回结果。如果最重要的部分相等，那么继续比较次重要的部分。在本例中，只有两个部分：姓和名。如果还有更多的部分，只需按这种方式扩展，依次比较各个部分直到找到不相等的两个部分或者比较到最次要的部分，然后返回比较结果。

331

现在看看具体的代码，下面这个程序建立一个姓名列表并对它进行排序[1]：

```
import java.util.*;

public class NameSort {
  public static void main(String[] args) {
    Name nameArray[] = {
      new Name("John", "Lennon"),
      new Name("Karl", "Marx"),
      new Name("Groucho", "Marx"),
      new Name("Oscar", "Grouch")
    };
    List<Name> names = Arrays.asList(nameArray);
    Collections.sort(names);
    System.out.println(names);
  }
}
```

此程序的输出如下：

```
[Oscar Grouch, John Lennon, Groucho Marx, Karl Marx]
```

对于compareTo方法的行为有四个限制，在这里不深入讨论它们，因为它们的技术性太强，请通过API文档研究它们。重要的是，实现Comparable的所有类都要服从这些限制，所以，如果你要编写实现Comparable的类，就应该阅读Comparable的文档。如果试图对违反这些限制

1. tutorial/collections/interfaces/examples/NameSort.java

的对象的列表进行排序，那么产生的行为是不确定的。从技术上讲，这些限制确保自然顺序是实现Comparable的类的对象上的完整顺序（total order）；这对于确保排序以合适的方式进行是必要的。

2. 比较器

如果你想按非自然顺序对某些对象进行排序，那么应该怎么办？或者，如果想对没有实现Comparable的对象进行排序，那么应该怎么办？为应付这两种情况，需要提供一个Comparator[1]——一个封装了顺序的对象。与Comparable接口类似，Comparator接口由一个方法组成：

```
public interface Comparator<T> {
  int compare(T o1, T o2);
}
```

compare方法比较它的两个实参并且返回一个负整数、零或正整数，分别表示第一个实参小于、等于或大于第二个实参。如果这些实参的类型对于Comparator不合适，那么compare方法抛出一个ClassCastException异常。 332

关于Comparable的许多规则也适用于Comparator。编写compare方法几乎与编写compareTo方法一样，区别只是前者通过实参得到两个对象。compare方法也必须服从对Comparable的compareTo方法施加的四个技术限制，原因是一样的：Comparator必须提供它要比较的对象的完整顺序。

假设你有一个称为Employee的类，如下：

```
public class Employee implements Comparable<Employee> {
  public Name name()       { ... }
  public int number()      { ... }
  public Date hireDate()   { ... }
  ...
}
```

我们假设Employee实例的自然顺序是雇员姓名上的Name顺序（与前一个例子定义的一样）。糟糕的是，老板要求我们按资历对雇员列表进行排序。这意味着，我们必须多做些工作，但并不太多。下面这个程序将产生所需的列表：

```
import java.util.*;
public class EmpSort {
  static final Comparator<Employee> SENIORITY_ORDER =
      new Comparator<Employee>() {
    public int compare(Employee e1, Employee e2) {
      return e2.hireDate().compareTo(e1.hireDate());
    }
  };

  // Employee database
  static final Collection<Employee> employees = ... ;

  public static void main(String[] args) {
    List<Employee>e = new ArrayList<Employee>(employees);
    Collections.sort(e, SENIORITY_ORDER);
    System.out.println(e);
  }
}
```

此程序中的Comparator相当简单。它依赖应用于hireDate访问器方法返回值的Date自然

1. docs/api/java/util/Comparator.html

顺序。注意，Comparator将它的第二个实参的雇佣日期传递给它的第一个实参，而不是相反。
333 原因是，最近雇佣的雇员是资历最短的。按雇佣日期排序将使列表按资历的逆序排序。

有时候使用另一种技术可以达到相同效果：保持实参次序，但是对比较的结果求反：

```
// Don't do this!!
return -r1.hireDate().compareTo(r2.hireDate());
```

你应该一直使用前一种技术，避免使用后一种，因为后者不能保证正常工作。原因在于，如果compareTo的实参小于调用它的对象，就会返回任何负的int值。下面是一个负的int值，对其求反后仍然是负值，看上去可能很奇怪：

```
-Integer.MIN_VALUE == Integer.MIN_VALUE
```

前一个程序中的Comparator可以对List排序，但它有一个欠缺：它不能用于对有序集合（比如TreeSet）进行排序，因为它产生一种与equals不相容的顺序。这意味着，这个Comparator会把equals方法认为不相等的对象当成相等的。具体说就是，在同一天被雇佣的两个雇员将被看成相等的。在对List进行排序时，这不是问题；但是在使用Comparator对有序集合进行排序时，它会失败。如果使用这个Comparator在TreeSet中插入在同一天被雇佣的多个雇员，那么只有第一个雇员被添加到集中，其他雇员会被当作重复的元素而被忽略。

为修正此问题，只需调整这个Comparator，使它产生与equals相容的顺序。换句话说，在使用compare时被认为相等的元素必须是那些在使用equals进行比较时也被认为相等的元素。实现方式是执行两部分比较（就像对Name所做的），其中第一部分是我们感兴趣的部分（在本例中，是雇佣日期），而第二个部分是一个唯一标识对象的属性。在这种情况下，雇员编号是合适的属性。修改后的Comparator如下所示：

```
static final Comparator<Employee> SENIORITY_ORDER =
                                    new Comparator<Employee>() {
  public int compare(Employee e1, Employee e2) {
    int dateCmp = e2.hireDate().compareTo(e1.hireDate());
    if (dateCmp != 0)
      return dateCmp;
    return (e1.number() < e2.number() ? -1 :
      (e1.number() == e2.number() ? 0 : 1));
  }
};
```

要注意的最后一点是：你可能打算将Comparator中最后的return语句换成以下较简单的形式：

334
```
return e1.number() - e2.number();
```

不要这么做，除非你绝对肯定没有人具有负的雇员编号！这个办法一般不合适，因为有符号的整数类型不够大，不足以代表任意两个有符号整数的差。如果i是一个大的正整数，而j是一个大的负整数，那么i - j将会溢出并且返回一个负整数。产生的Comparator违反了我们说过的四个技术限制之一（传递性），并且会产生极麻烦又微妙的错误。这不是个纯理论的问题，确实有人被这种问题弄得焦头烂额。

11.2.7 SortedSet接口

SortedSet[1]是一种按升序维护其元素的Set，排序是根据元素的自然顺序进行，或根据在创建SortedSet时提供的Comparator进行。除了普通的Set操作，SortedSet接口还提供了

1. docs/api/java/util/SortedSet.html

以下操作：

- **范围视图**——在有序的集上执行任意范围操作。
- **端点**——返回有序集中的第一个或最后一个元素。
- **Comparator访问**——返回用于对集进行排序的Comparator（如果有的话）。

SortedSet接口如下：

```
public interface SortedSet<E> extends Set<E> {
  // Range-view
  SortedSet<E> subSet(E fromElement, E toElement);
  SortedSet<E> headSet(E toElement);
  SortedSet<E> tailSet(E fromElement);

  // Endpoints
  E first();
  E last();

  // Comparator access
  Comparator<? super E> comparator();
}
```

1. 集操作

SortedSet从Set继承的操作的行为在有序集和一般集上是一样的，只有两处例外：

- iterator操作返回的Iterator按顺序遍历有序集。
- toArray返回的数组按顺序包含有序集的元素。

335

Java平台的SortedSet实现的toString方法返回一个字符串，其中按顺序包含有序集的所有元素，但是接口并不保证这一点。

2. 标准构造器

根据约定，所有通用的Collection实现都要提供一个以Collection为实参的转换构造器，SortedSet实现也不例外。在TreeSet中，这个构造器创建一个按照元素的自然顺序安排元素顺序的实例。这可能是一个错误。更好的办法是，动态地检查指定的集合是否是SortedSet实例，如果是，就按照相同的标准（比较器或者自然顺序）对新的TreeSet进行排序。因为TreeSet使用这种方式，所以它还提供以SortedSet为实参的构造器，返回包含相同元素的新TreeSet，其中按照相同标准对元素进行排序。注意，根据实参的编译时类型（而不是运行时类型）决定调用两种构造器中的哪一种（以及是否保持排序标准）。

根据约定，SortedSet实现也提供以Comparator为实参的构造器，它返回按照指定Comparator排序的空集。如果把null传递给这个构造器，它就返回按照其元素的自然顺序对元素进行排序的集。

3. 范围视图

范围视图（range-view）操作类似于List接口提供的那些范围视图操作，但有一个重大差异。即使直接修改后端的有序集，有序集的范围视图仍然是合法的。这是因为，有序集的范围视图的端点是元素空间中的绝对点，而不是后端集合中的特定元素（这是列表中的情况）。有序集的范围视图实际上只是个“窗口”，集内属于这个窗口的部分就位于元素空间的指定部分。对范围视图的修改会被写回后端的有序集，反之亦然。所以，与列表上的范围视图不同，可以长时间使用有序集上的范围视图。

有序集提供三种范围视图操作。第一种是subSet，它与subList一样以两个端点为实参。端点不是索引，而是对象，而且它必须可以与有序集中的元素进行比较，比较方式是使用这个Set的Comparator或其元素的自然顺序（采用Set本身所用的比较方式）。与subList相同，这

个范围是半开放的，范围包括它的低端点，但不包括它的高端点。

因此，下面的代码行计算在称为dictionary的字符串SortedSet中，在“doorbell”和
336 “pickle”之间（包括“doorbell”，但不包括“pickle”）有多少个单词：

```
int count = dictionary.subSet("doorbell", "pickle").size();
```

通过类似的方式，下面的代码行删除所有以字母“f”开头的元素：

```
dictionary.subSet("f", "g").clear();
```

类似的技巧可以用来输出一个表，告诉你以每个字母开头的单词各有多少个：

```
for (char ch = 'a'; ch <= 'z'; ) {
  String from = String.valueOf(ch++);
  String to = String.valueOf(ch);
  System.out.println(from + ": " +
                     dictionary.subSet(from, to).size());
}
```

假设你想要查看闭区间（closed interval）而不是开区间，也就是说，它包括两个端点。如果元素类型允许计算元素空间中的下一个给定值，那么只需使用从lowEndpoint到successor(highEndpoint)的subSet。你可能不知道，在String的自然顺序中字符串“s”的下一个字符串是s+“\0”（即s附加一个null字符）。

因此，下面的代码行计算字典中在“doorbell”和“pickle”之间（包括“doorbell”和“pickle”）有多少个单词：

```
count = dictionary.subSet("doorbell", "pickle\0").size();
```

类似的技术可以用于查看开区间（open interval），也就是说不包括端点。从lowEndpoint到highEndpoint的开区间等于从successor(lowEndpoint)到highEndpoint的半开区间。以下代码计算“doorbell”和“pickle”之间（不包括“doorbell”和“pickle”）的单词数：

```
count = dictionary.subSet("doorbell\0", "pickle").size();
```

SortedSet接口还包含另外两个范围视图操作，headSet和tailSet。它们都以一个Object作为实参。前者返回从后端SortedSet的起点到（但不包括）指定对象的视图。后者返回后端SortedSet的结尾部分的视图，从指定对象到后端SortedSet的结尾。因此，下面的代码允许你以两个分离的卷（a～m和n～z）的形式查看字典：

```
SortedSet<String> volume1 = dictionary.headSet("n");
SortedSet<String> volume2 = dictionary.tailSet("n");
```
337

4. 端点操作

SortedSet接口包含返回有序集中第一个和最后一个元素的操作，很显然，这两个操作被称为first和last。除了明显的用途，last还解决了SortedSet接口的一点欠缺。你可能想使用SortedSet在Set中向前或者向后迭代。向前移动很容易：只需得到tailSet并迭代。不幸的是，没有向后移动的简单方式。

下面的语句获取元素空间中第一个小于指定对象o的元素：

```
Object predecessor = ss.headSet(o).last();
```

这是在有序集内从某一点开始向后寻找元素的好方式。可以重复应用此方式进行向后迭代，但是这么做的效率非常低，需要查询返回的每个元素。

5. 比较器访问器

SortedSet接口包含一个称为comparator的访问器方法，它返回用于对集进行排序的Comparator；如果集按照元素的自然顺序排序，那么它返回null。有了这个方法，有序集就可以以相同的顺序被复制到新的有序集中。前面第2小节中的SortedSet构造器使用了这个方法。

11.2.8 SortedMap接口

SortedMap[1]是一种按升序维护其项目的Map，排序是根据键的自然顺序，或根据在创建SortedMap时提供的Comparator进行的。11.2.6节中讨论了自然顺序和Comparator。除了普通的Map操作，SortedMap接口还提供了以下操作：

- **范围视图**——在有序的映射上执行任意范围操作。
- **端点**——返回有序映射中的第一个或最后一个键。
- **Comparator访问**——返回用于对映射进行排序的Comparator（如果有的话）。

338

下面的接口是SortedSet的Map形式。

```
public interface SortedMap<K, V> extends Map<K, V>{
  Comparator<? super K> comparator();
  SortedMap<K, V> subMap(K fromKey, K toKey);
  SortedMap<K, V> headMap(K toKey);
  SortedMap<K, V> tailMap(K fromKey);
  K firstKey();
  K lastKey();
}
```

1. 映射操作

SortedMap从Map继承的操作的行为在有序映射和一般映射上是一样的，但有两个差异：

- 在任何有序映射的Collection视图上，iterator操作返回的Iterator按顺序遍历集合。
- Collection视图的toArray操作返回的数组按顺序包含键、值或项目。

在所有Java平台的SortedMap实现中，Collection视图的toString方法返回一个字符串，其中按顺序包含视图中的所有元素，但是接口并不保证这一点。

2. 标准构造器

根据约定，所有通用Map实现都要提供一个以Map为实参的标准构造器，SortedMap实现也不例外。在TreeMap中，这个构造器创建一个按照键的自然顺序对其项目进行排序的实例。这可能是一个错误。更好的办法是，动态地检查指定的Map实例是否是SortedMap；如果是，就按照相同的标准（比较器或者自然顺序）对新的映射进行排序。因为TreeMap使用这种方式，所以它也提供以SortedMap为实参的构造器，返回新的TreeMap，包含和给定的SortedMap相同的映射，并且按照相同标准进行排序。注意，根据实参的编译时类型（而不是运行时类型）决定SortedMap构造器是否先于普通的Map构造器被调用。

根据约定，SortedMap实现还提供以Comparator为实参的构造器，它返回按照指定的Comparator排序的空映射。如果把null传递给这个构造器，它就返回按照其元素的键的自然顺序排序的Map。

339

3. 与SortedSet比较

因为这个接口是SortedSet的Map形式，所以11.2.7节中的所有语法和代码示例只需稍加修改就可以应用于SortedMap。

11.2.9 接口小结

核心集合接口是Java集合框架的基础。

Java集合框架的层次结构由两个独立的接口树构成：

- 第一个树从Collection接口开始，它提供所有集合使用的基本功能，比如add和remove方法。其子接口——Set、List和Queue——用于更加专门化的集合。

---

1. docs/api/java/util/SortedMap.html

Set接口不允许重复的元素。这种特性在对一副纸牌或者学生记录这样的集合进行排序时非常有用。Set接口有一个子接口SortedSet，它提供对集中的元素进行排序的功能。

List接口提供有序集合，用于需要精确控制把每个元素插入到哪个位置的情况。你可以通过元素的确切位置从List中获得元素。

Queue接口提供额外的插入、提取和检查操作。典型情况下，Queue中的元素根据FIFO原则进行排序。

- 第二个树从Map接口开始，与Hashtable类似，它映射键和值。

Map的子接口SortedMap按照升序或者Comparator指定的顺序维护其键—值对。

这些接口允许操作集合而不涉及集合所代表的细节。

问题和练习：接口

问题

340

1. 这一节讲到了三种遍历List的方式。描述它们并且说明每种方式的局限。
2. 考虑四个核心接口Set、List、Queue和Map。对于下面的四个赋值，指出哪种核心接口最适合哪个赋值，并且解释为什么使用它实现赋值：
   - Whimsical玩具公司（Whimsical Toys Inc，WTI）需要记录公司所有员工的名字。从这些员工记录中，每个月随机选择一个员工免费获得一个玩具。
   - WTI决定以员工的名字命名每个新产品——但是只使用姓名中的名，并且每个名字只使用一次。要求你准备一个唯一的名列表。
   - WTI决定只使用最常见的名字命名他们的玩具。计算使用每个名字的员工的数量。
   - WTI得到了本地曲棍球队的季票，要发给员工。为这个活动创建一个等待列表。
3. 下面的程序应该输出字符串“Blue”。但是，它抛出了一个错误，为什么？

```
import java.util.*;

public class SortMe {
  public static void main(String args[]) {
    SortedSet<StringBuffer> s = new TreeSet<StringBuffer>();
    s.add(new StringBuffer("Red"));
    s.add(new StringBuffer("White"));
    s.add(new StringBuffer("Blue"));
    System.out.println(s.first());
  }
}
```

练习

1. 编写一个程序，按照随机顺序输出其实参。不要生成实参数组的副本。
2. 修改示例程序FindDups，让它使用SortedSet，而不是使用Set。指定一个Comparator，在排序和识别集的元素时忽略大小写。
3. 编写一个方法，以List<String>为实参，并且对每个元素使用String.trim[1]。为了达到这个目的，你需要从“问题1”中描述的三种迭代语法中选择一种。其中两种不能得出你希望的结果，所以一定要编写程序以表明这个方法确实有效！

341

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/collections/interfaces/QandE/answers.html

---

1. docs/api/java/lang/String.html

11.3 实现

实现是用于存储集合的数据对象，它们实现了11.2节所述的接口。本节讨论下面几种实现：

- **通用实现**——最常用的实现，为日常使用而设计。关于它的总结见表11-3。
- **专用实现**——被设计为用于特殊情况和提供非标准的性能特征、使用限制或者行为。
- **并发实现**——被设计为支持高并发性，通常用在单一线程性能不足的情况下。这些实现是java.util.concurrent包的一部分。
- **包装器实现**——和其他类型的实现组合使用，经常和通用实现组合在一起，以便提供附加的或者受限的功能。
- **简便实现**——迷你型实现，典型情况下通过静态工厂方法使用，它为特殊集合（比如单元素集）提供简便有效的通用实现替代方案。
- **抽象实现**——帮助构造定制实现的框架实现，在后面的11.5节中介绍。这是一个高级主题，并不是非常难，但是需要它的人相当少。

表 11-3 通用实现

接口	实现				
	散列表	可变尺寸的数组	树	链表	散列表和链表
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Map	HashMap		TreeMap		LinkedHashMap

342

从表11-3可以看出，Java集合框架为Set、List和Map接口提供了几种通用实现。每种情况都有一种实现——HashSet、ArrayList和HashMap——它们明显是大多数应用程序所使用的，其他内容也是这样的。注意，表中没有SortedSet和SortedMap这两行。这两种接口各有一种实现（TreeSet和TreeMap），列在了Set和Map行里。有两种通用的Queue实现——LinkedList也是List实现，而PriorityQueue[1]在表中省略了。这两种实现的语义差别很大：LinkedList提供FIFO（先进先出）语义，而PriorityQueue按照其元素的值对元素进行排序。

每种通用实现都提供其接口中包含的所有可选操作。它们都允许null元素、键和值。它们都没有被同步（线程安全）。它们都具有快速失败的迭代器（fail-fast iterator），这种迭代器检测迭代过程中的非法并发修改，并且快速而干净地失败，以免在将来不确定的时间发生任意的不确定行为。它们都实现Serializable，都支持公共的clone方法。

这些实现未被同步，这种情况与过去的情况不同：原先的集合Vector和Hashtable是被同步的。采用现在的处理方式是因为，集合常在不必进行同步的情况下使用。这样的情况包括单线程情况、只读情况以及作为一个自身进行同步的更大型数据对象的一部分。一般来说，让用户为他们不使用的特性付出代价不是良好的API设计方法。而且，不必要的同步在某些情况下可能造成死锁。

如果你需要线程安全的集合，那么11.3.5节描述的同步包装器允许将任何集合转换为同步集合。因此，同步特性对于通用的集合实现是可选的，而对于旧的实现是强制的。另外，

---

1. docs/api/java/util/PriorityQueue.html

java.util.concurrent包提供BlockingQueue接口的并发实现，它扩展Queue，并且提供ConcurrentMap接口的并发实现来扩展Map。这些实现提供比同步实现更高的并发性。

通常，你应该关注接口，而不是实现。所以本节中没有编程示例。对于大多数情况，对实现的选择只影响性能。如11.2节所述，首选的做法是：在创建Collection时选择一个实现，并且立即将新集合赋给具有对应的接口类型的变量（或者将集合传递给需要此接口类型的实参的方法）。采用这种方式，程序就不会依赖于给定实现中的任何附加方法，这样，如果有涉及性能或者行为细节的充分理由，程序员就可以随时改变实现。

343

后面的章节简要地讨论实现。实现的性能由以下词汇描述：常量时间（constant-time）、对数（log）、线性（linear）、n log(n)和二次方程式（quadratic）。这些词汇都是指执行操作的时间复杂性（time complexity）的渐近线上沿（asymptotic upper bound）。这些词汇是相当绕口的，如果你不知道它们的含义，也不要紧。如果想了解更多这方面的知识，请参考优秀的算法教科书。要记住的是，这种性能度量方式有局限性。有时，名义上较慢的实现对于你使用的集合大小可能会更快。拿不定主意时，应该实际测试性能！

11.3.1　Set 实现

Set实现被分为通用实现和专用实现。

1. 通用的Set实现

3个通用的Set实现是HashSet、TreeSet和LinkedHashSet。决定使用哪个实现是非常简单的。HashSet比TreeSet快得多（对于大多数操作，是常量时间与对数时间的对比关系），但是不提供排序保障。如果你需要使用SortedSet中的操作，或者需要按值排序的迭代，那么使用TreeSet。否则，使用HashSet。在大多数情况下使用HashSet是合适的。

LinkedHashSet在某种程度上介于HashSet和TreeSet之间。LinkedHashSet被实现为散列表，其中运行了链表，它提供按插入排序（insertion-ordered）的迭代（从最早插入的到最近插入的），并且运行速度几乎和HashSet一样快。LinkedHashSet实现使客户不受HashSet提供的未指定的、通常混乱的次序影响，而且免去了和TreeSet相关的额外代价。

关于HashSet要记住的一件事是，迭代是与项目数量和存储桶数量的总和（即容量）线性相关的。因此，选择过高的初始容量会浪费空间和时间。另一方面，选择过低的初始容量会浪费时间，因为每次它被迫增加容量时复制数据结构要花费时间。如果你没有指定初始容量，那么默认值为16。在过去，选择素数作为初始容量有一些优势。现在不是这样了。在内部，容量总是取整到2的幂。使用int构造器指定初始容量。下面的代码行分配一个初始容量为64的HashSet：

344

```
Set<String> s = new HashSet<String>(64);
```

HashSet类有另一个用于调优的参数，称为负载因子（load factor）。如果你非常关注HashSet的空间消耗，那么阅读HashSet的文档来了解更多的信息。否则，使用默认设置就可以了；这几乎总是正确的选择。

如果你接受默认的负载因子，但是想要指定初始容量，那么选择的数字应该大约是你预期集会增长到的尺寸的两倍。如果你的预期有偏差，可能会浪费一点儿时间或者空间，也可能二者都浪费，但是问题不会太大。

和HashSet一样，LinkedHashSet有相同的调优参数，但是迭代时间不受容量的影响。TreeSet没有调优参数。

2. 专用的Set实现

有两个专用的Set实现——EnumSet[1]和CopyOnWriteArraySet[2]。

EnumSet是用于枚举类型的高性能Set实现。枚举集的所有成员都必须是相同的枚举类型。在内部，它表示为位向量，典型情况下是单一的long值。枚举集支持对枚举类型的范围的迭代。例如，给定一周的星期的枚举声明，你可以迭代星期。EnumSet类提供静态工厂，使操作更加方便：

```
for (Day d : EnumSet.range(Day.MONDAY, Day.FRIDAY))
  System.out.println(d);
```

枚举集还为传统的位标志提供丰富的、类型安全的替换方式：

```
EnumSet.of(Style.BOLD, Style.ITALIC)
```

CopyOnWriteArraySet是通过复制数组支持的Set实现。所有导致变化的操作（比如add、set和remove）都通过制作数组的新副本而实现，从不需要加锁。即使迭代也可以与元素插入和删除安全地并发进行。和大多数Set实现不同，add、remove和contains方法与集的大小成比例。这个实现只适用于很少修改但是频繁迭代的集。它非常适合维护必须防止复制的事件处理器列表。 345

11.3.2 List 实现

List实现分为通用实现和专用实现。

1. 通用的List实现

有两个通用的List实现——ArrayList和LinkedList。在大多数时候，你可能会使用ArrayList。它提供常量时间的位置访问而且速度很快。它不必为List中的每个元素分配一个节点对象，而且在同时移动多个元素时它可以利用System.arraycopy。可以把ArrayList看作没有同步开销的Vector。

如果你经常将元素添加到List的开头，或者迭代List以便从其内部删除元素，那么应该考虑使用LinkedList。这些操作在LinkedList中需要常量时间，但在ArrayList中需要线性时间。但是你要在性能上付出高昂的代价。位置访问在LinkedList中需要线性时间，但在ArrayList中是常量时间。此外，LinkedList的常量因子要差得多。如果你认为应该使用LinkedList，那么在作出决定之前，请分别使用LinkedList和ArrayList测量性能，ArrayList通常要快一些。

ArrayList有一个调优参数——初始容量（initial capacity）。它是指在ArrayList被迫增长之前其中可以容纳的元素数量。LinkedList没有用于调整性能的参数。它有7个可选操作，其中之一是clone。另外6个是addFirst、getFirst、removeFirst、addLast、getLast和removeLast。LinkedList还实现Queue接口。

2. 专用的List实现

CopyOnWriteArrayList[3]是通过复制数组支持的List实现。这个实现在性质上和CopyOnWriteArraySet类似。不必进行同步（即使在迭代的过程中），并且确保迭代器不会抛出ConcurrentModificationException异常。这个实现非常适合维护这样的事件处理器列表：其中很少发生改动，遍历很频繁，而且可能很消耗时间。 346

---

1. docs/api/java/util/EnumSet.html
2. docs/api/java/util/concurrent/CopyOnWriteArraySet.html
3. docs/api/java/util/concurrent/CopyOnWriteArrayList.html

如果你需要同步，那么Vector将比用Collections.synchronizedList进行同步的ArrayList快一点儿。但是Vector具有旧式操作的负担，所以要谨慎，应该坚持使用List接口操作Vector，否则以后就不能替换这个实现。

如果List的尺寸是固定的——也就是说，你肯定不会使用remove、add或除containsAll以外的任何批量操作——那么有第三个值得考虑的选择。更多信息请参考11.3.6节中的Arrays.asList。

11.3.3　Map实现

Map实现分为专用实现、通用实现和并发实现三组。

1. 通用的Map实现

3个通用的Map实现是HashMap、treeMap和LinkedHashMap。如果你需要SortedMap操作或者按键排序的Collection视图迭代，就使用treeMap；如果你希望获得最大速度，并且不关心迭代顺序，就使用HashMap；如果你希望获得近似HashMap的性能和按照插入排序的迭代，就使用LinkedHashMap。在这方面，Map的情况和Set类似。同样，11.2.2节中介绍的内容也适用于Map实现。

LinkedHashMap提供LinkedHashSet不具备的两个功能。当你创建LinkedHashMap时，可以按照键访问（而不是按照插入）进行排序。换句话说，与很少被查看的值关联的键被安排到映射的末尾。另外，LinkedHashMap提供removeEldestEntry方法，可以覆盖它，以便在新的映射对加入到映射中时，自动地强制实现删除旧映射对的策略。这使定制缓存的实现变得非常容易。

例如，这个覆盖方法将允许映射增加到100个项目，然后在每次添加新项目时删除最旧的项目，以便维持100个项目的稳定状态：

```
private static final int MAX_ENTRIES = 100;

protected boolean removeEldestEntry(Map.Entry eldest) {
  return size() > MAX_ENTRIES;
}
```

347

2. 专用的Map实现

有3个专用Map实现——EnumMap[1]、WeakHashMap[2]和IdentityHashMap[3]。EnumMap在内部实现为array，它是用于枚举键的高性能Map实现。这个实现结合了Map接口的丰富功能和安全性以及数组的高速访问。如果你希望把枚举映射到值，就总应该使用EnumMap，而不使用数组。

WeakHashMap是只存储对其键的弱引用的Map接口实现。当它的键不再在WeakHashMap之外引用时，只存储弱引用允许键一值对被垃圾收集。这个类提供了利用弱引用的强大功能的最快途径。它非常适于实现“类似注册表”的数据结构，当项目的键不再能够通过任何途径到达时，这个项目就消失了。

IdentityHashMap是基于散列表的基于标识的Map实现。这个类非常适于保持拓扑的对象图转换，比如串行化或者深度复制。为了执行这样的转换，你需要维护基于标识的“节点表”，这个表跟踪已经看到的对象。基于标识的映射也用在动态调试器和类似的系统中，用来维护对象到元信息的映射。最后，基于标识的映射非常适于阻止“欺骗攻击”（通过故意调用反常的

---

1. docs/api/java/util/EnumMap.html
2. docs/api/java/util/WeakHashMap.html
3. docs/api/java/util/IdentityHashMap.html

equals方法），因为IdentityHashMap从不对其键调用equals方法。这个实现的另外一个优势是它的速度快。

3. 并发的Map实现

java.util.concurrent[1]包包含ConcurrentMap[2]接口（它扩展了Map，带有原子性的putIfAbsent、remove和replace方法），以及这个接口的ConcurrentHashMap[3]实现。

ConcurrentHashMap是通过散列表支持的高并发的高性能实现。当执行检索时这个实现不会阻塞，并且允许客户选择更新的并发级别。它的目的是替换Hashtable：除了实现 348
ConcurrentMap之外，它支持Hashtable特有的所有旧式方法。同样，如果你不需要旧的操作，就要小心地使用ConcurrentMap接口操作它。

11.3.4 Queue 实现

Queue实现分为通用实现和并发实现两组。

1. 通用的Queue实现

在前一节提到，LinkedList实现Queue接口，提供用于add、poll等方法的FIFO队列操作。

PriorityQueue类是基于堆（heap）数据结构的优先级队列。这个队列按照构造时指定的顺序对元素进行排序，这个顺序可以是元素的自然顺序，也可以是显式Comparator决定的顺序。

队列检索操作——poll、remove、peek和element——访问队列的头元素。队列的头是按照指定的顺序得出的最小元素。如果最小值涉及多个元素，那么头就是这些元素之一；最小值和这些元素之间的关系任意确定。

PriorityQueue及其迭代器实现Collection和Iterator接口的所有可选方法。iterator方法中提供的迭代器不确保按照任何特定顺序遍历PriorityQueue的元素。如果需要有序地进行遍历，可以考虑使用Arrays.sort（pq.toArray()）。

2. 并发的Queue实现

java.util.concurrent包包含一组同步的Queue接口和类。BlockingQueue扩展Queue，带有检索元素时等待队列变为非空的操作，以及存储元素时等待空间变为可用的操作。这个接口通过如下类实现：

- **LinkedBlockingQueue**[4]——通过链接节点支持的可选有界的FIFO阻塞队列。
- **ArrayBlockingQueue**[5]——通过数组支持的有界FIFO阻塞队列。
- **PriorityBlockingQueue**[6]——通过堆支持的无界阻塞优先级队列。 349
- **DelayQueue**[7]——通过堆支持的基于时间的调度队列。
- **SynchronousQueue**[8]——使用BlockingQueue接口的简单汇合机制。

---

1. docs/api/java/util/concurrent/package-summary.html
2. docs/api/java/util/concurrent/ConcurrentMap.html
3. docs/api/java/util/concurrent/ConcurrentHashMap.html
4. docs/api/java/util/concurrent/LinkedBlockingQueue.html
5. docs/api/java/util/concurrent/ArrayBlockingQueue.html
6. docs/api/java/util/concurrent/PriorityBlockingQueue.html
7. docs/api/java/util/concurrent/DelayQueue.html
8. docs/api/java/util/concurrent/SynchronousQueue.html

11.3.5　包装器实现

包装器实现将它们的所有实际工作委托给指定的集合，但在此集合提供的功能之上添加了额外的功能。对于设计模式爱好者来说，这就是装饰器（decorator）模式的一个例子。尽管它看起来有点儿怪异，但是实际上很简单。

这些实现是匿名的，库没有提供公共类，而是提供了一个静态工厂方法。所有这些实现都可以在Collections类中找到，其中只包含静态方法。

1. 同步包装器

同步包装器为任意集合添加自动同步（线程安全）。6个核心集合接口Collection、Set、List、Map、SortedSet和SortedMap都有一个静态工厂方法：

```
public static <T> Collection<T>
  synchronizedCollection(Collection<T> c);
public static <T> Set<T>
  synchronizedSet(Set<T> s);
public static <T> List<T>
  synchronizedList(List<T> list);
public static <K,V> Map<K,V>
  synchronizedMap(Map<K,V> m);
public static <T> SortedSet<T>
  synchronizedSortedSet(SortedSet<T> s);
public static <K,V> SortedMap<K,V>
  synchronizedSortedMap(SortedMap<K,V> m);
```

这些方法都返回一个由指定集合支持的同步的（线程安全的）Collection。为了保证串行化访问，对后端集合的所有访问都必须通过返回的集合来完成。保证这一点的简单办法是，

350　不保留对后端集合的引用。使用下面的技巧创建同步集合：

```
List<Type> list =
  Collections.synchronizedList(new ArrayList<Type>());
```

与一般的同步集合（比如Vector）一样，以这种方式创建的集合是线程安全的。

面对并发的访问，用户必须在迭代时在返回的集合上手工进行同步。原因是，迭代通过对集合的多个调用完成，这些操作必须被组合成单个原子操作。在包装器同步的集合上进行迭代的语法如下：

```
Collection<Type> c =
  Collections.synchronizedCollection(myCollection);
synchronized(c) {
  for (Type e : c)
    foo(e);
}
```

如果使用了显式迭代器，那么必须从同步块中调用iterator方法。不遵守这条建议可能会导致不确定的行为。在同步Map的Collection视图上进行迭代的语法与之类似。当用户在Map的任何Collection视图上进行迭代时，用户必须在同步的Map上进行同步，而不是在Collection视图本身上进行同步，如下面的示例所示：

```
Map<KeyType, ValType> m =
  Collections.synchronizedMap(new HashMap<KeyType, ValType>());
  ...
Set<KeyType> s = m.keySet();
  ...
synchronized(m) { // Synchronizing on m, not s!
  while (KeyType k : s)
    foo(k);
}
```

使用包装器实现的一个次要的副作用是，不能执行被包装实现的任何非接口（noninterface）操作。所以举例来说，在前面的List例子中，你不能在被包装的ArrayList上调用ArrayList的ensureCapacity操作。

2. 不可修改的包装器

同步包装器把功能添加到被包装的集合上，但是不可修改的包装器与之不同，它禁止了某些功能。具体地说，它们取消了修改集合的能力，采用的办法是截取会修改集合的所有操作并且抛出UnsupportedOperationException异常。不可修改的包装器有两个主要用途： 351

- 使集合在被创建之后不能被修改。在这种情况下，不保留对后端集合的引用是个好办法。这样可以绝对保证集合不可修改。
- 允许特定客户对数据结构进行只读访问。你保留对后端集合的引用，但是将对此包装器的引用交付给客户。通过这种方式，可以维持完全的访问，而客户只能查看数据，不能修改数据。

与同步包装器类似，6个核心Collection接口都有一个静态工厂方法。

```
public static <T> Collection<T>
  unmodifiableCollection(Collection<? extends T> c);
public static <T> Set<T>
  unmodifiableSet(Set<? extends T> s);
public static <T> List<T>
  unmodifiableList(List<? extends T> list);
public static <K,V> Map<K, V>
  unmodifiableMap(Map<? extends K, ? extends V> m);
public static <T> SortedSet<T>
  unmodifiableSortedSet(SortedSet<? extends T> s);
public static <K,V> SortedMap<K, V>
  unmodifiableSortedMap(SortedMap<K, ? extends V> m);
```

3. 可控接口包装器

Collections.checked接口包装器用于泛型集合。这些实现返回指定集合的动态类型安全视图，如果客户试图添加类型错误的元素，就会抛出ClassCastException异常。Java语言中的泛型机制提供编译时（静态）类型检查，但是使这种机制失效的可能性是存在的。动态的类型安全视图完全消除了这种可能性。

11.3.6 简便实现

本节描述几个小型的实现，如果你不需要通用实现的全部功能，那么可以使用这些简便实现，它们比通用实现更方便、更高效。本节中的所有实现都可以通过静态工厂方法使用，而不是通过public类。

1. 数组的List视图

Arrays.asList方法返回它的数组参数的一个List视图。对这个List的修改会被写到数组中，反之亦然。集合的尺寸就是数组的尺寸，而且不能被修改。如果在此List上调用add或 352
remove方法，那么会导致UnsupportedOperationException异常。

这个实现的一般用途是作为基于数组的API和基于集合的API之间的桥梁。它允许将数组传递给需要Collection或List的方法。但是，这个实现还有另一个用途。如果你需要一个固定尺寸的List，那么这个实现比任何通用的List实现更高效。下面是其语法：

```
List<String> list = Arrays.asList(new String[size]);
```

注意，没有保留对后端数组的引用。

2. 不可变的多副本List

有时候，你会需要一个包含同一元素的多个副本的不可变List。Collections.nCopies方法就返回这样的列表。这个实现有两个主要用途。第一个用途是初始化一个新创建的List；例如，假设你想将一个ArrayList初始化为包含1000个null元素，下面的代码完成此工作：

```
List<Type> list =
  new ArrayList<Type>(Collections.nCopies(1000, (Type)null));
```

当然，每个元素的初始值不必是null。第二个主要用途是扩展一个现有的List。例如，假设你想在一个List<String>的末尾添加字符串"fruit bat"的69个副本。我们不清楚为什么要这么做，但是假设你需要这么做。下面是具体做法：

```
lovablePets.addAll(Collections.nCopies(69, "fruit bat"));
```

采用以一个索引和一个Collection为实参的addAll形式，可以将新元素添加到List的中间而不是末尾。

3. 不可变的单元素Set

有时候，你需要一个不可变的单元素（singleton）Set，其中只包含一个指定的元素。Collections.singleton方法就返回这样的Set。这个实现的一个用途是，从一个Collection中删除指定元素的所有实例：

```
c.removeAll(Collections.singleton(e));
```

类似的语法可以用于从Map中删除映射到指定值的所有元素。例如，假设你有一个Map job，它将人映射到他们的职业，假设你想删除所有律师，下面这行代码可以完成此工作：

353

```
job.values().removeAll(Collections.singleton(LAWYER));
```

这个实现的另一个用途是，为接受值集合的方法提供单个输入值。

4. 空Set、List和Map常量

Collections类提供返回空Set、List和Map的方法——emptySet、emptyList和emptyMap。这些常量的主要用途是，在你根本不想提供任何值时，它们可以作为接受Collection的方法的输入，就像下面这个例子：

```
tourist.declarePurchases(Collections.emptySet());
```

11.3.7　实现小结

实现是用于存储集合的数据对象，它们实现11.2节中描述的接口。

Java集合框架（Java Collections Framework）提供这些核心接口的若干通用实现：

- 对于Set接口，HashSet是最常用的实现。
- 对于List接口，ArrayList是最常用的实现。
- 对于Map接口，HashMap是最常用的实现。
- 对于Queue接口，LinkedList是最常用的实现。

每个通用实现都提供其接口中包含的所有可选操作。

Java集合框架还为需要非标准性能、使用限制和其他不常见行为的情况提供了若干专用实现。

java.util.concurrent包包含若干集合实现，它们是线程安全的，但是不由单一排他锁来管理。

Collections类（而不是Collection接口）提供对集合进行操作或者返回集合的静态方法，它们被称为包装器实现。

最后，有若干简便实现，当你不需要通用实现的全部功能时，它们比通用实现效率更高。

简便实现是通过静态工厂方法使用的。 354

问题和练习：实现

问题

1. 你计划编写一个程序，使用几个基本集合接口：Set、List、Queue和Map。但你不确定哪个实现的效果最好，所以决定使用通用实现，直到你更清楚地了解程序在现实世界中的工作情况为止。这些实现是哪些实现？
2. 如果你需要一个Set实现提供按照值排序的迭代，应该使用哪个类？
3. 使用哪个类访问包装器实现？

练习

1. 编写一个程序把一个文本文件读入到List，这个文本文件由第一个命令行参数指定。程序随机输出文件中的一些行，输出行的数量由第二个命令行参数指定。使程序一次性正确地分配集合的大小，而不是随着文件的读入逐步扩展集合。提示：为了确定文件中总的行数，可以使用java.io.File.length[1]获得文件的长度，然后除以假设的行的平均长度。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/collections/implementations/QandE/answers.html

11.4 算法

这里描述的多态算法（polymorphic algorithm）是Java平台提供的可重用的功能块。这些算法都来自Collections类，它们都采用静态方法的形式，这些静态方法的第一个实参是要在其上执行操作的集合。Java平台提供的绝大多数算法在List实例上进行操作，但是其中的几个可以在任意Collection实例上进行操作。本节简要描述以下算法： 355

- 排序
- 混序
- 常规数据操作
- 搜索
- 组合
- 查找极值

11.4.1 排序

sort算法对List重新排序，使它的元素根据次序关系按升序排列。此操作有两种形式。简单的形式以一个List为参数，并且根据它的元素的自然顺序（natural ordering）排序。如果你不熟悉自然顺序的概念，那么请阅读11.2.6节。

sort操作使用一个略微优化的合并排序（merge sort）算法，它的速度快并且稳定。

- **快速**——这个算法保证以n log(n)时间运行，并且本质上快于类似的有序列表。测试经验表明它与高度优化的快速排序一样快。快速排序一般被认为比合并排序快，但是不稳定，而且不保证n log(n)性能。

---

1. docs/api/java/io/File.html

- **稳定**——它不对相等的元素重新排序。如果你根据不同的属性对相同的列表重复排序，那么这很重要。如果用户的邮件程序先按邮寄日期对收件箱排序，然后再按发信人排序，用户自然希望来自某个发信人的一系列信件仍然按邮寄日期排序。只有在第二次排序是稳定的情况下，才能保证这一点。

下面这个小程序[1]按照字母表顺序输出它的实参：

```
import java.util.*;

public class Sort {
  public static void main(String[] args) {
    List<String> list = Arrays.asList(args);
    Collections.sort(list);
    System.out.println(list);
  }
}
```

356

我们来运行这个程序：

```
% java Sort i walk the line
```

下面是产生的输出：

```
[i, line, the, walk]
```

我们提供这个程序只是为了说明算法是多么易于使用。

sort的第二个形式除了List之外，还采用一个Comparator实参，并且用这个Comparator对元素进行排序。假设你想按长度的反序输出前面例子中的变位词组——最大的变位词组先输出。下面的程序给出实现这个任务的办法，它使用了sort方法的第二个形式。

前面讲过，变位词组作为值存储在Map中，采用List实例的形式。反序输出代码迭代Map的值视图，把通过了最小长度测试的每个List放进一个List的List。然后，代码使用一个需要List实例的Comparator对这个List进行排序，并且实现长度顺序的反转。最后，代码迭代这个有序List，输出它的元素（变位词组）。下面的代码[2]替换Anagrams示例main方法末尾的输出代码：

```
// Make a List of all anagram groups above size threshold.
List<List<String>> winners = new ArrayList<List<String>>();
for (List<String> l : m.values())
  if (l.size() >= minGroupSize)
    winners.add(l);

// Sort anagram groups according to size
Collections.sort(winners, new Comparator<List<String>>() {
  public int compare(List<String> o1, List<String> o2) {
    return o2.size() - o1.size();
  }});

// Print anagram groups.
for (List<String> l : winners)
  System.out.println(l.size() + ": " + l);
```

在11.2.5节中的同一个字典上运行此程序，设置相同的最小变位词组长度（8），会产生下面的输出：

357

```
12: [apers, apres, asper, pares, parse, pears, prase, presa,
                                          rapes, reaps, spare, spear]
```

---

1. tutorial/collections/algorithms/examples/Sort.java
2. tutorial/collections/algorithms/examples/Anagrams2.java

```
11: [alerts, alters, artels, estral, laster, ratels, salter,
                                        slater, staler, stelar, talers]
10: [least, setal, slate, stale, steal, stela, taels, tales,
                                                  teals, tesla]
9: [estrin, inerts, insert, inters, niters, nitres, sinter,
                                                  triens, trines]
9: [capers, crapes, escarp, pacers, parsec, recaps, scrape,
                                                  secpar, spacer]
9: [palest, palets, pastel, petals, plates, pleats, septal,
                                                  staple, tepals]
9: [anestri, antsier, nastier, ratines, retains, retinas,
                                        retsina, stainer, stearin]
8: [lapse, leaps, pales, peals, pleas, salep, sepal, spale]
8: [aspers, parses, passer, prases, repass, spares, sparse,
                                                  spears]
8: [enters, nester, renest, rentes, resent, tenser, ternes,
                                                  treens]
8: [arles, earls, lares, laser, lears, rales, reals, seral]
8: [earings, erasing, gainers, reagins, regains, reginas,
                                        searing, seringa]
8: [peris, piers, pries, prise, ripes, speir, spier, spire]
8: [ates, east, eats, etas, sate, seat, seta, teas]
8: [carets, cartes, caster, caters, crates, reacts, recast,
                                                  traces]
```

11.4.2 混序

shuffle算法的作用与sort相反，它破坏List中可能存在的任何次序。也就是说，这个算法根据来自一个随机源的输入重新排列List的次序，这样，所有可能的排列的出现几率是相同的(假设使用的是一个公正的随机源)。这个算法在实现游戏时很有用。例如，它可以用于对Card对象的List（代表一副牌）进行混序。另外，它还有助于产生测试用的案例。

此操作有两个形式。第一个形式采用一个List实参，并且使用默认的随机源。第二个形式要求调用者提供一个用作随机源的Random[1]对象。11.2.3节的第3小节中有一个使用此算法的例子。 358

11.4.3 常规数据操作

Collections类为在List对象上执行常规数据操作提供了5个算法。这些算法都非常简单：

- **reverse**——颠倒List中元素的次序。
- **fill**——用指定的值覆盖List中的每个元素。这个操作有助于对List进行重新初始化。
- **copy**——它采用两个实参，一个目标List和一个源List，并且将源List中的元素复制到目标List，以覆盖它的内容。目标List必须至少与源List一样长。如果目标List较长，那么多余的元素不受影响。
- **swap**——交换List中位于指定位置的元素。
- **addAll**——把所有指定的元素添加到Collection中。添加的元素可以是单独指定的，也可以是一个数组。

11.4.4 搜索

binarySearch算法在有序的List中搜索指定的元素。这个算法有两个形式。第一个形式以一个List和要搜索的元素（“搜索键”）为实参。这个形式假设List按照元素的自然次序进行

---

1. docs/api/java/util/Random.html

升序排序。第二个形式除了List和搜索键之外，还有一个Comparator实参，并且假设List按照指定的Comparator进行升序排序。在调用binarySearch前，可以使用sort算法对List进行排序。

两种形式的返回值是一样的。如果List包含搜索键，那么返回它的索引。如果不包含，那么返回的值是(-(insertion point)-1)，其中的insertion point是此值会被插入List的位置，这是大于此值的第一个元素的索引，如果List中的所有元素都小于指定的值，那么这就是list.size()。这个怪异的公式保证返回值当且仅当找到搜索键的情况下大于等于零。它基本上就是将一个布尔值（found）和一个整数（index）组合为一个int返回值。

下面的语法寻找指定的搜索键，如果此搜索键还不存在，那么将它插入合适的位置，这个语法适用于binarySearch操作的两种形式：

```
int pos = Collections.binarySearch(list, key);
if (pos < 0)
  l.add(-pos-1);
```

359

11.4.5　组合

frequency和disjoint算法检查一个或多个Collection的组合的一些特性：

- **frequency**——计算指定集合中指定元素出现的次数。
- **disjoint**——判断两个Collection是否不相交，就是说，它们是否不包含共同的元素。

11.4.6　查找极值

min和max算法分别返回指定的Collection中包含的最小元素和最大元素。这两个操作都有两个形式。简单形式只有一个Collection实参，并且根据元素的自然顺序返回最小或最大元素。第二个形式除了Collection参数之外，还有一个Comparator实参，并且根据指定的Comparator返回最小或最大元素。

11.5　定制集合实现

许多程序员从来不需要实现自己的集合类，使用本章前面几节描述的实现已经可以完成许多工作。但是，有时候你可能想编写自己的实现。有了Java平台提供的抽象实现，这种工作非常简单。在讨论如何编写实现之前，我们先讨论一下为什么需要编写实现。

11.5.1　编写实现的原因

下面列出了实现自己定制的Collection的一些原因。这里并没有列出所有原因。

- **持久性**——所有内置的Collection实现都驻留在主内存中，并且在程序退出时消失。如果你希望在程序下一次启动时集合仍然存在，可以通过在外部数据库上存储集合来实现。可能有多个程序并发地访问这样的集合。

360

- **应用的特殊需要**——这是个非常广泛的范畴。一个例子是包含实时遥测数据的不可改变的Map。键可能代表位置，值可以通过get操作从这些位置的传感器读取。
- **高性能，专用**——许多数据结构由于用途的限制，能够提供比通用实现更好的性能。例如，假设一个List包含长期使用的相同元素值。这样的列表出现在文本处理过程中，它们可以采用运转周期编码（runlength encoded）——运转可以表示为包含重复的元素和连续重复的次数的单一对象。这个例子之所以有趣，是因为它对性能的两个方面产生相

反的效果：它需要的空间比ArrayList少，但是处理时间比其要长。

- **高性能，通用**——Java集合框架的设计者试图为每个接口提供最好的通用实现，但是许许多多数据结构已经投入使用，而且新的数据结构不断被开发出来。你可能可以让某些操作更快！
- **增强的功能**——假设你需要一个高效的袋（bag）实现（也称为多集）：这是一个提供常量时间的包含检查并且允许重复元素的Collection。很明显，这样的集合需要在HashMap的基础上实现。
- **简便**——除了Java平台提供的简便实现之外，你可能希望提供更便利的其他实现。例如，你可能经常需要一个代表连续范围内的Integer的List实例。
- **适配器**——假设你使用的旧式API是具有自己特殊的集合的API。你可以编写一个适配器实现，使这些集合能够在Java集合框架内操作。适配器实现（adapter implementation）可以封装某类型的对象，并且使它们的行为类似于另一类型的对象，采取的办法是将对后一类型的操作转换为对前一类型的操作。

11.5.2 如何编写定制实现

编写定制实现轻松得让人吃惊。Java集合框架提供抽象实现的设计目的明显就是为了帮助编写定制实现。我们从一个例子开始，这是Arrays.asList的一个实现： 361

```
public static <T> List<T> asList(T[] a) {
  return new MyArrayList<T>(a);
}

private static class MyArrayList<T> extends AbstractList<T> {

  private final T[] a;

  MyArrayList(T[] array) {
    a = array;
  }

  public T get(int index) {
    return a[index];
  }

  public T set(int index, T element) {
    T oldValue = a[index];
    a[index] = element;
    return oldValue;
  }

  public int size() {
    return a.length;
  }
}
```

不管你信不信，此代码与java.util.Arrays中包含的实现非常接近。它就这么简单！你提供一个构造器以及get、set和size方法，剩下的事都由AbstractList去做。这样你就自动地得到了ListIterator、批量操作、搜索操作、散列码计算、比较和字符串表示。

假设你想让此实现的速度稍微快一些。抽象实现的API文档精确地描述了每个方法是如何实现的，所以你能知道为了得到想要的性能应该覆盖哪些方法。前面这个实现的性能已经很好了，但还可以再提高。具体地说，toArray方法迭代List，每次复制一个元素。通过给出内部

表示，克隆数组要快得多而且更加合理：

```
public Object[] toArray() {
  return (Object[]) a.clone();
}
```

通过添加这个覆盖的方法和几个类似的方法，这个实现就与java.util.Arrays中的实现完全一样了。总体来说，使用其他抽象实现稍微困难一些，因为你必须编写自己的迭代器，但
362 是这仍然不是非常难。

下面总结了抽象实现：

- **AbstractCollection**[1]——一个既不是Set也不是List的Collection。至少必须提供iterator和size方法。
- **AbstractSet**[2]——一个Set。它的用途和AbstractCollection相同。
- **AbstractList**[3]——由一个随机访问的数据存储（比如数组）支持的List。至少必须提供位置访问方法（get以及可选的set、remove和add）和size方法。这个抽象类负责实现listIterator（和iterator）。
- **AbstractSequentialList**[4]——由一个顺序访问的数据存储（比如链表）支持的List。至少必须提供listIterator和size方法。这个抽象类负责实现位置访问方法。（这与AbstractList相反。）
- **AbstractQueue**[5]——至少必须提供offer、peek、poll和size方法，以及支持remove的iterator。
- **AbstractMap**[6]——一个Map。至少必须提供entrySet视图。这通常用AbstractSet类实现。如果这个Map是可修改的，那么还必须提供put方法。

编写定制实现的过程如下：

(1) 从前面的列表中选择合适的抽象实现类。

(2) 为类的所有抽象方法提供实现。如果你的定制集合是可修改的，那么还必须覆盖一个或多个具体的方法。抽象实现类的API文档会告诉你应该覆盖哪些方法。

(3) 测试实现，如果必要的话，还要调试实现。你现在有了一个可以工作的定制集合实现。

(4) 如果关心性能，那么请阅读抽象实现类的API文档，了解你继承的所有方法实现。如果
363 其中有些似乎太慢，就覆盖它们。如果你覆盖了任何方法，那么确保在进行覆盖前后测试方法的性能。你对性能提高要作出多大努力取决于实现的应用程度以及性能的重要程度。（这一步常常被忽视。）

11.6　互操作性

在本节中，你将学习互操作性的两个方面：兼容性和API设计。

- 兼容性：这个主题介绍如何使当前集合与在Java平台添加Collection之前就存在的旧式API一起工作。

---

1. docs/api/java/util/AbstractCollection.html
2. docs/api/java/util/AbstractSet.html
3. docs/api/java/util/AbstractList.html
4. docs/api/java/util/AbstractSequentialList.html
5. docs/api/java/util/AbstractQueue.html
6. docs/api/java/util/AbstractMap.html

- API设计：这个主题介绍如何设计新的API，以便它们之间能够无缝地相互操作。

11.6.1 兼容性

Java集合框架的设计目的是，确保核心集合接口（参见11.2节）和Java平台的早期版本中表示集合的那些类型（Vector、Hashtable、数组和Enumeration[1]）之间能够完全互操作。在本节中，你将学习如何在传统的集合和新的集合之间进行相互转换。

1. 向上兼容

假设你需要把一个返回旧式集合的API和另一个需要实现集合接口的对象的API结合使用。为使这两个API顺利地互操作，你必须在旧式集合转换为新的集合。幸运的是，Java集合框架简化了这个工作。

假设旧式API返回一个对象数组，而新式API需要一个Collection。集合框架有一个简便实现，它允许将对象数组当成List。可以使用Arrays.asList将数组传递给需要Collection或List的任何方法：

```
Foo[] result = oldMethod(arg);
newMethod(Arrays.asList(result));
```

如果旧式API返回一个Vector或Hashtable，那么你什么都不用做，因为Vector已经被改进为实现List接口，而Hashtable已经被改进为实现Map接口。因此，Vector可以被直接传递给需要Collection或List的任何方法调用：

364

```
Vector result = oldMethod(arg);
newMethod(result);
```

类似地，Hashtable可以被直接传递给需要Map的任何方法调用：

```
Hashtable result = oldMethod(arg);
newMethod(result);
```

比较少见的情况是，API可能返回一个代表对象集合的Enumeration。Collections.list方法可以把Enumeration转换为Collection：

```
Enumeration e = oldMethod(arg);
newMethod(Collections.list(e));
```

2. 向下兼容

假设你使用一个返回新集合的API，它与另一个需要旧式集合的API协作。为使这两个API顺利地互操作，你必须将新集合转换为旧式集合。同样，Java集合框架简化了这个工作。

假设新API返回一个Collection，而旧API需要一个Object数组。你可能注意到了，Collection接口包含一个toArray方法，它是专门为这种情况而设计的：

```
Collection c = newMethod();
oldMethod(c.toArray());
```

如果旧的API需要一个String（或其他类型）的数组，而不是Object数组，那么怎么办？只需使用toArray的另一种形式——它以一个数组作为输入：

```
Collection c = newMethod();
oldMethod((String[]) c.toArray(new String[0]));
```

如果旧的API需要一个Vector，那么使用标准的集合构造器很方便：

```
Collection c = newMethod();
oldMethod(new Vector(c));
```

---

1. docs/api/java/util/Enumeration.html

如果旧的API需要一个Hashtable，那么处理方式与上面类似：

```
Map m = newMethod();
oldMethod(new Hashtable(m));
```

365

最后，如果旧的API需要一个Enumeration，那么怎么办？这种情况并不常见，但是有时会发生，Collections.enumeration方法用于处理此情况。这个静态工厂方法采用一个Collection实参，并且返回Collection的元素上的Enumeration：

```
Collection c = newMethod();
oldMethod(Collections.enumeration(c));
```

11.6.2　API设计

在这个简短但重要的一节中，你将学习几条简单的指导方针，它们会帮助你的API与其他所有遵循这些指导方针的API顺利地互操作。实际上，这些指导方针定义了API要想成为新的集合群体中的合格“成员”应该做些什么。

1. 参数

如果你的API包含一个需要以集合作为输入的方法，那么最重要的是，将有关的参数类型声明为集合接口类型之一（参见11.2节）。绝不要使用实现类型（参见11.3节），因为这会破坏基于接口的集合框架的灵活性；如果只使用接口类型，就可以在不理会实现细节的情况下操作集合。

而且，应该坚持使用特殊性尽可能小的类型。例如，如果可以使用Collection（参见11.2.1节），就不要使用List（参见11.2.3节）或Set（参见11.2.2节）。但这并不是说你决不应该使用List或Set作为输入；如果一个方法依赖于这些接口的属性，那么就应该使用它们。例如，Java平台提供的许多算法需要以List作为输入，因为List是有序的，而这些算法要依赖于这一点。但是，总体原则是，用于输入的最佳类型就是最通用的类型：Collection和Map。

警告　决不要定义自己的特殊的Collection类并且要求以这个类的对象作为输入。如果这么做，就会失去Java集合框架（参见11.1节）提供的所有优势。

2. 返回值

在返回值方面，灵活性要比输入参数方面大得多。可以返回一个实现或扩展一个集合接口

366

的任何类型的对象。这可以是一个接口，或者扩展或实现集合接口的专用类型。

例如，有人设想了一个图像处理包，它返回一个实现List的新类（称为ImageList）的对象。除了List操作，ImageList还可以支持此应用程序可能需要的任何特定操作。例如，它可能提供indexImage操作，这个操作返回一个图像，其中包含ImageList中每个图形的缩略图。重要的是，即使API以ImageList实例作为输出，它也应该接受任意Collection（或者List）实例作为输入。

在某种意义上，返回值的原则应该与输入参数相反：最好返回可应用的最特殊化的集合接口，而不是最通用的。例如，如果你肯定总是会返回SortedMap，那么应该使有关方法的返回类型为SortedMap，而不是Map。与一般的Map实例相比，建立SortedMap实例花费的时间更多，但SortedMap实例的功能更强大。既然你的模块已经花时间建立了SortedMap，那么让用户访问那些增强的功能是很有意义的。这样，用户就能够将返回的对象传递给需要SortedMap的方法，也能够传递给需要Map的方法。

3. 旧式API

目前有很多API定义它们自己的特殊集合类型。很不幸，Java平台的前两个主要版本中没有

提供集合框架，这是无法改变的事实。如果你有这样的API，可以使用下面的处理方法。

如果可能的话，改进你的旧式集合类型，实现一种标准集合接口。这样，返回的所有集合就可以和其他基于集合的API进行平稳的互操作。如果这不可能（例如，因为一个或者多个先前存在的类型签名和标准集合接口冲突），那么可以定义适配器类（adapter class）来包装你的旧式集合对象，让它像标准集合那样工作。（Adapter类是定制实现的一个例子，参见11.5节。）

如果可能的话，按照输入指导方针改进你的API的新调用，让它接受标准集合接口的对象。这样的调用可以和接受旧式集合类型的调用并存。如果这不可能，可以为你的旧式类型提供构造器或者静态工厂，它们接受一个实现标准接口的对象，并且返回包含相同元素（或者映射）的旧式集合。以上任何一种途径都允许用户把任意的集合传递给你的API。 367

第12章 并　　发

计算机用户认为他们的系统理所当然地可以同时进行多项工作。他们假设在字处理程序工作的同时，其他应用程序可以下载文件、管理打印队列和播放音频。即使单一的应用程序也经常被认为可以同时进行一项以上的工作——例如，播放音频的应用程序必须同时从网络读取数字音频、解压缩、管理重放和更新其显示。即使字处理程序也应该时刻准备对键盘和鼠标事件作出响应，不管它是否忙于处理文本或者更新显示。可以这样进行操作的软件被称为并发（concurrent）软件。

Java平台被设计为完全支持并发编程，Java编程语言和Java类库具有基本的并发性支持。从版本5.0开始，Java平台还包括高级并发API。本章介绍Java平台的基本并发支持，并且总结`java.util.concurrent`包中的一些高级API。

12.1 进程和线程

在并发编程中，有两个基本的执行单位：进程（process）和线程（thread）。在Java编程语言中，并发编程主要涉及线程。但是，进程也很重要。

计算机系统中通常有很多活动的进程和线程。即使在只有一个执行核心（因此在任何给定时刻实际执行的线程只有一个）的系统中也是如此。通过使用称为时间片的OS特性，在进程和线程之间共享单一核心的处理时间。

越来越常见的情况是计算机系统具有多个处理器，或者处理器具有多个执行核心。这在很大程度上增强了系统对进程和线程的并发执行能力——但即使在不具备多个处理器或者多个执行核心的简单系统上，并发也是可能的。

12.1.1 进程

进程具有自成一体的执行环境。一个进程通常具有一组完整的、私有的基本运行时资源；特别是，每个进程都具有它自己的内存空间。

进程经常被看作程序或者应用程序的同义语。但是，用户看到的单一应用程序可能实际上是一组相互协作的进程。为了实现进程之间的通信，大多数操作系统都支持进程间通信（Inter Process Communication，IPC）资源，比如管道和套接字。IPC不仅用于同一个系统上进程之间的通信，也用于不同系统上进程间的通信。

Java虚拟机的大多数实现都作为单一进程运行。Java应用程序可以使用`ProcessBuilder`[1]对象创建额外的进程。多进程应用程序超出了本章的讨论范围。

12.1.2 线程

线程有时被称为轻量进程（lightweight process）。进程和线程都提供一个执行环境，但是创

1. docs/api/java/lang/ProcessBuilder.html

建一个新线程需要的资源比创建新进程要少。

线程存在于进程内——每个进程都至少有一个线程。线程共享进程的资源，包括内存和打开的文件。这是为了进行有效的通信，但可能会造成潜在问题。

多线程执行是Java平台的本质特性。每个应用程序都具有至少一个线程；如果算上进行内存管理和信号处理等工作的“系统”线程，那么可能有若干个线程。但是从应用程序程序员的角度来说，只从一个线程开始，这个线程被称为主线程（main thread）。这个线程具有创建其他线程的能力，我们将在下一节中进行演示。 370

12.2 Thread 对象

每个线程都和Thread类[1]的一个实例相关联。使用Thread对象创建并发应用程序的基本策略有两种：

- 为了直接控制线程的创建和管理，应用程序每当需要初始化异步任务时，只需实例化Thread。
- 为了将线程管理与应用程序的其他部分分离开，需要把应用程序的任务传递给一个执行器（executor）。

这一节介绍了Thread对象的使用。执行器和其他高级并发对象一起讨论（参见12.7节）。

12.2.1 定义和启动线程

创建Thread实例的应用程序必须提供线程中运行的代码。可以通过两个途径完成这个工作：

- **提供Runnable对象**。Runnable接口[2]定义单一方法run，目的是包含线程中执行的代码。Runnable对象被传递给Thread构造器，如HelloRunnable[3]示例所示：

```
public class HelloRunnable implements Runnable {

  public void run() {
    System.out.println("Hello from a thread!");
  }

  public static void main(String args[]) {
    (new Thread(new HelloRunnable())).start();
  }
}
```

- **子类化Thread**。Thread类本身实现Runnable，但是它的run方法不进行任何操作。应用程序可以子类化Thread，提供它自己的run实现，如HelloThread[4]示例所示： 371

```
public class HelloThread extends Thread {

  public void run() {
    System.out.println("Hello from a thread!");
  }

  public static void main(String args[]) {
    (new HelloThread()).start();
  }
```

1. docs/api/java/lang/Thread.html
2. docs/api/java/lang/Runnable.html
3. tutorial/essential/concurrency/example/HelloRunnable.java
4. tutorial/essential/concurrency/example/HelloThread.java

```
}
```

注意，两个示例都调用Thread.start来启动新的线程。

应该使用哪种语法？第一种语法使用Runnable对象，它更加通用，因为Runnable对象可以子类化Thread之外的类。第二种语法在简单应用程序中更加容易使用，但是其局限性在于你的任务类必须是Thread的后裔。本章把重点放在第一种方法上，它把Runnable任务和执行任务的Thread对象分离开。这种方法不仅更加灵活，而且适用于后面所要介绍的高级线程管理API。

Thread类定义了很多用于线程管理的方法。包括static方法，它们提供关于调用方法的线程的状态信息，或者影响调用方法的线程的状态。从其他线程调用的其他方法涉及到管理线程和Thread对象。我们将在下面的几节中例证其中的一些方法。

12.2.2　使用 sleep 暂停执行

Thread.sleep使当前线程的执行挂起一段指定的时间。这可以有效地使应用程序的其他线程或者运行在计算机系统上的其他应用程序可以使用处理器时间。sleep方法也可以用于定时，如后面的示例所示，也可以用于等待另一个有时间要求的线程，如后面一节的示例SimpleThreads所示。

提供两个sleep的重载版本：一个以毫秒指定睡眠时间，另一个以纳秒指定睡眠时间。但是并不保证这些睡眠时间的精确性，因为它们受到底层OS提供的设施的限制。另外，中断可以终止睡眠时间，我们将在下一节中介绍。在任何一种情况下，都不能假设调用sleep就会按照
372 指定的时间精确地挂起线程。

示例SleepMessages[1]使用sleep按4秒的间隔输出消息：

```
public class SleepMessages {

  public static void main(String args[])
                      throws InterruptedException {
    String importantInfo[] = {
      "Mares eat oats",
      "Does eat oats",
      "Little lambs eat ivy",
      "A kid will eat ivy too"
    };

    for (int i = 0; i < importantInfo.length; i++) {
      // Pause for 4 seconds
      Thread.sleep(4000);
      // Print a message
      System.out.println(importantInfo[i]);
    }
  }
}
```

注意，main方法声明它可以抛出InterruptedException异常。当另一个线程中断了已启动sleep的当前线程时就会抛出这个异常。因为这个应用程序没有定义会导致中断的另一个线程，所以不必费心去捕获InterruptedException异常。

12.2.3　中断

中断（interrupt）指示线程停止正在进行的工作去进行其他工作。由程序员决定线程如何响

---

1. tutorial/essential/concurrency/example/SleepMessages.java

应中断，但是终止线程非常常见。这是本章中重点强调的一种应用方式。

线程通过调用被中断线程的Thread对象的interrupt方法来发送中断。为了使中断机制正确地工作，被中断的线程必须支持它自己的中断。 373

1. 支持中断

一个线程如何支持它自己的中断呢？这取决于线程当前在做什么。如果线程频繁地调用抛出InterruptedException异常的方法，捕获到这个异常之后，线程就会从run方法返回。例如，假设示例SleepMessages中的主要消息循环在线程的Runnable对象的run方法中。那么就可以对它作出如下改动以便支持中断：

```
for (int i = 0; i < importantInfo.length; i++) {
  // Pause for 4 seconds
  try {
      Thread.sleep(4000);
  } catch (InterruptedException e) {
      // We've been interrupted: no more messages.
      return;
  }
  // Print a message
  System.out.println(importantInfo[i]);
}
```

很多抛出InterruptedException异常的方法（比如sleep）被设计为，当接收到中断时，立即取消当前操作并返回。

如果一个线程长时间没有调用抛出InterruptedException异常的方法，该怎么办？那么就必须定期地调用Thread.interrupted，如果接收到中断，它就返回true。例如：

```
for (int i = 0; i < inputs.length; i++) {
  heavyCrunch(inputs[i]);
  if (Thread.interrupted()) {
    // We've been interrupted: no more crunching.
    return;
  }
}
```

在这个简单的示例中，代码简单地检测中断，如果接收到中断，就退出线程。在更加复杂的应用程序中，抛出InterruptedException异常可能更有意义：

```
if (Thread.interrupted()) {
  throw new InterruptedException();
}
```

这使中断处理代码集中在catch子句中。 374

2. 中断状态标志

使用内部标志实现中断机制，这个标志被称为中断状态（interrupt status）。调用Thread.interrupt设置这个标志。当线程通过调用静态方法Thread.interrupted检查中断时，中断状态被清除。一个线程可以使用非静态方法Thread.isInterrupted查询另一个进程的中断状态，这个方法不改变中断状态标志。

按照约定，通过抛出InterruptedException异常而退出的所有方法，当抛出异常时，都会清除中断状态。但是，总有可能另一个调用interrupt的线程立即再次设置中断状态。

12.2.4 join

join方法让一个线程等待另一个线程的完成。如果t是一个Thread对象，其线程正在执行，那么

```
t.join();
```

会导致当前线程暂停执行，直到t的线程终止。join的重载版本允许程序员指定等待时间。但是和sleep一样，join的计时取决于OS，所以不能假设join等待的时间和指定的时间确切相等。

和sleep一样，join通过抛出InterruptedException异常并且退出的方式来响应中断。

12.2.5 SimpleThreads 示例

下面的示例结合了这一节中介绍的一些概念。SimpleThreads[1]由两个线程构成。第一个主线程是每个Java应用程序都有的。主线程从Runnable对象创建新的线程MessageLoop，并且等待它的完成。如果MessageLoop线程的完成花费了过长时间，那么主线程就会中断它。

MessageLoop线程输出一系列消息。如果在输出完所有消息之前中断它，MessageLoop线程就输出一条消息并且退出：

375

```
public class SimpleThreads {

  // Display a message, preceded by the name
  // of the current thread
  static void threadMessage(String message) {
    String threadName = Thread.currentThread().getName();
    System.out.format("%s: %s%n", threadName, message);
  }

  private static class MessageLoop implements Runnable {
    public void run() {
      String importantInfo[] = {
        "Mares eat oats",
        "Does eat oats",
        "Little lambs eat ivy",
        "A kid will eat ivy too"
      };
      try {
        for (int i = 0; i < importantInfo.length; i++) {
          // Pause for 4 seconds
          Thread.sleep(4000);
          // Print a message
          threadMessage(importantInfo[i]);
        }
      } catch (InterruptedException e) {
          threadMessage("I wasn't done!");
      }
    }
  }

  public static void main(String args[])
      throws InterruptedException {

    // Delay, in milliseconds before we interrupt MessageLoop
    // thread (default one hour).
    long patience = 1000 * 60 * 60;

    // If command line argument present,
    // gives patience in seconds.
    if (args.length > 0) {
      try {
          patience = Long.parseLong(args[0]) * 1000;
```

1. tutorial/essential/concurrency/example/SimpleThreads.java

```
    } catch (NumberFormatException e) {
        System.err.println("Argument must be an integer.");
        System.exit(1);
    }
}

threadMessage("Starting MessageLoop thread");
long startTime = System.currentTimeMillis();
Thread t = new Thread(new MessageLoop());
t.start();

threadMessage("Waiting for MessageLoop thread to finish");
// loop until MessageLoop thread exits
while (t.isAlive()) {
  threadMessage("Still waiting...");
  // Wait maximum of 1 second for MessageLoop thread to
  // finish.
  t.join(1000);
  if (((System.currentTimeMillis() - startTime) > patience)
                                          && t.isAlive()) {
    threadMessage("Tired of waiting!");
    t.interrupt();
    // Shouldn't be long now -- wait indefinitely
    t.join();
  }

}
threadMessage("Finally!");
  }
}
```

376

12.3 同步

线程通信主要是通过共享访问字段和字段引用的对象完成的。这种通信形式非常有效，但是有可能出现两种错误：线程干扰（thread interference）和内存一致性错误（memory consistency error）。用来防止这些错误的工具是同步（synchronization）。

12.3.1 线程干扰

查看下面称为Counter[1]的示例类：

377

```
class Counter {
  private int c = 0;

  public void increment() {
    c++;
  }

  public void decrement() {
    c--;
  }

  public int value() {
    return c;
  }
```

1. tutorial/essential/concurrency/example/Counter.java

```
}
```

Counter被设计为在每次调用increment时将c加1，而每次调用decrement时将c减1。但是，如果多个线程引用Counter对象，线程之间的干扰就可能导致预期之外的结果。

当运行在不同线程中的两个操作对相同数据进行操作时（即发生交错（interleave）），就会出现干扰。这就是说，两个操作由多个步骤构成，并且操作步骤的序列重叠了。

对Counter实例的操作似乎不可能交错，因为两个操作对于c都是单一的简单语句。但是，即使简单语句也可以被虚拟机转换为多个步骤。我们不分析虚拟机采取的特定步骤——知道单一表达式c++可以分解为三个步骤就足够了：

(1) 获得c的当前值。

(2) 将获得的值加1。

(3) 把增加后的值存储回c。

表达式c--也可以按照相同的方式分解，只是第二个步骤进行减操作，而不是加操作。

假设线程A调用increment语句，线程B几乎同时调用decrement语句。如果c的初始值为0，那么交错的操作可能是如下顺序：

(1) 线程A：获得c的值。

(2) 线程B：获得c的值。

(3) 线程A：递增获得的值；结果为1。

378

(4) 线程B：递减获得的值；结果为－1。

(5) 线程A：把结果存储回c；现在c为1。

(6) 线程B：把结果存储回c；现在c为－1。

线程A的结果丢失了，被线程B的结果覆盖。这种特定的交错只是其中一种可能。在不同环境下可能是线程B的结果丢失，或者完全没有错误。因为交错情况是不可预期的，所以线程交错错误难以检测和纠正。

12.3.2　内存一致性错误

内存一致性错误（Memory consistency error）出现在相同的数据具有不一致的视图的不同线程中。导致内存一致性错误的原因很复杂，并且超出了本教材的讨论范围。幸运的是，程序员不需要详细地了解这些起因。程序员需要的是避免这些错误的策略。

避免内存一致性错误的关键在于了解发生前（happens-before）关系。这种关系确保被一个特定语句写的内存对另一个特定语句是可见的。为了了解这一点，我们分析下面的示例。假设定义和初始化一个简单的int字段：

```
int counter = 0;
```

counter字段在两个线程A和B之间共享。假设线程A递增counter：

```
counter++;
```

然后，线程B马上输出counter：

```
System.out.println(counter);
```

如果这两个语句是在同一个线程中执行的，那么就能安全地假定输出的值为“1”。但是，如果两个语句是在不同的线程中执行的，那么输出的值就可能为“0”，因为不能确保线程A对counter的改动对线程B是可见的——除非程序员在这两个语句之间建立了发生前关系。

有几个操作可以创建发生前关系。其中之一就是同步，我们将在下面几个小节中进行介绍。

我们已经看到过两个创建发生前关系的操作。

- 当一个语句调用Thread.start时，和这个语句具有发生前关系的每个语句也都和新线程执行的每个语句具有发生前关系。创建新线程的代码的影响对新线程是可见的。 379
- 当线程终止并且导致另一个线程中的Thread.join返回时，被终止的线程执行的所有语句和成功连接之后的所有语句都具有发生前关系。现在线程中代码的影响对执行连接的线程是可见的。

创建发生前关系的操作清单请参考java.util.concurrent package包[1]的Summary页面。

12.3.3 同步方法

Java编程语言提供两种基本的同步语法。本节介绍比较简单的一种——同步方法（synchronized method）。下一节中介绍另一种语法——同步语句（synchronized statement）。

为了使方法同步，只需把synchronized关键字添加到方法的声明中即可：

```
public class SynchronizedCounter {
  private int c = 0;

  public synchronized void increment() {
    c++;
  }

  public synchronized void decrement() {
    c--;
  }

  public synchronized int value() {
    return c;
  }
}
```

如果count是SynchronizedCounter[2]的一个实例，那么使这些方法同步有两个作用：

- 首先，对同一个对象的同步方法的两个调用不可能交错。当一个线程执行对象的一个同步方法时，调用同一个对象的同步方法的所有其他线程都会阻塞（暂停执行），直到第一个线程对对象的操作完成为止。
- 其次，当同步方法存在时，它自动地和同一个对象的同步方法的任何后续调用都建立发生前关系。这样就确保对象状态的改动对所有线程都是可见的。 380

注意，构造器不能同步——在构造器中使用synchronized关键字会出现语法错误。使构造器同步没有任何意义，因为当构造对象时，只有创建对象的线程能够访问它。

警告 当构造将在线程之间共享的对象时，一定不要过早地“泄露”对象的引用。例如，假设你希望使名为instances的List包含类的每个实例。你可能试图把这行代码加到你的构造器中：

```
instances.add(this);
```

但是，在对象的构造完成之前，其他线程可能会使用instances访问对象。

同步方法是防止线程干扰和内存一致性错误的简单策略：如果对象对于多个线程是可见的，那么对这个对象的变量的所有读写操作都通过同步方法完成。（一个重要的例外是final字段，

---

1. docs/api/java/util/concurrent/package-summary.html
2. tutorial/essential/concurrency/example/SynchronizedCounter.java

创建对象之后不能修改这种字段；所以在构造对象之后，可以通过非同步线程安全地读取它。）这种策略是有效的，但是在活性方面可能出现问题。（参见本章后面的12.4节。）

12.3.4　内在锁和同步

同步是围绕被称为内在锁（intrinsic lock）或者监视器锁（monitor lock）的内部实体构建的。（API规范经常把这个实体简单地称为“监视器”。）内在锁在同步的两个方面起作用：强制对对象状态的独占访问，以及建立可见性所需的发生前关系。

每个对象都具有与其相关联的内在锁。按照约定，需要对对象的字段进行独占和一致性访问的线程，在进行访问之前，必须获得（acquire）这个对象的内在锁，访问操作完成之后必须释放（release）内在锁。在从获得锁到释放锁的时间段内，线程被称为拥有（own）内在锁。只
381 要线程拥有内在锁，其他线程就不能获得同一个锁。试图获得锁的其他线程将被阻塞。

当线程释放内在锁时，就在这个操作和同一个锁的任何后续获得操作之间建立了发生前关系。

1. 同步方法中的锁

当线程调用同步方法时，它自动获得对象的这个方法的内在锁，并且在方法返回时释放锁。即使返回是由未捕获的异常导致的，也会释放锁。

你可能会感到奇怪，当调用静态同步方法时会发生什么，因为静态方法和类相关联，而不是和对象相关联。在这种情况下，线程获得和这个类相关联的Class对象的内在锁。因此，对类的静态字段的访问由和类的任何实例的锁所不同的锁控制。

2. 同步语句

创建同步代码的另一个途径是使用同步语句（synchronized statement）。和同步方法不同，同步语句必须指定提供内在锁的对象：

```
public void addName(String name) {
  synchronized(this) {
    lastName = name;
    nameCount++;
  }
  nameList.add(name);
}
```

在这个例子中，addName方法需要同步对lastName和nameCount的改动，但是还需要避免同步其他对象的方法的调用。（从同步代码调用其他对象的方法可能产生问题，这将在12.4节中讨论。）如果没有同步语句，就必须使用单独的非同步方法达到调用nameList.add的目的。

同步语句也用于通过精细调整的同步来改进并发性。例如，假设MsLunch类有两个实例字段c1和c2，它们不会一起使用。必须同步这两个字段的所有更新，但是没有理由禁止c1的更新和c2的更新交错——并且这样做会导致不必要的阻塞而降低并发性。不使用同步方法或使用和
382 this相关联的阻塞，而是创建两个对象单独地提供锁定：

```
public class MsLunch {
  private long c1 = 0;
  private long c2 = 0;
  private Object lock1 = new Object();
  private Object lock2 = new Object();

  public void inc1() {
    synchronized(lock1) {
      c1++;
    }
  }
```

```
  public void inc2() {
    synchronized(lock2) {
      c2++;
    }
  }
}
```

使用这种语法要非常谨慎。必须非常确定交错访问相关字段的安全。

3. 可重入同步

回忆一下，一个线程不能获得另一个线程拥有的锁。但是，线程可以获得它已经拥有的锁。允许线程多次获得同一个锁，就允许可重入同步（reentrant synchronization）。这种情况是指同步代码直接或者间接地调用也包含了同步代码的方法，并且两个代码集都使用同一个锁。如果没有可重入同步，同步代码就必须采取很多额外的预防措施避免线程阻塞自己。

12.3.5 原子访问

在编程中，原子（atomic）操作是必须同时整体完成的操作。原子操作不能在中间停止。原子操作要么完全发生，要么完全不发生。直到操作完成之前，原子操作的副作用是不可见的。

我们前面见过的递增表达式（比如c++）不能被描述为原子操作。即使是非常简单的表达式，也可以定义成可以分解为其他操作的复杂操作。但是，你可以指定一些操作是原子操作：

- 引用变量和大部分原始变量（除long和double之外的所有类型）的读取和写入都是原子的。
- 被声明为volatile的所有变量（包括long和double变量）的读取和写入是原子的。

383

原子操作不可能交错，所以使用它们时不必担心线程干扰的问题。但是，这并不意味着完全不需要对原子操作进行同步，因为内存一致性错误仍然可能出现。使用volatile变量减少了内存一致性错误的风险，因为volatile变量的任何写入操作都会和对同一个变量的后续读取建立发生前关系。这就是说，对volatile变量的改动对于其他线程总是可见的。另外，这也意味着，当线程读取volatile变量时，它不仅了解volatile变量的最新更新，而且知道导致改动的代码的副作用。

警告 volatile变量的描述只适用于Java平台5.0和以后的版本。Java平台以前的版本使用稍有不同的语义来描述volatile变量。

和通过同步代码访问变量相比，使用简单的原子变量访问的效率更高，但是程序员需要更加谨慎，以便避免内存一致性错误。并根据应用程序的规模和复杂性，判断是否需要额外的处理。

java.util.concurrent包中的一些类提供不依靠同步的原子方法。我们将在12.7节中讨论它们。

12.4 活性

并发应用程序按照及时方式执行的能力被称为活性（liveness）。这一小节介绍最常见的活性问题类型——死锁，并且简要地介绍另外两种活性问题——饿死和活锁。

12.4.1 死锁

死锁（deadlock）描述这样的情况：两个或者多个线程相互等待，以致永远阻塞。下面是一

个例子。

384 Alphonse和Gaston是朋友，都非常讲礼貌。礼貌的一个严格规则是，当你向一个朋友鞠躬时，必须保持鞠躬状态，直到你的朋友向你还礼。不幸的是，这一规则没有考虑到这样的可能性：两个朋友可能同时向对方鞠躬。下面的示例应用程序Deadlock[1]模拟了这种可能性：

```
public class Deadlock {
  static class Friend {
    private final String name;
    public Friend(String name) {
      this.name = name;
    }
    public String getName() {
      return this.name;
    }
    public synchronized void bow(Friend bower) {
      System.out.format("%s: %s has bowed to me!%n",
        this.name, bower.getName());
      bower.bowBack(this);
    }
    public synchronized void bowBack(Friend bower) {
      System.out.format("%s: %s has bowed back to me!%n",
        this.name, bower.getName());
    }
  }

  public static void main(String[] args) {
    final Friend alphonse = new Friend("Alphonse");
    final Friend gaston = new Friend("Gaston");
    new Thread(new Runnable() {
      public void run() { alphonse.bow(gaston); }
    }).start();
    new Thread(new Runnable() {
      public void run() { gaston.bow(alphonse); }
    }).start();
  }
}
```

运行Deadlock，当两个线程试图调用bowBack时，它们非常可能被阻塞。每个阻塞都不会结束，因为每个线程都在等待另一个线程退出bow。

12.4.2　饿死和活锁

饿死和活锁是比死锁少见一些的问题，但它们仍然是并发软件的每个设计者都可能遇到的
385 问题。

1. 饿死

饿死（starvation）描述这样的情况：一个线程不能获得对共享资源的常规访问，并且不能继续工作。当共享资源被“贪婪”线程长期占有而不可用时，就会发生这种情况。例如，假设一个对象提供的同步方法经常需要很长时间才能返回。如果一个线程频繁地调用这个方法，那么需要频繁地同步访问同一个对象的其他线程就经常会被阻塞。

2. 活锁

一个线程经常对另一个线程的操作作出响应。如果另一个线程的操作也对这个线程的操作作出响应，那么就可能导致活锁（livelock）。和死锁一样，发生活锁的线程不能进行进一步操

1. tutorial/essential/concurrency/example/Deadlock.java

作。但是，线程没有被锁定——它们只是忙于相互响应，以致不能恢复工作。这种情况可以比喻为两个人在走廊中相遇。Alphonse避让到自己的左边让Gaston通过，而Gaston同时避让到自己的右边让Alphonse通过。发现他们仍然挡住了对方，Alphonse就避让到自己的右边，而Gaston同时避让到了左边。他们还是挡住了对方，所以就没完没了了。

12.5 保护块

线程经常必须对它们的操作进行协调。最常用的协调语法就是保护块（guarded block）。这样的块首先检查一个条件，在块继续执行之前，这个条件必须为true。为了使保护块正确工作，需要采用很多步骤。

例如，假设guardedJoy是一个方法，它在另一个线程设置共享变量joy之前不能继续工作。在理论上，这样的方法可以简单地进行循环，直到满足条件为止，但是使用循环是浪费的，因为它在等待时仍在执行：

```
public void guardedJoy() {
  // Simple loop guard. Wastes processor time. Don't do this!
  while(!joy) {}
  System.out.println("Joy has been achieved!");
}
```

效率更高的保护块调用Object.wait[1]暂停当前线程。在另一个线程发出通知，表明一些特定事件可能已经发生（但未必是这个线程等待的事件）之前，wait的调用不会返回： 386

```
public synchronized void guardedJoy() {
  // This guard only loops once for each special event,
  // which may not be the event we're waiting for.
  while(!joy) {
    try {
      wait();
    } catch (InterruptedException e) {}
  }
  System.out.println("Joy and efficiency have been achieved!");
}
```

注意 常常在检查等待条件的循环内调用wait。不要假设你正在等待的特定条件的中断，或假设条件仍然为真。

和暂停执行的很多方法一样，wait可以抛出InterruptedException异常。在这个例子中，我们可以简单地忽略这个异常——因为我们只关心joy的值。

为什么guardedJoy的这个版本是同步的？假设d是我们用于调用wait的对象。当一个线程调用d.wait时，它必须拥有d的内在锁——否则就会抛出错误。在同步方法内调用wait是获得内在锁的一个简单途径。

调用wait时，线程释放锁并且暂停执行。在未来的某个时刻，另一个线程获得同一个锁并且调用Object.notifyAll，通知等待这个锁的所有线程，一些重要的事情发生了：

```
public synchronized void notifyJoy() {
  joy = true;
  notifyAll();
}
```

1. docs/api/java/lang/Object.html

第二个线程释放锁之后的某个时刻，第一个线程重新获得锁，并且通过wait调用返回，从而继续执行。

注意　还有第二个通知方法notify，它唤醒单一进程。因为notify不允许你指定要唤醒的线程，所以它只适用于大规模并行的应用程序——就是具有大量线程，所有线程都进行类似处理工作的程序。在这样的应用程序中，你不必关心唤醒的是哪个线程。

387

我们来使用保护块创建一个生产者－消费者（Producer-Consumer）应用程序。这种应用程序在两个线程之间共享数据：生产者（producer）创建数据，而消费者（consumer）对数据进行某些操作。两个线程共享对象进行通信。协调是至关重要的：在生产者线程交付数据之前，消费者线程绝对不能试图获得数据，并且如果消费者线程还没有获得旧的数据，那么生产者线程绝不能试图交付新的数据。

在这个示例中，数据是一系列文本消息，通过类型为Drop的对象[1]共享这些数据：

```
public class Drop {
  // Message sent from producer to consumer.
  private String message;
  // True if consumer should wait for producer to send message,
  // false if producer should wait for consumer
  // to retrieve message.
  private boolean empty = true;

  public synchronized String take() {
    // Wait until message is available.
    while (empty) {
      try {
        wait();
      } catch (InterruptedException e) {}
    }
    // Toggle status.
    empty = true;
    // Notify producer that status has changed.
    notifyAll();
    return message;
  }

  public synchronized void put(String message) {
    // Wait until message has been retrieved.
    while (!empty) {
      try {
        wait();
      } catch (InterruptedException e) {}
    }
    // Toggle status.
    empty = false;
    // Store message.
    this.message = message;
    // Notify consumer that status has changed.
    notifyAll();
  }
}
```

388

Producer[2]中定义的生产者线程发送一系列类似的消息。字符串“DONE”表示已经发送了所有消息。为了模拟现实应用程序不可预知的本质，生产者线程在两个消息之间暂停随机的

1. tutorial/essential/concurrency/example/Drop.java
2. tutorial/essential/concurrency/example/Producer.java

时间间隔：

```
import java.util.Random;

public class Producer implements Runnable {
  private Drop drop;

  public Producer(Drop drop) {
    this.drop = drop;
  }

  public void run() {
    String importantInfo[] = {
      "Mares eat oats",
      "Does eat oats",
      "Little lambs eat ivy",
      "A kid will eat ivy too"
    };
    Random random = new Random();

    for (int i = 0; i < importantInfo.length; i++) {
      drop.put(importantInfo[i]);
      try {
        Thread.sleep(random.nextInt(5000));
      } catch (InterruptedException e) {}
    }
    drop.put("DONE");
  }
}
```

Consumer[1]中定义的消费者线程简单地获得消息并且输出消息，直到获得字符串“DONE”为止。这个线程也暂停随机的时间间隔：

```
import java.util.Random;

public class Consumer implements Runnable {
  private Drop drop;

  public Consumer(Drop drop) {
    this.drop = drop;
  }

  public void run() {
    Random random = new Random();
    for (String message = drop.take();
                         ! message.equals("DONE");
                            message = drop.take()) {
      System.out.format("MESSAGE RECEIVED: %s%n", message);
      try {
        Thread.sleep(random.nextInt(5000));
      } catch (InterruptedException e) {}
    }
  }
}
```

最后是ProducerConsumerExample[2]中定义的主线程，它启动生产者和消费者线程：

```
public class ProducerConsumerExample {
  public static void main(String[] args) {
```

1. tutorial/essential/concurrency/example/Consumer.java
2. tutorial/essential/concurrency/example/ProducerConsumerExample.java

```
        Drop drop = new Drop();
        (new Thread(new Producer(drop))).start();
        (new Thread(new Consumer(drop))).start();
    }
}
```

注意　编写Drop类是为了演示保护块。为了避免从头开始，在试图编写自己的数据共享对象之前，请参考第11章中已有的数据结构。更多信息请参考“问题和练习：并发”一节。

390

12.6　不可变对象

如果一个对象在构造之后不能改变其状态，那么这个对象就被认为不可变（immutable）。最大程度地依靠不可变对象，被广泛地公认为创建简单可靠代码的合理策略。

不可变对象特别适用于并发应用程序。因为它们不能改变状态，所以线程干扰不能破坏它们，并且不会处于不一致性状态。

程序员常不愿使用不可变对象，因为他们担心，和更新现有对象相比，创建新对象的代价比较高。对象创建的影响经常被高估了，并且和不可变对象相关联的一些效率的提高可以抵消创建成本。这包括降低了垃圾收集带来的开销，以及消除了预防不可变对象被改变所需的代码。

下面几节使用一个类，其实例是可变的，并且可以从它派生具有不可变实例的类。通过这种方式，介绍这种转换的总体原则，并且演示不可变对象的一些优势。

12.6.1　同步类示例

SynchronizedRGB[1]类定义表示颜色的对象。每个对象把颜色表示为三个整数（它们代表主要颜色值）和一个表示颜色名称的字符串。

```
public class SynchronizedRGB {

    // Values must be between 0 and 255.
    private int red;
    private int green;
    private int blue;
    private String name;

    private void check(int red, int green, int blue) {
      if (red < 0 || red > 255
          || green < 0 || green > 255
              || blue < 0 || blue > 255) {
        throw new IllegalArgumentException();
      }
    }
    public SynchronizedRGB(int red, int green,
                           int blue, String name) {
      check(red, green, blue);
      this.red = red;
      this.green = green;
      this.blue = blue;
      this.name = name;
    }
```

391

1. tutorial/essential/concurrency/example/SynchronizedRGB.java

```
public void set(int red, int green, int blue, String name) {
  check(red, green, blue);
  synchronized (this) {
    this.red = red;
    this.green = green;
    this.blue = blue;
    this.name = name;
  }
}

public synchronized int getRGB() {
  return ((red << 16) | (green << 8) | blue);
}

public synchronized String getName() {
  return name;
}

public synchronized void invert() {
  red = 255 - red;
  green = 255 - green;
  blue = 255 - blue;
  name = "Inverse of " + name;
}
}
```

必须小心地使用SynchronizedRGB，以避免出现不一致状态。例如，假设一个线程执行如下代码：

392

```
SynchronizedRGB color = new SynchronizedRGB(0, 0, 0,
                                            "Pitch Black");
...
int myColorInt = color.getRGB();      // Statement 1
String myColorName = color.getName(); // Statement 2
```

如果另一个线程在Statement 1之后，但在Statement 2之前调用color.set，那么myColorInt的值就不会匹配myColorName的值。为了避免这种结果，这两个语句必须绑定到一起：

```
synchronized (color) {
  int myColorInt = color.getRGB();
  String myColorName = color.getName();
}
```

这种类型的不一致性只可能出现在可变对象中——SynchronizedRGB的不可变版本不会出现这种问题。

12.6.2 定义不可变对象的策略

下面的规则定义创建不可变对象的简单策略。并不是文档说明为“不可变”的所有类都遵守这些规则。这并不是说这些类的创建者不负责任——他们可能有合理的原因相信他们类的实例在构造之后永远不会改变。但是，这些策略需要经过复杂的分析，因此不适用于初学者。

(1) 不要提供“设置器”方法——修改字段或者字段引用的对象的方法。

(2) 将所有字段都声明为final或者private。

(3) 不允许子类覆盖的方法。完成这一工作的最简单方法是把类声明为final。更加复杂的方式是声明构造器为private，并且在工厂方法中构造实例。

(4) 如果实例字段包含对可变对象的引用，则不允许改动这些对象：

393

- 不提供修改可变对象的方法。

- 不共享对可变对象的引用。绝不存储对于传递给构造器的外部可变对象的引用；如果必要的话，创建副本，并且存储对副本的引用。类似地，在必要时创建内部可变对象的副本，以便避免在你的方法中返回原始对象。

在SynchronizedRGB中应用这些策略需要如下步骤：

(1) 这个类中有两个设置器方法。第一个是set，它任意地转换对象，在这个类的不可变版本中没有这个方法。第二个是invert，它创建新的对象，而不是修改已有对象，所以不可变版本可以包含它。

(2) 所有字段已经被声明为private；可以进一步把它们限定为final。

(3) 类本身声明为final。

(4) 只有一个字段引用对象，并且这个对象本身是不可变的。因此，不必设置保护措施来防止改变“被包含的”可变对象的状态。

作出这些改动之后，我们就得到了ImmutableRGB[1]：

```
final public class ImmutableRGB {

  // Values must be between 0 and 255.
  final private int red;
  final private int green;
  final private int blue;
  final private String name;

  private void check(int red, int green, int blue) {
    if (red < 0 || red > 255
        || green < 0 || green > 255
            || blue < 0 || blue > 255) {
      throw new IllegalArgumentException();
    }
  }

  public ImmutableRGB(int red, int green,
                      int blue, String name) {
    check(red, green, blue);
    this.red = red;
    this.green = green;
    this.blue = blue;
    this.name = name;
  }


  public int getRGB() {
    return ((red << 16) | (green << 8) | blue);
  }

  public String getName() {
    return name;
  }

  public ImmutableRGB invert() {
    return new ImmutableRGB(255 - red, 255 - green,
                            255 - blue, "Inverse of " + name);
  }
}
```

394

1. tutorial/essential/concurrency/example/ImmutableRGB.java

12.7 高级并发对象

到目前为止，本章重点介绍的是低级别API，它们曾经是Java平台最基本的组成部分。这些API足以胜任基本的任务，但是更加高级的任务需要更高级别的构造块。对于充分利用现代多处理器和多核心系统功能的大规模并发应用程序来说，这尤其重要。

在这一小节中，我们将了解Java平台5.0版本引入的高级并发特性。大多数这些特性在java.util.concurrent包中实现。在Java集合框架（Java Collections Framework）中也有新的并发数据结构。

12.7.1 Lock对象

同步代码依靠简单类型的可重入锁。这种类型的锁易于使用，但是有很多局限性。java.util.concurrent.locks包[1]支持更加复杂的锁定语法。我们在这里不详细分析这个包，而是重点介绍其最基本的接口Lock[2]。

Lock对象的工作方式和同步代码使用的隐式锁非常类似。和隐式锁一样，每一时刻只有一个线程能够拥有Lock对象。通过与其相关联的Condition对象[3]，Lock对象也支持wait / notify机制。

和隐式锁相比，Lock对象的最大优势在于能够阻挡获得锁的企图。如果锁不能立即可用或者在超时时间到期之前可用（如果指定了超时的话），tryLock方法就会进行阻挡。如果另一个线程在获得锁之前发送中断，lockInterruptibly方法就会阻挡。

我们来使用Lock对象解决12.4节中提到的死锁问题。Alphonse和Gaston训练他们自己注意，一个朋友将要鞠躬了。我们通过这种方式改进模型：要求我们的Friend对象在鞠躬之前必须获得两个参与者的锁，下面是改进后的模型Safelock[4]的源代码。为了演示这种语法的通用性，我们假设Alphonse和Gaston没有认识清楚他们新得到的安全鞠躬的能力，以至于还在不停地相互鞠躬：

395

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.Random;

public class Safelock {
  static class Friend {
    private final String name;
    private final Lock lock = new ReentrantLock();

    public Friend(String name) {
      this.name = name;
    }

    public String getName() {
      return this.name;
    }

    public boolean impendingBow(Friend bower) {
      Boolean myLock = false;
```

1. docs/api/java/util/concurrent/locks/package-summary.html
2. docs/api/java/util/concurrent/locks/Lock.html
3. docs/api/java/util/concurrent/locks/Condition.html
4. tutorial/essential/concurrency/example/Safelock.java

```
    Boolean yourLock = false;
    try {
      myLock = lock.tryLock();
      yourLock = bower.lock.tryLock();
    } finally {
      if (! (myLock && yourLock)) {
        if (myLock) {
          lock.unlock();
        }
        if (yourLock) {
          bower.lock.unlock();
        }
      }
      return myLock && yourLock;
    }

    public void bow(Friend bower) {
      if (impendingBow(bower)) {
        try {
          System.out.format("%s: %s has bowed to me!%n",
                            this.name, bower.getName());
          bower.bowBack(this);
        } finally {
          lock.unlock();
          bower.lock.unlock();
        }
      } else {
          System.out.format("%s: %s started to bow " +
            "to me, but saw that I was already " +
            "bowing to him.%n", this.name, bower.getName());
        }
      }

      public void bowBack(Friend bower) {
        System.out.format("%s: %s has bowed back to me!%n",
                          this.name, bower.getName());
      }
    }

    static class BowLoop implements Runnable {
      private Friend bower;
      private Friend bowee;

      public BowLoop(Friend bower, Friend bowee) {
        this.bower = bower;
        this.bowee = bowee;
      }

      public void run() {
        Random random = new Random();
        for (;;) {
          try {
              Thread.sleep(random.nextInt(10));
          } catch (InterruptedException e) {}
          bowee.bow(bower);
        }
      }
    }

    public static void main(String[] args) {
```

396

```
        final Friend alphonse = new Friend("Alphonse");
        final Friend gaston = new Friend("Gaston");
        new Thread(new BowLoop(alphonse, gaston)).start();
        new Thread(new BowLoop(gaston, alphonse)).start();
    }
}
```

12.7.2 执行器

在前面的例子中，新的线程（由其Runnable对象定义）完成的任务和线程本身（由Thread对象定义）之间紧密连接。这适用于小型应用程序，但是在大型应用程序中，把线程管理和创建工作与应用程序的其余部分分离开更有意义。封装这些功能的对象被称为执行器（executor）。

1. 执行器接口

java.util.concurrent包定义3个执行器接口：

- Executor，一个简单接口，它支持启动新任务。
- ExecutorService，Executor的子接口，它添加了帮助管理独立任务和执行器本身的生存周期的特性。
- ScheduledExecutorService，ExecutorService的子接口，支持未来和/或周期性的任务执行。

典型情况下，引用执行器对象的变量被声明为这3种接口类型之一，而不是执行器类类型。

(1) Executor接口

Executor接口[1]提供单一方法execute，设计它的目的是替换常见的线程创建语法。如果r是Runnable对象，而e是Executor对象，就可以把

```
(new Thread(r)).start();
```

替换为

```
e.execute(r);
```

但是，execute的定义不那么特定。低级别语法创建新的线程并且立即启动它。根据Executor的实现，execute可能完成相同操作，但是更有可能使用现有的工作者线程运行r，或者把r放入队列等待线程变为可用。（我们将在12.7.2节的第2小节中讨论工作者线程。）

java.util.concurrent中执行器实现的设计目的是充分利用更加高级的ExecutorService和ScheduledExecutorService接口，但是它们也可以处理基本的Executor接口。

(2) ExecutorService接口

ExecutorService接口[2]通过类似的，但是更加通用的submit方法补充execute。和execute一样，submit接受Runnable对象，但是也接受Callable对象[3]，它允许任务返回一个值。submit方法返回一个Future对象[4]，它用于获得Callable返回值并且管理Callable和Runnable任务的状态。

ExecutorService还提供用于提交Callable对象的大型集合的方法。最后，ExecutorService提供很多方法用于管理执行器的关闭。为了支持立即关闭，任务应该能够正确地处理中断（参见12.2.3节）。

397

398

1. docs/api/java/util/concurrent/Executor.html
2. docs/api/java/util/concurrent/ExecutorService.html
3. docs/api/java/util/concurrent/Callable.html
4. docs/api/java/util/concurrent/Future.html

(3) ScheduledExecutorService接口

ScheduledExecutorService接口[1]通过schedule方法补充其父接口ExecutorService，schedule方法在指定的延迟之后执行Runnable或者Callable任务。另外，这个接口定义scheduleAtFixedRate和scheduleWithFixedDelay，它们按照定义的时间间隔重复地执行指定的任务。

2. 线程池

java.util.concurrent中的大多数执行器实现使用线程池（thread pool），线程池由工作者线程（worker thread）构成。这种类型的线程独立于它执行的Runnable和Callable任务存在，并且经常用于执行多个任务。

使用工作者线程使创建线程造成的开销最小化。线程对象使用极大量的内存，而且在大型应用程序中，分配和释放很多线程对象会造成巨大的内存管理开销。

线程池的一种常见类型是固定线程池（fixed thread pool）。这种类型的池总包含数量固定的正在运行的线程；如果仍在使用的一个线程由于某种原因终止了，就自动使用新的线程替换它。通过内部队列把任务提交到池，只要活动的任务数量超过了线程数量，内部队列就保存额外任务。

固定线程池的一个重要优势在于，使用它的应用程序会适度地降级（degrade gracefully）。为了了解这一点，考虑一个Web服务器应用程序，其中每个HTTP请求由单独线程处理。如果应用程序简单地为每个新的HTTP请求创建新的线程，并且系统接收到的请求超过它能够立即处理的数量，当这些线程的开销超出系统容量时，应用程序就会突然停止对所有请求的响应。通过限制可以创建线程的数量，应用程序不会在接收到HTTP请求时立即处理它们，但是会在系统能够支持的情况下尽快处理它们。

创建使用固定线程池的执行器的一个简单方式是，调用java.util.concurrent.Executors[2]中的newFixedThreadPool工厂方法。这个类还提供下面的工厂方法：

- newCachedThreadPool方法使用可扩展的线程池创建执行器。这个执行器适用于启动很多生存周期短暂的任务的应用程序。

399

- newSingleThreadExecutor方法创建每次只执行单一任务的执行器。
- 若干工厂方法是上述执行器的ScheduledExecutorService版本。

如果上述工厂方法提供的执行器都不能满足你的要求，可以构造java.util.concurrent.ThreadPoolExecutor[3]或者java.util.concurrent.ScheduledThreadPoolExecutor[4]的实例，它们将提供额外的选择。

12.7.3 并发集合

java.util.concurrent包括Java集合框架（Java Collections Framework）的很多补充。下面是按照提供的集合接口最容易分类的一些：

- BlockingQueue[5]定义先进先出数据结构，当试图向排满的队列进行添加，或者从空队列取出时，它会阻塞或者超时。

---

1. docs/api/java/util/concurrent/ScheduledExecutorService.html
2. docs/api/java/util/concurrent/Executors.html
3. docs/api/java/util/concurrent/ThreadPoolExecutor.html
4. docs/api/java/util/concurrent/ScheduledThreadPoolExecutor.html
5. docs/api/java/util/concurrent/BlockingQueue.html

- ConcurrentMap[1]是java.util.Map[2]的子接口，它定义有用的原子操作，这些操作只在键存在时才删除或者替换键一值对，或者只在键不存在时才添加键一值对。使这些操作原子化可以帮助避免同步。ConcurrentMap的标准通用实现是ConcurrentHashMap[3]，它是HashMap[4]的并发版本。
- ConcurrentNavigableMap[5]是ConcurrentMap的子接口，它支持近似匹配。ConcurrentNavigableMap的标准通用实现是ConcurrentSkipListMap[6]，它是TreeMap[7]的并发版本。

400

通过在把对象添加到集合的操作和访问或者删除这个对象的后续操作之间建立发生前关系，所有这些接口都帮助避免出现内存一致性错误（参见12.3.2节）。

12.7.4 原子变量

java.util.concurrent.atomic[8]包定义支持对单一变量执行原子操作的类。所有类都具有get和set方法，它们的工作方式和对volatile变量的读取和写入一样。就是说，set和对同一个变量的后续get具有发生前关系。原子方法compareAndSet也具有内存一致性特性，就像应用于整数原子变量的简单原子算术方法一样。

为了了解如何使用这个包，我们回顾一下最初用于演示线程接口的Counter类：

```
class Counter {
  private int c = 0;
  public void increment() {
    c++;
  }
  public void decrement() {
    c--;
  }
  public int value() {
    return c;
  }
}
```

使Counter避免线程干扰的一种方式是使其方法同步，如SynchronizedCounter所示：

```
class SynchronizedCounter {
  private int c = 0;
  public synchronized void increment() {
    c++;
  }
  public synchronized void decrement() {
    c--;
  }
  public synchronized int value() {
    return c;
  }
}
```

401

1. docs/api/java/util/concurrent/ConcurrentMap.html
2. docs/api/java/util/Map.html
3. docs/api/java/util/concurrent/ConcurrentHashMap.html
4. docs/api/java/util/HashMap.html
5. docs/api/java/util/concurrent/ConcurrentNavigableMap.html
6. docs/api/java/util/concurrent/ConcurrentSkipListMap.html
7. docs/api/java/util/TreeMap.html
8. docs/api/java/util/concurrent/atomic/package-summary.html

对于这个简单的类，同步是可接受的解决方案。但是对于更加复杂的类，我们可能希望避免同步对活性不必要的影响。把int字段替换为AtomicInteger使我们能够防止线程干扰，且不必借助同步手段，如AtomicCounter[1]所示：

```
import java.util.concurrent.atomic.AtomicInteger;

class AtomicCounter {
  private AtomicInteger c = new AtomicInteger(0);

  public void increment() {
    c.incrementAndGet();
  }

  public void decrement() {
    c.decrementAndGet();
  }

  public int value() {
    return c.get();
  }

}
```

12.8 推荐阅读

- *Concurrent Programming in Java: Design Principles and Pattern （2nd Edition）*，作者Doug Lea。由一流专家编写的综合性资料，作者也是Java平台的并发框架的架构设计者。
- *Java Concurrency in Practice*，作者是Brian Göetz、Tim Peierls、Joshua Bloch、Joseph Bowbeer、David Holmes和Doug Lea。为初学者编写的使用指南。
- *Effective Java Programming Language Guide*，作者Joshua Bloch。虽然这是程序设计的一般性指南，但是书中关于线程的章节包括用于并发程序设计的必备“最佳实践”。
- *Concurrency: State Models & Java Programs （2nd Edition）*，作者是Jeff Magee和Jeff Kramer。通过建模和实用示例介绍了并发程序设计。

402

问题和练习：并发

问题

可以把Thread对象传递给Executor.execute吗？这样的调用有意义吗？

练习

1. 编译和运行BadThreads.java[2]：

```
public class BadThreads {

  static String message;

  private static class CorrectorThread extends Thread {

    public void run() {
      try {
```

1. tutorial/essential/concurrency/example/AtomicCounter.java
2. tutorial/essential/concurrency/QandE/BadThreads.java

```
            sleep(1000);
        } catch (InterruptedException e) {}
        // Key statement 1:
        message = "Mares do eat oats.";
    }
}

public static void main(String args[])
                        throws InterruptedException {
    (new CorrectorThread()).start();
    message = "Mares do not eat oats.";
    Thread.sleep(2000);
    // Key statement 2:
    System.out.println(message);
  }
}
```

这个应用程序应该输出“Mares do eat oats.”。它能确保总是这样工作吗？如果不行，为什么？改变两次调用Sleep的参数能有帮助吗？描述改变程序保证程序正常工作的两种方式。 403

2. 修改12.5节中的生产者－消费者的例子，使用标准库类替换Drop类。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/essential/concurrency/QandE/answers.html 404

第13章

正则表达式

本章介绍如何使用java.util.regex API[1]与正则表达式进行模式匹配。虽然这个包接受的语法和Perl编程语言[2]类似，但是并不需要事先了解Perl。本章从基础讲起，然后逐步深入讲解更加高级的技术。

13.1 简介

13.1.1 正则表达式是什么

正则表达式（regular expression）是根据字符串集合内每个字符串共享的共同特性来描述字符串集合的一种途径。正则表达式可以用于搜索、编辑或者处理文本和数据。必须了解创建正则表达式的特定语法——这超出了Java编程语言的一般语法。正则表达式的复杂性各不相同。但是了解了如何构造正则表达式的基础之后，就能够解释（或者创建）任何正则表达式。

本章讲解java.util.regex API支持的正则表达式语法，并且提供若干实例以便演示各种对象如何交互。在正则表达式的领域中，有很多形式可供选择，比如grep、Perl、Tcl、Python、PHP和awk。java.util.regex API中的正则表达式语法和Perl最为类似。

13.1.2 这个包如何表示正则表达式

java.util.regex包主要由三部分构成：Pattern[3]、Matcher[4]和PatternSyntaxException[5]。

- Pattern对象是正则表达式编译后的表达形式。Pattern类没有提供公共构造器。为了创建模式，首先必须调用它的一个public static compile方法，这样会返回一个Pattern对象。这些方法接受正则表达式作为第一个实参；本章下面几页将讲解所需的语法。
- Matcher对象是解释模式和对输入字符串执行匹配操作的引擎。和Pattern类一样，Matcher没有定义公共构造器。通过调用Pattern对象的matcher方法获得Matcher对象。
- PatternSyntaxException对象是不可控异常，它指出正则表达式模式中的语法错误。

本章的最后几节将详细讲解上述每个类。但是首先必须了解如何构造正则表达式。因此，下一小节介绍测试示例，以后将重复地使用它来介绍语法。

---

1. docs/api/java/util/regex/package-summary.html
2. http://www.perl.com
3. docs/api/java/util/regex/Pattern.html
4. docs/api/java/util/regex/Matcher.html
5. docs/api/java/util/regex/PatternSyntaxException.html

13.2 测试示例

本节定义一个可重用的测试示例RegexTestHarness.java[1]，用于讲解这个API支持的正则表达式结构。运行这段代码的命令是java RegexTestHarness；不接受命令行参数。这个应用程序重复地循环，提示用户输入正则表达式和输入字符串。使用这个测试示例是可选的，但是你会发现使用它分析后面章节讨论的测试案例是很方便的。 406

```
import java.io.Console;
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class RegexTestHarness {

  public static void main(String[] args){
    Console console = System.console();
    if (console == null) {
      System.err.println("No console.");
      System.exit(1);
    }
    while (true) {

      Pattern pattern =
      Pattern.compile(console.readLine("%nEnter your " +
                                       regex: "));

      Matcher matcher =
      pattern.matcher(console.readLine("Enter input string " +
                                       to search: "));

      boolean found = false;
      while (matcher.find()) {
        console.format("I found the text \"%s\" starting " +
                       "at index %d and ending at " +
                       "index %d.%n", matcher.group(),
                       matcher.start(), matcher.end());
        found = true;
      }
      if(!found){
        console.format("No match found.%n");
      }
    }
  }
}
```

在继续下一小节之前，请首先保存和编译这段代码，以便确保开发环境支持所需的包。

13.3 字符串字面量

这个API提供的模式匹配的最基本形式是字符串字面量的匹配。例如，如果正则表达式为foo，输入字符串为foo，那么匹配结果就成功，因为两个字符串相等。试一试下面的测试示例： 407

```
Enter your regex: foo
Enter input string to search: foo
I found the text "foo" starting at index 0 and ending at index 3.
```

1. tutorial/essential/regex/examples/RegexTestHarness.java

这个匹配成功。注意，当输入字符串包含3个字符时，开始索引为0，结束索引为3。按照约定，范围包含开始索引，而不包含结束索引，如图13-1所示。

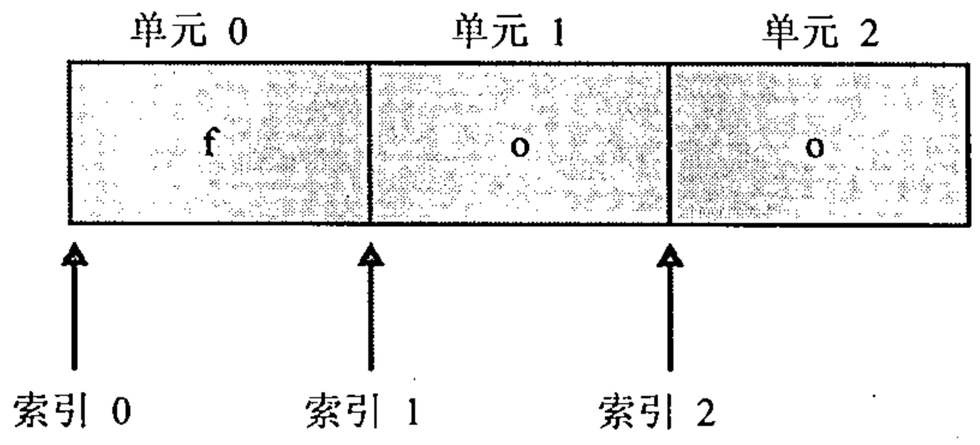


图13-1　String字面量“foo”，标出了编号的单元和索引值

字符串中的每个字符都在自己的单元（cell）中，索引位置指向每个单元之间。字符串“foo”从索引0开始，到索引3结束，尽管字符本身只占用单元0、1和2。

在后续的匹配中，你会注意到一些重叠；下一个匹配的开始索引和前一个匹配的结束索引相同：

```
Enter your regex: foo
Enter input string to search: foofoofoo
I found the text "foo" starting at index 0 and ending at index 3.
I found the text "foo" starting at index 3 and ending at index 6.
I found the text "foo" starting at index 6 and ending at index 9.
```

元字符

这个API还支持很多影响匹配模式的方式的特殊字符。把正则表达式改为cat.，输入字符串改为cats。输出如下：

```
Enter your regex: cat.
Enter input string to search: cats
I found the text "cats" starting at index 0 and ending at index 4.
```

这个匹配仍然成功，尽管点号（.）没有出现在输入字符串中。匹配成功的原因在于点号是元字符（metacharacter）——被匹配器解释为具有特殊含义的字符。元字符“.”的含义是“任
408 何一个字符”，这就是这个例子中匹配成功的原因。

这个API支持的元字符有：([{ \ ^ - $ | }]) ? * +.。

注意　在某些情况下，前面列出的特殊字符不被当作元字符对待。随着你更多地学习如何构造正则表达式，就会遇到这种情况。但是，你可以使用这个清单检查一个特殊字符是否被当作元字符。例如，字符!、@和#永远都不会具有特殊含义。

有两种方式可以强制元字符作为普通字符：

- 在元字符前面加上反斜线，或者
- 用\Q（开始引用）和\E（结束引用）把元字符括起来。

使用这种技术时，\Q和\E可以放在表达式内的任何位置，前提是\Q先出现。

13.4　字符类

如果浏览Pattern类规范会看到总结支持正则表达式结构的表。表13-1描述字符类。

左边一列指定正则表达式结构，右边一列描述每个结构在什么情况下匹配。

表13-1 字符类

[abc]	a、b或者c（简单类）
[^abc]	除a、b或者c之外的任何字符（非）
[a-zA-Z]	a到z，或者A到Z（包含）（范围）
[a-d[m-p]]	a到d，或者m到p：[a-dm-p]（并）
[a-z&&[def]]	d、e或者f（交）
[a-z&&[^bc]]	a到z，除b和c之外：[ad-z]（减）
[a-z&&[^m-p]]	a到z，不包括m到p：[a-lq-z]（减）

注意 短语“字符类”中“类”这个词不表示.class文件。在正则表达式的上下文表述中，字符类是括在方括号内的字符集合。它表示这些字符将和给定输入字符串内的单一字符成功匹配。

409

简单类

字符类最基本的形式是方括号中简单并排放置的字符集合。例如，正则表达式[bcr]at将和单词“bat”、“cat”或者“rat”匹配，因为它定义一个字符类（接受“b”、“c”或者“r”）作为其第一个字符：

```
Enter your regex: [bcr]at
Enter input string to search: bat
I found the text "bat" starting at index 0 and ending at index 3.

Enter your regex: [bcr]at
Enter input string to search: cat
I found the text "cat" starting at index 0 and ending at index 3.

Enter your regex: [bcr]at
Enter input string to search: rat
I found the text "rat" starting at index 0 and ending at index 3.

Enter your regex: [bcr]at
Enter input string to search: hat
No match found.
```

在上面的例子中，只有当第一个字符和字符类定义的字符之一匹配时，整体匹配才成功。

1. 非

为了匹配列出的字符之外的所有字符，需要在字符类的开头插入“^”。这种技术被称为非（negation）：

```
Enter your regex: [^bcr]at
Enter input string to search: bat
No match found.

Enter your regex: [^bcr]at
Enter input string to search: cat
No match found.

Enter your regex: [^bcr]at
Enter input string to search: rat
No match found.
```

```
Enter your regex: [^bcr]at
Enter input string to search: hat
I found the text "hat" starting at index 0 and ending at index 3.
```

410 只有当输入字符串的第一个字符不包含字符类中定义的任何字符时，匹配才成功。

2. 范围

有时候，你会希望定义一个字符类包含一个范围内的值，比如字母“a”到“h”或者数字“1”到“5”。为了指定范围，只需在要匹配的第一个和最后一个字符之间插入“-”即可，比如[1-5]或者[a-h]。也可以在类中连着放置不同范围，以便进一步扩展匹配的可能性。例如[a-zA-Z]将匹配字母表中的任何字母：a到z（小写）或者A到Z（大写）。

下面是范围和非的一些例子：

```
Enter your regex: [a-c]
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.

Enter your regex: [a-c]
Enter input string to search: b
I found the text "b" starting at index 0 and ending at index 1.

Enter your regex: [a-c]
Enter input string to search: c
I found the text "c" starting at index 0 and ending at index 1.

Enter your regex: [a-c]
Enter input string to search: d
No match found.
```

```
Enter your regex: foo[1-5]
Enter input string to search: foo1
I found the text "foo1" starting at index 0 and ending at index 4.

Enter your regex: foo[1-5]
Enter input string to search: foo5
I found the text "foo5" starting at index 0 and ending at index 4.

Enter your regex: foo[1-5]
Enter input string to search: foo6
No match found.

Enter your regex: foo[^1-5]
Enter input string to search: foo1
No match found.

Enter your regex: foo[^1-5]
Enter input string to search: foo6
I found the text "foo6" starting at index 0 and ending at index 4.
```

3. 并

也可以使用并（union）创建由两个或者多个独立字符类构成的单一字符类。为了创建并，
411 只需在一个类中嵌套另一个类，比如[0-4[6-8]]。这个并创建的单一字符类匹配数字0、1、2、3、4、6、7和8。

```
Enter your regex: [0-4[6-8]]
Enter input string to search: 0
I found the text "0" starting at index 0 and ending at index 1.

Enter your regex: [0-4[6-8]]
Enter input string to search: 5
```

```
No match found.

Enter your regex: [0-4[6-8]]
Enter input string to search: 6
I found the text "6" starting at index 0 and ending at index 1.

Enter your regex: [0-4[6-8]]
Enter input string to search: 8
I found the text "8" starting at index 0 and ending at index 1.

Enter your regex: [0-4[6-8]]
Enter input string to search: 9
No match found.
```

4. 交

为了创建只和其所有嵌套类共有的字符匹配的单一字符类，需要使用&&，比如[0-9&&[345]]。这个交创建只和两个字符类共有的数字（3、4和5）匹配的单一字符类：

```
Enter your regex: [0-9&&[345]]
Enter input string to search: 3
I found the text "3" starting at index 0 and ending at index 1.

Enter your regex: [0-9&&[345]]
Enter input string to search: 4
I found the text "4" starting at index 0 and ending at index 1.

Enter your regex: [0-9&&[345]]
Enter input string to search: 5
I found the text "5" starting at index 0 and ending at index 1.

Enter your regex: [0-9&&[345]]
Enter input string to search: 2
No match found.

Enter your regex: [0-9&&[345]]
Enter input string to search: 6
No match found.
```

下面的例子显示两个范围的交：

412

```
Enter your regex: [2-8&&[4-6]]
Enter input string to search: 3
No match found.

Enter your regex: [2-8&&[4-6]]
Enter input string to search: 4
I found the text "4" starting at index 0 and ending at index 1.

Enter your regex: [2-8&&[4-6]]
Enter input string to search: 5
I found the text "5" starting at index 0 and ending at index 1.

Enter your regex: [2-8&&[4-6]]
Enter input string to search: 6
I found the text "6" starting at index 0 and ending at index 1.

Enter your regex: [2-8&&[4-6]]
Enter input string to search: 7
No match found.
```

5. 减

最后，可以使用减（subtraction）去掉一个或者多个嵌套字符类，比如[0-9&&[^345]]。

这个例子创建从0到9的所有值，但除3、4和5之外的单一字符类：

```
Enter your regex: [0-9&&[^345]]
Enter input string to search: 2
I found the text "2" starting at index 0 and ending at index 1.

Enter your regex: [0-9&&[^345]]
Enter input string to search: 3
No match found.

Enter your regex: [0-9&&[^345]]
Enter input string to search: 4
No match found.

Enter your regex: [0-9&&[^345]]
Enter input string to search: 5
No match found.

Enter your regex: [0-9&&[^345]]
Enter input string to search: 6
I found the text "6" starting at index 0 and ending at index 1.

Enter your regex: [0-9&&[^345]]
Enter input string to search: 9
I found the text "9" starting at index 0 and ending at index 1.
```

413 现在我们介绍了如何创建字符类，在阅读下一小节之前你可能希望回顾一下表13-1。

13.5 预定义字符类

Pattern API包含很多有用的预定义字符类（predefined character class），它们提供常用正则表达式便利的简写方式。

在表13-2中，左边一列的每个结构是右边一列的字符类的简写方式。例如，\d表示数字范围（0-9），而\w表示单词字符（任何小写字母、任何大写字母、下划线或者任何数字）。应该尽可能使用预定义类。它们使你的代码更容易阅读，并且排除易混淆的字符类造成的错误。

表13-2 预定义字符类

	任何字符（可能匹配，也可能不匹配行终止符）
\d	数字：[0-9]
\D	非数字：[^0-9]
\s	空白字符：[\t\n\x0B\f\r]
\S	非空白字符：[^\s]
\w	单词字符：[a-zA-Z_0-9]
\W	非单词字符：[^\w]

以反斜线开头的结构被称为转义结构（escaped construct）。我们在3.1.2节的第2小节简单介绍了转义结构，其中提到了用于引用的反斜线、\Q和\E。如果你在字符串字面量中使用转义结构，就必须在反斜线前面再加上一个反斜线，以便能够编译字符串。例如：

```
private final String REGEX = "\\d"; // a single digit
```

在这个例子中，\d是正则表达式；附加的反斜线是编译代码所必须的。但是，测试示例直接从控制台读取表达式，所以附加的反斜线不是必须的。

下面的例子演示预定义字符类的使用：

```
Enter your regex: .
Enter input string to search: @
I found the text "@" starting at index 0 and ending at index 1.

Enter your regex: .
Enter input string to search: 1
I found the text "1" starting at index 0 and ending at index 1.
Enter your regex: .
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.

Enter your regex: \d
Enter input string to search: 1
I found the text "1" starting at index 0 and ending at index 1.

Enter your regex: \d
Enter input string to search: a
No match found.

Enter your regex: \D
Enter input string to search: 1
No match found.

Enter your regex: \D
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.

Enter your regex: \s
Enter input string to search:
I found the text " " starting at index 0 and ending at index 1.

Enter your regex: \s
Enter input string to search: a
No match found.

Enter your regex: \S
Enter input string to search:
No match found.

Enter your regex: \S
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.

Enter your regex: \w
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.

Enter your regex: \w
Enter input string to search: !
No match found.

Enter your regex: \W
Enter input string to search: a
No match found.

Enter your regex: \W
Enter input string to search: !
I found the text "!" starting at index 0 and ending at index 1.
```

414

415

在前三个例子中，正则表达式是简单的.（“点号”元字符表示“一个任何字符”）。因此，前三个例子都匹配成功（其中随机地使用一个@字符、一个数字和一个字母）。其余的例子分别使用表13-2中的正则表达式结构。可以参考这个表来分析每个匹配背后的逻辑：

- \d匹配所有数字。
- \s匹配空白。
- \w匹配单词字符。

另一种方式是使用大写字母表示相反含义：

- \D匹配非数字。
- \S匹配非空白。
- \W匹配非单词字符。

13.6　量词

量词（quantifier）允许你指定要匹配的出现次数。为了方便起见，Pattern API规范的三个部分描述greedy、reluctant和possessive量词，如表13-3所示。乍看上去，你可能认为量词X?、X??和X?+的功能完全一样，因为它们都匹配“X，一次或者完全没有”。在本节快结束时将解释它们实现的微妙区别。

表13-3　量　词

量　词			含　义
greedy	reluctant	possessive	
X?	X??	X?+	X，一次或者完全没有
X*	X*?	X*+	X，零次或者多次
X+	X+?	X++	X，一次或者多次
X{n}	X{n}?	X{n}+	X，正好n次
X{n,}	X{n,}?	X{n,}+	X，至少n次
X{n,m}	X{n,m}?	X{n,m}+	X，至少n次，但是不超过m次

通过创建3个不同的正则表达式，我们从greedy量词开始分析：字母a后面是?、*或者+。

416 我们来看一看把这些表达式和空白输入字符串""进行测试会发生什么情况：

```
Enter your regex: a?
Enter input string to search:
I found the text "" starting at index 0 and ending at index 0.

Enter your regex: a*
Enter input string to search:
I found the text "" starting at index 0 and ending at index 0.

Enter your regex: a+
Enter input string to search:
No match found.
```

13.6.1　零长度匹配

在前面的例子中，前两个匹配成功，因为表达式a?和a*都允许字母a的出现次数为0。你还会注意到，开始和结束索引都为0，这和我们到目前为止见过的任何例子都不同。空白输入字符串“”没有长度，所以测试简单地和位于索引0的“无内容”匹配。这种类型的匹配被称为零长

度匹配（zero-length match）。零长度匹配可能发生在这样几种情况下：在空白输入字符串中、在输入字符串的开头、在输入字符串的最后一个字符之后或者在输入字符串的任何两个字符之间。零长度匹配很容易识别，因为它们总在同一个索引位置开始和结束。

我们再通过几个例子分析零长度匹配。把输入字符串改为单一字母“a”，你会注意到一些有趣的事情：

```
Enter your regex: a?
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.
I found the text "" starting at index 1 and ending at index 1.

Enter your regex: a*
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.
I found the text "" starting at index 1 and ending at index 1.

Enter your regex: a+
Enter input string to search: a
I found the text "a" starting at index 0 and ending at index 1.
```

字母“a”找到了3个量词，但是前两个例子还找到了索引1位置的零长度匹配；就是在输入字符串最后会一个字符之后的位置。记住，匹配器发现字符“a”位于索引0和索引1之间的单元中，并且我们的测试示例会一直循环到再也找不到匹配为止。根据使用的量词，位于最后一个字符之后的索引位置的“无内容”可能触发匹配，也可能不触发匹配。 417

现在把输入字符串改为一行中有5个字母“a”，你会得到如下结果：

```
Enter your regex: a?
Enter input string to search: aaaaa
I found the text "a" starting at index 0 and ending at index 1.
I found the text "a" starting at index 1 and ending at index 2.
I found the text "a" starting at index 2 and ending at index 3.
I found the text "a" starting at index 3 and ending at index 4.
I found the text "a" starting at index 4 and ending at index 5.
I found the text "" starting at index 5 and ending at index 5.

Enter your regex: a*
Enter input string to search: aaaaa
I found the text "aaaaa" starting at index 0 and ending at index 5.
I found the text "" starting at index 5 and ending at index 5.

Enter your regex: a+
Enter input string to search: aaaaa
I found the text "aaaaa" starting at index 0 and ending at index 5.
```

表达式a？寻找每个字符的单独匹配，因为它匹配“a”出现零次或者一次的情况。表达式a*找到两个单独匹配：第一个匹配字母“a”的所有出现，然后是最后一个字符之后位于索引5位置的零长度匹配。最后，a＋匹配字母“a”的所有出现，忽略位于最后索引位置出现的“无内容”。

讲到这里，你可能想知道，如果前两个量词遇到不是“a”的字母会得到什么结果。例如，如果遇到字母“b”（比如“ababaaaab”），会怎么样？

我们来看一下：

```
Enter your regex: a?
Enter input string to search: ababaaaab
I found the text "a" starting at index 0 and ending at index 1.
```

```
I found the text "" starting at index 1 and ending at index 1.
I found the text "a" starting at index 2 and ending at index 3.
I found the text "" starting at index 3 and ending at index 3.
I found the text "a" starting at index 4 and ending at index 5.
I found the text "a" starting at index 5 and ending at index 6.
I found the text "a" starting at index 6 and ending at index 7.
I found the text "a" starting at index 7 and ending at index 8.
I found the text "" starting at index 8 and ending at index 8.
I found the text "" starting at index 9 and ending at index 9.

Enter your regex: a*
Enter input string to search: abababaaaab
I found the text "a" starting at index 0 and ending at index 1.
I found the text "" starting at index 1 and ending at index 1.
I found the text "a" starting at index 2 and ending at index 3.
I found the text "" starting at index 3 and ending at index 3.

I found the text "aaaa" starting at index 4 and ending at index 8.
I found the text "" starting at index 8 and ending at index 8.
I found the text "" starting at index 9 and ending at index 9.

Enter your regex: a+
Enter input string to search: abababaaaab
I found the text "a" starting at index 0 and ending at index 1.
I found the text "a" starting at index 2 and ending at index 3.
I found the text "aaaa" starting at index 4 and ending at index 8.
```

418

尽管字母“b”出现在单元1、3和8中，输出也报告了这些位置的零长度匹配。正则表达式a? 并不专门搜索字母“b”；它仅仅搜索字母“a”的出现（或者不出现）。如果量词允许匹配“a”零次，那么输入字符串中不是“a”的任何内容的结果都是零长度匹配。按照前面例子中讨论的规则匹配剩余的a。

为了匹配正好出现n次的模式，只需在花括号中指定次数：

```
Enter your regex: a{3}
Enter input string to search: aa
No match found.

Enter your regex: a{3}
Enter input string to search: aaa
I found the text "aaa" starting at index 0 and ending at index 3.

Enter your regex: a{3}
Enter input string to search: aaaa
I found the text "aaa" starting at index 0 and ending at index 3.
```

这里，正则表达式a{3}搜索一行中字母“a”的3个出现。第一个测试失败，因为输入字符串中a出现的次数不够匹配要求。第二个测试的输入字符串中正好包含3个a，这就触发了匹配。第三个测试也触发匹配，因为在输入字符串的开头正好有3个a。后面跟着的任何内容都和第一个匹配不相关。如果在这个位置之后再次出现模式，就会触发后续的匹配：

```
Enter your regex: a{3}
Enter input string to search: aaaaaaaaa
I found the text "aaa" starting at index 0 and ending at index 3.
I found the text "aaa" starting at index 3 and ending at index 6.
I found the text "aaa" starting at index 6 and ending at index 9.
```

为了要求模式至少出现n次，需要在数字后面加上逗号：

```
Enter your regex: a{3,}
```

```
Enter input string to search: aaaaaaaaa
I found the text "aaaaaaaaa" starting at index 0 and ending at index 9.
```

419

对于相同的输入字符串，这个测试只找到一个匹配，因为一行中的9个a满足“至少”3个a的要求。

最后，为了指定出现次数的上限，需要在括号中加上第二个数字：

```
Enter your regex: a{3,6}
        // find at least 3 (but no more than 6) a's in a row
Enter input string to search: aaaaaaaaa
I found the text "aaaaaa" starting at index 0 and ending at index 6.
I found the text "aaa" starting at index 6 and ending at index 9.
```

这里，按照6个字符的上限强制停止第一个匹配。第二个匹配包含剩下的任何内容，正好是3个a——匹配允许的最小字符数量。如果输入字符串中少一个字符，就没有第二个匹配，因为只剩下了2个a。

13.6.2 使用量词的捕获组和字符类

到目前为止，我们只对包含一个字符的输入字符串测试了量词。实际上，一次只能把量词附加到一个字符之后，所以正则表达式abc+的含义是“a，后面是b，再后面是c出现一次或者多次”。它的含义不是“abc”一次或者多次。但是，也可以把量词附加到字符类（参见13.4节）和捕获组（见13.7节），比如[abc]+（a或b或c，一次或者多次），或者(abc)+（组“abc”，一次或者多次）。

下面我们演示，指定组(dog)在一行中出现三次：

```
Enter your regex: (dog){3}
Enter input string to search: dogdogdogdogdogdog
I found the text "dogdogdog" starting at index 0 and ending at index 9.
I found the text "dogdogdog" starting at index 9 and ending at index 18.

Enter your regex: dog{3}
Enter input string to search: dogdogdogdogdogdog
No match found.
```

第一个例子找到三个匹配，因为量词应用于整个捕获组。但是，删掉括号之后匹配失败，因为现在量词{3}只应用于字母“g”。

类似的，我们可以对整个字符类应用量词：

420

```
Enter your regex: [abc]{3}
Enter input string to search: abccabaaaccbbbc
I found the text "abc" starting at index 0 and ending at index 3.
I found the text "cab" starting at index 3 and ending at index 6.
I found the text "aaa" starting at index 6 and ending at index 9.
I found the text "ccb" starting at index 9 and ending at index 12.
I found the text "bbc" starting at index 12 and ending at index 15.

Enter your regex: abc{3}
Enter input string to search: abccabaaaccbbbc
No match found.
```

在第一个例子中，量词{3}应用于整个字符类，但是第二个例子中只应用于字母“c”。

13.6.3 greedy、reluctant 和 possessive 量词的区别

greedy、reluctant和possessive量词之间有微妙的区别。

greedy量词被看作“贪婪的”，因为它们在试图搜索第一个匹配之前读完（或者说吃掉）整

个输入字符串。如果第一个匹配尝试（整个输入字符串）失败，匹配器就会在输入字符串中后退一个字符并且再次尝试，重复这个过程，直到找到匹配或者没有更多剩下的字符可以后退为止。根据表达式中使用的量词，它最后试图匹配的内容是1个或者0个字符。

但是，reluctant量词采取相反的方式：它们从输入字符串的开头开始，然后逐步地一次读取一个字符搜索匹配。它们最后试图匹配的内容是整个输入字符串。

最后，possessive量词总是读完整个输入字符串，尝试一次（而且只有一次）匹配。和greedy量词不同，possessive从不后退，即使这样做能允许整体匹配成功。

为了演示，我们分析输入字符串xfooxxxxxxfoo：

```
Enter your regex: .*foo  // greedy quantifier
Enter input string to search: xfooxxxxxxfoo
I found the text "xfooxxxxxxfoo" starting at index 0 and ending at index 13.

Enter your regex: .*?foo  // reluctant quantifier
Enter input string to search: xfooxxxxxxfoo
I found the text "xfoo" starting at index 0 and ending at index 4.
I found the text "xxxxxxfoo" starting at index 4 and ending at index 13.

Enter your regex: .*+foo // possessive quantifier
Enter input string to search: xfooxxxxxxfoo
No match found.
```

421

第一个例子使用greedy量词.*搜索“任何内容”零次或者多次，后面是字母f、o、o。因为是greedy量词，所以表达式的.*部分首先读完整个字符串。这样，整个表达式不会成功，因为最后三个字母（“f”“o”“o”）已经被消耗了。所以匹配器缓慢地一次后退一个字母，一直后退到最右侧出现“foo”为止，这里匹配成功并且搜索停止。

但是第二个例子使用的量词是reluctant量词，所以它首先消耗“无内容”。因为“foo”没有出现在字符串的开头，所以迫使它消耗掉第一个字母（x），这样就在索引0和4的位置触发第一个匹配。我们的测试示例继续处理，直到输入字符串耗尽为止。它在索引4和13找到了另一个匹配。

第三个例子找不到匹配，因为是possessive量词。这种情况下，.*+消耗整个输入字符串，在表达式的结尾没有剩下满足“foo”的内容。possessive量词用于处理所有内容，但是从不后退的情况；在没有立即发现匹配的情况下，它的性能优于功能相同的greedy量词。

13.7　捕获组

前面我们讲过如何把量词一次附加到一个字符、字符类或者捕获组之后。但是到目前为止，我们还没有详细讨论过捕获组的概念。

捕获组（capturing group）是把多个字符当作一个单元对待的一种方式。通过把字符括在括号内创建捕获组。例如，正则表达式(dog)创建包含字母“d”、“o”和“g”的一个组。输入字符串和捕获组匹配的那一部分将被保存在内存中，以便以后通过反向引用再次使用（见13.7.2节的讨论）。

13.7.1　编号

如Pattern API中所述，按照从左到右的顺序计算捕获组的前括号数目，给捕获组编号。例如，在表达式((A)(B(C)))中，有4个这样的组：

(1) ((A).(B(C)))

(2) (A)

(3) (B(C))

(4) (C)

422

为了查出表达式中有多少个组，可以对匹配器对象调用groupCount方法。groupCount方法返回一个int，表示匹配器模式中出现的捕获组的数量。在这个例子中，groupCount将返回4，显示这个模式包含4个捕获组。

还有一个特殊的组，组0，它总是代表整个表达式。这个组不包括在groupCount报告的总数内。以(? 开头的组是纯粹的非捕获组（non-capturing group），它不捕获文本，也不计入组的总数。（后面的13.9节中将给出非捕获组的例子。）

了解组如何编号非常重要，因为一些Matcher方法接受指定特定捕获组编号的int作为参数：

- **public int start(int group)**——返回前一个匹配操作期间，给定组捕获的子序列的开始索引。
- **public int end(int group)**——返回前一个匹配操作期间，给定组捕获的子序列的最后一个字符的索引加1。
- **public String group(int group)**——返回前一个匹配操作期间，给定组捕获的输入子序列。

13.7.2 反向引用

输入字符串中和捕获组匹配的部分被保存在内存中，以便以后通过反向引用（backreference）再次使用。反向引用在正则表达式中被指定为反斜线（\），后面跟着一个数字，指出要再次使用的组的编号。例如，表达式（\d\d）定义匹配一行中两个数字的一个捕获组，以后可以通过反向引用\1再次使用它。

为了匹配任意两个数字，后面跟着完全相同的两个数字，可以使用（\d\d）\1作为正则表达式：

```
Enter your regex: (\d\d)\1
Enter input string to search: 1212
I found the text "1212" starting at index 0 and ending at index 4.
```

如果改动最后两个数字，匹配就会失败：

```
Enter your regex: (\d\d)\1
Enter input string to search: 1234
No match found.
```

对于嵌套捕获组，反向引用的工作方式完全相同：指定一个反斜线，后面跟着要再次使用的组的编号。

423

13.8 边界匹配器

到目前为止，我们只对是否在特定输入字符串的某个位置找到匹配感兴趣。我们从未关心过匹配发生在什么位置。

通过使用边界匹配器（boundary matcher）指定这样的信息，可以使模式匹配更加精确。例如，可能你希望找到特定单词，但是只搜索它出现在一行的开头或者末尾的情况。或者可能你希望知道匹配是否出现在单词边界或者前一个匹配的末尾。

表13-4列出并且解释所有边界匹配器。

表13-4　边界匹配器

^	一行的开头
$	一行的末尾
\b	单词边界
\B	非单词边界
\A	输入的开头
\G	前一个匹配的末尾
\Z	输入的末尾，但是只用于最后的终止符（如果有的话）
\z	输入的末尾

下面的例子演示边界匹配器^和$。前面讲过，^匹配一行的开头，而$匹配末尾：

```
Enter your regex: ^dog$
Enter input string to search: dog
I found the text "dog" starting at index 0 and ending at index 3.

Enter your regex: ^dog$
Enter input string to search:      dog
No match found.

Enter your regex: \s*dog$
Enter input string to search:           dog
I found the text "          dog" starting at index 0 and ending at index 15.
Enter your regex: ^dog\w*
Enter input string to search: dogblahblah
I found the text "dogblahblah" starting at index 0 and ending at index 11.
```


第一个例子成功是因为模式占据了整个输入字符串。第二个例子失败，因为输入字符串的开头包含额外空白。第三个例子指定的表达式允许数量不限的空白，后面在行的末尾是“dog”。第四个例子要求“dog”出现在行的开头，后面跟着数量不限的单词字符。

为了检查模式是否在单词边界开始或者结束（而不是更长的字符串内的子字符串），只需在模式的一边加上\b；例如\bdog\b：

```
Enter your regex: \bdog\b
Enter input string to search: The dog plays in the yard.
I found the text "dog" starting at index 4 and ending at index 7.

Enter your regex: \bdog\b
Enter input string to search: The doggie plays in the yard.
No match found.
```

为了在非单词边界匹配表达式，使用\B：

```
Enter your regex: \bdog\B
Enter input string to search: The dog plays in the yard.
No match found.

Enter your regex: \bdog\B
Enter input string to search: The doggie plays in the yard.
I found the text "dog" starting at index 4 and ending at index 7.
```

为了要求匹配只出现在前一个匹配的末尾，使用\G：

```
Enter your regex: dog
Enter input string to search: dog dog
I found the text "dog" starting at index 0 and ending at index 3.
```

```
I found the text "dog" starting at index 4 and ending at index 7.

Enter your regex: \Gdog
Enter input string to search: dog dog
I found the text "dog" starting at index 0 and ending at index 3.
```

第二个例子只找到一个匹配，因为“dog”的第二次出现不是从前一个匹配的末尾开始的。

13.9 Pattern 类的方法

到目前为止，我们只使用测试示例创建了Pattern对象的最基本形式。这一小节介绍高级技术，比如创建带有标志的模式和使用嵌入标志表达式。并且介绍我们还没有讨论过的其他一些有用的方法。

425

13.9.1 创建带有标志的模式

Pattern类定义了另一个compile方法，它接受一组影响匹配模式的方式的标志。标志参数是一个位掩码，它可能包括以下任何公共静态字段：

- **Pattern.CANON_EQ**——启用规范相等性。如果指定了这个标志，当（并且仅当）两个字符的完整规范分解匹配时，才认为它们匹配。例如，当指定了这个标志时，表达式“a\u030A”和字符串“\u00E5”匹配。默认情况下，匹配不考虑规范相等性。指定这个标志可能会导致性能的降低。
- **Pattern.CASE_INSENSITIVE**——启用不区分大小写的匹配。默认情况下，不区分大小写的匹配假定只匹配US-ASCII字符集中的字符。通过同时指定UNICODE_CASE和这个标志，可以启用识别Unicode的不区分大小写匹配。也可以通过嵌入标志表达式(? i)启用不区分大小写匹配。指定这个标志可能导致轻微的性能降低。
- **Pattern.COMMENTS**——允许在模式中使用空白和注释。在这种模式下，空白被忽略，并且嵌入的注释（#开始，直到行末尾）被忽略。也可以通过嵌入标志表达式(? x)启用注释模式。
- **Pattern.DOTALL**——启用dotall模式。在dotall模式下，表达式.匹配任何字符，包括行终止符。默认情况下，这个表达式不匹配行终止符。也可以通过嵌入标志表达式(? s)启用dotall模式。（表达式中的s是“单行”模式的助记符，在Perl中把这种模式称为单行模式。）
- **Pattern.LITERAL**——启用模式的字面量解析。指定了这个标志时，指定模式的输入字符串被当作字面量字符的序列。输入序列中的元字符或者转移序列没有特殊含义。和这个标志结合使用时，标志CASE_INSENSITIVE和UNICODE_CASE保留它们对匹配的影响。其他标志则变成多余的。没有用于启用字面量解析的嵌入标志字符。
- **Pattern.MULTILINE**——启用多行模式。在多行模式下，表达式^和$分别正好在行终止符或者输入序列的末尾的之前或者之后。默认情况下，这些表达式只匹配整个输入序列的开头和结尾。也可以通过嵌入标志表达式(? m)启动多行模式。
- **Pattern.UNICODE_CASE**——启用识别Unicode的大小写折叠。指定了这个标志时，不区分大小写的匹配（由CASE_INSENSITIVE标志启动）按照和Unicode标准一致的方式完成。默认情况下，不区分大小写的匹配假定只匹配US-ASCII字符集中的字符。也可以通过嵌入标志表达式(?u)启用识别Unicode的大小写折叠。指定这个标志可能导致性能降低。

426

- **Pattern.UNIX_LINES**——启用UNIX行模式。在这种模式下，在.、^和$的行为中只把‘\n’识别为行终止符。也可以使用嵌入标志表达式(? d)启动UNIX行模式。

下面我们将修改测试示例RegexTestHarness.java，使用区分大小写的匹配创建模式。首先，修改代码调用compile的替代版本：

```
Pattern pattern =
Pattern.compile(console.readLine("%nEnter your regex: "),
Pattern.CASE_INSENSITIVE);
```

然后编译和运行测试示例，得到如下输出：

```
Enter your regex: dog
Enter input string to search: DoGDOg
I found the text "DoG" starting at index 0 and ending at index 3.
I found the text "DOg" starting at index 3 and ending at index 6.
```

如你所见，字符串字面量“dog”和两次出现匹配，而不管大小写如何。为了编译带有多个标志的模式，需要使用位或操作符“|”分隔要包含的标志。为了表达明确，下面的代码示例硬编码正则表达式，而没有从控制台读取：

```
pattern = Pattern.compile("[az]$",
          Pattern.MULTILINE | Pattern.UNIX_LINES);
```

你也可以指定一个int变量：

```
final int flags =
  Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE;
Pattern pattern = Pattern.compile("aa", flags);
```

13.9.2　嵌入标志表达式

可以使用嵌入标志表达式（embedded flag expression）启用各种标志。嵌入标志表达式是compile的两个实参版本的替代方式，在正则表达式本身中指定。下面的例子使用原始的测试
427 示例RegexTestHarness.java，并且用嵌入标志表达式(? i)启用不区分大小写的匹配：

```
Enter your regex: (?i)foo
Enter input string to search: FOOfooFoOfoO
I found the text "FOO" starting at index 0 and ending at index 3.
I found the text "foo" starting at index 3 and ending at index 6.
I found the text "FoO" starting at index 6 and ending at index 9.
I found the text "foO" starting at index 9 and ending at index 12.
```

同样，所有匹配成功，没有区分大小写。

和Pattern的公共可访问字段对应的嵌入标志表达式如表13-5所示。

表13-5　嵌入标志表达式

常　量	等效的嵌入标志表达式
Pattern.CANON_EQ	无
Pattern.CASE_INSENSITIVE	(?i)
Pattern.COMMENTS	(?x)
Pattern.MULTILINE	(?m)
Pattern.DOTALL	(?s)
Pattern.LITERAL	无
Pattern.UNICODE_CASE	(?u)
Pattern.UNIX_LINES	(?d)

13.9.3 使用 matches(String,CharSequence)方法

Pattern类定义了一个方便的matches方法，它允许你快速地检查模式是否出现在给定的输入字符串中。和所有公共静态方法一样，通过其类名称调用matches，比如Pattern.matches("\\d","1");。在这个例子中，方法返回true，因为数字“1”匹配正则表达式\d。

13.9.4 使用 split(String)方法

split方法是用于从进行匹配操作的模式的任何一侧收集文本的非常有用的工具。比如在下面的SplitDemo.java[1]中，split方法可以从字符串“one:two:three:four:five”中提取单词“one two three four five”：

428

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class SplitDemo {

  private static final String REGEX = ":";
  private static final String INPUT
                          = "one:two:three:four:five";

  public static void main(String[] args) {
    Pattern p = Pattern.compile(REGEX);
    String[] items = p.split(INPUT);
    for(String s : items) {
      System.out.println(s);
    }
  }
}
```

输出如下：

```
one
two
three
four
five
```

为了简单起见，我们匹配一个字符串字面量，冒号（:），没有使用复杂的正则表达式。因为我们仍然使用Pattern和Matcher对象，所以你可以使用split方法获得任何正则表达式的任何一侧的文本。下面是个相同的例子SplitDemo2.java[2]，它被修改为在数字处进行拆分：

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class SplitDemo2 {
  private static final String REGEX = "\\d";
  private static final String INPUT
                          = "one9two4three7four1five";

  public static void main(String[] args) {
    Pattern p = Pattern.compile(REGEX);
    String[] items = p.split(INPUT)
    for(String s : items) {
      System.out.println(s);
    }
```

1. tutorial/essential/regex/examples/SplitDemo.java
2. tutorial/essential/regex/examples/SplitDemo2.java

```
    }
}
```

输出如下：

```
one
two
three
four
five
```

13.9.5 其他工具方法

你会发现下面的方法也有一些用处：

- **public static String quote(String s)**——返回指定的String的字面量模式String。这个方法生成的String可以用于创建匹配s的Pattern，就像它是字面量模式一样。不会赋予输入序列中的元字符或者转移序列特殊的含义。
- **public String toString()**——返回这个模式的String表示。这是被编译的这个模式的正则表达式。

13.9.6 java.lang.String 中和 Pattren 等同的方法

java.lang.String[1]也通过若干方法支持正则表达式，这些方法模拟java.util.regex.Pattern的行为。为了便于参考，下面列出它们API中的关键摘要：

- **public boolean matches(String regex)**——判断这个字符串是否匹配给定的正则表达式。调用这个方法的*str*.matches(*regex*)形式产生的结果和表达式Pattern.matches(*regex*, *str*)完全一样。
- **public String[] split(String regex, int limit)**——按照给定正则表达式的匹配拆分这个字符串。调用这个方法的*str*.split(*regex*, *n*)产生的结果和表达式Pattern.compile (*regex*).split(*str*, *n*)完全一样。
- **public String[] split(String regex)**——按照给定正则表达式的匹配拆分这个字符串。这个方法的结果与使用给定表达式和限制实参0调用两个实参的拆分方法相同。结果数组内不包括末尾的空白字符串。

430

还有一个替换方法，它使用另一个CharSequence替换CharSequence：

- **public String replace(CharSequence target, CharSequence replacement)**——使用指定的字面量替换序列，替换这个字符串中和字面量目标序列匹配的每个子字符串。从字符串的开始到末尾进行替换操作，例如，使用“b”替换字符串“aaa”中的“aa”，结果为“ba”，而不是“ab”。

13.10 Matcher 类的方法

这一小节介绍Matcher类的其他一些有用的方法。为了方便起见，按照功能对下面列出的方法分组。

13.10.1 索引方法

索引方法（index method）提供有用的索引值，精确地指出在输入字符串中的什么位置找到

1. docs/api/java/util/regex/String.html

了匹配：

- **public int start()**——返回前一个匹配的开始索引。
- **public int start(int group)**——返回前一个匹配操作期间，给定组捕获的子序列的开始索引。
- **public int end()**——返回最后一个匹配的字符之后的偏移量。
- **public int end(int group)**——返回前一个匹配操作期间，给定组捕获的子序列的最后一个字符之后的偏移量。

13.10.2 研究方法

研究方法（study method）检查输入字符串，并且返回一个布尔值，指出是否找到了模式。

- **public boolean lookingAt()**——从一个区域的开头开始试图把输入序列和模式进行匹配。
- **public boolean find()**——试图查找输入序列中和模式匹配的下一个子序列。 431
- **public boolean find(int start)**——重新设置这个匹配器，然后从指定的索引开始，试图查找输入序列中和模式匹配的下一个子序列。
- **public boolean matches()**——试图把整个区域和模式进行匹配。

13.10.3 替换方法

替换方法（replacement method）是用于替换输入字符串中文本的有用的方法：

- **public Matcher appendReplacement(StringBuffer sb, String replacement)**——实现非终结符的附加和替换步骤。
- **public StringBuffer appendTail(StringBuffer sb)**——实现终结符的附加和替换步骤。
- **public String replaceAll(String replacement)**——使用给定的替换字符串，替换输入序列中和模式匹配的每个子序列。
- **public String replaceFirst(String replacement)**——使用给定的替换字符串，替换输入序列中和模式匹配的第一个子序列。
- **public static String quoteReplacement(String s)**——返回指定String的字面量替换String。这个方法生成的String像Matcher类的appendReplacement方法中的字面量替换s那样工作。生成的String和当作字面量序列的s中的字符序列匹配。不会赋予反斜线（\）和美元符号（$）特殊含义。

13.10.4 使用 start 和 end 方法

下面是一个例子MatcherDemo.java[1]，它计算输入字符串中单词“dog”的出现次数： 432

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class MatcherDemo {

  private static final String REGEX = "\\bdog\\b";
  private static final String INPUT =
                              "dog dog dog doggie dogg";
```

1. tutorial/essential/regex/examples/MatcherDemo.java

```
public static void main(String[] args) {

  Pattern p = Pattern.compile(REGEX);
  Matcher m = p.matcher(INPUT); // get a matcher object
  int count = 0;

  while(m.find()) {
    count++;
    System.out.println("Match number "+count);
    System.out.println("start(): "+m.start());
    System.out.println("end(): "+m.end());
  }
 }
}
```

输出为

```
Match number 1
start(): 0
end(): 3
Match number 2
start(): 4
end(): 7
Match number 3
start(): 8
end(): 11
```

可以发现，这个例子使用单词边界确保字母“d”“o”“g”不仅仅是更长的单词中的子字符串。它还给出关于匹配出现在输入字符串中什么位置的一些有用信息。start方法返回前一个匹配操作过程中，给定组捕获的子序列的开始索引，而end方法返回匹配的最后一个字符的索引加1。

433

13.10.5　使用 matches 和 lookingAt 方法

matches和lookingAt方法都试图把输入序列和模式进行匹配。但是，二者的区别在于matches要求整个输入字符串匹配，而lookingAt不是这样。两个方法总是从输入字符串的开头开始。下面是完整代码MatchesLooking.java[1]：

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class MatchesLooking {

  private static final String REGEX = "foo";
  private static final String INPUT = "fooooooooooooooooo";
  private static Pattern pattern;
  private static Matcher matcher;

  public static void main(String[] args) {

    // Initialize
    pattern = Pattern.compile(REGEX);
    matcher = pattern.matcher(INPUT);

    System.out.println("Current REGEX is: "+REGEX);
    System.out.println("Current INPUT is: "+INPUT);
```

1. tutorial/essential/regex/examples/MatchesLooking.java

```
        System.out.println("lookingAt(): "+matcher.lookingAt());
        System.out.println("matches(): "+matcher.matches());
      }
    }
```

程序输出如下：

```
Current REGEX is: foo
Current INPUT is: fooooooooooooooooo
lookingAt(): true
matches(): false
```

13.10.6 使用 replaceFirst(String)和 replaceAll(String)

replaceFirst和replaceAll方法提供和给定正则表达式匹配的文本。就像它们的名字所指出的，replaceFirst替换第一次出现，而replaceAll替换所有出现。下面是代码ReplaceDemo.java[1]：

434

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class ReplaceDemo {

  private static String REGEX = "dog";
  private static String INPUT = "The dog says meow. " +
                                "All dogs say meow.";
  private static String REPLACE = "cat";

  public static void main(String[] args) {
    Pattern p = Pattern.compile(REGEX);
    Matcher m = p.matcher(INPUT); // get a matcher object
    INPUT = m.replaceAll(REPLACE);
    System.out.println(INPUT);
  }
}
```

输出如下：

```
The cat says meow. All cats say meow.
```

在第一个版本中，dog的所有出现都被替换为cat。但是为什么不再进一步呢？除了替换简单的字面量（例如dog）之外，还可以替换匹配任何正则表达式的文本。这个方法的API声称“给定正则表达式a*b、输入aabfooaabfooabfoob和替换字符串-，对这个表达式的匹配器调用这个方法将生成字符串-foo-foo-foo-”。

下面是ReplaceDemo2.java[2]的代码：

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class ReplaceDemo2 {
  private static String REGEX = "a*b";
  private static String INPUT = "aabfooaabfooabfoob";
  private static String REPLACE = "-";
  public static void main(String[] args) {
    Pattern p = Pattern.compile(REGEX);
    Matcher m = p.matcher(INPUT); // get a matcher object
    INPUT = m.replaceAll(REPLACE);
    System.out.println(INPUT);
```

1. tutorial/essential/regex/examples/ReplaceDemo.java
2. tutorial/essential/regex/examples/ReplaceDemo2.java

```
    }
  }
```

输出如下：

435

```
-foo-foo-foo-
```

为了只替换模式的第一次出现，只需把调用replaceAll换成调用replaceFirst即可。它接受相同参数。

13.10.7 appendReplacement(StringBuffer,String)和appendTail(StringBuffer)方法

Matcher类还提供appendReplacement和appendTail方法用于文本替换。下面的例子RegexDemo.java[1]使用这两个方法达到和replaceAll相同的目的：

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class RegexDemo {
  private static String REGEX = "a*b";
  private static String INPUT = "aabfooaabfooabfoob";
  private static String REPLACE = "-";
  public static void main(String[] args) {
    Pattern p = Pattern.compile(REGEX);
    Matcher m = p.matcher(INPUT); // get a matcher object
    StringBuffer sb = new StringBuffer();
    while(m.find()){
      m.appendReplacement(sb,REPLACE);
    }
    m.appendTail(sb);
    System.out.println(sb.toString());
  }
}
```

输出如下：

```
-foo-foo-foo-
```

13.10.8 java.lang.String中和Matcher等同的方法

为了便利起见，String类也模拟了几个Matcher方法：

- **public String replaceFirst(String regex, String replacement)**——使用给定的替换，替换这个字符串中和给定正则表达式匹配的第一个子字符串。调用这个方法的*str*.replaceFirst(*regex*, *repl*)形式产生的结果和表达式Pattern.compile(*regex*).matcher (*str*).replaceFirst(*repl*)相同。

436

- **public String replaceAll(String regex, String replacement)**——使用给定的替换，替换这个字符串中和给定正则表达式匹配的每个子字符串。调用这个方法的*str*.replaceAll(*regex*,*repl*) 形式产生的结果和表达式Pattern.compile(*regex*).matcher(*str*). replaceAll(*repl*)相同。

13.11 PatternSyntaxException类的方法

PatternSyntaxException是不可控异常，指出正则表达式模式中的语法错误。

1. tutorial/essential/regex/examples/RegexDemo.java

PatternSyntaxException类提供如下方法帮助你判断发生了什么错误：

- **public String getDescription()**——获得错误描述。
- **public int getIndex()**——获得错误索引。
- **public String getPattern()**——获得错误的正则表达式模式。
- **public String getMessage()**——返回一个多行字符串，其中包含错误语法及其索引的描述、错误的正则表达式模式，以及模式内错误索引的可视指示。

下面的源代码RegexTestHarness2.java[1]更新了我们的测试示例，检查错误的正则表达式：

```
import java.io.Console;
import java.util.regex.Pattern;
import java.util.regex.Matcher;
import java.util.regex.PatternSyntaxException;

public class RegexTestHarness2 {

  public static void main(String[] args){

    Pattern pattern = null;
    Matcher matcher = null;

    Console console = System.console();
    if (console == null) {
      System.err.println("No console.");
      System.exit(1);
    }

    while (true) {
      try{
        pattern =
        Pattern.compile(console.readLine("%nEnter your " +
                                         regex: "));

        matcher =
        pattern.matcher(console.readLine("Enter input " +
                                         "string to search: "));
      }
      catch(PatternSyntaxException pse){
        console.format("There is a problem with " +
                       "the regular expression!%n");
        console.format("The pattern in question is: %s%n",
                       pse.getPattern());
        console.format("The description is: %s%n",
                       pse.getDescription());
        console.format("The message is: %s%n",
                       pse.getMessage());
        console.format("The index is: %s%n",pse.getIndex());
        System.exit(0);
      }
      boolean found = false;
      while (matcher.find()) {
        console.format("I found the text \"%s\" " +
                       "starting at index %d and ending " +
                       "at index %d.%n", matcher.group(),
                            matcher.start(), matcher.end());
```

1. **tutorial/essential/regex/examples/RegexTestHarness2.java**

```
            found = true;
          }
          if(!found){
            console.format("No match found.%n");
          }
        }
      }
    }
```

为了运行这个测试，输入?i)foo作为正则表达式。这个错误是常见的情况，程序员忘记了嵌入标志表达式(?i)中的前括号。这样做将生成如下结果：

```
Enter your regex: ?i)
There is a problem with the regular expression!
The pattern in question is: ?i)
The description is: Dangling meta character '?'
The message is: Dangling meta character '?' near index 0
?i)
^
The index is: 0
```


从输出中我们可以发现，语法错误是索引0位置的不确定的元字符（问号）。缺少前括号是错误的原因。

13.12　小结

在本章中，你了解到正则表达式是根据字符串集合中每个字符串共享的共同特性，描述这个字符集合的一种方式。Java编程语义通过java.util.regex包支持正则表达式，主要通过Pattern、Matcher和PatternSyntaxException类。

- Pattern对象是正则表达式编译后的表示。Pattern类没有提供公共构造器。为了创建模式，必须调用它的public static compile方法之一，它们都会返回一个Pattern对象。
- Matcher对象是解释模式和对输入字符串进行匹配操作的引擎。和Pattern类一样，Matcher没有定义公共构造器。通过调用Pattern对象的Public matcher方法获得Matcher对象。
- PatternSyntaxException对象是不可控异常，指出正则表达式模式中的语法错误。

这个API支持的模式匹配的最基本形式是字符串字面量的匹配。也可以指定元字符——具有特殊含义的字符，由匹配器解释它们。

字符类是括在方括号内的字符集合。它指定将成功匹配给定输入字符串中的单一字符的字符。你可以定义自己的字符类，或者使用API中包含的预定义字符类。

量词允许你指定匹配出现的次数。有三种不同类型的量词greedy、reluctant和possessive：

- greedy量词被认为是“贪婪的”，因为试图进行第一个匹配之前，它们强制匹配器读完（或者说吃掉）整个输入字符串。如果第一个匹配尝试（整个输入字符串）失败，匹配器就会在输入字符串中后退一个字符并且再次尝试，重复这个过程，直到找到匹配或者没有更多剩下的字符可以后退为止。根据表达式中使用的量词，它最后试图匹配的内容是1个或者0个字符。
- reluctant量词采取相反的方式：它们从输入字符串的开头开始，然后逐步地一次读取一个字符搜索匹配。它们最后试图匹配的内容是整个输入字符串。
- possessive量词总是读完整个输入字符串，尝试一次（而且只有一次）匹配。和greedy量

词不同，possessive从不后退，即使这样做能允许整体匹配成功。

捕获组提供把多个字符当作一个单元对待的方式。把字符括在括号内创建捕获组，并且按照从左到右的顺序，计算它们的前括号从而进行编号。输入字符串中和捕获组匹配的部分被保存在内存中，以便以后通过反向引用再次使用。反向引用在正则表达式中被指定为反斜线“\”，后面跟着一个数字，指出要再次使用的组的编号。

通过指定输入字符串中的匹配位置，边界匹配器使匹配更加精确。regex API提供用于如下位置的边界匹配器：一行的开头、一行的末尾、单词边界、非单词边界、输入的开头、输入的末尾和前一个匹配的末尾。

最后，你详细地学习了Pattern、Matcher和PatternSyntaxException类，了解了它们的附加功能，包括java.lang.String中和它们等同的方法。

13.13 附加资源

关于正则表达式结构的更准确的描述，我们推荐*Mastering Regular Expressions*这本书，作者Jeffrey E. F. Friedl（O'Reilly-2003年出版）。

问题和练习：正则表达式

问题

1. java.util.regex包中的三个公共类是什么？分别描述它们的用途。
2. 考虑字符串字面量“foo”。开始索引是什么？结束索引是什么？解释这些数字的含义。
3. 普通字符和元字符之间有什么区别？分别给出二者的例子。
4. 如何强制元字符的行为像普通字符那样？
5. 括在方括号内的字符集合叫作什么？它的用途是什么？ 440
6. 有三个预定义类：\d、\s和\w。分别描述它们，并且使用方括号重新编写它们。
7. 分别使用\d、\s和\w编写两个简单的表达式，和相反的字符集合匹配。
8. 分析正则表达式(dog){3}。识别两个子表达式。这个表达式和什么字符串匹配？

练习

使用反向引用编写一个表达式，它与名和姓相同的人名匹配。

答案

可以在以下位置找到“问题”和“练习”的答案：

tutorial/essential/regex/QandE/answers.html 441

第14章 平台环境

应用程序运行在平台环境（platform environment）中，平台环境由底层操作系统、Java虚拟机、类库和启动应用程序时提供的各种配置数据定义。本章介绍一些用于检查和配置平台环境的API和应用程序。

14.1 配置工具

本节介绍一些配置工具，它们帮助应用程序访问其启动环境。

14.1.1 属性

属性（property）是作为键/值对（key / value pair）管理的配置值。在每个对中，键和值都是String[1]值。键标识符用于获得值，它很像用于获得变量值的变量名称。例如，能够下载文件的应用程序可能使用一个名为“download.lastDirectory”的属性跟踪用于最近下载的目录。

为了管理属性，需要创建java.util.Properties[2]的实例。这个类提供用于如下目的的

方法：

- 从流把键/值对加载到Properties对象中。
- 从其键获得值。
- 列出键和它们的值。
- 对键进行枚举。
- 把属性保存到流。

关于流的介绍请参考10.1节。

Properties扩展了java.util.Hashtable[3]。从Hashtable继承的一些方法支持如下操作：

- 检查特定的键或者值是否在Properties对象中。
- 获得当前的键/值对数量。
- 删除一个键及其值。
- 把键/值对添加到Properties清单中。
- 对值或者键进行枚举。
- 通过其键获得一个值。
- 查看Properties对象是否为空。

---

1. docs/api/java/lang/String.html
2. docs/api/java/util/Properties.html
3. docs/api/java/util/Hashtable.html

安全考虑 访问属性涉及由当前安全管理器批准的问题。这一小节中的示例代码片段假定是独立应用程序中的代码，默认情况下，这样的应用程序没有安全管理器。根据applet运行所处的浏览器的不同，applet中的相同代码可能无法工作。关于applet的安全限制的信息，请参考docs/books/tutorial/deployment/applet/proper- ties.html。

System类维护定义当前工作环境的配置的Properties对象。更多关于属性的介绍，参见14.2.2节。本节的剩余部分将会讲解如何使用属性管理应用程序配置。

1. 应用程序生存周期中的属性

图14-1演示典型的应用程序在其执行过程中将如何使用Properties对象管理其配置数据。 444

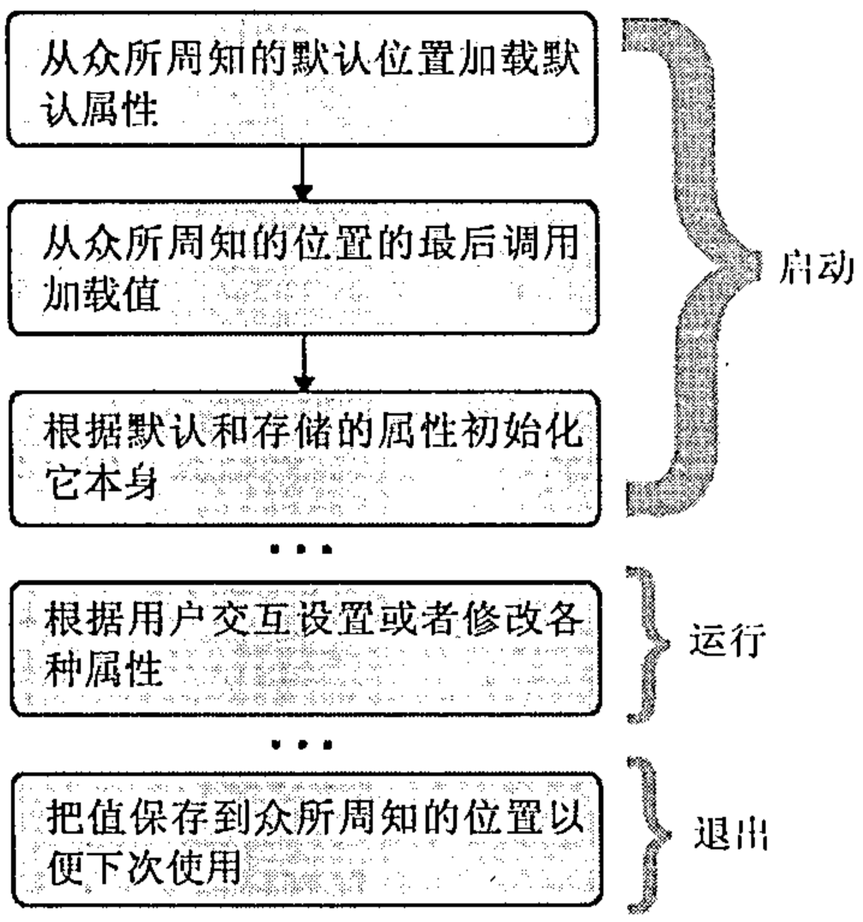


图14-1 属性角色

- **启动**——当应用程序启动时，进行前三个方框中描述的操作。首先，应用程序从众所周知的位置把默认属性加载到Properties对象中。通常，默认属性和应用程序的.class以及其他源文件一起存储在磁盘上的文件中。

  接下来，应用程序创建另一个Properties对象，并且加载上次应用程序运行时保存的属性。很多应用程序在每个用户的基础上存储属性，所以这个步骤中加载的属性通常在特定目录的指定文件中，由这个应用程序在这个用户的主目录中维护这个特定目录。最后，应用程序使用默认或者存储的属性初始化它自己。

  这里的关键在于一致性。应用程序加载和保存属性所使用的位置必须是一致的，以便它可以在下次执行时找到属性。
- **运行**——在应用程序的执行期间，用户可能在Preferences窗口中改动某些设置，并且更新Properties对象反映这些改动。如果用户的改动要在未来的会话中恢复，就必须存储它们。
- **退出**——在退出时，应用程序把属性保存到众所周知的位置，以便在下次启动应用程序时再次加载它们。

2. 设置Properties对象

445

下面的Java代码执行前一小节所描述的前两个步骤——加载默认属性和加载存储的属性：

```
...
// create and load default properties
```

```
Properties defaultProps = new Properties();
FileInputStream in = new FileInputStream("defaultProperties");
defaultProps.load(in);
in.close();

// create application properties with default
Properties applicationProps = new Properties(defaultProps);

// now load properties from last invocation
in = new FileInputStream("appProperties");
applicationProps.load(in);
in.close();
...
```

首先，应用程序设置默认的Properties对象。如果其他位置没有显式地设置值，那么就使用这个对象包含的属性集合。然后load方法从磁盘上名为defaultProperties的文件读取默认值。

接下来，应用程序使用不同的构造器创建第二个Properties对象applicationProps，它的默认值包含在defaultProps中。当获得属性时默认值开始起作用。如果在applicationProps中找不到属性，那么就搜索默认清单。

最后，代码从名为appProperties的文件把属性集合加载到applicationProps中。这个文件中的属性是上次调用应用程序时保存的那些属性，如下一小节所述。

3. 保存属性

下面的示例使用Properties.store写出前一个示例的应用程序属性。不需要保存默认属性，因为它们从不会改变：

```
FileOutputStream out = new FileOutputStream("appProperties");
applicationProps.store(out, "---No Comment---");
out.close();
```

store方法需要一个流去写入它，以及一个字符串作为输出开头的注释。

446

4. 获得属性信息

应用程序设置了其Properties对象之后，应用程序就可以查询这个对象，获得它包含的关于各种键和值的信息。应用程序启动后从Properties对象获得信息，以便能够根据用户做出的选择初始化它自己。Properties类有若干用于获得属性信息的方法：

contains(Object value)
containsKey(Object key)

如果Properties对象中包含指定的值或者键，则返回true。Properties从Hashtable继承这些方法。因此它们接受Object实参，但是只使用String值。

getProperty(String key)
getProperty(String key, String default)

返回指定属性的值。第二个版本提供默认值。如果没有找到这个键，则返回默认值。

list(PrintStream s)
list(PrintWriter w)

把所有属性写入到指定的流或者写入器。这对调试很有帮助。

elements()
keys()
propertyNames()

返回包含Properties对象中的键或者值的Enumeration（就像方法名称表述的）。keys方法只返回对象本身的键；propertyNames方法还返回默认属性的键。

stringPropertyNames()

和propertyNames类似，但是返回Set<String>，并且只返回键和值都是字符串的那些属性的名称。注意，这个Set对象和Properties对象没有关联，所以一个对象中的改动不会影响另一个对象。

size()

返回键/值对的当前数量。

5. 设置属性

在应用程序的执行期间，用户和应用程序的交互可能影响属性设置。这些改动反映在Properties对象中，以便应用程序退出（并且调用store方法）时保存它们。下面的方法改变Properties对象中的属性：

447

setProperty(String key, String value)

把键/值对存入Properties对象中。

remove(Object key)

删除和键相关联的键/值对。

注意 刚才介绍的一些方法是在Hashtable中定义的，因此接受String类型之外的键和值参数。即使方法允许使用其他类型，但是总要使用String类型的键和值。另外，不要对Properties对象调用Hashtable.set或者Hastable.setAll；而使用Properties.setProperty。

14.1.2 命令行参数

Java应用程序可以从命令行接受任意数量的参数。这就允许用户在启动应用程序时指定配置信息。

用户在调用应用程序时输入命令行参数，并且在要运行的类名称后面指定实参。例如，假设有一个名为Sort的Java应用程序，它对文件中的行进行排序。为了对名为friends.txt的文件中的数据进行排序，用户可以输入：

```
java Sort friends.txt
```

应用程序运行时系统通过一个String数组把命令行参数传递给应用程序的main方法。在前面的例子中，传递给Sort应用程序的命令行参数放在一个数组中，数组包含单一String："friends.txt"。

1. 输出命令行参数

Echo[1]示例显示其命令行参数，一个实参一行：

```
public class Echo {
  public static void main (String[] args) {
    for (String s: args) {
      System.out.println(s);
    }
  }
}
```

下面的例子显示用户可能如何运行Echo。用户的输入用斜体显示：

448

java Echo Drink Hot Java

---

1. tutorial/essential/environment/examples/Echo.java

```
Drink
Hot
Java
```

注意，应用程序在单独的一行显示每个单词——Drink、Hot和Java。这是因为空格字符分开了命令行参数。为了使Drink、Hot和Java被解释为单一实参，用户可以通过把它们括在引号中把它们连接起来：

```
java Echo "Drink Hot Java"
Drink Hot Java
```

2. 解析数字命令行参数

如果需要应用程序支持命令行数字参数，它就必须把表示数字的String（比如“34”）转换为数字值。下面的代码片段把命令行参数转换为int：

```
int firstArg;
if (args.length > 0) {
  try {
      firstArg = Integer.parseInt(args[0]);
  } catch (NumberFormatException e) {
      System.err.println("Argument must be an integer");
      System.exit(1);
  }
}
```

如果args[0]的格式非法，parseInt就会抛出NumberFormatException异常。Number类（Integer、Float、Double等）具有parseXXX方法，它把表示数字的String转换为数字类型的对象。

14.1.3　环境变量

很多操作系统使用环境变量（environment variable）把配置信息传递给应用程序。和Java平台中的属性一样，环境变量是键/值对，键和值都是字符串值。因为各种操作系统之间的差异，以及各种命令行解释器之间的差异，设置和使用环境变量的约定各不相同。为了了解操作系统如何把环境变量传递给应用程序，请参考系统文档。

1. 查询环境变量

在Java平台上，应用程序使用System.getEnv获得环境变量值。不带实参的getEnv返回只读的java.util.Map实例，其中映射键是环境变量名称，而映射值是环境变量值。EnvMap[1]示例中演示这种情况：

449

```
import java.util.Map;

public class EnvMap {
  public static void main (String[] args) {
    Map<String, String> env = System.getenv();
    for (String envName : env.keySet()) {
      System.out.format("%s=%s%n", envName, env.get(envName));
    }
  }
}
```

带有String实参的getEnv返回指定变量的值。如果这个变量没有定义，getEnv则返回null。Env[2]示例通过这种方式使用getEnv查询在命令行中指定的特定环境变量：

---

1. tutorial/essential/environment/examples/EnvMap.java
2. tutorial/essential/environment/examples/Env.java

```
public class Env {
  public static void main (String[] args) {
    for (String env: args) {
      String value = System.getenv(env);
      if (value != null) {
        System.out.format("%s=%s%n", env, value);
      } else {
        System.out.format("%s is not assigned.%n", env);
      }
    }
  }
}
```

2. 把环境变量传递给新进程

当Java应用程序使用ProcessBuilder[1]对象创建新进程时，传递给新进程的环境变量的默认集合和提供给应用程序的虚拟机进程的集合相同。应用程序使用ProcessBuilder.environment改变这个集合。

450

3. 平台相关性问题

在不同系统实现环境变量的方式之间有很多微妙区别。例如，Windows忽略环境变量值中的大小写，而UNIX则不同。使用环境变量的方式也各不相同。例如Windows在名为USERNAME的环境变量中提供用户名，而UNIX实现可能通过USER、LOGNAME或者二者结合的方式提供用户名。

为了使可移植性最大化，如果在系统属性中提供了相同的值，就不要引用环境变量。例如，如果操作系统提供用户名，它总在系统属性user.name中可用。

14.1.4 其他配置工具

这里总结一些其他的配置工具。

首选项API（Preferences API）允许应用程序在和实现相关的存储中存储和获得配置数据。支持异步更新，并且相同的首选项集合可以被多线程（甚至多个应用程序）安全地更新。关于更多信息，请参阅首选项API指南（Preferences API Guide）[2]。

部署在JAR档案文件（JAR archive）中的应用程序使用清单（manifest）描述文件的内容。关于更多信息，请参阅第16章。

Java Web Start应用程序的配置包含在JNLP文件中。关于更多信息，请参阅第17章。

Java Plug-in applet的一部分配置由用于把applet嵌入Web页面的HTML标记确定。根据applet和浏览器的不同，这些标记可以包括<applet>、<object>、<embed>和<param>。关于更多信息，请参阅第18章。

java.util.ServiceLoader[3]提供简单的服务提供器（service provider）工具。服务提供器是服务（service）——众所周知的接口和（通常是抽象的）类的集合的实现。服务提供器中的类通常实现服务中定义的接口并且子类化服务中定义的类。服务提供器可以作为扩展安装（参见docs/books/tutorial/ext/index.html）。也可以通过把提供器添加到类路径或者通过平台特定的一些其他方式，使提供器可用。

451

1. docs/api/java/lang/ProcessBuilder.html
2. docs/guide/preferences/index.html
3. docs/api/java/util/ServiceLoader.html

14.2　系统工具

System[1]类实现很多系统工具。在14.1节中已经介绍了一些。这一小节介绍一些其他的系统工具。

14.2.1　命令行I/O对象

System提供若干预定义的I/O对象，它们适用于需要从命令行启动的Java应用程序。这些对象实现大多数操作系统提供的标准I/O流，以及适用于输入密码的控制台对象。关于更多信息，参见10.1.5节。

14.2.2　系统属性

在14.1.1节中，我们分析了应用程序使用Properties对象维护其配置的方式。Java平台本身使用一个Properties对象维护它自己的配置。System类维护一个描述当前工作环境的配置的Properties对象。系统属性包括的信息和当前用户、Java运行时的当前版本以及用于分隔文件路径名部分的字符有关。

表14-1描述一些最重要的系统属性。

表14-1　一些重要的系统属性

键	含　义
"file.separator"	分隔文件路径部分的字符。在UNIX中是“/”；在Windows中是“\”
"java.class.path"	用于查找包含类文件的目录和JAR文件的路径。类路径的元素由path.separator属性中指定的平台特定的字符分隔
"java.home"	Java运行时环境（Java Runtime Environment，JRE）的安装目录
"java.vendor"	JRE厂商名称
"java.vendor.url"	JRE厂商的URL
"java.version"	JRE版本号
"line.separator"	操作系统用于分隔文本文件中的行的序列
"os.arch"	操作系统架构
"os.name"	操作系统名称
"os.version"	操作系统版本
"path.separator"	java.class.path中使用的路径分隔符
"user.dir"	用户工作目录
"user.home"	用户主目录
"user.name"	用户账户名

安全考虑　安全管理器限制对系统属性的访问。这在applet中经常是一个问题，它经常禁止读取某些系统属性，并且禁止写入任何系统属性。关于在applet中访问系统属性的更多信息，请参阅14.2.2节的第1小节。

1. 读取系统属性

System类有两个用于读取系统属性的方法：getProperty和getProperties。

1. docs/api/java/lang/System.html

System类的getProperty方法有两个不同版本。两个版本都获得实参列表中指定的属性的值。两个getProperty方法中比较简单的那个采用一个属性键作为实参。例如，为了获得path.separator的值，可以使用如下语句：

```
System.getProperty("path.separator");
```

getProperty方法返回包含这个属性的值的字符串。如果属性不存在，getProperty的这个版本返回null。

getProperty的另一个版本需要两个String实参：第一个实参是要搜索的键，如果找不到键或者键没有值，那么第二个实参就是返回的默认值。例如，下面的getProperty调用搜索名为subliminal.message的System属性。这不是合法的系统属性，所以这个方法没有返回null，而是返回第二个实参提供的默认值："Buy StayPuft Marshmallows!"：

```
System.getProperty("subliminal.message",
                   "Buy StayPuft Marshmallows!");
```

452 ~ 453

System类提供的访问属性值的最后一个方法是getProperties，它返回一个Properties对象。这个对象包含系统属性定义的完整集合。

2. 写系统属性

为了修改现有的系统属性集合，需要使用System.setProperties。这个方法采用Properties对象作为实参，这个对象被初始化为包含要设置的属性。这个方法使用Properties对象代表的新集合替换整个系统属性集合。

警告 改变系统属性有潜在的危险性，并且应该慎重进行。很多系统属性在启动后不重新读取，并且用于提供信息。改变一些属性可能造成不可预计的副作用。

下一个示例PropertiesTest[1]创建一个Properties对象，并且从myProperties.txt初始化它：

```
subliminal.message=Buy StayPuft Marshmallows!
```

然后PropertiesTest使用System.setProperties安装新的Properties对象作为当前系统属性的集合：

```
import java.io.FileInputStream;
import java.util.Properties;

public class PropertiesTest {

  public static void main(String[] args) throws Exception {
    // set up new properties object
    // from file "myProperties.txt"
    FileInputStream propFile = new FileInputStream(
                                    "myProperties.txt");
    Properties p = new Properties(System.getProperties());
    p.load(propFile);

    // set the system properties
    System.setProperties(p);
    // display new properties
    System.getProperties().list(System.out);
  }
}
```

454

1. tutorial/essential/environment/examples/PropertiesTest.java

注意PropertiesTest是如何创建Properties对象p的，它被用作setProperties的实参：

```
Properties p = new Properties(System.getProperties());
```

这个语句使用当前系统属性的集合，初始化新的属性对象p，这个集合在这个小应用程序示例中是运行时系统初始化的属性集合。然后，应用程序把附加的属性从文件myProperties.txt加载到p中，并且设置系统属性为p。结果是把myProperties.txt中列出的属性添加到了启动运行时系统创建的属性集合。注意，应用程序可以不使用任何默认的Properties对象创建p，比如：

```
Properties p = new Properties();
```

另外注意，系统属性的值可以被覆盖！例如，如果myProperties.txt包含如下一行，那么系统属性java.vendor就被覆盖了：

```
java.vendor=Acme Software Company
```

总的来说，注意不要覆盖系统属性。

setProperties方法改变当前运行中的应用程序的系统属性集合。这些改变不具有持久性。也就是说，即使在一个应用程序中改变系统属性，也不会影响Java解释器将来对这个应用程序或者其他应用程序的调用。运行时系统每次启动时重新初始化系统属性。如果对系统属性的改动需要持久化，那么应用程序就必须在退出之前把值写入某个文件，并且在启动时再次读取它们。

14.2.3　安全管理器

安全管理器（security manager）是定义应用程序安全策略的对象。这一策略指定不安全或者敏感的操作。安全策略不允许的任何操作都会抛出SecurityException[1]异常。应用程序也可以查询其安全管理器，从而了解允许哪些操作。

典型情况下，Web applet在运行时使用浏览器或者Java Plug-in提供的安全管理器。其他类型的应用程序一般运行不带有安全管理器，除非应用程序本身定义了安全管理器。如果不带安全管理器，那么应用程序就没有安全策略，并且其操作不受限制。

455

这一节讲解应用程序如何和现有的安全管理器进行交互。更详细的信息，包括如何设计安全管理器的信息，请参阅安全指南（Security Guide）[2]。

1. 和安全管理器进行交互

安全管理器是SecurityManager[3]类型的对象，为了获得对这个对象的引用，需要调用System.getSecurityManager:

```
SecurityManager appsm = System.getSecurityManager();
```

如果没有安全管理器，这个方法则返回null。

应用程序具有了对安全管理器对象的引用之后，就可以请求进行特定操作的许可。标准库中的很多类都是这样。例如，System.exit使用退出状态终止Java虚拟机，它调用SecurityManager.checkExit确保当前线程具有关闭应用程序的许可。

SecurityManager定义了很多其他的用于验证其他类型操作的方法。例如，SecurityManager.checkAccess验证线程访问，SecurityManager.checkProperty Access验证对

---

1. docs/api/java/lang/SecurityException.html
2. docs/guide/security/index.html
3. docs/api/java/lang/SecurityManager.html

指定的属性的访问。每个操作或者操作组都有其自己的checkXXX()方法。

另外，checkXXX()方法的集合表示已经属于安全管理器保护范围的操作集合。典型情况下，应用程序不必直接调用任何checkXXX()方法。

2. 识别安全违规

很多平常不带有安全管理器的操作，当带有安全管理器运行时，可以抛出SecurityException异常。即使调用没有指定为抛出SecurityException异常的方法，也是如此。例如，分析下面用于读取文件的代码：

```
reader = new FileReader("xanadu.txt");
```

没有安全管理器的情况下，这个语句的执行不出现错误，前提是xanadu.txt文件存在并且可读。但是，假设这个语句被插入到一个Web applet中，典型情况下Web applet在不允许文件输入的安全管理器之下运行。可能会出现下面的错误消息：

456

```
appletviewer fileApplet.html
Exception in thread "AWT-EventQueue-1"
  java.security.AccessControlException: access denied
  (java.io.FilePermission characteroutput.txt write)
  ...
```

注意这种情况下抛出的特定异常，java.security.AccessControlException[1]是SecurityException的子类。

14.2.4 系统中的杂项方法

这一小节介绍前面几节没有提到的System中的一些方法。

arraycopy方法高效地在数组之间复制数据。关于更多信息，请参阅3.1.3节。

currentTimeMillis和nanoTime方法非常适于测量应用程序执行过程中的时间间隔。为了测量以毫秒为单位的时间间隔，可以在间隔的开始和结束时两次调用currentTimeMillis，然后从返回的第二个值中减去第一个值。类似的，可以两次调用nanoTime测量以纳秒为单位的时间间隔。

注意 currentTimeMillis和nanoTime的精度受到操作系统提供的时间服务的限制。不要假设currentTimeMillis精确到最接近的毫秒，或者nanoTime精确到最接近的纳秒。另外，不应该使用currentTimeMillis和nanoTime确定当前时间。可以使用高级方法，比如java.util.Calendar.getInstance[2]。

---

exit方法使Java虚拟机关闭，带有实参指定的整数退出状态。退出状态可以被启动应用程序的进程使用。按照约定，退出状态0表示应用程序正常终止，而任何其他值都是错误码。

14.3 PATH 和 CLASSPATH

本节讲解如何在Microsoft Windows、Solaris和Linux系统中使用PATH和CLASSPATH环境变量。关于当前信息，请参考Java开发工具（Java Development Kit，JDK）软件包包含的安装指导。

457

---

1. docs/api/java/security/AccessControlException.html
2. docs/api/java/util/Calendar.html

安装好软件之后，JDK目录将具有如图14-2所示的结构。bin目录包含编译器和启动器。

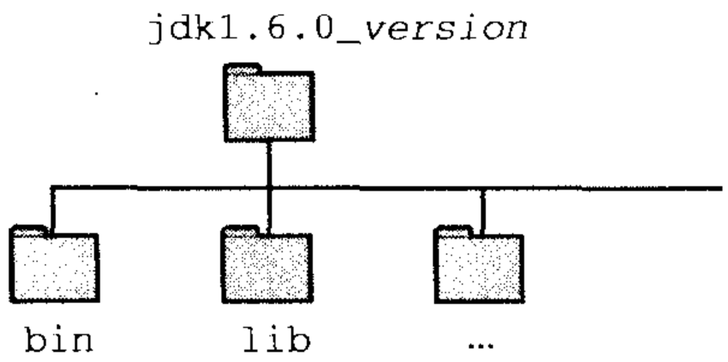


图14-2　JDK目录结构

14.3.1　更新 PATH 变量（Microsoft Windows NT／2000／XP）

你可以正常运行Java应用程序，而不必设置PATH变量。也可以为了方便起见，选择性地设置它。

如果你希望从任何目录都能方便地运行可执行文件（javac.exe、java.exe、javadoc.exe等），而不必输入命令的完整路径，那么可以设置PATH变量。如果没有设置PATH变量，那么每次运行可执行文件时都需要指定它的完整路径，比如：

```
C:\Program Files\Java\jdk1.6.0\bin\javac MyClass.java
```

永久地设置PATH很有帮助，这使它在重新启动之后保持不变。为了永久地设置，需要把jdk1.6.0的bin目录的完整路径添加到PATH变量中。按照下面的说明设置PATH。

为了永久改变CLASSPATH变量，需要使用Control Panel中的System图标。确切的步骤因Windows版本的不同而不同。

PATH可以是以分号（;）分隔的一系列目录。Microsoft Windows从左到右按照顺序在PATH目录中搜索程序。在路径中只能同时有JDK的一个bin目录（第一个后面的那些会被忽略），所以如果已经有了一个bin目录，就可以更新这个特定项目。

14.3.2　更新 PATH 变量（Solaris 和 Linux）

你可以正常运行Java应用程序，而不必设置PATH变量。但是，如果你希望从任何目录都能
458 运行可执行文件（javac、java、javadoc等），而不必输入命令的完整路径，就应该设置PATH变量。如果你没有设置PATH变量，那么每次运行可执行文件时都需要指定它的完整路径，比如：

```
% /usr/local/jdk1.6.0/bin/javac MyClass.java
```

为了查看是否正确地设置了路径，执行：

```
% java -version
```

如果找到的话这将输出java工具的版本。如果版本旧了或者出现错误java：Command not found，那么就是没有正确地设置路径。

为了永久地设置路径，可以在启动文件中设置路径。

对于C shell（csh），这样编辑启动文件（~/.cshrc）：

```
set path=(/usr/local/jdk1.6.0/bin )
```

对于bash，这样编辑启动文件（~/.bashrc）：

```
PATH=/usr/local/jdk1.6.0/bin:
export PATH
```

对于ksh，由环境变量ENV指定启动文件。这样设置路径：

```
PATH=/usr/local/jdk1.6.0/bin:
```

```
export PATH
```

对于sh，这样编辑设置文件（~/.profile）：

```
PATH=/usr/local/jdk1.6.0/bin:
export PATH
```

然后，加载启动文件，并且通过再次使用java命令验证设置后的路径。

对于C shell（csh）：

```
% source ~/.cshrc
% java -version
```

对于ksh、bash或者sh：

```
% . /.profile
% java -version
```

459

14.3.3 检查 CLASSPATH 变量（所有平台）

CLASSPATH变量是通知应用程序(包括JDK工具)在什么位置搜索用户类的一种方式。(JRE、JDK平台和扩展中的类应该通过其他途径来定义，比如引导类路径或者扩展目录。)

指定类路径的首选方式是使用-cp命令行开关。这允许分别为每个应用程序设置CLASSPATH，而不会影响其他应用程序。设置CLASSPATH可能比较需要技巧并且应该谨慎执行。

类路径的默认值为“.”，含义是只搜索当前目录。指定CLASSPATH变量或者-cp命令行开关会覆盖这个值。

在Microsoft Windows NT / 2000 / XP系统上，要检查是否设置了CLASSPATH，可以执行：

```
C:> echo %CLASSPATH%
```

在Solaris或者Linux系统上，可以执行：

```
% echo $CLASSPATH
```

如果没有设置CLASSPATH，就会出现CLASSPATH：Undefined variable错误（Solaris或者Linux系统），或者仅仅是%CLASSPATH%（Microsoft Windows NT / 2000 / XP系统）。

为了修改CLASSPATH，可以使用与修改PATH变量相同的方法。

类路径通配符允许在类路径中包含.jar文件的整个目录，不必单独地显式指定它们。关于更多信息，包括对类路径的介绍和如何清除CLASSPATH的详细讲解，请参阅docs/books/tutorial/technotes/tools/windows/classpath.html。

问题和练习：平台环境

问题

一个程序员安装了包含在一个.jar文件中的新的库。为了从他的代码访问这个库，他将CLASSPATH环境变量设置为指向新的.jar文件。现在他发现试图启动简单的应用程序时出现了错误消息：

460

```
java Hello
Exception in thread "main"
  java.lang.NoClassDefFoundError: Hello
```

在这个例子中，Hello类被编译为.class文件存放在当前目录中，但是似乎java命令找不到它。错误出在哪里？

练习

编写一个应用程序PersistentEcho，它具有如下特性：

(1) 如果带命令行参数运行PersistentEcho，它会输出这些实参。它还把输出的字符串保存到一个属性，并且把这个属性保存到名为PersistentEcho.txt的文件。
(2) 如果不带命令行参数运行PersistentEcho，它就查找名为PERSISTENTECHO的环境变量。如果这个变量存在，PersistentEcho就输出它的值，并且按照处理命令行参数的方式保存这个值。
(3) 如果不带命令行参数运行PersistentEcho，并且没有定义PERSISTENTECHO环境变量，PersistentEcho就从PersistentEcho.txt获得属性值并且输出这个值。

答案

可以在以下位置找到“问题”和“练习”的答案：

461　tutorial/essential/environment/QandE/answers.html

第15章

Swing

如何使用Java编程语言编写具有图形用户界面的程序？简短的回答是使用Swing工具包！本章概述核心Java平台的图形功能，重点介绍Swing。

本章没有介绍如何实现这些特性。关于“如何”使用Swing工具包，请参阅*Creating a GUI with JFC/Swing*[1]。

15.1 Swing包概述

本节简要介绍Java SE平台上用于开发具有图形用户界面（GUI）的程序的功能。然后，通过一个演示程序（提供了示例代码）展现这些功能。

后面的15.2节将会更加详细地讨论这些功能。

15.1.1 Swing是什么

为了创建带有图形用户界面的Java程序，就需要学习Swing。

Swing工具包包含用于构建GUI和为Java应用程序添加交互性的丰富的组件集合。Swing包含从现代工具包所能获得的所有组件：表控件、列表控件、树控件、按钮和标签。

但是，Swing远远超出了简单组件工具包的范畴。它包含丰富的撤销支持、可灵活定制的文本包、集成的国际化和可访问性支持。为了真正利用Java平台的跨平台功能，Swing支持数量众多的界面外观（look and feel），包括创建你自己的界面外观的功能。其中，Synth是一种专门为你设计的界面外观，使创建定制的界面外观的功能更加容易使用。但如果没有基本的用户界面元素（拖放、事件处理、可定制绘制和窗口管理），Swing也不能成为其一个组件工具包。

Swing是Java基础类（Java Foundation Classes，JFC）的一部分。JFC还包含对GUI程序很重要的其他特性，比如添加丰富的图形功能，以及创建可以用不同语言编写、可以由不同输入设备的用户使用的程序。

1. Swing GUI组件

Swing工具包包含丰富的组件：从基本的组件（按钮和复选框），到丰富复杂的组件（比如表和文本）。即使是看上去简单的组件（比如文本字段），也能提供复杂的功能（比如格式化的文本输入或者密码字段行为）。有适合大多数需求的文件浏览器和对话框，如果还不够，可以进行定制。如果Swing提供的组件不能完全满足你的要求，则可以利用基本的Swing组件功能创建自己的组件。

2. Java 2D API

为了使你的应用程序与众不同、表达视觉信息或者为GUI添加图表、图像或动画，你会希望使用Java 2D API。因为Swing是在2D包上构造的，所以在Swing组件内利用2D API轻而易举。

---

1. /tutorial/uiswing/index.html

添加图像、阴影和组合，这些使用Java 2D都很容易完成。

3. 可插拔的界面外观支持

任何使用Swing组件的程序员都可以选择界面外观。Sun和Apple发布的JFC类提供与平台相匹配的界面外观。Synth包允许创建自己的界面外观。GTK+界面外观使Swing程序可以使用数百种现成的界面外观。

程序可以指定运行平台的界面外观，也可以指定总使用的Java界面外观，并且无需重新编
464 译就可以工作。或者，你可以忽略这些事务，让UI管理器进行选择。

4. 数据传输

通过剪切、复制、粘贴和拖放的数据传输对几乎所有的应用程序都是至关重要的。对数据传输的支持内置到了Swing中，并且可以在应用程序的Swing组件之间、Java应用程序之间以及Java和其他应用程序之间工作。

5. 国际化

这一特性允许开发人员构造的应用程序可以和世界各地的用户交互，使用他们自己的语言和文化习惯。应用程序可以接受具有数千个不同字符的语言的输入，比如汉语、日语和韩语。

使用Swing的布局管理器可以很容易地应用UI要求的特殊方向。例如，在从右到左书写文字的地区，UI会以从右到左的方式出现。这种支持是自动的，只需你编写一次UI代码，它就能够随着文本的本地化而改变，以从左到右或者从右到左的方式工作，而且使组件具有适当的型号。

6. 可访问性API

残疾人使用专门的辅助软件作为中介手段。这样的软件需要获得正在运行的应用程序的大量信息，以便以替代的媒介表示它们：屏幕阅读器使用合成语音读出屏幕内容，或者通过盲文显示表达内容；屏幕放大镜跟踪提示符和键盘焦点；屏上键盘显示动态的键盘，用来控制菜单选项、工具栏项目和对话框控件；语音控制系统使用户可以通过声音控制操作。可访问性API使这些辅助软件能够获得所需的信息，并且通过程序操作那些构成图形用户界面的元素。

7. 撤销框架API

Swing的撤销框架允许开发人员提供撤销和重做支持。撤销支持内置在Swing的文本组件中。对于其他组件，Swing支持撤销和重做数量不限的操作，并且很容易应用到应用程序中。例如，你可以很容易地撤销表元素的添加和删除。

8. 灵活的部署支持

如果你希望程序运行在浏览器窗口中，可以把它创建为applet，然后使用Java Plug-in运行它，Java Plug-in支持各种浏览器，比如Internet Explorer、Firefox和Safari。如果你希望创建可以从浏
465 览器启动的程序，可以使用Java Web Start。当然，你的应用程序也可以在浏览器之外，作为独立的桌面应用程序运行。

关于部署应用程序的更多信息，请参阅第17章和第18章。

本章概述Swing的功能，首先通过一个演示程序展示很多Swing特性。当你准备好进行编程时，在线指南*Creating a GUI with JFC/Swing*可以帮助你利用这些特性的编程技术。

我们来分析一个展示了很多这些特性的演示程序。

15.1.2　Swing 演示程序

PasswordStore[1]演示程序演示Swing丰富特性集合的一部分。PasswordStore允许用户管理各个主机的登录信息。它还生成密码并评估密码的有效性，允许存储关于特定主机的注释或者指

---

1. 点击tutorial/ui/overview/demo.html上的launch按钮。

定表示主机的图标。

下面重点介绍PasswordStore应用程序的一些特定特性。[1]

1. 主机信息

在启动程序时，主机列表显示在Swing的列表组件中。使用View菜单可以在表和列表之间切换视图。

在两个视图中，Host/Account Filter文本字段可以用于动态地限制包含输入字符串的主机或者账户名称的项目。

(1) 列表视图

可以将Swing的列表组件定制为包含可视数据。如图15-1所示，主机名称左侧可选的微型图标代表主机。右侧的图形使用颜色和比例填充反映密码的强度（红色＝弱，黄色＝中等，绿色＝良好）。随着用户在下面的文本字段中进行输入或修改，条形图动态地变化。用户已经在过滤器文本字段中输入了“oo”，它和下面两项匹配：Heirloom Seeds和Pacific Zoo Shop。 466

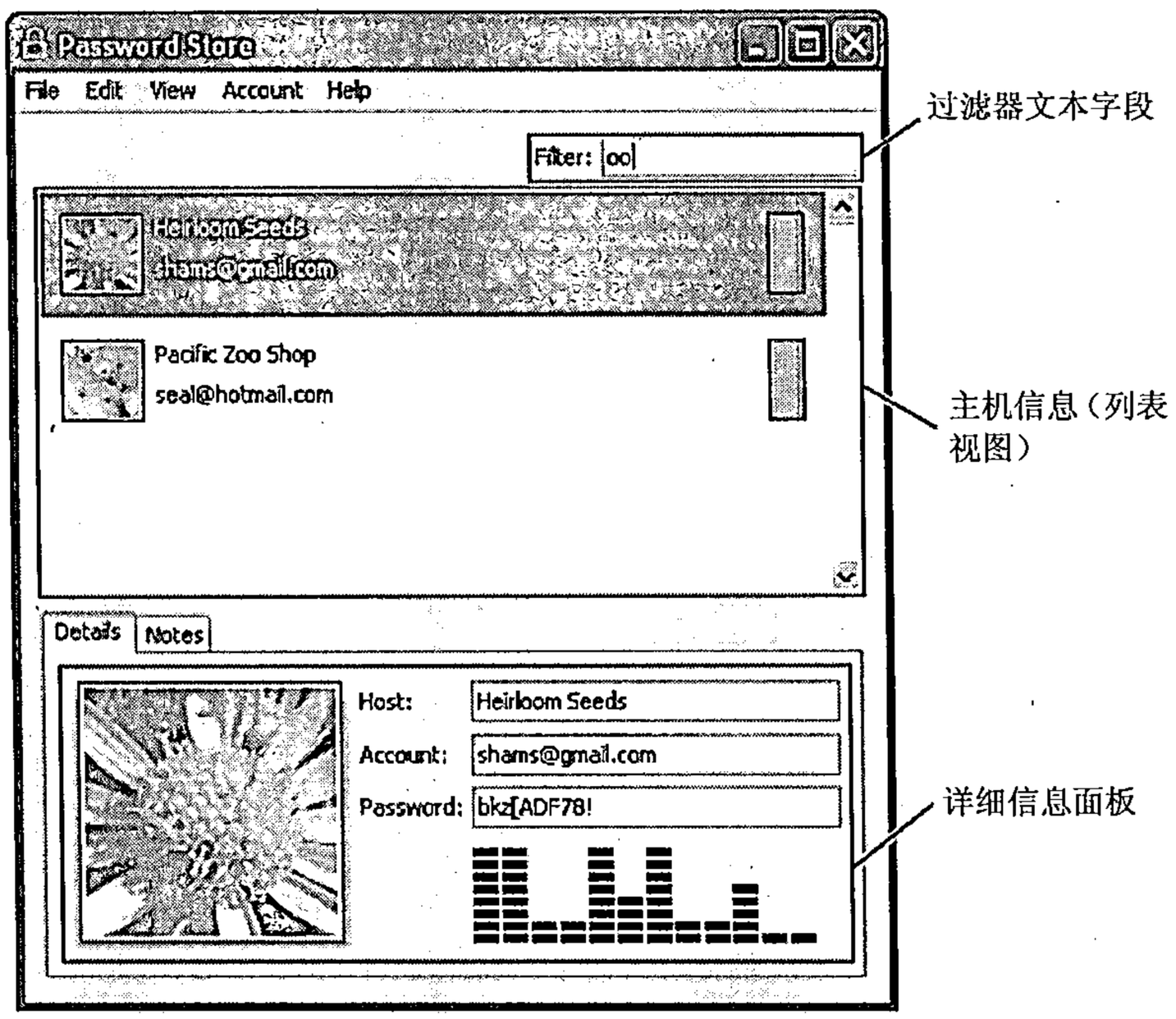


图15-1 主机信息（列表视图）和Filter文本字段

(2) 表视图

Swing表组件（图15-2）允许用户通过拖动列标题重新安排列。此外，还可以通过点击列标题对列进行排序。如果你点击的列还没有突出显示为主要排序列，那么它就会变成主要排序列，以升序排序。点击当前的主要排序列，就会切换排序顺序。例如，如果列1没有被选中，点击它就会使它成为被选中的列，并且按照升序排序其中的数据。再次点击列1，将按照降序排序。点击列2将使列2成为主要排序列，按照升序排序。 467

1. 感谢http://www.freeimages.co.uk提供演示程序中使用的骰子、花、药丸和袖珍表图像。感谢Steve Amstrup提供北极熊和幼熊的图像。感谢http://www.fws.gov提供的山峰图像。感谢http://grin.hq.nasa.gov提供的螺旋星系图像。

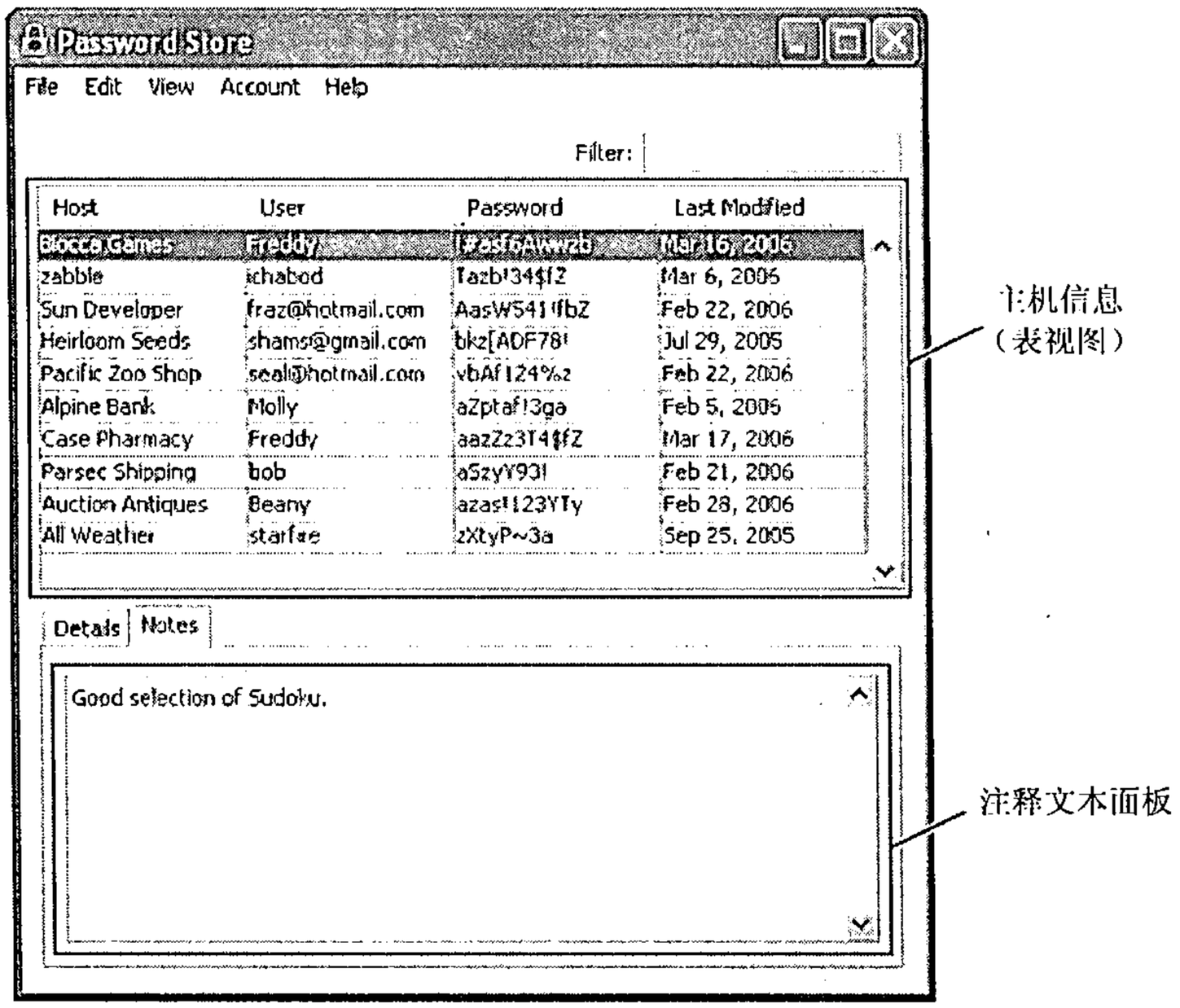


图15-2　主机信息（表视图）

2. Details / Notes选项卡式面板

主机信息下面的选项卡式面板允许用户选择Details面板和Notes文本面板，这使窗口的整体面积小一些，不会过多占用空间。

(1) Details面板

可以通过把图像（jpg、png、gif或者tif）拖动到左侧的图标区域，或者点击图像并打开文件浏览器，为图标区域指定图像。

用于输入或者修改主机名称、登录名和密码的文本字段支持剪切、复制、粘贴、拖放、撤销和重做。

当用户输入或者修改密码时，2D条形图动态地显示密码的分布状态。如果当前显示列表视
468 图，那么列表中相应的颜色条也动态地改变。

(2) Notes文本面板

用户可以在这个文本组件中保存所选择的主机的注释。如果文本面板包含URI，Swing的文本组件提供的功能允许点击URI，并且启动浏览器窗口自动打开这个URI。

3. 巧妙的2D图形

PasswordStore通过若干方式使用定制的图形来增强UI：在列表视图中，使用图像代表每个主机；颜色条——强度指示器（Strength Visualizer）代表密码的有效性；以及动态的条形图——密码指示器（Password Visualizer），代表密码的分布状态。当你要添加图像时，可以通过拖放图像（在Details面板中），或者通过点击图像并打开文件浏览器，为列表视图自动生成微型图标（见图15-3）。

注意　这个演示程序只用于演示目的，而不用于密码的实际分析。

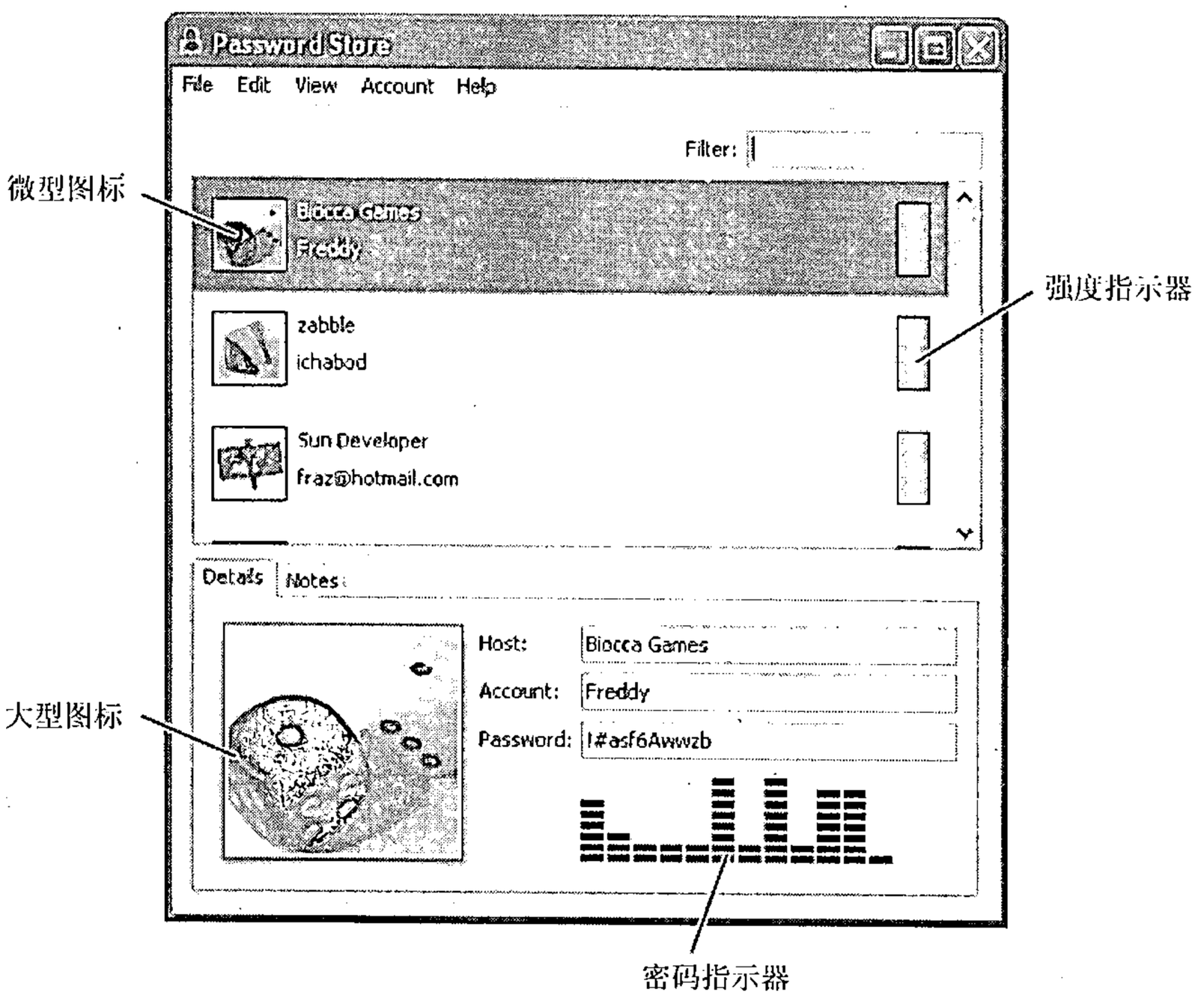


图15-3 2D图形

4. 多种界面外观

使用View菜单，可以在三种界面外观之间切换：Java（称为Metal）、Motif / CDE和本机界面外观：Microsoft Windows系统的Windows、Mac OS X系统的Aqua等。

5. 撤销和重做

撤销和重做按照预期处理文本，而且可以处理操作。例如，可以使用Account > Generate Password菜单生成密码，如果不喜欢新的密码，可以使用Edit > Undo或者control－Z快捷键撤销它。类似的，也可以使用Edit > Redo或者使用control－Y快捷键重做撤销的操作。

PasswordStore演示程序的复杂程度适合小型Swing应用程序，它展示了Swing功能的一部分。网上有源代码[1]可供下载，但是对其实现的详细讨论超出了本章的范围。关于这个应用程序的架构和实现的更多信息，请参阅java.net上Scott Violet编写的Excellent Blog[2]。

注意 如果PasswordStore是产品程序，很可能对密码数据库进行加密；但是，由于发布这类信息受美国法律限制，这里没有包含加密功能。

15.2 Swing 的特性

PasswordStore展示了Swing应用程序的丰富功能。本节讨论使用Java SE平台的应用程序可用的一般特性，特别介绍Swing工具包。

---

1. tutorial/ui/overview/PasswordStore-1.0-src.zip
2. http://weblogs.java.net/blog/zixle

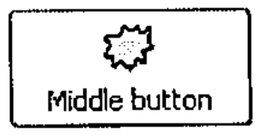
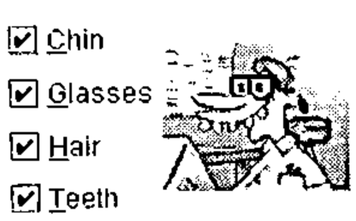
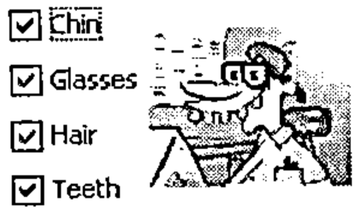
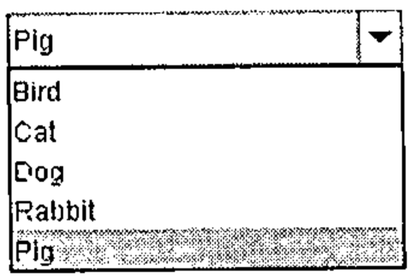
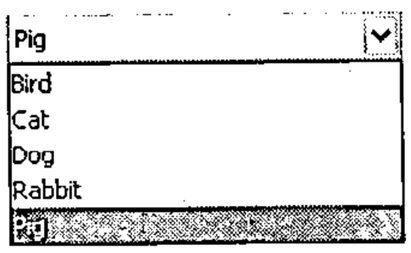
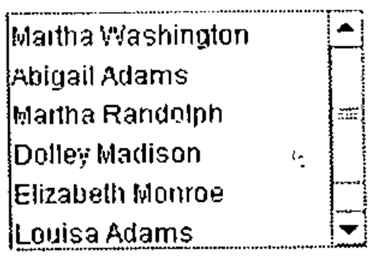
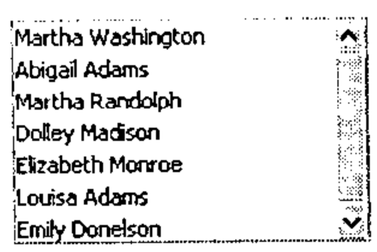
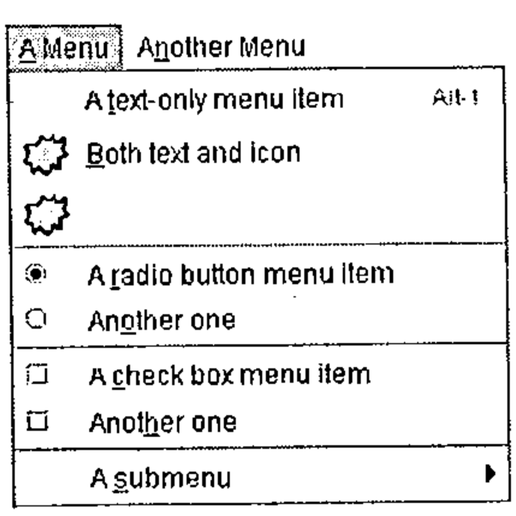
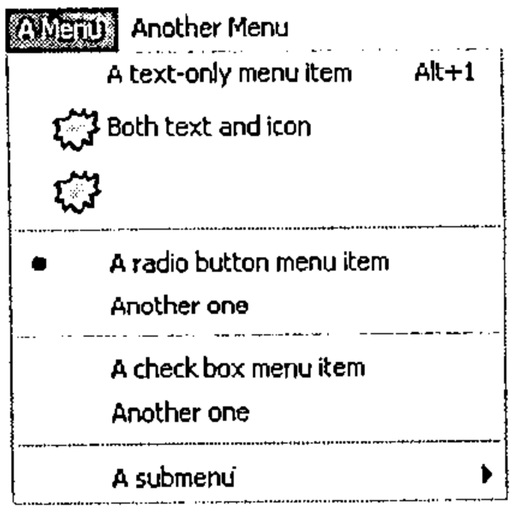
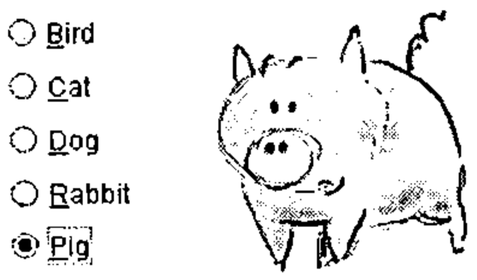
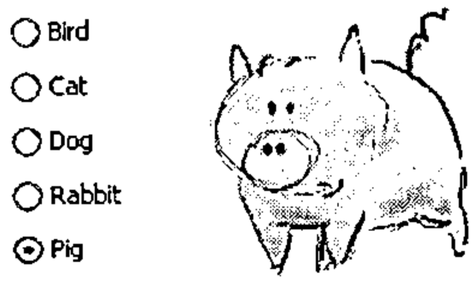
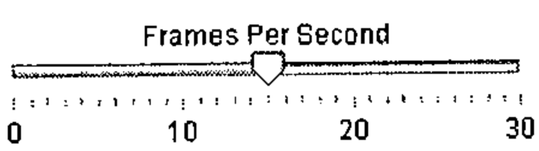
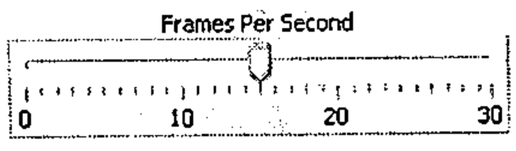
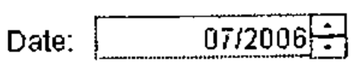
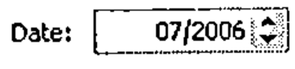
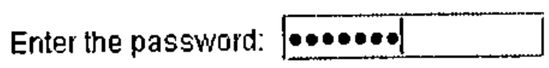
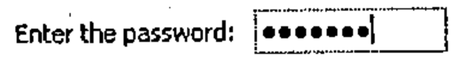
15.2.1　Swing 组件的可视化指导

本节以Java和Windows界面外观显示Swing组件。

1. 基本控件（表15-1）

470　简单组件主要用于用户输入；它们也可能显示简单状态。

表15-1　基本控件

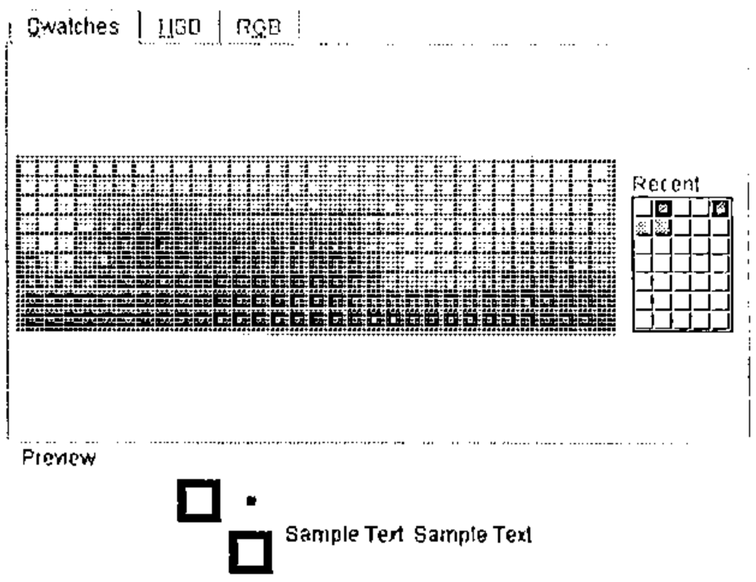
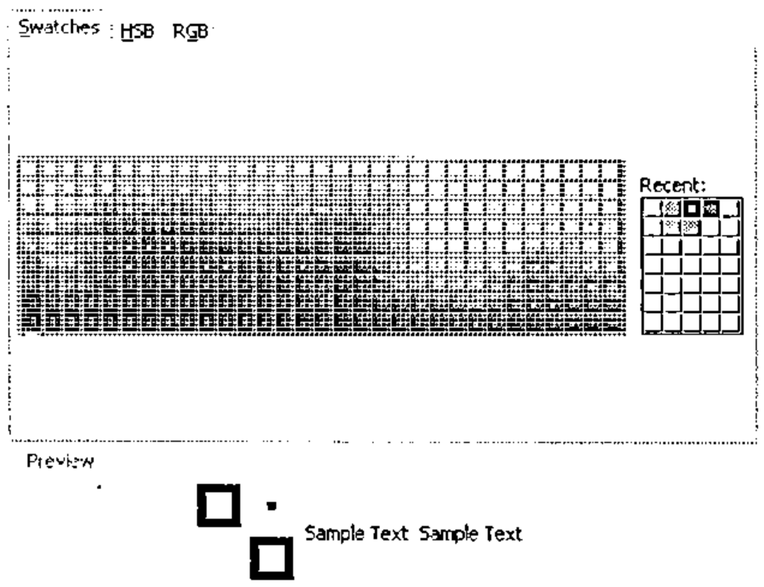
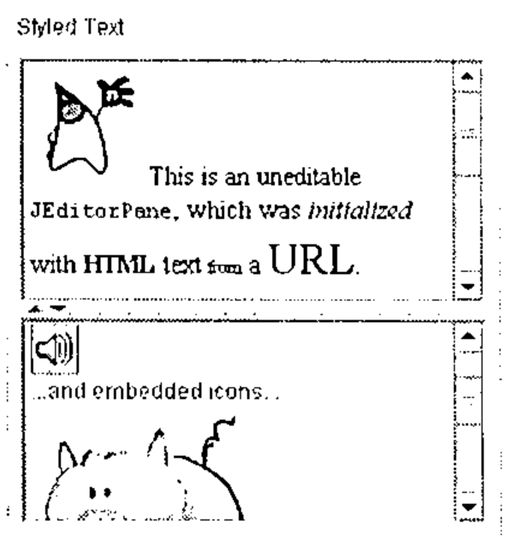
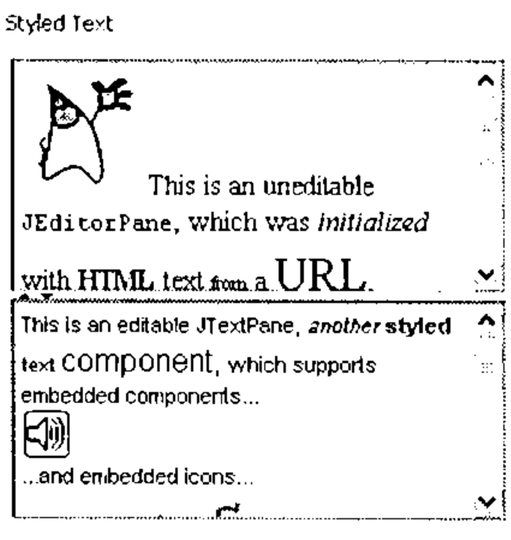
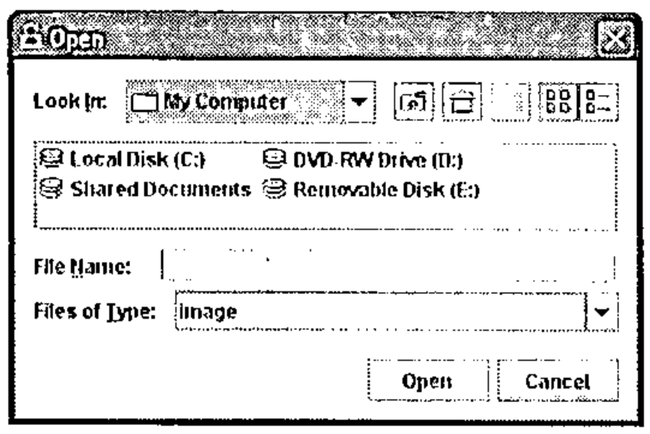
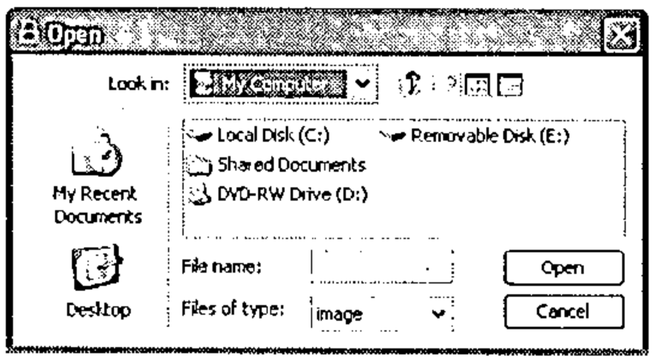
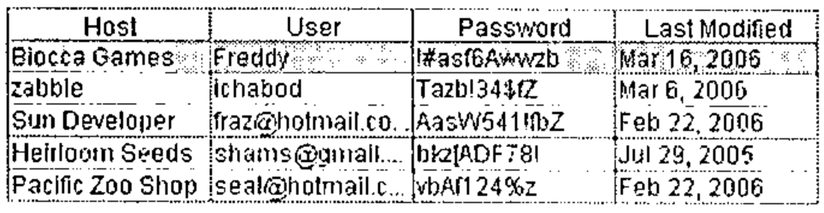
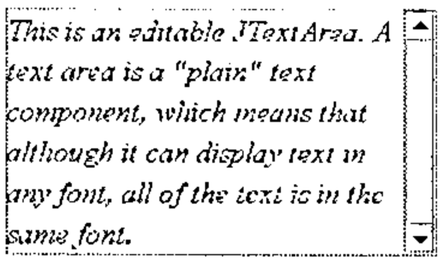
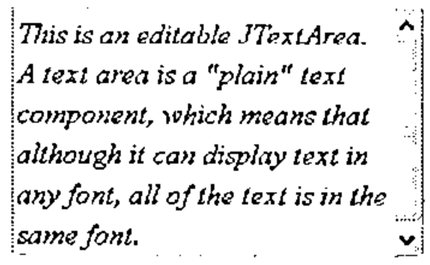
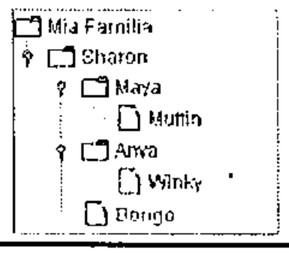
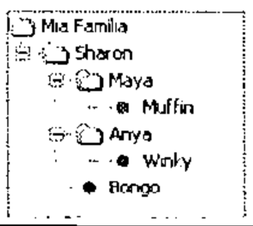
名　称	Java界面外观	Windows界面外观
JButton		
JCheckBox		
JComboBox		
JList		
JMenu		
JRadioButton		
JSlider		
JSpinner		
JPasswordField		

471

2. 高度格式化信息的交互性显示（表15-2）

这些组件显示高度格式化的信息，用户可以修改这些信息（如果你选择的话）。

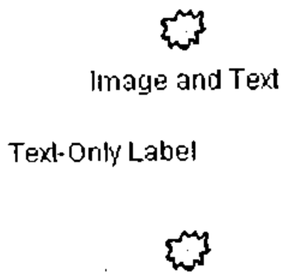
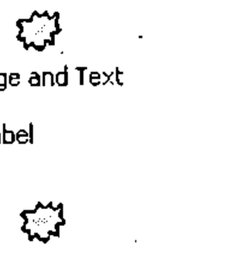
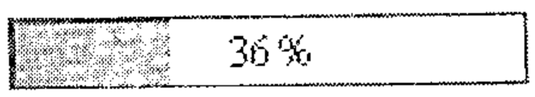
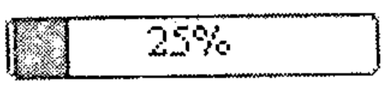
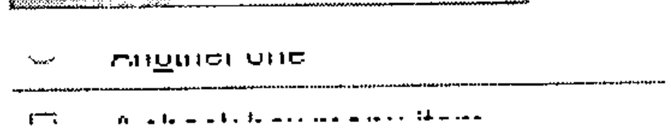
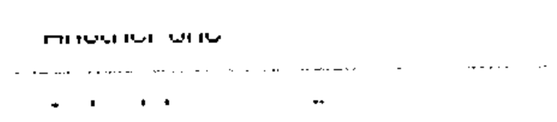
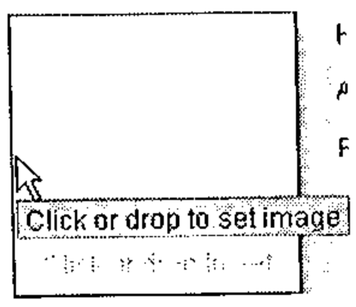
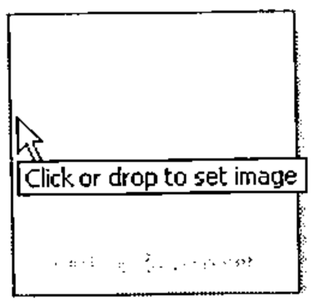
表15-2 高度格式化信息的交互性显示

Java界面外观	Windows界面外观
JColorChooser	
	
JEditorPane和JTextPane	
	
JFileChooser	
	
JTable	
	
JTextArea	
	
JTree	
	

3. 不可编辑的信息显示（表15-3）

这些组件专门用来向用户提供信息。

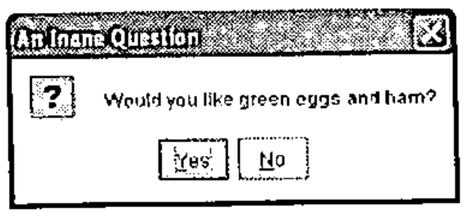
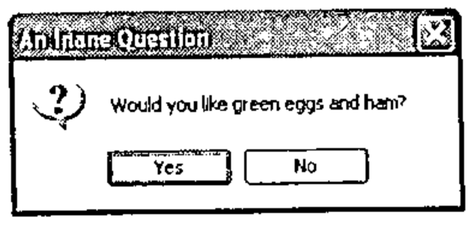
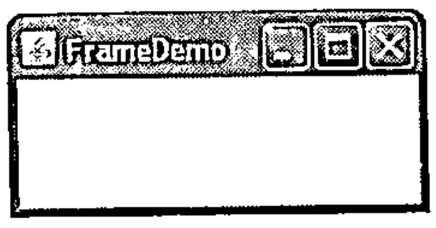
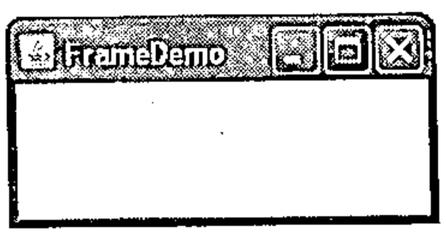
表15-3　不可编辑的信息显示

名　称	Java界面外观	Windows界面外观
JLabel		
JProgressBar		
JSeparator		
JToolTip		

4. 顶级容器（表15-4）

在任何Swing应用程序中都必须至少有一个这种组件。

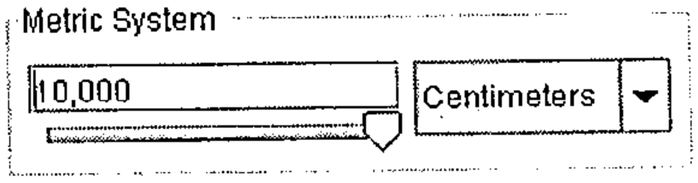
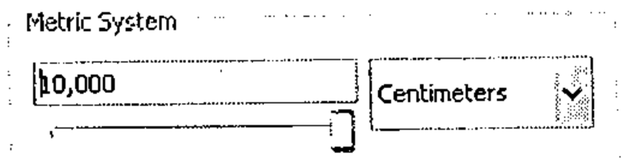
表15-4　顶级容器

名　称	Java界面外观	Windows界面外观
JApplet		
JDialog		
JFrame		

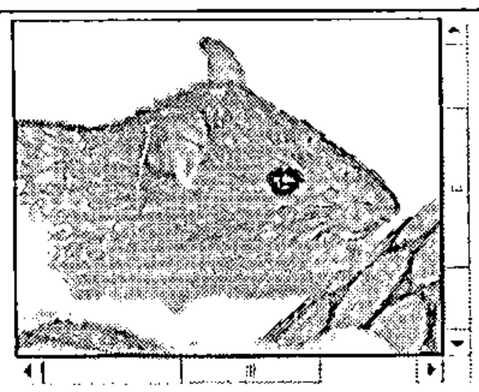
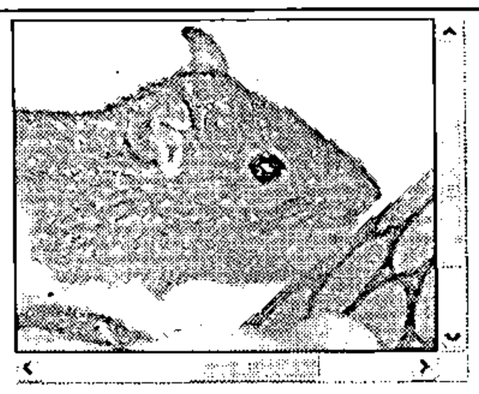
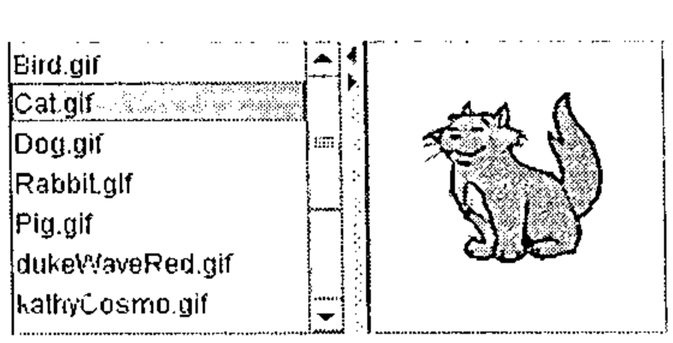
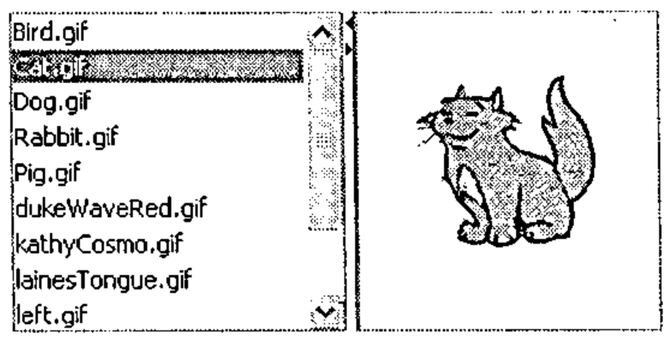
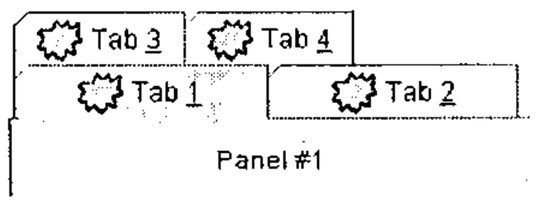
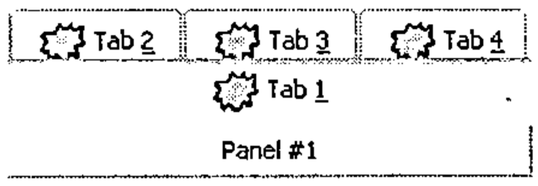
5. 通用容器（表15-5）

这些通用容器用在大多数Swing应用程序中。

表15-5　通用容器

Java界面外观	Windows界面外观
JPanel	
	
JScrollPane	

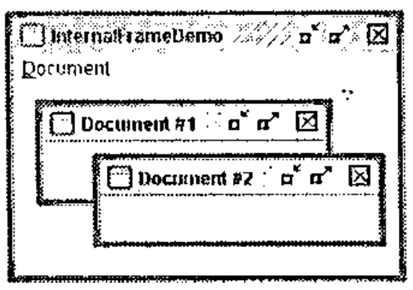
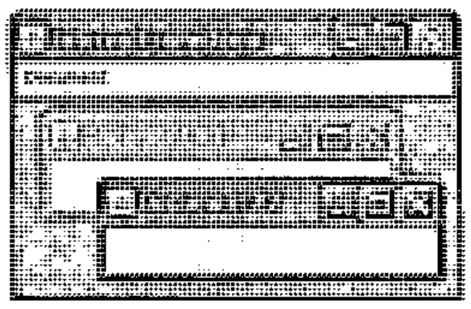
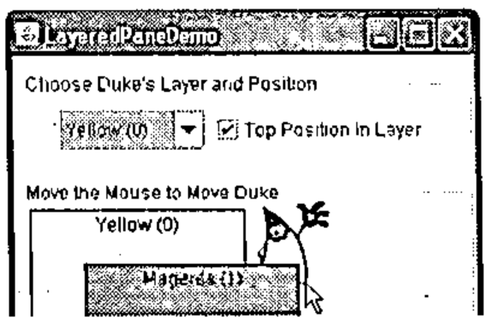
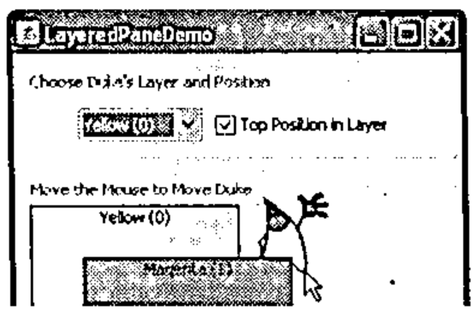
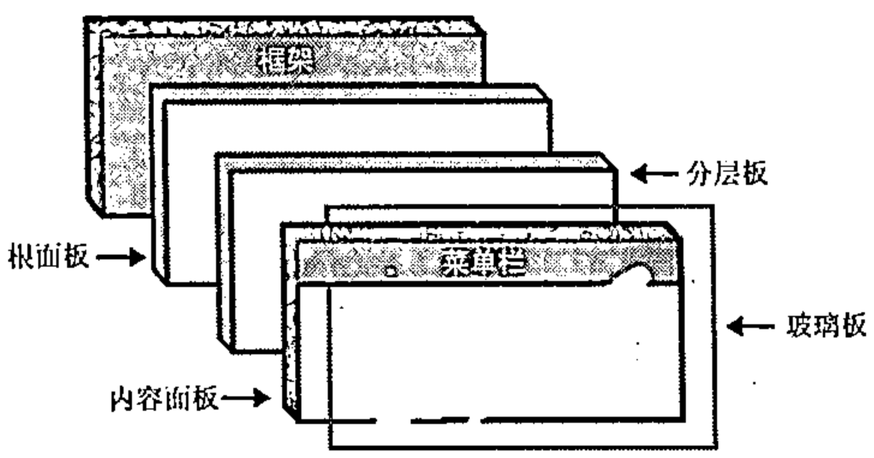
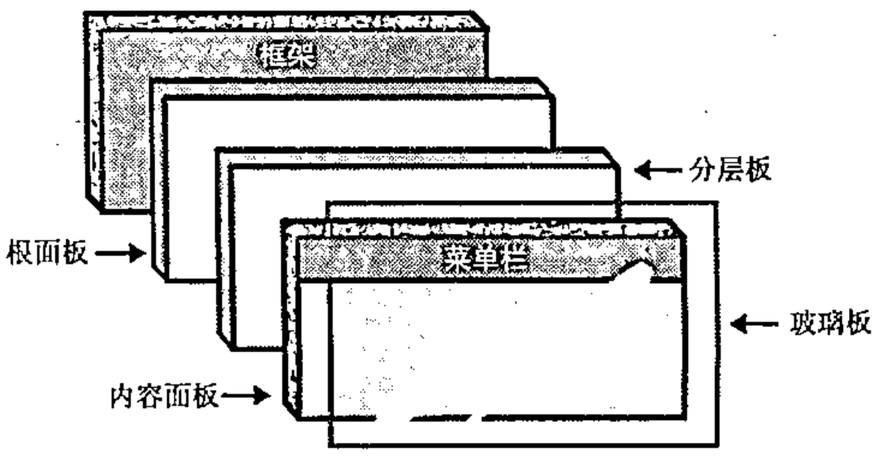
（续）

Java界面外观	Windows界面外观
	
JSplitPane	
	
JTabbedPane	
	
J'ToolBar	
	

6. 专用容器（表15-6）

这些专用容器在UI中扮演特殊角色。

表15-6 专用容器

Java界面外观	Windows界面外观
JInternalFrame	
	
JLayeredPane	
	
JRootPane	
	

15.2.2　可插拔的界面外观

Swing工具包允许你决定如何配置应用程序的界面外观。如果你没有指定界面外观，Swing UI管理器则决定使用哪一种。设置界面外观的选项包括：

- 让Swing UI管理器进行选择。如果程序没有指定特定的界面外观，Swing UI管理器检查用户是否指定了首选项。如果没有指定首选项或者首选项不可用，则使用默认的界面外观。默认的界面外观由JRE的提供者决定。对于Sun公司提供的JRE，就使用Java界面外观（称为Metal）。Java界面外观适用于所有平台。
- 使用本机平台的界面外观。如果应用程序运行在Microsoft Windows XP计算机上，则使用Windows界面外观。在Mac OS平台上，使用Aqua界面外观。在UNIX平台上（比如Solaris或者Linux），根据用户的桌面选择决定使用GTK+界面外观，还是CDE/Motif界面外观。
- 指定特定的界面外观。Swing发布了4种界面外观：Java（也称为Metal）、Microsoft Windows、GTK+和CDE / Motif。GTK+界面外观要求指定一个主题，并且有很多可以在因特网上免费下载。
- 使用Synth包创建你自己的界面外观。
- 使用外部提供的界面外观。

476

如图15-4到图15-8所示，PasswordStore有三种界面外观可供选择。感谢Incors[1]提供Alloy和Synthetica界面外观。

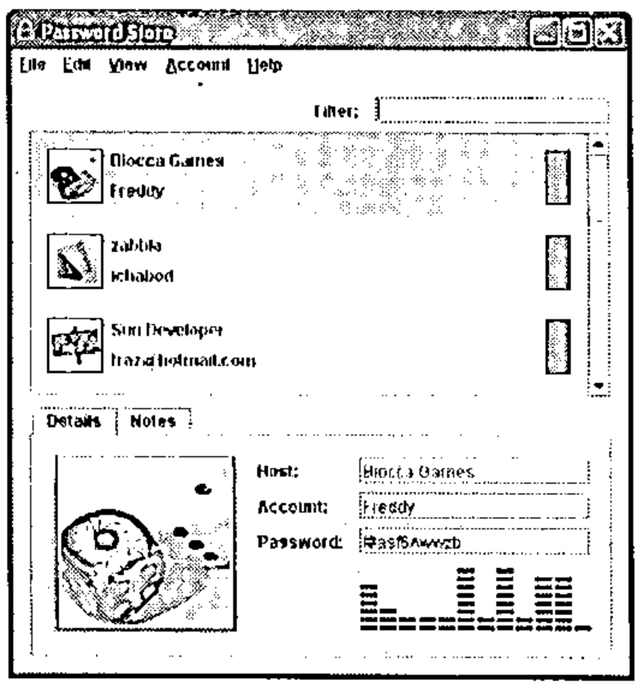


图 15-4　Java 界面外观

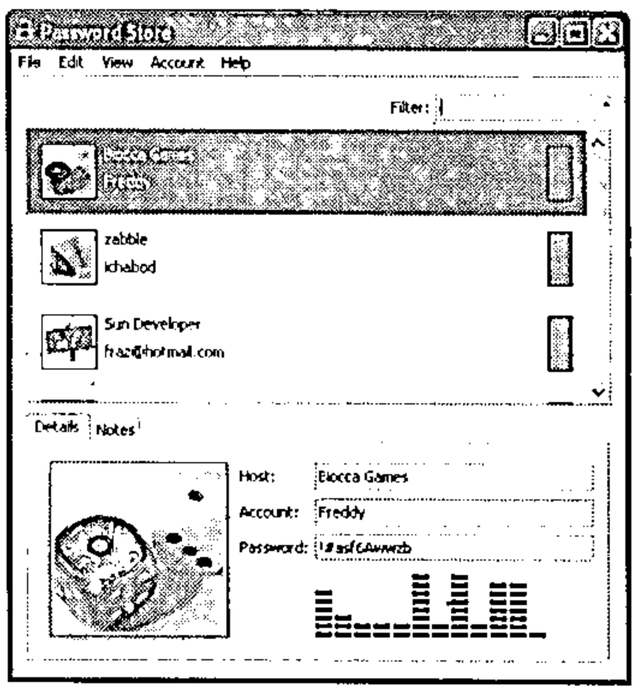


图 15-5　Windows 界面外观

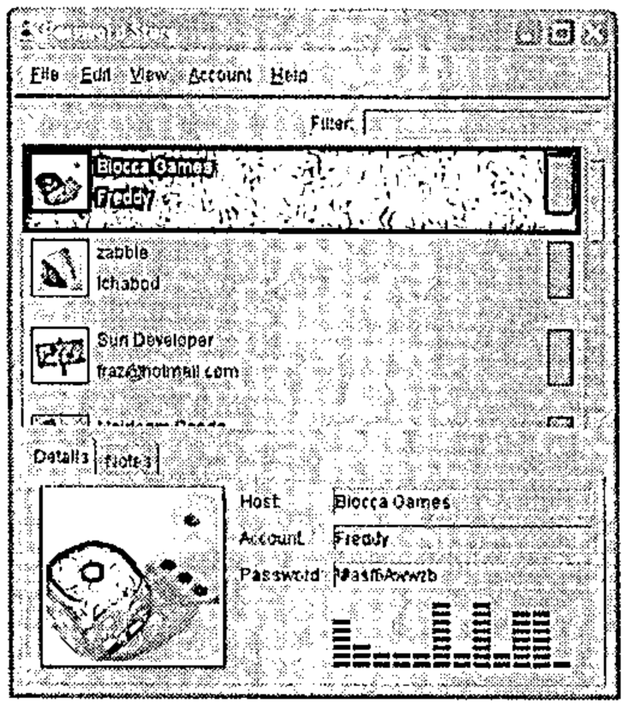


图15-6　CDE / Motif界面外观

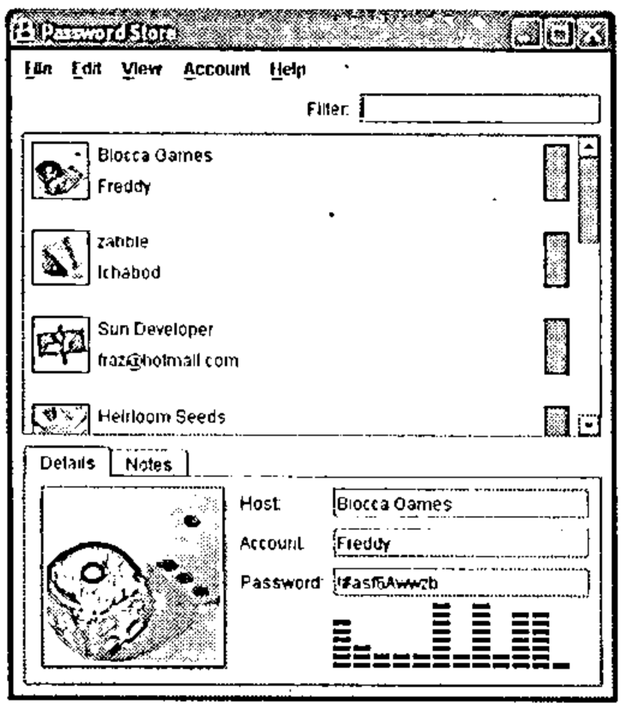


图15-7　默认的Alloy界面外观

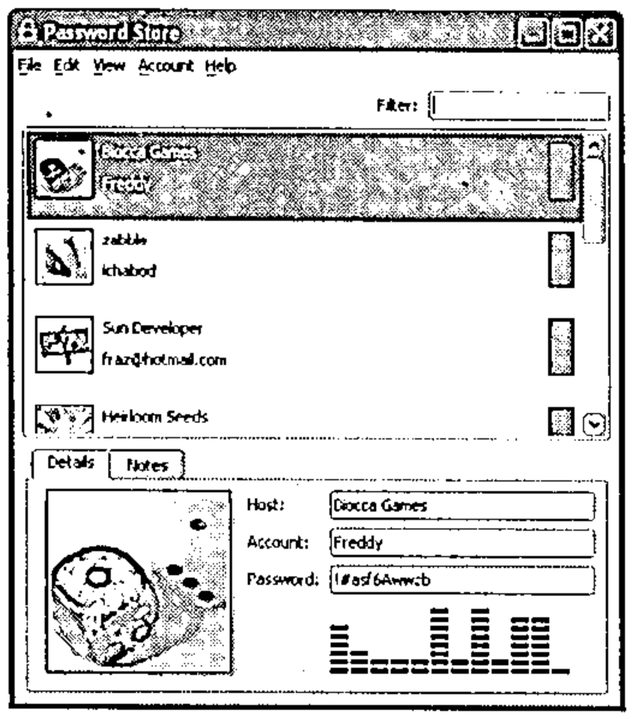


图15-8　Synthetica界面外观

477
~
479

1. http://www.incors.com/lookandfeel/

15.2.3 拖放和数据传输

Swing工具包支持在同一个Java应用程序内的组件之间，在不同Java应用程序之间，以及在Java和本机应用程序之间传输数据。可以通过拖动方式传输数据，或者通过剪贴板使用剪切、复制和粘贴传输数据。

1. 拖放

可以很容易地启用很多Swing组件的拖放支持(有时候只需要一行代码)。例如，启用JTable(Swing的表组件）的拖放、复制和粘贴支持非常简单。需要提供的只是表示选择的数据和如何从剪贴板获得数据——就这么简单！

2. 剪切、复制和粘贴

大多数基于文本的组件（比如编辑器板和文本字段）都支持剪切 / 复制和粘贴到组件外。当然，需要创建菜单项并且“连接”到适当的操作。其他组件（比如列表和树）通过很少量的处理就可以支持剪切、复制和粘贴。

PasswordStore支持各种方式的数据传输：

- 列表和树视图中的文本支持剪切、复制和粘贴。
- Details面板中的文本字段、Filter文本字段和Notes文本板支持剪切 / 复制、粘贴和拖放。
- Details面板中的公司图标区域接受拖放过来的图像（jpg、png、gif或者tif)。

15.2.4 国际化和本地化

国际化（internationalization）过程是设计应用程序，使用户可以使用他的文化首选项运行应用程序，而无需修改或者重新编译代码。这些文化首选项统称为地区（locale)，包括（但是不限于）语言、货币格式、时间和日期格式以及数字格式。

国际化程序被设计为把文本元素（比如状态消息和GUI组件标签）存储在源代码之外的资源包中，并且动态获得。把地区特定的信息和代码分离，是允许程序用不同语言和不同首选项运行且无需重新编译的途径。

480

本地化（localization）过程把文本翻译为特定语言，并且添加任何地区特定的组件。把应用程序本地化为一种语言，然后在这个地区运行应用程序时，Swing从资源包获得本地化的字符串，并且布局管理器相应地改变组件的大小。

例如，使用英语的人编写了一个应用程序，遵守国际化规则；然后，应用程序被本地化为日语和西班牙语。当用户把语言系统首选项（Language System Preference）集合设置为日语并且运行应用程序时，Swing会探测到这一情况。当应用程序出现时，菜单、标签、按钮等都显示日语文本，并且相应地改变组件的大小。如果这个用户退出程序，把语言系统首选项设置为西班牙语，然后重新启动应用程序，它就显示西班牙语，并且按照新的字符集相应地改变组件大小。

Swing的布局管理器知道地区设置如何影响UI——不必为每个地区创建新的布局。例如，在从右到左书写文字的地区，布局管理器会按照相同的方向安排组件（如果指定的话)。也支持*Bidi*文本（混合方向的文本，例如希伯来人和阿拉伯人使用的)。

设计每个程序时都应该时刻考虑到国际化问题：GUI组件标签、状态消息、货币值、日期、电话号码和地址格式都不应该硬编码到程序中。程序经过国际化之后，语言专家可以在以后进行实际的翻译工作，无需任何重新编译处理。

如图15-9和图15-10所示，PasswordStore被本地化为日语和阿拉伯语。

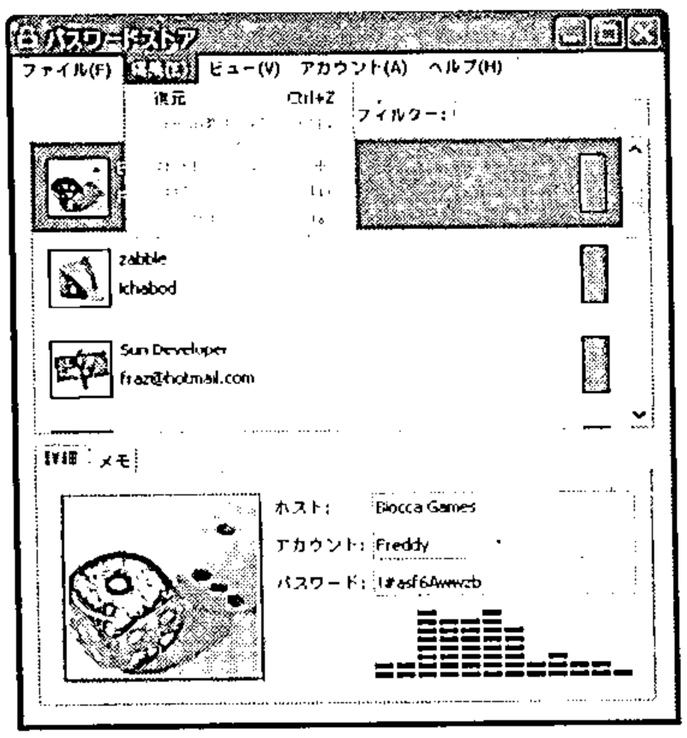


图 15-9　使用日语的 PasswordStore

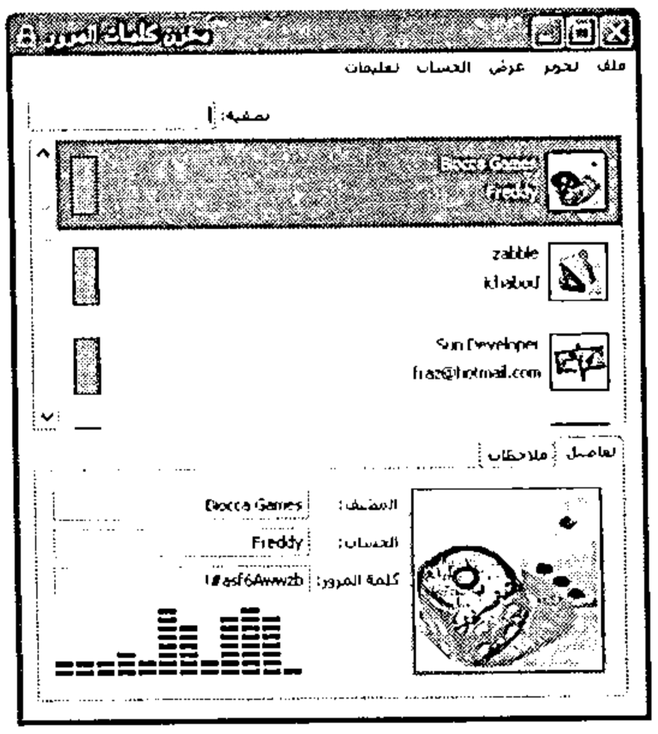


图 15-10　使用阿拉伯语的 PasswordStore

15.2.5　可访问性

辅助技术使永久或者临时残疾的用户也可以使用计算机。这包括范围很广的技术和设备——语音界面、放大镜、屏幕阅读器、Closed Caption字幕、键盘增强等。在很多国家和地区，包括美国、澳大利亚、加拿大和欧盟，都有要求程序功能平滑使用辅助技术的法律[1]。

所有Swing组件都内置了一定程度的可访问性，但是可以遵照某些简单原则达到完全可访问性。例如，尽可能使用工具提示、键盘替换方式和图像的文本描述。

PasswordStore演示遵守可访问性原则。在图15-11中，你能看到工具提示文本的例子。

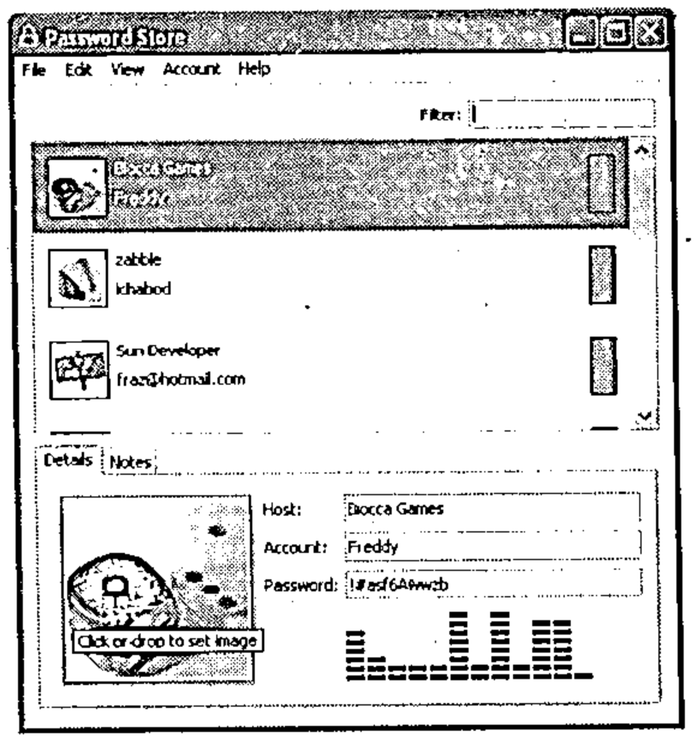


图15-11　带有工具提示的PasswordStore

15.2.6　和桌面集成

Java SE 6引入的Desktop API[2]使Java应用程序可以无缝地和桌面集成。支持三种类型的集成：

- 启动主机系统的默认浏览器打开指定的统一资源标识符（Uniform Resource Identifier，URI）的能力。
- 启动主机系统的默认电子邮件客户程序的能力。
- 启动应用程序打开、编辑或者输出与这些应用程序相关联的文件的能力。

在PasswordStore演示程序的Notes文本板中可以看到这种情况。点击图15-12中显示的链接，就会在默认浏览器中打开指定的URI。

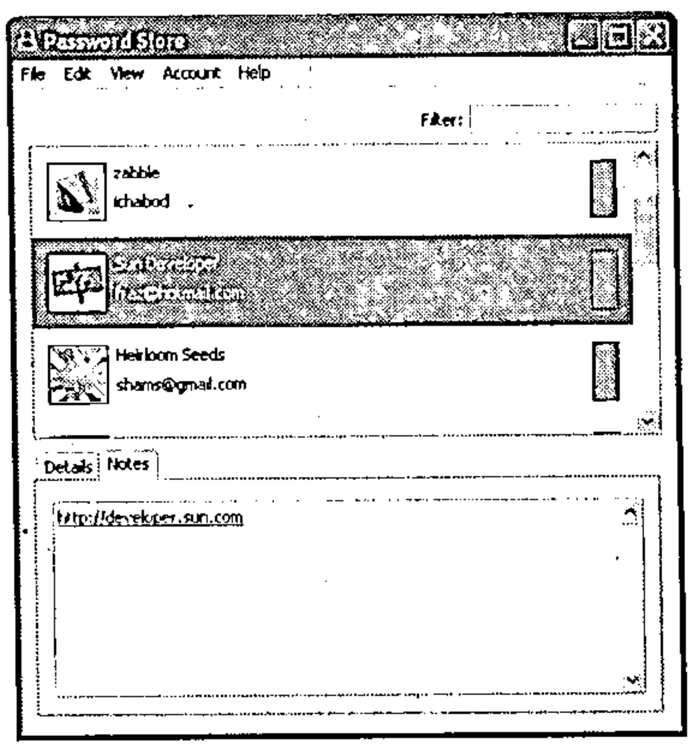


图15-12　点击URI，它会在默认浏览器中打开

1. 关于更多信息，请参考Sun公司的可访问性程序相关法律（Accessibility Program Relevant Laws）：http://www.sun.com/access/background/laws.html
2. docs/api/java/awt/Desktop.html

关于更多信息，请参阅*Using the Desktop API in Java SE 6*一文[1]。

15.2.7 系统托盘图标支持

一些平台（比如Microsoft Windows）的桌面包含系统托盘（system tray），如图15-13所示。



图15-13 Windows XP系统上的系统托盘

在Microsoft Windows系统中，它被称为“工具栏状态区域（Taskbar Status Area）”；在Gnome系统中是“通知区域（Notification Area）”；在KDE中是“系统托盘（System Tray）”。不管它叫什么，系统托盘由所有应用程序共享。

在支持系统托盘的系统上，应用程序可以把小型图标（称为托盘图标（tray icon））插入系统托盘。这个图标可以用于通知用户应用程序状态的改变，或者需要采取特别操作。点击托盘图标可以打开应用程序窗口。也可以把弹出菜单和工具提示附加到托盘图标。

481
~
484

Java SE 6中添加了系统托盘支持[2]。关于更多信息，请参阅*New System Tray Functionality in Java SE 6*一文[3]。

问题和练习：图形化用户界面

问题

1. Swing支持多种界面外观吗？
2. 是或否：Java界面外观是唯一一种可以跨越所有平台工作的界面外观。
3. 是或否：Swing的撤销框架（Undo Framework）支持撤销（和重做）数量不限的操作。
4. Swing的列表组件（JList）只能显示文本吗？
5. 提供开箱即用的重做支持的Swing组件是什么？
6. Java应用程序能够和本机桌面组件集成吗？
7. 可以在Web上部署Swing应用程序吗？

答案

可以在以下位置找到“问题”和“练习”的答案：

```
tutorial/ui/features/QandE/answers.html
```

485

1. http://java.sun.com/developer/technicalArticles/J2SE/Desktop/mustang/desktop_api/
2. docs/api/java/awt/SystemTray.html
3. http://java.sun.com/developer/technicalArticles/J2SE/Desktop/mustang/systemtray/

第16章 把程序打包到JAR文件中

本章和后面两章讨论Java应用程序和applet的部署。

Java版本1.4.2、1.5.0和1.6.0提供了两种部署技术：

- Java Web Start，用于部署Java应用程序。
- Java Plug-in，用于部署Java applet。

当你希望应用程序运行在浏览器中，或者应用程序和Web页面内容紧密集成时，我们建议使用Java Plug-in部署applet。

当你希望应用程序在用户的桌面上独立运行时，我们建议使用Java Web Start。

Java Web Start要求将应用程序打包到JAR文件中，本章将对此进行讲解。

Java档案（Java Archive，JAR）文件可以把多个文件打包到单一档案文件中。典型情况下，JAR文件包含与applet和应用程序相关联的类文件和辅助资源。

JAR文件格式提供很多优势：

- **安全**——你可以数字化签名JAR文件的内容。识别你签名的用户可以选择性地给你的软件授予（通过其他方式所不能获得的）安全许可。
- **减少下载时间**——如果你的applet打包到JAR文件中，那么applet的类文件和相关资源可以在单一HTTP事务中下载到浏览器，无需为每个文件打开新的链接。
- **压缩**——JAR格式允许你压缩文件，以便进行高效存储。
- **对扩展进行打包**——扩展框架提供一种途径，通过它可以把功能添加到Java核心平台，而JAR文件格式定义了对扩展进行打包的方式。Java 3D[1]和JavaMail[2]是Sun开发的扩展的例子。使用JAR文件格式，可以把自己的软件转换为扩展。
- **包密封**——存储在JAR文件中的包可以选择性地进行密封，让包可以实施版本一致性。密封JAR文件中的包意味着，包中定义的所有类都必须在同一个JAR文件中。
- **包版本**——JAR文件可以保存和它包含的文件有关的数据，比如厂商和版本信息。
- **可移植性**——处理JAR文件的机制是Java平台核心API的标准组成部分。

Java开发工具包（Java Development Kit）的文档包含Jar工具的信息：

- Java档案文件指南（Java Archive（JAR）Files Guide）[3]。
- JAR文件规范（JAR File Specification）[4]。

16.1 使用JAR文件：基础

JAR文件以ZIP文件格式进行打包，可以使用它们执行无损数据压缩、存档、解压缩和档案

1. http://java.sun.com/products/java-media/3D
2. http://java.sun.com/products/javamail/
3. docs/technotes/guides/jar/index.html
4. docs/technotes/guides/jar/jar.html

解包的任务。这些任务属于最常见的JAR应用，仅仅使用这些基本特性，就可以认识到JAR文件的很多优势。

488

如果你希望利用JAR文件格式提供的高级功能（比如电子签名），那么首先需要熟悉基础操作。

为了使用JAR文件执行基本任务，可以使用Java开发工具包提供的Java档案工具（Java Archive Tool）。因为使用jar命令调用Java档案工具，所以本书把它称为“Jar工具”。

作为这一小节中将介绍的一些主题的大纲和预览，表16-1总结了常见的JAR文件操作。

表16-1 常见的JAR文件操作

操 作	命 令
创建JAR文件	`jar cf jar-file input-file(s)`
查看JAR文件的内容	`jar tf jar-file`
提取JAR文件的内容	`jar xf jar-file`
提取JAR文件中的特定文件	`jar xf jar-file archived-file(s)`
运行打包为JAR文件的应用程序（需要Main-class* 清单头）	`java -jar app.jar`
调用打包为JAR文件的applet	`<applet code=AppletClassName.class archive="JarFileName.jar" width=width height= height></applet>`

* tutorial/deployment/jar/appman.html

本节讲解如何执行最常见的JAR文件操作，每个基本特性都带有示例。

Java开发工具包的文档包含Jar工具的参考页面：

- 用于Windows平台的Jar工具参考[1]。

489

- 用于Solaris平台的Jar工具参考[2]。

16.1.1 创建 JAR 文件

创建JAR文件的命令的基本格式是：

```
jar cf jar-file input-file(s)
```

这个命令中使用的选项和实参是：

- c选项表示希望创建（create）JAR文件。
- f选项表示希望输出到文件（file），而不是到标准输出。
- *jar-file*是希望创建的JAR文件的名称。可以把JAR文件命名为任何名称。按照约定，JAR文件名被加上.jar扩展名，但这不是必须的。
- *input-file(s)*实参是你希望添加到JAR文件中的以空格分隔的一个或者多个文件的清单。*input-file(s)*实参可以包含通配符*符号。如果任何“输入文件”是目录，那么这些目录的内容就被递归地添加到JAR档案中。

c和f选项出现的前后顺序没关系，但是它们之间绝不能有任何空格。

这个命令将创建压缩的JAR文件并且存放到当前目录中。命令还将创建JAR档案的默认清单（manifest）文件（见16.2.1节）。

---

1. docs/technotes/tools/windows/jar.html
2. docs/technotes/tools/solaris/jar.html

注意　JAR文件中的元数据（比如条目名称、注释和清单的内容）必须以UTF8编码。

可以把表16-2所示的附加选项添加到基本命令的cf选项中。

表16-2　Java命令选项

选　项	描　述
v	构造JAR文件时把详细（verbose）输出发送到标准输出。详细输出告诉你添加到JAR文件的每个文件的名称
0（零）	表示不希望压缩JAR文件
M	表示不生成默认的清单文件
m	用于包含从现有的清单文件获得的清单信息。使用这个选项的格式是： `jar cmf existing-manifest jar-file input-file(s)` 关于这个选项的更多信息，请参阅16.2.2节。 **警告**　清单必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。
-C	在命令的执行过程中改变目录。参阅16.1.1节的第1小节

注意　当创建JAR文件时，创建的时间存储在JAR文件中。因此，即使JAR文件的内容没有改变，多次创建JAR文件时，产生的文件也不会完全相同。因此在构造环境中使用JAR文件时应该意识到这一点。我们推荐你使用清单文件中的版本信息控制JAR文件的版本，而不要使用创建时间；参见16.2.5节。

示例

我们来看一个示例，它是一个简单的applet——TicTacToe。这个演示程序包含字节码类文件、音频文件和图像，它们的结构如图16-1所示。

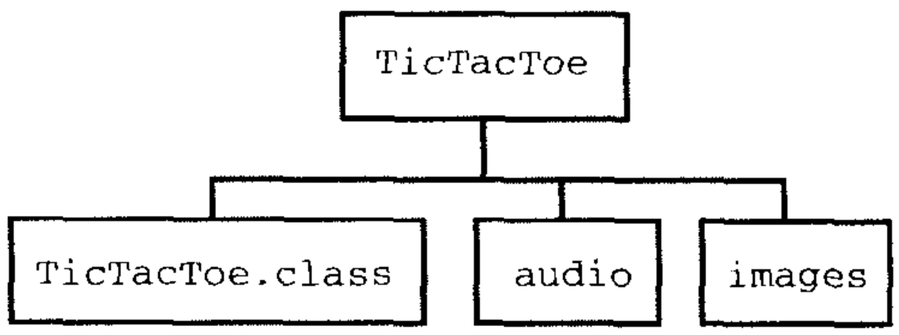


图16-1　TicTacToe文件夹层次结构

audio和images子目录包含这个applet使用的音频文件和GIF图像。

490 ~ 491

你可以从*jar/examples*目录[1]获得这些文件。为了把这个演示程序打包到一个名为TicTacToe.jar的JAR文件中，你需要在TicTacToe目录内运行这个命令：

```
jar cvf TicTacToe.jar TicTacToe.class audio images
```

实参audio和images代表目录，所以Jar工具将递归地把它们和它们的内容存放到JAR文件中。生成的JAR文件TicTacToe.jar将存放在当前目录中。因为命令使用了v选项要求详细输出，所以在运行命令时你会看到类似这样的内容：

```
adding: TicTacToe.class (in=3825) (out=2222) (deflated 41%)
```

1. tutorial/deployment/jar/examples/TicTacToe.java

```
adding: audio/ (in=0) (out=0) (stored 0%)
adding: audio/beep.au (in=4032) (out=3572) (deflated 11%)
adding: audio/ding.au (in=2566) (out=2055) (deflated 19%)
adding: audio/return.au (in=6558) (out=4401) (deflated 32%)
adding: audio/yahoo1.au (in=7834) (out=6985) (deflated 10%)
adding: audio/yahoo2.au (in=7463) (out=4607) (deflated 38%)
adding: images/ (in=0) (out=0) (stored 0%)
adding: images/cross.gif (in=157) (out=160) (deflated -1%)
adding: images/not.gif (in=158) (out=161) (deflated -1%)
```

从这些输出可以看到，JAR文件TicTacToe.jar被压缩了。默认情况下，Jar工具会压缩文件。可以使用0（零）选项关闭压缩特性，命令就像这样：

```
jar cvf0 TicTacToe.jar TicTacToe.class audio images
```

举例来说，为了加快浏览器加载JAR文件的速度，你可能希望避免压缩。总的来说，未压缩的JAR文件比压缩文件加载得更快，因为加载过程中不需要解压缩。但是，对于比较大的未压缩文件来说，付出的代价是通过网络的下载时间可能更长。

Jar工具可以接受使用通配符*符号的参数。只要TicTacToe目录中没有任何不想要的文件，就可以使用下面的替代命令构造JAR文件：

```
jar cvf TicTacToe.jar *
```

虽然详细输出并未指出，但是Jar工具自动地把清单文件添加到JAR档案中，清单文件的路径名称为META-INF/MANIFEST.MF。关于清单文件的更多信息，请参阅16.2节。

在前面的示例中，档案中的文件保持它们的相对路径名称和目录结构。Jar工具提供-C选项，可以创建不保持存档的文件的相对路径的JAR文件。它模仿的是TAR的-C选项。

举个例子，假设你希望把TicTacToe使用的音频文件和gif文件添加到JAR文件中，并且希望所有文件都位于最高级别，不带有目录层次结构。为了达到这个目的，你可以从images和audio目录的上层目录使用这个命令：

```
jar cf ImageAudio.jar -C images . -C audio .
```

这个命令的-C images部分指示Jar工具进入images目录，-C images后面的.指示Jar工具存档这个目录的全部内容。然后，命令的-C audio.部分对audio目录进行同样的处理。这样产生的JAR文件就具有如下内容：

```
META-INF/MANIFEST.MF
cross.gif
not.gif
beep.au
ding.au
return.au
yahoo1.au
yahoo2.au
```

相反，假设你使用的命令不包含-C选项：

```
jar cf ImageAudio.jar images audio
```

那么产生的JAR就具有这样的内容：

```
META-INF/MANIFEST.MF
images/cross.gif
images/not.gif
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
```

16.1.2　查看JAR文件的内容

查看JAR文件内容的命令的基本格式是：

```
jar tf jar-file
```

我们来分析这个命令中使用的选项和实参：

493

- t选项表示希望查看JAR文件的目录（table of contents）。
- f选项表示从命令行指定要查看其内容的JAR文件。
- jar-file实参是要查看其内容的JAR文件的路径和名称。

t和f选项的前后顺序没关系，但是它们之间绝不能有任何空格。

这个命令将把JAR文件的目录显示到标准输出。

你可以选择添加详细选项v，以便在输出中显示与文件长度和最后修改日期有关的额外信息。

示例

我们来使用Jar工具列出前一个小节创建的TicTacToe.jar文件的内容：

```
jar tf TicTacToe.jar
```

这个命令把JAR文件的内容发送到标准输出：

```
META-INF/MANIFEST.MF
TicTacToe.class
audio/
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
images/
images/cross.gif
images/not.gif
```

正如预期，这个JAR文件包含TicTacToe类文件以及音频和图像目录。输出还显示JAR文件包含默认的清单文件META-INF/MANIFEST.MF，它是由Jar工具自动存放在档案中的。关于更多信息，请参阅16.2.1节。

使用正向斜线显示所有路径名称，无论你使用的是什么平台或者操作系统。JAR文件中的路径总是相对路径；你永远都不会在路径开头看到C：。

如果你使用v选项，Jar工具就会显示附加信息：

```
jar tvf TicTacToe.jar
```

494　例如，JAR文件TicTacToe的详细输出类似这样：

```
 256 Mon Apr 18 10:50:28 PDT 2005 META-INF/MANIFEST.MF
3885 Mon Apr 18 10:49:50 PDT 2005 TicTacToe.class
   0 Wed Apr 20 16:39:32 PDT 2005 audio/
4032 Wed Apr 20 16:39:32 PDT 2005 audio/beep.au
2566 Wed Apr 20 16:39:32 PDT 2005 audio/ding.au
6558 Wed Apr 20 16:39:32 PDT 2005 audio/return.au
7834 Wed Apr 20 16:39:32 PDT 2005 audio/yahoo1.au
7463 Wed Apr 20 16:39:32 PDT 2005 audio/yahoo2.au
   0 Wed Apr 20 16:39:44 PDT 2005 images/
 157 Wed Apr 20 16:39:44 PDT 2005 images/cross.gif
 158 Wed Apr 20 16:39:44 PDT 2005 images/not.gif
```

16.1.3　提取JAR文件的内容

提取JAR文件内容的基本命令是：

```
jar xf jar-file [archived-file(s)]
```

我们来看看这个命令中的选项和实参：

- x选项表示希望从JAR档案提取（extract）文件。
- f选项表示在命令行指定要从中提取文件的JAR文件（file），而不是通过标准输入指定。
- jar-file是从中提取文件的JAR文件的文件名（或者路径和文件名）。
- archived-file(s)实参是用空格分隔的要从档案提取的文件的清单。如果没有这个实参，Jar工具就提取档案中的所有文件。

和以前一样，x和f选项出现在命令中的前后顺序没关系，但是它们之间绝不能有空格。

提取文件时，Jar工具生成想要的文件的副本，并且把它们写入到当前目录中，重新生成档案中文件的目录结构。原始的JAR文件保持不变。

警告 提取文件时，Jar工具会覆盖和被提取文件的路径名称相同的任何现有文件。

示例

我们从前几个小节使用过的JAR文件TicTacToe中提取一些文件。回忆一下，TicTacToe.jar的内容是：

```
META-INF/MANIFEST.MF
TicTacToe.class
audio/
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
images/
images/cross.gif
images/not.gif
```

假设你希望提取TicTacToe类文件和cross.gif图像文件。为了达到这个目的，可以使用这个命令：

```
jar xf TicTacToe.jar TicTacToe.class images/cross.gif
```

这个命令完成两个工作：

- 它把TicTacToe.class的副本存放到当前目录。
- 如果没有images目录，它就创建这个目录，并且把cross.gif存放到里面。

原始的JAR文件TicTacToe保持不变。

可以通过相同的方式从JAR文件提取任意数量的文件。当命令没有指定要提取哪些文件时，Jar工具就会提取档案中的所有文件。例如，可以使用这个命令提取TicTacToe档案中的所有文件：

```
jar xf TicTacToe.jar
```

16.1.4 更新 JAR 文件

Jar工具提供u选项。通过修改JAR文件的清单，或者添加文件，可以使用u选项更新现有JAR文件的内容。

添加文件的基本命令使用这样的格式：

```
jar uf jar-file input-file(s)
```

在这个命令中：

- u选项表示希望更新（update）现有的JAR文件。
- f选项表示从命令行指定要更新的JAR文件。
- jar-file是要更新的现有JAR文件。
- input-file(s)是以空格分隔的希望添加到JAR文件的一个或者多个文件的清单。

496

档案中和要添加的文件的路径名称相同的任何文件都会被覆盖。

创建新的JAR文件时，你可以选择使用-C选项指示目录的改动。关于更多信息，请参阅16.1.1节。

示例

回顾一下，TicTacToe.jar有如下内容：

```
META-INF/MANIFEST.MF
TicTacToe.class
audio/
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
images/
images/cross.gif
images/not.gif
```

假设你希望把文件images/new.gif添加到JAR文件中。可以在images目录的上层目录使用这个命令完成这一工作：

```
jar uf TicTacToe.jar images/new.gif
```

修改后的JAR文件将包含如下内容：

```
META-INF/MANIFEST.MF
TicTacToe.class
audio/
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
images/
images/cross.gif
images/not.gif
images/new.gif
```

497

可以使用-C选项在命令执行过程中改变目录。例如：

```
jar uf TicTacToe.jar -C images new.gif
```

这个命令在把new.gif添加到JAR文件之前改变images目录。把new.gif添加到档案中时，它的路径名称中不会包含images目录，结果是JAR文件的内容如下：

```
META-INF/MANIFEST.MF
TicTacToe.class
audio/
audio/beep.au
audio/ding.au
audio/return.au
audio/yahoo1.au
audio/yahoo2.au
images/
images/cross.gif
images/not.gif
new.gif
```

16.1.5 运行 JAR 打包的软件

现在你已经了解了如何创建JAR文件，那么如何实际运行打包的代码呢？考虑这三种情况：

- JAR文件包含从浏览器内运行的applet。
- JAR文件包含从命令行调用的应用程序。
- JAR文件包含希望用作扩展的代码。

这一小节讨论前两种情况。Alan Sommerer编写的*Extension Mechanism*[1]讲解如何使用JAR文件作为扩展。

1. 打包在JAR文件中的applet

为了从HTML文件调用任何applet以便在浏览器中运行，需要使用APPLET标记。关于更多信息，请参阅18章。如果applet被打包为JAR文件，所需操作的唯一区别是，使用ARCHIVE参数指定JAR文件的相关路径。

举个例子，我们使用（又一次！）Java开发工具包发布的TicTacToe演示applet。HTML文件中的APPLET标记这样调用演示程序：

498

```
<applet code=TicTacToe.class
  width=120 height=120>
</applet>
```

如果TicTacToe演示程序被打包在名为TicTacToe.jar的JAR文件中，可以通过修改APPLET标记，简单地添加ARCHIVE参数：

```
<applet code=TicTacToe.class
  archive="TicTacToe.jar"
  width=120 height=120>
</applet>
```

ARCHIVE参数指定包含TicTacToe.class的JAR文件的相对路径。这个例子假设JAR和HTML文件在同一个目录中。如果不是，则需要在ARCHIVE的参数值中包含JAR文件的相对路径。例如，如果JAR文件在比HTML文件低一级的一个applets目录中，那么APPLET就像这样：

```
<applet code=TicTacToe.class
  archive="applets/TicTacToe.jar"
  width=120 height=120>
</applet>
```

2. JAR文件作为应用程序

可以使用Java解释器运行JAR打包的应用程序。基本命令是：

```
java -jar jar-file
```

-jar标志通知解释器，应用程序被打包为JAR文件格式。只能指定一个JAR文件，它必须包含应用程序特定的所有代码：

在执行这个命令之前，应确保运行时系统知道JAR文件中的哪个类是应用程序的入口点。

为了指出哪个类是应用程序的入口点，必须把Main-Class头添加到JAR文件的清单。头的形式为：

```
Main-Class: classname
```

头的值classname是应用程序的入口点的类名称。

关于更多信息，请参阅16.2.3节。

499

在清单文件中设置了Main-Class之后，就可以从命令行运行应用程序了：

---

1. tutorial/ext/index.html

```
java -jar app.jar
```

16.2　处理清单文件：基础

JAR文件支持的功能范围广泛，包括电子签名、版本控制、包密封以及其他。是什么赋予了JAR文件如此多的功能？回答是JAR文件的清单（manifest）。

清单是一个特殊的文件，它包含关于JAR文件中打包的文件的信息。通过设置清单包含的这些“元”信息，可以让JAR文件用于各种用途。

这一小节讲解清单文件的内容，以及如何处理它，还有使用基本特性的示例。

清单格式的规范是JDK文档的一部分。

16.2.1　了解默认的清单

创建JAR文件时，它自动获得一个默认的清单文件。档案中只能有一个清单文件，并且总使用这个路径名称：

```
META-INF/MANIFEST.MF
```

创建JAR文件时，默认的清单文件包含如下简单内容：

```
Manifest-Version: 1.0
Created-By: 1.6.0 (Sun Microsystems Inc.)
```

这两行显示清单中的条目采用“头：值”对的形式。头的名称和它的值之间用分号分隔。默认的清单遵守清单规范版本1.0，由JDK版本1.6.0创建。

清单也可以包含关于档案中打包的其他文件的信息。根据所使用JAR文件的目的决定清单中应该记录哪些文件信息。默认的清单不假设它应该记录其他文件的什么信息。

500　默认的清单中不包含摘要信息。为了了解摘要和签名，请参阅16.3节。

16.2.2　修改清单文件

在创建JAR文件的过程中，使用m命令行选项把定制信息添加到清单。本节讨论m选项。

Jar工具自动地把路径名称为META-INF/MANIFEST.MF的默认清单（参见16.2.1节）添加到已创建的所有JAR文件中。通过修改默认的清单，可以启用特殊的JAR文件功能，比如包密封（参见16.2.6节）。典型情况下，修改默认的清单涉及把专用的头（header）添加到清单，以便允许JAR执行特定的功能。

为了修改清单，必须准备一个包含希望添加到清单的信息的文本文件。然后使用Jar工具的m选项把文件中的信息添加到清单。

警告　用于创建清单的文本文件必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。

这个格式的基本命令是：

```
jar cmf jar-file manifest-addition input-file (s)
```

我们来看看这个命令中的选项和实参：

- c选项表示希望创建（create）JAR文件。
- m选项表示希望把现有文件的信息合并（merge）到你要创建的JAR文件的清单文件中。
- f选项表示希望输出到一个文件（file）（你创建的JAR文件），而不是到标准输出。

- *manifest-addition*是一个现有文本文件的名称（或者路径和名称），希望把这个文件的内容添加到JAR文件的清单中。
- *jar-file*是你希望得到的JAR文件的名称。
- *input-file(s)*实参是以空格分隔的，你希望存放到JAR文件中的一个或者多个文件的清单。

m和f选项必须和相应的实参顺序相同。

501

注意 清单的内容必须用UTF-8编码。

这一小节剩余部分的演示可以对清单文件作出特定的修改。

16.2.3 设置应用程序的入口点

如果把应用程序打包在JAR文件中，就需要通过某种途径指出JAR文件内的哪个类是应用程序的入口点。使用清单中的Main-Class头提供这一信息，它的一般形式是：

```
Main-Class: classname
```

*classname*的值是应用程序的入口点的类名称。

回忆一下，入口点是具有签名为public static void main（String[] args）的方法的类。

在清单中设置了Main-Class头之后，可以使用下面的java命令形式运行JAR文件：

```
java -jar JAR-name
```

Main-Class头中指定的类的main方法会被执行。

1. 示例

我们希望在运行JAR文件时，执行包MyPackage中的MyClass类的main方法。

我们首先创建一个名为Manifest.txt的文本文件，其内容如下：

```
Main-Class: MyPackage.MyClass
```

警告 这个文本文件必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。

当创建名为MyJar.jar的JAR文件时，使用如下命令：

```
jar cfm MyJar.jar Manifest.txt MyPackage/*.class
```

502

这样创建的JAR文件带有的清单具有如下内容：

```
Manifest-Version: 1.0
Created-By: 1.6.0 (Sun Microsystems Inc.)
Main-Class: MyPackage.MyClass
```

使用如下命令运行JAR文件时，就会执行MyClass的main方法：

```
java -jar MyJar.jar
```

2. 使用JAR工具设置入口点

e标志（表示“入口点（entrypoint）”）创建或者覆盖清单的Main-Class属性。在创建或者更新JAR文件时可以使用它。使用它指定应用程序的入口点，而不必编辑或者创建清单文件。

例如，这个命令创建app.jar，其中清单的Main-Class属性被设置为MyApp：

```
jar cfe app.jar MyApp MyApp.class
```

你可以使用下面的命令直接调用这个应用程序：

```
java -jar app.jar
```

如果入口点的类名称在包中，可以使用“.”（点号）字符作为分界符。例如，如果Main.class在一个名为foo的包中，可使用下面的方式指定入口点：

```
jar -cfe Main.jar foo.Main foo/Main.class
```

16.2.4　把类添加到JAR文件的类路径

你可能需要从一个JAR文件内引用另一个JAR文件中的类。

例如一种典型情况是：一个打包到JAR文件中的applet的清单引用另一个JAR文件（或者若干不同的JAR文件），applet使用其他这些JAR文件作为工具。

你需要在applet或者应用程序的清单文件的Class-Path头字段中指定要包含的类。Class-Path头采用如下形式：

```
Class-Path: jar1-name jar2-name directory-name/jar3-name
```

通过在清单中使用Class-Path头，可以避免在调用Java运行你的应用程序时指定长的
503 -classpath标志：

注意　Class-Path头指向本地网络上的类或者JAR文件，而不是JAR文件中的JAR文件或者可以通过网际协议访问的类。为了把JAR文件内的JAR文件中的类加载到类路径，你必须编写定制的代码去加载这些类。例如，如果MyJar.jar包含另一个JAR文件MyUtils.jar，则不能使用MyJar.jar的清单中的Class-Path头把MyUtils.jar加载到类路径。

示例

我们希望加载MyUtils.jar中的类以便在MyJar.jar中使用。这两个JAR文件在同一个目录中。

首先我们创建一个名为Manifest.txt的文本文件，其内容如下：

```
Class-Path: MyUtils.jar
```

警告　这个文本文件必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。

然后，我们使用下面的命令创建名为MyJar.jar的JAR文件：

```
jar cfm MyJar.jar Manifest.txt MyPackage/*.class
```

创建出的JAR文件的清单的内容如下：

```
Manifest-Version: 1.0
Class-Path: MyUtils.jar
Created-By: 1.6.0 (Sun Microsystems Inc.)
```

现在，当你运行MyJar.jar时，会把MyUtils.jar中的类加载到类路径中。

16.2.5　设置包版本信息

你可能希望在JAR文件的清单中包含包版本信息。在清单中使用表16-3列出的头来提供这
504 些信息。

表16-3 清单中的头

头	定 义
Name	规范的名称
Specification-Title	规范的标题
Specification-Version	规范的版本
Specification-Vendor	规范的厂商
Implementation-Title	实现的标题
Implementation-Version	实现的构造编号
Implementation-Vendor	实现的厂商

可以为每个包分配一组这样的头。版本信息头应该直接出现在包的Name头下面。这个例子显示所有版本信息头：

```
Name: java/util/
Specification-Title: Java Utility Classes
Specification-Version: 1.2
Specification-Vendor: Sun Microsystems, Inc.
Implementation-Title: java.util
Implementation-Version: build57
Implementation-Vendor: Sun Microsystems, Inc.
```

关于包版本头的更多信息，请参阅包版本规范（Package Versioning specification）[1]。

示例

我们希望在前面MyJar.jar的清单例子中包含头。

首先，我们创建一个名为Manifest.txt的文本文件，其内容如下：

```
Name: java/util/
Specification-Title: Java Utility Classes
Specification-Version: 1.2
Specification-Vendor: Sun Microsystems, Inc.
Implementation-Title: java.util
Implementation-Version: build57
Implementation-Vendor: Sun Microsystems, Inc.
```

警告 这个文本文件必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。

505

然后，使用如下命令创建名为MyJar.jar的JAR文件：

```
jar cmf MyJar.jar Manifest.txt MyPackage/*.class
```

这样创建的JAR带有的清单具有如下内容：

```
Manifest-Version: 1.0
Created-By: 1.6.0 (Sun Microsystems Inc.)
Name: java/util/
Specification-Title: Java Utility Classes
Specification-Version: 1.2
Specification-Vendor: Sun Microsystems, Inc.
Implementation-Title: java.util
Implementation-Version: build57
Implementation-Vendor: Sun Microsystems, Inc.
```

1. docs/technotes/guides/versioning/spec/versioning2.html

16.2.6　在JAR文件中密封包

可以选择性地密封JAR文件中的包，这意味着这个包中定义的所有类都必须存档在同一个JAR文件中。例如，可能你希望密封包，以便确保软件中的类之间的版本一致性。

通过把Sealed头添加到清单从而密封JAR文件中的包，Sealed头的一般形式是：

```
Name: myCompany/myPackage/
Sealed: true
```

myCompany/myPackage/的值是要密封的包的名称。

注意，包的名称必须以“/”结束。

1. 示例

我们希望在JAR文件MyJar.jar中密封两个包：firstPackage和secondPackage。

首先，我们创建一个名为Manifest.txt的文本文件，其内容如下：

```
Name: myCompany/firstPackage/
Sealed: true

Name: myCompany/secondPackage/
Sealed: true
```

506

警告　这个文本文件必须以换行或者回车结束。如果最后一行没有以换行或者回车结束，就不能被正确解析。

然后，我们使用下面的命令创建名为MyJar.jar的JAR文件：

```
jar cmf MyJar.jar Manifest.txt MyPackage/*.class
```

这样创建的JAR文件的清单具有如下内容：

```
Manifest-Version: 1.0
Created-By: 1.6.0 (Sun Microsystems Inc.)
Name: myCompany/firstPackage/
Sealed: true
Name: myCompany/secondPackage/
Sealed: true
```

2. 密封JAR文件

如果你希望确保一个包中的所有类都来自于相同的代码源，那么可以使用JAR密封。密封的JAR指定这个JAR定义的所有包都是密封的，除非在每个包的基础上覆盖这个设置。

为了密封JAR文件，需要将清单头Sealed的值设置为true。例如：

```
Sealed: true
```

指定这个档案文件中的所有包都是密封的，除非使用清单条目中的Sealed属性覆盖特定的包。

关于如何密封JAR文件的进一步信息，请参阅JDC TechTips[1]。

16.3　JAR文件的签名和验证

你可以选择使用自己的电子“签名”签署JAR文件。验证你签名的用户可以授予你的JAR打包的软件安全许可（通常不会具有的）。反过来，你可以验证要使用的带有签名的JAR文件的签名。

1. http://java.sun.com/developer/JDCTechTips/2001/tt0130.html

这一小节介绍如何使用Java开发工具包提供的工具签名和验证JAR文件。 507

16.3.1 了解签名和验证

Java平台允许你数字化地签名JAR文件。数字化地签名文件和用笔签署书面文件的原因一样——让读者知道你写了这个文件，或者至少你批准了这个文件。

例如，当你在一封信上签名时，每个认识你签名的人都能确定是你写了这封信。类似的，当你数字化地签名文件时，任何“认识”你数字签名的人都知道这个文件来自于你。“识别”电子签名的过程被称为验证（verification）。

签名和验证的功能是Java平台的安全架构的重要组成部分。安全由安全策略（policy）控制，在运行时实施。你可以配置策略以便授予applet和应用程序安全许可。

例如，你可以授权一个applet执行通常被禁止的操作，比如读取和写入本地文件，或者运行本地可执行程序。如果你下载的一些代码具有某一可信任实体的签名，那么可以把这一情况作为决定为代码分配哪些安全许可的判断标准。

一旦你（或者你的浏览器）验证了一个applet来自于可信任的来源，就可以让平台放宽安全限制，让这个applet执行一般情况下被禁止的操作。可信的applet具有策略文件（policy file）指定的自由程度。

Java平台通过被称为公共密钥（public key）和私有密钥（private key）的特殊数字进行签名和验证。公共密钥和私有密钥成对出现，它们起相互补充的作用。

私有密钥是你用来签名文件的电子“笔”。就像它的名称所暗示的，你的私有密钥只有你自己知道，所以其他人不可能“伪造”你的签名。使用你的私有密钥签名后的文件只能由相应的公共密钥验证。

但是，只有公共密钥和私有密钥还不足以真正地验证签名。即使你验证了一个签名文件包含匹配的密钥对，仍然需要某些途径来确认公共密钥确实来自它声称的来源。

因此，进行签名和验证工作还需要另一个元素。这个额外的元素是证书（certificate），签名者把它包含在签名后的JAR文件中。证书是来自于公认的证书权威机构的数字化签名的声明，它指出谁拥有特定的公共密钥。证书权威机构是在业界内被广泛信任的实体（典型情况下是专门从事数字安全的公司），它们为密钥和密钥的所有者签署和发布证书。在签名的JAR文件的情况中，证书表示谁拥有这个JAR文件中包含的公共密钥。 508

当你签名JAR文件时，你的公共密钥和相关证书被存放在档案内，以便希望验证你的签名的任何人都可以方便地获得它们。

数字签名的过程如下：

- 签名者使用私有密钥签名JAR文件。
- 相应的公共密钥及其证书存放在JAR文件中，以便希望验证签名的任何人都可以方便地获得它们。

1. 摘要和签名文件

当你签名JAR文件时，档案的清单中为档案中的每个文件添加一个条目（参见16.2节）。下面是这样一个条目的例子：

```
Name: test/classes/ClassOne.class
SHA1-Digest: TD1GZt8G11dXY2p4olSZPc5Rj64=
```

摘要值是签名时文件内容的散列（或者说编码的）表示。只有文件本身变化时，文件的摘要才会变化。

签名JAR文件时，自动产生一个签名文件（signature file）并且存放在JAR文件的META-INF

目录中，同一个目录还包含档案的清单。签名文件的扩展名为.SF。下面是签名文件内容的一个例子：

```
Signature-Version: 1.0
SHA1-Digest-Manifest: h1yS+K9"f7DyIltZrt1+LxvgqaMYM=
Created-By: 1.6.0 (Sun Microsystems Inc.)

Name: test/classes/ClassOne.class
SHA1-Digest: fcav7Sh1G6i86xPepmitOVo4vWY=

Name: test/classes/ClassTwo.class
SHA1-Digest: xrQem9snnPhLySDiZyclM1sFdtM=

Name: test/images/ImageOne.gif
SHA1-Digest: kdHbE7kL9ZHLgK7akHttYV4X1a0=

Name: test/images/ImageTwo.gif
SHA1-Digest: mF0D5zpk68R4oaxEqoS9Q7nhm60=
```

如你所见，签名文件包含档案中文件的摘要条目，看上去类似清单中的摘要一值条目。但是，清单中的摘要值从文件本身计算而来，而签名文件中的摘要值从清单中相应的条目计算而
509 来。签名文件还包含整个清单的摘要值（参见前面示例中的SHA1-Digest- Manifest头）。

验证签名文件时，每个文件的摘要被重新计算并且和清单中的记录进行比较，以便确保JAR文件的内容在签名之后没有改动过。附加的检查是，清单文件本身的摘要值被重新计算，并且和签名文件中记录的值进行比较。

你可以在JDK文档的Manifest Format页面得到关于签名文件的更多信息。

2. 签名块文件

当签名JAR文件时，除了签名文件之外，一个签名块（signature block）文件被自动存放在META-INF目录中。与清单文件或者签名文件不同，签名块文件是人不可阅读的。

签名块文件包含对验证至关重要的两个元素：

- 使用签名者的私有密钥生成的JAR文件的数字签名。
- 包含签名者公共密钥的证书，希望验证这个签名JAR文件的任何人都要使用它。

典型情况下，签名块文件的扩展名为.DSA，表示它们由数字签名算法（Digital Signature Algorithm）创建。如果使用和一些其他标准算法相关联的密钥进行签名，扩展名可能是其他的。

3. 相关文档

关于密钥、证书和证书权威机构的更多信息，请参阅：

- JDK安全工具（JDK Security Tools）[1]。
- X.509证书（X.509 Certificates）[2]。

关于Java平台的安全架构的更多信息，请参阅相关文档：

510
- Java SE中的安全特性（Security Features in Java SE）[3]。
- Java密码扩展（Java Cryptography Extension）的Web站点[4]。
- JDK安全文档（JDK Security Documentation）[5]。

---

1. docs/technotes/tools/
2. docs/technotes/guides/security/cert3.html
3. docs/books/tutorial/security/
4. http://java.sun.com/products/jce/
5. docs/technotes/tools/

16.3.2 签名 JAR 文件

使用JAR签名和验证工具签名JAR文件。使用jarsigner命令调用JAR签名和验证工具，所以我们把它简称为“Jarsigner”。

为了签名JAR文件，必须首先拥有私有密钥。私有密钥和与其相关联的公共密钥证书存放在由密码保护的数据库中，这个数据库称为密钥存储库（keystore）。密钥存储库可以保存很多潜在签名者的密钥。密钥存储库中的每个密钥都可以通过别名（alias）标识，典型情况下别名是拥有密钥的签名者的名字。例如，属于Rita Jones的密钥可能使用别名“rita”。

用于签名JAR文件的命令的基本格式是：

```
jarsigner jar-file alias
```

在这个命令中：

- jar-file是要签名的JAR文件的路径名。
- alias是一个别名，标识要用于签名JAR文件的私有密钥和密钥的相关证书。

Jarsigner会提示你输入密钥存储库和别名的密码。

这个命令的基本形式假设要使用的密钥存储库的文件名为.keystore，存放在你的主目录中。这个命令分别使用名称x.SF和x.DSA创建签名和签名块文件，其中x是别名的前8个字母，全部转换为大写。这个基本命令将使用签名后的文件覆盖原始的JAR文件。

在实际工作中，你可能希望在这个命令中加上表16-4中显示的一个或者多个选项，这些选项必须放在jar-file路径名的前面。

511

表16-4 Jarsigner命令选项

选　项	描　述
-keystore *url*	指定密钥存储库（如果你不使用默认数据库.keystore）
-storepass *password*	允许在命令行输入密钥存储库的密码，而不必通过提示
-keypass *password*	允许在命令行输入别名的密码，而不必通过提示
-sigfile *file*	指定.SF和.DSA的基本名称（如果你不希望采用别名作为基本名称）。*file*只能由大写字母（A-Z）、数字（0-9）、连字符（-）和下划线（_）构成
-signedjar *file*	指定生成的签名JAR文件的名称（如果你不希望签名后的文件覆盖原始的未签名文件）

1. 示例

我们来看一看使用Jarsigner工具签名JAR文件的几个例子。在这些例子中，我们假设：

- 你的别名是“johndoe”。
- 你希望使用的密钥存储库在当前工作目录中一个名为“mykeys”的文件中。
- 密钥存储库的密码是“abc123”。
- 别名的密码是“mypass”。

基于这些假设，可以使用这个命令签名名为app.jar的JAR文件：

```
jarsigner -keystore mykeys -storepass abc123
          -keypass mypass app.jar johndoe
```

因为这个命令没有使用-sigfile选项，所以创建出的.SF和.DSA文件名为JOHNDOE.SF和JOHNDOE.DSA。因为没有使用-signedjar选项，所以生成的签名文件将覆盖原始的app.jar版本。

我们来看看使用不同的选项组合会怎么样：

```
jarsigner -keystore mykeys -sigfile SIG
          -signedjar SignedApp.jar app.jar johndoe
```

512

这一次，会提示你输入密钥存储库和别名的密码，因为没有在命令行指定密码。签名文件和签名块文件将被分别命名为SIG.SF和SIG.DSA，而且签名后的JAR文件SignedApp.jar将存放在当前目录中。未签名的原始JAR保持不变。

2. 其他信息

JAR签名和验证工具的完整参考页面的在线版本在安全工具总结（Summary of Security Tools）[1]中。

16.3.3　验证签名的JAR文件

典型情况下，验证签名后的JAR文件由Java运行时环境（Java Runtime Environment）负责。浏览器会验证它下载的签名applet。解释器使用-jar选项调用的签名应用程序由运行时环境验证。

然而，你可以使用Jarsigner工具自己验证签名的JAR文件。例如，如果希望测试你准备好的签名的JAR文件，就可以这么做。

验证签名的JAR文件的基本命令是：

```
jarsigner -verify jar-file
```

这个命令将验证JAR文件的签名，并且确认档案中的文件从签名之后没有改变过。如果验证成功，你会看到如下消息：

```
jar verified.
```

如果你试图验证未签名的JAR文件，会出现如下消息：

```
jar is unsigned. (signatures missing or not parsable)
```

如果验证失败，会显示相关消息。例如，如果JAR文件的内容在签名之后改变过，验证文件时就会出现类似下面的消息：

```
jarsigner: java.lang.SecurityException: invalid SHA1
signature file digest for test/classes/Manifest.class
```

513

16.4　使用和JAR相关的API

Java平台包含若干用于处理JAR文件的类。其中一些API是：

- java.util.jar包[2]
- java.net.JarURLConnection类[3]
- java.net.URLClassLoader类[4]

为了使你了解这些新的API的可能作用，这一小节通过分析一个名为JarRunner的示例应用程序的内部工作指导你的学习。

16.4.1　示例——应用程序JarRunner

JarRunner允许通过从命令行指定JAR文件的URL，运行打包到这个JAR文件中的一个应用程序。例如，如果名为TargetApp的应用程序被打包到位于http://www.xxx.yyy/TargetApp.

---

1. docs/technotes/guides/security/SecurityToolsSummary.html
2. docs/api/java/util/jar/package-summary.html
3. docs/api/java/net/JarURLConnection.html
4. docs/api/java/net/URLClassLoader.html

jar的JAR文件中，你可以使用这个命令运行应用程序：

```
java JarRunner http://www.xxx.yyy/TargetApp.jar
```

为了使JarRunner工作，它必须能够执行如下任务，所有任务都使用新的API完成：

- 访问远程JAR文件并且建立到这个文件的通信链接。
- 查看这个JAR文件的清单，查明档案中的哪个类是主类。
- 加载JAR文件中的这个类。

JarRunner由两个类构成：JarRunner和JarClassLoader。JarRunner把大部分JAR处理任务委托给JarClassLoader类。JarClassLoader扩展java.net.URLClassLoader类。在阅读下面的章节之前，你可以先浏览一下JarRunner和JarClassLoader的源代码： 514

- JarRunner.java[1]
- JarClassLoader.java[2]

16.4.2 JarClassLoader 类

JarClassLoader类扩展java.net.URLClassLoader。就像它的名称所暗示的，URLClassLoader的设计目的是用于加载类和资源，通过搜索一组URL访问这些类和资源。URL可以引用目录或者JAR文件。

除了子类化URLClassLoader之外，JarClassLoader还利用其他两个新的和JAR相关的API（java.util.jar包和java.net.JarURLConnection类）的特性。在这一小节中，我们将详细分析JarClassLoader的构造器和两个方法。

1. JarClassLoader构造器

这个构造器采用java.net.URL的实例作为实参。传递给这个构造器的URL将在JarClassLoader中的其他位置使用，用于搜索从之加载类的JAR文件：

```
public JarClassLoader(URL url) {
   super(new URL[] { url });
   this.url = url;.
}
```

URL对象被传递给超类URLClassLoader的构造器，它采用URL[]数组（而不是单一URL实例）作为实参。

2. getMainClassName方法

使用JAR打包应用程序的URL构造JarClassLoader对象之后，需要通过一种方式判断JAR文件中的哪个类是应用程序的入口点。这是getMainClassName方法的工作： 515

```
public String getMainClassName() throws IOException {
  URL u = new URL("jar", "", url + "!/");
  JarURLConnection uc = (JarURLConnection)u.openConnection();
  Attributes attr = uc.getMainAttributes();
  return attr != null
    ? attr.getValue(Attributes.Name.MAIN_CLASS)
    : null;
}
```

可以回顾16.1.5节的第2小节，打包到JAR文件的应用程序的入口点由JAR文件的清单的Main-Class头指定。为了了解getMainClassName方法如何访问Main-Class头值，我们来详细地分析这个方法，特别要注意它使用的新的JAR处理特性。

---

1. tutorial/deployment/jar/interfaces/examples/JarRunner.java
2. tutorial/deployment/jar/interfaces/examples/JarClassLoader.java

(1) JarURLConnection类和JAR的URL

getMainClassName方法使用java.net.JarURLConnection类指定的JAR URL格式。JAR文件的URL语法如这个例子所示：

```
jar:http://www.xxx.yyy/jarfile.jar!/
```

最后的!/分隔符表示URL引用整个JAR文件。跟在这个分隔符后面的所有内容引用特定的JAR文件内容，比如这个例子：

```
jar:http://www.xxx.yyy/jarfile.jar!/mypackage/myclass.class
```

getMainClassName方法的第一行是：

```
URL u = new URL("jar", "", url + "!/");
```

这个语句构造表示JAR URL的新URL对象，把!/分隔符附加到URL之后，用于创建JarClassLoader实例。

(2) java.net.JarURLConnection类

这个类代表应用程序和JAR文件之间的通信链接。它提供了用于访问JAR文件的清单的方法。getMainClassName的第二行是：

```
JarURLConnection uc = (JarURLConnection)u.openConnection();
```

在这个语句中，第一行创建的URL实例打开一个URLConnection。然后把URLConnection实例
516 类型转换为JarURLConnection，以便可以利用JarURLConnection的JAR处理特性。

(3) 获得清单属性：java.util.jar.Attributes

通过JarURLConnection连接到JAR文件之后，可以使用JarURLConnection的getMainAttributes方法访问JAR文件的清单中的头信息。这个方法返回java.util.jar.Attributes的实例，该类把JAR文件清单的头名称映射到和它们相关联的字符串值。getMainClassName方法的第三行创建Attributes对象：

```
Attributes attr = uc.getMainAttributes();
```

为了获得清单的Main-Class头的值，getMainClassName方法的第四行调用Attributes.getValue方法：

```
return attr != null
  ? attr.getValue(Attributes.Name.MAIN_CLASS)
  : null;
```

这个方法的实参Attributes.Name.MAIN_CLASS指出所希望获得Main-Class头的值。(Attributes.Name类还提供用于指定其他标准清单头的静态字段，比如MANIFEST_ VERSION、CLASS_PATH和SEALED。)

3. invokeClass方法

我们已经看到JarClassLoader如何识别JAR打包的应用程序中的主类。最后一个要分析的方法是JarClassLoader.invokeClass，它允许调用主类来启动JAR打包的应用程序：

```
public void invokeClass(String name, String[] args)
  throws ClassNotFoundException,
    NoSuchMethodException,
    InvocationTargetException
{
  Class c = loadClass(name);
  Method m = c.getMethod("main", new Class[]
                                    { args.getClass() });
  m.setAccessible(true);
  int mods = m.getModifiers();
  if (m.getReturnType() != void.class ||
```

```
    !Modifier.isStatic(mods) || !Modifier.isPublic(mods)) {
    throw new NoSuchMethodException("main");
  }
  try {
      m.invoke(null, new Object[] { args });
  } catch (IllegalAccessException e) {
  // This should not happen, as we have disabled access checks
  }
}
```

517

invokeClass方法采用两个实参：应用程序的入口点类的名称和传递给入口点类的main方法的字符串实参数组。首先加载主类：

```
Class c = loadClass(name);
```

loadClass方法是从java.lang.ClassLoader继承的。

加载主类之后，使用java.lang.reflect包的反射API把参数传递给类并且启动它。关于反射的介绍，可以参考“反射API（Reflection API）”指南[1]。

16.4.3 JarRunner 类

使用这种形式的命令启动JarRunner应用程序：

```
java JarRunner url [arguments]
```

在前一节中，我们知道JarClassLoader能够从给定的URL识别和加载JAR打包的应用程序的主类。因此，为了完成JarRunner应用程序，我们需要从命令行获得URL和所有实参，并且把它们传递给JarClassLoader的实例。这些任务由JarRunner应用程序的入口点JarRunner类完成。

在这个类的开头，从命令行指定的URL创建java.net.URL对象：

```
public static void main(String[] args) {
  if (args.length < 1) {
    usage();
  }
  URL url = null;
  try {
      url = new URL(args[0]);
  } catch (MalformedURLException e) {
      fatal("Invalid URL: " + args[0]);
  }
  ...
```

如果args.length < 1，就表示命令行没有指定URL，所以输出一条消息。如果第一个命令行参数是合理的URL，那么就创建新的URL对象来表示它。

518

接下来，JarRunner创建JarClassLoader的新实例，把命令行指定的URL传递给构造器：

```
JarClassLoader cl = new JarClassLoader(url);
```

在前一小节中我们见过，JarRunner通过JarClassLoader利用JAR处理API。

传递给JarClassLoader构造器的URL是你希望运行的JAR打包应用程序的URL。然后，JarRunner调用类加载器的getMainClassName方法，识别应用程序的入口点：

```
String name = null;
try {
    name = cl.getMainClassName();
} catch (IOException e) {
```

---

1. tutorial/reflect/index.html

```
        System.err.println("I/O error while loading JAR file:");
        e.printStackTrace();
        System.exit(1);
    }
    if (name == null) {
        fatal("Specified jar file does not " +
              "contain a 'Main-Class' manifest attribute");
    }
```

关键语句用粗体突出显示。其他语句用于错误处理。

JarRunner识别出应用程序的入口点类之后，只剩下两个步骤：把任何实参传递给应用程序，以及实际启动应用程序。JarRunner使用这些代码执行这两个步骤：

```
// Get arguments for the application
String[] newArgs = new String[args.length - 1];
System.arraycopy(args, 1, newArgs, 0, newArgs.length);
// Invoke application's main class
try {
    cl.invokeClass(name, newArgs);
} catch (ClassNotFoundException e) {
    fatal("Class not found: " + name);
} catch (NoSuchMethodException e) {
    fatal("Class does not define a 'main' method: " + name);
} catch (InvocationTargetException e) {
    e.getTargetException().printStackTrace();
    System.exit(1);
}
```

519

第一个命令行参数是JAR打包应用程序的URL。因此，要传递给应用程序的任何实参在args数组的第1个和之后的元素中。JarRunner接受这些实参，并且创建传递给应用程序的名为newArgs的新数组（在上面以粗体显示）。然后，JarRunner把入口点的类名称和新的实参清单传递给JarClassLoader的invokeClass方法。我们在前一节中看到，invokeClass将加载应用程序的入口点类，把任何实参传递给它，并且启动应用程序。

问题和练习：JAR文件

问题

1. 如何调用打包为JAR文件的applet？
2. jar命令中的-e选项起什么作用？
3. JAR文件中的清单的重要性是什么？
4. 如何修改JAR的清单文件？

答案

可以在以下位置找到“问题”和“练习”的答案：

520

tutorial/deployment/jar/QandE/answers.html

第17章

Java Web Start

Java Web Start提供了强大的功能，可以通过一次点击启动功能完整的应用程序。用户可以下载和启动应用程序，比如完整的电子表格程序或者因特网聊天客户程序，而无需经过复杂的安装过程。

使用Java Web Start，用户可以通过点击Web页面上的链接启动Java应用程序。链接指向JNLP文件，它指示Java Web Start下载、缓存和运行应用程序。

Java Web Start向Java开发人员和用户提供了很多部署优势：

- 通过Java Web Start，可以把一个Java应用程序放在Web服务器上，部署到种类众多的平台上，包括Windows 2003 / Vista / 2000 / XP、Linux和Solaris。
- Java Web Start支持多个并存的Java标准版（Java Standard Edition）平台的版本。特定应用程序可以请求使用特定的Java版本，不会和其他应用程序的不同需求发生冲突。根据应用程序的需求和用户的环境，Java Web Start自动下载和安装所需的Java平台版本。
- 用户可以独立于Web浏览器启动Java Web Start。可以离线或者不访问浏览器。桌面快捷方式也可以启动应用程序，从而向用户提供与本机应用程序相同的体验。
- Java Web Start利用Java平台固有的安全性。默认情况下，应用程序对本地磁盘和网络资源的访问受到限制。用户可以安全地运行从不信任的来源获得的应用程序。
- 使用Java Web Start启动的应用程序在本地缓存，以便提高性能。
- Java Web Start通过其内置的applet查看器，为applet提供有限的支持。但是，它的目的不是成为功能齐全的applet环境（就像Java Plug-in提供的那样）。Java Web Start的applet查看器具有某些局限；例如，不能指定类文件作为资源，并且不接受策略文件。

  在Java 1.4.2和以后的版本中，Java Web Start作为JRE的一部分安装。为了使用Java Web Start应用程序，用户不必独立地安装它或者执行额外的任务。

本章的目的是让你对Java Web Start有初步了解，并不包含所有可用的文档。关于Java Web Start的更多信息，请参阅：

- Java Web Start指南（Java Web Start Guide）[1]。
- Java Web Start FAQ[2]。
- JNLP规范（JNLP Specification）[3]。
- `javax.jnlp` API文档（javax.jnlp API Documentation）[4]。
- Java Web Start开发者网站（Java Web Start Developers Site）[5]。

1. docs/technotes/guides/javaws/developersguide/contents.html
2. docs/technotes/guides/javaws/developersguide/faq.html
3. http://jcp.org/en/jsr/detail?id=56
4. docs/jre/api/javaws/jnlp/index.html
5. http://java.sun.com/products/javawebstart/developers.html

17.1　运行 Java Web Start 应用程序

用户可以通过下面的方式运行Java Web Start：

- 从浏览器运行Java Web Start应用程序。
- 从Java缓存查看器（Java Cache Viewer）运行应用程序。
- 从桌面运行应用程序。

522

注意　要使用Java Web Start部署的应用程序，用户必须具有Java运行时环境（Java Runtime Environment，JRE）的兼容版本。不需要完整的JDK。

17.1.1　从浏览器运行 Java Web Start 应用程序

只需点击浏览器中应用程序的JNLP文件链接，就可以使用Java Web Start运行应用程序，比如：

```
<a href="Notepad.jnlp">Launch Notepad Application</a>
```

然后，Java Web Start根据JNLP文件中的指令加载并且运行应用程序。

17.1.2　从 Java 缓存查看器运行 Java Web Start 应用程序

如果你使用Java 1.6.0版本，就可以通过Java缓存查看器运行Java Web Start应用程序。

当Java Web Start第一次加载应用程序时，应用程序的JNLP文件的信息被存储在本地的Java缓存查看器中。为了再次加载这个应用程序，不需要返回第一次启动应用程序的Web页面，只需打开Java缓存查看器。

为了打开Java缓存查看器，需要：

(1) 打开控制面板（Control Panel）。

(2) 双击Java图标。打开Java控制面板（Java Control Panel）。

(3) 选择General选项卡。

(4) 点击View。打开Java缓存查看器。

查看器中列出应用程序，如图17-1所示。

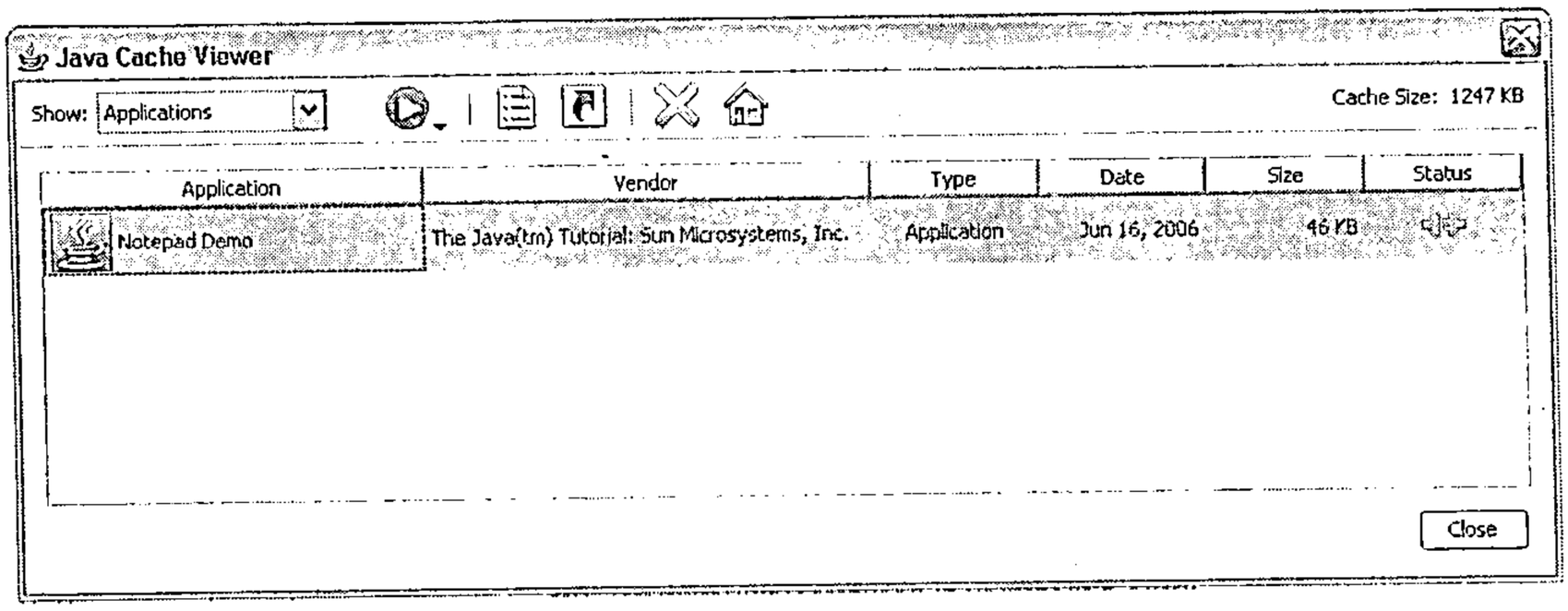


图17-1　Java缓存查看器应用程序

为了运行应用程序，选择它并且点击Run按钮，（图1），或者双击这个应用程序。应用程序像从Web页面启动一样开始运行。

17.1.3 从桌面运行 Java Web Start 应用程序

通过Java缓存查看器，可以把应用程序的快捷方式添加到桌面上。只需选择应用程序，右键点击并且选择Install Shortcuts，或者点击安装图标，（图1）。这样就把快捷方式添加到了桌面上，如图17-2所示。



图17-2 Notepad Demo应用程序的快捷方式

然后，你可以启动Java Web Start应用程序，就像启动任何本机应用程序一样。

17.2 部署 Java Web Start 应用程序

这一小节介绍使用Java Web Start部署应用程序的基础知识。部署应用程序涉及如下步骤：

(1) 设置Web服务器。

(2) 创建JNLP文件。

(3) 把应用程序存放到Web服务器上。

(4) 创建Web页面。

523 ~ 524

这一小节使用Notepad示例应用程序演示Java Web Start技术。你可以在JDK安装目录内的\demo\plugin\jfc\Notepad目录中找到Notepad应用程序示例的全部源文件。

17.2.1 设置 Web 服务器

在使用Java Web Start把应用程序部署到Web上之前，必须确保所使用的Web服务器可以处理JNLP文件。

配置Web服务器，将带有.jnlp扩展名的文件设置为application/x-java-jnlp-file MIME类型。

如何设置JNLP的MIME类型取决于你使用的Web服务器。例如，对于Apache Web服务器，你只需把这一行：

```
application/x-java-jnlp-file JNLP
```

添加到mime.types文件中即可。

对于其他Web服务器，请查阅文档中关于设置MIME类型的说明。

17.2.2 创建 JNLP 文件

使用Java Web Start运行应用程序的关键是Java网络启动协议（Java Network Launching Protocol，JNLP）文件。JNLP文件是XML文件，包含通知Java Web Start如何运行应用程序的元素和属性。

1. JNLP文件示例

下面是Notepad演示程序的JNLP文件[1]：

525

```
<?xml version="1.0" encoding="utf-8"?>
<!-- JNLP File for Notepad -->
```

1. docs/books/tutorialJWS/deployment/webstart/ex5/Notepad.jnlp

```
<jnlp spec="1.0+"
      codebase="http://java.sun.com/docs/books/
                tutorialJWS/deployment/webstart/examples/"
      href="Notepad.jnlp">
  <information>
    <title>Notepad Demo</title>
    <vendor>
      The Java(tm) Tutorial: Sun Microsystems, Inc.
    </vendor>
    <description>Notepad Demo</description>
    <homepage href="http://java.sun.com/docs/books/
              tutorial/deployment/webstart/running.html"/>
    <description kind="short">
      ClickMeApp uses 3 custom classes plus several
      standard ones
    </description>
    <offline-allowed/>
  </information>
  <resources>
    <jar href="Notepad.jar"/>
   <j2se version="1.6+"
         href="http://java.sun.com/products/autodl/j2se"/>
  </resources>
  <application-desc main-class="Notepad"/>
</jnlp>
```

2. JNLP文件内容

表17-1描述示例JNLP文件中的元素和属性。

526

注意 表17-1并不包括JNLP文件的所有可能内容。关于更多信息，请参阅Java网络启动协议和API规范（Java Network Launching Protocol & API Specification）（JSR-56）[1]。

表17-1 JNLP文件的元素和属性

元素	属性	描述	起始版本	是否必须
jnlp		这是JNLP文件的xml主元素。所有内容都包含在jnlp元素之内	1.0	是
	spec	spec属性可以是1.0、1.5.0或者1.6.0，或者使用通配符（比如1.0+）。它表示这个JNLP文件可以使用的JNLP规范的最低版本	1.0	
	codebase	codebase属性指定JNLP文件中href属性指定的所有相对URL的基础位置	1.0	
	href	href指定JNLP文件本身的URL	1.0	
	version	启动的应用程序的版本，还有JNLP文件本身的版本	1.0	
information		information元素包含描述应用程序及其来源的其他元素	1.0	是
	os	指定这一信息元素应该针对的操作系统	1.5.0	
	arch	指定这一信息元素应该针对的架构	1.5.0	
	platform	指定这一信息元素应该针对的平台	1.5.0	

1. http://java.sun.com/products/javawebstart/download-spec.html

（续）

元素	属性	描述	起始版本	是否必须
	locale	指定这一信息元素应该针对的地区	1.5.0	
title		title元素指定应用程序的标题	1.0	是
vendor		vendor元素指定应用程序的提供者	1.0	是
homepage		应用程序的主页	1.0	
	href	指向可以找到关于这个应用程序的更多信息的URL	1.0	是
description		描述应用程序的简短陈述	1.0	
	kind	表示这个描述是什么类型的指示器，合法的值是one-line、short和tooltip	1.0	
icon		描述可以用于向用户标识这个应用程序的图标	1.0	
	href	指向图标文件的URL；可以是下列格式之一：gif、jpg、png和ico	1.0	是
	kind	指出建议的图标用途；可以是default、selected、disabled、rollover、splash或者shortcut.	1.0	
	width	可以用于表示图像的分辨率	1.0	
	height	可以用于表示图像的分辨率	1.0	
	depth	可以用于表示图像的分辨率	1.0	
offline-allowed		表示当客户系统没有连接到网络时可以操作这个应用程序	1.0	
shortcut		shortcut元素可以用于表示用于桌面集成的应用程序首选项	1.5.0	
	online	可以用于描述创建在线或者离线运行的快捷方式的应用程序首选项	1.5.0	
desktop		可以用于表示把快捷方式放到用户桌面上的应用程序首选项	1.5.0	
menu		可以用于表示把菜单项放到用户的开始菜单上的应用程序首选项	1.5.0	
	sub-menu	可以用于表示在什么位置安排菜单项的应用程序首选项	1.5.0	
association		可以用于提示JNLP客户，它希望被操作系统注册为特定扩展名和特定mime类型的首选处理器	1.5.0	
	extensions	包含应用程序要求注册为用它处理的文件扩展名的清单（以空格分隔）	1.5.0	
	mime-type	包含应用程序要求注册为用它处理的mime类型	1.5.0	
related-content		描述可能和应用程序集成的附加相关内容	1.5.0	
	href	指向相关内容的URL	1.5.0	是
update		update元素用于表示JNLP客户应该如何更新应用程序的首选项	1.6.0	
	check	表示JNLP客户应该何时检查更新的首选项。它可以是always、timeout或者background	1.6.0	
	policy	表示JNLP客户在启动应用程序之前知道有更新可用时，应该如何处理更新的首选项。它可以是always、prompt-update或者prompt-run	1.6.0	

527

528

（续）

元素	属性	描述	起始版本	是否必须
security		这个元素可以用于请求增强的许可	1.0	
all-permissions		请求应用程序要带全部许可运行	1.0	
j2ee-application client-permissions		请求应用程序运行时的许可集合满足J2EE应用程序客户（J2EE Application Client）环境的安全规范	1.0	
resources		描述应用程序需要的所有资源	1.0	是
	os	指定资源元素应该针对的操作系统	1.0	
	arch	指定资源元素应该针对的架构	1.0	
	locale	指定资源元素应该针对的地区		
java（或者java se）		指定应用程序和Java的什么版本一起运行	1.6.0（java）	
	version	描述使用的版本范围的有序清单	1.0	是
	href	表示java的这个版本的提供者和下载位置的URL	1.0	
	java-vm-args	表示启动Java时，应用程序希望JNLP客户使用的标准和非标准虚拟机实参的附加集合	1.0	
	initial-heap-size	表示Java堆的初始长度	1.0	
	max-heap-size	表示Java堆的最大长度	1.0	
jar		指定作为应用程序类路径的一部分的JAR文件	1.0	是
	href	JAR文件的URL	1.0	是
	version	要求的JAR文件版本。要求使用基于版本的下载协议	1.0	
	main	表示这个jar文件是否包含具有应用程序主方法的类	1.0	
	download	可以用于表示这个jar文件可以推迟下载，或者在需要时下载	1.0	
	size	表示以字节为单位的JAR文件的下载长度	1.0	
	part	可以用于把资源分组在一起，以便同时下载	1.0	
nativelib		指定在其根目录中包含本机库的JAR文件	1.0	
	href	JAR文件的URL	1.0	是
	version	要求的JAR文件版本。要求使用基于版本的下载协议	1.0	
	download	可以用于表示这个jar文件可以推迟下载	1.0	
	size	表示以字节为单位的JAR文件的下载长度	1.0	
	part	可以用于把资源分组在一起，以便同时下载	1.0	
extension		包含指向和这个应用程序一起使用的附加component-desc或者installer-desc的指针	1.0	
	href	附加扩展JNLP文件的URL	1.0	是
	version	附加扩展JNLP文件的版本	1.0	
	name	附加扩展JNLP文件的名称	1.0	
ext-download		可以用在extension元素中，表示组件扩展中包含的部分	1.0	

529

530

（续）

元素	属性	描述	起始版本	是否必须
	ext-part	描述可以预期在扩展中找到的部分的名称	1.0	是
	download	可以用于表示这个扩展需要立即下载，还是可以推迟下载	1.0	
	part	表示这个JNLP文件中包含扩展的部分的名称	1.0	
package		可以用于向JNLP客户说明哪些JAR文件中实现了哪些包	1.0	
	name	包含在给定部分的JAR文件中的包名称	1.0	是
	part	包含JAR文件的部分名称，这些JAR文件包含给定的包名称	1.0	是
	recursive	可以用于表示（以给定名称开头的）所有包名称可以在给定部分中找到	1.0	
property		定义将通过System.getProperty和System.getProperties方法可用的系统属性	1.0	
	name	系统属性的名称	1.0	是
	value	要设置的值	1.0	是
		注意 JNLP文件必须包含application-desc、applet-desc、component-desc或者installer-desc之一	1.0	是
application-desc		表示这是应用程序的JNLP文件	1.0	
	main-class	包含应用程序的public static void main(String[])方法的类的名称	1.0	是
argument		每个argument包含一个（按照顺序）传递给main的附加实参	1.0	
applet-desc		表示这是applet的JNLP文件	1.0	
	main-class	这是主applet类的名称	1.0	是
	documentbase	URL形式的applet文档基	1.0	
	name	applet的名称	1.0	是
	width	以像素为单位的applet宽度	1.0	是
	height	以象素为单位的applet高度	1.0	是
param		可以传递给applet的参数集合	1.0	
	name	这个参数的名称	1.0	是
	value	这个参数的值	1.0	是
component-desc		表示这是组件扩展的JNLP文件	1.0	
installer-desc		表示这是安装的扩展的JNLP文件	1.0	
	main-class	包含安装器的public static void main（String[]）方法的类的名称	1.0	是

3. 对JNLP文件进行编码

Java Web Start支持使用Java平台支持的任何字符编码对JNLP文件进行编码。关于Java字符编码的更多信息，请参阅“支持的编码指南（Supported Encodings Guide）”[1]。为了对JNLP文

1. docs/technotes/guides/intl/encoding.doc.html

件进行编码，需要在文件的XML序言中指定编码。例如，下面这行表示JNLP文件使用UTF-16编码：

```
<?xml version="1.0" encoding="utf-16"?>
```

注意　XML序言本身必须编码为UTF－8。

17.2.3　把应用程序放到服务器上

使用Java Web Start部署应用程序的下一个步骤很简单，只需把应用程序的所有JAR和JNLP文
533 件放到Web服务器上即可。必须确保JAR文件位于JNLP文件中jar元素的href属性所指定的位置。

17.2.4　创建 Web 页面

完成前面的步骤之后，编写Web页面的准备就绪，用户使用Web页面访问你的应用程序。对于已经安装了Java Web Start的用户，为他们在Web页面中添加到应用程序的链接非常简单；但是，还必须为还没有安装Java Web Start的用户设计Web页面。

1. 添加到JNLP文件的基本链接

为了允许你的用户从Web页面启动应用程序，必须在Web页面上包含到应用程序的JNLP文件的链接。为了添加链接，可以使用标准的HTML链接语法，链接带有指定JNLP文件位置的href属性：

```
<a href="Notepad.jnlp">Launch Notepad Application</a>
```

如果Java Web Start已经安装在客户计算机上，当用户点击这个链接时，Java Web Start就会根据JNLP文件中的指令执行应用程序。

2. 为没有安装Java Web Start的用户添加链接

对于可能没有安装Java Web Start的用户，必须通过编写Web页面中的脚本来完成如下任务：

(1) 检查用户使用哪种浏览器。

(2) 检查是否安装了Java Web Start。

(3) 如果没有安装Java Web Start，可以自动安装，也可以将用户引导到下载页面。

关于这些步骤的更多信息和示例脚本，请参阅Java Web Start指导（Java Web Start Guide）。

17.3　开发 Java Web Start 应用程序

在大多数情况下，使用Java Web Start部署应用程序和部署独立的Java应用程序是一样的。但是，有一些打包方面的考虑因素，在下面几节中介绍。

17.3.1　把应用程序打包到 JAR 文件中

为了使用Java Web Start部署应用程序，必须把应用程序打包为一个或者多个JAR文件。特
534 别的是，你必须把应用程序的文件打包到一个或者多个JAR文件中去。

注意　如果必要的话，其他资源（比如图像和属性文件）可以放在JAR文件之外，通过HTTP请求获得。但是，把资源存储在JAR文件中是首选方式，因为JAR文件会被Java Web Start缓存在本地计算机上。

如果应用程序需要不受限制地访问本地系统，那么所有的JAR文件和项目都必须经过签名。关于更多信息，请参阅17.5节。

17.3.2 读取 JAR 文件中的资源

使用getResource方法从JAR文件读取资源。例如，下面的代码示例从JAR文件获得图像：

```
// Get current classloader
ClassLoader cl = this.getClass().getClassLoader();
// Create icons
Icon saveIcon  = new
                 ImageIcon(cl.getResource("images/save.gif"));
Icon cutIcon   = new
                 ImageIcon(cl.getResource("images/cut.gif"));
```

这个示例假设应用程序的JAR文件中包含如下项目：

- images/save.gif
- images/cut.gif

17.3.3 不被信任的应用程序

除非签名你的应用程序（这要求用户接受你的证书），通过Java Web Start部署的应用程序是不被信任的。不被信任的应用程序受到如下限制：

- 应用程序不能访问本地磁盘。
- 应用程序的所有JAR文件都必须从同一个服务器下载。
- 应用程序只能连接到下载JAR文件的那台服务器。
- 不能删除或者替换安全管理器。
- 应用程序不能使用本机库。
- 应用程序对系统属性的访问受到限制。应用程序只能对已知安全的系统属性集合进行读取/写入访问，还可以对applet可以访问的属性集合进行只读访问。

关于更多信息，请参阅17.5节。

另外，关于在应用程序中使用特殊的Java Web Start包的更多信息，请参阅下一节。

535

17.4 JNLP API

Java平台包括JNLP API，它允许向通过Java Web Start部署的应用程序提供附加信息。

JNLP API包含在javax.jnlp包中（这个包在jnlp.jar中提供，作为Java开发工具包版本6的一部分），javax.jnlp包在sample/jnlp/servlet目录中。javax.jnlp还在javaws.jar中提供，它位于JRE的lib目录中。

表17-2提供javax.jnlp包中的主要接口和类的概述。关于更多信息，请参阅API文档和Java Web Start指导（Java Web Start Guide）。

警告 Java Web Start警告使用javax.jnlp接口和类的用户，如果允许不被信任的应用程序访问文件系统、使用系统打印机、使用系统剪贴板或者增加PersistenceService使用的磁盘空间，那么可能有潜在的安全风险。

表17-2　javax.jnlp包中的接口和类

接口或者类	描　述
BasicService	为以下目的提供方法： • 判断应用程序的代码基 • 判断应用程序是否在线 • 判断当前平台和JNLP客户是否支持Web浏览器 • 在浏览器中打开指定的URL
ClipboardService	为把内容复制到和复制出客户系统的剪贴板提供方法。ClipboardService接口用于在沙箱中运行的应用程序
536 DownloadService	提供控制如何缓存应用程序资源的功能
ExtendedService	为当前JNLP API提供附加支持。它允许应用程序在客户的文件系统中打开指定的文件
FileContents	封装文件的名称和内容。这个类的对象由FileOpenService、FileSaveService和PersistenceService接口使用
FileOpenService	为应用程序提供在本地文件系统上打开文件的功能
FileSaveService	提供用于把文件导出到本地磁盘的方法，即使是在受限制的执行环境中运行的应用程序也可以这么做 **注意**　这个接口为潜在不被信任的Web部署的应用程序提供的磁盘访问级别，和Web浏览器通过其Save As对话框提供的内容访问级别一样。
JNLPRandomAccessFile	支持读取和写入随机访问文件。随机访问文件的行为和存储在文件系统中的大型字节数组一样
PersistenceService	提供用于在客户系统本地存储数据的方法，即使是在沙箱中运行的应用程序也可以这么做 **注意**　这个方法类似基于HTML的应用程序的cookie机制。cookie允许在客户系统本地存储少量数据。这些数据可以由浏览器安全地管理，并且只能被和存储数据的页面来自于同一个URL的HTML页面获得
PrintService	提供用于访问打印机的方法，即使是在受限制执行环境中运行的应用程序也可以这么做 **注意**　从Java Web Start 5.0开始，你可以使用Java Printing API，所以不需要使用这个接口。如果应用程序运行在沙箱中，Java Web Start会打开安全对话框要求用户允许打印
SingleInstanceService 537	向应用程序提供一些方法，用于把它自己注册为单元素，以及注册用于处理其他应用程序实例传递来的实参的监听器

17.5　Java Web Start 和安全

本节介绍通过Java Web Start部署的应用程序的安全基础知识。

17.5.1　Java Web Start 安全基础

默认情况下，使用Java Web Start启动的应用程序运行在受限制的环境中，这种环境被称为沙箱（sandbox）。在这种沙箱中，Java Web Start：

- 阻止可能影响本地文件的恶意代码，从而保护用户。
- 阻止可能试图访问或者破坏网络上的数据的代码，从而保护企业。

Java Web Start启动的未签名的JAR文件在沙箱中运行，这意味着它们不能访问本地文件或者网络。

17.5.2 Java Web Start 部署的签名 JAR 文件

Java Web Start支持签名的JAR文件，使应用程序可以在沙箱之外工作，并可以访问本地文件和网络。

Java Web Start验证JAR文件的内容从签名之后是否改变。如果数字签名的验证失败，Java Web Start就不会运行应用程序。

当用户第一次把应用程序作为签名的JAR文件运行时，Java Web Start打开一个对话框，根据签名者的证书显示应用程序的来源。然后，用户可以根据这些信息做出是否运行应用程序的决定。

关于更多信息，请参阅16.3节。

17.5.3 安全和 JNLP 文件

为了使签名的JAR文件能够访问本地文件和网络，必须在JNLP文件中指定安全设置。security元素包含应用程序的安全设置。

下面的例子为应用程序提供对客户计算机的完全访问，条件是应用程序的所有JAR文件都具有签名：

```
<security>
  <all-permissions/>
</security>
```

17.5.4 HTTPS 证书的动态下载

538

Java Web Start动态地导入证书，就像浏览器那样。为了完成这个工作，Java Web Start设置它自己的https处理器（使用java.protocol.handler.pkgs系统属性）来初始化SSLSocketFactory[1]和HostnameVerifier[2]的默认值。它使用HttpsURLConnection.setDefaultSSLSocketFactory[3]和HttpsURLConnection.setDefaultHostnameVerifier方法设置默认值。

如果你的应用程序使用这两个方法，就要确保在Java Web Start初始化https处理器之后调用它们，否则你定制的处理器会被替换为Java Web Start的默认处理器。

通过下面的操作，可以确保使用自己定制的SSLSocketFactory和HostnameVerifiter：

- 安装你自己的https处理器来替换Java Web Start的https处理器。关于更多信息，请参阅文档*A New Era for Java Protocol Handlers*[4]。
- 在你的应用程序中，只在创建第一个https URL对象（这会首先执行Java Web Start的https处理器的初始化代码）之后，调用HttpsURLConnection.setDefaultSSLSocketFactory或者HttpsURLConnection.setDefaultHostnameVerifier。

17.6 常见的 Java Web Start 问题

问题：我的浏览器把很多应用程序的JNLP文件显示为纯文本。

你的Web服务器很可能没有把JNLP文件识别为正确的MIME类型。更多信息请参阅17.2.1节。

而且，如果你使用代理服务器，那么通过更新Web服务器上资源的时间戳，使代理更新它们的缓存，以便确保返回文件的更新版本。

---

1. docs/api/javax/net/ssl/SSLSocketFactory.html
2. docs/api/javax/net/ssl/HostnameVerifier.html
3. docs/api/javax/net/ssl/HttpsURLConnection.html
4. http://java.sun.com/developer/onlineTraining/protocolhandlers/

问题：当我在Internet Explorer中点击JNLP文件的链接时，Java Web Start启动，出现消息："Could not load file/URL specified: C:\Documents and Settings\...\application[1].jnlp"。

539 这个问题只在Internet Explorer中出现。通常是由Web服务器或者代理服务器的非缓存指令造成的，这使Internet Explorer不把JNLP文件写入本地磁盘。

当对基于Tomcat的服务器进行升级时，这个问题经常出现，因为之后的版本默认情况下为在web.xml文件中安全约束内的任何资源设置非缓存指令。尝试从安全约束中删除相关的URI。

这个问题的出现原因也可能是，缓存满了或者Internet Explorer关闭了缓存。

问题：我的浏览器似乎能找到JNLP文件，但是Java Web Start说找不到。

可能的原因是你的浏览器和Java Web Start使用不同的代理设置。修改Java Web Start的代理设置：

1. 打开Java应用程序缓存查看器（Java Application Cache Viewer）。
2. 从Edit菜单选择Preferences。打开Java控制面板（Java Control Panel）。
3. 点击Network Settings。打开Network Settings对话框。
4. 选择Use proxy server，输入代理服务器名称和端口号（通常为8080）。
5. 点击OK。

问题和练习：Java Web Start

问题

1. 在运行Java Web Start应用程序的链接中，哪个文件被指定为a标记的href属性？
2. Web服务器必须识别哪种MIME类型才能启动Java Web Start应用程序？
3. 在应用程序的JNLP文件中，resources元素内必须指定哪两个元素？
4. 哪个接口提供控制如何缓存Java Web Start应用程序资源的功能？
   a. BasicService
   b. DownloadService
   c. PersistenceService
   d. ExtendedService

540 5. 是或否：默认情况下，Java Web Start应用程序在沙箱中运行。
6. 是或否：如果Java Web Start应用程序运行在安全沙箱中，应用程序的JAR文件可以在另一个服务器上。
7. 为了使Java Web Start应用程序支持安全沙箱之外的操作，必须对其JAR文件进行什么处理？

练习

1. 编写添加到JNLP文件的XML代码，以便请求让应用程序具有对客户系统的完全访问。
2. 你的Java Web Start应用程序有两个图标（one.gif和two.gif），在JAR文件的images目录中。编写应用程序代码来访问这些图像。

答案

可以在以下位置找到"问题"和"练习"的答案：

541 tutorial/deployment/webstart/QandE/answers.html

第18章

applet

本章介绍applet的基础知识，applet较普通应用程序的优势来自在Web页面中加载applet，把应用程序转换为applet，以及applet工作的方式。

applet是一种特殊类型的Java程序，启用Java技术的浏览器可以从因特网下载和运行applet。applet通常嵌入Web页面并且运行在浏览器的上下文环境中。applet必须是`java.applet.Applet`类的子类，这个类提供applet和浏览器环境之间的标准接口。

Swing提供了Applet的一个特殊子类，称为`javax.swing.JApplet`，所有使用Swing组件构造GUI的applet都会使用它。

如果在Web页面中加载applet，浏览器会通过调用某些方法管理applet的生存周期。

1. applet的生存周期

基本上，`Applet`类有4个方法，所有applet都是在它们的基础上构建的：

- **init**——这个方法的目的是提供applet所需的初始化。在applet标记的param属性后面调用它。
- **start**——在init方法之后自动调用这个方法。当用户访问其他页面之后，返回包含applet的页面时，也会调用它。
- **stop**——当用户离开包含applet的页面时，会自动调用这个方法。你可以使用这个方法停止动画。
- **destroy**——只有当浏览器正常关闭时，才调用这个方法。

因此，applet被初始化一次且只有一次，在其生存周期内可以开始和停止一次或者多次，并且只能销毁一次。

关于更多信息，请参阅18.1.2节。

2. 什么时候应该编写applet而不是应用程序

在Java的早期，和Java应用程序相比，Java applet的一个非常重要的优势是applet可以很容易地通过Web部署，而Java应用程序则需要烦琐的安装过程。此外，因为从因特网下载applet，所以默认情况下它们必须运行在受限制的安全环境中（称为“沙箱”），以便确保它们不会在用户的计算机上执行任何破坏性操作，比如读取/写入文件系统。

但是，随着Java Web Start（参见17章）的引入，Java应用程序也可以很容易地通过Web部署，同样可以运行在安全环境中。这意味着，Java applet和Java应用程序之间的显著区别仅仅是applet运行在Web浏览器的上下文环境中，通常嵌入在HTML页面内，而Java应用程序在浏览器之外独立运行。因此，applet特别适合在Web页面中提供比HTML更强大的交互性或者动画功能，比如图形化游戏、复杂的编辑或者交互性数据可视化。最终用户可以在不离开浏览器的情况下访问这些功能。

3. 在Web页面中加载applet

为了在Web页面中加载applet，必须使用正确的applet标记指定applet类。下面是一个简单例子：

```
<applet code=AppletWorld.class width="200" height="200">
</applet>
```

为了部署和测试可以使用JDK附带的轻型appletviewer应用程序运行applet。例如，如果AppletWorld.html是HTML文件名称，那么可以运行这个命令：

```
appletviewer AppletWorld.html
```

一旦知道你的applet可以运行在appletviewer中之后，把applet的Web页面加载到浏览器窗口中，测试applet在Web浏览器中的运行情况。浏览器可以从因特网，或者从部署过程中使用的工作目录中获得类文件。如果applet加载到浏览器中后改动了applet的代码，就必须重新编译applet，并且按下Shift和浏览器中的Reload按钮以便加载新版本。

544

本章的目的是让你对applet和Java Plug-in做初步了解，并未包括所有可用的文档。关于更多信息，请参阅Java Plug-in开发者指导（Java Plug-in Developer Guide）[1]。

18.1　applet入门

运行在线指南中的HelloWorld applet[2]，或者自己编译和运行这个applet（图18-1）。它是一个简单的Java类，在一个小的长方形内输出字符串“Hello World”。



图18-1　HelloWorld applet的屏幕截图

下面是HelloWorld applet[3]的源代码：

```
import javax.swing.JApplet;
import java.awt.Graphics;

public class HelloWorld extends JApplet {
  public void paint(Graphics g) {
    g.drawRect(0, 0,
               getSize().width - 1,
               getSize().height - 1);
    g.drawString("Hello world!", 5, 15);
  }
}
```

这样的applet通常由*Java Plug-in*管理和运行。当你下载Java SE运行时环境（Java SE Runtime Environment，JRE）时自动包含Java Plug-in，Java Plug-in扩展Web浏览器的功能，允许applet运行在Sun的Java SE运行时环境之下，而不是Web浏览器附带的Java运行时环境。Java Plug-in可以用于Mozilla系列浏览器和Internet Explorer。

545

把应用程序转换为applet

应用程序是独立程序，由具有main方法的至少一个类构成。applet和应用程序有显著区别。首先，applet没有被自动调用的用来启动程序的main方法。与应用程序不同的是，在applet执行的过程中，在不同位置调用若干方法。Java applet和应用程序之间的区别在于它们如何运行。应用程序通常通过由Java解释器（比如JDK 6中的java工具）加载应用程序的主类运行。

---

1. docs/technotes/guides/plugin/developer_guide/contents.html
2. tutorial/deployment/applet/examples/helloWorld.jar
3. tutorial/deployment/applet/examples/HelloWorld.java

把应用程序转换为applet程序的基本步骤如下。

- 需要创建java.applet.Applet的子类，覆盖init方法，用它初始化applet的资源，和main初始化应用程序资源的方式一样。
- init可能被调用多次，并且应该适当地设计它。而且，需要在init中把顶级的Panel添加到applet；通常添加到main中的Frame。就是这么简单！

通过分析实例应用程序SwingUI.java[1]及其相应的applet程序[2]，你将了解如何把应用程序转换为applet。

比较这两个程序时，你可能会注意到二者的主要区别：

- applet类声明为public，以便appletviewer能够访问它。
- applet类派生于Applet/JApplet，而应用程序派生于Frame/JFrame。
- applet版本没有main方法。
- 在applet中，应用程序的构造器被替换为start和init方法。
- GUI组件直接添加到Applet，然而在应用程序中，GUI组件添加到JFrame对象的内容板。

18.1.1 扩展 Applet 或者 JApplet

下面清单中以粗体显示的第一行开始定义HelloWorld类的块：

546

```
import javax.swing.JApplet;
import java.awt.Graphics;

public class HelloWorld extends JApplet {
  public void paint(Graphics g) {
    g.drawRect(0, 0,
               getSize().width - 1,
               getSize().height - 1);
    g.drawString("Hello world!", 5, 15);
  }
}
```

applet从Applet或者JApplet继承大量功能，包括和浏览器进行通信以及向用户显示图形用户界面（GUI）。

使用Swing（Java的GUI工具包）的GUI组件的applet应该扩展javax.swing.JApplet基类，它提供和Swing的GUI工具的最佳集成。

JApplet提供与Swing的JFrame和JDialog组件相同的“RootPane”顶级组件结构，而Applet仅仅提供简单的面板。关于如何使用这一特性的详细介绍，请参阅*How to Use Root Panes*[3]。

当applet不使用Swing的GUI组件时，它可以扩展java.applet.Applet。如果applet使用Java图形库（比如绘图或者游戏）和或只使用AWT组件完成所有绘制工作，就可能出现这种情况。

18.1.2 applet 的生存周期

运行Simple applet[4]（图18-2）。

---

1. tutorial/deployment/applet/SwingUI.java
2. tutorial/deployment/applet/ApptoAppl.java
3. tutorial/uiswing/components/rootpane.html
4. tutorial/deployment/applet/examples/simple.jar



图18-2　Simple applet的屏幕截图

下面是Simple[1]的源代码。Simple applet在遇到其生存周期内的重要里程碑时（比如用户
547 第一次访问包含applet的页面）会显示一个描述性字符串。使用Simple applet的页面和在它上面构造的页面可以演示很多applet中的共同概念。如果看不懂这些Java源代码，则需要参考第2章来学习这种语言。

```
/*
 * Java SE 6 Version
 */

import java.applet.Applet;
import java.awt.Graphics;

// No need to extend JApplet, since we don't add
// any components; we just paint.
public class Simple extends Applet {

  StringBuffer buffer;

  public void init() {
    buffer = new StringBuffer();
    addItem("initializing... ");
  }

  public void start() {
    addItem("starting... ");
  }

public void stop() {
  addItem("stopping... ");
}

public void destroy() {
  addItem("preparing for unloading...");
}

private void addItem(String newWord) {
  System.out.println(newWord);
  buffer.append(newWord);
  repaint();
}

public void paint(Graphics g) {
  // Draw a Rectangle around the applet's display area.
  g.drawRect(0, 0,
  getWidth() - 1,
  getHeight() - 1);

  // Draw the current string inside the rectangle.
  g.drawString(buffer.toString(), 5, 15);
 }
}
```

548

注意　在这个示例中，我们扩展Applet类，而不是Swing JApplet类，因为我们在这个applet中不需要添加Swing组件。

1. tutorial/deployment/applet/examples/Simple.java

1. 加载applet

加载这个applet时，应该会看到上面的文本“initializing ... starting ... ”。加载applet之后，会发生如下情况：

- 创建applet的控制类（Applet的子类）的实例。
- applet对自己进行初始化。
- applet开始运行。

2. 离开和返回applet的页面

当用户离开这个页面时（比如进入另一个页面），浏览器停止applet。当用户返回页面时，浏览器开始运行applet。

浏览器提示 一些浏览器当用户返回这个页面时重新加载applet。在至少一种浏览器中存在一个错误，在没有重新加载时applet会多次初始化自己。

---

3. 重新加载applet

某些浏览器允许用户重新加载applet，这个过程包括卸载applet和再次加载它。在applet被卸载前，它有机会停止自己并且执行最后的清理工作（final cleanup），这样applet可以释放它占有的任何资源。在此之后，applet被卸载，然后再次加载，如18.1.2节的第1小节所述。

试验 如果浏览器或者其他applet浏览器能够很容易地重新加载，就重新加载它。当重新加载applet时发生情况，请参阅18.3.2节的第2小节。（关于标准输出的信息，请参阅18.2.2节。）卸载applet时，应该看到“stopping ...”和“preparing for unloading ...”。但在applet的GUI中看不到这些信息，因为在能够显示文本之前applet被卸载了。重新加载applet时，应该看到“initializing ...”和“starting ...”，就像第一次加载applet时的情况。

549

---

4. 退出浏览器

当用户退出浏览器（或者显示applet的任何其他应用程序）时，applet有机会停止自己并在浏览器退出前执行最后的清理工作。

5. 小结

applet可以以下面的方式对主要事件做出响应：

- 它可以对本身进行初始化。
- 它可以开始运行。
- 它可以停止运行。
- 它可以执行最后的清理工作，以便准备卸载。

下一节描述与这4个反应相对应的四个applet方法。

18.1.3 里程碑方法

与其他的applet一样，Simple applet提供Applet类的一个子类。Simple类覆盖了四个Applet方法，这样它就可以对主要事件做出反应：

- **init**——用于在每次加载（或者重新加载）applet时初始化（initialize）applet。
- **start**——用于开始（start）applet的执行，比如在加载applet时或用户重新访问包含applet的页面时。
- **stop**——用于停止（stop）applet的执行，比如在用户离开applet的页面时或者退出浏览器时。
- **destroy**——用于执行最后的清理工作（final cleanup），以准备卸载。

下面是这些方法的接口：

```
public class Simple extends JApplet {
  ...
  public void init() { ... }
  public void start() { ... }
  public void stop() { ... }
  public void destroy() { ... }
  ...
}
```

550

本书中讨论和使用init、start、stop和destroy方法。关于更多信息，请参阅JApplet API规范（JApplet API Specification）[1]。

覆盖这些方法

并非每个applet都需要完全覆盖这些方法。有些简单的applet根本不覆盖它们。例如，“HelloWorld”applet（见1.2节）没有覆盖这些方法，因为这个applet除了绘制本身之外什么也没有做。

“HelloWorld”applet只使用paint方法显示一个字符串一次。（18.1.4节中讨论paint方法。）但是大多数applet进行更多的处理。

(1) init方法

init方法对于耗时不多的一次性初始化工作是很有用的。总的来说，init方法应该包含通常放在构造器中的代码。applet通常不应该有构造器的原因是，在它的init方法被调用之前不保证有完整的环境。例如，Applet的图像装载方法不能在构造器内工作。另一方面，init方法是调用图像装载方法的好方法，因为这些方法可以很快地返回。

浏览器提示　有时候一些浏览器在加载applet之后多次调用init方法。关于详细信息，请参阅18.1.2节。

---

(2) start方法

在初始化之后进行某些操作（除了直接响应用户操作之外）的applet必须覆盖start方法。start方法要么执行applet的工作，要么（很可能）启动一个或多个线程来执行工作。在后面的18.3.4节中有更多关于线程的信息。在18.1.4节中，你将更多地了解如何处理表示用户操作的事件。

(3) stop方法

大多数覆盖了start方法的applet也应该覆盖stop方法，stop方法应该暂停执行applet，以便在用户不观看此applet页面时applet不占用系统资源。例如，有一个显示动画的applet，当用户没有观看它时，它应该停止绘制动画。

551

(4) destroy方法

许多applet不需要覆盖destroy方法，因为它们的stop方法（在destroy前被调用）做了停止执行applet所需的所有工作。但是，对于需要释放其他资源的applet，就需要覆盖destroy方法。

注意　应该使destroy方法的实现尽可能简短，因为不能保证这个方法会完全执行。Java虚拟机（Java Virtual Machine）可能在长的destroy方法完成之前就退出了。

18.1.4　用于绘制和事件处理的方法

1. 使用paint方法

为了在浏览器页面中绘制applet的外观表示，需要使用paint方法。

---

1. docs/api/javax/swing/JApplet.html

例如，Simple applet通过覆盖paint方法定义它在屏幕上的外观：

```
public void paint(Graphics g) {
  // Draw a Rectangle around the applet's display area.
  g.drawRect(0, 0,
    getWidth() - 1,
    getHeight() - 1);

  // Draw the current string inside the rectangle.
  g.drawString(buffer.toString(), 5, 15);
}
```

applet从抽象窗口工具包（Abstract Window Toolkit，AWT）的Container类继承paint方法。

2. 事件处理

applet从Container类继承了一组事件处理方法。

Container类定义了若干方法（比如processKeyEvent和processMouseEvent）用于处理特定类型的事件，并且定义了名为processEvent的包含所有功能的方法。 552

为了响应事件，applet必须覆盖适当的事件特定方法。例如下面的程序SimpleClick[1]，实现MouseListener并且覆盖mouseClicked方法：

```
/*
 * Java(TM) SE 6 version.
 */

import java.awt.event.MouseListener;
import java.awt.event.MouseEvent;
import java.applet.Applet;
import java.awt.Graphics;

// No need to extend JApplet, since we don't add
// any components; we just paint.
public class SimpleClick extends Applet
    implements MouseListener {

  StringBuffer buffer;

  public void init() {
    addMouseListener(this);
    buffer = new StringBuffer();
    addItem("initializing... ");
  }

  public void start() {
    addItem("starting... ");
  }

  public void stop() {
    addItem("stopping... ");
  }

  public void destroy() {
    addItem("preparing for unloading...");
  }

  void addItem(String newWord) {
    System.out.println(newWord);
    buffer.append(newWord);
```

1. tutorial/deployment/applet/examples/SimpleClick.java

```
    repaint();
  }
553 public void paint(Graphics g) {
    // Draw a Rectangle around the applet's display area.
    g.drawRect(0, 0,
               getWidth() - 1,
               getHeight() - 1);

    // Draw the current string inside the rectangle.
    g.drawString(buffer.toString(), 5, 15);
  }

  // The following empty methods could be removed
  // by implementing a MouseAdapter (usually done
  // using an inner class).
  public void mouseEntered(MouseEvent event) {
  }
  public void mouseExited(MouseEvent event) {
  }
  public void mousePressed(MouseEvent event) {
  }
  public void mouseReleased(MouseEvent event) {
  }

  public void mouseClicked(MouseEvent event) {
    addItem("click!... ");
  }
}
```

运行生成的applet[1]（见图18-3）。当你在它的长方形中点击鼠标时，会显示“click! ...”。



图18-3 SimpleClick applet的屏幕截图

18.1.5 用于添加 UI 组件的方法

Simple applet的显示代码（在它的paint方法中实现）的缺陷是它不支持滚动。一旦它显示的文本达到了长方形显示区的末尾，就无法看到新的文本。图18-4显示这个问题的现象。



554 图18-4 Simple applet演示程序的输出显示缺少滚动功能

这个问题的最简单的解决办法是使用预制的、行为正确的用户界面（UI）组件。

注意 本节省略了很多详细信息。为了真正学习使用UI组件，请参阅*Creating a GUI with JFC/Swing*[2]。

---

1. 预制的UI组件

Swing提供下列UI组件（括号中列出实现每个组件的类）：

- 按钮（javax.swing.JButton）。
- 复选框（javax.swing.JCheckBox）。
- 单行文本字段（javax.swing.JTextField）。

---

1. tutorial/deployment/applet/examples/simpleClick.jar
2. tutorial/uiswing/index.html

- 大型文本显示和编辑区域（javax.swing.JTextArea）。
- 标签（javax.swing.JLabel）。
- 列表（javax.swing.JList）。
- 弹出（javax.swing.Popup）。
- 滚动条（javax.swing.JScrollBar）。
- 滑动块（javax.swing.JSlider）。
- 绘图区域（java.awt.Canvas）。
- 菜单（javax.swing.JMenu、javax.swing.JMenuBar、javax.swing.JMenuItem和javax.swing.JCheckBoxMenuItem）。
- 容器（javax.swing.JPanel、javax.swing.JWindow及其子类）。

2. 在applet中使用UI组件的方法

因为JApplet类继承了AWT的Container类，所以很容易把组件添加到applet以及使用布局管理器控制组件在屏幕上的位置。下面是applet可以使用的Container方法：

- **add**——添加指定的Component。
- **remove**——删除指定的Component。
- **setLayout**——设置布局管理器。

555

3. 为Simple applet添加不可编辑的文本字段

为了让Simple applet使用可以滚动的不可编辑文本字段，我们可以使用JTextField类。下面是修改后的ScrollingSimple类[1]：

```
/*
 * Java(TM) SE 6 version.
 */

import javax.swing.JApplet;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

// Since we're adding a Swing component, we now need to
// extend JApplet. We need to be careful to access
// components only on the event-dispatching thread.
public class ScrollingSimple extends JApplet {

  JTextField field;

  public void init() {
    // Execute a job on the event-dispatching thread:
    // creating this applet's GUI.
    try {
        SwingUtilities.invokeAndWait(new Runnable() {
          public void run() {
            createGUI();
          }
        });
    } catch (Exception e) {
        System.err.println("createGUI didn't " +
                           "successfully complete");
    }
    addItem(false, "initializing... ");
  }
```

1. tutorial/deployment/applet/examples/ScrollingSimple.java

```
    private void createGUI() {
      // Create the text field and make it uneditable.
      field = new JTextField();
      field.setEditable(false);

      // Set the layout manager so that the text field will be
      // as wide as possible.
      setLayout(new java.awt.GridLayout(1,0));

      // Add the text field to the applet.
      add(field);
    }
    public void start() {
      addItem(false, "starting... ");
    }

    public void stop() {
      addItem(false, "stopping... ");
    }

    public void destroy() {
      addItem(false, "preparing for unloading...");
      cleanUp();
    }

    private void cleanUp() {
      // Execute a job on the event-dispatching thread:
      // taking the text field out of this applet.
      try {
          SwingUtilities.invokeAndWait(new Runnable() {
            public void run() {
              remove(field);
            }
          });
      } catch (Exception e) {
          System.err.println("cleanUp didn't " +
                             "successfully complete");
      }
      field = null;
    }

    private void addItem(boolean alreadyInEDT, String newWord) {
      if (alreadyInEDT) {
          addItem(newWord);
      } else {
          final String word = newWord;
          // Execute a job on the event-dispatching thread:
          // invoking addItem(newWord).
          try {
              SwingUtilities.invokeAndWait(new Runnable() {
                public void run() {
                  addItem(word);
                }
              });
          } catch (Exception e) {
              System.err.println("addItem didn't " +
                                 "successfully complete");
          }
      }
    }
    // Invoke this method ONLY from the event-dispatching thread.
    private void addItem(String newWord) {
```

556

557

```
        String t = field.getText();
        System.out.println(newWord);
        field.setText(t + newWord);
      }
    }
```

createGUI方法创建不可编辑的文本字段（一个JTextField实例）。它设置applet的布局管理器，使文本字段尽可能宽（可以在*Laying Out Components within a Container*[1]中更多地了解布局管理器），然后把文本字段添加到applet。

运行生成的applet[2]见图18-5。



图18-5 ScrollingSimple applet的屏幕截图

18.1.6 applet 能做什么，不能做什么

这一小节概述applet受到的限制和它们具有的特殊功能。在18.3.1节中有更详细的介绍。

1. 安全限制

每种浏览器都实现安全策略，防止applet危害系统安全。本节介绍当前浏览器应用的安全策略。但是，安全策略的实现因浏览器的不同而不同。此外，安全策略在不断地改变。例如，如果浏览器被开发为只用于可信任的环境中，那么其安全策略可能比我们在这里描述的要宽松很多。

当前浏览器对通过网络加载的任何applet加以如下限制：

- applet不能加载库或者定义本地方法。
- applet一般不能读取或者写入执行它的主机上的文件。
- 除了连接它所来自的主机之外，applet不能建立网络连接。
- applet不能启动执行它的主机上的任何程序。
- applet不能读取某些系统属性。
- applet生成的窗口和应用程序生成的窗口不同。

每种浏览器都有实现其安全策略的SecurityManager对象。当SecurityManager检测到违规操作时，它就会抛出SecurityException异常。applet可以捕获这个SecurityException异常并且作出适当响应。

2. applet功能

java.applet提供一个API，它使applet具有应用程序不具有的某些功能。

下面是当前浏览器和其他applet查看器允许applet进行的一些其他操作：

- applet通常可以建立到它来自的主机的网络连接。
- 运行在Web浏览器中的applet可以很容易地显示HTML文档。
- applet可以调用同一个页面上的其他applet的公共方法。
- 从本地文件系统（从用户的CLASSPATH中的目录）加载的applet没有对从网络加载的applet的那些限制。

18.2 利用 applet API 的优势

applet API使你可以利用applet和Web浏览器的紧密关系。这个API是javax.swing.JApplet

1. tutorial/uiswing/layout/index.html
2. tutorial/deployment/applet/examples/scrollingSimple.jar

类和java.applet.AppletContext接口提供的。

applet可以使用这些API进行如下操作：

- 得到浏览器的里程碑的通知。
- 加载相对于applet的URL或运行applet的页面的URL指定的数据文件。
- 显示简短的状态字符串。
- 使浏览器显示一个文档。
- 寻找同一个页面中正在运行的其他applet。
- 播放声音。
- 得到用户在<APPLET>标记中指定的参数。

559 本节将依次讨论这些主题，除了里程碑方法之外（init、start等），里程碑方法在18.1.3节中介绍。关于如何在applet中使用非applet特定的API的信息，请参阅18.3节。

18.2.1　查找和加载数据文件

当applet需要从一个用相对URL（相对URL就是没有完整地指定文件位置的URL）指定的文件中加载数据时，applet通常使用代码基或文档基构成完整的URL。

代码基（code base）指定了从其中加载applet的类的目录，JApplet类的getCodeBase方法返回代码基。

文档基（document base）指定包含此applet的HTML页面的目录。JApplet类的getDocumentBase方法返回文档基。

除非<APPLET>标记指定了代码基，否则代码基和文档基指向相同服务器上的相同目录。

applet可能需要的数据，或者需要作为备份的数据，通常是相对于代码基指定的。applet开发者指定的数据（常常是使用参数指定的）通常是相对于文档基指定的。

注意　出于安全原因，浏览器限制不可信applet可以读取的URL。例如，大多数浏览器不允许不可信applet使用“..”进入代码基或文档基上层的目录。另外，因为不可信applet不能读取除applet来源机器上以外的文件，所以如果文档与不可信applet在不同的服务器上，那么文本基一般没有用。

JApplet类定义了加载图像和加载声音的方法的简便形式，可以相对于一个基URL指定图像和声音。例如，假设applet具有图18-6所示的目录结构之一。

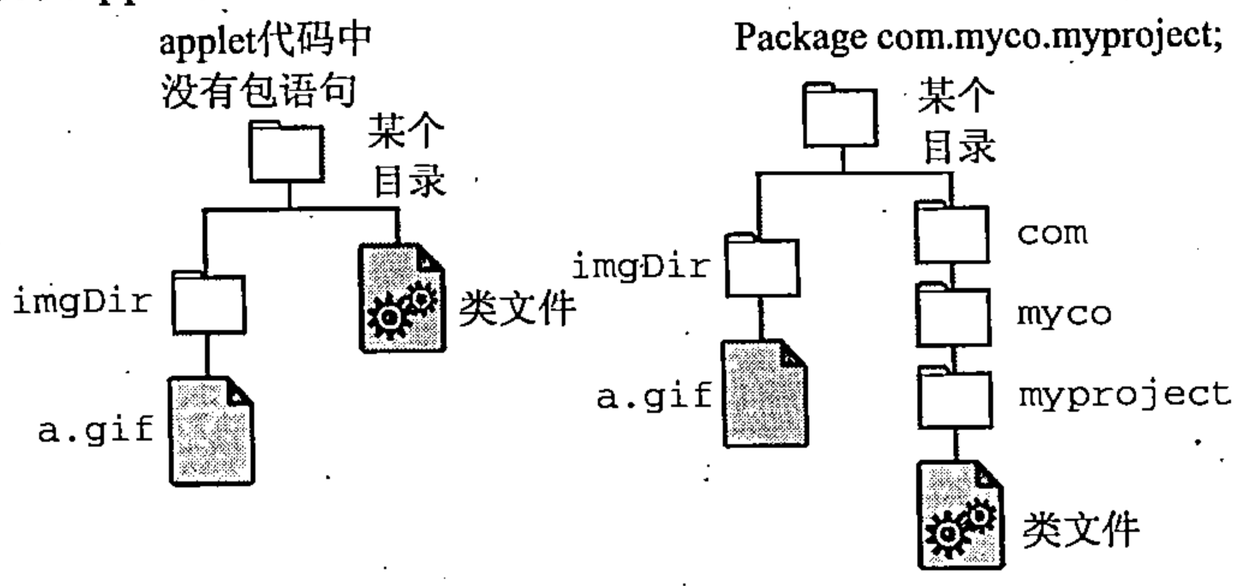


图18-6　两个目录结构显示图像文件和类文件在独立的位置中，但是具有不同的结构

要想使用imgDir目录下的a.png图像文件创建Image对象，applet可以使用以下代码：

```
Image image = getImage(getCodeBase(), "imgDir/a.png");
```

18.2.2 显示简短状态字符串

所有浏览器都允许applet显示短状态字符串。页面上的所有applet以及浏览器本身共享同一个状态栏。

绝不能将关键信息放在状态栏中。如果许多用户可能需要此信息，那么它应该被显示在applet区域中。如果少数高级用户可能需要此信息，可以考虑在标准输出中显示它（参见18.3.2节的第2小节）。

状态栏通常不很引人注目，而且可能被其他applet或浏览器本身覆盖。出于这些原因，状态栏最好适用于不重要的临时信息。例如，一个applet要加载几个图像文件，它可能会显示当前正在加载的图像文件的名称。

applet使用showStatus[1]方法显示状态栏，这个方法是JApplet类从Applet类继承的。

下面是使用这个方法的一个例子：

```
showStatus("MyApplet: Loading image file " + file);
```

注意 请不要在状态栏中滚动文本。浏览器用户不喜欢这样的状态栏！

560
|
561

18.2.3 在浏览器中显示文档

你希望让applet显示格式化的HTML文本吗？这里有一个很容易的办法：使浏览器显示文本。

通过使用AppletContext showDocument方法，applet可以告诉浏览器显示哪个URL以及在哪个浏览器窗口中显示。

showDocument有两种形式：

```
public void showDocument(java.net.URL url)
public void showDocument(java.net.URL url, String targetWindow)
```

单参数形式的showDocument只是让浏览器显示指定的URL处的文档，没有指定在哪个窗口中显示此文档。

术语提示 在这段讨论中，框架（frame）不是指Swing的JFrame，而是指浏览器窗口中的HTML框架。

双实参形式的showDocument允许指定在哪个窗口或者哪个HTML框架中显示文档。第二个实参可以是以下值：

- **"_blank"**将文档显示在一个新的没有名称的窗口中。
- "*windowName*"将文档显示在名为*windowName*的窗口中。如果需要的话，就创建这个窗口。
- **"_self"**将文档显示在包含此applet的窗口和框架中。
- **"_parent"**将文档显示在applet的框架的父框架中。如果applet框架没有父框架，那么"_parent"与"_self"效果一样。
- **"_top"**将文档显示在顶层框架中。如果applet的框架就是顶层框架，那么“_top”与"_self"效果一样。

1. docs/api/java/applet/Applet.html

运行一个applet[1]，它允许你试验两种形式的showDocument的每个选项（见图18-7）。此applet显示一个窗口，你可以在其中输入一个URL并选择任何showDocument选项（见图18-8）。当按回车或点击Show document按钮时，applet调用showDocument。



图18-7　Bring up URL window applet的屏幕截图

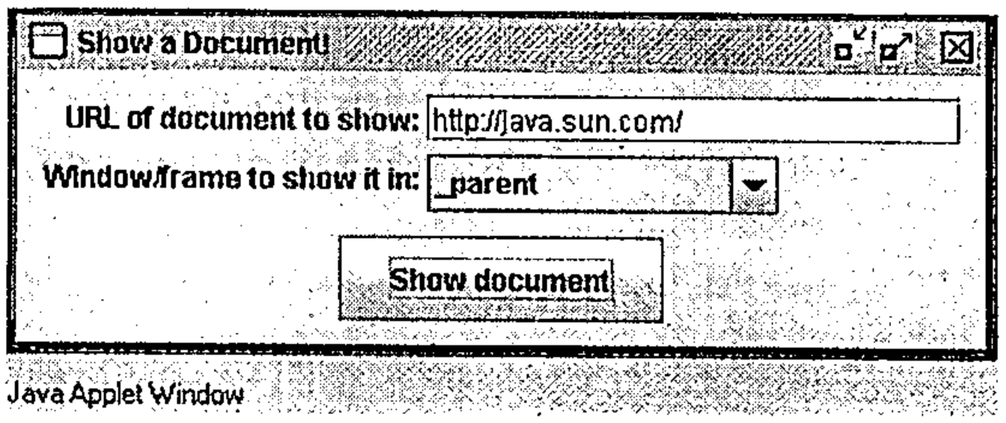


图18-8　ShowDocument applet的屏幕截图

下面是调用showDocument的applet代码[2]：

```
  ...// In an Applet subclass:
  urlWindow = new URLWindow(getAppletContext());
  ...
class URLWindow extends Frame {
  ...
  public URLWindow(AppletContext appletContext) {
    ...
    this.appletContext = appletContext;
    ...
  }
  ...
public boolean action(Event event, Object o) {
  ...
  String urlString = /* user-entered string */;
  URL url = null;
  try {
     url = new URL(urlString);
  } catch (MalformedURLException e) {
     ...// Inform the user and return...
  }
  if (url != null) {
    if (/* user doesn't want to specify the window */) {
       appletContext.showDocument(url);
    } else {
       appletContext.showDocument(url,
         /* user-specified window */ );
    }
  }
  ...
```

562
~
563

18.2.4　向其他 applet 发送消息

applet可以找到其他applet并向它们发送消息，但是有以下几条安全限制：

- 许多浏览器要求这些applet源自相同的服务器。
- 许多浏览器还要求这些applet源自相同服务器的相同目录（即具有相同的代码基）。
- Java平台API要求这些applet在相同浏览器窗口中的相同页面上运行。

---

1. tutorial/deployment/applet/examples/showDocument.jar
2. tutorial/deployment/applet/examples/ShowDocument.java

注意 某些浏览器允许applet调用其他applet的方法（即使applet位于相同浏览器中的不同页面上），条件是这些applet都来自相同的代码基。Java API不支持这种applet间的通信方式，所以并不是所有浏览器都支持此方式。

applet寻找另一个applet的办法有两种，一个是使用AppletContext getApplet方法按applet的名称寻找，另一个是使用AppletContext getApplets方法寻找页面上的所有applet。如果执行成功，这两个方法都向调用者返回一个或多个Applet对象。一旦找到一个Applet对象，调用者就可以调用此对象上的方法。

1. 按名称寻找applet：getApplet方法

getApplet方法遍历当前页面上的所有applet，检查是否有applet具有指定的名称。如果有，getApplet就返回这个applet的Applet对象。

在默认情况下，applet没有名称。若想让applet具有名称，必须在将applet添加到页面的HTML代码中指定名称。可以通过两种方式指定applet的名称：

- 通过在applet的<APPLET>标记中指定NAME属性，例如：

```
<APPLET CODEBASE=example/ CODE=Sender.class
  WIDTH=450
  HEIGHT=200
  NAME="buddy" >
...
</APPLET>
```

- 通过在<PARAM>标记中指定NAME参数，例如：

```
<APPLET CODEBASE=example/ CODE=Receiver.class
  WIDTH=450
  HEIGHT=50>
<PARAM NAME="name" value="old pal">
...
</APPLET>
```

564

浏览器提示 尽管至少有一种启用Java技术的浏览器会执行大小写敏感的搜索，但getApplet方法的预期行为是执行大小写不敏感的搜索。例如，getApplet("old pal")和getApplet("OLD PAL")都会找到名为“Old Pal”的applet。

两个applet[1]演示按名称寻找（见图18-9和图18-10）。第一个applet（Sender）寻找第二个applet（Receiver）。当Sender找到Receiver时，它通过调用Receiver的一个方法（以实参形式传递Sender的名字）向Receiver发送一条消息。Receiver对此方法调用的反应是将它最左边的文本字符串改为“Received message from *sender-name!*”。

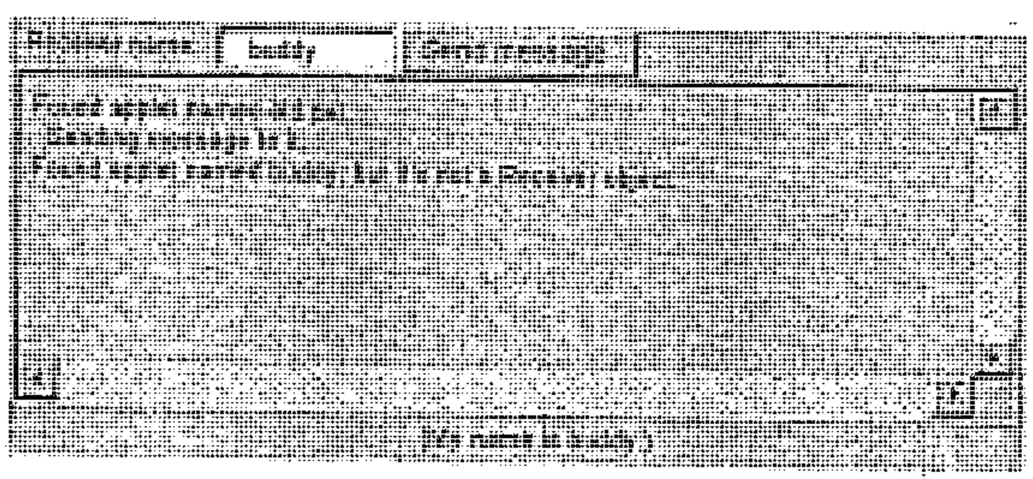
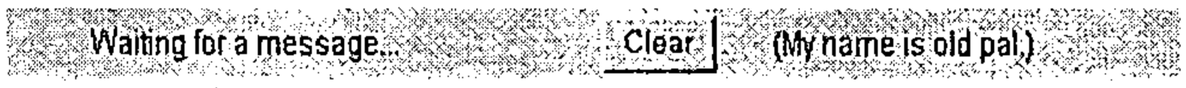


图18-9 Sender applet的屏幕截图

1. tutorial/deployment/applet/examples/sender.jar



565

图18-10　Receiver applet的屏幕截图

试验　点击第一个applet（Sender）的Send message按钮。某些状态信息将出现在Sender的窗口中，而Receiver将用它自己的状态字符串确认收到了消息。在看到Receiver的状态字符串后，点击它的Clear按钮进行复位。在Sender的标有“Receiver name:”的文本域中，输入buddy并按回车键。因为buddy是Sender自己的名字，所以它可以找到一个名为buddy的applet，但是它不会向它自己发送消息，因为它不是一个Receiver实例。

下面是Sender程序[1]用来寻找Receiver并与之进行通信的代码。在你自己的applet中无需改动就可以使用的代码以粗体显示：

```
Applet receiver = null;
String receiverName = nameField.getText();  // Get name
                                            // to search for.
receiver = getAppletContext().getApplet(receiverName);
```

接下来，Sender确保已找到Receiver而且它是正确的类（Receiver[2]）的实例。如果一切顺利，Sender就向Receiver发送一条消息：

```
if (receiver != null) {
  // Use the instanceof operator to make sure the applet
  // we found is a Receiver object.
  if (!(receiver instanceof Receiver)) {
     status.appendText("Found applet named " +
                       receiverName + ", " +
                       "but it's not a Receiver object.\n");
  } else {
     status.appendText("Found applet named " +
                       receiverName + ".\n" +
                       " Sending message to it.\n");
     // Cast the receiver to be a Receiver object
     // (instead of just an Applet object) so that the
     // compiler will let us call a Receiver method.
     ((Receiver)receiver).processRequestFrom(myName);
  }
566 } ...
```

从applet的角度来看，它的名称被存储在名为NAME的参数中。它可以使用Applet getParameter方法得到参数的值。例如，Sender用以下代码得到它的名字：

```
myName = getParameter("NAME");
```

关于使用getParameter的更多信息，请参阅18.2.6节的第2小节。

本节中的applet例子执行单向通信——从Sender到Receiver。如果希望接收者能够向发送者发送消息，那么只需让发送者把对本身的引用（this）传递给接收者。例如：

```
((Receiver)receiver).startCommunicating(this);
```

2. 寻找页面上的所有applet：getApplets方法

GetApplets方法返回页面上所有applet的列表（确切地说，是一个Enumeration[3]）。由于安全原因，很多浏览器和applet查看器实现getApplets的方式使它只返回那些与调用

1. tutorial/deployment/applet/examples/Sender.java
2. tutorial/deployment/applet/examples/Receiver.java
3. docs/api/java/util/Enumeration.html

getApplets的applet源自相同主机的applet。运行一个applet[1]，它简单地列出可以在它的页面上找到的所有applet（见图18-11）：

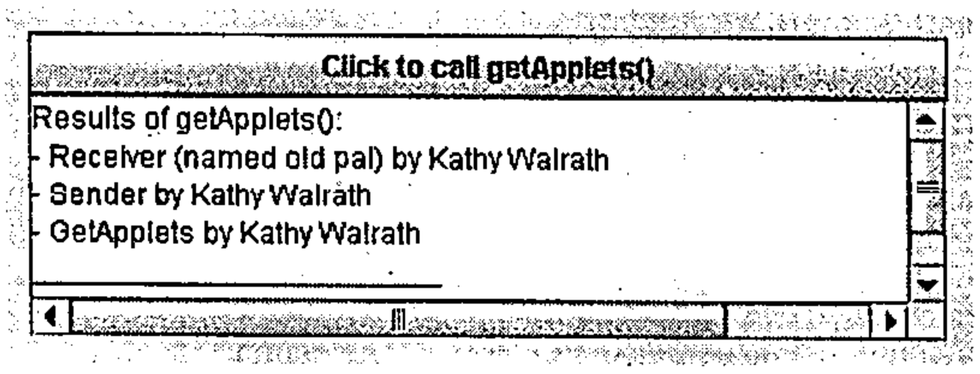


图18-11 GetApplets applet的屏幕截图

567

下面的代码包含与调用getApplets的方法相关的部分[2]：

```
public void printApplets() {
  // Enumeration will contain all applets on this page
  // (including this one) that we can send messages to.
  Enumeration e = getAppletContext().getApplets();
  ...
  while (e.hasMoreElements()) {
    Applet applet = (Applet)e.nextElement();
    String info = ((Applet)applet).getAppletInfo();
    if (info != null) {
        textArea.appendText("- " + info + "\n");
    } else {
        textArea.appendText("- " +
                            applet.getClass().getName() + "\n");
    }
  }
  ...
}
```

18.2.5 播放声音

Java Swing包（javax.swing）中的JApplet类和Java Applet包（java.applet）中的AudioClip[3]接口对播放声音提供基本支持。当前，Java API只支持一种声音格式：8位，μ—law，8000Hz，单声道的Sun“.au”文件。可以使用audiotool应用程序在Sun工作站上创建这种格式。也可以使用音频格式转换程序把其他音频格式转换为这种文件。

1. 和声音有关的方法

下面是和声音有关的Applet方法。每个方法的双实参形式采用基URL（通常由geTDocumentBase或者getCodeBase返回）和声音文件相对于基URL的位置：

- **getAudioClip(URL)**和**getAudioClip(URL,String)**——返回一个实现AudioClip接口的对象。
- **play(URL)**和**play(URL,String)**——播放对应于指定的URL的AudioClip。

AudioClip接口定义如下方法：

- **loop**——开始重复地播放片段。
- **play**——播放片段一次。
- **stop**——停止播放片段。与循环和一次性播放一起使用。

568

1. tutorial/deployment/applet/examples/getApplets.java
2. tutorial/deployment/applet/examples/GetApplets.java
3. docs/api/java/applet/AudioClip.html

2. 示例

运行名为SoundExample[1]的applet，它演示关于声音的一些情况（见图18-12）。注意，出于演示的目的，applet在每个声音的加载时间上加了10秒钟。如果声音文件比我们的大，或者用户的连接比我们的慢，这种延迟就具有现实意义。



图18-12　SoundExample applet的屏幕截图

SoundExample applet为在applet中加载和播放多个声音提供了基本架构。因此，它比较复杂。声音加载和播放代码基本上可以归结为：

```
AudioClip onceClip, loopClip;
onceClip = applet.getAudioClip(getCodeBase(), "bark.au");
loopClip = applet.getAudioClip(getCodeBase(), "train.au");
onceClip.play();     // Play it once.
loopClip.loop();     // Start the sound loop.
loopClip.stop();     // Stop the sound loop.
```

如果在离开applet的页面后applet仍然制造噪音，则会产生反感，所以SoundExample applet在用户离开页面时停止声音的循环播放，在用户回来时恢复播放。这是使用SoundExample的stop和start方法实现的，如下所示：

```
public void stop() {
  // If one-time sound were long, we'd stop it here, too.
  // looping is a boolean instance variable that's initially
  // false. It's set to true when the "Start sound loop" button
  // is clicked and to false when the "Stop sound loop" or
  // "Reload sounds" button is clicked.
  if (looping) {
    loopClip.stop();    // Stop the sound loop.
  }
}
public void start() {
  if (looping) {
    loopClip.loop();    // Restart the sound loop.
  }
}
```

569

SoundExample apple使用三个类：

- JApplet的子类SoundExample[2]，它控制applet的执行
- Hashtable的子类SoundList[3]，它保存AudioClip。这超出了这个applet所需的功能，但是，如果你编写的applet使用很多声音文件，那么这样的类就很有用。
- Thread子类SoundLoader[4]，这个类的每个实例在后台加载AudioClip。在applet初始化期间，applet通过为每个声音文件创建SoundLoader从而预加载它。

在后台线程（使用SoundLoader）预加载声音文件，可以减少用户等待和applet交互的时间，提高了感觉上的性能。通过减少init方法中花费的时间来达到这个目的。如果在applet的init方法中简单地调用getAudioClip，就会在getAudioClip返回之前花费一段时间，这意味着此时applet不能执行其init方法中的其他语句，并且不会调用applet的start方法。（对于这个SoundExample applet，调用start方法的延迟没有关系。）

---

1. tutorial/deployment/applet/examples/sound.jar
2. tutorial/deployment/applet/examples/SoundExample.java
3. tutorial/deployment/applet/examples/SoundList.java
4. tutorial/deployment/applet/examples/SoundLoader.java

在后台线程中加载声音的另一个优势是，允许applet正确地（并且立即）响应一般可能播放声音的用户输入，即使声音文件还没有被加载。例如，你简单地使用Applet play方法，那么用户第一次让applet播放特定声音文件时，当加载声音时applet的绘制和事件处理被冻结。相反，这个applet会检测声音文件还没有加载，并且适当地做出响应。

在18.3.4节中详细讨论这个示例。

18.2.6 定义和使用 applet 参数

参数对于applet的作用类似于命令行参数对于应用程序的作用。参数允许用户定制applet的操作。通过定义参数，可以提高applet的灵活性，使applet可以在多种情况下工作，而不必重新编写代码和重新编译。

下面几个小节从applet程序员的角度讨论参数。为了从用户的角度了解参数，请参阅18.2.7节的第1小节。 570

1. 决定支持哪些参数

这一小节解释当实现参数时应该提出的四个问题：

- 这个applet应该允许用户配置什么？
- 参数的名称应该是什么？
- 每个参数应该采用什么类型的值？
- 每个参数的默认值应该是什么？

这一小节最后讨论一个示例<APPLET>标记中定义的参数。

(1) 这个applet应该允许用户配置什么

applet应该支持哪些参数取决于applet的用途和用户需要的灵活性程度。显示图像的applet可能需要用于指定图像位置的参数。类似，播放声音的applet可能需要用于指定声音的参数。

除了指定资源（比如图像和声音文件）位置的参数，applet有时也提供用于指定applet的外观和操作细节的参数。例如，动画applet可能允许用户指定每秒显示的图像数量。applet也可能允许用户修改它显示的字符串。任何事情都是可能的。

(2) 参数的名称应该是什么

一旦决定了applet将支持哪些参数，就需要决定它们的名称。下面是一些典型的参数名称：

- SOURCE或SRC——用于图像文件这样的数据文件。
- *XXX*SOURCE（例如，IMAGESOURCE）——在applet中用于让用户指定多个数据文件类型。
- *XXX*S——用于以*XXX*的列表为值的参数（其中的*XXX*同样可能是IMAGE）。
- NAME——只用于applet的名称。applet的名称用于applet间的通信，请参考18.2.4节。

保持名称的含义清晰比保持名称简短更重要。不要使用<APPLET>标记属性的名称，在18.5.2节中对这个标记作出了说明。 571

注意 尽管本书常常使用全部大写引用参数名称，但参数名称实际上是大小写不敏感的。例如，IMAGESOURCE和imageSource指的是相同的参数。另一方面，参数值是大小写敏感的，除非在解释参数值前对其进行转换（比如使用String toLowerCase方法）。

---

(3) 每个参数应该采用什么类型的值

参数值都是字符串。无论用户是否在参数值的前后加引号，参数值都是以字符串的形式传递给applet的。但是，applet可以以多种方式解释字符串。

applet通常采用以下类型之一解释参数值：

- URL。
- 整数。
- 浮点数。
- 布尔值——通常是“true”/“false”或“yes”/“no”。
- 字符串——例如，用作窗口标题的字符串。
- 以上值的列表。

(4) 每个参数的默认值应该是什么

applet应该为每个参数提供合适的默认值，这样即使用户没有指定某个参数或指定了不正确的值，applet仍可以执行。例如，一个动画applet应该为每秒显示的图像数量提供合理的设置。这样，如果用户没有指定相关的参数，applet仍然可以工作得很好。

(5) 示例：<APPLET>标记示例

下面是一个典型的<APPLET>标记：

```
<APPLET CODE=SampleApplet.class CODEBASE=example
  WIDTH=350 HEIGHT=60>
<PARAM NAME=windowClass VALUE=BorderWindow>
<PARAM NAME=windowTitle VALUE="BorderLayout">
<PARAM NAME=buttonText
  VALUE="Click here to see a BorderLayout in action">
</APPLET>
```

572

当用户没有指定参数值时，applet使用合理的默认值。例如，如果用户没有指定窗口的标题，applet就使用窗口的类型作为标题。

下一小节介绍如何从用户那里获得参数。

2. 编写代码支持参数

applet使用Applet getParameter方法得到用户为applet参数指定的值。GetParameter方法的定义如下：

```
public String getParameter (String name)
```

你的applet可能需要将getParameter返回的字符串转换为另一种形式，比如整数。java.lang包提供的类（比如Integer）帮助你把字符串转换为原始类型。下面的代码把一个参数的值转换为整数：

```
int requestedWidth = 0;
...
String windowWidthString = getParameter("WINDOWWIDTH");
if (windowWidthString != null) {
  try {
     requestedWidth = Integer.parseInt(windowWidthString);
  } catch (NumberFormatException e) {
     // Use default width.
  }
}
```

注意，如果用户没有为WINDOWWIDTH参数指定值，那么上面的代码使用默认值0，applet将这个值解释为“使用窗口的自然尺寸”。尽可能地提供默认值是很重要的。

除了使用getParameter方法得到applet特有的参数值外，还可以使用getParameter得到applet的<APPLET>标记的属性值。<APPLET>标记属性的清单见18.5.2节。

3. 提供关于参数的信息

既然已经为用户提供了这些有用的参数，就需要帮助用户正确地设置参数值。当然，applet

的文档应该描述每个参数，并且向用户提供设置参数的例子和提示。但是工作还没完，还应该实现getParameterInfo方法，让它返回关于你的applet参数的信息。浏览器可以使用此信息帮助用户设置applet的参数值。

573

下面的代码是一个实现getParameterInfo方法的例子：

```
public String[][] getParameterInfo() {
  String[][] info = {
    // Parameter Name  Kind of Value     Description
    {"imagesource",    "URL",           "a directory"},
    {"startup",        "URL",           "displayed at startup"},
    {"background",     "URL",           "displayed " +
                                          "as background"},
    {"startimage",     "int",           "start index"},
    {"endimage",       "int",           "end index"},
    {"namepattern",    "URL",           "used to generate " +
                                          "indexed names"},
    {"pause",          "int",           "milliseconds"},
    {"pauses",         "ints",          "milliseconds"},
    {"repeat",         "boolean",       "repeat or not"},
    {"positions",      "coordinates",   "path"},
    {"soundsource",    "URL",           "audio directory"},
    {"soundtrack",     "URL",           "background music"},
    {"sounds",         "URLs",          "audio samples"},
  };
  return info;
}
```

如你所见，getParameterInfo方法必须返回一个包含三个字符串的数组。在每个三字符串数组中，第一个字符串是参数名称。第二个字符串提示用户此参数需要什么类型的值。第三个字符串描述此参数的含义。

18.2.7 使用 APPLET 标记

这一小节讲解使用<APPLET>标记时最应该了解的事情。首先给出<APPLET>标记最简单的形式。然后讨论此简单形式之外的一些最常见的扩充：<PARAM>标记、替代的HTML代码和文本、CODEBASE属性和ARCHIVE属性。对<APPLET>标记的详细说明参见18.5.2节。

这是<APPLET>标记最简单的形式：

```
<APPLET CODE=AppletSubclass.class WIDTH=anInt HEIGHT=anInt>
</APPLET>
```

此标记让浏览器装载一个其Applet子类名为*AppletSubclass*的applet，并将它显示在一个指定宽度和高度的区域中。

574

1. 指定参数

某些applet允许用户使用参数定制applet的配置，如18.2.6节所示。例如，AppletButton（本书中多次使用的一个applet，它提供打开窗口的按钮）允许用户通过指定参数BUTTONTEXT的值设置按钮的文本。

开发者使用<PARAM>标记提供参数值。<PARAM>标记应该紧跟着它们所影响的applet的<APPLET>标记：

```
<APPLET CODE=AppletSubclass.class WIDTH=anInt HEIGHT=anInt>
<PARAM NAME=parameter1Name VALUE=aValue>
<PARAM NAME=parameter2Name VALUE=anotherValue>
</APPLET>
```

下面是一个使用<PARAM>标记的例子：

```
<APPLET CODE="Animator.class" WIDTH=460 HEIGHT=160>
<PARAM NAME="imageSource" VALUE="images/Beans">
<PARAM NAME="backgroundColor" VALUE="0xc0c0c0">
<PARAM NAME="endImage" VALUE=10>
<PARAM NAME="soundSource" VALUE="audio">
<PARAM NAME="soundtrack" VALUE="spacemusic.au">
<PARAM NAME="sounds"
  VALUE="1.au|2.au|3.au|4.au|5.au|6.au|7.au|8au|9.au|0.au">
<PARAM NAME="pause" VALUE=200>
...
</APPLET>
```

2. 指定替代的HTML代码和文本

注意前面的HTML例子中的省略号（“...”）。这个例子省略了什么？它省略了替代的HTML代码（alternative HTML code），这些HTML代码只由不理解<APPLET>标记的浏览器解释。替代的HTML代码是出现在<APPLET>和</APPLET>标记之间，任何<PARAM>标记之后的任何文本。启用Java技术的浏览器会忽略替代的HTML代码。

要为启用Java技术的浏览器和其他理解<APPLET>标记的浏览器指定替代文本，应该使用ALT属性。如果浏览器因为某种原因不能显示applet，它可以显示applet的ALT文本。

我们在本书的在线版本中一直使用替代的HTML代码，以便向用户说明那些无法显示的applet。替代的HTML代码常常包括applet的一个或多个图像。前面所示的Animator例子的完整HTML代码如下所示：

575

```
<APPLET CODE="Animator.class" WIDTH=460 HEIGHT=160
  ALT="If you could run this applet, you'd see some animation">
<PARAM NAME="imageSource" VALUE="images/Beans">
<PARAM NAME="backgroundColor" VALUE="0xc0c0c0">
<PARAM NAME="endImage" VALUE=10>
<PARAM NAME="soundSource" VALUE="audio">
<PARAM NAME="soundtrack" VALUE="spacemusic.au">
<PARAM NAME="sounds"
  VALUE="1.au|2.au|3.au|4.au|5.au|6.au|7.au|8au|9.au|0.au">
<PARAM NAME="pause" VALUE=200>
Your browser is completely ignoring the &lt;APPLET&gt; tag!
</APPLET>
```

不理解<APPLET>标记的浏览器会忽略前面的HTML代码中的所有内容，但是不忽略以“Your”开头的一行。理解<APPLET>标记的浏览器会忽略此行中的所有内容。如果无法运行applet，理解applet的浏览器可能显示ALT文本。

3. 指定applet目录

默认情况下，浏览器在包含<APPLET>标记的HTML文件的目录中寻找applet的类和档案文件。（如果applet的类在一个包中，浏览器就使用包名称构造HTML文件的目录下的目录路径。）但是，有时将applet的文件放在其他地方是有好处的。可以使用CODEBASE属性告诉浏览器applet的文件在哪个目录中：

```
<APPLET CODE=AppletSubclass.class CODEBASE=aURL
  WIDTH=anInt HEIGHT=anInt>
</APPLET>
```

如果*aURL*是一个相对URL，那么它被解释为相对于HTML文档的位置。通过使*aURL*成为绝对URL，几乎可以从任何地方装载applet，甚至可以从另一个HTTP服务器装载。

本书常常使用CODEBASE＝"*someDirectory/*"，因为我们将每章的示例分组到子目录中。例如，下面这个<APPLET>标记包含18.1.2节中的Simple applet：

```
<APPLET CODE=Simple.class CODEBASE="example/"
```

```
  WIDTH=500 HEIGHT=20>
</APPLET>
```

图18-13显示当CODEBASE被设置为“example/”时，类文件相对于HTML文件的位置。 576

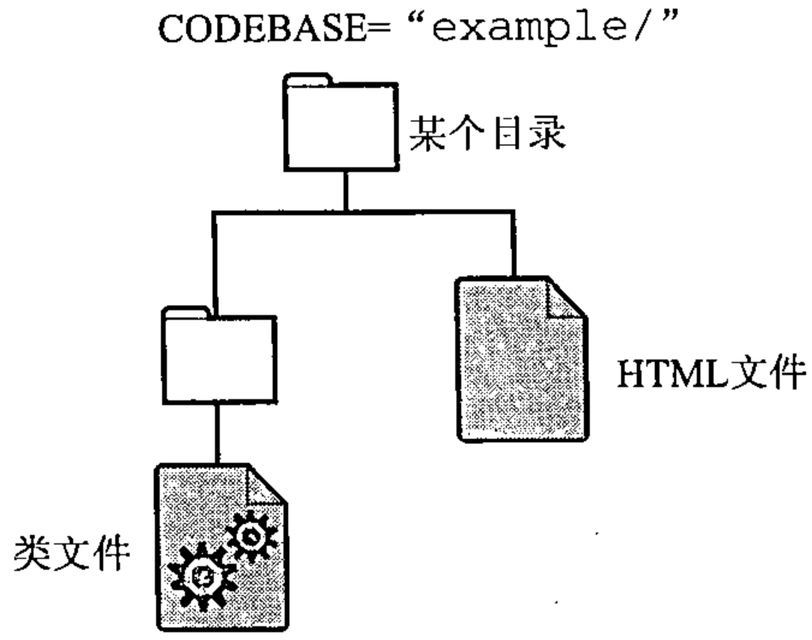


图18-13 当CODEBASE被设置为“example/”时类文件的位置

图18-14显示当把CODEBASE指定为绝对URL时，applet类可能的位置。

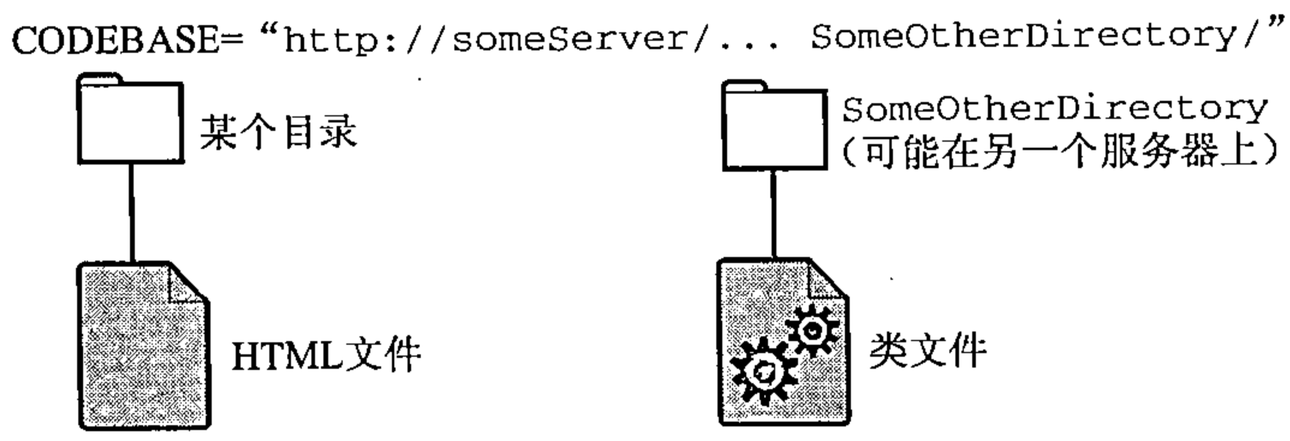


图18-14 当CODEBASE被指定为绝对URL时类文件的位置

4. 把applet的文件组合为一个文件

如果你的applet有多个文件，那么应该考虑提供一个档案文件，此文件将applet的文件打包为单个文件。applet是否需要档案文件取决于几个因素，包括applet的尺寸、性能考虑因素以及你的用户可能具备的环境。

档案文件减少了applet的总下载时间。节省时间主要是因为减少了浏览器必须建立的HTTP连接数量。每个HTTP连接的启动都可能花费几秒。这意味着，对于多文件的applet，连接时间会拖长传输时间。可以通过压缩档案文件中的文件进一步减少下载时间。

如果指定一个或多个档案文件，那么applet类加载器在搜索applet类文件的目录中寻找这些档案文件。然后，applet类加载器在档案文件中寻找applet的类文件。如果一个文件不在档案中，那么浏览器一般会像档案文件不存在那样尝试装载此文件。 577

标准的Java档案格式称为JAR，它是在JDK 1.1中引入的，它建立在ZIP文件格式之上。使用<APPLET>标记的ARCHIVE属性指定JAR文件。可以指定多个档案文件，它们之间用逗号分隔：

```
<APPLET CODE="AppletSubclass.class" ARCHIVE="file1, file2"
  WIDTH=anInt HEIGHT=anInt>
</APPLET>
```

不幸的是，并非所有的浏览器都理解相同的档案格式，或者使用相同的HTML代码指定applet档案。注意了解关于浏览器支持档案的最新信息。为了学习如何创建JAR，请参阅16.1.1节。

5. 其他<APPLET>标记属性

这一小节并未讨论<APPLET>标记的每个属性。其他属性包括ALIGN、VSPACE和HSPACE，你可能熟悉它们，因为HTML的<IMG>标记也使用它们。<APPLET>标记还允许通过指定OBJECT

属性（而不是使用CODE）指定类文件，从而加载串行化的（保存的）applet。最后，你可以使用NAME属性命名applet。关于<APPLET>标记的详细介绍，请参阅18.5.2节。

18.3　编写 applet 时的考虑事项

本章的前两节已经讨论了applet专用的所有API。但是，大多数applet依赖于许多不是applet专用的API。本节提供关于使用Java平台API的提示，讲解受到applet与浏览器的密切关系影响的领域。

18.3.1　安全限制

Java平台的主要目的之一是让用户感到在他们的浏览器内运行任何的applet都是安全的。为达到此目的，我们在开始的时候采取保守态度，可能过多地限制了applet的功能。随着时间的推
578 移，applet可能得到越来越多的能力。

这一小节从安全限制如何影响applet设计的角度出发，讲解当前applet的安全限制。关于applet安全的更多信息，请参考*Frequently Asked Questions —Applet Security*[1]。

每种applet查看器都具有SecurityManager对象，它会检查applet的安全违规情况。当SecurityManager检测到违规时，它创建并且抛出SecurityException对象。通常，SecurityException构造器会把一条警告消息输出到标准输出。applet可以捕获SecurityException异常并且相应地作出响应，比如通过再次向用户保证和通过采取“更加安全”（但是更加不理想）的方式完成任务。

一些applet查看器会“吞掉”SecurityException异常，以致applet永远得不到SecurityException。例如JDK Applet Viewer的AppletContext getApplet和getApplets方法只是捕获并忽略任何SecurityException。用户在标准输出中看不到错误消息，但是至少applet从这些方法得到了合法结果。这具有一定的意义，因为getApplets应该能够返回它找到的任何合法applet，即使遇到了非法的applet。（如果一个applet来自于调用getApplets的applet的同一台主机，Applet Viewer就认为这个applet是合法的。）

为了了解安全管理器和它们检查的安全违规的种类，请参阅14.2.3节。

现有的applet查看器（包括Web浏览器）实施如下限制：

- **applet不能加载库或者定义本地方法。**applet只能使用它们自己的Java代码和applet查看器提供的Java API。最低限度，每种applet查看器必须提供对java.*包中定义的API的访问。
- **applet一般不能读取或者写入执行它的主机上的文件。**对于这条规则，JDK Applet Viewer实际上允许某些用户指定的例外，但是旧的浏览器通常不会。任何applet查看器中的applet都可以读取使用完整URL（而不是文件名）指定的文件。对于不能读写文件这个问题的一个解决办法是，让applet将数据转发到它的来源主机上的一个应用程序。这个应用程序可以读写它自己的主机上的数据文件。更多例子请参阅18.3.5节。
- **除了连接它的来源主机之外，applet不能建立网络连接。**对这个限制的一个解决办法是，让applet和它的来源主机上的一个应用程序一起工作。这个应用程序可以连接到网络上
579 的任何位置。更多例子请参阅18.3.5的第2小节。
- **applet不能在执行它的主机上启动任何程序。**同样，applet可以使用服务器端应用程序作为替代方式。

---

1. http://java.sun.com/sfaq/

- **applet不能读取某些系统属性。**更多信息请参阅18.3.3节。
- **applet显示的窗口外观和应用程序显示的不同。**你可以通过名称“Java Applet Window”（显示在窗口底部）识别applet窗口，应用程序窗口的底部没有任何名称。这帮助用户区分applet窗口和其他可信的应用程序的窗口。

图18-15和图18-16分别显示了应用程序作为applet和作为应用程序运行时的窗口。图18-15显示当程序在Microsoft Windows平台上作为应用程序运行时的窗口。图18-16显示当程序作为applet运行在Windows平台的Mozilla浏览器中时的窗口。

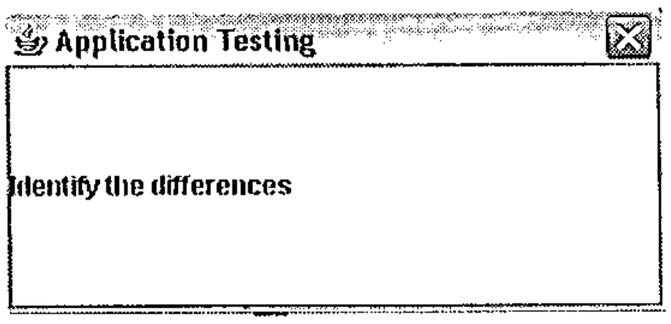


图18-15 作为应用程序运行的程序

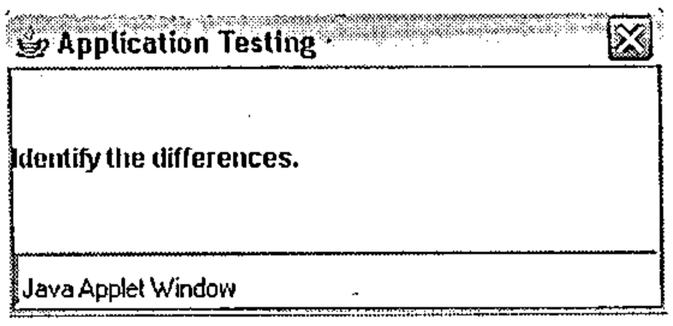


图18-16 作为applet运行的同一个程序

580

如你所见，applet窗口有一个标签，通知用户它正作为applet运行。

18.3.2 创建用户界面

大多数applet都有GUI。这是每个applet都在一个浏览器窗口中出现这一事实的自然结果。因为JApplet类是Applet类的子类，Applet类又是AWT Panel类的子类，并且参与AWT事件和绘制模型，所以创建applet的GUI与创建应用程序的GUI一样容易。实际上则更容易，因为applet的窗口（即浏览器窗口）已经存在了。

除了GUI，根据需要提供或获取的信息类型，applet还可以使用另外几种UI类型。某些applet播放声音，以便向用户提供反馈或者提供其他效果。applet可以通过它定义的参数从用户那里得到配置信息。为了向用户提供文本信息，applet可以使用它的GUI或显示一个简短的状态字符串（用于不重要的文本）或向标准输出或标准错误流显示信息（用于调试）。

关于声音、参数和状态字符串的信息，请参阅18.2节。

1. 创建GUI

本节讨论applet GUI特有的几个问题。在你阅读*Creating a GUI with JFC/Swing*，特别是其中的“How to Make Applets”[1]一节之前，本小节中的一些信息可能对你来说没有意义。这一资料讨论本节中涉及的所有GUI概念：

- **applet出现在预先存在的浏览器窗口中。**这有两个影响。首先，与基于GUI的应用程序不同，applet不必创建用于显示它们自己的窗口。如果有充分理由，applet可以创建窗口，但是它们通常只是将自己显示在浏览器窗口中。其次，根据浏览器的实现，除非你的applet在把组件添加到本身之后调用validate，否则可能不会显示applet的组件。幸运的是，调用validate没有任何影响。
- **applet的背景颜色可能与页面颜色不匹配。**默认情况下，applet具有白色的背景颜色。但是，HTML页面可以具有其他背景颜色，而且可以使用背景图案。如果applet设计者和页面设计者不小心的话，applet与页面的背景颜色可能不同，这会使applet在页面中很突出，而且会造成绘制applet时明显的闪动。一个解决方案是，定义一个指定applet背景颜色的applet参数。JApplet类可以使用JComponent的setBackground方法将applet的背景颜

581

1. tutorial/uiswing/components/applet.html

色设置为Web页面中指定的颜色。通过使用背景颜色参数，页面设计者可以选择与页面颜色匹配的applet颜色。可以在18.2.6节中了解参数。

- **每个applet都具有用户指定的预先决定的尺寸。**<APPLET>标记要求指定applet的宽度和高度。Web设计者可以以像素为单位，或者通过指定浏览器窗口的百分比，设置applet的尺寸。注意，即使设置的空间量对一种平台很理想，但是applet平台特定的部分（比如按钮）在另一种平台上可能需要不同的空间量。可以这样缓解这种问题：让包含applet的页面指定比可能需要的空间量略多的空间，然后使用一种灵活的布局（比如GridBagLayout和BorderLayout类）调整多余的空间。
- **applet使用Applet的getImage方法加载图像。**Applet类提供一种简便的getImage形式，它有两个实参，第一个实参指定一个基URL，第二个实参指定图像文件相对于基URL的位置。Applet的getCodeBase和getDocumentBase方法提供大多数applet使用的基URL。applet经常需要（或者作为备份需要）的图像通常被指定为相对于加载applet代码的位置（代码基（code base））。applet用户（经常在HTML文件中使用参数）指定的图像通常相对于包含applet的页面的位置（文档基（document base））。
- **applet的类（还有它们使用的数据文件）是通过网络加载的，这个过程可能较慢。**applet可以通过几种方式减少感觉上的启动时间。applet子类可以很小，它随即显示一条状态消息。如果不马上使用applet的类或者数据，applet就可以在后台线程中预加载这些类或者数据。

例如，AppletButton[1]类的start方法启动一个线程，获得按钮打开的窗口的Class[2]对象。applet这样做的主要目的是确保用户指定的类名称是合法的。另外的优势是，获得Class对象会强制在实例化类之前加载类文件。当用户要求创建窗口时，applet实例化窗口类的速度就会快得多，因为窗口类文件已经加载了。

582

2. 向标准输出和错误流显示诊断信息

在调试applet时，向标准输出或标准错误流显示诊断信息可能是宝贵的手段。当applet中发生未被捕获的异常时，也可以在标准输出或标准错误流中看到消息。applet还提供使用标准错误流的选项。

显示标准输出和标准错误流的确切位置是不同的，这取决于applet查看器是如何实现的、查看器运行在什么平台上以及（有时候）如何启动浏览器或者applet查看器。例如，当从UNIX shell窗口启动Applet Viewer时，发送到标准输出和标准错误的字符串出现在这个shell窗口中，除非你对输出进行了重定向。当从X Window菜单启动Applet Viewer时，标准输出和错误出现在控制台窗口中。

applet使用System.out.print(*String*)和System.out.println(*String*)向标准输出进行显示。向标准错误流进行显示与之类似，用System.err替代System.out即可。下面是一个向标准输出进行显示的例子：

```
// Where instance variables are declared:
boolean DEBUG = true;
...
// Later, when we want to print some status:
if (DEBUG) {
  System.out.println("Called someMethod(" + x + "," + y + ")");
}
```

---

1. tutorial/uiswing/layout/example-swing/AppletButton.java
2. docs/api/java/lang/Class.html

注意 向标准输出和标准错误流进行显示相对来说速度很慢。如果你遇到了一个与计时相关的问题，那么将消息发送到这些流可能帮助不大。

在发布你的applet前，要确保关闭所有调试输出。.

18.3.3 获得系统属性

为了了解当前的工作环境，applet可以读取系统属性值。系统属性是键/值对，它们包含运行applet的操作系统等信息。在14.1.1节中对系统属性有深入讲解。 583

applet可以获得一些系统属性，但不是全部。

1. applet可以读取的系统属性

表18-1列出applet可以读取的系统属性。

表 18-1 合法系统属性

键	含 义
“file.separator”	文件分隔符（例如“/”）
“java.class.version”	Java 类版本号
“java.vendor”	Java 厂商专有字符串
“java.vendor.url”	Java 厂商 URL
“java.version”	Java 版本号
“line.separator”	行分隔符
“os.arch”	操作系统架构
“os.name”	操作系统名称
“path.separator”	路径分隔符（例如“:”）

为了从applet内读取系统属性，applet使用System类的方法getProperty。例如：

```
String newline = System.getProperty("line.separator");
```

运行一个applet[1]，它读取对所有applet都可用的属性（见图18-17）。

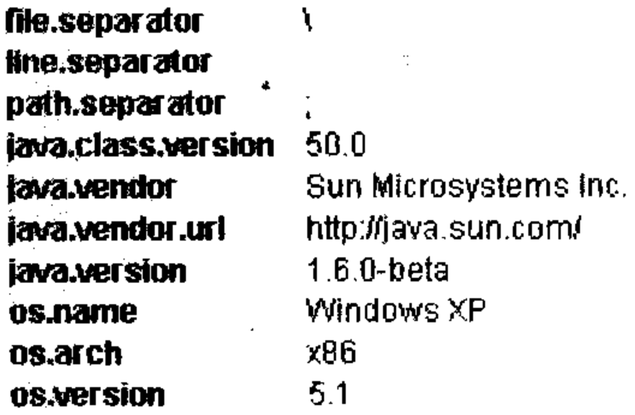


图18-17 GetOpenProperties applet的屏幕截图

源代码在GetOpenProperties.java中[2]。

584

2. 禁止访问的系统属性

出于安全原因，现有的浏览器或者applet查看器不允许applet读取的系统属性如表18-2所示。

1. tutorial/deployment/applet/examples/properties.jar
2. tutorial/deployment/applet/examples/GetOpenProperties.java

表18-2　禁止访问的系统属性

键	含　义
“java.class.path”	Java类路径
“java.home”	Java安装目录
“user.dir”	用户的当前工作目录
“user.home”	用户的主目录
“user.name”	用户的账户名

18.3.4　applet 中的线程

注意　本节假设你已熟悉线程，所以不解释基本的线程代码。如果你还不熟悉线程，那么在阅读本节之前请参考12.1节。

每个applet都可以在多个线程中运行。applet的GUI在事件调度线程上创建。具体由哪些线程执行主要由里程碑方法（init、start、stop和destroy）取决于运行applet的应用程序。但是应用程序不会从事件处理线程调用里程碑方法。

很多浏览器可能为页面中的每个applet分配一个线程，并将这个线程用于所有对applet里程碑方法的调用。一些浏览器为每个applet分配一个线程组，这样就很容易杀死属于某个applet的所有线程。无论在哪种情况下，都要保证一个applet的任何里程碑方法创建的每个线程都属于相同的线程组。

运行PrintThread applet[1]（见图18-18）。PrintThread是SimpleApplet经过修改的版本，它输出调用init、start、stop、destroy和update方法的线程和线程组。[2]

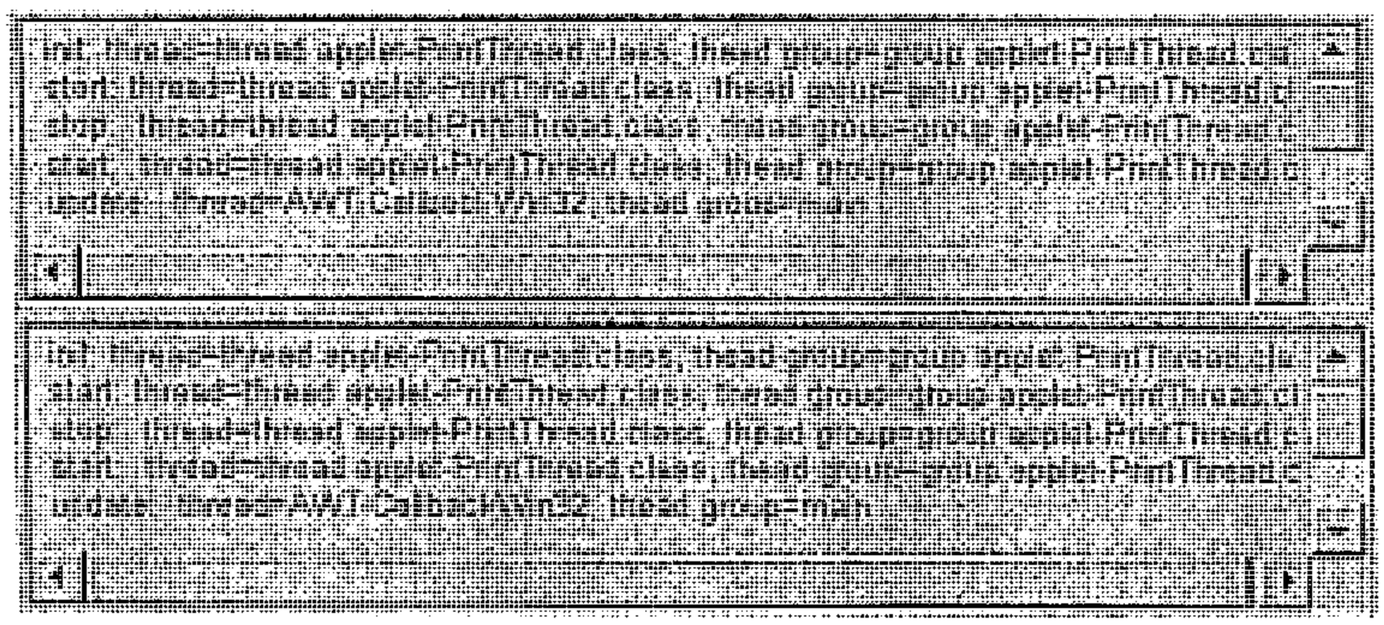


图18-18　PrintThread applet的屏幕截图

通常为了查看在卸载过程中调用的方法（比如destroy）的输出时需要查看标准输出。为了查看在浏览器中运行的applet的标准输出，从浏览器的Tools菜单打开Java Console。关于标准输出流的信息请参阅18.3.2的第2小节。

那么，applet为什么需要创建和使用它自己的线程呢？假设一个applet在它的init方法中执行某些耗时较长的初始化工作，例如加载声音文件。调用init的线程在init返回之前不能做其他的任何事。在某些浏览器中，这可能意味着在applet完成本身的初始化之前，浏览器不能显示

1. tutorial/deployment/applet/examples/threads.jar
2. tutorial/deployment/applet/examples/PrintThread.java

applet或它后面的任何东西。所以，如果此applet在页面的顶部，那么在applet完成初始化之前，页面上什么也不会显示。

即使在那些为每个applet创建单独线程的浏览器中，将耗时较长的任务放入一个applet创建的线程中也是有意义的，这样applet就可以在等待耗时较长的任务完成的同时执行其他任务。

规则 如果applet执行耗时较长的任务，就应该创建和使用它自己的线程来执行此任务。

applet通常会执行两种耗时较长的任务：反复执行的任务和一次性执行的任务。下一小节给出两种任务的例子。

applet中的线程：示例

这一小节讨论在applet中使用线程的两个例子。第一个applet是AnimatorApplet，显示如何使用线程执行重复的任务。第二个applet是SoundExample，显示如何使用线程执行一次性任务。SoundExample的特性在18.2.5节中讲解过。

585 ~ 586

这一小节并不讲解基本线程代码。为了了解Java的线程实现，请参阅12.2.1节。

(1) 使用线程执行重复的任务

重复地执行相同任务的applet通常会用一个线程来执行此任务的while（或do...while）循环。典型的例子是执行定时动画的applet，比如电影播放器或游戏。动画applet需要一个定期要求重新绘制的线程。另一个例子是读取服务器端应用程序提供的数据的applet。（这样的例子参见18.3.5节的第2小节。）

applet通常在它的start方法中创建用于执行重复性任务的线程。在此处创建线程就可以在用户离开页面时很容易地停止此线程。需要做的只是实现stop方法，让它停止线程。当用户返回applet的页面时，再次调用start方法，applet可以再次创建用于执行重复性任务的线程。

下面是AnimatorApplet的start方法和stop方法的实现：

```
public void start() {
  if (frozen) {
      // Do nothing. The user has requested that we
      // stop changing the image.
  } else {
      // Start animating!
      if (animatorThread == null) {
        animatorThread = new Thread(this);
      }
      animatorThread.start();
  }
}

public void stop() {
  animatorThread = null;
}
```

new Thread(this)中的this表示由applet提供此线程的体。它通过实现java.lang.Runnable接口来完成，此接口要求applet提供一个形成线程体的run方法。我们稍后将更多地讨论AnimatorApplet的run方法。

注意，AnimatorApplet类中没有调用Thread stop方法。原因是，调用Thread stop方法相当于直接杀死线程。这是让线程停止操作的一种生硬而激烈的方式。相反地，应该以某种方式编写线程的run方法，让线程在通知它时平稳地退出。发出通知的方式可以是把类型为Thread类型的实例变量设置为null。

587

在AnimatorApplet中，这个实例变量被称为animatorThread。start方法将它设置为指向新创建的Thread对象。在需要杀死此线程时，applet将animatorThread设置为null。这并不是通过使线程被垃圾收集来杀死线程（在线程可以运行时，它不能被垃圾收集），但是线程在循环的顶部检查animatorThread，并根据animatorThread的值决定是继续运行还是退出。下面是相关的代码：

```
public void run() {
  ...
  while (Thread.currentThread() == animatorThread) {
    ...// Display a frame of animation and then sleep.
  }
}
```

如果animatorThread指向的线程与当前执行的线程相同，那么线程将继续执行。另一方面，如果animatorThread是null，那么线程则退出。如果animatorThread指向另一个线程，那么说明发生了赛跑状态：在stop之后快速地调用start（或者这个线程在其循环内花了很长的时间），以至在这个线程到达循环的顶端之前，start就创建了另一个线程。无论赛跑状态的原因是什么，这个线程都应该退出。

关于动画applet的更多信息，请参阅*Creating a GUI with JFC/Swing*中的“Threds in Applets”一节。

(2) 使用线程执行一次性初始化

如果你的applet需要执行耗时较长的一些初始化任务，那么应该考虑在线程中执行初始化。例如，任何要求建立网络连接的任务一般应该在一个后台线程中执行。幸运的是，GIF、PNG和JEPG图像的加载都是使用线程自动地在后台完成的，这不必担心。

但不幸的是，声音的加载并不保证在后台进行。在当前实现中，Applet getAudioClip方法在加载完所有音频数据前不返回。其结果是，如果你希望预加载声音，就可能需要创建一个或多个线程来进行。

使用线程执行applet的一次性初始化任务是典型生产者/消费者场景的一个变形。执行任务
588 的线程是生产者，而applet是消费者。12.3节中讨论了在生产者/消费者场景中如何使用Java线程。

SoundExample严格地遵循同步线程中表现的模型。和同步线程示例一样，SoundExample使用3个类[1]：

- 生产者：SoundLoader，一个Thread的子类。
- 消费者：SoundExample，一个Applet的子类。和同步线程消费者示例不同，SoundExample不是Thread；它甚至没有实现Runnable接口。但是，SoundExample实例方法由至少两个线程执行，这取决于执行SoundExample applet的应用程序。
- 存储对象：SoundList，一个Hashtable的子类。和同步线程示例中的CubbyHole不同，如果还没有存储声音数据，SoundList返回null。这对applet有意义，因为它需要能够立即响应要求播放声音的用户，即使声音还没有加载。

关于SoundExample的更多信息，请参阅18.2.5节。

18.3.5　处理服务器端应用程序

和其他Java程序一样，applet可以使用java.net包中定义的API通过网络进行通信。唯一的区别是，出于安全原因，applet只能和它来源于的主机进行通信。

---

1. tutorial/deployment/ applet/examples/SounExample.java

注意 根据加载applet的网络环境以及运行applet的浏览器，applet可能不能和它来源于的主机进行通信。例如，运行在防火墙之内的主机经常不能获得防火墙之外的信息。其结果是，一些浏览器可能不允许applet和防火墙之外的主机进行通信。

很容易查明applet来源于哪台主机。只需使用Applet getCodeBase方法和java.net.URL getHost方法，例如：

```
String host = getCodeBase().getHost();
```

得到正确的主机名称之后，就可以使用在线指南中定制网络（Custom Networking）文档中介绍的所有联网代码。[1]

589

注意 并非所有浏览器都无缺陷地支持全部联网代码。例如，一种使用广泛的兼容Java技术的浏览器就不支持发送到URL。

1. 简单网络客户applet示例

这一小节介绍一个作为applet的客户程序[2]。这个客户不仅和applet来源于的主机进行通信，而且具有图形UI，使用循环按照期望获得任意数量的名言。你可以通过把它包含在页面中，运行这个applet；下面是页面的HTML代码：

```
<APPLET CODE=QuoteClientApplet.class WIDTH=500 HEIGHT=100>
</APPLET>
```

quoteApplet.html[3]页面包含前面的HTML代码。把这个页面保存到本地HTTP服务器上的一个文件，可以使用它和运行在这台HTTP服务器上的服务器端应用程序进行通信。还必须把applet的编译后的形式保存到相同目录。

在applet获得名言之前，需要在applet来源于的主机上运行服务器程序。然后记下服务器正在监听的端口号。把这个端口号输入applet之后，它就连接到了服务器，就能够获得一句名言。下面是详细的说明，以及运行中的服务器和applet的表现：

(1) 编译QuoteServer.java[4]和QuoteServerThread.java[5]。一个文本文件（one-liners.txt[6]）应该和生成的类文件放在同一个目录中。

(2) 在为applet类文件服务（通过HTTP）的计算机上，对QuoteServer类调用解释器。例如，如果使用URL http://mymachine/quoteApplet.html浏览applet的页面，就需要在名为mymachine的主机上运行服务器。

(3) 记录名言服务器显示的端口号。

(4) 在applet的文本字段中输入这个端口号。

(5) 按下Send按钮向服务器请求名言。可以看到名言出现在文本区域中。

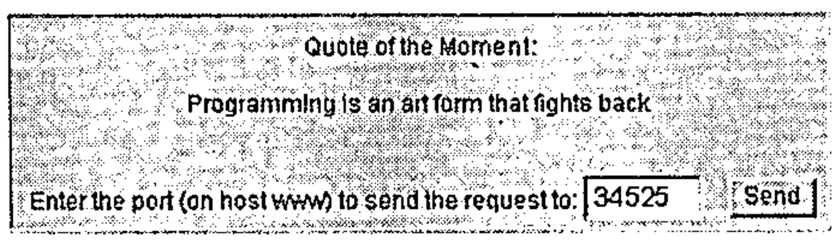


图18-19显示运行中的applet。

图18-19 QuoteServer applet的屏幕截图

1. tutorial/networking/index.html
2. tutorial/deployment/applet/examples/QuoteClientApplet.java
3. tutorial/deployment/applet/examples/quoteApplet.html
4. tutorial/deployment/applet/examples/QuoteServer.java
5. tutorial/deployment/applet/examples/QuoteServerThread.java
6. tutorial/deployment/applet/examples/one-liners.txt

2. 使用服务器处理安全限制

在18.1.6节中解释过，applet受到很多安全限制。例如，applet不能执行文件I/O，不能建立网络连接（除非是连接它们来源的主机），而且不能启动程序。

解决这些安全限制的一个途径是，使用在applet的主机上执行的服务器应用程序。服务器不能解决applet的每种限制，却能够实现更多功能。例如，服务器可能不能在运行applet的主机上保存文件，但是它可以把文件保存在applet来源的主机上。

这一小节介绍一个服务器的示例，它允许两个applet相互通信。这两个applet不必运行在同一个页面上或在同一个浏览器中或者在同一台计算机上。只要这两个applet来自于同一台计算机，它们就可以通过运行在来源计算机上的服务器进行通信。示例使用套接字，在套接字详解（All About Sockets）文档中详细介绍了套接字。[1]

下面是源文件：

- **TalkClientApplet.java**[2]——客户applet的源文件。编译它之后，使用下面的标记把它包含在HTML页面中，就可以运行了：

```
<APPLET CODE=TalkClientApplet.class WIDTH=550 HEIGHT=200>
</APPLET>
```

590 ~ 591

  `talk.html`[3]页面包含上面的HTML代码。把这个页面保存在本地HTTP服务器上后，就可以使用它和TalkServer进行通信。

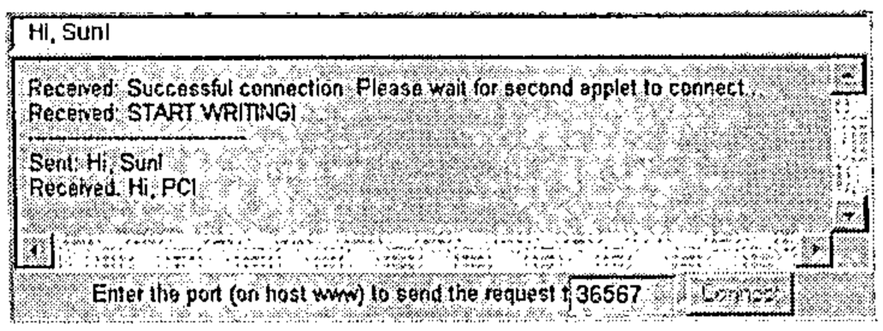
- **TalkServer.java**[4]和**TalkServerThread.java**[5]——服务器applet的源文件。编译这两个文件之后，通过对`TalkServer`调用解释器，可以在applet的来源主机上运行服务器applet。

运行服务器的步骤和前一个示例完全一样（参见18.3.5节的第1小节）。在applet的来源主机上运行服务器，记录applet应该使用的端口号。然后将两个applet（它们运行在不同计算机上）初始化为通过服务器端口号通话。初始化完成之后，在每个applet的文本字段中输入一个字符串。然后按下回车键把消息发送给另一个applet。

下面是运行中的服务器：

```
www% java TalkServer
TalkServer listening on rendezvous port: 36567
```

图18-20和图18-21显示运行中的applet。



592　图18-20　示例通话输出——计算机＃1

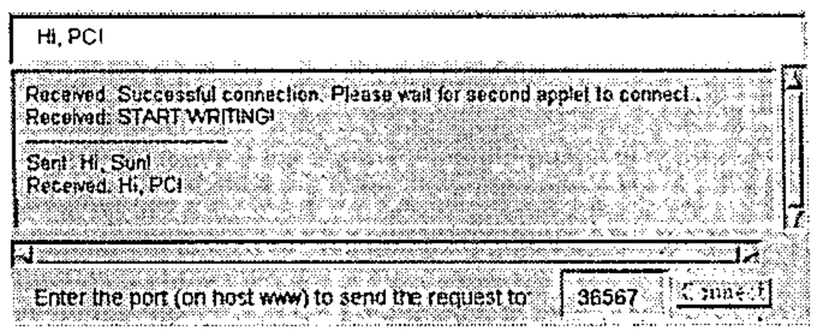


图18-21　示例通话输出——计算机＃2

18.4　完成 applet

18.4.1　发布 applet 前的工作

暂停！在发布你的applet前，必须确保以下所有问题都得出肯定的回答。

---

1. tutorial/networking/sockets/index.html
2. tutorial/deployment/applet/examples/TalkClientApplet.java
3. tutorial/deployment/applet/examples/talk.html
4. tutorial/deployment/applet/examples/TalkServer.java
5. tutorial/deployment/applet/examples/TalkServerThread.java

1. 是否已经删除或禁止了调试输出？

尽管调试输出（一般是用System.out.println创建的）对开发人员很有用，但常常会打扰或激怒用户。如果需要为用户提供文本化的反馈，应该在applet的显示区域或窗口底部的状态区域中显示。关于使用状态区域的信息见18.2.2节。

2. 当applet从屏幕上消失时是否停止运行？

当浏览器被最小化或正在显示不包含applet的页面时，大多数applet不再使用CPU资源。如果你的applet代码没有显式地启动任何线程，那么就没有问题。

如果你的applet代码启动了任何线程，那么应该实现stop方法，让它停止和销毁线程（通过设置为null），除非你有真正合理的理由不这么做。实现stop方法的例子见18.3.4节的第1小节。

3. 如果applet执行一些可能打扰用户的程序，比如播放声音或动画，那么是否为用户提供了停止这些操作的办法？

对你的用户体贴些。让他们有办法停止applet，而不必关闭页面。在响应鼠标点击的applet中，可以实现mouseDown方法，以便通过鼠标点击暂停或者恢复运行烦人的线程。例如：

```
boolean frozen = false; // an instance variable

public boolean mouseDown(Event e, int x, int y) {
  if (frozen) {
     frozen = false;
     start();
  } else {
     frozen = true;
     stop();
  }
  return true;
}
```

593

18.4.2 圆满完成的 applet

前一节列出了一些方式，通过这些方式可以避免你的applet用户对你不满。本节提供的一些其他方式可以使你的applet尽可能地令人满意。

- **使你的applet尽可能地灵活。**通常可以通过定义参数，使applet可以用于各种情况，而不必重新编写代码。更多信息请参考18.2.6节。
- **使你的applet可访问。**可以将你的applet设计为所有用户都能访问。更多信息请参阅指南的“Swing”一章中“如何支持辅助技术（How to Support Assistive Technologies）”一节[1]。
- **实现getParameterInfo方法。**现在实现这个方法可能使你的applet在将来更容易定制。当前，还没有浏览器使用这个方法。但是，我们预期浏览器能很快使用此方法，帮助产生一个允许用户交互地设置参数值的GUI。关于实现getParameterInfo方法的信息，请参考18.2.6节的第3小节。
- **实现getAppletInfo方法。**这个方法返回一个描述applet的简短信息性字符串。虽然当前还没有浏览器使用这个方法，但是我们预期将来浏览器能使用它。下面是实现getAppletInfo方法的一个例子：

```
public String getAppletInfo() {
  return "GetApplets by Kathy Walrath";
}
```

1. tutorial/uiswing/misc/access.html

18.5　部署 applet

本节讲解HTML作者如何和何时使用applet、object和embed标记把Java applet添加到Web
[594] 页面中，并且提供在因特网和内联网上部署applet以及在不同浏览器中使用applet的指导方针。

18.5.1　一般的部署考虑事项

如何部署applet取决于用户是通过因特网还是内联网访问Web页面，以及他们使用的浏览器的类型。收集关于你的用户的信息，然后遵守下面的总体指导方针。

1. 在因特网和内联网上部署applet

在部署applet时：

- 如果用户通过因特网，或者通过网内使用不同浏览器的内联网访问Web页面，则使用applet标记。
- 如果通过内联网，并且你知道用户都使用哪种浏览器访问Web页面，则使用object或者embed标记。

2. 为特定浏览器部署applet

在部署applet时：

- 如果只针对Internet Explorer，则使用object标记。
- 如果只针对Mozilla系列的浏览器，则使用embed标记。

如果你必须把applet部署到多种浏览器的环境中，那么遵守18.5.5节提供的指导方针。

18.5.2　使用 applet 标记

使用applet标记把applet部署到多种浏览器的环境中。

关于applet标记的完整介绍，请参阅W3 HTML规范[1]。

注意　HTML规范声明applet标记已被废除，应该使用object标记。但是，规范对于浏览器应该如何实现object标记来支持Java applet的说明含混不清，并且当前浏览器的支持不一致。
[595] 因此建议你继续使用applet标记作为在所有平台上跨浏览器部署Java applet的一致性方式。

下面是applet标记的一个例子：

```
<applet code=Applet1.class width="200" height="200">
Your browser does not support the applet tag.
</applet>
```

对于Internet Explorer和Mozilla系列浏览器，如果安装了Java Plug-in（版本1.3.1_01a或更高），那么就会调用最新安装的Java Plug-in版本来运行applet。

注意　如果JRE没有安装在本地计算机上，就不能使用applet标记自动下载JRE。

18.5.3　使用 object 标记

使用object标记部署只用于Internet Explorer的applet。

关于object标记的完整介绍，请参阅W3 HTML规范。

---

1. http://www.w3.org/TR/1999/REC-html401-19991224/

下面是object标记的一个例子：

```
<OBJECT
  classid="clsid:8AD9C840-044E-11D1-B3E9-00805F499D93"
  width="200" height="200">
  <PARAM name="code" value="Applet1.class">
</OBJECT>
```

1. classid属性

classid属性来定义使用Java Plug-in的哪个版本。

下面的例子是classid属性最常用的形式。这个例子指示Internet Explorer使用最新安装的Java Plug-in版本：

```
classid="clsid:8AD9C840-044E-11D1-B3E9-00805F499D93"
```

下面是classid属性的另一种形式：

```
classid="clsid:CAFEEFAC-xxxx-yyyy-zzzz-ABCDEFFEDCBA"
```

596

在这种形式中，“xxxx”、“yyyy”和“zzzz”是标识要使用的Java Plug-in的特定版本的四位数字。

例如，为了使用Java Plug-in版本1.6.0，需要指定：

```
classid="clsid:CAFEEFAC-0016-0000-0000-ABCDEFFEDCBA"
```

2. codebase属性

在系统没有安装JRE的情况下，使用可选的codebase属性指定下载JRE的位置。

codebase属性有两种形式：

- **codebase=<URL>**——如果本地没有安装classid属性指定的JRE，那么通过这种形式提示用户从指定的URL下载JRE。
- **codebase=<URL>#Version=major, minor, micro, update**——通过这种形式，Java Plug-in会比较本地安装的JRE classid和必须的JRE版本，如果必须的版本更高，那么就提示用户下载和安装JRE的更新版本。

下面是使用codebase属性设置从Sun的Java Web网站进行自动下载的一个例子：

```
<object
  classid="clsid:8AD9C840-044E-11D1-B3E9-00805F499D93"
  width="200" height="200"
  codebase="http://java.sun.com/update/1.6.0/
    jinstall-6-windows-i586.cab#Version=6,0,0,99">
  <param name="code" value="Applet1.class">
</object>
```

注意 在这个例子中，代码codebase=http://java.sun.com...被分为两行以便容易阅读。在实际的HTML文件中应该只占一行。

Sun公司把JRE安装器的每个版本都打包为Microsoft的cabinet（.cab）文件格式。你可以浏览这些版本的清单和相应的.cag文件名。[1]

597

18.5.4 使用 embed 标记

使用embed标记部署只用于Mozilla系列浏览器的applet。

下面是embed标记的一个例子：

1. docs/technotes/guides/deployment/deployment-guide/autodl-files.html

```
<embed code="Applet1.class"
  width="200" height="200"
  type="application/x-java-applet;version=1.6.0"
  pluginspage="http://java.sun.com/javase/downloads"/>
```

type属性有两种形式：

- **type="application/x-java-applet;version=1.6.0"**——通过这种形式，调用支持MIME类型application/x-java-applet;version=1.6.0的已经安装的最高版本JRE来运行applet。如果本地安装的JRE的版本号等于或者高于指定的版本号，则使用本地安装的JRE。否则引导用户到pluginspage属性值指定的URL，并且提示用户下载和安装必须的JRE。
- **type="application/x-java-applet;jpi-version=1.6.0_01**——通过这种形式，调用jpi-version的值（在这个例子中是1.6.0_01）指定的JRE版本来运行applet。否则引导用户到pluginspage属性值指定的URL，并且提示用户下载和安装必须的JRE。

18.5.5　在混合浏览器环境中部署 applet

可以通过下面两种方式，部署applet以便Internet Explorer和Mozilla系列浏览器的用户都可以使用：

- 通过纯HTML。
- 通过JavaScript。

1. 使用纯HTML

598

使用纯HTML的方式在多种浏览器环境中部署applet时，要注意如下事项：

- **Internet Explorer：**
  - 识别object标记。
  - 忽略comment标记的内容。
- **Mozilla浏览器：**
  - 忽略使用classid属性的object标记。
  - 解释comment标记的内容。

参考下面HTML页面中的代码示例：

```
<object classid="clsid:CAFEEFAC-0016-0000-0000-ABCDEFFEDCBA"
  <param name="code" value="Applet1.class">
    <comment>
      <embed code="Applet1.class"
            type="application/x-java-applet;jpi-version=1.6">
        <noembed>
          No Java Support.
        </noembed>
      </embed>
    </comment>
</object>
```

2. 使用JavaScript

除了前面描述的纯HTML方式，还可以使用JavaScript在混合浏览器环境中部署applet。

通过JavaScript，可以：

(1) 通过appName变量检测用户的浏览器。

(2) 使用document.write()方法，根据appName变量的值写入标记：

a. 如果浏览器的名称等于“Netscape”，则写入embed标记。

b. 如果浏览器的名称等于“Microsoft Internet Explorer”，则写入object标记。

在下面的例子中，document.write()方法为每个用户动态地输出embed或者object标记： 599

```
<html>
<script language="Javascript">

  var _app = navigator.appName;

  if (_app == 'Netscape') {
    document.write('<embed code="Applet1.class"',
      'width="200"',
      'height="200"',
      'type="application/x-java-applet;version=1.6">');
    }
  else if (_app == 'Microsoft Internet Explorer') {
    document.write('<OBJECT ',
      'classid="clsid:8AD9C840-044E-11D1-B3E9-00805F499D93"',
      'width="200"',
      'height="200">',
      '<PARAM name="code" value="Applet1.class">',
      '</OBJECT>');
    }
  else {
    document.write('<p>Sorry, unsupported browser.</p>');
    }

</script>
</html>
```

可以使用HTML转换器（HTML Converter）工具[1]帮助生成在混合环境中使用的object和embed标记。

18.6 解决常见的 applet 问题

本节讨论编写applet时可能遇到的一些常见问题。每个问题的后面都给出了可能的解决方案。

问题：Applet Viewer声称在我的HTML页面中没有<APPLET>标记，但确实有<APPLET>标记。

- 检查是否有applet结束标记：</APPLET>。

问题：我重新编译了我的applet，但applet查看应用程序没有显示新版本，即使让它重新装载页面也不行。

- 在许多applet查看器中（包括浏览器），重新加载并不可靠。这就是我们建议使用SDK Applet Viewer的原因，每次修改applet时都会重新调用它。 600
- 如果有applet的旧版本，不管你怎么操作，确保不要将applet的旧副本放在CLASSPATH中的目录中。关于CLASSPATH环境变量的信息，请参阅7.1.4节。

问题：当applet被绘制到使用另一种颜色的页面上时，它的背景颜色不匹配或者闪烁。

- 需要设置applet的背景颜色，使它与页面颜色相适应。详细介绍请参阅18.3.2节的第1小节。

问题：Applet的getImage方法不起作用。

- 确保从init方法中或在init之后调用的方法中调用getImage。如果从构造器调用getImage方法，就不会起作用。

问题：我已经将applet类文件复制到HTTP服务器，但是applet不起作用。

- 你的applet是否定义了多个类？如果是，确保HTTP服务器上有每个类的类文件

1. docs/technotes/guides/plugin/developer_guide/html_converter.html

(*ClassName*.class)。即使所有类都是在一个源文件中定义的，编译器也会为每个类生成一个类文件。

- 是否将applet的所有数据文件（例如图像和声音文件）复制到了服务器？
- 确保applet的所有类文件和数据文件可以被任何人读取。
- 确保applet的类文件和数据文件在传输期间没有被破坏。一个常见的原因是使用FTP的ASCII模式（而不是二进制（BINARY）模式）传输文件。

问题：我的Web页面不能加载applet。我在Java控制台中看到错误消息："java.lang.UnsupportedClassVersionError: Bad version number in .class file"。

- 问题的原因在于，编译applet的源代码使用的Java版本比你安装在系统上的版本要新。你使用的JRE如果不是最新版本，确保下载和安装最新的Java SE运行时环境（Java SE Runtime Environment，JRE）[1]。

开发者提示　可以使用JDK 6的编译时选项source－1.2和target－1.2编译你的applet，这样就可以使用JRE的旧版本运行它们。

601

问题和练习：Java applet

问题

1. applet可以扩展哪些类？
2. 当其中的数据改变时，如何刷新浏览器中的applet GUI？
3. 使用start()方法做什么？
4. 是或否：applet可以建立到因特网上任何主机的网络连接。
5. 如何从applet代码中获得APPLET标记指定的参数值？
6. 是或否：applet可以运行多个线程。
7. 把下面的标记名称和后面的描述列表匹配起来：

   (1) EMBED标记。

   (2) APPLET标记。

   (3) OBJECT标记。

   a. 用于在多种浏览器的环境中部署applet。

   b. 用于在只使用Mozilla系列浏览器的环境中部署applet。

   c. 用于在只使用Internet Explorer的环境中部署applet。

练习

1. 对于使用Exercise类的applet，编写APPLET标记把ButtonName参数设置为Save。
2. 编写方法在浏览器中显示一个applet，使内容包含在围绕短语"Exercise Applet"的长方形中。将applet的尺寸设置为比Web页面指定的尺寸小一个像素，并且短语在坐标5，15的位置开始。

答案

可以在以下位置找到"问题"和"练习"的答案：

602

```
tutorial/deployment/applet/QandE/answers.html
```

1. http://java.sun.com/javase/downloads

附录 A

Java语言关键字

表A-1中列出Java语言中的关键字。你不能使用这些单词作为程序中的标识符。尽管当前并未使用，但关键字const和goto是保留的。true、false和null看起来像关键字，但它们实际上是字面量，所以也不能将它们用作程序中的标识符。

表 A-1　Java 语言关键字

abstract	continue	for	new	switch
assert[3]	default	goto[1]	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum[4]	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp[2]	volatile
const[1]	float	native	super	whilex

注：1. 未使用。

2. 在1.2中添加。

3. 在1.4中添加。

4. 在5.0中添加。

附录 B

Java编程语言认证考试的准备

Sun公司提供Solaris管理员和不同类型的Java程序员的认证考试。而且Sun公司还为每种类型的认证提供培训课程和实践测验。

本书中的很多资料都和成为“Sun认证的Java平台程序员”的考试准备工作有关。

下面是一些有用的链接：

- `http://www.sun.com/training/certification`。提供对各种认证及其相关学习工具的概述。
- `http://www.sun.com/training/certification/java/index.html`。Java技术认证的专门信息。
- `http://www.sun.com/training/certification/objectives`。每种Java认证的“测试目标（Testing Objectives）”的链接列表——测验中涵盖内容的清单。

如果打开最后一个链接并且选择Sun认证的Java平台程序员（Sun Certified Programmer for the Java Platform）的测试目标，就会看到包含7个领域的项目的目标列表：

(1) 声明、初始化和范围。

(2) 流控制。

(3) API内容。

(4) 并发。

(5) 面向对象（OO）概念。

(6) 集合 / 泛型。

(7) 基础。

在下面的介绍中，引用“Sun认证的Java平台程序员”的测试目标列表中的每个领域项目，然后指出本书中包含相关资料的章节。

声明 本书的目的不是作为认证考试的入门读物，因为没有为每个主题提供适合考试深度的资料。提供下面的章节引用是为了让读者可以使用本书作为准备考试的众多资料之一。

第 1 节：声明、初始化和范围

项目 1

“开发声明类（包括抽象类和所有形式的嵌套类）、接口和枚举的代码，并且包括正确地使用包和导入语句（包括静态导入）。”

- 声明类（4.1.1节）。
- 抽象方法和抽象类（5.2.11节）。
- 嵌套类（4.4节）。

- 接口（5.1节）。
- 枚举类型（4.5节）。
- 创建和使用包（7.1节）。
- 导入语句、静态导入：使用包成员（7.1.3节）。

项目 2

“开发声明接口的代码。开发实现或者扩展一个或者多个接口的代码。开发声明抽象类的代码。开发扩展抽象类的代码。”

- 定义接口（5.1.4节）。
- 实现接口（5.1.5节）。
- 抽象方法和抽象类（5.2.11节）。

606

项目 3

“开发代码来声明、初始化和使用基本数据类型、数组、枚举和对象，作为静态、实例和局部变量。此外，为变量名称使用合法标识符。”

- 基本数据类型（3.1.2节）。
- 数组（3.1.3节）。
- 枚举类型（4.5节）。
- 了解实例成员和类成员（4.3.4节）。
- 变量（3.1节）。
- 声明成员变量（4.1.2节）。

项目 4

“开发声明静态和非静态方法的代码，并且（如果适当）使用符合JavaBeans命名标准的方法名称。此外，开发代码来声明和使用可变长度实参清单。”

- 理解实例和类成员（4.3.4节）。
- 把信息传递给方法或构造器（4.1.5节）。

项目 5

“给定一个代码示例，判断一个方法是否正确地重载或者覆盖另一个方法，以及识别方法的合法返回值（包括协变返回）。”

- 覆盖和隐藏方法（5.2.6节）。
- 重载：定义方法（4.1.3节）。
- 从方法返回值（4.3.1节）。

项目 6

“给定一个类和超类的集合，开发一个或者多个类的构造器。给定一个类的声明，判断是否将创建默认构造器，如果会创建，判断构造器的行为。给定一个嵌套或者非嵌套类的代码清单，编写代码来实例化这个类。”

- 为类提供构造器（4.1.4节）。
- 使用关键字super（5.2.8节）。

607

- 创建对象（4.2.1节）。
- 嵌套类（4.4节）。

第 2 节：流控制

项目 1

“开发实现if或者switch语句的代码；并且识别这些语句的合法实参类型。”

- if-then和if-then-else语句（3.4.1节）。
- switch语句（3.4.2节）。

项目 2

“开发实现所有形式的循环和迭代器的代码，包括for循环、增强for循环（for -each）、do、while、标签、break和continue的使用；并且解释在循环执行期间和执行之后循环计数器变量具有的值。”

- for语句（3.4.4节）。
- while和do-while语句（3.4.3节）。
- 分支语句（3.4.5节）。

项目 3

“开发利用断言的代码，并且区分正确和错误的使用方式。”

- 断言示例：问题和练习：类。

项目 4

“开发代码，利用异常和异常处理子句（try、catch和finally），并且声明和覆盖抛出异常的方法。”

- 捕获和处理异常（9.3节）。
- try块（9.3.1节）。
- catch块（9.3.2节）。
- finally块（9.3.3节）。
- 结合使用（9.3.4节）。

608

项目 5

“识别代码段中指定位置出现的异常的影响。注意，异常可能是运行时异常、可控异常或者错误。”

- 捕获或者指定需求（9.2节）。
- 指定方法抛出的异常（9.4节）。

项目 6

“识别造成抛出任何下列异常的情况：ArrayIndexOutOfBoundsException、AssertionError、ClassCastException、IllegalArgumentException、IllegalStateException、NullPointerException、NumberFormatException、ExceptionInInitializerError、StackOverflowError或者NoClassDefFoundError。了解虚拟机抛出哪种异常，并且认识

从程序中会抛出其他异常的情况。”

- ArrayIndexOutOfBoundsException：结合使用（9.3.4节）。
- NullPointerException：如何抛出异常（9.5节）。
- NumberFormatException：命令行参数（14.1.2节）。
- ClassCastException、IllegalStateException：对象排序（11.2.6节）。
- IllegalStateException：Queue接口（11.2.4节）。
- IllegalStateException：List接口（11.2.3节）。

第 3 节：API 内容

项目 1

“开发代码来使用原始包装器类（比如Boolean、Character、Double、Integer等）和/或自动装箱和拆箱。讨论String、StringBuilder和StringBuffer之间的区别。”

- 数字类（8.1.1节）。
- 字符（8.2节）。
- 字符串（8.3节）。
- StringBuilder类（8.3.8节）。

609

项目 2

“给定一个涉及文件系统定位、读取文件或者写入文件的场景，使用java.io中的下列类(有时需要结合使用）开发正确的解决方案：BufferedReader、BufferedWriter、File、FileReader、FileWriter和PrintWriter。”

- FileReader、FileWriter：字符流（10.1.2节）。
- BufferedReader、BufferedWriter：缓冲流（10.1.3节）。
- PrintWriter：格式化（10.1.4节）。
- File：File对象（10.2.1节）。

项目 3

“开发代码，使用java.io中的下列API对对象进行串行化和 / 或反串行化：DataInputStream、DataOutputStream、FileInputStream、FileOutputStream、ObjectInputStream、ObjectOutputStream和Serializable。”

- 数据流（10.1.6节）。
- 对象流（10.1.7节）。

项目 4

“使用java.text包中的标准J2SE API，为特定地区正确地格式化或者解析日期、数字和货币值；并且给定一个场景，如果使用默认地区或者特定地区，判断要使用的正确方法。描述java.util.Locale类的用途和使用方式。”

项目 5

“使用java.util和java.util.regex包中的标准J2SE API，编写代码来格式化或者解析字符串或者流。对于字符串，编写代码来使用Pattern和Matcher类以及String.split方法。

认识和使用用于匹配的正则表达式模式（限于：.（点号）、*（星号）、+（加号）、?、\d、\s、\w、[]和()）。*、+和? 的使用只限于greedy量词，括号操作符只用于分组机制，不用于在匹配过程中捕获内容。对于流，编写代码来使用Formatter和Scanner类以及PrintWriter.format/printf方法。认识和使用格式字符串中的格式化参数（限于：%b、%c、%d、%f和%s）。”

- String split方法：处理字符串中的字符（8.3.6节）。

610

- Pattern类的方法（13.9节）。
- Matcher类的方法（13.10节）。
- 扫描（10.1.4节）。
- 格式化（10.1.4节）。
- PrintWriter，格式化参数：格式化（10.1.4节）。
- 格式化参数：格式化数字输出（8.1.2节）。

第 4 节：并发

项目 1

“编写代码，使用java.lang.Thread和java.lang.Runnable定义、实例化和启动新的线程。”

- 定义和启动线程（12.2.1节）。
- SimpleThreads示例（12.2.5节）。

项目 2

“识别线程可以处于的状态，并且指出线程可以从一种状态转换为另一种状态的途径。”

项目 3

“给定一个场景，编写代码来正确利用对象锁定，以便保护静态或者实例变量不受并发访问问题的影响。”

- 线程干扰（12.3.1节）。
- 内存一致性错误（12.3.2节）。
- 同步方法（12.3.3节）。
- 内在锁和同步（12.3.4节）。
- 原子访问（12.3.5节）。

项目 4

611

“给定一个场景，编写代码来正确使用wait、notify或者notifyAll。”

- 保护块（12.5节）。

第 5 节：面向对象（OO）概念

项目 1

“开发代码在类中实现紧密封装、松耦合和高内聚性，并且解释其优势。”

项目 2

“给定一个场景，开发代码来演示多态的使用。此外，判断何时必须进行类型转换，并且识别与对象引用类型转换相关的编译时和运行时错误。”

- 继承（5.2节）。

项目 3

“对于构造器、实例或者静态变量以及实例或者静态方法，解释修饰符对继承的影响。”

- 继承（5.2节）。
- 覆盖和隐藏方法（5.2.6节）。
- 使用关键字super（5.2.8节）。

项目 4

“给定一个场景，开发代码来声明和/或调用覆盖的或者重载的方法，开发代码来声明和/或调用超类的、覆盖的或重载的构造器。”

- 覆盖和隐藏方法（5.2.6节）。
- 定义方法（4.1.3节）。
- 使用关键字super（5.2.8节）。

项目 5

“开发代码来实现‘is-a’和/或‘has-a’关系。”

612

第 6 节：集合/泛型

项目 1

“给定一个设计场景，判断应该使用哪个集合类和/或接口正确地实现设计方案，包括使用Comparable接口。”

- 对象排序（11.2.6节）。

项目 2

“区分相应的hashCode和equal方法的正确和错误覆盖，并且解释==和equal方法之间的区别。”

- Object作为超类（5.2.9节）。

项目 3

“编写代码来使用Collections API的泛型版本，特别是Set、List和Map接口和实现类。认识非泛型Collections API的局限，以及如何重构代码以便使用泛型版本。”

- 泛型类型（6.2节）。

项目 4

“开发代码，在类/接口中正确地使用类型参数、实例变量、方法实参和返回类型；编写泛

型方法或者利用通配类型的方法，并且理解这两种方式之间的相似性和区别。”

- 声明类（4.1.1节）。
- 定义接口（5.1.4节）。
- 把信息传递给方法或构造器（4.1.5节）。
- 从方法返回值（4.3.1节）。
- 泛型类型（6.2节）。
- 泛型方法和构造器（6.3节）。
- 通配符（6.6节）。

613

项目5

“使用java.util中的功能编写代码，对列表进行排序、执行折半搜索或者把列表转换为数组。使用java.util包中的功能编写代码，对数组进行排序、执行折半搜索或者把数组转换为列表。使用java.util.Comparator和java.lang.Comparable接口影响列表和数组的排序。此外，了解原始包装器类和java.lang.String的‘自然顺序’对排序的影响。”

- 算法（11.4节）。

第7节：基础

项目1

“给定一个代码示例和场景，编写代码使用正确的访问修饰符、包声明和导入语句（通过访问或者继承）与示例中的代码进行交互。”

- 控制对类成员的访问（4.3.3节）。
- 使用包成员（7.1.3节）。

项目2

“给定一个类和命令行示例，判断预期的运行时行为。”

- 命令行参数（14.1.2节）。
- 命令行I/O对象（14.2.1节）。

项目3

“把对象引用和原始值传递给方法，判断在对参数执行赋值或者其他修改操作时，对对象引用或者原始值的作用。”

614

- 把信息传递给方法或构造器（4.1.5节）。

项目4

“给定一个代码示例，识别对象在哪个位置变为可被垃圾收集，并且判断垃圾收集系统保证什么和不保证什么。了解System.gc和finalization。”

- 使用对象（4.2.2节）。

项目5

“给定JAR文件之内和/或之外部署的类的完全限定名称，为这个类构造正确的目录结构。给

定代码示例和类路径，判断类路径是否允许代码成功编译。”

- 管理源文件和类文件（7.1.4节）。

项目 6

“编写代码，正确地应用适当操作符产生期望的结果，操作符包括赋值操作符（限于：=、+=和-=）、算术操作符（限于：+、-、*、/、%、++和--）、关系操作符（限于：<、<=、>、>=、==和!=）、instanceof操作符、逻辑操作符（限于：&、|、^、!、&&和||）和条件操作符（?:）。编写代码判断两个对象或者两个原始值是否相等。”

- 赋值、算术和一元操作符（3.2.1节）。
- 相等、关系和条件操作符（3.2.2节）。
- Object作为超类（5.2.9节）。
- 比较字符串和部分字符串（8.3.7节）。

索　引

索引中的页码为英文原书的页码，与书中边栏的页码一致。

符号

- （minus sign）（减号）
  in regular expressions（正则表达式中），409，411
  operator（操作符），55～58，203
-- operator（--操作符），55，58，67
_ （underscore）（下划线）
  in constant names（常量名称中），45，115
  in package names（包名称中），187
  in variable names（变量名称中），44
, （comma）（逗号）
  in numbers（数字中），200，203，271～272
  in regular expressions（正则表达式中），419
; （semicolon）（分号）
  in class paths（类路径中），193，458
  in statements（语句中），28，67
: （colon）（冒号），in class paths（类路径中），193
! operator（!操作符），58
!/ separator（!/分隔符），516
!= operator（!=操作符），55，59～60
? （question mark）（问号）
  in regular expressions（正则表达式中），409，416～419，423，427～428
  in wildcards（通配符中），177
?: operator（?:操作符），55，60
/ （forward slash）（正斜线）
  file name separator（文件名分隔符），191，494
  operator（操作符），55～57，203
// in comments（注释中），25
/* in comments（注释中），24
/** in comments（注释中），25
/= operator（/=操作符），55
. （dot）（点号）
  in class paths（类路径中），460，503
  in method invocations（方法调用中），105
  in numbers（数字中），200，203，271～272
  in regular expressions（正则表达式中），408，414，416，426
  in variable names（变量名称中），104
... （ellipsis）（省略号），95
^ （caret）（脱字符）
  in regular expressions（正则表达式中），409～410，424，426
  operator（操作符），55，62
^= operator（^=操作符），55
~ operator（~操作符），62
' （single quote）（单引号），escape sequence for（用于转义序列），49，212
" （double quote）（双引号）
  escape sequence for（用于转义序列），49，212
  in literals（字面量中），213
() （parentheses）（括号）
  in annotations（注解中），133
  in declarations（声明中）90，105
  in expressions（表达式中），66～67
  in regular expressions（正则表达式中），409，420，422
[] （square brackets）（方括号）
  in arrays（数组中），51～52
  in regular expressions（正则表达式中）409～410
{} （braces）（花括号）
  in arrays（数组中），52
  in blocks（块中），68，70，117
  in declarations（声明中），87～88，90
  in regular expressions（正则表达式中），409，419
@ （at）
  in annotations（注解中），133
  in Javadoc（Javadoc中），135
$ （dollar sign）（美元符号）
  in DecimalFormat patterns（DecimalFormat模式中），203
  in regular expressions（正则表达式中），409，424，426
  in variable names（变量名称中），44
* （asterisk）（星号）
  in import statements（import语句中），188～189
  in command line（命令行中），492
  in regular expressions（正则表达式中），409，416～419
  operator（操作符），55～57，203
/ in comments（注释中的/），24
= operator（=操作符），55
\ （backslash）（反斜线）
  file name separator（文件名分隔符），191
  in eseape sequences（转义序列中），49，212，409，414
  in regular expressions（正则表达式中），409，414，423
& （ampersand）（与）
  in generic types（泛型类型中），174

operator（操作符），55，62
&&
in regular expressions（正则表达式中），409，412～413
operator（操作符），55，60～61
&= operator（&=操作符），55
（pound sign）（磅符）
in DecimalFormat patterns（DecimalFormat模式中），203
in regular expressions（正则表达式中），426
% （percent sign）（百分号）
format specifier（格式说明符），199，274
operator（操作符），55～57，203
%= operator（%=操作符），55
+ （plus sign）（加号）
in regular expressions（正则表达式中），409，416～419
operator（操作符），55～58，203，215
++ operator（++操作符），55，58，67
+= operator（+=操作符），55，57
< operator（<操作符），55，59～60
<< operator（<<操作符），55，62
<<= operator（<<=操作符），55
<= operator（<=操作符），55，59～60
<> （angle brackets）（尖括号），169，296
= operator（=操作符），48，51，56
return values of（返回值），66
-= operator（-=操作符），55
== operator（==操作符），55，59～60，158
> operator（>操作符），55，59～60
>= operator（>=操作符），55，59～60
>> operator（>>操作符），55，62
>>= operator（>>=操作符），55
>>> operator（>>>操作符），55，62
>>>= operator（>>>=操作符），55
| （vertical bar）（竖线）
in regular expressions（正则表达式中），409
operator（操作符），55，62，427
|= operator（|=操作符），55
|| operator（||操作符），55，60～61

数字

0（zero），in octal numbers（八进制数字中的0），48
0x in hexadecimal numbers（十六进制数字中的0x），48

A

abs method（abs方法），204
abstract keyword（abstract关键字），161～163，603
Abstract Window Toolkit（AWT）（抽象窗口工具包），189，547，552，581
AbstractXXX implementations（AbstractXXX实现），363
AccessControlException，457
access modifiers（访问修饰符）
classes and（类），87～89，110～112，127，151
constants and（常量），143
fields and（字段），151
interfaces and（接口），142～143
methods and（方法），90，151，154
acos method（acos方法），207
add method（add方法）
for applets（用于applet），555
for collections（用于集合），298～299，303，307～309，317，345，353，363
addAll method（addAll方法），300，304，307～309，353，359
addFirst method（addFirst方法），346
addLast method（addLast方法），346
algorithms（collections）（算法（集合）），300，309，314，316，355～360
composition（组合），360
finding extreme values（查找极值），360
polymorphic, defined（定义多态），294
routine data manipulation（常规数据操作），358～359
searching（搜索），359
shuffling（混序），358
sorting（排序），356～358
Alloy look and feel（Alloy界面外观），477，479
alt attribute（alt属性），575
ampersand（与）（见&）
Anagrams example（Anagrams示例），326～328
Anagrams2 example（Anagrams2示例），357
angle brackets（尖括号）（见<>）
AnimatorApplet applet，587
annotations（注解），132～137
comments and（注释），133
naming（命名），135
types in（类型中），133
Apache，525
APIs （Application Programming Interfaces）（应用程序编程接口），141
array-based vs. collection-based（基于数组和基于集合），353
compatibility of（兼容性），364～366
design of（设计），366～367
interoperability of（互操作性），294～295，364～367
JAR-related（JAR相关），514～520
legacy（旧式），367
append method（append方法），227～228
appendReplacement method（appendReplacement方法），432，436
appendTail method（appendTail方法），432，436
applet tag（applet标记），498～499，544，564，572，574～578，595～596
alternate HTML code in（替代HTML代码），575～576

closing（结束），600
Applet class（Applet类），543，546
extending（扩展），546～547
AppletButton applet，575，582
AppletContext interface（AppletContext接口），559
applets，543～602
accessibility of（可访问性），594
API of，559～578
background color of（背景颜色），581，601
capabilities of（功能），559
common problems with（常见问题），600～602
debugging（调试），583，593
deploying（部署），594～600
directories of（目录），576～577
displaying documents in（显示文档），562～564
download time of（下载时间），488
event-handling methods in（事件处理方法），552～554
final cleanup（最后的清理工作），550
finding all on a page（查找页面中所有），567～568
finding by name（按名称查找），564～567
finishing（完成），593～594
flexibility of（灵活性），594
GUIs in，554，581～583
leaving and returning to Web pages（离开和返回Web页面），549
life cycle of（生存周期），543～544，547～550
loading（加载），544～545，549
loading data files（加载数据文件），560
loops in（循环），587
networking（网络），589～593
non-editable text fields in（不可编辑的文本文件），555～558
overriding methods in（覆盖方法），551
packed in JAR files（打包到JAR文件中），498～499，577～578
parameters in（参数），570～574
playing sounds（播放声音），568～570，588，593
quitting the browser（退出浏览器），550
reloading（重新加载），549，600
representation of（表示），552
security and（安全），444，508，558～559，564，578～581，585，591～593
sending messages to other applets（向其他applet发送消息），564～568
status line and（状态栏），561，581
threads in（线程），585～589
vs. applications（应用程序），544，546，580
appletviewer application（appletviewer应用程序），544
applet viewers（applet查看器）（见Web browsers）
reloading applets（重新加载applet），549
security and（安全），567，578～581，585
applications（应用程序）
accessibility of（可访问性），465，482
command-line arguments in（命令行参数），448～449
concurrent（并发），369
efficiency of（有效性），269
entry points of（入口点），502～503，518
exiting（退出），445～446
launching（启动），521
life cycle of（生存周期），444～445
liveness of（活性），384～386
localizing（本地化），481
look and feel of（界面外观），464，476～479
measuring time intervals in（测量时间间隔），457
packed in JAR files（打包到JAR文件中），499～500
performance of（性能），294，361，363
pluggability of（可插拔性），35
portability of（可移植性），5
reusing（重用），295
running（运行），7，12～13，15，18，20，23，445
security and（安全），455～457，535～536
starting up（启动），445～446
vs. applets，544，546，580
archive attribute（archive属性），574
args variable（args变量），44
arguments（参数，实参），93～97
command-line（见command-line arguments）
number of（数字），94～95
of reference data types（引用数据类型），96～97
primitive（原始），96
vs. parameters（参数），94
ArithmeticDemo example（ArithmeticDemo示例），56～57
arithmetic operators（算术操作符），55～57，203
ArrayBlockingQueue class（ArrayBlockingQueue类），349
arraycopy method（arraycopy方法），53，346，457
ArrayCopyDemo example（ArrayCopyDemo示例），53
ArrayDemo example（ArrayDemo示例），49，51
ArrayIndexOutOfBoundsException，237，240～246
ArrayList implementation（ArrayList实现），307，309，314，342～343，346
initial capacity of（初始容量），346
performance of（性能），346
synchronization and（同步），346
arrays（数组），49～53，293
assigning values to（赋值），52
collections and（集合），301
copying（复制），53
creating（创建），51～52
declaring（声明），50～51
elements of（元素），49，52

indexes in（索引），49，52
initializing（初始化），51～52
length of（长度），49，52～53
lists and（列表），352
looping through（循环），76
multidimensional（多维），52，78
naming（命名），51
of characters（字符），213
of strings（字符串），220
Arrays class（Arrays类），310
asin method（asin方法），207
asList method（asList方法），310，347，352～353，364
assert keyword（assert关键字），603
assignments（赋值），48，51，55～56，67
compound（复合），55，57
return values of（返回值），66
asterisk（星号）（见*）
at（见@）
atan method（atan方法），207
atan2 method（atan2方法），207
atomic actions（原子操作），383～384
AtomicCounter cxample（AtomicCounter示例），402
AtomicInteger class（AtomicInteger类），196
AtomicLong class（AtomicLong类），196
Attributes class（Attributes类），517
AudioClip interface（AudioClip接口），568～570
audiotool application（audiotool应用程序），568
autoflush（自动刷新），269

B

backslash（反斜线）（见\）
backspace（退格），49
BadThreads example（BadThreads示例），403
BasicMathDemo example（BasicMathDemo示例），204～205
BasicService interface（BasicService接口），536
Bicycle class（Bicycle类），35～116，148～151，155
BicycleDemo example（BicycleDemo示例），36～37
bidi texts（bidi文本），481
BigDecimal class（BigDecimal类），46，196，271，282，284
natural ordering（自然顺序），329
BigInteger class（BigInteger类），196，271
natural ordering（自然顺序），329
binarySearch method（binarySearch方法），316，359
BitDemo example（BitDemo示例），62～63
bit shift operators（移位操作符），55，62～63
bitwise operators（位操作符），55，62～63
BlockDemo example（BlockDemo示例），68
BlockingQueue implementation（BlockingQueue实现），318，343，349～350，400
blocks（块），68
boolean data type（布尔数据类型），46，603
default value of（默认值），47
Boolean class（Boolean类），329
BorderLayout class（BorderLayout类），582
boxing（装箱），196，211
braces（花括号）（见{}）
break statement（break语句），71～74，77～79，240，603
labeled（带标签），78
unlabeled（无标签），77～78
BreakDemo example（BreakDemo示例），77～78
BreakWithLabelDemo example（BreakWithLabelDemo示例），78
browsers（见Web browsers）
BufferedInputStream class（BufferedInputStream类），269
BufferedOutputStream class（BufferedOutputStream类），269
BufferedReader class（BufferedReader类），267，269
BufferedWriter class（BufferedWriter类），269
buffers（缓冲区），269
bugs（见errors）
byte data type（byte数据类型），45，603
default value of（默认值），47
expressing（表示），48
switch statement and（switch语句），71
Byte class（Byte类），216
natural ordering（自然顺序），329
switch statement and（switch语句），71
bytecodes（字节码），2，7，14，20
byteValue method（byteValue方法），197

C

C++ programming language（C++编程语言），5，26
Standard Template Library（STL）（标准模板库），294
cache implementation（缓存实现），347
Calendar class（Calendar类），284，457
Callable interface（Callable接口），398
call stacks（调用栈），233，243，255～257
Canvas Swing component（Canvas Swing组件），555
capacity method（capacity方法），226
capturing groups（捕获组），422～423
backreferences in（反向引用），423
quantifiers and（量词），420～421
carriage return（回车），49，212，267
case statement（case语句），71～74，603
catch block（catch块），239～245，250，257，374，603
Catch or Specify Requirement（捕获或者指定需求），235～236
bypassing（绕过），236
cd command（cd命令），17，21～22，29

CDE/Motif look and feel（CDE/Motif界面外观），477～478
ceil method（ceil方法），204
change method（change方法），279
char data type（char数据类型），46，210～212，603
  converting to strings（转换为字符串），211
  default value of（默认值），47
  escape sequences in（转义序列），49
  expressing（表示），48
  getting by index（按照索引获得），218～219
  switch statement and（switch语句），71
Character class（Character类），210～212
  natural ordering（自然顺序），329
  switch statement and（switch语句），71
character classes（字符类），409～413
  intersections of（交），412～413
  negation of（否），410
  predefined（预定义），414～416
  quantifiers and（量词），420～421
  ranges of（范围），411
  simple（简单），409～413
  subtractions of（减），413
  unions of（并），411～412
charAt method（charAt方法），213，218～219
CharSequence interface（CharSequence接口），220
checkXXX methods（checkXXX方法），456
class keyword（class关键字），25，603
  definitions of（定义），25
Class class（Class类），159
ClassCastException，169，329，333，352
classes（类），35～37，85～97（见inheritance；nested classes）
  abstract（抽象），161～163
  adapter（适配器），367
  base or parent（基类或者父类）（见superclasses）
  child, derived, or extended（子类、派生类或者扩展类）（见subclasses）
  constructors for（构造器），92～93，103
  controlling access to（控制访问），110～112
  declaring（声明），87～88
  deprecated（废弃的），134
  grouping（分组），122
  hierarchy of（层次结构），108，142，148
  instantiating（实例化），43，100～101
  interfaces implemented by（实现接口），143～144，163
  naming（命名），186
  static initialization blocks in（静态初始化块），117
  wrapper（包装器），196
class files（类文件），191～192
classid attribute（classid属性），596～597
class library（类库）（见Java API（Application Programming Interface））
ClassNotFoundException，284
CLASSPATH variable（CLASSPATH变量），30，192～193，457～460，601
class paths（类路径），192～193
class variables（类变量）（见fields；static）
clear method（clear方法），300，322
ClipboardService interface（ClipboardService接口），536
clone method（clone方法），156～158，343，346
Cloneable interface（Cloneable接口），157，161
CloneNotSupportedException，157
close method（close方法），271
cmd command（cmd命令），16
code（代码）
  case sensitivity in（区分大小写），11，15，21
  error-prone（容易出错），70，241，258
  platform-independent（平台无关），286
  readability of（可读性），45，95，122～123，191，241，252～255
codebase attribute（codebase属性），574，576～577，597
CollationKey class（CollationKey类）
  natural ordering（自然顺序），329
Collection interface（Collection接口），176，295～296，298～301
  array operations of（数组操作），301
  bulk operations of（批处理操作），300
  implementations of（实现），363
  standard conversion constructor of（标准转换构造器），298
  view methods of（视图方法），322～326
  wrappers for（包装器），350，352
collections（集合），293～367
  benefits of（优势），294～295
  concurrent（并发），400～401
  elements of（元素）（见elements）
  filtering（过滤），300
  frameworks of（框架），294
  hierarchy of（层次结构），176，366
  immutable（不可变），352
  older APIs and（旧式API），301
  ordered（有序），297
  read-only access to（只读访问），352
  synchronized（同步），343，350～351
  traversing（遍历），76，299～300
  with no duplicate elements（无重复元素），297，301～306
Collections class（Collections类）
  empty constants in（空常量），354
  methods in（方法），350～353
  polymorphic algorithms in（多态算法），355～306
colon（冒号）（见：）

comma（逗号）（见,）
command line（命令行），276
command-line arguments（命令行参数），26，448～449
  echoing（回显），448～449
  numeric（数字），449
comments（注释），24～25，168
  annotations and（注解），133
Comparable interface（Comparable接口），161，328～335
Comparator interface（Comparator接口），332～339，349，357，359～360
compare method（compare方法），332～335
compareAndSet method（compareAndSet方法），401
compareTo method（compareTo方法）
  for files（用于文件），287
  for objects（用于对象），329～332
  for primitive data types（用于基本数据类型），197
  for strings（用于字符串），225
compareToIgnoreCase method（compareToIgnoreCase方法），225
ComparisonDemo example（ComparisonDemo示例），59
comparisons（比较），55
  interclass（类间），329
  of numbers（数字），59～60，204
  of object（对象），61～62
compatibility（兼容性），364～366
  backward（向下），365～366
  binary（二进制），178
  upward（向上），364～365
compile method（compile方法），426～428
compilers（编译器）
  error handling（错误处理）（见errors; compile-time）
  information for（信息），132，152
  verifying types of objects（验证对象类型），296
ComputeResult class（ComputeResult类），231
concat method（concat方法），214～215
ConcatDemo example（ConcatDemo示例），57
concurrency（并发），369～404
ConcurrentHashMap implementation（ConcurrentHashMap实现），348～349，400
ConcurrentMap interface（ConcurrentMap接口），343，348～349，400
ConcurrentModificationException，346
ConcurrentNavigableMap interface（ConcurrentNavigableMap接口），400
ConcurrentSkipListMap interface（ConcurrentSkipListMap接口），400
ConditionalDemo1 example（ConditionalDemo1示例），60
ConditionalDemo2 example（ConditionalDemo2示例），60～61
conditionals（条件），55，60～61，69
Console（控制台），277～279
const keyword（const关键字），603
constants（常量），112，115，128
  compile-time（编译时），115
  for upper and lower bounds（上限和下限），197
  importing（导入），190，203
  in interfaces（接口中），140，143
  naming（命名），45，115
constructors（构造器），99～103
  calling methods from（调用方法），161
  chaining（链式），156
  conversion（转换），298
  declaring（声明），92～99
  default（默认），103
  for enum types（用于枚举类型），130
  generic（泛型），172～173
  inheritance and（继承），155～156
  invoking（调用），109～110，148
  no-argument（无实参），93，103，110，156
  synchronization and（同步），381
Consumer example（Consumer示例），389～390
Container class（Container类），552，555
containers（容器）（见collections）
contains method（contains方法），298，345，447
containsAll method（containsAll方法），300，304，323～326，347
containsKey method（containsKey方法），320～322，447
containsValue method（containsValue方法），320～322
continue statement（continue语句），79～81，240，603
  labeled（带标签），80
  unlabeled（无标签），79
ContinueDemo example（ContinueDemo示例），79～80
ContinueWithLabelDemo example（ContinueWithLabelDemo示例），80
control flow statements（控制流语句），68～82
  branching（分支），77～82
  decision-making（决策），69～71
controlling access（控制访问）（见access modifiers）
conversions（转换），199～201，274～275
copy method（copy方法），316，359
CopyBytes example（CopyBytes示例），263～266
CopyCharacters example（CopyCharacters示例），265～266
CopyLines example（CopyLines示例），267
CopyOnWriteArrayList implementation（CopyOnWriteArrayList实现），346
CopyOnWriteArraySet implementation（CopyOnWriteArraySet实现），345

core collection interfaces（核心集合接口），294～342
compatibility of（兼容性），364～366
hierarchy of（层次结构），295～297
implementations of（实现），366
modification operations in（修改操作），296
cos method（cos方法），207
Countdown example（Countdown示例），318
Counter example（Counter示例），377，401
CreateObjectDemo example（CreateObjectDemo示例），98～106
createTempFile method（createTempFile方法），289
currentTimeMillis method（currentTimeMillis方法），457

D

d in 64-bit double literals（在64位双精度字面量中的d），48
D in 64-bit double literals（在64位双精度字面量中的D），48
data encapsulation（数据封装），34
DataInput interface（DataInput接口），279，290
DataInputStream class（DataInputStream类），279～282
DataOutput interface（DataOutput接口），279，290
DataOutputStream class（DataOutputStream类），279～282
DataStreams example（DataStreams示例），279～282
data types（数据类型），89
reference（引用），96～97，145
returned by expressions（表达式返回），66
switch statement and（switch语句），71
Date class（Date类），328～329
dates（日期），200，284
Deadlock example（Deadlock示例），384～385
Deal example（Deal示例），314～315
debugging（调试），35，160，167，249
output（输出），593
DecimalFormat class（DecimalFormat类），202～203
declaration statements（声明语句），67
decode method（decode方法），198
default statement（default语句），71～74，603
DelayQueue class（DelayQueue类）350
delete method（delete方法）228，289
deleteOnExit method（deleteOnExit方法），289
@deprecated Javadoc tag（@deprecated Javadoc标记），134
@Deprecated annotation type（@Deprecated注解类型），134
Desktop API（桌面API），483～484
adding Java Web Start applications to（添加Java Web Start应用程序），524
destroy method（destroy方法），544，550，552，585
dir command（dir命令），17，29
directories（目录），289
changing（改变），17，21～22，29
creating（创建），21
current（当前），16，21～22，29
home（主），16，21
in JAR files（JAR文件中），492～493
disjoint method（disjoint方法），360
do keyword（do关键字），603
documentation（文档），133～134，168，296
dollar sign（美元符号）（见$）
dot（点号）（见.）
double data type（double数据类型），46，603
default value of（默认值），47
expressing（表示），48
Double class（Double类），216
natural ordering（自然顺序），329
double quote（双引号）（见"）
doubleValue method（doubleValue方法），197
do-while statement（do-while语句），74～75
skipping the current iteration（跳过当前迭代），79
terminating（终止），77
DoWhileDemo example（DoWhileDemo示例）75
DownloadService interface（DownloadService接口），536
Drop example（Drop示例）388，390

E

e in scientific notation（科学记数法中的e），48
E
as type parameter（作为类型参数），171
constant（常量），204
in scientific notation（科学记数法中），48
Echo example（Echo示例），448～449
element method（element方法），317
elementAt method（elementAt方法），307，309
elements（in collections）（元素）（集合中），296，447
adding（添加），298，303，323
checking（检查），298
counting（计数），298，303
manipulating（操作），358～359
minimum and maximum（最大和最小），360
not duplicated（无重复），301～306
null，301，308
ordering（顺序），328～335
randomization of（随机），358
removing（删除），297～298，303，323
searching（搜索），359

sorting（排序），318，338，356～358
ellipsis（省略号）（见...）
else statement（else语句），603
emacs text editor（emacs文本编辑器），19
embed tag（embed标记），595，598～600
emptyList method（emptyList方法），354
emptyMap method（emptyMap方法），354
emptySet method（emptySet方法），354，363
EmptyStackException，247
encapsulation（封装），122～123
end method（end方法），423，431～433
endsWith method（endsWith方法），225
EnhancedForDemo example（EnhancedForDemo示例），77
ensureCapacity method(ensureCapacity方法)，227，351
entrySet method（entrySet方法），322～323
enum keyword（enum关键字），128，603
Enum class（Enum类），130
Enumeration collection（Enumeration集合），567
  compatibility and（兼容性），364～366
EnumMap implementation（EnumMap实现），348
EnumSet implementation（EnumSet实现），345
EnumTest example（EnumTest示例），129
enum types（enum类型），128～132，345
  constructors for（常量），130
  naming（命名），128
Env example（Env示例），450
environment（环境），443～461
  properties of（属性），443～448，583～585
  restricted（限制），538，544
environment variables（环境变量），449～451
  passing to new processes（传递给新的进程），450
  platforms and（平台），451
  querying（查询），449～450
EnvMap example（EnvMap示例），449～450
EOFException，282
equality operators（等于操作符）（见comparisons）
equals method（equals方法），156，158～160，197，225，301，309，318，322，331，334
equalsIgnoreCase method(equalsIgnoreCase方法)，225
Error class（Error类），235～236，248，252
error messages（错误消息），27～30，51，167，239，277
errors（错误），233，235
  compile-time（编译时），27～29，104，107，135，152，154，156，167，176～178，188
  memory consistency（内存一致性），379～381，384，401
  runtime（运行时），29～30，167，296
  semantic（语义），28～29
  syntax（语法），12，27～28，437～439
escape sequences（转义序列），211～212
  in regular expressions（正则表达式中），414
  in Unicode（Unicode中），48
Exception class（Exception类），248，251，258
exception handlers(异常处理代码)，233，236～245，257～258
  associating with try blocks(和try块相关联)，238～240
  catching multiple exceptions（捕获多个异常），251
exception objects（异常对象），233
exceptions（异常），152，233～260
  advantages of（优势），253～258
  catching（捕获），234，236～245，257
  chained（链式），249～250
  checked（可控），235～236，253
  class hierarchy of（类层次结构），251
  external（外部）（见errors）
  grouping（分组），257～258
  runtime（运行时），236，248，252
  throwing（抛出），233，235，243，246～252
  unchecked（不可控），236，252～253
  user-defined（用户定义），240，246，250～252
exclamation sign（叹号）（见!）
execute method（execute方法），398
Executor interface（Executor接口），397～398
executors（执行器），371，397～400
Executors class（Executors类），399
ExecutorService interface（ExecutorService接口），397～398
exit method（exit方法），457
exp method（exp方法），205
ExponentialDemo example（ExponentialDemo示例），206
exponents（指数），205
expressions（表达式），66～67
  compound（复合），66
expression statements（表达式语句），67
ExtendedService interface（ExtendedService接口），537
extends keyword(extends关键字)，38，87，143，173～174，177，603
extensions（扩展），451，488，498

F

f in 32-bit float literals（32位浮点数字面量中的f），48
F in 32-bit float literals（32位浮点数字面量中的F），48
fields（字段），43，88（见variables）
  controlling access to（控制访问），110～112
  declaring（声明），47，150
  default values of（默认值），47～48
  deprecated（已废弃的），134
  final（最终），381，393

hiding（隐藏），150，154
inherited（继承），150
initializing（初始化），47，116～118
non-static（非静态），43，112，115，118
private（私有），393
referencing（引用），103～105
reusing（重用），148
shadowing（阴影化），95，109
static（静态），44，112～115，163
static final（静态最终）（见constants）
FIFO（first-in, first-out）（先进先出），297，317，343，349
File class（File类），39，286～289
natural ordering（自然顺序），329
FileContents interface（FileContents接口），537
file descriptors（文件描述符），159
FileInputStream class（FileInputStream类）263，266～267
Filename class（Filename类），221～224
FilenameDemo example（FilenameDemo示例），223
FileNotFoundException，235，240，257
FileOpenService interface（FileOpenService接口），537
FileOutputStream class（FileOutputStream类），263，266～267
FileReader class（FileReader类），265～267
files（文件）
creating（创建），289
I/O and（输入/输出），286～291
manipulating（操作），288～289
modification data/time of（修改日期/时间），289
names of（名称），286～288
random access to（随机访问），289～291
FileSaveService interface（FileSaveService接口），537
FileStuff example（FileStuff示例），288
FileWriter class（FileWriter类），265～267，286
fill method（fill方法），316，359
final modifier（final修饰符），603
constants and（常量），115，143
fields and（字段），44
methods and（方法），118，160～161
finalize method（finalize方法），156，159
finally block（finally块），240～245，264，603
find method（find方法），431
FindDups example（FindDups示例），303
FindDups2 example（FindDups2示例），305
first method（first方法），338
float data type（float数据类型），46，282，603
default value of（默认值），47
expressing（表示），48
Float class（Float类），216
natural ordering（自然顺序），329
floatValue method（floatValue方法），197
floor method（floor方法），204
flush method（flush方法），269
for statement（for语句），75～77，299～300，603
enhanced（增强），76
nested（嵌套），78
skipping the current iteration（跳过当前迭代），79
terminating（终止），77
ForDemo example（ForDemo示例），75～76
format method（format方法），198～200，215～216，269，273～276
Format example（Format示例），275
format specifiers（格式说明符），199～201，274～276
format strings（格式字符串），199，274
Formatter class（Formatter类），199
formatting（格式化），272～276
form feed（换页），49，212
forward slash（正斜线）（见/）
Frame class（Frame类），546
Freq example（Freq示例），320～321
frequency method（frequency方法），360
functions（函数）（见methods）
Future interface（Future接口），398

G

garbage collection（垃圾收集），5，106，159
weak references and（弱引用），348
generic types（泛型类型），169～172，296
declaring（声明），169
instantiating（实例化），178
invoking（调用），170
get method（get方法），307，309，320～322，363，401
getAbsolutePath method（getAbsolutePath方法），287
getApplet method（getApplet方法），564～567，579
getAppletInfo method（getAppletInfo方法），594
getApplets method（getApplets方法），567～568，579
getAudioClip method（getAudioClip方法），568
getCanonicalPath method（getCanonicalPath方法），287
getCause method（getCause方法），249
getChars method（getChars方法），214
getClass method（getClass方法），156，159
getCodeBase method（getCodeBase方法），560，582，589
getDescription method（getDescription方法），437
getDocumentBase method（getDocumentBase方法），560，582
getEnv method（getEnv方法），449～450
getFields method（getFields方法），159
getFirst method（getFirst方法），346

getHost method（getHost方法），589
getImage method（getImage方法），582，601
getIndex method（getIndex方法），437
getInterfaces method（getInterfaces方法），159
getLast method（getLast方法），346
getMainAttributes method（getMainAttributes方法），517
getMainClassName method（getMainClassName方法），515～517，519
getMessage method（getMessage方法），437
getMethods method（getMethods方法），159
getName method（getName方法），287
GetOpenProperties applet，584
getParameter method（getParameter 方法），573
getParameterInfo method（getParameterInfo方法），573～574，594
getParent method（getParent方法），287
getPattern method（getPattern方法），437
getProperty method（getProperty方法），447，452～454
getResource method（getResource方法），535
getSecurityManager method（getSecurityManager方法），456
getSimpleName method（getSimpleName方法），159
getStackTrace method（getStackTrace方法），250
getSuperclass method（getSuperclass方法），159
getValue method（getValue方法），517
goto method（goto方法），603
graphical user interfaces（GUI）（图形用户界面），4，40，463（见Swing）
  creating by applets（applet创建），581～583
GridBagLayout class（GridBagLayout类），582
group method（group方法），423
groupCount method（groupCount方法），423
GTK+, look and feel（GTK+界面外观），464
GTK+ look and feel（GTK+界面外观），477
guarded blocks（保护块），386～390

H

hashCode method（hashCode方法），156，160，301，309，318，322，331
HashMap implementation（HashMap实现），320，322，342～343，347，361
HashSet implementation（HashSet实现），302，304～305，342～345
  initial capacity of（初始容量），344
  load factor of（负载因子），345
Hashtable collection（Hashtable集合），293，297，320，343，348～349，444，448，570
  compatibility and（兼容性），364～366
hasNext method（hasNext方法），299，311
headSet method（headSet方法），377
HelloRunnable example（HelloRunnable示例），371
HelloThread example（HelloThread示例），371～372
HelloWorld，6～26
  applet，545，551
  for Microsoft Windows（用于Microsoft Windows），14～19
  for Solaris OS（用于Solaris OS），19～23
  for UNIX/Linux（用于UNIX/Linux），19～23
HelloWorldApp example（HelloWorldApp示例），25
HostnameVerifier class（HostnameVerifier类），539
HTML Converter tool（HTML转换工具），600
HTML specification（HTML规范），595
HTTP requests（HTML请求），399
HTTPS Certificates（HTTPS认证），539

I

I/O（输入/输出），261～293
  atomic（原子），383～384
  from the command line（从命令行），276～279，452
  line-oriented（面向行），267～268
  non-stream file（非流文件），286～291
  of complex objects（复杂对象），284～285
  of objects（对象），282～285
  of primitive data type values（基本数据类型值），279～282
  scalable（可伸缩），291
I/O streams（I/O流）（见streams; I/O）
IdentityHashMap implementation（IdentityHashMap实现），348
IDE Project（IDE工程），7～9
if statement（if语句），603
IfElseDemo example（IfElseDemo示例），70
if-then statement（if-then语句），69～70
if-then-else statement（if-then-else语句），60，70～71
  comparing to switch（和switch比较），72
IllegalAccessException，248
IllegalStateException，317
images, in Swing applets（图像，在Swing applet中）（见icons）
ImmutableRGB example（ImmutableRGB示例），394
implementations（实现），294，342～355
  abstract（抽象），342，361～364
  adapter（适配器），361
  anonymous（匿名），350
  concurrent（并发），342，348～350
  convenience（简便），342，352～354
  custom（定制），360～364，367
  documenting（文档记录），296
  general-purpose（通用），342～347，349，352

performance of（性能），344
special-purpose（专用），342，345～346，348
wrapper（包装器），342～343，350～352
implements keyword（implements关键字），39，87～88，143，603
import statement（import语句），188～190，203，603
indexOf method（indexOf方法），307～309
indexOfSubList method（indexOfSubList方法），316
information-hiding（隐藏信息），34
inheritance（继承），37～38，147～165
multiple（多），141～142，145
init method（init方法），543，546，550～551，570，585，601
initCause method（initCause方法），249
initializer blocks（初始化块），118
inner classes（内部类），122～127
anonymous（匿名），127
controlling access to（控制访问），127
instantiating（实例化），124
local（局部），127
InputStream class（InputStream类），263
InputStreamReader class（InputStreamReader类），267，277
insert method（insert方法），227～228
insertElementAt method（insertElementAt方法），308～309
instanceof operator（instanceof操作符），55，61～62，152，603
InstanceofDemo example（InstanceofDemo示例），61
instances（实例），35，43，100～101，113
of inner classes（内部类），123
testing（测试），61～62
instance variables（实例变量）（见fields; non-static）
int data type（int数据类型），45～46，603
default value of（默认值），47
expressing（表示），48
switch statement and（switch语句），71
Integer class（Integer类），198，216
natural ordering（自然顺序），329
switch statement and（switch语句），71
interface keyword（interface关键字），142，603
interfaces（接口），38～39，139～147
abstract classes and（抽象类），161～162
acccss control of（访问控制），142
body of（体），142
collection（集合）（见core collection interfaces）
defining（定义），140，142～143，161
extending other interfaces（扩展其他接口），142
implementing（实现），88，140～144，163
rewriting（重写），145～146
using as types（用作类型使用），144～145
internationalization（国际化），265
in Swing（Swing中），465，480～482
Internet domain names（因特网域名），186，191
Internet Explorer，540，545，595～599
Inter Process Communication（IPC）（进程间通信），370
interrupt method（interrupt方法），373
interrupted method（interrupted方法），374～375
InterruptedException，373～375，387
intValue method（intValue方法），197
invokeClass method（invokeClass方法），517～518
IOError，236
IOException，237，239～240，243～246，257
isAnnotation method（isAnnotation方法），159
isEmpty method（isEmpty方法），298，303，320～322
isEnum method（isEnum方法），159
isInterface method（isInterface方法），159
isInterrupted method（isInterrupted方法），375
isLetter method（isLetter方法），211
isLowerCase method（isLowerCase方法），211
isUpperCase method（isUpperCase方法），211
isWhitespace method（isWhitespace方法），211，270
iterator method（iterator方法），298～300，303，323，335，339，351，363
Iterator class（Iterator类），303，310～313，335，339
iterators（迭代器），298～300
fail-fast（快速失败），343

J

JApplet Swing component（JApplet Swing组件），474，543，546，559
extending（扩展），546～547
Jar tool（Jar工具），489
setting entry points with（设置入口点），503
JAR files（JAR文件夹），451，487～520
benefits of（好处），488
creating（创建），489～493，534
extracting（提取），495～496
paths in（路径），494
reading（读取），535
referencing classes in other JAR files from（引用其他JAR文件中的类），503～504
running（运行），498～500
sealing（密封），507
signing（签名），507～513，538
uncompressed（未压缩），492
updating（更新），496～498
verifying（验证），508，513
viewing（查看），493～495
JarClassLoader example（JarClassLoader示例），514～519

JarRunner example（JarRunner示例），
514～515，518～520
Jarsigner tool（Jarsigner工具），511～513
JarURLConnection class（JarURLConnection类），
514～517
java launcher（java启动器），2，4，7，15，20，23，
29～30，499
arguments（实参），30
case sensitivity in（区分大小写），11，15，21
Java programming language（Java编程语言），1～5
java.awt package（java.awt包），189
java.io package（java.io包），263～291
java.nio.* packages（java.nio.*包），291
java.util.concurrent package
（java.util.concurrent包），
342，348～350，369～404
java.util.concurrent.atomic package
（java.util.concurrent.atomic包），401～402
java.util.concurrent.locks package
（java.util.concurrent.locks包），395～397
java.util.jar package（java.util.jar包），514～
515
java.util.regex package（java.util.regex包），
405～441
Java 2D，4，464
Java 3D，488
Java API（Java Application Programming Interface）（Java
应用程序编程接口），3～4，26，488
Java Archive Tool（Java档案工具）（见Jar tool）
JavaBeans，5
javac compiler（javac编译器），2，4，7，14，20，27
case sensitivity in（区分大小写），11，15，21
Java Cache Viewer，523，540
Java Collections Framework(Java集合框架)(见collections)
javadoc tool（javadoc工具），4
doc comments and（文档注释），25
tags in（标记），135
Java Foundation Classes（JFC）（Java基础类），464
Java HotSpot virtual machine（Java HotSpot虚拟机），2
Java IDL API，5
Java look and feel（Java界面外观），470～477
JavaMail，488
Java Naming and Directory Interface API（Java命名和目录
接口API），5
Java Network Launching Protocol（Java网络启动协议）(见
JNLP files)
Java platform（Java平台），3～6
API Specification（API规范），40
earlier versions of（早期版本），293，522
multithreaded execution and（多线程执行），370
supported encodings on（支持的编码），533
Java Plug-in，451，465，487，545
versions of（版本），596
Java Remote Method Invocation over Internet Inter-ORB
Protocol（Java RMI-IIOP）Technology（通过因特网ORB
间协议的Java远程方法调用技术），5
Java RMI，5
JavaScript，599～600
Java SE Development Kit 6（见JDK 6）
Java SE Runtime Environment（Java SE运行时环境）（见
JRE（Java SE Runtime Environment））
Java Virtual Machine（Java VM）（Java虚拟机），2～4，7
failures in（特性），248，252
terminating（终止），289
Java Web Start，5，451，466，487，521～541
benefits of（好处），521
common problems with（常见问题），539～540
deploying applications（部署应用程序），524～534
importing certificates（导入证书），539
running applications（运行应用程序），522～524
security and（安全），538～539，544
signing applications（签名应用程序），538
javax.jnlp package（javax.jnlp包），536～537
JButton Swing component（JButton Swing组件），471，
555
JCheckBox Swing component（JCheckBox Swing组件），
471，555
JCheckBoxMenuItem Swing component
（JCheckBoxMenuItem Swing组件），555
JColorChooser Swing component（JColorChooser
Swing组件），472
JComboBox Swing component（JComboBox Swing组件），
471
JDBC API，5
JDialog Swing component（JDialog Swing组件），474，
547
JDK 6（Java SE开发工具包6），4，6，14，19，27，136，
463，483，485，601
JDK Applet Viewer，579，600
JEditorPane Swing component（JEditorPane Swing组
件），472
JFileChooser Swing component（JFileChooser Swing
组件），473
JFrame Swing component（JFrame Swing组件），474，
546～547
JInternalFrame Swing component（JInternalFrame
Swing组件），476
JLabel Swing component（JLabel Swing组件），474，555
JLayeredPane Swing component（JLayeredPane Swing
组件），476
JList Swing component（JList Swing组件），471，555
JMenu Swing component（JMenu Swing组件），471，555

JMenuBar Swing component（JMenuBar Swing组件），555
JMenuItem Swing component（JMenuItem Swing组件），555
JNLP files（JNLP文件），451，523，525
  contents of（内容），526～533
  creating（创建），525～533
  encoding（编码），533
  links to（链接），534
  security and（安全），538
JNLP API，536～537
join method（join方法），375
JPanel Swing component（JPanel Swing组件），475，555
JPasswordField Swing component（JPasswordField Swing组件），471
JProgressBar Swing component（JProgressBar Swing组件），474
JRadioButton Swing component（JRadioButton Swing组件），471
JRE（Java SE运行时环境），523，545，596～597，601
JRootPane Swing component（JRootPane Swing组件），476
JScrollBar Swing component（JScrollBar Swing组件），555
JScrollPane Swing component（JScrollPane Swing组件），475
JSeparator Swing component（JSeparator Swing组件），474
JSlider Swing component（JSlider Swing组件），471，555
JSpinner Swing component（JSpinner Swing组件），471
JSplitPane Swing component（JSplitPane Swing组件），475
JTabbedPane Swing component（JTabbedPane Swing组件），475
JTable Swing component（JTable Swing组件），473
JTextArea Swing component（JTextArea Swing组件），473，555
JTextField Swing component（JTextField Swing组件），555～556
JTextPane Swing component（JTextPane Swing组件），472
JToolBar Swing component（JToolBar Swing组件），475
JToolTip Swing component（JToolTip Swing组件），474
JTree Swing component（JTree Swing组件），473
JWindow Swing component（JWindow Swing组件），555

K

K type parameter（K类型参数），172
keys method（keys方法），447
keySet method（keySet方法），322～323
keywords（关键字），603
  variable names and（变量名称），45

L

last method（last方法），338
lastIndexOf method（lastIndexOf方法），221～224，307～309
lastIndexOfSubList method（lastIndexOfSubList方法），316
length method（length方法），53，213～214，219，226
line feed（换行），49，212，215，267
line terminators（行终止符），199～200，267～268，274
  in regular expressions（正则表达式中），427
LinkedBlockingQueue class（LinkedBlockingQueue类），349
LinkedHashMap implementation（LinkedHashMap实现），320～321，342，347
LinkedHashSet implementation（LinkedHashSet实现），302，342，344～345
LinkedList implementation（LinkedList实现），307，318，342～343，346，349
  performance of（性能），346
Linux（见UNIX/Linux）
list method（list方法），289，447
List interface（List接口），297，306～316
  algorithms of（算法），316
  arrays and（数组），352
  basic operations of（基本操作），308～309
  comparing to Vector（与Vector相比），307～309
  empty constants of（空常量），354
  implementations of（实现），307，342～343，345～347，353，363
  order of elements in（元素顺序），356～359
  positional access and search operations of（位置访问和搜索操作），309～310
  range-view operation in（范围视图操作），313～315
  wrappers for（包装器），350，352
listeners（监听器）（见event listeners）
listFiles method（listFiles方法），289
listIterator method（listIterator方法），307，311，363
ListOfNumbers example（ListOfNumbers示例），236～246
listRoots method（listRoots方法），289
lists（列表）
  cursor positions in（指针位置），312
  iterating backward（向后迭代），311
literals（字面量），48～409
  class（类），49
  concatenating with strings（和字符串连接），215
locales（地区），200，271～272，274，480
  character sets in（字符集），265

overriding（覆盖），272
Lock interface（Lock接口），395～397
lockInterruptibly method（lockInterruptibly方法），395
locks（锁）
deadlocks（死锁），384～385，395
intrinsic（内在），381～383，387
liveloeks（活锁），386
starvation（饿死），386
log method（log方法），205
logarithms（对数），205
logging（记录日志），250
logical operators（逻辑操作符），55，60～61
bitwise（位），62
long data type（long数据类型），46，603
default value of（默认值），47
expressing（表示），48
Long class（Long类），216
natural ordering（自然顺序），329
longValue method（longValue方法），197
lookingAt method（lookingAt方法），431，434
loop method（loop方法），568
loops（循环），50，74～77
in applets（applet中），587
infinite（无限），74，76
nested（嵌套），78，80
skipping the current iteration（跳过当前迭代），79
terminating（终止），77
ls command（ls命令），22，30

M

Mac OS（Mac操作系统），2～3，6～7
main method（main方法），25～26，30，36，44，546
manifest files（清单文件），492，494，500～507
default（默认），500
modifying（修改），500～502
package sealing in（包密封），506～507
package versions in（包版本），504～506
referencing classes in（引用类），503～504
Map interface（Map接口），295～297，319～328，449
basic operations of（基本操作），320～322
bulk operations in（批处理操作），322
comparing to Hashtable（与Hashtable相比），320
empty constants of（空常量），354
implementations of（实现），320，342～343，347～349，363
standard conversion constructor of（标准转换构造器），322
viewing as a Collection（作为Collection查看），322～326
wrappers for（包装器），350，352
Matcher class（Matcher类），406
methods of（方法），431～437
MatcherDemo example（MatcherDemo示例），432～434
matches method（matches方法），225，428，430，432，434
MatchesLooking example（MatchesLooking示例），434
Math class（Math类），190，203～208
max method（max方法），204，360
MAX_VALUE constant（MAX_VALUE常量），197
members（成员），44
controlling access to（访问控制），151
memory（内存）
freeing（释放），106，159，240～241
leaks（泄漏），5，241，264
reserving for a variable（为变量保留），99
saving in large arrays（在大型数组中节省），45
methods（方法），34
abstract（抽象），161～163
accessor（访问器），213
as members（作为成员），44
atomic（原子），384
body of（体），90，94，140
called from constructors（从构造器调用），161
class（类），115，152～153
controlling access to（控制访问），110～112
declarations of（声明），90～97，246
deprecated（废弃的），134～135
exiting from（退出），81
final（最终），118，160～161，393
generic（泛型），172～173
hiding（隐藏），152
importing（导入），190，203
inherited（继承），150
instance（实例），115，150，152
invoking（调用），67，105～106
naming（命名），90～91
overloading（重载），154
overriding（覆盖），108，135，152～154
package-private（包私有），111
passing to methods（传递给方法），94
returning values from（返回值），81，90，105～109
reusing（重用），117，148
specifying exceptions in（指定异常），245～246
static（静态），114～115，150，163
static private（静态私有），117
synchronized（同步），380～382，387
method signatures（方法签名），91
in interface declarations（接口声明中），140，142
in method declarations（方法声明中），91
inheritance and（继承），150，152，154
Microsoft Windows，2～3，6～7，14～19

common error messages on（常见错误消息），27，29
environment variables on（环境变量），193，451，458，460
file name separators on（文件名分隔符），191
file names on（文件名），286～287
path separators on（路径分隔符），193
root directories on（根目录），289
system tray on（系统托盘），484
MIME types（MIME类型），525，539
min method（min方法），204，360
MIN_VALUE constant（MIN_VALUE常量），197
mkdir method（mkdir方法），21，289
modifiers（修饰符）（见access modifiers）
modularity（模块化），34
monitor locks（监视器锁定）（见locks; intrinsic）
mouseDown method（mouseDown方法），593
Mozilla，545，595～596，598～599
MultiDimArrayDemo example（MultiDimArrayDemo示例），52
multimaps（多映射），326～328
multisets（多集），361

N

N type parameter（N类型参数），172
name attribute（name属性），564
NameSort example（NameSort示例），331～332
nanoTime method（nanoTime方法），457
native keyword（native关键字），603
nCopies method（nCopies方法），353
NegativeArraySizeException，248
nested classes（嵌套类），122～128
as members（作为成员），44
controlling access and（控制访问），151
importing（导入），189
non-static（非静态）（见inner classes）
static（静态），122～123，127
NetBeans IDE，6～14
new keyword（new关键字），48，51，92，99～101，104～105，170，213，603
newCachedThreadPool method（newCachedThreadPool方法），399
newFixedThreadPool method（newFixedThreadPool方法），399
newline（换行）（见line terminators）
newSingleThreadExecutor method（newSingleThreadExecutor方法），400
next method（next方法），299
nextIndex method（nextIndex方法），312
NoSuchElementException，317
NoSuchMethodError，30
Notepad text editor（记事本文本编辑器），14～16
Notepad demo（Notepad演示程序），525～526
notify method（notify方法），157
notifyAll method（notifyAll方法），157
null value（null值），47，49，62，100，106，197，249，252
NullPointerException，236，248，253，331
Number class（Number类），195～198，216～218
NumberFormatException，449
numbers（数字），195～210
comparing（比较），204
converting to strings（转换为字符串），197～198，217～218
currency values and（货币值），282
formatting（格式化），198～203，274
random（随机），208
separators for（分隔符），200～203，271～272
number systems（数字系统），48，197
converting between（相互转换），197～198

O

object tag（object标记），595～600
Object class（Object类），93，103，148，156～160
ObjectInput interface（ObjectInput接口），282
ObjectInputStream class（ObjectInputStream类），282
object-oriented programming（面向对象编程），33～41
ObjectOutput interface（ObjectOutput接口），282
ObjectOutputStream class（ObjectOutputStream类），282
object references（对象引用），56，158
I/O and（输入/输出），284～285
immutability and（不可变），393
objects（对象），33～35，97～106
assigning（赋值），175
casting（类型转换），151
comparing（比较），158，197
converting to primitive types（转换为原始类型），197
converting to strings（转换为字符串），160，197，215
copying（复制），156～158
creating（创建），67，99～103
hash codes of（散列码），160
immutable（不可变），47，331，391～394
initializing（初始化），99，101～103
instantiating（实例化），99
passing to methods（传递给方法），94，96～97
referencing（引用），105～106，114，123
reusing（重用），35
serialization of（串行化），282
testing（测试），61～62
using（使用），103～106
verifying types of（验证类型），296

ObjectStreams example（ObjectStreams示例），282～284
offer method（offer方法），317，363
operators（操作符），55～65
  precedence of（优先级），55，67
  prefix/postfix（前缀/后缀），55，58
OutputStream class（OutputStream类），263
OutputStreamWriter class（OutputStreamWriter类），267
@Override annotation type（@Override注解类型），135，152

P

package statement（package语句），184～195，603
package members（包成员），187～191
  importing（导入），188
  public（公共），187
  referring to（引用），187～188
packages（包），3，39～40，183～195
  apparent hierarchy of（外表层次结构），189
  controlling access to（控制访问），110～112
  creating（创建），184～195
  exceptions in（异常），245～246，251
  importing（导入），188～189
  naming（命名），186～187，191
  sealing（密封），488，506～507
  unnamed（未命名），186
  versioning（版本），488，504～506
pages（页面）（见Web pages）
paint method（paint方法），551～552
palindromes（回文），213～214，219，228～230
Panel class（Panel类），581
param tag（param标记），565，574～575
parameterized types（参数化类型），170
parameters（参数），44，88，90，366
  data types of（数据类型），94
  different lists of（不同列表），91
  naming（命名），95～96
  of reference data types（引用数据类型），96～97
  primitive（原始），96
  shadowing（阴影化），95，109
  vs. arguments（实参），94
parentheses（括号）（见()）
parseXXX methods（parseXXX方法），198，217，449
PassPrimitiveByValue class（PassPrimitiveByValue类），96
Password example（Password示例），277～279
passwords（密码），277～279
PasswordStore demo（PasswordStore演示程序），466～470
  accessibility of（可访问性），482～483
  data transfer support in（数据传输支持），480
  integrating with the Desktop（和桌面集成），483～484
  localized（本地化），481～482
  look and feel（界面外观），479
  look and feels of（界面外观），477
PATH variable（PATH变量），27，457～459
Pattern class（Pattern类），406（见regular expressions）
  boundary matchers in（边界匹配器），424～425
  capturing groups in（捕获组），422～423
  character classes in（字符类），409～422
  methods of（方法），425～431
PatternSyntaxException，406
  methods of（方法），437～439
peek method（peek方法），317，363
percent sign（百分号）（见%）
performance（性能），294，361，363
  boosting（提高），2
  implementations and（实现），343～344，346
Perl，405
PI constant（PI常量），190，204
Pico text editor（Pico文本编辑器），19
Planet class（Planet类），130
play method（play方法），568
poll method（poll方法），297，317，363
Popup Swing component（Popup Swing组件），555
pound sigh（镑符）（见#）
pow method（pow方法），205
Preferences API（首选项API），451
PrePostDemo example（PrePostDemo示例），58
primitive data types（基本数据类型），45～49，96，195（见numbers）
  assigning literals to（字面量赋值），48
  comparing（比较），158
  converting to objects（转换为对象），197
  reserving memory for（保留内存），99
  tokens and（记号），271
  wrappers for（包装器），196
print method（print方法），198，273～274，583
printf method（printf方法），95，198～200
println method（println方法），160，198，215，267，269，273～274，583
PrintStream class（PrintStream类），198～200，272～277
PrintThread applet，585
PrintWriter class（PrintWriter类），240～241，267，269，272～276
PriorityBlockingQueue class（PriorityBlockingQueue类），349
PriorityQueue implementation（PriorityQueue实现），343，349
private modifier（private修饰符），123，603
  classes and（类），87～89，110～112，122，127
  inheritance and（继承），151
  methods and（方法），90

problems（问题）（见errors）
ProcessBuilder，370，450
processes（进程），370
  lightweight（轻量）（见thrcads）
processEvent method（processEvent方法），552
processKeyEvent method（processKeyEvent方法），552
processMouSeEvent method（processMouSeEvent方法），552
Producer example（Producer示例），389
ProducerConsumerExample example（ProducerConsumerExample示例），390
programs（程序）（见applications）
properties（属性）
  managing（命名），443
  saving（保存），446
  setting（设置），446～448
  system（系统），452～455，583～585
Properties class（Properties类），443～448
PropertiesTest example（PropertiesTest示例），454
propertyNames method（propertyNames方法），447
protected modifier（protected修饰符），603
  classes and（类），110～112，122，127
  inheritance and（继承），150～151
proxy servers（代理服务器），539
public modifier（public修饰符），603
  classes and（类），87～89，110～112，122，127
  constants and（常量），143
  inheritance and（继承），150～151
  interfaces and（接口），142
  methods and（方法），26，90，142
  package members and（包成员），187
  types and（类），185
put method（put方法），320～322，363
putAll method（putAll方法），322
pwd command（pwd命令），22，30

Q

qualified names（限定名称），187～188
quantifiers（量词），416～422
  types of（类型），421～422
question mark（问号）（见?）
Queue interface（Queue接口），297，316～319
  implementations of（实现），342～343，349～350，363
queues（队列），
  bounded（有界），317
  order of elements in（元素顺序），317
quote method（quote方法），430
QuoteClientApplet applet，590～591
quoteReplacement method（quoteReplacement方法），432
QuoteServer applet，590
QuoteServerThread applet，590

R

radians（弧度），206
random method（random方法），208
Random class（Random类），208，358
RandomAccessFile class（RandomAccessFile类），290～291
raw types（原始类型），178
readDouble method（readDouble方法），280
reader method（reader方法），277
Reader class（Reader类），265
readInt method（readInt方法），280
readLine method（readLine方法），267，279
readObject method（readObject方法），284～285
readPassword method（readPassword方法），279
readUTF method（readUTF方法），280
Receiver applet，565～567
Rectangle class（Rectangle类），101～103，143～144
Reflection API（反射API），518
RegexTestHarness example（RegexTestHarness示例），406～407，427
RegexTestHarness2 example（RegexTestHarness2示例），437～439
regionMatches method（regionMatches方法），224～225
RegionMatchesDemo example（RegionMatchesDemo示例），224
regular expressions（正则表达式），221，225，271，405～441
  backreferences in（反向引用），423
  boundary matchers in（边界匹配器），424～425
  capturing groups in（捕获组），422～423
  case sensitivity in（区分大小写），426
  character classes in（字符类），409～413
  commonly used（常用），414～416
  embedded flag expressions in（嵌入标志表达式），427～428
  metacharacters in（元字符），408～409
  non-capturing groups in（非捕获组），423
  quantifiers in（量词），416～422
  syntax errors in（语法错误），437～439
  zero-length matches in（零长度匹配），417～420
Relatable interface（Relatable接口），143～145
relational operators（关系操作符）（见comparisons）
remove method（remove方法）
  for applets（用于applet），555
  for collections（用于集合），297～300，303，307～309，311，317，345，353，363
  for properties（用于属性），448

removeAll method (removeAll方法), 300, 304, 323～326, 353
removeDups method (removeDups方法); 303
removeEldestEntry method (removeEldestEntry方法), 347
removeElementAt method (removeElementAt方法), 309
removeFirst method (removeFirst方法), 346
removeLast method (removeLast方法), 346
renameTo method (renameTo方法), 289
replace method (replace方法), 221, 228, 314, 431
replaceAll method (replaceAll方法), 221, 316, 432, 434～437
ReplaceDemo example (ReplaceDemo示例), 434～436
ReplaceDemo2 example (ReplaceDemo2示例), 435
replaceFirst method (replaceFirst方法), 221, 432, 434～436
reserved words (保留字) (见keywords)
retainAll method (retainAll方法), 300, 304, 323～326
return statement (return语句), 81～82, 106～107, 240, 603
return types (返回类型), 81, 90, 106～109, 366～367
  by values (按值), 105
  class names as (类名称), 107～109
  covariant (协变), 108, 152
  interface names as (接口名称), 109
reverse method (reverse方法), 228～229, 316, 359
rint method (rint方法), 204
Root example (Root示例), 273～274
Root2 example (Root2示例), 274
rotate method (rotate方法), 316
round method (round方法), 204
run method (run方法), 371
Runnable interface (Runnable接口), 371～372, 398, 587
runtime (运行时)
  checks at (检查), 152
  examining annotations at (检查注解), 132, 136
  handling errors (处理错误), 233～235, 241～246
  performance boosting at (性能提高), 2
  terminating (终止), 234
RuntimeException, 235～236, 248, 252～253

S

S type parameter (S类型参数), 172
Safelock example (Safelock示例), 395～397
sandboxes (沙箱), 538, 544
Scanner class (Scanner类), 270～272
scanning (扫描), 270～272
  locales and (地区), 271～272
ScanSum example (ScanSum示例), 271～272
ScanXan example (ScanXan示例), 270～271
schedule method (schedule方法), 399
ScheduledExecutorService interface (ScheduledExecutorService接口), 398～399
ScheduledThreadPoolExecutor class (ScheduledThreadPoolExecutor类), 400
ScrollingSimple applet, 555～558
security (安全), 261
  applets and, 558～560, 564, 567, 578～581, 585, 591～593
  certificates and (证书), 508
  digitally signed files and (数字签名文件), 488, 507～513
  Java Web Start and, 538～539
  keystores and (密钥存储库), 511
  password entry and (密码输入), 277
  public and private keys (公共和私有密钥), 508
  violation of (违规), 456～457
SecurityException, 455～456, 559, 579
SecurityManager class (SecurityManager类), 456, 559, 579
security managers (安全管理器), 444, 455～457
  accessing to system properties (访问系统属性), 452
semicolon (分号) (见;)
Sender applet, 565～567
sequences (序列) (见collections; ordered)
Serializable interface (Serializable接口), 282, 343
servers (服务器) (见Web servers)
ServiceLoader class (ServiceLoader类), 451
services (服务), 451
set method (set方法), 307～309, 345, 363, 401
Set interface (Set接口), 295～297, 301～306
  array operations of (数组操作), 306
  basic operations of (基本操作), 303
  bulk operations of (批处理操作), 304～306
  empty constants of (空常量), 354
  implementations of (实现), 302, 342～345, 353, 363
  wrappers for (包装器), 350, 352
setDefaultHostnameVerifier method (setDefaultHostnameVerifier方法), 539
setDefaultSSLSocketFactory method (setDefaultSSLSocketFactory方法), 539
setElementAt method (setElementAt方法), 307～309
setLastModified method (setLastModified方法), 289
setLayout method (setLayout方法), 555
setLength method (setLength方法), 227

setProperties method（setProperties方法），454～455
setProperty method（setProperty方法），448
setSize method（setSize方法），308
short data type（short数据类型），46，603
  default value of（默认值），47
  expressing（表示），48
  switch statement and（switch语句），71
Short class（Short类），216
  natural ordering（自然顺序），329
  switch statement and（switch语句），71
shortValue method（shortValue方法），197
showDocument method（showDocument方法），562～564
ShowDocument applet，562～564
showStatus method（showStatus方法），561
Shuffle example（Shuffle示例），310，314，316，358
shuffle method（shuffle方法），310
signature block files（签名块文件），510
signature files（签名文件），509
Simple applet，547～550，552
SimpleClick applet，552～554
SimpleThreads example（SimpleThreads示例），372，375～377
sin method（sin方法），207
single quote（单引号）（见'）
singleton method（singleton方法），301，353
size method（size方法），303，320～322，363，447
  for collections（用于集合），298
slash（斜线）（见/）
sleep method（sleep方法），372～373，375
SleepMessages example（SleepMessages示例），373
Smalltalk's collection hierarchy（Smalltalk的集合层次结构），294
Socket class（Socket类），39
sockets（套接字），267
software（软件）（见applications）
Solaris OS（Solaris操作系统），2～3，6～7，19～23
  environment variables on（环境变量），458～460
  file names on（文件名），286～287
sort method（sort方法），316，328～335，356～359
Sort example（Sort示例），356
SortedMap interface（SortedMap接口），297，338～340
  operations in（操作），338～339
  standard conversion constructor in（标准转换构造器），339
  wrappers for（包装器），350，352
SortedSet interface（SortedSet接口），295～297，335～338
  Comparator access operations in（Comparator访问操作），335，338
  endpoint operations in（终点操作），335，338
  range-view operations in（范围视图操作），335～338
  standard conversion constructor of（标准转换构造器），336
  wrappers for（包装器），350，352
SoundExample class applet，569～570，589
SoundList class（SoundList类），570，589
SoundLoader class（SoundLoader类），570，589
source files（源文件），191～193
  adding code to（添加代码），7，9～11
  compiling（编译），7，11～12，14，16～18，20
  creating（创建），14～16，20～22
  naming（命名），11，15，21
  renaming nodes in（重命名节点），13
  skeleton（框架），7，9
  package statement in（package语句），184～195
split method（split方法），220，428～430
SplitDemo example（SplitDemo示例），428
SplitDemo2 example（SplitDemo2示例），429
sqrt method（sqrt方法），205
square brackets（方括号）（见[]）
square root（平方根），205
SSLSocketFactory class（SSLSocketFactory类），539
StackOfInts class（StackOfInts类），125～127
stacks（栈），124
stack traces（栈踪迹），236，249～250
  printing（输出），257
Standard Error（标准错误），276～277，583
Standard Input（标准输入），276～277
Standard Output（标准输出），276～277，583
start method（start方法），372，423，431～433，543，546，550～551，570，585，587
startsWith method（startsWith方法），225
statements（语句），67～68
  synchronized（同步），382～383
static modifier（static修饰符），603
  classes and（类），117
  constants and（常量），115，143
  fields and（字段），44，112～114
  methods and（方法），26
static initialization blocks（静态初始化块），117
stop method（stop方法），544，550～551，568，585，587，593
streams，I/O（流，I/O），261～285
  buffered（缓冲），268～269，291
  byte（字节），263～267，269，277
  character（字符），265～269
  closing（结束），264～265
  data（数据），279～282
  object（对象），282～285
  standard（标准），276～277
  unbuffered（非缓冲），269

strictfp method（strictfp方法），603
String class（String类），39，47，160，212～232
  constructors for（构造器），213
  default value of（默认值），47
  escape sequences in（转义序列），49
  expressing（表示），48
  natural ordering（自然顺序），328～329
  regular expressions in（正则表达式中），430～431，436～437
StringBuilder class（StringBuilder类），224～230
string builders（字符串构建器），224～230
  capacity of（容量），226～227
  length of（长度），226～227
StringDemo example（StringDemo示例），213～214，228～230
StringIndexOutOfBoundsException，223
stringPropertyNames method（stringPropertyNames方法），447
strings（字符串），212～232
  comparing（比较），224
  concatenating（连接），57，214～215
  converting to numbers（转换为数字），197～198，216～217
  converting to objects（转换为对象），197
  creating（创建），212～213
  immutability of（不可变），213
  length of（长度），213～214
  multiline（多行），215
  replacing（替换），221
  sorting（排序），356
  substrings from（子字符串），219
subclasses（子类），37～165
  constructors for（构造器），155～156
  controlling access to（控制访问），111，148，150～151，154
  final methods and（最终方法），118
subList method（subList方法），307～308，313～315
submit method（submit方法），398
subSequence method（subSequence方法），220
subSet method（subSet方法），336～338
substring method（substring方法），219，221～224
subtyping（子类型），175～177
Sun Certified Programmer for the Java Platform（Sun公司认证的Java平台程序员），605～615
super keyword（super关键字），151，154～156，177，603
superclasses（超类），37，87，135，145，148
  accessing（访问），154～155
  constructors for（构造器），155～156
  private members in（私有成员），151
@SuppressWarnings annotation type（@SuppressWarnings注解类型），135
swap method（swap方法），309，316，359
Swing，4，463～485，547
  basic controls（基本控件），470～471，555
  cut, copy, and paste（剪切、复制和粘贴），480
  drag-and-drop support（拖放支持），480
  general-purpose containers（通用容器），474～475
  interactive displays（交互性显示），472
  internationalization（国际化），465，480～482
  list component（列表组件），466
  look and feel（界面外观），479
  look and feels（界面外观），464，470
  special-purpose containers（专用容器），475～476
  system tray support（系统托盘支持），484
  tabbed pane（选项卡式面板），468～469
  table component（表组件），467
  top-level containers（顶级容器），474
  undo framework（撤销框架），465，470
  uneditable information displays（不可编辑的信息显示），473～474
switch block（switch块），71～74，603
  terminating（终止），77
SwitchDemo example（SwitchDemo示例），72
SwitchDemo2 example（SwitchDemo2示例），73
synchronization（同步）
  intrinsic locks and（内在锁），381～383
  of collections（集合），343，346，349～351
  of threads（线程），377～384
  reentrant（可重入），383
synchronized keyword（synchronized关键字），380，603
synchronizedCollection method（synchronizedCollection方法），350～351
SynchronizedCounter example（SynchronizedCounter示例），380～381，401
synchronizedList method（synchronizedList方法），346，350～351
synchronizedMap method（synchronizedMap方法），350～351
SynchronizedRGB example（SynchronizedRGB示例），391～393
synchronizedSet method（synchronizedSet方法），350～351
synchronizedSortedMap method（synchronizedSortedMap方法），350～351
synchronizedSortedSet method（synchronizedSortedSet方法），350～351
SynchronousQueue class（SynchronousQueue类），350
Synthetica look and feel（Synthetica界面外观），477，479
Synth package（Synth包），464，477
System class（System类），26，444，452～457

System.console，277
System.err，273，276～277，583
System.in，276～277
System.out，198，273，276～277，583

T

T type parameter（T类型参数），172
tab（制表符），49，212
tailSet method（tailSet方法），337
TalkClientApplet applet，591
TalkServer applet，592
TalkServerThread applet，592
tan method（tan方法），207
ternary operator（三元操作符），55，60
TestFormat example（TestFormat示例），201
this keyword（this关键字），109～110，603
  using with constructors（用于构造器），109～110
Thread class（Thread类），371～377，570
  subclasses of（子类），371～372
ThreadPoolExecutor class（ThreadPoolExecutor类），400
thread pools（线程池），399～400
threads（线程），369～404
  blocked（阻塞），384～385
  coordinating（协调），386～390
  defining（定义），371～372
  execution history of（执行历史），249
  in applets（applet中），585～589
  interference of（干扰），377～379
  locks and（锁），381～383
  main（主），370
  memory consistency errors and（内存一致性错误），379～381，384，401
  suspending（暂停），372～373，386
  synchronization of（同步），377～384
  system（系统），370
  using to perform initialization tasks（用于执行初始化任务），588
throw statement（throw语句），246～247，603
Throwable class（Throwable类），239，246～249，258
  constructors for（构造器），249
throws keyword（throws关键字），235，246，257，603
TicTacToe example（TicTacToe示例），491～498～499
toArray method（toArray方法），301，335，339，362
toDegrees method（toDegrees方法），207
tokens（记号）
  breaking input into（分割为），270～271
  for primitive types（用于原始类型），271
  separators for（分隔符），270～271
  translating（转换），271～272
toLowerCase method（toLowerCase方法），211，220
toRadians method（toRadians方法），206～207
toString method（toString方法），156，160，198，211，215，218，228，273～274，287，331，336，339，430
ToStringDemo example（ToStringDemo示例），218
toUpperCase method（toUpperCase方法），211，220
transient keyword（transient关键字），603
TreeMap implementation（TreeMap实现），320～321，339，342～343，347
TreeSet implementation（TreeSet实现），302，304，334，336，342～345
TrigonometricDemo example（TrigonometricDemo示例），206～207
trigonometric functions（三角函数），206～208
trim method（trim方法），220
troubleshooting（错误检查）（见errors）
try block（try块），235，238～245，603
tryLock method（tryLock方法），395
type erasure（类型擦除），178～179
type inference（类型接口），173
type parameters（类型参数）
  bounded（有界），173～174
  formal（正式），170
  multiple（多），171
  naming（命名），171～172
types（类型），183
  in interfaces（接口中），140
  multiple in one source file（一个源文件中多个），185
  package-private（包私有），185
  public（公共），185
type variables（类型变量），169～171

U

U type parameter（U类型参数），172
UnaryDemo example（UnaryDemo示例），58
unary operators（一元操作符），55，57～59，62
unboxing（拆箱），196，211
underscore（下划线）（见_）
Unicode encoding（Unicode编码），46，265
  escape sequences in（转义序列），48
Uniform Resource Identifiers（URI）（统一资源标识），483
UNIX/Linux，2～3，6～7，19～23
  common error messages on（常见错误消息），27，29～30
  environment variables on（环境变量），193，451，458～460
  file name separators on（文件名分隔符），191
  file names on（文件名），287
  path separators on（路径分隔符），193
  root directories on（根目录），289
unmodifiableXXX methods（unmodifiableXXX方法），352

unset command（unset命令），30
UnsupportedClassVersionError，601
UnsupportedOperationException，296，352～353
URL class（URL类），515，518
URLClassLoader implementation（URLClassLoader实现），514～515
useDelimiter method（useDelimiter方法），271
UTF-8 encoding（UTF-8编码），281，502，533
utilities（工具）
  configuration（配置），443～451
  system（系统），452～457

V

V type parameter（V类型参数），172
valueOf method（valueOf方法），198，216
ValueOfDemo example（ValueOfDemo示例），216～217
values method（values方法），130，322～323
varargs，（可变参数）94～95
variables（变量），34，43～55
  as members（作为成员），44
  assigning literals to（字面量赋值），48
  atomic（原子），401～402
  declaring（声明），45，67，99～100
  initializing（初始化），28
  local（局部），44，47，48
  naming（命名），44～45，89～90
  reference（引用），100
  referring to arrays（引用数组），50～51
  volatile，384
Vector collection（Vector集合），293，297，307～309，343，346，351
  compatibility and（兼容性），364～366
verify method（verify方法），279
vertical bar（竖线）（见|）
vi text editor（vi文本编辑器），19
void keyword（void关键字），39，81，107，603
volatile keyword（volatile关键字），384，603

W

wait method（wait方法），157，386～390
WarningDemo example（WarningDemo示例），179
warning messages（警告消息），134，179
  suppressing（抑制），132，135
WeakHashMap implementation（WeakHashMap实现），348
Web browsers（Web浏览器），543
  displaying documents in（显示文档），562～564
  frames in（框架），562
  quitting（退出），550
  reloading applets（重新加载applet），549
  running applets（运行applet），544，559～578
  security and（安全），558～560，564，567，578～581，585
  status lines of（状态栏），561
  that don't understand the <APPLET> tag（不理解<APPLET>标记），575～576
Web pages（Web页面）
  adding applets to（添加applet），594～600
  creating（创建），533～534
Web servers（Web服务器）
  applets and，589～593
  plaeing applications on（存放应用程序），533
  setting up（设置），525
while statement（while语句），74～75，603
  skipping the current iteration（跳过当前迭代），79
  terminating（终止），77
WhileDemo example（WhileDemo示例），74
white space（空白），211
  leading and trailing（前导和结尾），220
  tokens and（记号），270
wildcards（通配符），177～178，188～189
  bounded（有界），177
  unbounded（无界），177
Windows（见Microsoft Windows）
Windows look and feel（Windows界面外观），470～478
wrappers（包装器），342～343，350～352
  checked interface（checked接口），352
  synchronization（同步），343，350～351
  unmodifiable（不可修改的），351～352
write method（write方法），273
writeDouble method（writeDouble方法），280
writeInt method（writeInt方法），280
writeObject method（writeObject方法），284～285
Writer class（Writer类），277
writer method（writer方法），265
writeUTF method（writeUTF方法），280～281

Z

ZIP archives（ZIP档案），289～291，488

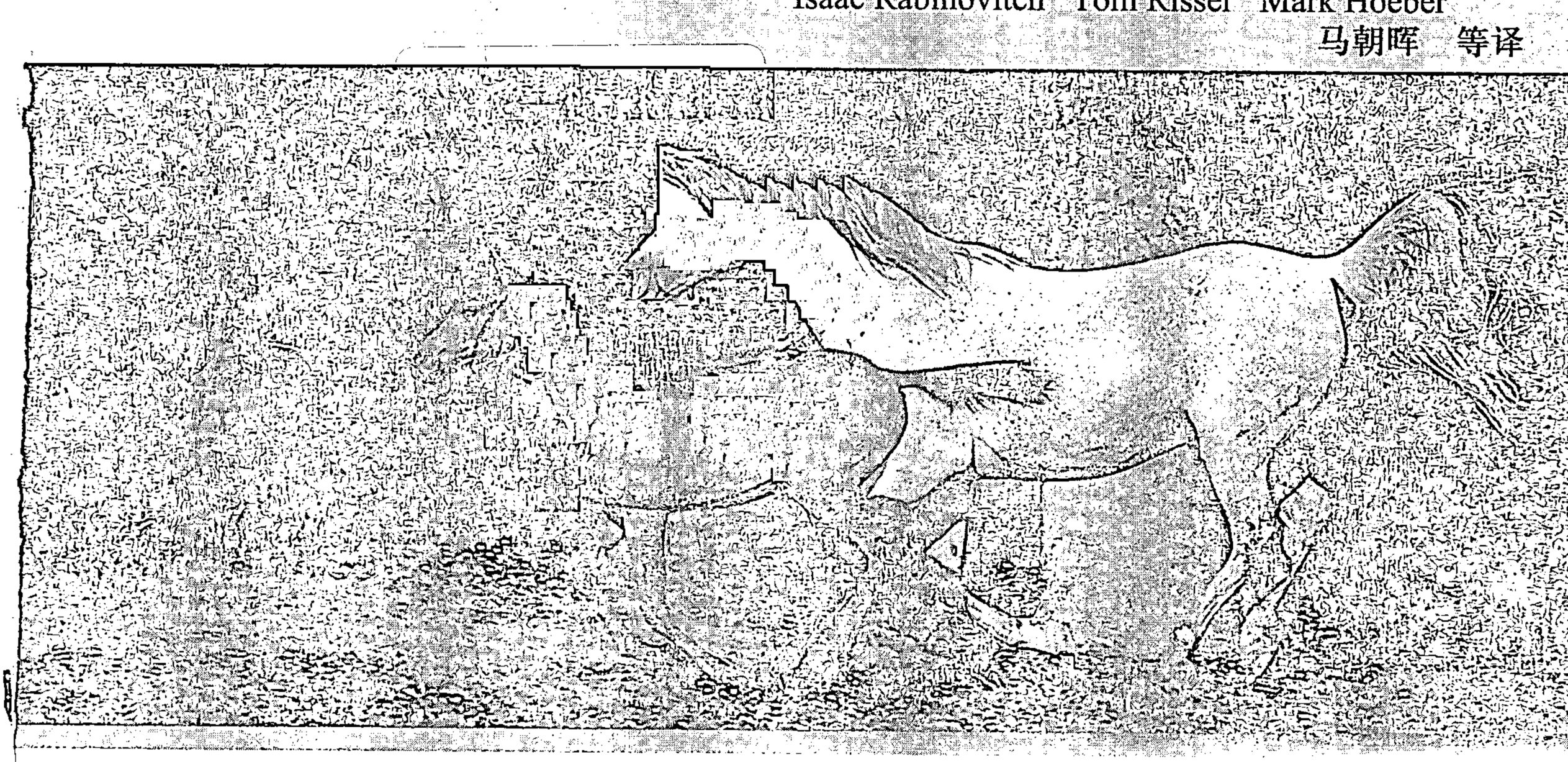
TURING 图灵计算机科学丛书

Java™教程（第4版）

The Java™ Tutorial: A Short Course on the Basics

Fourth Edition

[美] Sharon Zakhour Scott Hommel Jacob Royal
Isaac Rabinovitch Tom Risser Mark Hoeber 著
马朝晖 等译



人民邮电出版社
北 京

TURING 图灵计算机科学丛书

Addison Wesley

Sun microsystems

Sun公司指定教材

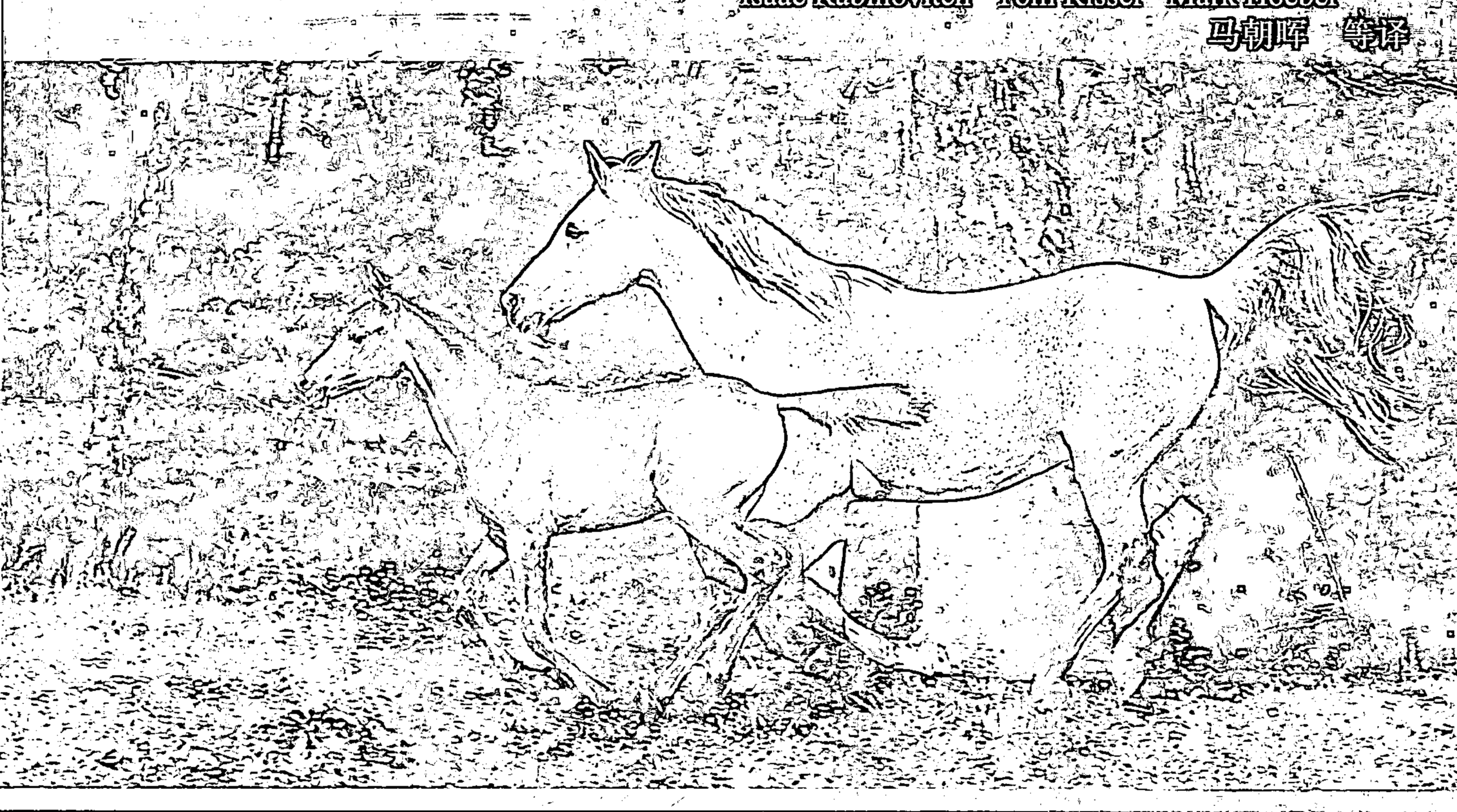
Java™ 教程

（第4版）

The Java™ Tutorial: A Short Course on the Basics

Fourth Edition

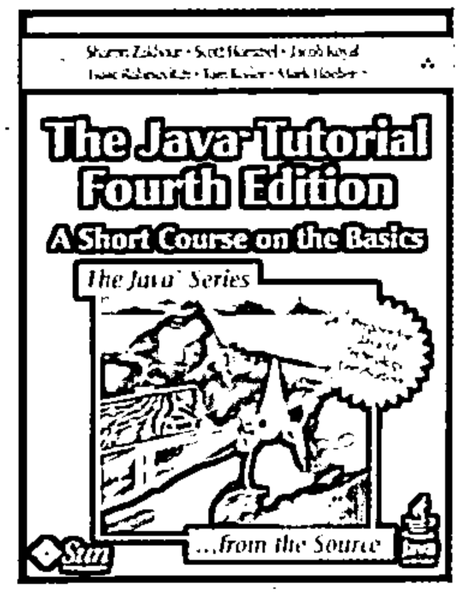
[美] Sharon Zakhour Scott Hommel Jacob Royal
Isaac Rabinovitch Tom Risser Mark Hoeber 著
马朝晖 等译



人民邮电出版社
POSTS & TELECOM PRESS



Java™教程（第4版）



The Java™ Tutorial
A Short Course on the Basics
Fourth Edition

“本书循序渐进地引导读者成为基础扎实、深思熟虑的Java程序员，既适合自学，也可以用于各种教学场合。”

——**Francis Glassborow**，C/C++用户协会前主席

“最初学习本书是多年前的事了，现在重读本书，仍能发现作者对许多关键的Java特性都有精彩的讲解。该书轻易就澄清了许多困扰我许久的问题，对于有一定Java语言基础的朋友也有相当大的参考价值。我强烈推荐。”

——**赵科**，JavaResearch.org创始人

本书是Sun公司指定的Java语言入门教程，多年来引领了一批又一批程序员步入Java殿堂。本书由Java语言和平台开发团队的多位成员亲自撰写，并由众多世界级Java专家精心审稿，有力地保证了权威性和准确性。

本书采用久经检验的交互式教学方法，提供了许多来自实战的完整示例，而且每一章都有问题和练习（网上资源给出部分答案），有助于读者实践和检查所学的知识。书中清晰地阐述了对象、类和数据结构的基础，并且详细讲解了异常、I/O和线程等内容。本书秉承了使此前各版本成为经典的各项特色，时至今日，仍然是Java程序员由入门到精通的绝佳选择，同时也适合作为高等院校和各类培训的教材。

作为最新版本，本书涵盖了Java SE 6，增加了讲述泛型、集合、Java Web Start、平台环境和正则表达式的章节，并大幅改写了线程、I/O、面向对象编程概念和语言基础等内容。此外，Sun公司专门开设的本书网站（http://java.sun.com/docs/books/tutorial/）上有丰富的资源，包括英文原版的内容（包括示例代码以及问题和练习答案）、不断更新的本书博客、最新JRE、JDK和Java API规范和其他文档。

Sun microsystems

Addison Wesley

www.PearsonEd.com

ISBN 978-7-115-16336-3
9 787115 163363 >
ISBN 978-7-115-16336-3/TP
定价：55.00元

PEARSON Education

本书相关信息请访问：图灵网站 http://www.turingbook.com
读者/作者热线：(010) 88593802
反馈/投稿/推荐信箱：contact@turingbook.com

上架建议 | 计算机/程序设计/Java

人民邮电出版社网址 www.ptpress.com.cn

序

能为《Java教程》的新版写序，我感到非常高兴。

Java平台显著的特征之一就是其文档的质量。这包括教程，JavaDoc方式的API详尽文档，还有讲述Java的很多书籍。有人可能质疑，文档资源的质量对Java平台的广泛采用而言是否具有和语言以及库本身的质量一样的意义。其实，它们是相辅相成的，其质量的好坏都非常关键。

数百万开发人员把他们的职业生涯投入到了Java平台形成的生态系统中。现在有无数的Java标准、API和工具。要学的东西太多了！而且，要了解的信息的数量还在不断增长。虽然我们这些软件专业人员应该感谢这种增长，但对开发人员来说这也是一种挑战。我希望本书能帮助读者迎接这一挑战，并且使读者掌握以至精通Java平台的过程容易一些。

就像Edsger Dijkstra[1]多年前说的那样，是否能成为真正“合格的程序员”取决于你自己；但是如果你没能做到，不要归咎于缺乏好的Java教程。很多敬业的专业人士已经为编写良好的资料尽了全力。

我希望新版能够达到甚至超越以前几个版本树立的高度。最终的评判者是你——读者。我认为本书不会令读者失望。

Gilad Bracha[2]
加利福尼亚州洛斯阿尔托斯市
2006 年 6 月

---

1. 计算机科学大师，1972年图灵奖得主。——编者注
2. Java顶级专家，Java语言规范的作者之一，曾是Sun公司的杰出工程师，目前供职于Cadence公司。——编者注

作 者 简 介

Sharon Zakhour是本书创作团队的领导，在Sun公司作为高级技术作家任职7年。她编写了*JFC Swing Tutorial*（第2版），并且在Swing和AWT团队工作多年。她毕业于加利福尼亚大学伯克利分校，获得计算机科学学士学位，从事程序员、开发支持工程师和技术作家的工作已20余年。

Scott Hommel是Sun公司的技术作家，他的工作是为Java SE编写文档。从1999年开始，他参与了JDK每个主要版本的工作，主要的工作是编写API规范说明和核心版本文档。现在他在位于佛蒙特州基灵顿的家中工作。

Jacob Royal是IT专业的硕士和信息系统方向的MBA。他编写过管理员指导、API参考、程序员指导，开发并编写过新的工具和代码，为很多公司（包括朗讯科技和Autodesk公司）编写过标准。他在Sun公司还参与了JSR API规范和Java部署指导的编写工作。

Isaac Rabinovitch是自由技术作家。他为Sun、Borland、SGI和很多其他公司编写过用户手册、程序员指南、管理员手册、API参考、版本说明和支持文档。他喜欢处理技术细节，从事计算机行业多年。

Tom Risser毕业于哈佛大学物理专业（获得学士学位）和加利福尼亚大学伯克利分校（获得博士学位）。他作为计算机行业的技术作家已达15年之久。

Mark Hoeber曾任Sun公司的高级技术作家。他作为技术作家已有12年，致力于为软件开发人员和系统管理人员编写文档。此外，Mark还参与过技术编写团队的内容管理系统的实现工作。

前　言

自从最初的JDK（Java Development Kit，Java开发工具包）于1995年5月发布以来，Sun公司的工程人员为改进和增强Java平台付出了艰苦的努力。本版与Java SE（Java Platform Standard Edition，Java平台标准版）6相一致，并且反映了这一版本的API。

本版介绍了从第3版（对应Java SE 1.3）出版以来Java平台添加的新特性，比如介绍泛型和注解的一章（第6章）。还有关于Java Web Start（第17章）、平台环境（第14章）和正则表达式（第13章）的几章。并发（原来的“线程”）（第12章）、I/O（第10章）、面向对象的编程概念（第2章）和语言基础（第3章）这几章都完全重新编写了。集合（第11章）从附录中提取出来成为独立的一章。新的附录B包含如何准备Java编程语言认证（Java Programming Language Certification）考试的信息。

本书的所有资料都经过了Sun公司Java开发人员的仔细审核，以便确保信息是准确的和最新的。

和前几版一样，本书以Sun公司网站上的在线教程为基础：

```
http://java.sun.com/docs/books/tutorial/
```

本书包含的信息（经常被称为“核心教程”或者“基础”）针对初级到中级程序设计人员的需要。掌握了本书的内容之后，读者可以在该网站上学习Java平台的其他知识。

像以往一样，我们的目的是编写容易阅读的实用程序员指南，并且用很多示例帮助读者理解程序。

本书面向的读者

本书既适合初学者，也适合有经验的程序员。

- 新程序员通过从头到尾阅读本书可以得到最大的收获，包括按照第1章中的分步讲解编译和运行自己的第一个程序。
- 具有过程式语言（比如C）经验的程序员可以从Java编程语言的面向对象概念和特性开始学习。
- 经验丰富的程序员可以跳到更高级的主题，比如关于泛型、并发或者Java Web Start的内容。

本书包含的内容可以满足各种不同经验的程序员的学习需要。

如何使用本书

本书的设计方式使读者既可以通读，也可以在各主题之间跳转。信息按照逻辑的顺序编排，并且尽可能避免向前引用。

书中的示例是在6.0版本下编译的。为了编译和运行大多数示例，读者需要下载这个版本。

本书的网上下载资源包含本书英文原版的内容[1]（包括示例，还有问题和练习的答案[2]），以及在线Java SE教程、Java SE开发工具包（Java SE Development Kit，JDK）[3]、相应的API规范和指导文档[4]。

在本书中你会看到这样的脚注：

```
docs/api/java/lang/Class.html
```

和

```
tutorial/deployment/applet/examples/TalkServer.java
```

根目录包含docs和tutorial目录，所以能在指定的位置找到这些脚注。要想定位到最新的在线页面，在docs脚注前面加上http://java.sun.com/javase/6/；在tutorial脚注前面加上http://java.sun.com/docs/books/即可：

```
http://java.sun.com/javase/6/docs/api/java/lang/Class.html

http://java.sun.com/docs/books/tutorial/deployment/applet/examples/TalkServer.java
```

本书英文原版的章节在网上下载资源的下列位置：

章	位 置
第 1 章 入门	tutorial/getStarted
第 2 章 面向对象的编程概念	tutorial/java/concepts
第 3 章 语言基础	tutorial/java/nutsandbolts
第 4 章 类和对象	tutorial/java/javaOO
第 5 章 接口和继承	tutorial/java/IandI
第 6 章 泛型	tutorial/java/generics
第 7 章 包	tutorial/java/package
第 8 章 数字和字符串	tutorial/java/data
第 9 章 异常	tutorial/essential/exceptions
第 10 章 基本 I/O	tutorial/essential/io
第 11 章 集合	tutorial/collections
第 12 章 并发	tutorial/essential/concurrency
第 13 章 正则表达式	tutorial/essential/regex
第 14 章 平台环境	tutorial/essential/environment
第 15 章 Swing	tutorial/ui
第 16 章 把程序打包到 JAR 文件中	tutorial/deployment/jar
第 17 章 Java Web Start	tutorial/deployment/webstart
第 18 章 applet	tutorial/deployment/applet

我们欢迎读者对本书多提宝贵意见。请使用如下的Tutorial反馈论坛：

```
http://developers.sun.com/contact/tutorial_feedback.jsp
```

---

1. http://java.sun.com/docs/books/tutorial/
2. 为了方便读者参考，图灵网站（www.turingbook.com）还提供了本书“问题和练习”部分的中文版答案。——编者注
3. http://java.sun.com/javase/downloads/index.jsp
4. http://java.sun.com/javase/6/does/api

致谢

许多因特网读者帮助我们保持并提高了本书的质量，他们给我们发电子邮件，热情地指出我们的输入错误和无效的链接，以及本书中可能引起误解的地方和需要重写的地方。

许多现任和前任的Java开发人员和文档编写人员给我们提了许多建议，解答了我们的许多问题，审阅了我们的内容，甚至参与了本书部分内容的编写。这个名单很长，其中我们特别要感谢Eric Armstrong、David Bristor、Martin Buchholz、Mandy Chung、Iris Garcia Clark、Margarita Fischer、Amy Fowler、Andy Herrick、Shannon Hickey、David Holmes、Jim Holmlund、Patrick Keegan、Peter Korn、Masayoshi Okutsu、Scott Seligman、Xueming Shen、Sundar Sivasubramanian、Pete Soper、Scott Violet和Kathy Walrath作出的贡献。我们还要特别感谢Sun公司其他高水平的作者和程序员，他们作为特邀作者参与了在线教程的编写。

我们要感谢以前的同事Joshua Bloch[1]（“集合”一章的原作者），他在百忙之中为我们提供了最新的信息。我们还要感谢特邀的审稿人Doug Lea和Brian Goetz的长期支持。

感谢我们的经理Debra Scott给予我们鼓励，并且给我们的工作提供了必要的支持。

我们的程序经理Mary Lautner和Ian Hardie跟踪进度、分担事务性工作，并以其幽默的天性使工作变得轻松。

插图设计者Michael Quillman和Dwayne Wolff制作了专业的图片，工作又快又好。

我们还要感谢SQE团队成员提供了有价值的反馈。

最后，感谢Java SE部门的经理让他们的工程师和插图设计者帮助编写本书，特别感谢：Uday Dhanikonda、Frances Ho、James Hsieh、Janet Koenig和Keith Yarwood。

1. Bloch曾是Java平台设计的灵魂人物，Java集合框架的设计者，现供职于Google公司。他的《Java解惑》一书（人民邮电出版社出版）是Java进阶最佳读物之一。——编者注

图书在版编目（CIP）数据

Java™ 教程：第 4 版 /（美）扎克霍尔（Zakhour, S.）等著；马朝晖等译.
—北京：人民邮电出版社，2007.9
（图灵计算机科学丛书）

ISBN 978-7-115-16336-3

Ⅰ. J… Ⅱ. ①扎…②马… Ⅲ. JAVA 语言—程序设计—教材 Ⅳ. TP312

中国版本图书馆 CIP 数据核字（2007）第 081151 号

内 容 提 要

本书是Sun公司指定Java教程的最新版。书中除了讲解面向对象的编程概念、语言基础、类和对象、接口和继承、包、数字和字符串、异常、基本I/O、集合、并发、Swing、把程序打包到JAR文件中、applet之外，还介绍了Java Web Start、平台环境和正则表达式，同时涵盖了Java 5.0和6.0引入的新特性，比如泛型和并发、注解。附录B中还给出如何准备Java编程语言认证考试方面的内容。此外，本书配套的网上资源还包含本书的示例以及问题和练习的答案、在线Java SE指南、Java SE开发工具包（Java SE Development Kit，JDK）、相应的API规范和指导文档等。

本书既适合作为高校相关专业Java程序设计语言课程的教材，也适合各种层次的Java程序员阅读。

图灵计算机科学丛书

Java™ 教程（第 4 版）

◆ 著 [美] Sharon Zakhour Scott Hommel Jacob Royal
Isaac Rabinovitch Tom Risser Mark Hoeber
译 马朝晖 等
责任编辑 杨海玲

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街 14 号
邮编 100061 电子函件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
北京顺义振华印刷厂印刷
新华书店总店北京发行所经销

◆ 开本：787 × 1092 1/16
印张：27
字数：746 千字 2007 年 9 月第 1 版
印数：1–4 000 册 2007 年 9 月北京第 1 次印刷

著作权合同登记号 图字：01-2005-5228 号

ISBN 978-7-115-16336-3/TP

定价：55.00 元

读者服务热线：(010)88593802 印装质量热线：(010)67129223